ABC80 BASIC-ROM

Kommenterad disassemblering

Z80-kod, $0000–$3FFF (16 KB), version 11273

Hör till boken *ABC80 inifrån*.

Innehåll

1 Inledning **1**

2 Systemvariabler och externa adresser **3**

3 Listning **6**

3.1 RST-vektorer och fasta ingångar . . . 6
3.2 Kallstart . . . 7
3.3 Kommandoloopen . . . 8
3.3.1 Anrop till tilläggs-ROM . . . 10
3.4 Enhetslistan . . . 11
3.5 PRTXT: skriv text på skärmen . . . 11
3.6 INLINE: radinmatning från tangentbordet . . . 14
3.7 GETKEY: vänta på en tangent . . . 15
3.8 Kassetten . . . 19
3.9 OPEN för kassett: leta efter filen . . . 22
3.10 CASRBL: hämta ett block från den avbrottsstyrda läsaren . . . 24
3.11 CASINT: avbrott från kassetten, ett per bit . . . 25
3.12 Felhantering . . . 28
3.13 INSLIN: lägg in, ersätt eller radera en programrad . . . 32
3.14 OPEN: öppna en fil . . . 34
3.15 DVCALL: anropa enhetsrutin nummer 0–8 . . . 36
3.16 TXTPRT: skriv text till blockad fil (enhetsrutin 4 för CAS) . . . 37
3.17 TXTINP: läs en textrad från blockad fil (enhetsrutin 3 för CAS) . . . 39
3.18 KWSRCH: sök nyckelord i tabell . . . 42
3.19 ED n: redigera rad n . . . 43
3.20 LIST [n][-m] eller LIST fil . . . 44
3.21 LOAD och MERGE . . . 46
3.22 REN [start][,steg]: numrera om programmet . . . 50
3.23 RUN [fil]: starta programmet . . . 52
3.24 EXESTM: utför en sats . . . 53
3.25 SAVE fil: spara programmet som BAC-fil . . . 54
3.26 PRELNK: förlänkning innan programmet körs . . . 56
3.27 SKPELM: hoppa över ett element i tokeniserad text . . . 59
3.28 DETOK: gör om en programrad till text (för LIST och ED) . . . 69
3.29 DETEXP: skriv ett kompilerat uttryck (omvänd polsk notation) som infix . . . 72
3.30 ROTTXT: flytta den senast skrivna texten bakåt . . . 78
3.31 FNAME: tolka ett filnamn [enhet:]namn[.typ] . . . 80
3.32 GETFLT: läs ett flyttal från texten . . . 83
3.33 FLTASC: skriv flyttalet vid HL som text vid DE . . . 88
3.34 TOKLIN: gör om en inskriven rad till intern kod . . . 96
3.35 TOKKW: kompilera en sats . . . 97
3.36 STTAB1/STTAB2: satsernas kompileringsrutiner . . . 100
3.37 CMPDEF: kompilera DEF FNx(p1,p2,...)=uttryck . . . 101
3.38 CMPFN: kompilera ett anrop `FNx(a1,a2,...)` i ett uttryck . . . 103
3.39 FN vid körning (funktionskod 0 i EXPTAB2) . . . 104
3.40 LET: kompilering . . . 106
3.41 DIM: kompilering . . . 107

3.42 DIMELM: $CC i uttryckskoden, dimensionera en variabel . . . 108
3.43 FOR: kompilering . . . 111
3.44 FOR vid körning . . . 112
3.45 NEXT vid körning . . . 114
3.46 IF: kompilering . . . 118
3.47 INPUT: kompilering . . . 119
3.48 INPUT vid körning . . . 120
3.49 READ vid körning . . . 121
3.50 RDVAL: läs ett värde ur texten vid HL och tilldela det . . . 122
3.51 ON: kompilering . . . 125
3.52 ON vid körning . . . 126
3.53 OPEN / PREPARE: kompilering . . . 128
3.54 OPEN / PREPARE vid körning . . . 128
3.55 OUT / POKE: kompilering . . . 130
3.56 PRINT: kompilering . . . 132
3.57 PRINT vid körning . . . 134
3.58 RETURN . . . 138
3.59 DOTADR: skärmadress och bit för en grafikpunkt . . . 140
3.60 Uttryckskompilatorn: en post per prioritetsnivå . . . 145
3.61 FNTYPE: typer för funktionernas resultat och argument . . . 147
3.62 TYPCNV: omvandla värdet till typen A (bit 0-1) . . . 148
3.63 PRIMARY: kompilera ett primärt uttryck . . . 149
3.64 CMPVAR: kompilera en variabel, med index . . . 150
3.65 VARNAM: läs ett variabelnamn . . . 153
3.66 CMPEXP: kompilera ett uttryck till omvänd polsk notation . . . 154
3.67 EXEXP: utför kompilerad uttryckskod (RST $38) . . . 160
3.68 Värdestacken när uttryckskoden körs . . . 160
3.69 EXPTAB1: rutiner för token $B6-$FF vid körning . . . 161
3.70 EXPTAB2: funktionerna, index = kod (token i FNKW - $80) . . . 163
3.71 ASGNF / ASGNI / ASGNS: tilldelning ($B6-$B8) . . . 164
3.72 Jämförelse av flyttal, $E2-$F3 . . . 168
3.73 FCMP: jämför flyttal . . . 169
3.74 ADDF / SUBF / MULF / DIVF . . . 170
3.75 Logiska operatorer, $DC-$E1 . . . 172
3.76 Jämförelse av heltal, $E2-$F3 . . . 173
3.77 ADD$ SUB$ MUL$ DIV$ COMP%: aritmetik på tal skrivna som text . . . 176
3.78 ASCARG: kopiera ett strängargument till bufferten (DE) . . . 179
3.79 VAR0 ($C0-$C2): variabel utan index . . . 181
3.80 VAR1x / VAR2x: element i vektor eller matris . . . 181
3.81 Flyttalsfunktioner med ett argument . . . 185
3.82 Potens: POWF ($FD), POWI ($FE), POWFI ($FF) . . . 186
3.83 Strängfunktionerna . . . 191
3.84 CONCAT ($F6): a$ + b$ . . . 192
3.85 Strängjämförelser: EQS NES GTS LES LTS GES . . . 195
3.86 EXINST: INSTR(i, a$, b$) . . . 197
3.87 EXVAL: VAL(s$) . . . 201
3.88 EXRND: RND och RND(x) . . . 202
3.89 FNPAR: adressen till en DEF FN-parameter . . . 209
3.90 Flyttalsaritmetik: FADD FSUB FMUL FDIV . . . 211
3.91 IMUL / IDIV: 16-bitars heltal utan tecken . . . 222
3.92 ASCII-aritmetik: AADD ASUB ACOMP ADIV AMUL . . . 224
3.93 ADIV: a = a / b . . . 226
3.94 AMUL: a = a * b . . . 227
3.95 ASPARS: läs ett tal i text . . . 230

3.96 ASALGN: ställ upp a och b så att decimalerna står under varandra 231
3.97 ASFIN: gör resultatet i a läsbart 234
3.98 Flyttalsfunktioner: ATN EXP INT LOG LOG10 SIN COS TAN SQR 238
3.99 FSIN / FCOS 244
3.100 FSQR: kvadratrot, siffra för siffra med udda tal 245
3.101 FTOINT: flyttalet vid HL till heltal i DE 249
3.102 INTTOF: heltalet i HL (med tecken) som flyttal vid DE (exponenten) 251
3.103 FPOLY: P(z) = c0 + c1 z + ... + cn z^n med Horners metod 252

4 Fel och egenheter **255**

A Felkoder **261**

B Tokens och tabeller **263**
B.1 Direktkommandon 263
B.2 Satser 263
B.3 Satser, tabell 2 264
B.4 Övriga nyckelord 265
B.5 Operatorer 265
B.6 Funktioner 266
B.7 Uttryckskod 267

C Symbolförteckning **268**

Inledning

ABC80:s BASIC-ROM, 16 kilobyte Z80-kod på $0000–$3FFF, i versionen med kontrollsumman 11273. Koden i listningen är inte publicerad med dokumentet utan fylldes i ur ROM:en när dokumentet byggdes; namnen och kommentarerna kommer ur arbetet med boken *ABC80 inifrån*, där ROM:ens uppbyggnad och arbetssätt beskrivs.
Varje rad visar adressen, byten, eventuellt ett namn och instruktionen, följd av en kommentar. En rad visar högst fyra byte; har instruktionen eller tabellen fler står det tre punkter. Talen skrivs hexadecimalt med $ framför och decimalt utan. Namnen på rutiner och variabler är valda för genomgången; ROM:en själv innehåller inga. Längre förklaringar står som block före den rutin de gäller. Instruktionernas namn är Zilogs.
Namnen och adresserna är länkar i PDF-filen. Osäkra tolkningar är markerade med (?). Uppgifter från andra källor har källan inom parentes; källförteckningen finns i boken.

Hur ROM:en arbetar

ROM:en tolkar inte programraden tecken för tecken när den körs. En inskriven rad översätts först (TOKLIN): nyckelorden blir tokens om en eller två byte, talen lagras i binär form, och uttrycken blir en följd av koder som körs med en värdestack, operanderna före operatorn. Det är den kompilerade raden som ligger i minnet; LIST gör om den till text igen (DETOK). Före körningen går PRELNK igenom hela programmet och sätter in adresser: målradens adress i hoppen, variablernas plats på heapen, och vilken NEXT som hör till vilken FOR. Sedan körs satserna en efter en från EXESTM, och uttryckskoden från EXEXP genom RST $38.
Talen är antingen heltal på 16 bitar eller flyttal om fem byte med sex decimala siffror. Den egna ASCII-aritmetiken räknar i stället direkt på strängar av siffror, för ADD$ och de andra.
Tabellen visar ROM:ens delar i adressordning. Adresserna är länkar till listningen, och varje del har avsnitt med egna rubriker i innehållsförteckningen.

Från	Innehåll
$0000	RST-vektorerna, de fasta ingångarna och kallstarten (INIT)
$00C6	kommandoloopen: läs en rad, kör den direkt eller lägg in den i programmet
$018C	enheterna: skärmen, tangentbordet (GETKEY, INLINE) och kassetten
$062E	felhanteringen, ERRTRP och felingångarna
$0750	programraderna (INSLIN), OPEN, enhetsanropen och textfilerna
$09F5	nyckelordssökningen (KWSRCH) och kommandona ED, LIST, LOAD, MERGE, REN, RUN och SAVE
$0E6E	länkningen före körningen (PRELNK, SKPELM)
$1195	från kompilerad kod till text igen (DETOK, DETEXP), filnamnen
$1513	tal från text och till text (GETFLT, FLTASC)
$1890	kompileringen av en rad (TOKLIN, TOKKW) och satstabellerna
$1A62	satserna, från DEF och LET till RETURN
$23B5	uttryckskompilatorn
$2841	uttryckskoden vid körning: operatorerna, variablerna och funktionerna
$33F7	flyttalsaritmetiken och heltalens multiplikation och division
$37A6	ASCII-aritmetiken
$3B77–$3FFF	flyttalsfunktionerna med sina konstanter och serier; från $3FF5 oanvänt

Efter listningen följer bilagor med felkoderna, nyckelordens tokens och tabeller, och alla namn i bokstavsordning. Först kommer RAM-variablerna.

Systemvariabler och externa adresser

Tabellen säger vad de fasta adresserna i RAM, och de adresser i tilläggskretsarna som ROM:en går till, används till. Den har alla sådana adresser som listningen använder, i adressordning. Namnen är valda för listningen; där en adress inte har fått något namn står den bara med sin adress i koden.

Systemvariablerna ligger på $FDD0–$FE3F, och efter dem kommer radbufferten på $FE40. IY pekar hela tiden på $FE16. När ROM:en en variabel genom IY står förskjutningen i kolumnen IY; i koden står då till exempel (IY+$0F) för FLAGS, inte namnet.

Byten på $FDD0–$FDED har två användare. ASCII-aritmetiken (ADD$ och de andra) sparar där var de två talen a och b börjar och slutar. Flyttalsfunktionerna (ATN, LOG, SIN och de andra) räknar där med tal om fem byte, och PRINT skriver heltal där. De räknar aldrig samtidigt. Ett flyttal nås genom adressen till sin sista byte, exponenten; tecknet står i byten före.

Adress	IY	Namn	Användning
$405C		V405C	tilläggskretsen: målet för RST $08
$604B		V604B	tilläggskretsen (skivminnets DOS): INIT anropar adressen vid kallstart om det står ett JP ($C3) där
$604E–$605D			tilläggskretsen: BYE, NAME, UNSAVE och KILL, som stubbarna på $014C–$017A går till
$6051		V6051	tilläggskretsen: skriver felmeddelandet i stället för ERRMSG, om bit 0 i EXTFLG är satt
$7000–$7006			tilläggskretsen: CMD och IEC$ (stubbarna $0147, $0166 och $016B)
$7C00–$7FFF			bildminnet, 24 rader om 40 tecken; radernas adresser står i ROWTAB
$FB00			stackens början; INIT sätter SP och STKTOP hit
$FB00–$FCFF			kassettens två buffertar om 256 byte, $FB00 och $FC00; skrivning går genom $FC00: namnblocket (CASPRE), sedan datablocken från $FC03 (CASBRS); läsning växlar mellan dem (CASINT, XOR $07 i den höga byten)
$FDD0		ASBBEG	ASCII-aritmetiken: var b börjar
$FDD0–$FDD4			flyttalsfunktionerna: x, en kopia av argumentet (FLDX); tecknet på $FDD3 (FNEGX), exponenten på $FDD4
$FDD2		ASBEND	ASCII-aritmetiken: var b slutar (byten $83 efter sista siffran)
$FDD4		ASABEG	ASCII-aritmetiken: var a börjar
$FDD5–$FDD9			flyttalsfunktionerna: ett tal till (ATN: x + 1; LOG: exponenten gånger LN 10; SIN: q, sedan INT(q) och sist vinkeln v)
$FDD5–$FDDA			PRINT: ett heltal som text, mellanslag eller ”-” och högst fem siffror
$FDD6		ASAEND	ASCII-aritmetiken: var a slutar (byten $83)
$FDDA–$FDDE			ATN: t^2
$FDDF–$FDE3			TAN: en kopia av argumentet, sedan dess COS
$FDE4		SINQ	SIN och COS: vinkelns kvadrant
$FDE7		ASARG1	ASCII-aritmetiken: var kopian av a$ står i bufferten på stacken
$FDE9		ASEND	ASCII-aritmetiken: SP när argumenten började; flyttalsfunktionerna: där resultatet ska skrivas
$FDEB		ASDEC	ASCII-aritmetiken: n, antalet decimaler

Adress	IY	Namn	Användning
$FDED		ASCODE	ASCII-aritmetiken: DE, platsen i uttryckets kod, medan räkningen pågår
$FDEF		NMICNT	byten före klockan. NMICLK läser BC här, så att B blir klockans låga byte, och skriver tillbaka C som den var
$FDF0			klockans låga byte; räknas ned vid varje NMI, 50 gånger i sekunden
$FDF1		CLOCKH	klockans två höga byte ($FDF1–$FDF2); räknas ned var 256:e NMI
$FDF3		CURROW	markörens rad (0–23); CURADR lämnar IX här
$FDF4		CURCOL	markörens kolumn (0–39)
$FDF5		KEYFLG	bit 7 = en tangent väntar (KEYINT sätter den, GETKEY läser)
$FDF7		RPTDLY	autorepetitionen: tiden till nästa upprepning, i varv om 256; 70 efter en ny tangent, sedan 8
$FDF8		CASST	kassettläsningen: $FF medan ett block tas emot, sedan 0 = rätt, $A5 = ETX saknas, $A3 = fel checksumma
$FDF9		CASBUF	kassetten: bufferten som läses till (0 = ingen läsning pågår)
$FDFB		EDPTR	INLINE: mallraden som TAB-tangenten hämtar tecken ur (ED)
$FDFD		CASBLK	kassetten: blocknumret; $FFFF före första datablocket
$FDFF		RCVPTR	kassettläsningen: DE, nästa byte i bufferten, mellan avbrotten
$FE01		RCVBC	kassettläsningen: C = skiftregistret, B = bitar kvar
$FE03		RCVCONT	kassettläsningen: där läsningen fortsätter vid nästa avbrott
$FE05		RCVSUM	kassettläsningen: checksumman
$FE07		BRKFLG	skilt från 0 om BREAK (CTRL-C) har tryckts; TSTBRK prövar och nollar
$FE0A		DEVHEAD	första posten i enhetslistan; INIT sätter den till DEVLST i ROM:en
$FE0F		EXTFLG	bit 0 = tilläggskretsen skriver felmeddelandena ($6051); INIT nollar
$FE19	+$03		FDRND: bit 7 satt = en sjunde siffra som är exakt 5 avrundar inte uppåt. Ingen instruktion skriver byten, och INIT nollar den inte
$FE1A	+$04		ASCII-aritmetiken: a:s längd, sedan svarets
$FE1B	+$05		ASCII-aritmetiken: a:s decimaler, sedan svarets
$FE1C	+$06	BOFA	programmets början, första sidan med RAM från $8000 (INIT)
$FE1E		EOFA	programmets slut, där slutmarkeringen $01 står
$FE20	+$0A	HEAP	slutet av variablerna, första lediga byte; CLEAR sätter EOFA + 1
$FE23	+$0D		filen som PRINT och INPUT går till; 0 = skärmen och tangentbordet
$FE24	+$0E		DIRECT: 1 = raden körs direkt, 0 = ett program körs. I direktläge gäller inte ON ERROR, felet får inget radnummer, och NODIR ger fel 2
$FE25	+$0F	FLAGS	bit 4 = TRACE, bit 5 = skyddat program (raderas vid READY)
$FE26	+$10		LNKOK: PRELNK sätter 1 när hela programmet är länkat, CLEAR 0; ROM:en läser den inte
$FE27		STKTOP	stackens början; SP sätts hit vid varje ny kommandorad och efter ett fel
$FE29		VARLST	första posten i variabellistan (0 = inga variabler)
$FE2B	+$15	VFE2B	kortvalet: det senaste värdet till port 1, som skrivs dit igen före varje OUT och INP
$FE2C		CURLIN	raden som körs, eller hade fel
$FE2E		DATPTR	READ: nästa värde i DATA-raderna
$FE30		DATLIN	READ: DATA-raden som läses
$FE32		FILLST	fillistan: första FCB i kedjan
$FE33	+$1D	VFE33	FILLST:s höga byte; 0 = inga öppna filer
$FE34	+$1E	ONERR	ON ERROR GOTO: radens adress; höga byten (IY+$1F) 0 = avstängt. Nollas när felet fångas, så ON ERROR gäller ett fel
$FE36		STMSP	SP när satsen började; ON ERROR återställer stacken därifrån

Adress	IY	Namn	Användning
$FE38	+$22		ERRCOD: koden för det senaste felet, som ERRCODE ger; CLEAR nollar
$FE39–$FE3D			RNDSED: slumptalsfröet, 40 bitar, lägsta byten på $FE3D (EXRND når den som IY + $27); RANDOMIZE fyller det ur R
$FE3E		FNFRAM	ramen för det FN-anrop som räknas, alltså var argumenten står
$FE40–$FEB7	+$2A	BUF1	radbufferten, högst 120 tecken (CMDLP, ED, INPUT). Medan programmet länkas (PRELNK) står i $FE40–$FE41 det innersta öppna FOR-blocket; höga byten (IY+$2B) 0 = inget
$FE42	+$2C		länkningen: antalet parametrar i den DEF FN-rad som länkas (LNKDEF); 0 utanför en sådan rad
$FE43	+$2D		länkningen: parametrarnas namn, två byte var (namnbyten och bokstaven), som LNKPAR jämför med
$FEB8			TOKLIN: den kompilerade raden, med längd och radnummer som i programmet; en rad utan nummer körs från koden på $FEBB

Listning

3.1 RST-vektorer och fasta ingångar $0000

Mellan RST-adresserna ligger JP-instruktioner som fasta ingångar för maskinkodsprogram:

```
$0002 GETKEY  vänta på tangent, A = tecken
$0005 INLINE  läs en rad från tangentbordet (HL = buffert, C = max)
$000B PRTXT   skriv text på skärmen (HL = text, BC = längd)
$0015 TXTINP  läs rad från blockad textfil (enhetsrutin 3)
$001B TXTPRT  skriv till blockad textfil (enhetsrutin 4)
$0023 TXTEND  avsluta blockad textfil
$002D OPEN    öppna en fil
$003B, $003E OPNSTR, $0041, $0044 KWSRCH: interpretatorns rutiner
```

RST-rutinerna:

```
RST $08  hopp till tilläggs-ROM på $405C
RST $10  fel; efter RST-instruktionen följer felkoden
RST $18  INC HL och hoppa över blanktecken, A = nästa tecken
RST $20  hoppa över blanktecken, A = nästa tecken
RST $28  kastar returadressen och går till $2841 (PUSH HL, sedan som RST $38)
RST $30  kastar returadressen och går till RST $38
RST $38  utför kompilerad uttryckskod vid DE ($2842, EXPTAB1)
```

```
0000  18 66        RESET:  JR   INIT            hoppa över ingångarna

0002  C3 EA 02     J0002:  JP   GETKEY

0005  C3 AA 02     J0005:  JP   INLINE

0008  C3 5C 40     RST08:  JP   V405C

000B  C3 A8 01     J000B:  JP   PRTXT

000E  00 00                DEFB $00,$00         två byte över, för få för ett JP
0010  E1           RST10:  POP  HL              HL = adressen till felkoden
0011  7E                   LD   A,(HL)
0012  C3 2E 06             JP   ERROR

0015  C3 A0 09     J0015:  JP   TXTINP

0018  C3 BE 25     RST18:  JP   SKIPSP          från nästa tecken, utan blanka; A = tecknet

001B  C3 12 09     J001B:  JP   TXTPRT

001E  00 00                DEFB $00,$00         två byte över
0020  C3 BF 25     RST20:  JP   SKIPS0          som RST $18, men från (HL) självt

0023  C3 85 09     J0023:  JP   TXTEND

0026  00 00                DEFB $00,$00         två byte över
0028  33           RST28:  INC  SP              kasta returadressen; HL blir retur
0029  33                   INC  SP
002A  C3 41 28             JP   EXEXPH          HL på stacken, nästa token

002D  C3 E2 07     J002D:  JP   OPEN            fast ingång i luckan

0030  33           RST30:  INC  SP              kasta returadressen
0031  33                   INC  SP
0032  18 04                JR   RST38           över avbrottstabellen
```

IM 2-*vektortabell* (*I* = 0). *PIO A* (*tangentbord*) *har vektor* $34 *och PIO B* (*kassett*) *har vektor* $36.

```
0034  1E 03        IM2VEC: DEFW KEYINT
0036  94 05                DEFW CASINT
0038  C3 42 28     RST38:  JP   EXEXP

003B  C3 6C 23     J003B:  JP   CMPSTX

003E  C3 C2 07     J003E:  JP   OPNSTR

0041  C3 78 14     J0041:  JP   FNAME

0044  C3 F5 09     J0044:  JP   KWSRCH
```

NMI*: realtidsklocka.* NMI *kommer 50 gånger per sekund. Räknaren CLOCK* ($FDF0-$FDF2, *24 bitar*) *räknas ned med 1 per* NMI*. Den läses in som BC =* ($FDEF)*: B =* $FDF0 *räknas med* DJNZ*, och vid noll räknas* $FDF1-$FDF2 *ned.* $FDEF (*C*) *skrivs tillbaka oförändrad.*

```
0047  C5           NMICLK: PUSH BC             B = låga byten, C = $FDEF orörd
0048  ED 4B EF FD         LD   BC,(NMICNT)
004C  10 10               DJNZ L005E           inte noll än: bara spara
004E  ED 43 EF FD         LD   (NMICNT),BC
0052  ED 4B F1 FD         LD   BC,(CLOCKH)     var 256:e NMI: de höga bytena
0056  0B                 DEC  BC
0057  ED 43 F1 FD         LD   (CLOCKH),BC
005B  C1                 POP  BC
005C  ED 45              RETN

005E  ED 43 EF FD L005E: LD   (NMICNT),BC
0062  C1                 POP  BC
0063  ED 45              RETN

0065  45                 DEFB $45             används inte ($00 i version 9913)
0066  18 DF       NMI:    JR   NMICLK
```

3.2 Kallstart $0068

Initierar PIO:erna, sätter IM 2, stacken på $FB00 och IY = $FE16. Letar sedan nedåt från $FA00 efter lägsta sidan (256 byte) med skrivbart minne, dock inte under $8000. Där börjar programmet (BOFA). Enhetslistan initieras och en eventuell tilläggs-ROM anropas om det står JP ($C3) på $604B.

```
0068  DB 07        INIT:   IN   A,($07)        port 7: återställ ABC-bussen (Markesjö 1978, s. 122)
006A  ED 5E               IM   2
006C  3E CF               LD   A,$CF          PIO B: bitläge ...
006E  D3 3B               OUT  ($3B),A
0070  3E 87               LD   A,$87          ... bit 0–2 och 7 in, bit 3–6 ut
0072  D3 3B               OUT  ($3B),A
0074  3E 34               LD   A,$34          PIO A: avbrottsvektor $34
0076  D3 39               OUT  ($39),A
0078  3E B7               LD   A,$B7          PIO A: avbrott på, mask följer
007A  D3 39               OUT  ($39),A
007C  3E 7F               LD   A,$7F          bara bit 7 (strobe) ger avbrott
007E  D3 39               OUT  ($39),A
0080  21 00 FB            LD   HL,$FB00       stackens topp
0083  F9                  LD   SP,HL
0084  22 27 FE            LD   (STKTOP),HL    stacken återställs hit vid fel och i CMDLP
0087  FD 21 16 FE         LD   IY,$FE16       IY = bas för systemvariablerna
008B  AF                  XOR  A
008C  ED 47               LD   I,A            I = 0: vektortabellen ligger på $00xx
008E  32 0F FE            LD   (EXTFLG),A     inga tillägg som skriver felen
0091  32 25 FE            LD   (FLAGS),A      inte TRACE, inget skyddat program
```

```
0094  32 33 FE           LD   (VFE33),A
0097  6F                 LD   L,A            HL = $FB00: provet börjar på sidan under
0098  D3 06              OUT  ($06),A        tyst ljudgenerator (SN76477; Markesjö 1978, s. 97)
009A  32 F5 FD           LD   (KEYFLG),A     ingen tangent väntar
009D  3D                 DEC  A              A = $FF
009E  D3 39              OUT  ($39),A        PIO A: bitläge ...
00A0  D3 39              OUT  ($39),A        ... alla bitar in
```

RAM-test: skriv komplementet av innehållet, läs tillbaka och återställ. Nollflagga = minnet gick att skriva.

```
00A2  25           L00A2:  DEC  H
00A3  7E                   LD   A,(HL)         spara innehållet
00A4  47                   LD   B,A
00A5  2F                   CPL                 skriv komplementet ...
00A6  77                   LD   (HL),A
00A7  AE                   XOR  (HL)           ... och läs: Z om det gick
00A8  70                   LD   (HL),B         innehållet tillbaka
00A9  20 05                JR   NZ,L00B0       inget RAM här
00AB  7C                   LD   A,H
00AC  FE 80                CP   $80
00AE  30 F2                JR   NC,L00A2       inte under $8000
00B0  24           L00B0:  INC  H              första sidan med RAM
00B1  22 1C FE             LD   (BOFA),HL      BOFA
00B4  CD 76 02             CALL CLS            sudda skärmen
00B7  21 8C 01             LD   HL,DEVLST      enhetslistan börjar i ROM
00BA  22 0A FE             LD   (DEVHEAD),HL
00BD  3A 4B 60             LD   A,(V604B)      tilläggs-ROM med JP på $604B?
00C0  FE C3                CP   $C3
00C2  CC 4B 60             CALL Z,V604B        ... anropa den
00C5  FB                   EI                  vidare in i COLD
```

3.3 Kommandoloopen $00C6

COLD: NEW + CLEAR. READY: om bit 5 i $FE25 är satt (programmet var laddat som skyddat, typ $83) raderas det först. Skriver ”ABC80”. CMDLP läser en rad till bufferten $FE40 (högst 120 tecken). Tom rad ignoreras. Börjar raden med ett direktkommando (CMDKW) körs det via CMDTAB med READY som returadress. Annars tokeniseras raden:

- Med radnummer läggs raden in i programmet (INSLIN).
- Utan radnummer utförs raden direkt.

```
00C6  CD 70 0A        COLD:   CALL NEW            inget program
00C9  CD 80 0A        ENDPRG: CALL CLEAR          inga variabler, filer stängda
00CC  FD CB 0F 6E     READY:  BIT  5,(IY+$0F)     skyddat program: radera
00D0  20 F4                   JR   NZ,COLD
00D2  21 2D 07                LD   HL,D072D       CR LF ABC80
00D5  01 07 00                LD   BC,$0007
00D8  CD A8 01                CALL PRTXT          skriv ABC80
00DB  CD 42 07        CMDLP:  CALL NEWLIN
00DE  FD 36 0E 01             LD   (IY+$0E),$01   $FE24 = 1: direktläge
00E2  2A 27 FE                LD   HL,(STKTOP)    stacken som efter INIT
00E5  F9                      LD   SP,HL
00E6  21 40 FE                LD   HL,$FE40       radbuffert
00E9  01 78 00                LD   BC,$0078       max 120 tecken
00EC  E5                      PUSH HL             bufferten sparas
00ED  CD AA 02                CALL INLINE
00F0  CD 3E 03                CALL TSTBRK         nolla en BREAK från inmatningen
00F3  E1                      POP  HL             HL = bufferten
00F4  E7              L00F4:  RST  $20            första tecknet utan blanka
```

```
00F5  FE 0D              CP   $0D
00F7  28 E2              JR   Z,CMDLP
00F9  11 20 0A           LD   DE,CMDKW       direktkommando?
00FC  CD F5 09           CALL KWSRCH
00FF  28 32              JR   Z,DOCMD        Z: A = kommandots nummer
0101  CD 90 18           CALL TOKLIN         tokenisera raden
0104  DA 69 06           JP   C,ERRSTP       CY: A = felkoden
0107  DD 6E 01           LD   L,(IX+$01)     HL = radnumret
010A  DD 66 02           LD   H,(IX+$02)
010D  7D                 LD   A,L
010E  B4                 OR   H
010F  28 08              JR   Z,L0119        radnummer 0: kör direkt
0111  CD 50 07           CALL INSLIN         annars in i programmet
0114  30 C5              JR   NC,CMDLP       NC: klart, nästa rad
0116  C3 2E 06           JP   ERROR          CY: A = $83, MINNET FULLT

0119  DD 7E 00     L0119: LD   A,(IX+$00)     längd 0: ingen sats på raden
011C  A7                 AND  A
011D  28 BC              JR   Z,CMDLP
011F  CD 42 07           CALL NEWLIN
0122  21 BB FE           LD   HL,$FEBB       koden i direktläge, sparas för RUNSTM
0125  E5                 PUSH HL
0126  FD 36 2C 00        LD   (IY+$2C),$00   inga DEF-parametrar
012A  CD AA 0E           CALL LNKELM         länka raden
012D  D1                 POP  DE             DE = koden
012E  CD 92 0D           CALL RUNSTM
0131  18 99              JR   READY          tillbaka via READY
```

Direktkommando: A = kommandonummer från KWSRCH. Hämta adressen ur CMDTAB och hoppa dit med READY som returadress.

```
0133  01 CC 00     DOCMD: LD   BC,READY
0136  C5                 PUSH BC
0137  11 5A 0A           LD   DE,CMDTAB      två byte per kommando
013A  87                 ADD  A,A
013B  4F                 LD   C,A
013C  06 00              LD   B,$00
013E  EB                 EX   DE,HL
013F  09                 ADD  HL,BC
0140  7E                 LD   A,(HL)
0141  23                 INC  HL
0142  66                 LD   H,(HL)
0143  6F                 LD   L,A
0144  EB                 EX   DE,HL          DE = rutinen
0145  D5                 PUSH DE             hoppa dit, READY som retur
0146  C9                 RET
```

3.3.1 Anrop till tilläggs-ROM $0147

Efter CALL EXTJP1/EXTJP2 följer adressen till en ingång i en tilläggs-ROM. Om det står JP ($C3) på den adressen hoppar man dit. Annars gäller:

EXTJP1: återvänder till anroparens anropare med CY = 1, A = $88
EXTJP2: fel 8 (FINNS EJ I DETTA SYSTEM)

```
0147  CD 56 01     L0147:  CALL EXTJP1          CMD kompileras: $7000
014A  00 70                 DEFB $00,$70

014C  CD 56 01     L014C:  CALL EXTJP1          UNSAVE: $605A (fel)
014F  5A 60                 DEFB $5A,$60

0151  CD 56 01     L0151:  CALL EXTJP1          NAME kompileras: $6054
0154  54 60                 DEFB $54,$60

0156  E3          EXTJP1: EX   (SP),HL         HL = adressen efter CALL
0157  7E                  LD   A,(HL)          måladressen, låga byten
0158  23                  INC  HL
0159  66                  LD   H,(HL)          höga byten
015A  6F                  LD   L,A             HL = måladressen
015B  7E                  LD   A,(HL)          första byten i tilläggskretsen
015C  FE C3               CP   $C3             JP-instruktion där?
015E  E3                  EX   (SP),HL         måladressen i returadressens ställe
015F  C8                  RET  Z               ja: hoppa dit via RET
0160  33                  INC  SP              nej: kasta måladressen
0161  33                  INC  SP
0162  3E 88               LD   A,$88           A = $80 + 8: fel 8
0164  37                  SCF                  CY = 1: felet till anroparen
0165  C9                  RET                  tillbaka förbi stubben

0166  CD 7F 01     L0166:  CALL EXTJP2          CMD körs: $7003
0169  03 70                 DEFB $03,$70

016B  CD 7F 01     EXIEC:  CALL EXTJP2          IEC$: $7006
016E  06 70                 DEFB $06,$70

0170  CD 7F 01     L0170:  CALL EXTJP2          KILL körs: $605D
0173  5D 60                 DEFB $5D,$60

0175  CD 7F 01     L0175:  CALL EXTJP2          NAME körs: $6057
0178  57 60                 DEFB $57,$60

017A  CD 7F 01     L017A:  CALL EXTJP2          BYE: $604E
017D  4E 60                 DEFB $4E,$60

017F  E3          EXTJP2: EX   (SP),HL         HL = adressen efter CALL
0180  7E                  LD   A,(HL)          måladressen, låga byten
0181  23                  INC  HL
0182  66                  LD   H,(HL)          höga byten
0183  6F                  LD   L,A             HL = måladressen
0184  7E          L0184:  LD   A,(HL)          hit också från DVCALL ($08BA)
0185  FE C3               CP   $C3
0187  C2 09 07           JP   NZ,L0709        saknas: fel 8
018A  E3                  EX   (SP),HL         hoppa dit, anroparen som retur
018B  C9                  RET
```

3.4 Enhetslistan $018C

DEVHEAD ($FE0A) pekar på första posten. Varje post är sju byte:

```
+0,+1  pekare till nästa post (0 = sista)
+2..+4 enhetsnamn, 3 tecken ("CAS", "PR " ...)
+5,+6  adress till enhetens hopptabell med 9 JP-instruktioner:
         0 OPEN     öppna befintlig fil för läsning
         1 PREPARE  skapa ny fil för skrivning
         2 CLOSE    stäng
         3 INPUT    läs en textrad
         4 PRINT    skriv text
         5 GET      läs nästa block (blockade filer)
         6 PUT      skriv block
         7, 8       används inte av CAS (fel 52)
```

Posten vars namn är tre blanktecken är standardenheten när inget namn anges. Den pekar på samma tabell som CAS:. PR: och IEC: har sina hopptabeller i tilläggskretsar ($7800, $7009). Eftersom listan börjar i RAM-variabeln DEVHEAD kan program och tilläggs-ROM:ar länka in nya enheter först i listan.

Uppslagning av enheten sker i OPEN ($07E2) och anrop av en rutin via DVCALL ($08A5).

```
018C  93 01        DEVLST: DEFW DEVDFL          nästa post: standardenheten
018E  43 41 53             DEFM "CAS"           CAS:
0191  A4 03                DEFW CASTAB          kassettens hopptabell
0193  9A 01        DEVDFL: DEFW DEVPR           nästa: PR:
0195  20 20 20             DEFM "   "           tre blanka: ingen enhet angiven
0198  A4 03                DEFW CASTAB          samma tabell som CAS:
019A  A1 01        DEVPR:  DEFW DEVIEC          nästa: IEC:
019C  50 52 20             DEFM "PR "           PR:
019F  00 78                DEFW $7800           hopptabell i en tilläggskrets
01A1  00 00        DEVIEC: DEFW $0000           sista posten
01A3  49 45 43             DEFM "IEC"
01A6  09 70                DEFW $7009           hopptabell i en tilläggskrets
```

3.5 PRTXT: skriv text på skärmen $01A8

In: HL = text, BC = antal tecken.

Styrtecken:

```
$07  pip (ljudgeneratorn via port 6)
$08  markören ett steg vänster
$0A  ny rad (rullar skärmen på sista raden)
$0C  sudda skärmen
$0D  markören till början av raden
$1B  ESC "=" rad+32 kolumn+32: flytta markören (som ADM-3A)
```

Övriga tecken skrivs med bit 7 nollställd. Efter kolumn 39 blir det automatiskt ny rad.

Ut: HL efter texten, BC = 0 och DE = markörens adress i bildminnet. A förstörs; IX bevaras.

```
01A8  DD E5        PRTXT:  PUSH IX              (egenhet)
01AA  CD 93 02     L01AA:  CALL CURADR          DE = markören, IX = $FDF3
01AD  79           L01AD:  LD   A,C
01AE  B0                   OR   B
01AF  CA E7 02             JP   Z,L02E7         slut: stäng
01B2  7E                   LD   A,(HL)
01B3  23                   INC  HL
01B4  0B                   DEC  BC
01B5  FE 07                CP   $07             BEL
01B7  28 32                JR   Z,L01EB
```

```
01B9  FE 08              CP   $08            BS
01BB  28 5D              JR   Z,L021A
01BD  FE 0A              CP   $0A            LF
01BF  28 25              JR   Z,L01E6
01C1  FE 0D              CP   $0D            CR
01C3  28 1B              JR   Z,L01E0
01C5  FE 0C              CP   $0C            FF
01C7  28 2B              JR   Z,L01F4
01C9  FE 1B              CP   $1B            ESC
01CB  28 2C              JR   Z,L01F9
01CD  E6 7F              AND  $7F            vanligt tecken
01CF  12                 LD   (DE),A         rakt in i bildminnet
01D0  13                 INC  DE
01D1  DD 34 01           INC  (IX+$01)       kolumn + 1
01D4  3A F4 FD           LD   A,(CURCOL)
01D7  FE 28              CP   $28            kolumn 40?
01D9  38 D2              JR   C,L01AD
01DB  CD 34 02           CALL CRLF           rad full: ny rad
01DE  18 CA              JR   L01AA          ny rad: hämta markören igen

01E0  DD 36 01 00  L01E0: LD   (IX+$01),$00  CR: kolumn 0
01E4  18 C4              JR   L01AA

01E6  CD 38 02     L01E6: CALL LINFD         LF: ny rad, samma kolumn
01E9  18 BF              JR   L01AA

01EB  AF           L01EB: XOR  A             pip
01EC  D3 06              OUT  ($06),A
01EE  3E 83              LD   A,$83          styrord för tonen
01F0  D3 06              OUT  ($06),A
01F2  18 B6              JR   L01AA

01F4  CD 76 02     L01F4: CALL CLS           FF: sudda skärmen
01F7  18 B1              JR   L01AA

01F9  EB           L01F9: EX   DE,HL         ESC: minst 3 tecken kvar?
01FA  21 FD FF           LD   HL,$FFFD
01FD  09                 ADD  HL,BC
01FE  EB                 EX   DE,HL
01FF  30 A9              JR   NC,L01AA       färre: ESC ignoreras
0201  7E                 LD   A,(HL)
0202  FE 3D              CP   $3D            ESC =
0204  20 A4              JR   NZ,L01AA       inte =: ESC ignoreras
0206  23                 INC  HL             hoppa över =
0207  0B                 DEC  BC
0208  1E 18              LD   E,$18          rad 0..23
020A  CD 27 02           CALL ESCARG
020D  DD 77 00           LD   (IX+$00),A     rad
0210  1E 28              LD   E,$28          kolumn 0..39
0212  CD 27 02           CALL ESCARG
0215  DD 77 01           LD   (IX+$01),A     kolumn
0218  18 90              JR   L01AA

021A  1B           L021A: DEC  DE            BS: kolumn - 1 om möjligt
021B  DD 35 01           DEC  (IX+$01)
021E  F2 AD 01           JP   P,L01AD        var inte 0: klart
0221  13                 INC  DE             kolumn 0: ångra
0222  DD 34 01           INC  (IX+$01)
0225  18 86              JR   L01AD
```

ESCARG: *läs en koordinat vid HL (tecknet - 32) och begränsa den till 0 till E-1. Ut: A = koordinaten; HL och BC ett tecken framåt.*

```
0227  7E           ESCARG: LD   A,(HL)
0228  23                   INC  HL
0229  0B                   DEC  BC
022A  D6 20                SUB  $20            koordinat - 32
022C  30 01                JR   NC,L022F
022E  AF                   XOR  A              negativ: 0
022F  BB           L022F:  CP   E
0230  D8                   RET  C
0231  7B                   LD   A,E            för stor: E - 1
0232  3D                   DEC  A
0233  C9                   RET
```

CRLF*: markören till kolumn 0 och ny rad.* LINFD*: ny rad i samma kolumn. In: IX =* $FDF3*. På rad 23 rullas skärmen: rad 1 till 23 kopieras upp ett steg via* ROWTAB *och sista raden suddas. Förstör A och DE; BC och HL bevaras. Efter en rullning pekar IX in i* ROWTAB*, så anroparen hämtar IX igen med* CURADR *(*PRTXT *på* $01AA*,* INLINE *när* GETKEY *anropas).*

```
0234  DD 36 01 00  CRLF:   LD   (IX+$01),$00
0238  DD 34 00      LINFD:  INC  (IX+$00)
023B  DD 7E 00               LD   A,(IX+$00)
023E  FE 18                  CP   $18
0240  D8                    RET  C              rad under 24: klart
0241  DD 36 00 17            LD   (IX+$00),$17   stanna på rad 23 och rulla
0245  DD 21 74 03            LD   IX,ROWTAB      rulla skärmen: kopiera rad n+1 till rad n
0249  3E 18                  LD   A,$18          24 rader: 23 flyttas, den sista suddas
024B  E5                     PUSH HL
024C  C5                     PUSH BC
024D  01 28 00      L024D:   LD   BC,$0028       40 tecken per rad
0250  DD 5E 00               LD   E,(IX+$00)     DE = rad n
0253  DD 56 01               LD   D,(IX+$01)
0256  3D                    DEC  A
0257  28 10                  JR   Z,L0269        sista raden: sudda den (CLRROW tar BC och HL från
                                                 stacken)
0259  DD 6E 02               LD   L,(IX+$02)     HL = rad n + 1
025C  DD 66 03               LD   H,(IX+$03)
025F  ED B0                  LDIR
0261  DD 23                  INC  IX             nästa post i ROWTAB
0263  DD 23                  INC  IX
0265  18 E6                  JR   L024D
```

CLRROW*: fyll raden vid DE (40 tecken) med blanktecken. Ut: DE efter raden. A, BC och HL bevaras;* CLS *räknar raderna i A.*

```
0267  E5           CLRROW: PUSH HL
0268  C5                   PUSH BC
0269  62           L0269:  LD   H,D             blanka i första tecknet ...
026A  6B                   LD   L,E
026B  36 20                LD   (HL),$20
026D  13                   INC  DE
026E  01 27 00             LD   BC,$0027
0271  ED B0                LDIR                ... och kopiera det framåt
0273  C1                   POP  BC
0274  E1                   POP  HL
0275  C9                   RET
```

CLS*: sudda alla 24 rader och sätt markören i övre vänstra hörnet. Förstör A, DE och IX.*

```
0276  3E 18         CLS:    LD   A,$18          24 rader
0278  DD 21 74 03           LD   IX,ROWTAB
027C  DD 5E 00     L027C:   LD   E,(IX+$00)
```

```
027F  DD 56 01          LD   D,(IX+$01)
0282  DD 23             INC  IX
0284  DD 23             INC  IX
0286  CD 67 02          CALL CLRROW          sudda raden
0289  3D                DEC  A
028A  20 F0             JR   NZ,L027C
028C  5F                LD   E,A             A = 0: rad 0, kolumn 0
028D  57                LD   D,A
028E  ED 53 F3 FD       LD   (CURROW),DE
0292  C9                RET
```

CURADR: *DE = adressen i bildminnet för markören (CURROW, CURCOL). Ut: IX = $FDF3, alltså IX+0 rad, IX+1 kolumn och IX+2 tangentflagga. Förstör A; HL bevaras. Kolumnen läggs till radens låga byte utan minnessiffra: den största låga byten i ROWTAB är $D0, och $D0 + 39 är mindre än $100.*

```
0293  E5          CURADR: PUSH HL
0294  2A F3 FD            LD   HL,(CURROW)     L = rad, H = kolumn
0297  7C                 LD   A,H
0298  26 00              LD   H,$00
029A  29                 ADD  HL,HL           två byte per rad i ROWTAB
029B  11 74 03           LD   DE,ROWTAB
029E  19                 ADD  HL,DE
029F  86                 ADD  A,(HL)          radens adress + kolumnen
02A0  23                 INC  HL
02A1  66                 LD   H,(HL)
02A2  6F                 LD   L,A
02A3  EB                 EX   DE,HL
02A4  E1                 POP  HL
02A5  DD 21 F3 FD        LD   IX,$FDF3
02A9  C9                 RET
```

3.6 INLINE: radinmatning från tangentbordet $02AA

In: HL = buffert, C = max antal tecken. Raden ekas på skärmen och avslutas med CR i bufferten.

```
CR         avslutar
$18        (CTRL-X) suddar hela raden
$08        suddar ett tecken
$09 (TAB)  hämtar nästa tecken ur mallraden som EDPTR pekar på
           (används av ED; INLINE sätter mallen till en tom rad)
```

INLIN2 tar mallens adress i DE.

Ut: HL vid radens CR i bufferten. A, B och DE förstörs; IX bevaras.

```
02AA  11 4E 03     INLINE: LD   DE,$034E        mall = $0D inne i CP $0D: tom
02AD  ED 53 FB FD  INLIN2: LD   (EDPTR),DE
02B1  DD E5                PUSH IX
02B3  41                   LD   B,C             B = platser kvar
02B4  CD 5B 03     L02B4:  CALL RUBOUT          radera allt inmatat
02B7  20 FB                JR   NZ,L02B4
02B9  CD 5B 03     L02B9:  CALL RUBOUT          BS: ett tecken bort
02BC  CD EA 02     L02BC:  CALL GETKEY
02BF  FE 09                CP   $09
02C1  CC 48 03             CALL Z,EDTAB         TAB: tecken från mallen
02C4  FE 0D                CP   $0D
02C6  28 1D                JR   Z,L02E5         CR: klart
02C8  FE 18                CP   $18             CTRL-X
02CA  28 E8                JR   Z,L02B4
02CC  FE 08                CP   $08             BS
02CE  28 E9                JR   Z,L02B9
```

```
02D0  FE 20              CP   $20             övriga styrtecken ignoreras
02D2  38 E8              JR   C,L02BC
02D4  77                 LD   (HL),A          in i bufferten ...
02D5  23                 INC  HL
02D6  12                 LD   (DE),A          ... och på skärmen
02D7  DD 34 01           INC  (IX+$01)        kolumn + 1
02DA  DD 7E 01           LD   A,(IX+$01)
02DD  FE 28              CP   $28
02DF  D4 34 02           CALL NC,CRLF         kolumn 40: ny rad
02E2  10 D8              DJNZ L02BC           plats kvar: nästa tangent
02E4  2B                 DEC  HL              full: sista tecknet blir CR
02E5  36 0D        L02E5: LD  (HL),$0D
02E7  DD E1        L02E7: POP IX
02E9  C9                 RET
```

3.7 GETKEY: vänta på en tangent $02EA

Sätter bit 7 i tecknet under markören medan den väntar. Tangenten kommer från avbrottsrutinen KEYINT, som sätter bit 7 i KEYFLG. Autorepetition: så länge tangenten hålls ned (bit 7 i port $38) räknas RPTCNT/RPTDLY ned. Första fördröjningen är 70 × 256 varv, och därefter upprepas tangenten efter var 8 × 256 varv.

Ut: A = teckenkod (7 bitar), DE = markörens adress och IX = $FDF3. HL bevaras.

```
02EA  FB           GETKEY: EI
02EB  CD 93 02             CALL CURADR          DE = markören, IX = $FDF3
02EE  EB                   EX   DE,HL
02EF  CB FE                SET  7,(HL)          markera markören
02F1  DD CB 02 7E  L02F1:  BIT  7,(IX+$02)      tangent från avbrottet?
02F5  20 1B                JR   NZ,L0312        ja: läs den nya tangenten
02F7  DB 38                IN   A,($38)         hålls tangenten ned?
02F9  87                   ADD  A,A             bit 7 (nedtryckt) -> CY
02FA  38 06                JR   C,L0302         ja: räkna mot repetitionen
02FC  DD 36 04 46          LD   (IX+$04),$46    nej: starta om fördröjningen
0300  18 EF                JR   L02F1

0302  DD 35 03     L0302:  DEC  (IX+$03)        inre räknare, 256 varv
0305  20 EA                JR   NZ,L02F1
0307  DD 35 04             DEC  (IX+$04)        yttre: 70 första gången, sedan 8
030A  20 E5                JR   NZ,L02F1
030C  DD 36 04 08          LD   (IX+$04),$08    8 omgångar till nästa repetition
0310  18 04                JR   L0316           repetera: läs tangenten igen

0312  DD CB 02 BE  L0312:  RES  7,(IX+$02)      kvittera tangenten
0316  DB 38        L0316:  IN   A,($38)         tangentens kod utan strobe
0318  E6 7F                AND  $7F
031A  CB BE                RES  7,(HL)          ta bort markören
031C  EB                   EX   DE,HL
031D  C9                   RET
```

KEYINT: tangentbordsavbrott (vektor $34). CTRL-C ($83 med strobebiten) sätter BRKFLG. Vid varje tangent skrivs $80 till ljudporten och KEYFLG, och repetitionens fördröjning startas om. Alla register bevaras.

```
031E  F5           KEYINT: PUSH AF
031F  DB 38                IN   A,($38)
0321  FE 83                CP   $83             CTRL-C (med strobebiten)
0323  20 03                JR   NZ,L0328
0325  32 07 FE             LD   (BRKFLG),A      BRKFLG = $83: TSTBRK ser det
0328  3E 80        L0328:  LD   A,$80           A = $80 både ...
032A  D3 06                OUT  ($06),A         ... till ljudet och ...
```

```
032C  32 F5 FD            LD   (KEYFLG),A      ... som tangentflagga (bit 7)
032F  3E 46               LD   A,$46           full fördröjning före första repetitionen
0331  32 F7 FD            LD   (RPTDLY),A
0334  F1                  POP  AF
0335  FB                  EI
0336  ED 4D               RETI
```

SCRPOS: *A = aktuell kolumn, E = radbredd (40), för skärmen.*

```
0338  3A F4 FD     SCRPOS: LD   A,(CURCOL)
033B  1E 28               LD   E,$28
033D  C9                  RET
```

TSTBRK: *NZ om BREAK (CTRL-C) har tryckts. Flaggan nollställs. Kommer tangentavbrottet mellan läsningen och skrivningen av 0, går BREAK förlorad; rutinen anropas bara en gång per sats, så det händer sällan.*

```
033E  3A 07 FE  TSTBRK: LD   A,(BRKFLG)      $83 efter CTRL-C, annars 0 (fel)
0341  A7                AND  A               Z = 1: inget CTRL-C
0342  3E 00              LD   A,$00           A = 0 utan att röra flaggorna
0344  32 07 FE          LD   (BRKFLG),A      nolla flaggan
0347  C9                RET                  Z säger om CTRL-C kom
```

EDTAB: *TAB i INLINE, hämta nästa tecken ur mallraden vid EDPTR. Ut: A = tecknet. Vid mallens CR blir A = TAB, som INLINE inte tar med, och EDPTR står kvar. HL bevaras.*

```
0348  E5           EDTAB:  PUSH HL              mallens nästa tecken
0349  2A FB FD             LD   HL,(EDPTR)
034C  7E                   LD   A,(HL)
034D  FE 0D                CP   $0D
034F  28 06                JR   Z,L0357
0351  23                   INC  HL
0352  22 FB FD             LD   (EDPTR),HL
0355  E1                   POP  HL
0356  C9                   RET

0357  3E 09        L0357:  LD   A,$09           mallen slut: ge TAB (ignoreras)
0359  E1                   POP  HL
035A  C9                   RET
```

RUBOUT: *backa ett tecken, inte förbi radens början. In: B = platser kvar, C = största antal, HL = bufferten, IX = $FDF3. Ut: Z om inget fanns att ta bort; annars NZ, HL och B ett tecken tillbaka och tecknet suddat på skärmen. Förstör A och DE.*

```
035B  78           RUBOUT: LD   A,B             inget inmatat: Z, inget görs
035C  B9                   CP   C
035D  C8                   RET  Z
035E  DD 35 01             DEC  (IX+$01)
0361  F2 6B 03             JP   P,L036B         kolumn >= 0: samma rad
0364  DD 35 00             DEC  (IX+$00)        annars sista kolumnen på raden ovanför
0367  DD 36 01 27          LD   (IX+$01),$27
036B  2B           L036B:  DEC  HL
036C  CD 93 02             CALL CURADR
036F  3E 20                LD   A,$20
0371  12                   LD   (DE),A          sudda tecknet
0372  04                   INC  B               en plats till; NZ
0373  C9                   RET
```

Bildminnets radadresser. Raderna ligger i tre grupper om 8. Inom en grupp är avståndet $80*, och grupperna börjar på* $7C00*,* $7C28 *och* $7C50*.*

```
0374  00 7C        ROWTAB: DEFW $7C00          rad 0: grupp 1 börjar
0376  80 7C                DEFW $7C80          rad 1
0378  00 7D                DEFW $7D00          rad 2
037A  80 7D                DEFW $7D80          rad 3
037C  00 7E                DEFW $7E00          rad 4
037E  80 7E                DEFW $7E80          rad 5
0380  00 7F                DEFW $7F00          rad 6
0382  80 7F                DEFW $7F80          rad 7
0384  28 7C                DEFW $7C28          rad 8: grupp 2 börjar
0386  A8 7C                DEFW $7CA8          rad 9
0388  28 7D                DEFW $7D28          rad 10
038A  A8 7D                DEFW $7DA8          rad 11
038C  28 7E                DEFW $7E28          rad 12
038E  A8 7E                DEFW $7EA8          rad 13
0390  28 7F                DEFW $7F28          rad 14
0392  A8 7F                DEFW $7FA8          rad 15
0394  50 7C                DEFW $7C50          rad 16: grupp 3 börjar
0396  D0 7C                DEFW $7CD0          rad 17
0398  50 7D                DEFW $7D50          rad 18
039A  D0 7D                DEFW $7DD0          rad 19
039C  50 7E                DEFW $7E50          rad 20
039E  D0 7E                DEFW $7ED0          rad 21
03A0  50 7F                DEFW $7F50          rad 22
03A2  D0 7F                DEFW $7FD0          rad 23
```

Korsreferenstabell för starten, skärmen och tangentbordet. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är byten vid IY+$0E (direktläget) och FLAGS vid IY+$0F (bit 4 TRACE, bit 5 skyddat program). Skärmrutinerna har IX = $FDF3, så att IX+0 och IX+1 är CURROW och CURCOL, IX+2 KEYFLG, IX+3 den inre repetitionsräknaren $FDF6 och IX+4 RPTDLY. CY-kolumnen gäller flaggorna: TSTBRK och RUBOUT svarar med Z. Det som en anropad rutin gör räknas med: PRTXT rullar och suddar genom CRLF, LINFD och CLS, INLINE väntar på tangenten i GETKEY, och COLD och ENDPRG nollar genom NEW och CLEAR också EOFA, VARLST, HEAP, fillistan, ONERR och felkoden. EXTJP1 och EXTJP2 hoppar till tilläggs-ROM:en med registren orörda när det står JP på måladressen; tabellen visar fallet när den saknas. * = kommer inte tillbaka; för dem visas bara stacken och variablerna, och det de gör innan de går vidare till nästa (INIT till COLD, COLD till READY, READY till CMDLP). NMICLK räknar klockan (NMICNT, CLOCKH) och bevarar alla register; de fasta hoppen $0002–$0044, vektortabellen och stubbarna efter $0147 har inga egna rader. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler									IY+	
Rutin	A	CY	BC	DE	HL	IX	stack	CURROW	CURCOL	KEYFLG	RPTDLY	EDPTR	BRKFLG	STKTOP	BOFA	DEVHEAD	EXTFLG	$0E	$0F
INIT*							U	U	U	U				U	U	U	U		U
COLD*															I				IU
ENDPRG*																			
READY*								IU	IU										I
CMDLP*							U	IU	IU	IU	IU	U	IU	I				U	
DOCMD*	I						U												
EXTJP1	U	U					I												
EXTJP2	X	X					I												
PRTXT	X	X	IU	U	IU			IU	IU										
ESCARG	U	X	IU	I	IU														
CRLF	X	X		X		IX		IU	U										
LINFD	X	X		X		IX		IU											

	register och stack							variabler										IY+	
Rutin	A	CY	BC	DE	HL	IX	stack	CURROW	CURCOL	KEYFLG	RPTDLY	EDPTR	BRKFLG	STKTOP	BOFA	DEVHEAD	EXTFLG	$0E	$0F
CLRROW				IU															
CLS	X			X		X		U	U										
CURADR	X	X		U		U		I	I										
INLINE	X	X	IX	X	IU			IU	IU	IU	IU	U							
INLIN2	X	X	IX	IX	IU			IU	IU	IU	IU	U							
GETKEY	U	X		U		U		I	I	IU	IU								
KEYINT										U	U		U						
SCRPOS	U			U					I										
TSTBRK	X	U											IU						
EDTAB	U	X										IU							
RUBOUT	X	U	IU	X	IU	I		IU	IU										

Rutin	**Anropar**	**Anropas från**
INIT	CLS, tilläggs-ROM:en vid V604B; går vidare till COLD	RESET
COLD, ENDPRG	NEW (COLD), CLEAR; går vidare till READY	READY (skyddat program); RUNLIN och STEXE2 (ENDPRG)
READY	COLD, PRTXT; går vidare till CMDLP	DOCMD och ERRSTP (som återhopp), CMDLP
CMDLP	NEWLIN, INLINE, TSTBRK, KWSRCH, TOKLIN, INSLIN, LNKELM, RUNSTM; DOCMD, ERRSTP, ERROR	READY (går in i den)
DOCMD	kommandot ur CMDTAB	CMDLP
EXTJP1	tilläggs-ROM:en	stubbarna för CMD, UNSAVE och NAME ($0147–$0151)
EXTJP2	tilläggs-ROM:en; annars fel 8	stubbarna för CMD, IEC$, KILL, NAME och BYE ($0166–$017A)
PRTXT	CURADR, CRLF, LINFD, CLS, ESCARG	den fasta ingången $000B, READY, CASOPN, ERRMSG, NEWLIN, PRINT, CMDED, CMDLIST, INPT, TRCPRT
ESCARG		PRTXT
CRLF, LINFD	CLRROW (rullningen)	PRTXT, INLINE
CLRROW		CLS, LINFD
CLS	CLRROW	INIT, PRTXT
CURADR		PRTXT, GETKEY, RUBOUT
INLINE, INLIN2	RUBOUT, GETKEY, EDTAB, CRLF	den fasta ingången $0005, CMDLP, INPUT (INLINE); CMDED (INLIN2)
GETKEY	CURADR	den fasta ingången $0002, INLINE, CMDLIST, GET
KEYINT		avbrottet från PIO A (vektor $34)
SCRPOS		GETPOS, TRCPRT
TSTBRK		CMDLP, CMDLIST, RUNSTM
EDTAB		INLINE
RUBOUT	CURADR	INLINE

3.8 Kassetten $03A4

Hopptabell för CAS: (och för standardenheten, vars namn är tre blanktecken), se enhetslistan.
Kassettfil:

```
Varje block är 256 byte. Det skrivs som 256 nollbitar, 3 x SYN
($16), STX ($02), 256 databyte, ETX ($03), en 16-bitars
checksumma (summan av databyten + 3) och sist en byte $00.
Namnblock: $FF $FF $FF + 11 tecken namn (8+3, blankutfyllt).
Datablock: $00, blocknummer (lo, hi), 253 byte data.
```

Bufferten är $FC00, eller vid läsning växelvis $FC00/$FB00. FCB-fälten (IX) som används:

```
+7     radbredd ($84 = 132)
+8,+9  buffertens dataområde (= $FC03)
+A,+B  aktuell position i bufferten
+D     lediga byte kvar i blocket
+E     bit 7: textdata har skrivits
```

```
03A4  C3 91 04     CASTAB: JP   CASOPN          0 OPEN: leta efter filen
03A7  C3 BF 03     L03A7:  JP   CASPRE          1 PREPARE: skriv namnblocket
03AA  C3 28 04     L03AA:  JP   CASCLO          2 CLOSE
03AD  C3 A0 09     L03AD:  JP   TXTINP          3 INPUT: en textrad
03B0  C3 12 09     L03B0:  JP   TXTPRT          4 PRINT: text
03B3  C3 09 05     L03B3:  JP   CASGET          5 GET: nästa block
03B6  C3 0E 04     L03B6:  JP   CASPUT          6 PUT: skriv blocket
03B9  C3 27 07     L03B9:  JP   L0727           7: fel 52, EJ TILL DENNA ENHET
03BC  C3 27 07     L03BC:  JP   L0727           8: likaså
```

CASPRE: *PREPARE för kassett. Bygger namnblocket i sidan* $FC00 ($FF $FF $FF *och de 11 tecknen från DE), startar motorn, väntar ungefär 6,8 s och skriver blocket. Sedan nollas* CASBUF, CASBLK *blir* $FFFF *så att första datablocket får nummer 0, och bufferten görs tom i* CASBRS. *In: DE = det tolkade namnet, IX = FCB. Ut: DE =* $FC03. *Förstör A, BC och HL.*

```
03BF  21 00 FC     CASPRE: LD   HL,$FC00        namnblocket byggs i $FC00
03C2  E5                   PUSH HL              $FC00 till CASWBL
03C3  36 00        L03C3:  LD   (HL),$00        nolla hela sidan
03C5  2C                   INC  L
03C6  20 FB                JR   NZ,L03C3
03C8  06 03                LD   B,$03           $FF $FF $FF = namnblock
03CA  36 FF        L03CA:  LD   (HL),$FF
03CC  23                   INC  HL
03CD  10 FB                DJNZ L03CA
03CF  EB                   EX   DE,HL           HL = namnet, DE = $FC03
03D0  01 0B 00             LD   BC,$000B        11 tecken namn
03D3  ED B0                LDIR
03D5  3E 20                LD   A,$20           motor på
03D7  D3 3A                OUT  ($3A),A
03D9  06 0C                LD   B,$0C           vänta på att bandet kommer upp i fart
03DB  2B           L03DB:  DEC  HL              12 x ca 65536 varv à 26 T, ca 6,8 s
03DC  7C                   LD   A,H
03DD  B5                   OR   L
03DE  20 FB                JR   NZ,L03DB
03E0  10 F9                DJNZ L03DB
03E2  E1                   POP  HL              HL = $FC00
03E3  CD 39 04             CALL CASWBL          skriv namnblocket
03E6  21 00 00             LD   HL,$0000
03E9  22 F9 FD             LD   (CASBUF),HL     CASBUF = 0: ingen mottagning
```

```
03EC  DD 36 0E 01          LD   (IX+$0E),$01     +E = 1 (död: OPEN nollar +E efteråt)
03F0  2B                   DEC  HL               blocknummer = -1
03F1  22 FD FD             LD   (CASBLK),HL
```

CASBRS: *gör bufferten* $FC00 *tom för skrivning:* $03 *först i dataområdet, FCB+8,+9 och +A,+B =* $FC03*, +D = 252 lediga byte och +7 = radbredd 132. Ingången* $03F7 *tar dataområdet i HL;* CASOPN *går dit med* $FC03 *eller* $FB03*. Ut: DE = dataområdet, HL = anroparens DE. A, BC och flaggorna bevaras.*

```
03F4  21 03 FC     CASBRS: LD   HL,$FC03
03F7  75           L03F7:  LD   (HL),L           första databyten $03 (ETX): tomt
03F8  DD 75 0A             LD   (IX+$0A),L
03FB  DD 75 08             LD   (IX+$08),L
03FE  DD 74 0B             LD   (IX+$0B),H
0401  DD 74 09             LD   (IX+$09),H
0404  DD 36 0D FC          LD   (IX+$0D),$FC     252 lediga byte
0408  DD 36 07 84          LD   (IX+$07),$84     radbredd 132
040C  EB                   EX   DE,HL            DE = bufferten
040D  C9                   RET
```

CASPUT: *PUT för kassett. Ökar* CASBLK*, skriver blockhuvudet (*$00*, nummer lo, hi) i de tre byten före dataområdet (FCB+8,+9) och skriver hela sidan med* CASWBL*. Sedan är* $FC00 *en tom buffert igen (*CASBRS*). In: IX = FCB. Ut: DE =* $FC03*. Förstör A, BC och HL.*

```
040E  2A FD FD     CASPUT: LD   HL,(CASBLK)
0411  23                   INC  HL
0412  22 FD FD             LD   (CASBLK),HL
0415  EB                   EX   DE,HL            DE = blocknumret
0416  DD 6E 08             LD   L,(IX+$08)
0419  DD 66 09             LD   H,(IX+$09)
041C  2B                   DEC  HL               huvudet före data: 0, lo, hi
041D  72                   LD   (HL),D
041E  2B                   DEC  HL
041F  73                   LD   (HL),E
0420  2B                   DEC  HL
0421  36 00                LD   (HL),$00
0423  CD 39 04             CALL CASWBL           HL = sidan, $FC00
0426  18 CC                JR   CASBRS           ny tom buffert
```

CASCLO: *CLOSE för kassett. Är bit 7 i FCB+E satt (text skriven med* TXTPRT*) skrivs sista blocket och ett slutblock med* TXTEND*. Sedan nollas +E och port* $3A*, och motorn stannar. Ut: A = 0. Med* TXTEND *förstörs också B och DE.*

```
0428  DD 7E 0E     CASCLO: LD   A,(IX+$0E)       bit 7: text skriven
042B  A7                   AND  A
042C  28 04                JR   Z,L0432
042E  87                   ADD  A,A              CY = bit 7
042F  DC 85 09             CALL C,TXTEND
0432  AF           L0432:  XOR  A
0433  DD 77 0E             LD   (IX+$0E),A
0436  D3 3A                OUT  ($3A),A          A = 0: motor av
0438  C9                   RET
```

CASWBL: *skriv sidan HL (L = 0, 256 byte) som ett block: 256 nollbitar, 3 SYN, STX, data, ETX och checksumman. Efter checksumman faller rutinen igenom in i* CASWBY *med C = 0 och skriver en nollbyte till: blocket blir 256 + 8 × 264 = 2368 bitar, ungefär 3,2 s. Ut: HL = samma sida, BC = 0, DE = checksumman. Förstör A.*

```
0439  0E 00        CASWBL: LD   C,$00            256 nollbitar som inledning
043B  CD 76 04     L043B:  CALL CASW0
043E  0D                   DEC  C
```

```
043F  20 FA              JR   NZ,L043B
0441  1E 03              LD   E,$03          3 x SYN
0443  0E 16        L0443: LD   C,$16
0445  CD 6A 04           CALL CASWBY
0448  1D                 DEC  E
0449  20 F8              JR   NZ,L0443
044B  0E 02              LD   C,$02          STX
044D  CD 6A 04           CALL CASWBY
0450  11 03 00           LD   DE,$0003       checksumman börjar på 3
0453  4E            L0453: LD   C,(HL)
0454  EB                 EX   DE,HL
0455  09                 ADD  HL,BC          B = 0 efter CASHLF: bara C adderas
0456  EB                 EX   DE,HL
0457  CD 6A 04           CALL CASWBY
045A  2C                 INC  L              sidan runt
045B  20 F6              JR   NZ,L0453
045D  0E 03              LD   C,$03          ETX
045F  CD 6A 04           CALL CASWBY
0462  4B                 LD   C,E            checksumma lo ...
0463  CD 6A 04           CALL CASWBY
0466  4A                 LD   C,D            ... och hi
0467  CD 6A 04           CALL CASWBY         faller igenom: en $00-byte till
```

CASWBY*: skriv byten C, minst signifikanta biten först. SCF före första* RR C *lägger en markeringsbit i bit 7. När den har skiftats ut är C = 0, och då har exakt 8 bitar skickats. Ut: B = C = 0, CY = 0. Förstör A.*

```
046A  37          CASWBY: SCF                 markeringsbiten
046B  CB 19               RR   C              bit 0 -> CY, CY -> bit 7
046D  CD 77 04    L046D:  CALL CASWBI         skriv biten i CY
0470  CB 19               RR   C              nästa bit; markeringen ett steg ned
0472  20 F9               JR   NZ,L046D       C = 0: åtta bitar
0474  A7                  AND  A              CY var markeringen: nolla
0475  C9                  RET
```

CASW0*: skriv en nollbit.* CASWBI*: skriv biten i CY.*

Skrivsignalen (bit 6 i port $3A*) vänds en gång per bitcell för en nolla. För en etta vänds den två gånger, med halv fördröjning emellan. Med anropet tar en nolla 3993 T och en etta 4065 T, ungefär 1,33 och 1,36 ms. Ut: B = 0 och CY = 0, så att* RR C *i* CASWBY *skiftar in nollor bakom markeringsbiten. Förstör A.*

```
0476  AF          CASW0:  XOR  A              CY = 0: nolla
0477  3E 96       CASWBI: LD   A,$96          nolla: 2 × 150 varv, en vändning
0479  30 09               JR   NC,CASHLF
047B  CB 3F               SRL  A              etta: 2 × 75 varv, två gånger
047D  CD 84 04            CALL CASHLF
0480  3E 96               LD   A,$96          andra halvan
0482  CB 3F               SRL  A
0484  47          CASHLF: LD   B,A            två slingor om A varv
0485  10 FE       L0485:  DJNZ L0485
0487  47                  LD   B,A
0488  10 FE       L0488:  DJNZ L0488
048A  DB 3A               IN   A,($3A)        vänd skrivsignalen
048C  EE 40               XOR  $40
048E  D3 3A               OUT  ($3A),A
0490  C9                  RET
```

3.9 OPEN för kassett: leta efter filen $0491

Läs block tills ett namnblock ($FF $FF $FF) med rätt namn hittas. Datablock hoppas över. Börjar det sökta namnet med blanktecken duger första filen, och dess namn skrivs ut som ”FOUND namn.ext”. Tidsgräns utan signal: A = $AA.

In: DE = det tolkade namnet, IX = FCB. Ut: NC, A = 0 och DE = dataområdet när filen är hittad. FCB:n pekar då på en tom buffert ($03 först), så att första läsningen hämtar block 0, och motorn går. Vid tidsgräns CY och A = $AA, med motorn avstängd. Andra läsfel går till ERROR (fel 37 eller 35), utom i ett datablock, som bara hoppas över. Förstör BC och HL.

```
0491  21 00 00     CASOPN: LD   HL,$0000
0494  22 F9 FD             LD   (CASBUF),HL      CASBUF = 0: mottagningen startas om
0497  D5           L0497:  PUSH DE               DE = namnet
0498  CD 2B 05    L0498:  CALL CASRBL
049B  D1                    POP  DE
049C  30 15                 JR   NC,L04B3         blocket läst utan fel
049E  FE AA                 CP   $AA              tidsgräns?
04A0  4F                    LD   C,A              C = felkoden
04A1  3E 00                 LD   A,$00
04A3  D3 3A                 OUT  ($3A),A          motor av
04A5  79                    LD   A,C
04A6  37                    SCF
04A7  C8                    RET  Z                tidsgräns: tillbaka med CY
04A8  7E                    LD   A,(HL)           läsfel: $00 först = datablock
04A9  A7                    AND  A
04AA  28 EB                 JR   Z,L0497          fel i ett datablock: läs vidare
04AC  AF           L04AC:  XOR  A
04AD  D3 3A                 OUT  ($3A),A
04AF  79                    LD   A,C
04B0  C3 2E 06              JP   ERROR            A = felkoden

04B3  01 A5 03     L04B3:  LD   BC,$03A5         B = 3, C = $A5
04B6  7E                    LD   A,(HL)
04B7  A7                    AND  A
04B8  28 DD                 JR   Z,L0497          datablock: hoppa över
04BA  7E           L04BA:  LD   A,(HL)           namnblock ($FF $FF $FF)?
04BB  23                    INC  HL
04BC  3C                    INC  A
04BD  20 ED                 JR   NZ,L04AC         varken $00 eller $FF: fel 37
04BF  10 F9                 DJNZ L04BA
04C1  1A                    LD   A,(DE)           tomt namn: ta första filen
04C2  FE 20                 CP   $20
04C4  28 19                 JR   Z,L04DF
04C6  D5                    PUSH DE
04C7  06 0B                 LD   B,$0B            jämför 11 tecken
04C9  1A           L04C9:  LD   A,(DE)
04CA  AE                    XOR  (HL)
04CB  20 CB                 JR   NZ,L0498         fel namn: nästa block
04CD  23                    INC  HL
04CE  13                    INC  DE
04CF  10 F8                 DJNZ L04C9
04D1  D1                    POP  DE
04D2  AF           L04D2:  XOR  A                A = 0, NC
04D3  11 FF FF              LD   DE,$FFFF
04D6  ED 53 FD FD           LD   (CASBLK),DE      nästa block ska vara nr 0
04DA  2E 03                 LD   L,$03            L = 3: $03 först, tom buffert
04DC  C3 F7 03              JP   L03F7            FCB:n som vid skrivning, DE = dataområdet
```

Tomt namn: skriv CR, LF, FOUND *och namnet ur namnblocket. Punkten stoppas in genom att typens tre tecken flyttas ett steg framåt i bufferten, och 12 tecken skrivs med* PRTXT*. Sedan som när namnet stämmer.*

```
04DF  E5           L04DF:  PUSH HL
04E0  21 01 05             LD   HL,D0501        HL = texten, 8 tecken
04E3  01 08 00             LD   BC,$0008
04E6  CD A8 01             CALL PRTXT
04E9  E1                   POP  HL
04EA  E5                   PUSH HL
04EB  01 2E 03             LD   BC,$032E        C = ".", B = 3
04EE  2E 0B                LD   L,$0B           från +$0B: typen
04F0  7E           L04F0:  LD   A,(HL)          typdelen ett steg, . in
04F1  71                   LD   (HL),C
04F2  4F                   LD   C,A
04F3  23                   INC  HL
04F4  10 FA                DJNZ L04F0
04F6  71                   LD   (HL),C
04F7  2E 03                LD   L,$03
04F9  0E 0C                LD   C,$0C           12 tecken: namn.typ
04FB  CD A8 01             CALL PRTXT
04FE  E1                   POP  HL
04FF  18 D1                JR   L04D2

0501  0D           D0501:  DEFB $0D             CR och LF före FOUND
0502  0A 46 4F 55…          DEFB $0A,"FOUND "
```

CASGET*: GET för kassett. Hämtar nästa block med* CASRBL*. Det måste börja med* $00 *och ha blocknumret* CASBLK *+ 1, annars fel 37 (*CASBLK *räknas upp ändå). Läsfel går också till* ERROR *med motorn avstängd; tidsgränsen blir fel 42. Ut: HL = blockets data (sidan + 3), även i FCB+8,+9; NC. Förstör A, BC och DE.*

```
0509  CD 2B 05     CASGET: CALL CASRBL
050C  4F                   LD   C,A             C = felkoden
050D  38 9D                JR   C,L04AC         fel: motorn av, ERROR
050F  ED 5B FD FD          LD   DE,(CASBLK)     väntat blocknummer
0513  13                   INC  DE
0514  ED 53 FD FD          LD   (CASBLK),DE
0518  97                   SUB  A               A = 0: blocktypen
0519  01 A5 03             LD   BC,$03A5        B = 3, C = $A5 (fel 37)
051C  AE           L051C:  XOR  (HL)            0, nr lo, nr hi
051D  20 8D                JR   NZ,L04AC
051F  23                   INC  HL
0520  7B                   LD   A,E             A = väntad nästa byte
0521  5A                   LD   E,D
0522  10 F8                DJNZ L051C
0524  DD 75 08             LD   (IX+$08),L      data efter huvudet
0527  DD 74 09             LD   (IX+$09),H
052A  C9                   RET
```

3.10 CASRBL: hämta ett block från den avbrottsstyrda läsaren $052B

Mottagningen sker under avbrott (CASINT/RCVBLK) medan programmet arbetar. CASST ($FDF8) är $FF medan ett block tas emot, och därefter resultatet:

```
0 = ok, $A5 = formatfel (ETX saknas), $A3 = checksummafel.
```

När ett block är klart startas mottagningen av nästa block i den andra bufferten. Ut: HL = bufferten, CY = 1 och A = felkod vid fel ($AA = tidsgräns utan signal).

Tidsgränsen: DE räknar 65 536 varv om 66 + 24 × 13 + 8 = 386 T, 25,3 miljoner T eller ungefär 8,5 s vid 2,9952 MHz (NMI-klockan tar lite till). Version 9913 har $3C i stället för $19 i den inre slingan: 841 T per varv, ungefär 18,4 s. I CASOPN blir tidsgränsen fel 21 (HITTAR EJ FILEN) via OPEN; kommer det block med andra namn fortsätter sökningen. Christensen anger 10 s väntan på kassettfilen, vilket bör vara version 11273:s 8,5 s (Christensen 1979).

Förstör B och DE.

```
052B  2A F9 FD     CASRBL: LD   HL,(CASBUF)
052E  7C                   LD   A,H              bara H prövas (OR H)
052F  B4                   OR   H
0530  CC 56 05             CALL Z,RCVSTA         ingen mottagning igång: starta
0533  11 00 00             LD   DE,$0000
0536  ED 53 F9 FD          LD   (CASBUF),DE      CASBUF = 0 medan vi väntar
053A  1B           L053A:  DEC  DE               DE = 0: 65536 varv
053B  06 19                LD   B,$19            tidsgräns
053D  10 FE        L053D:  DJNZ L053D
053F  7A                   LD   A,D
0540  B3                   OR   E
0541  3E AA                LD   A,$AA
0543  37                   SCF
0544  C8                   RET  Z                tidsgräns: A = $AA, CY = 1
0545  3A F8 FD             LD   A,(CASST)        blocket klart?
0548  3C                   INC  A
0549  28 EF                JR   Z,L053A          $FF: mottagning pågår
054B  3D                   DEC  A
054C  E5                   PUSH HL
054D  F5                   PUSH AF
054E  CC 56 05             CALL Z,RCVSTA         starta nästa block
0551  F1                   POP  AF
0552  E1                   POP  HL
0553  C0                   RET  NZ               fel: A = koden, CY = 1 (SCF)
0554  AF                   XOR  A
0555  C9                   RET                   klart: A = 0, CY = 0, HL = blocket
```

RCVSTA: starta mottagningen av ett block. CASST = $FF (pågår), och PIO B får vektor $36 och avbrott när bit 7 blir låg. RCVBC = $3200: de första 50 bitarna hoppas över, sedan börjar korutinen i RCVBLK. Buffertsidan växlar: H = 0 ger $FC00, annars $FC och $FB om vartannat. Första byten blir $AA, som visar att inget har kommit än. Motorn startas med bit 6 först 0 och sedan 1. In: HL = förra bufferten. Ut: HL = den nya, även i CASBUF och RCVPTR. Förstör A och DE.

```
0556  F3           RCVSTA: DI
0557  3E FF                LD   A,$FF
0559  32 F8 FD             LD   (CASST),A        CASST = $FF: mottagning pågår
055C  3E 36                LD   A,$36            PIO B: vektor $36
055E  D3 3B                OUT  ($3B),A
0560  3E 97                LD   A,$97            på, aktivt lågt, mask följer
0562  D3 3B                OUT  ($3B),A
0564  3E 7F                LD   A,$7F            mask: bara bit 7 bevakas
0566  D3 3B                OUT  ($3B),A
0568  EB                   EX   DE,HL            HL sparas i DE
```

```
0569  21 00 32          LD   HL,$3200        hoppa över 50 bitar innan synkningen börjar
056C  22 01 FE          LD   (RCVBC),HL
056F  21 D0 05          LD   HL,$05D0        korutinen börjar i RCVBLK
0572  22 03 FE          LD   (RCVCONT),HL
0575  EB                EX   DE,HL           HL tillbaka, DE = $05D0
0576  7C                LD   A,H             förra bufferten; H = 0: ingen
0577  A7                AND  A
0578  20 05             JR   NZ,L057F
057A  21 00 FC          LD   HL,$FC00
057D  18 03             JR   L0582

057F  EE 07       L057F: XOR  $07            $FC <-> $FB
0581  67                LD   H,A
0582  22 F9 FD    L0582: LD   (CASBUF),HL
0585  22 FF FD          LD   (RCVPTR),HL
0588  36 AA             LD   (HL),$AA        markering: ingen data än
058A  3E 20             LD   A,$20           motor på med bit 6 = 0 ...
058C  D3 3A             OUT  ($3A),A
058E  3E 60             LD   A,$60           ... och sedan bit 6 = 1
0590  D3 3A             OUT  ($3A),A
0592  FB                EI
0593  C9                RET
```

3.11 CASINT: avbrott från kassetten, ett per bit $0594

Avbrottet kommer på en flank i början av bitcellen. Rutinen pulsar bit 6 i port $3A (nollställer flanklatchen i kassettgränssnittet), väntar ungefär 3/4 bitcell och läser bit 7.

CY = inverterad bit 7 = den mottagna biten. Avbrottet är aktivt lågt ($97 i RCVSTA), så en flank drar bit 7 låg; har en flank kommit mitt i cellen blir CY = 1, en etta som CASWBI skriver den. Biten skiftas in i RCVBC (C). När B bitar är mottagna hoppar rutinen till adressen i RCVCONT med A = byten och DE = RCVPTR. RCVSUM summerar mottagna byte.

Mottagarens huvudprogram (RCVBLK) fungerar alltså som en korutin: RCVBYT/RCVBIT sparar returadressen i RCVCONT och avslutar avbrottet. Nästa gång B bitar är klara fortsätter programmet efter anropet.

```
0594  FB          CASINT: EI                 EI genast: PIO A får avbryta
0595  F5                  PUSH AF
0596  C5                  PUSH BC
0597  DB 3A               IN   A,($3A)       bit 6 = 0 ...
0599  E6 BF               AND  $BF
059B  D3 3A               OUT  ($3A),A
059D  4F                  LD   C,A           C = porten med bit 6 = 0
059E  3E 78               LD   A,$78         120 varv, ca 1 560 T
05A0  47                  LD   B,A
05A1  10 FE       L05A1:  DJNZ L05A1
05A3  47                  LD   B,A           B = 120 igen
05A4  3E 40               LD   A,$40
05A6  B1                  OR   C
05A7  D3 3A               OUT  ($3A),A       ... bit 6 = 1, ca 1 680 T efter avbrottet
05A9  10 FE       L05A9:  DJNZ L05A9         ca 1 560 T till
05AB  47                  LD   B,A           B = porten med bit 6 = 1
05AC  DB 3A               IN   A,($3A)       bit 7, ca 3 250 T efter avbrottet
05AE  87                  ADD  A,A           CY = inverterad bit 7
05AF  3F                  CCF
05B0  79                  LD   A,C
05B1  D3 3A               OUT  ($3A),A       bit 6 = 0 ...
05B3  78                  LD   A,B
```

```
05B4  D3 3A              OUT   ($3A),A       ... och 1: nollställd för nästa flank
05B6  ED 4B 01 FE        LD    BC,(RCVBC)    B = bitar kvar, C = byten hittills
05BA  CB 19              RR    C             skifta in biten
05BC  10 68              DJNZ  L0626         fler bitar i byten
05BE  79                 LD    A,C           A = den färdiga byten
05BF  E5                 PUSH  HL
05C0  D5                 PUSH  DE
05C1  2A 05 FE           LD    HL,(RCVSUM)   summera byten
05C4  09                 ADD   HL,BC         B = 0 efter DJNZ: bara C
05C5  22 05 FE           LD    (RCVSUM),HL
05C8  2A 03 FE           LD    HL,(RCVCONT)
05CB  ED 5B FF FD        LD    DE,(RCVPTR)
05CF  E9                 JP    (HL)          fortsätt korutinen
```

RCVBLK: mottagarens huvudprogram, en korutin som körs i CASINT. Söker SYN ($16) bit för bit, väntar ut resten av SYN och kräver STX ($02), tar emot 256 byte till sidan vid RCVPTR och kontrollerar ETX och checksumman (summan av data och ETX, som CASWBL räknar den). Resultatet läggs i CASST: 0, $A5 eller $A3. Sedan stängs avbrottet från PIO B av och bit 6 nollas.

```
05D0  06 01        RCVBLK: LD    B,$01         sök SYN bit för bit
05D2  CD 1C 06             CALL  RCVBIT        B = 1: en bit i taget
05D5  FE 16                CP    $16
05D7  20 F7                JR    NZ,RCVBLK
05D9  CD 1A 06     L05D9:  CALL  RCVBYT        vänta bort resten av SYN
05DC  FE 16                CP    $16
05DE  28 F9                JR    Z,L05D9
05E0  FE 02                CP    $02           STX?
05E2  20 EC                JR    NZ,RCVBLK
05E4  21 00 00             LD    HL,$0000
05E7  22 05 FE             LD    (RCVSUM),HL   summan från första databyten
05EA  CD 1A 06     L05EA:  CALL  RCVBYT        256 databyte
05ED  12                   LD    (DE),A        DE = RCVPTR
05EE  1C                   INC   E
05EF  20 F9                JR    NZ,L05EA
05F1  CD 1A 06             CALL  RCVBYT
05F4  FE 03                CP    $03           ETX?
05F6  3E A5                LD    A,$A5         fel 37 (formatfel)
05F8  20 12                JR    NZ,L060C
05FA  ED 5B 05 FE          LD    DE,(RCVSUM)   summan: data + ETX ($03)
05FE  CD 1A 06             CALL  RCVBYT
0601  AB                   XOR   E             jämför checksumman
0602  5F                   LD    E,A
0603  CD 1A 06             CALL  RCVBYT
0606  AA                   XOR   D
0607  B3                   OR    E
0608  28 02                JR    Z,L060C
060A  3E A3                LD    A,$A3         fel 35 (checksummafel)
060C  32 F8 FD     L060C:  LD    (CASST),A
060F  3E 07                LD    A,$07         PIO B: avbrott av
0611  D3 3B                OUT   ($3B),A
0613  DB 3A                IN    A,($3A)
0615  E6 BF                AND   $BF
0617  D3 3A                OUT   ($3A),A       bit 6 = 0
0619  C5                   PUSH  BC            skenadress; avbrotten är av
```

RCVBYT: vänta på nästa byte ($B = 8$), RCVBIT: på B bitar. Returadressen sparas i RCVCONT och avbrottet avslutas. När B bitar är mottagna fortsätter CASINT där, med A = byten, DE = RCVPTR och B = 0.

```
061A  06 08        RCVBYT: LD    B,$08         en hel byte
061C  E1           RCVBIT: POP   HL            returadressen i RCVBLK
061D  22 03 FE             LD    (RCVCONT),HL  ... blir fortsättningen
```

```
0620  ED 53 FF FD          LD   (RCVPTR),DE     pekaren i bufferten
0624  D1                   POP  DE              sparade av CASINT ($05BF/$05C0)
0625  E1                   POP  HL
0626  ED 43 01 FE  L0626:  LD   (RCVBC),BC      B = bitar kvar, C = byten
062A  C1                   POP  BC
062B  F1                   POP  AF
062C  ED 4D                RETI                 avbrottet slut
```

Korsreferenstabell för kassetten. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken, i systemvariablerna och i FCB:n som IX pekar på. FCB-kolumnerna: +7 radbredden, +8 dataområdets början (+8,+9), +$0A positionen i blocket (+$0A,+$0B), +$0D de lediga byten och +$0E bit 7 = text skriven. Rutinerna nås genom hopptabellen CASTAB, som DVCALL hoppar in i; CASBRS gör bufferten tom och är också ingången L03F7 med HL = dataområdet. Det som en anropad rutin gör räknas med: CASPRE och CASPUT skriver blocket med CASWBL, CASOPN och CASGET läser med CASRBL, och CASCLO avslutar en textfil genom TXTEND (som skriver med CASPUT). CASOPN skriver FOUND och namnet genom PRTXT, och vid läsfel stänger CASOPN och CASGET av motorn och går till ERROR. CASINT är avbrottet från kassetten; den bevarar alla register och tar emot blocket bit för bit i RCVBLK, RCVBYT och RCVBIT, som fortsätter vid nästa avbrott där de slutade (RCVCONT). CASHLF är fördröjningen för en halv bitcell. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler							FCB				
Rutin	A	CY	BC	DE	HL	IX	stack	CASST	CASBUF	CASBLK	RCVPTR	RCVBC	RCVCONT	RCVSUM	+7	+8	+$0A	+$0D	+$0E
CASPRE	X	X	X	IU	X	I			U	U					U	U	U	U	U
CASBRS				IU	U	I									U	U	U	U	
CASPUT	X	X	X	U	X	I				IU					U	IU	U	U	
CASCLO	U	X	X	X		I				IU					U	IU	IU	U	IU
CASWBL	X	X	U	U	I														
CASWBY	X	X	IU																
CASW0	X	U	U																
CASWBI	X	IU	U																
CASOPN	U	U	X	IU	X	I		I	IU	U	U	U	U		U	U	U	U	
CASGET	X	U	X	X	U	I		I	IU	IU	U	U	U			U			
CASRBL	U	U	X	X	U			I	IU		U	U	U						
RCVSTA	X	X		X	IU			U	U		U	U	U						
CASINT								U			IU	IU	IU	IU					

Rutin	**Anropar**	**Anropas från**
CASPRE	CASWBL; går vidare till CASBRS	CASTAB (PREPARE, genom DVPREP)
CASBRS		CASPRE, CASPUT, CASOPN (vid L03F7)
CASPUT	CASWBL; går vidare till CASBRS	CASTAB (PUT, genom DVPUT)
CASCLO	TXTEND (när text har skrivits)	CASTAB (CLOSE, genom CLOSE)
CASWBL	CASW0, CASWBY	CASPRE, CASPUT
CASWBY	CASWBI	CASWBL
CASW0, CASWBI	CASHLF	CASWBL (CASW0); CASWBY (CASWBI)
CASOPN	CASRBL, PRTXT, CASBRS (vid L03F7); fel till ERROR	CASTAB (OPEN, genom DVOPEN)
CASGET	CASRBL; fel till ERROR	CASTAB (GET, genom DVGET)
CASRBL	RCVSTA	CASOPN, CASGET
RCVSTA		CASRBL
CASINT	RCVBLK, RCVBYT, RCVBIT (där de slutade förra gången)	avbrottet från kassetten (vektor $36)

3.12 Felhantering $062E

ERROR: A = felkod ($80 + nummer). Om ett program körs, felet finns i listan ERRTRP och ON ERROR GOTO är aktivt ($FE35 skilt från 0), sparas koden i $FE38. Stacken återställs och programmet fortsätter på ON ERROR-raden.

Annars ERRSTP: stacken återställs, radnumret för den rad som kördes letas fram och ”ERR n LINE m” skrivs ut. Sedan går man tillbaka till kommandoloopen. Med A = 0 skrivs ”STOP” i stället.

Om bit 0 i EXTFLG är satt får tilläggs-ROM:en ($6051) skriva ut felet (till exempel som text).

```
062E  5F           ERROR:  LD   E,A              E = felkoden
062F  FD 7E 0E             LD   A,(IY+$0E)       direktläge: inget ON ERROR
0632  A7                    AND  A
0633  7B                    LD   A,E
0634  20 33                 JR   NZ,ERRSTP        direktläge: stoppa
0636  21 59 06              LD   HL,ERRTRP        kan felet fångas? (egenhet)
0639  01 10 00              LD   BC,$0010         16 koder
063C  ED B1                CPIR
063E  20 29                JR   NZ,ERRSTP        inte i ERRTRP: stoppa
0640  FD 56 1F             LD   D,(IY+$1F)       D = ONERR:s höga byte, 0 = av
0643  7A                    LD   A,D
0644  B7                    OR   A
0645  7B                    LD   A,E
0646  28 21                JR   Z,ERRSTP
0648  FD 73 22             LD   (IY+$22),E       felkoden sparas för ERRCODE
064B  FD 5E 1E             LD   E,(IY+$1E)       DE = ON ERROR-radens adress
064E  FD 36 1F 00          LD   (IY+$1F),$00     ON ERROR gäller en gång
0652  2A 36 FE             LD   HL,(STMSP)
0655  F9                   LD   SP,HL            SP som vid satsens början
0656  C3 71 0D             JP   RUNLIN           fortsätt på ON ERROR-raden
```

Felkoder som kan fångas med ON ERROR GOTO.

```
0659  87 84 8C 9E… ERRTRP: DEFB $87,$84,$8C,$9E,$A2,$93,$95,$A3
                                                 fel 7, 4, 12, 30, 34, 19, 21 och 35
0661  A4 A5 A6 A7…         DEFB $A4,$A5,$A6,$A7,$A8,$A9,$AA,$AB
                                                 fel 36-43; 36, 39-41 och 43 ger ROM:en aldrig
```

ERRSTP: *felet stoppar programmet. In: A = felkoden, 0 för STOP. Stacken återställs, och om ett program körde letas raden fram vars adress står i* CURLIN, *så att meddelandet får radnumret. Sedan* READY.

```
0669  2A 27 FE     ERRSTP: LD   HL,(STKTOP)      stacken som i kommandoloopen
066C  F9                   LD   SP,HL
066D  F5                   PUSH AF               felkoden på stacken
066E  AF                   XOR  A                HL = 0: inget radnummer
066F  6F                   LD   L,A
0670  67                   LD   H,A
0671  FD B6 0E             OR   (IY+$0E)         HL = 0 i direktläge: inget radnr
0674  20 1E                JR   NZ,L0694
0676  2A 1C FE             LD   HL,(BOFA)        leta rätt på raden som kördes
0679  44           L0679:  LD   B,H              BC = radens början
067A  4D                   LD   C,L
067B  5E                   LD   E,(HL)           E = radens längd
067C  16 00                LD   D,$00
067E  19                   ADD  HL,DE            HL = nästa rad
067F  7B                   LD   A,E
0680  EB                   EX   DE,HL
0681  6C                   LD   L,H              HL = 0 (H var 0)
0682  FE 01                CP   $01              längd 1: programslut, inget radnr
```

```
0684  28 0E              JR   Z,L0694
0686  2A 2C FE           LD   HL,(CURLIN)      adressen till raden som körde
0689  ED 52              SBC  HL,DE            CURLIN >= nästa rad: vidare
068B  EB                 EX   DE,HL
068C  30 EB              JR   NC,L0679
068E  03                 INC  BC               radnumret efter längdbyten
068F  0A                 LD   A,(BC)
0690  6F                 LD   L,A
0691  03                 INC  BC
0692  0A                 LD   A,(BC)
0693  67                 LD   H,A
0694  F1           L0694: POP  AF               A = felkoden
0695  CD 9B 06           CALL ERRMSG
0698  C3 CC 00           JP   READY
```

ERRMSG: *skriv felmeddelandet. In: A = felkoden (0 = STOP), HL = radnumret (0 = inget). Meddelandet byggs i 80 byte på stacken: CR LF BEL,* STOP *eller* ERR n *och* LINE m *om HL inte är 0. Om bit 0 i* EXTFLG *är satt får tilläggs-ROM:en* ($6051) *ta hand om felets text; NC tillbaka betyder att den gjorde det. Förstör A, BC, DE och HL.*

```
069B  EB           ERRMSG: EX   DE,HL           DE = radnumret
069C  21 B0 FF             LD   HL,$FFB0        80 byte buffert på stacken
069F  39                   ADD  HL,SP
06A0  F9                   LD   SP,HL
06A1  E5                   PUSH HL
06A2  36 0D                LD   (HL),$0D        CR LF BEL först
06A4  23                   INC  HL
06A5  36 0A                LD   (HL),$0A
06A7  23                   INC  HL
06A8  36 07                LD   (HL),$07
06AA  23                   INC  HL
06AB  EB                   EX   DE,HL           DE = skrivpekaren, HL = radnumret
06AC  E5                   PUSH HL              radnumret
06AD  A7                   AND  A
06AE  20 0A                JR   NZ,L06BA        A = 0: STOP
06B0  21 3A 07             LD   HL,D073A        STOP
06B3  01 04 00             LD   BC,$0004
06B6  ED B0                LDIR
06B8  18 1A                JR   L06D4

06BA  21 0F FE     L06BA:  LD   HL,$FE0F        tilläggs-ROM skriver felet?
06BD  CB 46                BIT  0,(HL)
06BF  37                   SCF
06C0  C4 51 60             CALL NZ,V6051        NC tillbaka: tillägget skrev felet
06C3  30 0F                JR   NC,L06D4
06C5  21 3E 07             LD   HL,D073E        ERR
06C8  01 04 00             LD   BC,$0004
06CB  ED B0                LDIR
06CD  E6 7F                AND  $7F             felnummer
06CF  6F                   LD   L,A
06D0  60                   LD   H,B             B = 0 efter LDIR: HL = felnumret
06D1  CD 62 18             CALL PRNUM           felnumret vid DE
06D4  E1           L06D4:  POP  HL
06D5  7C                   LD   A,H
06D6  B5                   OR   L
06D7  28 0D                JR   Z,L06E6         HL = 0: ingen radtext
06D9  E5                   PUSH HL
06DA  21 34 07             LD   HL,D0734         LINE
06DD  01 06 00             LD   BC,$0006
06E0  ED B0                LDIR
06E2  E1                   POP  HL
```

```
06E3  CD 62 18           CALL PRNUM          radnumret
06E6  E1          L06E6: POP  HL             HL = buffertens början
06E7  EB                 EX   DE,HL
06E8  B7                 OR   A
06E9  ED 52              SBC  HL,DE          BC = textens längd
06EB  4D                 LD   C,L
06EC  44                 LD   B,H
06ED  EB                 EX   DE,HL
06EE  CD A8 01           CALL PRTXT          skriv meddelandet
06F1  21 50 00           LD   HL,$0050       bufferten bort
06F4  39                 ADD  HL,SP
06F5  F9                 LD   SP,HL
06F6  C9                 RET
```

Felingångar: en `RST $10` *för varje felkod, så att man kan hoppa hit med* `JP`.

```
06F7  D7          L06F7: RST  $10
06F8  B5                 DEFB $B5           fel 53: FELAKTIG RAD

06F9  D7          L06F9: RST  $10
06FA  80                 DEFB $80           fel 0: EJ TILLÅTET ÖKA "DIM"

06FB  D7          L06FB: RST  $10
06FC  81                 DEFB $81           fel 1: FEL ANTAL INDEX

06FD  D7          L06FD: RST  $10
06FE  82                 DEFB $82           fel 2: OTILLÅTET SOM KOMMANDO

06FF  D7          L06FF: RST  $10
0700  83                 DEFB $83           fel 3: MINNET FULLT

0701  D7          L0701: RST  $10
0702  84                 DEFB $84           fel 4: FÖR STORT FLYTTAL

0703  D7          L0703: RST  $10
0704  85                 DEFB $85           fel 5: FÖR STORT INDEX

0705  D7          L0705: RST  $10
0706  86                 DEFB $86           fel 6: HITTAR EJ DETTA RADNUMMER

0707  D7          L0707: RST  $10
0708  87                 DEFB $87           fel 7: FÖR STORT HELTAL

0709  D7          L0709: RST  $10
070A  88                 DEFB $88           fel 8: FINNS EJ I DETTA SYSTEM

070B  D7          L070B: RST  $10
070C  89                 DEFB $89           fel 9: INDEX UTANFÖR STRÄNGEN

070D  D7          L070D: RST  $10
070E  8A                 DEFB $8A           fel 10: TEXTEN FÅR EJ PLATS I STRÄNGEN

070F  D7          L070F: RST  $10
0710  95                 DEFB $95           fel 21: HITTAR EJ FILEN

0711  D7          L0711: RST  $10
0712  96                 DEFB $96           fel 22: OTILLÅTEN SATS

0713  D7          L0713: RST  $10
0714  8C                 DEFB $8C           fel 12: FELAKTIGT TAL

0715  D7          L0715: RST  $10
0716  98                 DEFB $98           fel 24: "NEXT" SAKNAS

0717  D7          L0717: RST  $10
0718  9C                 DEFB $9C           fel 28: FEL VARIABEL EFTER "NEXT"

0719  D7          L0719: RST  $10
```

```
071A  9B                DEFB $9B            fel 27: "NEXT" UTAN "FOR"

071B  D7          L071B: RST  $10
071C  9D                DEFB $9D            fel 29: "RETURN" UTAN "GOSUB"

071D  D7          L071D: RST  $10
071E  9A                DEFB $9A            fel 26: FEL I ON-UTTRYCK

071F  D7          L071F: RST  $10
0720  9E                DEFB $9E            fel 30: DATA SLUT

0721  D7          L0721: RST  $10
0722  9F                DEFB $9F            fel 31: FEL DATA TILL KOMMANDO

0723  D7          L0723: RST  $10
0724  A0                DEFB $A0            fel 32: FILEN EJ ÖPPNAD

0725  D7          L0725: RST  $10
0726  B2                DEFB $B2            fel 50: KVADRATROT UR NEGATIVT TAL

0727  D7          L0727: RST  $10
0728  B4                DEFB $B4            fel 52: EJ TILL DENNA ENHET

0729  D7          L0729: RST  $10
072A  B9                DEFB $B9            fel 57: FUNKTIONEN EJ DEFINIERAD

072B  D7          L072B: RST  $10
072C  BC                DEFB $BC            fel 60: BIT ADRESS 16 BITAR

072D  0D          D072D: DEFB $0D           CR, början på ABC80 och på NEWLIN:s CR LF
072E  0A 41 42 43…      DEFB $0A,"ABC80"
0734  20 4C 49 4E… D0734: DEFB " LINE "
073A  53 54 4F 50  D073A: DEFB "STOP"
073E  45 52 52 20  D073E: DEFB "ERR "
```

NEWLIN: *skriv CR LF. DE och HL bevaras; A och BC förstörs.*

```
0742  D5          NEWLIN: PUSH DE
0743  E5                  PUSH HL
0744  21 2D 07            LD   HL,D072D
0747  01 02 00            LD   BC,$0002     CR LF, början av D072D
074A  CD A8 01            CALL PRTXT
074D  E1                  POP  HL
074E  D1                  POP  DE
074F  C9                  RET
```

Korsreferenstabell för felen. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är direktläget vid IY+$0E och felkoden för ERRCODE vid IY+$22. ONERR är adressen till ON ERROR-raden, med höga byten 0 när ON ERROR är av. Felingångarna $06F7–$072B är en RST $10 med felkoden efter, och har inga egna rader. Det som en anropad rutin gör räknas med: ERRMSG skriver genom PRTXT (markören som i tabellen efter $03A2) och bygger texten med PRNUM i 80 byte under SP, som den tar bort igen; finns ett tillägg som skriver felen (bit 0 i EXTFLG), anropas det på $6051. * = kommer inte tillbaka; för dem visas bara stacken och variablerna. ERROR går vidare till ERRSTP när felet inte fångas, och tabellen visar bara det ERROR själv gör. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler								IY+	
Rutin	A	CY	BC	DE	HL	IX	stack	CURROW	CURCOL	STKTOP	BOFA	CURLIN	ONERR	STMSP	EXTFLG	$0E	$22
RST10*							I										
ERROR*	I						U						IU	I		I	U

	register och stack							variabler								IY+	
Rutin	A	CY	BC	DE	HL	IX	stack	CURROW	CURCOL	STKTOP	BOFA	CURLIN	ONERR	STMSP	EXTFLG	$0E	$22
ERRSTP*	I						U	IU	IU	I	I	I			I	I	
ERRMSG	IX	X	X	X	IX			IU	IU						I		
NEWLIN	X	X	X					IU	IU								

Rutin	**Anropar**	**Anropas från**
RST10	ERROR med koden efter RST $10	felingångarna $06F7–$072B och RST $10 överallt
ERROR	RUNLIN (fångat); annars ERRSTP	RST10, CMDLP, CASOPN, INPUT, LNKFN
ERRSTP	ERRMSG; går vidare till READY	ERROR, CMDLP, CMDMERG
ERRMSG	PRNUM, PRTXT; tillägget på $6051	ERRSTP, CMDMERG
NEWLIN	PRTXT	CMDLP, CMDED, CMDLIST, CMDRUN, INPT, TRCPRT

3.13 INSLIN: lägg in, ersätt eller radera en programrad $0750

IX pekar på den tokeniserade raden (+0 längd, +1,+2 radnummer).

Raden ska in där FINDLN hittar numret eller första högre nummer. Resten av programmet, till och med $01:an på EOFA, flyttas med skillnaden mellan ny och gammal längd (gammal = 0 för en ny rad): LDDR uppåt om raden växer, LDIR nedåt om den krymper; EOFA följer med. Längd 0 (bara radnumret) raderar raden. Före en flyttning uppåt görs CLEAR, så att HEAP = EOFA + 1, och MEMRES kontrollerar minnet. Räcker det inte blir det via CMPERR tillbaka med CY = 1 och A = $83 (fel 3, MINNET FULLT); programmet är då orört. Därefter kopieras raden in, och CLEAR görs eftersom variablerna ligger efter programmet.

Ut: CY = 0; Z = 1 om raden blev sist (MERGE fortsätter på NZ). Räcker minnet inte: CY = 1 och A = $83. IX bevaras; A, BC, DE och HL förstörs, och CLEAR stänger filerna i FILLST.

```
0750  DD 5E 01     INSLIN: LD   E,(IX+$01)
0753  DD 56 02             LD   D,(IX+$02)
0756  CD 39 0F             CALL FINDLN          finns radnumret?
0759  01 00 00             LD   BC,$0000        BC = 0: numret är nytt, gammal längd 0
075C  20 01                JR   NZ,L075F        NZ: raden finns inte
075E  4E                   LD   C,(HL)          C = gamla radens längd
075F  DD 7E 00     L075F:  LD   A,(IX+$00)
0762  91                   SUB  C               A = ny - gammal längd
0763  28 30                JR   Z,L0795         lika lång: skriv över
0765  38 42                JR   C,L07A9         kortare: flytta ned
0767  E5                   PUSH HL              HL = radens plats
0768  47                   LD   B,A             B = hur mycket raden växer
0769  C5                   PUSH BC
076A  CD 80 0A             CALL CLEAR           HEAP = EOFA + 1
076D  16 00                LD   D,$00
076F  C1                   POP  BC
0770  C5                   PUSH BC
0771  58                   LD   E,B             DE = lika många byte
0772  CD 5A 33             CALL MEMRES          plats i minnet?
0775  C1                   POP  BC
0776  3E 83                LD   A,$83           fel 3, MINNET FULLT
0778  DA F7 1B             JP   C,CMPERR
077B  78                   LD   A,B             A = tillväxten
```

```
077C  06 00              LD   B,$00
077E  EB                 EX   DE,HL          DE = EOFA+1 från MEMRES
077F  E1                 POP  HL             HL = radens plats
0780  E5                 PUSH HL
0781  09                 ADD  HL,BC          HL = efter gamla raden
0782  D5                 PUSH DE             EOFA+1 undan
0783  EB                 EX   DE,HL
0784  ED 52              SBC  HL,DE          BC = raderna efter, till EOFA
0786  4D                 LD   C,L
0787  44                 LD   B,H
0788  D1                 POP  DE             DE = EOFA+1
0789  6F                 LD   L,A
078A  26 00              LD   H,$00
078C  1B                 DEC  DE             DE = gamla EOFA
078D  19                 ADD  HL,DE
078E  22 1E FE           LD   (EOFA),HL      nytt slut = gammalt + skillnaden
0791  EB                 EX   DE,HL          HL = gamla EOFA, DE = nya
0792  ED B8              LDDR                flytta upp, bakifrån
0794  E1           L0794: POP  HL             HL = radens plats
0795  EB           L0795: EX   DE,HL          DE = radens plats
0796  DD E5              PUSH IX
0798  E1                 POP  HL             HL = nya raden (IX)
0799  4E                 LD   C,(HL)         BC = nya längden (B = 0)
079A  79                 LD   A,C
079B  A7                 AND  A
079C  28 02              JR   Z,L07A0        längd 0: raden raderas
079E  ED B0              LDIR                kopiera in raden
07A0  1A           L07A0: LD   A,(DE)
07A1  FE 01              CP   $01            Z: raden är sist
07A3  F5                 PUSH AF
07A4  CD 80 0A           CALL CLEAR          variablerna efter programmet är borta
07A7  F1                 POP  AF
07A8  C9                 RET

07A9  E5           L07A9: PUSH HL
07AA  09                 ADD  HL,BC          HL = efter gamla raden
07AB  EB                 EX   DE,HL
07AC  2A 1E FE           LD   HL,(EOFA)      HL = EOFA
07AF  ED 52              SBC  HL,DE
07B1  44                 LD   B,H
07B2  4D                 LD   C,L
07B3  03                 INC  BC             BC = resten, med $01:an
07B4  26 FF              LD   H,$FF          HL = skillnaden (negativ)
07B6  6F                 LD   L,A
07B7  19                 ADD  HL,DE
07B8  EB                 EX   DE,HL          DE = dit, HL = efter gamla raden
07B9  ED B0              LDIR                flytta ned, framifrån
07BB  1B                 DEC  DE             DE på den flyttade $01:an
07BC  ED 53 1E FE        LD   (EOFA),DE      EOFA = den flyttade $01:an
07C0  18 D2              JR   L0794
```

3.14 OPEN: öppna en fil $07C2

OPNSTR: filnamnet står i programtexten på HL, inom " eller ', eller fram till CR. Ut till OPEN: HL = namnet, C = längd.

OPEN: A = typ av öppning:

```
0  OPEN
1  PREPARE
2  för LOAD: prova namn.BAC, och finns den inte namn.BAS
3  PREPARE med .BAS som standard (text, som LIST till fil)
4+ PREPARE med .BAC som standard (SAVE)
```

Namnet tolkas (via $1478) till 8+3 tecken och ett enhetsnamn, som letas upp i enhetslistan (fel 21 om enheten saknas). Det tolkade namnet ligger i 14 byte på stacken, och enhetens rutin får det i DE. FCB-fältet +3,+4 pekar sedan på posten i listan (på hopptabellspekaren), +9 = 0, och efter öppningen +5 = 1 (öppen), +6 = 0, +E = 0. Svarar enheten CY blir det fel 21, vad den än lagt i A; bara typ 2 prövar .BAS när .BAC gav CY med A = 0.

In till OPEN: A = typen, HL = namnet, BC = längden, IX = FCB. Ut: NC; alla fel går till ERROR. Förstör A, BC, DE och HL.

```
07C2  F5           OPNSTR: PUSH AF
07C3  7E                   LD   A,(HL)
07C4  23                   INC  HL
07C5  FE 22                CP   $22
07C7  28 07                JR   Z,L07D0
07C9  FE 27                CP   $27
07CB  28 03                JR   Z,L07D0
07CD  2B                   DEC  HL              utan citattecken: till CR
07CE  3E 0D                LD   A,$0D
07D0  47           L07D0:  LD   B,A
07D1  0E FF                LD   C,$FF           C = längden
07D3  E5                   PUSH HL
07D4  7E           L07D4:  LD   A,(HL)
07D5  23                   INC  HL
07D6  0C                   INC  C
07D7  B8                   CP   B
07D8  28 04                JR   Z,L07DE
07DA  FE 0D                CP   $0D             CR slutar alltid
07DC  20 F6                JR   NZ,L07D4
07DE  06 00        L07DE:  LD   B,$00
07E0  E1                   POP  HL
07E1  F1                   POP  AF
07E2  EB           OPEN:   EX   DE,HL           (fel)
07E3  21 F2 FF             LD   HL,$FFF2        14 byte för FNAME på stacken
07E6  39                   ADD  HL,SP
07E7  F9                   LD   SP,HL
07E8  EB                   EX   DE,HL           HL = texten, DE = platsen
07E9  DD E5                PUSH IX
07EB  F5                   PUSH AF              A = typen
07EC  D5                   PUSH DE              DE = platsen, till IX
07ED  CD 78 14             CALL FNAME           tolka filnamnet
07F0  DD E1                POP  IX              IX = det tolkade namnet
07F2  38 78                JR   C,L086C         felaktigt namn: fel 21
07F4  ED 5B 0A FE          LD   DE,(DEVHEAD)    gå igenom enhetslistan
07F8  7A           L07F8:  LD   A,D
07F9  B3                   OR   E
07FA  CA 0F 07             JP   Z,L070F         fel 21, enheten finns inte
07FD  EB                   EX   DE,HL
07FE  CD 1D 10             CALL RDDE            DE = nästa post, HL = namnet i posten
0801  DD 7E 0B             LD   A,(IX+$0B)      jämför enhetsnamnet
```

```
0804  BE                CP   (HL)
0805  23                INC  HL
0806  20 F0             JR   NZ,L07F8
0808  DD 7E 0C          LD   A,(IX+$0C)
080B  BE                CP   (HL)
080C  23                INC  HL
080D  20 E9             JR   NZ,L07F8
080F  DD 7E 0D          LD   A,(IX+$0D)
0812  BE                CP   (HL)
0813  23                INC  HL
0814  20 E2             JR   NZ,L07F8
0816  DD E5             PUSH IX
0818  D1                POP  DE             DE = namnet, till enheten
0819  F1                POP  AF             A = typen
081A  DD E1             POP  IX             IX = FCB
081C  DD 75 03          LD   (IX+$03),L     FCB+3,+4: tabellpekaren i posten
081F  DD 74 04          LD   (IX+$04),H
0822  DD 36 09 00       LD   (IX+$09),$00
0826  A7                AND  A              A = typ av öppning
0827  28 40             JR   Z,L0869
0829  3D                DEC  A
082A  28 1A             JR   Z,L0846
082C  3D                DEC  A
082D  28 1C             JR   Z,L084B
082F  3D                DEC  A
0830  21 D1 08          LD   HL,D08D1       typ 3: .BAS
0833  28 03             JR   Z,L0838
0835  21 CE 08          LD   HL,D08CE       typ 4+: .BAC
0838  E5          L0838: PUSH HL
0839  21 08 00          LD   HL,$0008       filtypen (namn+8) blank?
083C  19                ADD  HL,DE
083D  7E                LD   A,(HL)
083E  E1                POP  HL
083F  FE 20             CP   $20
0841  20 03             JR   NZ,L0846       typ angiven: behåll
0843  CD BD 08          CALL SETEXT         nej: standardtypen
0846  CD 8D 08     L0846: CALL DVPREP        PREPARE
0849  18 24             JR   L086F

084B  21 08 00     L084B: LD   HL,$0008      typ 2: filtyp angiven?
084E  19                ADD  HL,DE
084F  7E                LD   A,(HL)
0850  FE 20             CP   $20
0852  20 15             JR   NZ,L0869
0854  21 CE 08          LD   HL,D08CE       nej: prova .BAC
0857  CD BD 08          CALL SETEXT
085A  CD 8A 08          CALL DVOPEN
085D  30 0D             JR   NC,L086C       fanns: klart
085F  A7                AND  A
0860  C2 0F 07          JP   NZ,L070F       A <> 0: fel 21 direkt
0863  21 D1 08          LD   HL,D08D1       finns inte: prova .BAS
0866  CD BD 08          CALL SETEXT
0869  CD 8A 08     L0869: CALL DVOPEN        OPEN
086C  DA 0F 07     L086C: JP   C,L070F       CY från enheten: alltid fel 21
086F  DD 36 05 01  L086F: LD   (IX+$05),$01  filen är öppen
0873  21 0E 00          LD   HL,$000E       H = 0, L = 14
0876  DD 74 0E          LD   (IX+$0E),H     +E = 0
0879  DD 74 06          LD   (IX+$06),H     +6 = 0: kolumn 0
087C  39                ADD  HL,SP          de 14 bytena bort
087D  F9                LD   SP,HL
087E  C9                RET
```

CLOSE: *stäng filen i FCB IX om den är öppen. FCB+5 nollas före enhetens CLOSE, så filen räknas som stängd även om enheten svarar CY. Registren blir sedan som enhetens CLOSE lämnar dem (för kassetten* CASCLO).

```
087F  DD 7E 05     CLOSE:  LD   A,(IX+$05)
0882  A7                   AND  A
0883  C8                   RET  Z                redan stängd
0884  DD 36 05 00          LD   (IX+$05),$00     stängd före enhetens CLOSE
0888  18 06                JR   DVCLOS
```

3.15 DVCALL: anropa enhetsrutin nummer 0–8 $088A

Raderna CALL DVCALL ligger efter varandra. Att anropa DVOPEN, DVPREP och så vidare ger alltså returadresser som skiljer sig med 3. DVCALL räknar ut rutinnumret (× 3) som returadressen minus $088D och lägger det till hopptabellens adress, som hämtas via FCB+3,+4. Står det inte JP ($C3) där blir det fel 8. BC, DE, HL och IX når enhetens rutin oförändrade, men A är då $C3 och flaggorna ändrade: ett värde i A kan inte skickas med.

```
088A  CD A5 08     DVOPEN: CALL DVCALL
088D  CD A5 08     DVPREP: CALL DVCALL
0890  CD A5 08     DVCLOS: CALL DVCALL
0893  CD A5 08     DVINP:  CALL DVCALL
0896  CD A5 08     DVPRT:  CALL DVCALL
0899  CD A5 08     DVGET:  CALL DVCALL
089C  CD A5 08     DVPUT:  CALL DVCALL
089F  CD A5 08             CALL DVCALL
08A2  CD A5 08             CALL DVCALL
08A5  E3           DVCALL: EX   (SP),HL          HL = returadressen
08A6  D5                   PUSH DE
08A7  EB                   EX   DE,HL            DE = returadressen
08A8  DD 6E 03             LD   L,(IX+$03)
08AB  DD 66 04             LD   H,(IX+$04)
08AE  7E                   LD   A,(HL)           HL = hopptabellen
08AF  23                   INC  HL
08B0  66                   LD   H,(HL)
08B1  6F                   LD   L,A
08B2  19                   ADD  HL,DE            tabellen + returadressen
08B3  11 8D 08             LD   DE,$088D         HL = tabellen + 3 × numret
08B6  A7                   AND  A
08B7  ED 52                SBC  HL,DE
08B9  D1                   POP  DE
08BA  C3 84 01             JP   L0184            JP där? hoppa
```

SETEXT: *kopiera 3 tecken från HL till filtypen (namn+8) i det tolkade namnet vid DE. Ut: HL efter de 3 tecknen. A, BC och DE bevaras.*

```
08BD  D5           SETEXT: PUSH DE
08BE  E5                   PUSH HL
08BF  21 08 00             LD   HL,$0008
08C2  19                   ADD  HL,DE
08C3  EB                   EX   DE,HL
08C4  E1                   POP  HL
08C5  C5                   PUSH BC
08C6  01 03 00             LD   BC,$0003
08C9  ED B0                LDIR
08CB  C1                   POP  BC
08CC  D1                   POP  DE
08CD  C9                   RET
```

```
08CE  42 41 43     D08CE:  DEFB "BAC"          binär programfil (SAVE)
08D1  42 41 53     D08D1:  DEFB "BAS"          programtext (LIST)
```

PRINT, INPUT *och* GETPOS *går till skärmen/tangentbordet, eller till filen om ett filnummer är valt* ($FE23 <> *0,* CURFIL).

```
PRINT   HL = text, BC = längd
INPUT   läs en rad, fel 34 vid filslut, annars enhetens fel
        HL = bufferten, C = platsen
GETPOS  A = kolumn, E = radbredd
```

Utan fil gäller PRTXT, INLINE *och* SCRPOS. *Med fil är IX = FCB efteråt, och registren blir som enhetens rutin lämnar dem (för kassetten* TXTPRT *och* TXTINP).

```
08D4  CD FC 08     PRINT:  CALL CURFIL         CY: IX = FCB
08D7  D2 A8 01             JP   NC,PRTXT       ingen fil: skärmen
08DA  C3 96 08             JP   DVPRT

08DD  CD FC 08     INPUT:  CALL CURFIL
08E0  D2 AA 02             JP   NC,INLINE
08E3  CD 93 08             CALL DVINP
08E6  D0                   RET  NC             raden läst
08E7  A7                   AND  A
08E8  20 02                JR   NZ,L08EC
08EA  3E A2                LD   A,$A2          A = 0: filslut, fel 34
08EC  C3 2E 06     L08EC:  JP   ERROR          A = felkoden

08EF  CD FC 08     GETPOS: CALL CURFIL
08F2  D2 38 03             JP   NC,SCRPOS
08F5  DD 7E 06             LD   A,(IX+$06)     fil: A = kolumn, E = radbredd
08F8  DD 5E 07             LD   E,(IX+$07)
08FB  C9                   RET
```

CURFIL*: Z, NC och A = 0 om inget filnummer är valt (IY+*$0D*,* $FE23*). Annars letar* FNDFIL *upp FCB:n: IX = FCB, NZ och CY. Saknas filen eller är den inte öppen blir det fel 32. Förstör A och IX; BC bevaras.*

```
08FC  FD 7E 0D     CURFIL: LD   A,(IY+$0D)
08FF  A7                   AND  A
0900  C8                   RET  Z              Z, NC: ingen fil vald
0901  C5                   PUSH BC
0902  CD E1 20            CALL FNDFIL         leta FCB för filnumret
0905  C2 23 07             JP   NZ,L0723       fel 32, FILEN EJ ÖPPNAD
0908  DD CB 05 46          BIT  0,(IX+$05)     bit 0: öppen?
090C  CA 23 07             JP   Z,L0723
090F  C1                   POP  BC
0910  37                   SCF                 NZ, CY: IX = FCB
0911  C9                   RET
```

3.16 TXTPRT: skriv text till blockad fil (enhetsrutin 4 för CAS)

$0912

Tecknen läggs i blockbufferten. LF tas bort. En följd av blanktecken lagras som $09 + antal. Blocket slutar med $03, och när det är fullt skrivs det med PUT. FCB+6 är aktuell kolumn.

Två eller fler blanktecken före CR stryks helt; ett ensamt står kvar. Efter en följd av blanktecken som följs av ett annat tecken än CR blir kolumnen ett steg för stor (prövat). $09 n läggs aldrig i blockets två sista byte: då skrivs blocket först.

In: HL = texten, BC = längden, IX = FCB. Ut: HL efter texten, BC = 0, positionen i FCB+A,+B och FCB+E = $80. Förstör A och DE.

```
0912  DD 5E 0A     TXTPRT: LD   E,(IX+$0A)      (egenhet)
0915  DD 56 0B             LD   D,(IX+$0B)
0918  78           L0918:  LD   A,B             BC = kvar av texten
0919  B1                   OR   C
091A  28 27                JR   Z,L0943
091C  7E                   LD   A,(HL)
091D  FE 0A                CP   $0A             LF tas bort
091F  28 1E                JR   Z,L093F
0921  DD 34 06             INC  (IX+$06)        kolumn + 1
0924  FE 0D                CP   $0D             CR: kolumn 0
0926  20 04                JR   NZ,L092C
0928  DD 36 06 00          LD   (IX+$06),$00
092C  FE 20        L092C:  CP   $20             flera blanktecken?
092E  20 05                JR   NZ,L0935
0930  23                   INC  HL
0931  BE                   CP   (HL)
0932  2B                   DEC  HL
0933  28 19                JR   Z,L094E
0935  ED A0        L0935:  LDI                  tecknet till bufferten
0937  DD 35 0D             DEC  (IX+$0D)        blocket fullt?
093A  CC 95 09             CALL Z,TXTBLK
093D  18 D9                JR   L0918

093F  23           L093F:  INC  HL
0940  0B                   DEC  BC
0941  18 D5                JR   L0918

0943  DD 73 0A     L0943:  LD   (IX+$0A),E      spara positionen
0946  DD 72 0B             LD   (IX+$0B),D
0949  DD 36 0E 80          LD   (IX+$0E),$80    text skriven: CLOSE gör TXTEND
094D  C9                   RET

094E  D5           L094E:  PUSH DE
094F  1E 00                LD   E,$00           E = antal blanka
0951  1C           L0951:  INC  E
0952  23                   INC  HL
0953  0B                   DEC  BC
0954  78                   LD   A,B
0955  B1                   OR   C
0956  28 0C                JR   Z,L0964
0958  DD 34 06             INC  (IX+$06)        även tecknet efter: kolumnen 1 för stor
095B  7E                   LD   A,(HL)
095C  FE 0D                CP   $0D
095E  28 22                JR   Z,L0982         CR: blankorna stryks
0960  FE 20                CP   $20
0962  28 ED                JR   Z,L0951
0964  3E 02        L0964:  LD   A,$02           högst 2 lediga: nytt block först
0966  DD BE 0D             CP   (IX+$0D)
0969  7B                   LD   A,E
096A  D1                   POP  DE
096B  F5                   PUSH AF
096C  D4 95 09             CALL NC,TXTBLK
096F  F1                   POP  AF
0970  EB                   EX   DE,HL
0971  36 09                LD   (HL),$09        $09 + antal blanktecken
0973  23                   INC  HL
0974  77                   LD   (HL),A
0975  23                   INC  HL
0976  EB                   EX   DE,HL
0977  DD 35 0D             DEC  (IX+$0D)        två byte mindre
097A  DD 35 0D             DEC  (IX+$0D)
097D  CC 95 09             CALL Z,TXTBLK
```

```
0980  18 96              JR   L0918

0982  D1          L0982:  POP  DE
0983  18 93              JR   L0918
```

TXTEND: skriv aktuellt block. Lägg sedan 6 nollbyte (slutmarkering) först i ett nytt block och skriv det också.

TXTBLK: avsluta blocket med $03 och skriv det (PUT). In: DE = platsen efter sista tecknet. Ut: DE = den nya bufferten från enheten; BC och HL bevaras, A förstörs. TXTEND tar positionen ur FCB:n och förstör A, B och DE.

```
0985  DD 5E 0A     TXTEND: LD   E,(IX+$0A)
0988  DD 56 0B             LD   D,(IX+$0B)
098B  CD 95 09             CALL TXTBLK
098E  06 06                LD   B,$06          6 nollbyte = filslut
0990  AF                   XOR  A
0991  12          L0991:  LD   (DE),A
0992  13                   INC  DE
0993  10 FC                DJNZ L0991          sedan in i TXTBLK: $03 efter nollorna
0995  C5          TXTBLK: PUSH BC
0996  E5                   PUSH HL
0997  EB                   EX   DE,HL          HL = bufferten
0998  36 03                LD   (HL),$03       blockslut
099A  CD 9C 08             CALL DVPUT          PUT
099D  E1                   POP  HL             DE = nya bufferten, från enheten
099E  C1                   POP  BC
099F  C9                   RET
```

3.17 TXTINP: läs en textrad från blockad fil (enhetsrutin 3 för CAS) $09A0

Kopierar till DE tills CR. $03 = blockslut: nästa block hämtas med GET. $09 n expanderas till n blanktecken. $00 = filslut (A = 0, CY = 1). Tecken över $7F ger fel 58, och för lång rad ger fel 20.

In: HL = mottagaren, C = platsen. Raden med CR måste vara kortare än C, annars fel 20 (prövat). Positionen i FCB:n sparas bara efter CR, och ett LF i filen följer med raden. Ut: NC och DE efter radens CR, HL = positionen i blocket. Vid fel CY och A = 0 (filslut), $94 (fel 20) eller $BA (fel 58). Förstör B.

```
09A0  EB            TXTINP: EX   DE,HL
09A1  DD 36 0D 01           LD   (IX+$0D),$01   +D = 1; läses inte vid läsning
09A5  DD 6E 0A              LD   L,(IX+$0A)     HL = positionen i blocket
09A8  DD 66 0B              LD   H,(IX+$0B)
09AB  7E            L09AB:  LD   A,(HL)
09AC  FE 03                 CP   $03            blockslut: läs nästa block
09AE  28 1D                 JR   Z,L09CD
09B0  FE 09                 CP   $09            komprimerade blanktecken
09B2  28 22                 JR   Z,L09D6
09B4  A7                    AND  A              $00: filslut
09B5  37                    SCF
09B6  C8                    RET  Z
09B7  3E BA                 LD   A,$BA          fel 58
09B9  F8                    RET  M
09BA  7E                    LD   A,(HL)
09BB  06 00                 LD   B,$00          C = platser kvar
09BD  ED A0                 LDI
09BF  E2 E9 09              JP   PO,L09E9       BC = 0: fel 20, även med CR sist
09C2  FE 0D                 CP   $0D            till och med CR
09C4  20 E5                 JR   NZ,L09AB
```

```
09C6  DD 74 0B            LD   (IX+$0B),H      positionen sparas bara här
09C9  DD 75 0A            LD   (IX+$0A),L
09CC  C9                  RET

09CD  C5           L09CD: PUSH BC
09CE  D5                  PUSH DE
09CF  CD 99 08            CALL DVGET           HL = nästa block
09D2  D1                  POP  DE
09D3  C1                  POP  BC
09D4  18 D5               JR   L09AB

09D6  23           L09D6: INC  HL
09D7  46                  LD   B,(HL)
09D8  23                  INC  HL
09D9  79                  LD   A,C             ryms n blanka?
09DA  90                  SUB  B
09DB  38 0C               JR   C,L09E9
09DD  28 0A               JR   Z,L09E9
09DF  4F                  LD   C,A             C minskas i förväg
09E0  EB                  EX   DE,HL
09E1  36 20        L09E1: LD   (HL),$20
09E3  23                  INC  HL
09E4  10 FB               DJNZ L09E1
09E6  EB                  EX   DE,HL
09E7  18 C2               JR   L09AB

09E9  3E 94        L09E9: LD   A,$94            fel 20, FÖR LÅNG RAD
09EB  37                  SCF
09EC  C9                  RET
```

Korsreferenstabell för OPEN och filrutinerna. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken, i systemvariablerna och i FCB:n som IX pekar på; IY-kolumnen är det valda filnumret vid IY+$0D ($FE23). FCB-kolumnerna: +3 enhetens hopptabellspekare (+3,+4), +5 bit 0 = öppen, +6 kolumnen, +7 radbredden, +8 dataområdets början (+8,+9), +$0A positionen i blocket (+$0A,+$0B), +$0D de lediga byten och +$0E bit 7 = text skriven. Det som en anropad rutin gör räknas med, och för enheten gäller kassetten (tabellen efter $062C, där också CASBLK, CASBUF och mottagningen står): OPEN och CLOSE går genom CASOPN, CASPRE och CASCLO, TXTBLK skriver med CASPUT och TXTINP läser med CASGET. PRINT, INPUT och GETPOS går utan fil till PRTXT, INLINE och SCRPOS, och med fil genom DVPRT och DVINP till TXTPRT och TXTINP. DVOPEN–DVPUT är rader med CALL DVCALL, som räknar ut rutinens nummer ur returadressen och hoppar till enhetens hopptabell med BC, DE, HL och IX orörda; A blir $C3. OPEN lägger det tolkade namnet i 14 byte på stacken och tar bort dem igen, och CURFIL letar upp FCB:n med FNDFIL. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler				FCB								IY+
Rutin	A	CY	BC	DE	HL	IX	stack	DEVHEAD	FILLST	CURROW	CURCOL	+3	+5	+6	+7	+8	+$0A	+$0D	+$0E	$0D
OPNSTR	IX	X	X	X	IX	I		I				U	U	U	U	U	U	U	U	
OPEN	IX	U	IX	X	IX	I		I				U	U	U	U	U	U	U	U	
CLOSE	X	X	X	X		I						I	IU		U	IU	IU	U	IU	
DVCALL	X	X	I	I	I	I	I					I								
SETEXT		X		I	IU															
PRINT	X	X	IU	X	IU	U			I	IU	IU	I	I	IU	U	IU	IU	IU	U	I
INPUT	U	U	IX	U	IU	U			I	IU	IU	I	I			U	IU	U		I
GETPOS	U	X		U		U			I		I		I	I	I					I
CURFIL	U	U				U			I				I							I

	register och stack							variabler				FCB								IY+
Rutin	A	CY	BC	DE	HL	IX	stack	DEVHEAD	FILLST	CURROW	CURCOL	+3	+5	+6	+7	+8	+$0A	+$0D	+$0E	$0D
TXTPRT	X	X	IU	X	IU	I						I		IU	U	IU	IU	IU	U	
TXTEND	X		X	X		I						I			U	IU	IU	U		
TXTBLK	X			IU		I						I			U	IU	U	U		
TXTINP	U	U	IX	U	IU	I						I				U	IU	U		

Rutin	**Anropar**	**Anropas från**
OPNSTR	går vidare till OPEN	den fasta ingången $003E, CMDED, CMDMERG, CMDSAVE
OPEN	FNAME, RDDE, SETEXT, DVPREP, DVOPEN; fel 21	den fasta ingången $002D, OPNSTR, PREPST
CLOSE	DVCLOS	CMDLIST, CMDMERG, PREPST, CLOSST, CLSALL
DVCALL	enhetens hopptabell (för kassetten CASTAB); annars fel 8	DVOPEN och DVPREP (OPEN), DVCLOS (CLOSE), DVINP (INPUT, CMDMERG), DVPRT (PRINT, CMDLIST), DVGET (TXTINP, CMDMERG), DVPUT (TXTBLK, CMDSAVE)
SETEXT		OPEN
PRINT	CURFIL, PRTXT eller DVPRT	PRTEXP, PRCRLF, PRTCOM
INPUT	CURFIL, INLINE eller DVINP; fel till ERROR	INPT, INPLIN
GETPOS	CURFIL, SCRPOS	PRTFIT, PRTCOM, EXTAB
CURFIL	FNDFIL; fel 32	PRINT, INPUT, GETPOS
TXTPRT	TXTBLK	den fasta ingången $001B, CASTAB (genom DVPRT)
TXTEND	TXTBLK	den fasta ingången $0023, CASCLO
TXTBLK	DVPUT	TXTPRT, TXTEND
TXTINP	DVGET	den fasta ingången $0015, CASTAB (genom DVINP)

```
09ED  1B          L09ED:  DEC  DE          tillbaka till tecknet som skilde
09EE  13          L09EE:  INC  DE
09EF  1A                  LD   A,(DE)
09F0  87                  ADD  A,A
09F1  30 FB               JR   NC,L09EE    fram till nästa posts kod (bit 7)
09F3  E1                  POP  HL          HL = textens början igen
09F4  C1                  POP  BC
```

3.18 KWSRCH: sök nyckelord i tabell $09F5

DE = tabell, HL = text. Varje post är en kod med bit 7 satt, följd av ordet. Tabellen slutar med $FF. Tabellens tecken $41–$5D (A–Z och Ä, Ö, Å) jämförs utan hänsyn till skiftläge. Efter varje lika tecken hoppas blanktecken i texten över, men inte före ordet. Ordet behöver bara stå först i texten: LISTX hittar LIST, med HL på X. BC bevaras.

Ut: Z och A = kod om ordet hittades, HL efter ordet och blanka efter det, DE efter nästa posts kod. Annars NZ, HL orört och DE vid $FF.

```
09F5  1A           KWSRCH: LD   A,(DE)
09F6  FE FF                CP   $FF            tabellens slut?
09F8  20 02                JR   NZ,L09FC
09FA  A7                   AND  A              ja: NZ = inte hittat
09FB  C9                   RET

09FC  C5           L09FC:  PUSH BC
09FD  E5                   PUSH HL             spara textens början
09FE  4F                   LD   C,A            C = postens kod
09FF  13                   INC  DE
0A00  1A           L0A00:  LD   A,(DE)         tabellens tecken
0A01  FE 41                CP   $41            bokstav? jämför utan skiftläge
0A03  38 09                JR   C,L0A0E        under $41: exakt lika
0A05  FE 5E                CP   $5E
0A07  30 05                JR   NC,L0A0E       över $5D: exakt lika
0A09  AE                   XOR  (HL)           0 om lika, utom bit 5
0A0A  E6 DF                AND  $DF            bortse från bit 5, versal/gemen
0A0C  18 01                JR   L0A0F

0A0E  AE           L0A0E:  XOR  (HL)           0 om lika
0A0F  F5           L0A0F:  PUSH AF             Z skyddas
0A10  CC BE 25             CALL Z,SKIPSP       lika: nästa tecken utom blanka
0A13  F1                   POP  AF             Z tillbaka
0A14  1A                   LD   A,(DE)         samma tabellbyte, flaggorna står kvar
0A15  13                   INC  DE             ändrar inga flaggor
0A16  28 E8                JR   Z,L0A00        lika: fortsätt i ordet
0A18  87                   ADD  A,A            olika: nästa posts kod?
0A19  30 D2                JR   NC,L09ED       nej: nästa post
0A1B  79                   LD   A,C            hittat: A = koden
0A1C  C1                   POP  BC             släpp textens början, HL efter ordet
0A1D  C1                   POP  BC
0A1E  BF                   CP   A              Z = hittat
0A1F  C9                   RET
```

Direktkommandon. Koden är index i CMDTAB (NEW *och* SCR = *samma*).

```
0A20  80 42 59 45  CMDKW:  DEFB $80,"BYE"
0A24  81 52 55 4E          DEFB $81,"RUN"
0A28  82 4C 49 53…         DEFB $82,"LIST"
0A2D  83 4E 45 57          DEFB $83,"NEW"
0A31  83 53 43 52          DEFB $83,"SCR"
0A35  84 43 4C 45…         DEFB $84,"CLEAR"
0A3B  85 4C 4F 41…         DEFB $85,"LOAD"
0A40  86 4D 45 52…         DEFB $86,"MERGE"
0A46  87 52 45 4E          DEFB $87,"REN"
0A4A  88 45 44             DEFB $88,"ED"
0A4D  89 53 41 56…         DEFB $89,"SAVE"
0A52  8A 55 4E 53…         DEFB $8A,"UNSAVE"
0A59  FF                   DEFB $FF
0A5A  7A 01        CMDTAB: DEFW L017A          $80 BYE: tilläggs-ROM:en
0A5C  4C 0D                DEFW CMDRUN         $81 RUN
```

```
0A5E  EF 0A           DEFW CMDLIST        $82 LIST
0A60  70 0A           DEFW NEW            $83 NEW och SCR
0A62  80 0A           DEFW CLEAR          $84 CLEAR
0A64  A8 0B           DEFW CMDLOAD        $85 LOAD
0A66  A9 0B           DEFW CMDMERG        $86 MERGE
0A68  8A 0C           DEFW CMDREN         $87 REN
0A6A  A1 0A           DEFW CMDED          $88 ED
0A6C  C6 0D           DEFW CMDSAVE        $89 SAVE
0A6E  4C 01           DEFW L014C          $8A UNSAVE: tilläggs-ROM:en
```

NEW (*och* SCR)*: töm programmet* (EOFA = BOFA *och slutmarkering* $01), TRACE *av och inte längre skyddat* (FLAGS *bit 4, 5*). *Sedan* CLEAR. CLEAR*: stäng alla filer och töm fillistan* (FILLST*:s höga byte = 0*), VARLST *= 0, LNKOK = 0,* ON ERROR *av* (ONERR*:s höga byte = 0*), *ERRCOD = 0,* HEAP = EOFA*+1. Förstör A och HL och det som enheternas* CLOSE *förstör; IX bevaras.*

```
0A70  FD CB 0F A6  NEW:    RES  4,(IY+$0F)     TRACE av
0A74  FD CB 0F AE          RES  5,(IY+$0F)     inte skyddat
0A78  2A 1C FE             LD   HL,(BOFA)
0A7B  22 1E FE             LD   (EOFA),HL
0A7E  36 01                LD   (HL),$01       slutmarkering
0A80  DD E5        CLEAR:  PUSH IX             IX bevaras, CLSALL ändrar den
0A82  CD CD 20             CALL CLSALL         stäng alla filer
0A85  DD E1                POP  IX
0A87  21 00 00             LD   HL,$0000
0A8A  FD 74 1D             LD   (IY+$1D),H     FILLST:s höga byte = 0: tom
0A8D  22 29 FE             LD   (VARLST),HL
0A90  FD 75 10             LD   (IY+$10),L     LNKOK = 0
0A93  FD 75 1F             LD   (IY+$1F),L     ON ERROR av
0A96  FD 75 22             LD   (IY+$22),L     ERRCOD = 0
0A99  2A 1E FE             LD   HL,(EOFA)      HEAP = EOFA+1
0A9C  23                   INC  HL
0A9D  22 20 FE             LD   (HEAP),HL
0AA0  C9                   RET
```

3.19 ED n: redigera rad n

$0AA1

Raden detokeniseras ($1195) i 126 byte på stacken och skrivs ut.

Den blir mall för INLIN2: varje tryck på TAB hämtar mallens nästa tecken, i tur och ordning och oberoende av vad som skrivits ("2" och 11 tryck på TAB på raden `10 PRINT 12345` ger `210 PRINT 12`). Den nya raden (i BUF1, högst 120 tecken) körs sedan som om den hade skrivits in ($00F4), så den kan lika gärna vara ett kommando. Inget tal eller något efter talet: fel 31. Raden saknas: fel 6. In: HL efter ED. Kommer inte tillbaka hit: raden går till kommandoslingan, och READY från DOCMD ligger kvar på stacken tills CMDLP sätter om SP.

```
0AA1  CD 42 07     CMDED:  CALL NEWLIN
0AA4  CD 1E 18             CALL GETLN          DE = radnumret
0AA7  DA 21 07             JP   C,L0721        inget tal: fel 31
0AAA  7E                   LD   A,(HL)         efter talet: bara CR
0AAB  FE 0D                CP   $0D
0AAD  C2 21 07             JP   NZ,L0721
0AB0  CD 39 0F             CALL FINDLN
0AB3  C2 05 07             JP   NZ,L0705       raden saknas: fel 6
0AB6  EB                   EX   DE,HL          DE = raden
0AB7  21 82 FF             LD   HL,$FF82       126 byte på stacken
0ABA  39                   ADD  HL,SP
0ABB  F9                   LD   SP,HL
0ABC  EB                   EX   DE,HL          HL = raden, DE = mallens plats
0ABD  D5                   PUSH DE             mallens början
```

```
0ABE  CD 95 11            CALL DETOK          mallen slutar med CR
0AC1  EB                  EX   DE,HL          HL efter mallens CR
0AC2  D1                  POP  DE
0AC3  D5                  PUSH DE
0AC4  36 0A               LD   (HL),$0A       LF efter
0AC6  23                  INC  HL
0AC7  ED 52               SBC  HL,DE          BC = mallens längd med CR LF
0AC9  44                  LD   B,H
0ACA  4D                  LD   C,L
0ACB  EB                  EX   DE,HL          HL = mallen
0ACC  CD A8 01            CALL PRTXT
0ACF  D1                  POP  DE             DE = mallen
0AD0  21 40 FE            LD   HL,$FE40       BUF1, högst 120 tecken
0AD3  0E 78               LD   C,$78
0AD5  E5                  PUSH HL
0AD6  CD AD 02            CALL INLIN2         TAB hämtar ur mallen (DE)
0AD9  21 80 00            LD   HL,$0080       SP förbi BUF1 och mallen
0ADC  39                  ADD  HL,SP
0ADD  D1                  POP  DE             DE = BUF1
0ADE  F9                  LD   SP,HL
0ADF  EB                  EX   DE,HL          HL = BUF1
0AE0  C3 F4 00            JP   L00F4          kör raden som om den skrivits in

0AE3  DD 36 0F 01  L0AE3: LD   (IX+$0F),$01   till fil: hela programmet
0AE7  3E 03               LD   A,$03
0AE9  CD C2 07            CALL OPNSTR
0AEC  AF                  XOR  A              A = 0 och Z: hela programmet
0AED  18 25               JR   L0B14
```

3.20 LIST [n][-m] eller LIST fil $0AEF

IX = 16 byte på stacken: en FCB (IX+0 till IX+E, som till en fil) och en flagga. IX+0 = radräknare (22 rader per sida), IX+F = 1 om listningen går till en fil (OPEN typ 3, .BAS).

Ett filnamn börjar med " eller ' eller en bokstav. Till fil skrivs alltid hela programmet (ett radintervall efter namnet läses inte), utan sidvisning; varje rad slutar med CR LF.

På skärmen: n, n-m, n,m, n- (till slutet) och -m (från början). Efter 22 rader väntar LIST på en tangent: CR avbryter, andra tangenter visar en rad till. BREAK avbryter också. Efter första talet får bara -, komma eller CR stå (annars fel 31); efter sista talet läses inget, så LIST 10-20X listar 10–20. Förstör A, BC, DE, HL och IX.

```
                  CMDLIST:
0AEF  DD 21 F0 FF         LD   IX,$FFF0       IX = 16 byte på stacken
0AF3  DD 39               ADD  IX,SP
0AF5  DD F9               LD   SP,IX
0AF7  DD 36 00 16         LD   (IX+$00),$16   IX+0 = 22 rader per sida
0AFB  DD 36 0F 00         LD   (IX+$0F),$00   IX+F = 0: till skärmen
0AFF  7E                  LD   A,(HL)         filnamn efter LIST?
0B00  FE 22               CP   $22
0B02  28 06               JR   Z,L0B0A
0B04  FE 27               CP   $27
0B06  28 02               JR   Z,L0B0A
0B08  FE 41               CP   $41            bokstav ($41 och uppåt)?
0B0A  30 D7        L0B0A: JR   NC,L0AE3       citattecken eller bokstav: fil
0B0C  E5                  PUSH HL
0B0D  CD 42 07            CALL NEWLIN
0B10  E1                  POP  HL
0B11  7E                  LD   A,(HL)
0B12  FE 0D               CP   $0D            bara LIST: Z, hela programmet
```

```
0B14  11 00 00     L0B14:  LD   DE,$0000       från 0 till 65535
0B17  01 FF FF             LD   BC,$FFFF
0B1A  28 1E                JR   Z,L0B3A
0B1C  CD 1E 18             CALL GETLN          DE = första raden (0 om inget tal)
0B1F  D5                   PUSH DE
0B20  7E                   LD   A,(HL)
0B21  FE 0D                CP   $0D
0B23  28 12                JR   Z,L0B37        bara n: BC = n, en rad
0B25  FE 2D                CP   $2D            -
0B27  28 05                JR   Z,L0B2E
0B29  FE 2C                CP   $2C            ,
0B2B  C2 21 07             JP   NZ,L0721
0B2E  DF           L0B2E:  RST  $18            förbi tecknet, A = nästa
0B2F  CD 1E 18             CALL GETLN          DE = sista raden; det som följer läses inte
0B32  30 03                JR   NC,L0B37
0B34  11 FF FF             LD   DE,$FFFF       inget tal efter -: till slutet
0B37  42           L0B37:  LD   B,D            BC = sista raden
0B38  4B                   LD   C,E
0B39  D1                   POP  DE
0B3A  C5           L0B3A:  PUSH BC             FINDLN förstör BC
0B3B  CD 39 0F             CALL FINDLN         HL = första raden >= DE
0B3E  C1           L0B3E:  POP  BC
0B3F  7E                   LD   A,(HL)         programmets slut?
0B40  EE 01                XOR  $01
0B42  28 57                JR   Z,L0B9B
0B44  CD 3E 03             CALL TSTBRK         BREAK? NZ; CY = 0 för SBC nedan
0B47  20 52                JR   NZ,L0B9B
0B49  54                   LD   D,H            DE = raden
0B4A  5D                   LD   E,L
0B4B  23                   INC  HL             HL = radnumret
0B4C  7E                   LD   A,(HL)
0B4D  23                   INC  HL
0B4E  66                   LD   H,(HL)
0B4F  6F                   LD   L,A
0B50  ED 42                SBC  HL,BC          radnummer > BC: slut
0B52  38 02                JR   C,L0B56
0B54  20 45                JR   NZ,L0B9B
0B56  EB           L0B56:  EX   DE,HL          HL = raden
0B57  C5                   PUSH BC
0B58  11 40 FE             LD   DE,$FE40       detokenisera i BUF1
0B5B  D5                   PUSH DE
0B5C  DD E5                PUSH IX
0B5E  CD 95 11             CALL DETOK          detokenisera raden
0B61  DD E1                POP  IX
0B63  E3                   EX   (SP),HL        HL = BUF1, nästa rad vid SP
0B64  EB                   EX   DE,HL
0B65  36 0A                LD   (HL),$0A       LF efter CR
0B67  23                   INC  HL
0B68  A7                   AND  A
0B69  ED 52                SBC  HL,DE          HL = längden med CR LF
0B6B  EB                   EX   DE,HL
0B6C  4B                   LD   C,E            BC = längden, HL = BUF1
0B6D  42                   LD   B,D
0B6E  DD E5                PUSH IX
0B70  DD CB 0F 46          BIT  0,(IX+$0F)     fil eller skärm
0B74  28 05                JR   Z,L0B7B
0B76  CD 96 08             CALL DVPRT
0B79  18 03                JR   L0B7E

0B7B  CD A8 01     L0B7B:  CALL PRTXT
0B7E  DD E1        L0B7E:  POP  IX
```

```
0B80  E1                POP  HL           HL = nästa rad
0B81  DD CB 0F 46       BIT  0,(IX+$0F)
0B85  20 B7             JR   NZ,L0B3E     till fil: ingen sidvisning
0B87  DD 35 00          DEC  (IX+$00)     22 rader, sedan vänta
0B8A  20 B2             JR   NZ,L0B3E
0B8C  DD 34 00          INC  (IX+$00)     räknaren 1: en rad i taget
0B8F  DD E5             PUSH IX
0B91  CD EA 02          CALL GETKEY       CR avbryter, annan tangent: en rad till
0B94  DD E1             POP  IX
0B96  FE 0D             CP   $0D
0B98  20 A4             JR   NZ,L0B3E     annan tangent: en rad till
0B9A  C1                POP  BC           släpp sista radnumret
0B9B  DD CB 0F 46 L0B9B: BIT 0,(IX+$0F)
0B9F  C4 7F 08          CALL NZ,CLOSE     stäng filen
0BA2  21 10 00          LD   HL,$0010     släpp de 16 byten
0BA5  39                ADD  HL,SP
0BA6  F9                LD   SP,HL
0BA7  C9                RET
```

3.21 LOAD och MERGE $0BA8

CMDLOAD börjar på $0BA8, där byten $3E $AF blir LD A,$AF (A skilt från 0, alltså NEW först). CMDMERG börjar på $0BA9 med XOR A (A = 0, inget NEW).

Filen öppnas med OPEN typ 2. Filen läses först som text (INPUT), rad för rad. Varje rad tokeniseras och läggs in som om den hade skrivits in.

En rad som inte går att tokenisera får felutskrift och sparas ändå, som token $95 + hela textraden (med numret; LIST visar 20 ??? 20 A=). En rad utan radnummer ger fel 16; raderna före står kvar.

Ger INPUT fel 58 (tecken över $7F) är det en BAC-fil med tokeniserade rader. Då ska buffertens första byte vara $82 eller $83 (fel 59 annars). $83 = skyddat program och sätter bit 5 i $FE25. In: HL = namnet. Ut: NC och filen stängd; fel går till ERRSTP efter CLOSE.

```
              CMDLOAD:
0BA8  3E              DEFB $3E          LOAD: LD A,$AF tillsammans med nästa byte
              CMDMERG:
0BA9  AF              XOR  A            MERGE: A = 0
0BAA  DD 21 F1 FF     LD   IX,$FFF1     IX = FCB om 15 byte på stacken
0BAE  DD 39           ADD  IX,SP
0BB0  DD F9           LD   SP,IX
0BB2  08              EX   AF,AF'       A = LOAD/MERGE undan
0BB3  3E 02           LD   A,$02        OPEN typ 2 (BAC eller BAS)
0BB5  CD C2 07        CALL OPNSTR
0BB8  08              EX   AF,AF'
0BB9  A7              AND  A
0BBA  C4 70 0A        CALL NZ,NEW       LOAD: NEW först
0BBD  21 40 FE  L0BBD: LD  HL,$FE40     BUF1, högst 120 tecken
0BC0  0E 78           LD   C,$78
0BC2  E5              PUSH HL
0BC3  CD 93 08        CALL DVINP        läs en textrad
0BC6  E1              POP  HL
0BC7  38 44           JR   C,L0C0D      CY: fel eller filslut, A = koden
0BC9  E7              RST  $20          tom rad hoppas över
0BCA  FE 0D           CP   $0D
0BCC  28 EF           JR   Z,L0BBD
0BCE  DD E5           PUSH IX           FCB:n undan
0BD0  E5              PUSH HL
```

```
0BD1  CD 90 18           CALL TOKLIN          tokenisera
0BD4  D1                 POP  DE              DE = textraden
0BD5  DD 6E 01           LD   L,(IX+$01)      HL = radnumret (IX = raden)
0BD8  DD 66 02           LD   H,(IX+$02)
0BDB  30 23              JR   NC,L0C00        NC: tokeniserad
0BDD  E5                 PUSH HL              radnumret undan
0BDE  D5                 PUSH DE
0BDF  DD E5              PUSH IX
0BE1  CD 9B 06           CALL ERRMSG          felet skrivs ut, raden sparas ändå
0BE4  D1                 POP  DE              DE = den tokeniserade raden
0BE5  21 03 00           LD   HL,$0003        efter längd och radnummer
0BE8  19                 ADD  HL,DE
0BE9  EB                 EX   DE,HL
0BEA  E3                 EX   (SP),HL         HL = textraden, den tokeniserade vid SP
0BEB  3E 95              LD   A,$95           $95 + hela textraden
0BED  12                 LD   (DE),A
0BEE  13                 INC  DE
0BEF  7E           L0BEF: LD   A,(HL)         kopiera till och med CR
0BF0  ED A0              LDI
0BF2  FE 0D              CP   $0D
0BF4  20 F9              JR   NZ,L0BEF
0BF6  E1                 POP  HL
0BF7  E5                 PUSH HL              IX = den tokeniserade raden
0BF8  DD E1              POP  IX
0BFA  7B                 LD   A,E             ny längd
0BFB  95                 SUB  L
0BFC  DD 77 00           LD   (IX+$00),A
0BFF  E1                 POP  HL              HL = radnumret
0C00  7C           L0C00: LD   A,H           radnummer 0: saknas
0C01  B5                 OR   L
0C02  3E 90              LD   A,$90           fel 16, RADNUMMER SAKNAS
0C04  28 17              JR   Z,L0C1D
0C06  CD 50 07           CALL INSLIN
0C09  DD E1              POP  IX              IX = FCB:n
0C0B  30 B0              JR   NC,L0BBD        NC: nästa rad; CY: fel från INSLIN
0C0D  FE BA        L0C0D: CP   $BA            $BA, fel 58 från INPUT: en BAC-fil
0C0F  28 16              JR   Z,L0C27
0C11  A7                 AND  A               A = 0: textfilens slut
0C12  20 0B              JR   NZ,L0C1F        annars fel A
0C14  CD 7F 08           CALL CLOSE           klart
0C17  21 0F 00           LD   HL,$000F        släpp FCB:n
0C1A  39                 ADD  HL,SP
0C1B  F9                 LD   SP,HL
0C1C  C9                 RET

0C1D  DD E1        L0C1D: POP  IX
0C1F  F5           L0C1F: PUSH AF              stäng och ge felet A
0C20  CD 7F 08           CALL CLOSE
0C23  F1                 POP  AF
0C24  C3 69 06           JP   ERRSTP          stäng och fel
```

BAC-fil: bufferten ska börja med `$82` (*vanligt program*) *eller* `$83` (*skyddat program, bit 5 i* `FLAGS` *sätts; i direktläget raderas det sedan vid* `READY`), *annars fel 59. En buffertadress 0 i FCB+8/9 ger fel 52. Sedan följer rader i samma format som i programminnet. Längdbyte* `$00` = *slut på blocket* (*nästa block läses med* `GET`), `$01` = *slut på filen.*

Raderna läggs först in med `INSLIN`, *som sorterar in dem. När* `INSLIN` *säger att raden hamnade sist* (*Z*) *går det över till en snabbare slinga på* `$0C5D`, *som bara lägger raderna efter varandra i slutet. Resten av filen sorteras alltså inte:* `MERGE` *av raderna 10, 30, 25 till ett program med 5 och 20 ger 5, 10, 20, 30, 25.*

```
0C27  DD 66 09     L0C27: LD   H,(IX+$09)      HL = bufferten (FCB+8,+9)
```

```
0C2A  7C                LD   A,H
0C2B  DD 6E 08          LD   L,(IX+$08)
0C2E  A7                AND  A
0C2F  3E B4             LD   A,$B4          fel 52: ingen buffert
0C31  28 DA             JR   Z,L0C0D
0C33  7E                LD   A,(HL)
0C34  FE 82             CP   $82            BAC, typ $82
0C36  28 0A             JR   Z,L0C42
0C38  FE 83             CP   $83            typ $83: skyddat program
0C3A  3E BB             LD   A,$BB          fel 59: okänd typ
0C3C  20 CF             JR   NZ,L0C0D
0C3E  FD CB 0F EE       SET  5,(IY+$0F)     skyddat program
0C42  23          L0C42: INC  HL           förbi typbyten
0C43  7E          L0C43: LD   A,(HL)       längdbyten
0C44  A7                AND  A
0C45  28 39             JR   Z,L0C80        $00: blocket slut
0C47  EE 01             XOR  $01            $01 = filslut
0C49  28 C2             JR   Z,L0C0D        A = 0 till $0C0D: klart
0C4B  E5                PUSH HL
0C4C  DD E3             EX   (SP),IX        IX = raden
0C4E  4E                LD   C,(HL)         HL = nästa rad
0C4F  06 00             LD   B,$00
0C51  09                ADD  HL,BC
0C52  E5                PUSH HL
0C53  CD 50 07          CALL INSLIN         sortera in raden
0C56  E1                POP  HL
0C57  DD E1             POP  IX
0C59  38 B2             JR   C,L0C0D        fel 3 från INSLIN
0C5B  20 E6             JR   NZ,L0C43       NZ: fler rader finns efter, fortsätt sortera
0C5D  7E          L0C5D: LD   A,(HL)       resten läggs sist, osorterat (egenhet)
0C5E  A7                AND  A
0C5F  28 24             JR   Z,L0C85        $00: blocket slut
0C61  5F                LD   E,A            DE = radens längd
0C62  16 00             LD   D,$00
0C64  EE 01             XOR  $01
0C66  28 A5             JR   Z,L0C0D        A = 0 till $0C0D: klart
0C68  E5                PUSH HL
0C69  CD 5A 33          CALL MEMRES         plats för raden?
0C6C  2B                DEC  HL             HL = EOFA, på $01:an
0C6D  3E 83             LD   A,$83
0C6F  38 9C             JR   C,L0C0D        fel 3, MINNET FULLT
0C71  42                LD   B,D            BC = längden
0C72  4B                LD   C,E
0C73  EB                EX   DE,HL
0C74  E1                POP  HL             HL = raden i bufferten
0C75  ED B0             LDIR                lägg raden sist
0C77  EB                EX   DE,HL          HL = efter raden
0C78  22 1E FE          LD   (EOFA),HL      ny EOFA och slutmarkering
0C7B  36 01             LD   (HL),$01
0C7D  EB                EX   DE,HL          HL = nästa rad i bufferten
0C7E  18 DD             JR   L0C5D

0C80  CD 99 08    L0C80: CALL DVGET         HL = nästa blocks data; CY prövas inte
0C83  18 BE              JR   L0C43

0C85  CD 99 08    L0C85: CALL DVGET         nästa block
0C88  18 D3              JR   L0C5D
```

Korsreferenstabell för programraderna och kommandona. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är FLAGS vid IY+$0F (bit 4 = TRACE, bit 5 = skyddat program), LNKOK vid IY+$10 och felkoden för ERRCODE vid IY+$22. Det som en anropad rutin gör räknas med. CLEAR

stänger filerna genom CLSALL och CLOSE, nollar VARLST, LNKOK, ON ERROR (ONERR:s höga byte) och felkoden och sätter HEAP till EOFA + 1; INSLIN och NEW gör alltid en CLEAR. KWSRCH: Z = ordet hittades; CY blir alltid 0. INSLIN: Z = raden blev sist, CY = minnet räckte inte (A = $83). CMDLIST, CMDLOAD och CMDMERG lägger en FCB på 15 eller 16 byte på stacken och tar bort den igen; den öppnas med OPNSTR (tabellen efter $09EC). CMDED och CMDLIST läser tangentbordet med INLIN2 och GETKEY, och CMDLIST prövar BREAK med TSTBRK (tabellen efter $03A2). CMDED går inte tillbaka: den inmatade raden går till CMDLP på $00F4, som om den hade skrivits in. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler									IY+		
Rutin	A	CY	BC	DE	HL	IX	stack	BOFA	EOFA	HEAP	VARLST	FILLST	ONERR	CURROW	CURCOL	EDPTR	$0F	$10	$22
INSLIN	U	U	X	X	X	I		I	IU	U	U	IU	U					U	U
KWSRCH	U	X		IU	IU														
NEW	X	X	X	X	X			I	U	U	U	IU	U				U	U	U
CLEAR	X	X	X	X	X				I	U	U	IU	U					U	U
CMDED*					I			I						IU	IU	U			
CMDLIST	X	X	X	X	IX	X		I						IU	IU				
CMDLOAD	X	X	X	X	IX	X		I	IU	U	U	IU	U	IU	IU		U	U	U
CMDMERG	X	X	X	X	IX	X		I	IU	U	U	IU	U	IU	IU		U	U	U

Rutin	**Anropar**	**Anropas från**
INSLIN	FINDLN, CLEAR, MEMRES; fel 3 tillbaka genom CMPERR	CMDLP, CMDMERG (textrader och BAC-filer)
KWSRCH	SKIPSP	den fasta ingången $0044, CMDLP, TOKKW, CMPFOR, CMPIF, CMPON, CMPOPN, STMEND, TOFLT, PRECLV, CMPNOT
NEW	går vidare till CLEAR	COLD, CMDTAB (NEW, SCR), CMDLOAD, CHAIN
CLEAR	CLSALL	ENDPRG, INSLIN, CMDTAB (CLEAR), CMDRUN; NEW går in i den
CMDED	NEWLIN, GETLN, FINDLN, DETOK, PRTXT, INLIN2; går vidare till CMDLP; fel 31, 6	CMDTAB (ED)
CMDLIST	OPNSTR, NEWLIN, GETLN, FINDLN, TSTBRK, DETOK, DVPRT eller PRTXT, GETKEY, CLOSE; fel 31	CMDTAB (LIST)
CMDLOAD, CMDMERG	OPNSTR, NEW (LOAD), DVINP, TOKLIN, ERRMSG, INSLIN, MEMRES, DVGET, CLOSE; fel till ERRSTP	CMDTAB (LOAD, MERGE), CMDRUN (LOAD)

3.22 REN [start][,steg]: numrera om programmet $0C8A

Utan tal blir både start och steg 10. Med bara start blir steget lika stort (`REN 100` ger 100, 200, 300). 0 ger fel 31, liksom ett annat tecken än komma efter start och ett blanktecken efter kommat (`REN 100, 5`); efter steget läses inget.

1. `LNKREF`: alla radreferenser (`$BF/$BE`) i programmet får adressen till raden de pekar på (fel 6 om raden saknas).
2. Alla rader får nya nummer: start, start+steg och så vidare. Går nästa nummer över 65535 numreras hela programmet om från början med 10, 20 och så vidare, även när raden som fick det sista numret var programmets sista (`REN 65000,500` med två rader ger 10, 20).
3. Varje radreferens får det nya numret från raden som adressen pekar på.

Eftersom referenserna följer adresserna och inte numren, blir de rätt även om numren byts helt. Vid fel 6 i steg 1 är radnumren orörda (bara referensernas adressfält har fyllts i fram till felet).

```
0C8A  7E          CMDREN: LD   A,(HL)        CR: bara REN (egenhet)
0C8B  11 0A 00            LD   DE,$000A      förvalt startnummer 10
0C8E  FE 0D               CP   $0D
0C90  28 09               JR   Z,L0C9B
0C92  CD 1E 18            CALL GETLN         startnummer
0C95  7A                  LD   A,D
0C96  B3                  OR   E
0C97  28 11               JR   Z,L0CAA       0: fel 31
0C99  7E                  LD   A,(HL)        tecknet efter talet, HL förbi det
0C9A  23                  INC  HL
0C9B  D5          L0C9B:  PUSH DE            startnumret, och steget om inget följer
0C9C  FE 0D               CP   $0D
0C9E  28 0D               JR   Z,L0CAD
0CA0  FE 2C               CP   $2C
0CA2  C2 21 07            JP   NZ,L0721      annat än komma: fel 31
0CA5  CD 1E 18            CALL GETLN         steg; utan blanka före talet
0CA8  7A                  LD   A,D
0CA9  B3                  OR   E
0CAA  CA 21 07    L0CAA:  JP   Z,L0721
0CAD  D5          L0CAD:  PUSH DE
0CAE  CD 01 0D            CALL LNKREF        steg 1: alla referenser får adresser
0CB1  C1                  POP  BC            BC = steg
0CB2  D1          L0CB2:  POP  DE            DE = startnummer
0CB3  2A 1C FE            LD   HL,(BOFA)     steg 2: nya radnummer
0CB6  7E          L0CB6:  LD   A,(HL)        längdbyten
0CB7  3D                  DEC  A
0CB8  28 15               JR   Z,L0CCF       $01: alla rader klara
0CBA  23                  INC  HL
0CBB  73                  LD   (HL),E        nytt radnummer
0CBC  23                  INC  HL
0CBD  72                  LD   (HL),D
0CBE  3D                  DEC  A             A = längden - 2
0CBF  85                  ADD  A,L           HL på nästa rad
0CC0  6F                  LD   L,A
0CC1  30 01               JR   NC,L0CC4
0CC3  24                  INC  H
0CC4  EB          L0CC4:  EX   DE,HL
0CC5  09                  ADD  HL,BC         nästa nummer, spill ger CY
0CC6  EB                  EX   DE,HL
0CC7  30 ED               JR   NC,L0CB6      NC: inget spill
0CC9  01 0A 00            LD   BC,$000A      spill: om från början, 10 och steg 10
0CCC  C5                  PUSH BC
0CCD  18 E3               JR   L0CB2
```

Steg 3: skriv in de nya radnumren i alla referenser.

```
0CCF  2A 1C FE     L0CCF:  LD   HL,(BOFA)      steg 3: referenserna
0CD2  7E           L0CD2:  LD   A,(HL)
0CD3  3D                   DEC  A
0CD4  C8                   RET  Z
0CD5  23                   INC  HL             förbi längd och radnummer
0CD6  23                   INC  HL
0CD7  23                   INC  HL
0CD8  CD FB 0E     L0CD8:  CALL SKPELM         nästa element; CY: radens slut
0CDB  38 F5                JR   C,L0CD2
0CDD  7E                   LD   A,(HL)
0CDE  06 01                LD   B,$01          B = en referens
0CE0  FE BF                CP   $BF            $BF = en referens
0CE2  28 06                JR   Z,L0CEA
0CE4  FE BE                CP   $BE            $BE = lista, nästa byte = antal
0CE6  20 F0                JR   NZ,L0CD8
0CE8  23                   INC  HL
0CE9  46                   LD   B,(HL)
0CEA  23           L0CEA:  INC  HL
0CEB  E5           L0CEB:  PUSH HL             HL på referensens radnummer
0CEC  23                   INC  HL
0CED  CD 1C 10             CALL RDDE1          DE = målradens adress
0CF0  7B                   LD   A,E
0CF1  B2                   OR   D
0CF2  28 08                JR   Z,L0CFC        adress 0: hoppas över
0CF4  E3                   EX   (SP),HL        hämta nytt radnummer från målraden
0CF5  13                   INC  DE             DE på målradens radnummer
0CF6  1A                   LD   A,(DE)
0CF7  77                   LD   (HL),A
0CF8  23                   INC  HL
0CF9  13                   INC  DE
0CFA  1A                   LD   A,(DE)
0CFB  77                   LD   (HL),A
0CFC  E1           L0CFC:  POP  HL             HL = nästa referens
0CFD  10 EC                DJNZ L0CEB          B referenser
0CFF  18 D7                JR   L0CD8
```

LNKREF*: gå igenom alla satser och ge varje radreferens* ($BF*, eller* $BE + *antal*) *adressen till målraden. Saknas raden blir det fel 6, och* CURLIN *pekar då på raden med referensen. Ut: DIRECT = 0. Förstör A, BC, DE och HL.*

```
0D01  FD 36 0E 00  LNKREF: LD   (IY+$0E),$00   DIRECT = 0: felet visar raden
0D05  2A 1C FE             LD   HL,(BOFA)
0D08  7E           L0D08:  LD   A,(HL)         längdbyten
0D09  3D                   DEC  A
0D0A  C8                   RET  Z              $01: klart
0D0B  23                   INC  HL             förbi längd och radnummer
0D0C  23                   INC  HL
0D0D  23                   INC  HL
0D0E  22 2C FE             LD   (CURLIN),HL    för felutskriften
0D11  CD FB 0E     L0D11:  CALL SKPELM         nästa element; CY: radens slut
0D14  38 F2                JR   C,L0D08
0D16  7E                   LD   A,(HL)
0D17  06 01                LD   B,$01          B = en referens
0D19  FE BF                CP   $BF            $BF: en referens
0D1B  28 06                JR   Z,L0D23
0D1D  FE BE                CP   $BE            $BE: B = antal
0D1F  20 F0                JR   NZ,L0D11
0D21  23                   INC  HL
0D22  46                   LD   B,(HL)
```

```
0D23  23           L0D23:  INC  HL
0D24  CD 1D 10     L0D24:  CALL RDDE          DE = radnummer
0D27  7A                   LD   A,D
0D28  B3                   OR   E
0D29  C4 33 0D             CALL NZ,REFADR     0 = ingen referens
0D2C  CD 23 10             CALL WRDE          adressfältet, 0 om numret är 0
0D2F  10 F3                DJNZ L0D24
0D31  18 DE                JR   L0D11
```

REFADR: *DE = radnummer. Ut: DE = radens adress; BC och HL bevaras, A förstörs. Saknas raden: fel 6.*

```
0D33  E5           REFADR: PUSH HL
0D34  C5                   PUSH BC
0D35  CD 39 0F             CALL FINDLN        hitta raden
0D38  C2 05 07             JP   NZ,L0705      fel 6, HITTAR EJ DETTA RADNUMMER
0D3B  EB                   EX   DE,HL         DE = radens adress
0D3C  C1                   POP  BC
0D3D  E1                   POP  HL
0D3E  C9                   RET
```

CHAIN *"fil": beräkna strängen* (RST $38)*, gör* NEW *och sätt CR efter namnet. Sedan fortsätter koden i* RUN*, som laddar och startar filen.* CHAIN *går också i direktläge. Variablerna nollas* (NEW, CLEAR)*, och namnet får vara ett uttryck, till exempel* LEFT$(*A$,5*)*.* CHAIN *"" gör* NEW *och kör det tomma programmet: det gamla programmet är alltså borta. Kommer inte tillbaka.*

```
0D3F  FF           CHAIN:  RST  $38            strängens värde på stacken (egenhet)
0D40  CD 70 0A             CALL NEW           NEW
0D43  E1                   POP  HL
0D44  E1                   POP  HL
0D45  C1                   POP  BC            HL = strängen, BC = längd
0D46  54                   LD   D,H           DE = namnet
0D47  5D                   LD   E,L
0D48  09                   ADD  HL,BC
0D49  36 0D                LD   (HL),$0D      CR i strängens eget minne
0D4B  EB                   EX   DE,HL         HL = namnet, vidare som RUN
```

3.23 RUN [fil]: starta programmet $0D4C

Med ett filnamn laddas filen först (CMDLOAD). Sedan: CLEAR, förlänkning (PRELNK), DATA-pekaren nollställs (RSTDAT) och slumptalsfröet nollställs (RNDINI). DIRECT = 0.

Stacken nollställs till STKTOP, och adressen $071B (fel 29, RETURN utan GOSUB) läggs överst. En RETURN som inte har någon GOSUB hamnar alltså där.

RUN radnr finns inte: allt efter RUN tas som ett filnamn, så RUN 100 försöker ladda filen "100". GOTO och GOSUB ger fel 2 i direktläge (NODIR), och bland direktkommandona (CMDKW) finns ingen CON eller CONT. Ett program kan alltså bara startas från början, och efter STOP eller BREAK kan det inte fortsätta. Detsamma gäller version 9913, där också CON och RUN radnr saknas (Christensen 1979). Kommer inte tillbaka: programmet slutar i ENDPRG, STOP eller ett fel.

```
0D4C  E7           CMDRUN: RST  $20            första tecknet utan blanka
0D4D  FE 0D                CP   $0D
0D4F  C4 A8 0B             CALL NZ,CMDLOAD    RUN fil: ladda först
0D52  CD 42 07             CALL NEWLIN        radbyte
0D55  CD 80 0A             CALL CLEAR         inga variabler, filerna stängda
0D58  CD 6E 0E             CALL PRELNK        förlänkning
0D5B  CD 93 22             CALL RSTDAT        första DATA
0D5E  CD 5D 22             CALL RNDINI        slumptalsfrö = 0
0D61  FD 36 0E 00          LD   (IY+$0E),$00  programläge
```

```
0D65  ED 5B 1C FE          LD   DE,(BOFA)        DE = första raden
0D69  2A 27 FE             LD   HL,(STKTOP)
0D6C  F9                   LD   SP,HL            stacken från början
0D6D  21 1B 07             LD   HL,$071B         "RETURN utan GOSUB" i botten av stacken
0D70  E5                   PUSH HL
```

RUNLIN: kör programmet från raden vid DE. Längdbyte $01 = slut, och då går det till ENDPRG (CLEAR, sedan READY). För varje rad sparas radadressen i CURLIN, TRACE skriver radnumret (bit 4 i FLAGS), och SP sparas i STMSP så att ON ERROR kan återställa stacken. Kommer inte tillbaka.

```
0D71  1A           RUNLIN: LD   A,(DE)           längdbyten
0D72  FE 01                CP   $01
0D74  CA C9 00             JP   Z,ENDPRG         programmets slut
0D77  ED 53 2C FE          LD   (CURLIN),DE      för fel och TRACE
0D7B  FD CB 0F 66          BIT  4,(IY+$0F)       TRACE?
0D7F  C4 2F 23             CALL NZ,TRCPRT
0D82  21 00 00             LD   HL,$0000
0D85  39                   ADD  HL,SP
0D86  22 36 FE             LD   (STMSP),HL       SP för ON ERROR
0D89  13                   INC  DE               förbi längd och radnummer
0D8A  13                   INC  DE
0D8B  13                   INC  DE
0D8C  CD 92 0D             CALL RUNSTM           utför raden
0D8F  13                   INC  DE               förbi CR
0D90  18 DF                JR   RUNLIN
```

RUNSTM: utför satserna på en rad, till CR. Före varje sats testas BREAK, som ger STOP. In: DE = första satsen. Ut: DE vid radens CR.

```
0D92  EB           RUNSTM: EX   DE,HL            HL = satsen för TSTBRK
0D93  CD 3E 03             CALL TSTBRK
0D96  C2 23 23             JP   NZ,BRKSTP        BREAK: STOP
0D99  EB                   EX   DE,HL            DE = satsen
0D9A  1A                   LD   A,(DE)
0D9B  FE 0D                CP   $0D
0D9D  C8                   RET  Z                CR: raden slut
0D9E  CD A3 0D             CALL EXESTM           DE efter satsen
0DA1  18 EF                JR   RUNSTM
```

3.24 EXESTM: utför en sats $0DA3

Byte under $80 (":" mellan satser) hoppas över. Satstoken $80–$96 är index i STEXE1. Token $86 betyder att nästa byte är $80 + index i STEXE2 (DIM, POKE, OPEN ...). Hoppet sker via $284D med BC = tabell. Varje sats har två rutiner: STTAB1/STTAB2 ($1A08) används när raden matas in och kompileras, och STEXE1/STEXE2 används när den körs. In: DE = satsen. Satsens rutin får DE efter token (efter båda byten för $86) och ska lämna DE efter satsen.

```
0DA3  01 14 0E     EXESTM: LD   BC,STEXE1
0DA6  1A           L0DA6:  LD   A,(DE)
0DA7  13                   INC  DE               DE efter token
0DA8  FE 80                CP   $80              satstoken?
0DAA  38 FA                JR   C,L0DA6
0DAC  87                   ADD  A,A              HL = 2*(token-$80)
0DAD  6F                   LD   L,A
0DAE  26 00                LD   H,$00
0DB0  C3 4D 28             JP   L284D            JP via (BC + HL)

0DB3  01 42 0E     EXEST2: LD   BC,STEXE2        $86: nästa byte väljer i STEXE2
0DB6  18 EE                JR   L0DA6
```

Token $87 *+ längd: hoppa framåt.* IF *lägger* $87 *före* THEN*-delen och* ELSE*-delen. När villkoret är falskt hoppar* IF *över* THEN*-delen. Kommer* THEN*-delen till* $87 *så hoppar den över* ELSE*-delen. In: DE vid längdbyten n. Ut: DE = längdbyten + n. Förstör HL.*

```
0DB8  EB          EXESKP: EX   DE,HL
0DB9  5E                  LD   E,(HL)          E = n
0DBA  16 00               LD   D,$00
0DBC  19                  ADD  HL,DE           framåt från längdbyten
0DBD  EB                  EX   DE,HL
0DBE  C9                  RET
```

NODIR*: fel 2 om satsen körs i direktläge (DIRECT <> 0). Anropas av* GOTO*,* GOSUB*,* RETURN*,* ON*,* ONERRG*,* STOP*,* DATA*,* DIM *och* DEF*, och av* LNKFOR*, så att* FOR *ger felet redan när raden länkas. Förstör A.*

```
0DBF  FD 7E 0E    NODIR:  LD   A,(IY+$0E)
0DC2  A7                  AND  A
0DC3  C8                  RET  Z               programläge: tillåtet
0DC4  D7                  RST  $10
0DC5  82                  DEFB $82             fel 2: OTILLÅTET SOM KOMMANDO
```

3.25 SAVE fil: spara programmet som BAC-fil $0DC6

Filen öppnas med OPEN typ 4. Bufferten fylls med $82 och sedan hela rader. När nästa rad inte får plats avslutas blocket med $00 och skrivs med PUT. Efter sista raden kommer $01 och en sista PUT.

Slutet (CLOSE) är gemensamt med LOAD på $0C0D.

FCB+$0D = antal lediga byte (CAS: 252). $82 står bara i första blocket; nästa block börjar där enhetens PUT lämnar DE. Enheten måste ge en buffert (FCB+8,+9), annars blir det fel 52. SAVE prövar inte CY från PUT, så ett fel märks bara om enheten själv ger det. En rad som inte får plats i ett tomt block ger tomma block utan slut. In: HL = namnet. Ut: NC och filen stängd.

```
                  CMDSAVE:
0DC6  DD 21 F1 FF         LD   IX,$FFF1        FCB om 15 byte på stacken (fel)
0DCA  DD 39               ADD  IX,SP
0DCC  DD F9               LD   SP,IX
0DCE  3E 04               LD   A,$04           OPEN typ 4 (skriv .BAC)
0DD0  CD C2 07            CALL OPNSTR
0DD3  DD 6E 08            LD   L,(IX+$08)
0DD6  DD 66 09            LD   H,(IX+$09)
0DD9  7C                  LD   A,H             H = 0: ingen buffert
0DDA  A7                  AND  A
0DDB  3E B4               LD   A,$B4           fel 52 om enheten saknar buffert
0DDD  28 32               JR   Z,L0E11
0DDF  36 82               LD   (HL),$82        BAC-huvud
0DE1  23                  INC  HL
0DE2  DD 4E 0D            LD   C,(IX+$0D)      C = plats i bufferten
0DE5  0D                  DEC  C               $82 tar en plats
0DE6  EB                  EX   DE,HL           DE = bufferten, efter $82
0DE7  2A 1C FE            LD   HL,(BOFA)       från första raden
0DEA  7E          L0DEA:  LD   A,(HL)          B = radens längd
0DEB  47                  LD   B,A
0DEC  EE 01               XOR  $01             $01: programmets slut
0DEE  28 1A               JR   Z,L0E0A         slutmarkering?
0DF0  79                  LD   A,C
0DF1  90                  SUB  B
0DF2  4F                  LD   C,A             C = platsen som blir kvar
0DF3  38 08               JR   C,L0DFD         får inte raden plats?
0DF5  7E          L0DF5:  LD   A,(HL)          kopiera raden, B byte
```

```
0DF6  23                INC  HL
0DF7  12                LD   (DE),A
0DF8  13                INC  DE
0DF9  10 FA             DJNZ L0DF5
0DFB  18 ED             JR   L0DEA

0DFD  E5          L0DFD: PUSH HL
0DFE  EB                EX   DE,HL          HL = bufferten
0DFF  36 00             LD   (HL),$00       $00 = blocket slut
0E01  CD 9C 08          CALL DVPUT          CY från PUT prövas inte
0E04  E1                POP  HL
0E05  DD 4E 0D          LD   C,(IX+$0D)     nytt block, DE från PUT
0E08  18 E0             JR   L0DEA

0E0A  EB          L0E0A: EX   DE,HL        HL = bufferten
0E0B  36 01             LD   (HL),$01       $01 = filslut
0E0D  CD 9C 08          CALL DVPUT
0E10  AF                XOR  A              A = 0: inget fel
0E11  C3 0D 0C    L0E11: JP   L0C0D
```

Satsernas körrutiner, index = token - `$80` (se `STKW1`). `$86` = byt till `STEXE2` och `$87` = hoppa framåt. `$95` är en rad som inte gick att tokenisera och ger fel 53 om den körs.

```
0E14  DB 1D       STEXE1: DEFW GOTO         $80 GOTO
0E16  DB 1D               DEFW GOTO         $81 GOTO efter THEN och ELSE
0E18  FD 1B               DEFW LET          $82 LET
0E1A  FD 1B               DEFW LET          $83 LET utan ordet
0E1C  9F 21               DEFW PRINTS       $84 PRINT
0E1E  9F 21               DEFW PRINTS       $85 ; (PRINT)
0E20  B3 0D               DEFW EXEST2       $86 sats i tabell 2
0E22  B8 0D               DEFW EXESKP       $87 hopp framåt
0E24  4B 1E               DEFW IF           $88 IF
0E26  78 1E               DEFW INPT         $89 INPUT
0E28  82 1F               DEFW INPLIN       $8A INPUT LINE
0E2A  C2 1C               DEFW FOR          $8B FOR
0E2C  21 1D               DEFW NEXT         $8C NEXT
0E2E  09 07               DEFW L0709        $8D fel 8
0E30  C5 1E               DEFW READ         $8E READ
0E32  86 22               DEFW RESTOR       $8F RESTORE
0E34  E9 1D               DEFW GOSUB        $90 GOSUB
0E36  B0 22               DEFW RETURN       $91 RETURN
0E38  09 20               DEFW ON           $92 ON
0E3A  73 22               DEFW DATA         $93 DATA
0E3C  41 20               DEFW ONERRG       $94 ON ERROR GOTO
0E3E  F7 06               DEFW L06F7        $95 felaktig rad: fel 53
0E40  AA 1A               DEFW DEF          $96 DEF FN
```

Körrutiner för tabell 2, index = kod i `STKW2`.

```
0E42  FA 1B       STEXE2: DEFW DIM          $80 DIM
0E44  44 21               DEFW POKE         $81 POKE
0E46  2A 21               DEFW OUTST        $82 OUT
0E48  09 07               DEFW L0709        $83 fel 8
0E4A  76 22               DEFW REM          $84 REM
0E4C  74 20               DEFW OPENST       $85 OPEN
0E4E  77 20               DEFW PREPST       $86 PREPARE
0E50  BF 20               DEFW CLOSST       $87 CLOSE
0E52  58 22               DEFW RANDOM       $88 RANDOMIZE
0E54  20 23               DEFW STOP         $89 STOP
0E56  C9 00               DEFW ENDPRG       $8A END
0E58  66 01               DEFW L0166        $8B CMD: tilläggs-ROM:en
0E5A  09 07               DEFW L0709        $8C fel 8
```

```
0E5C  09 07              DEFW L0709          $8D fel 8
0E5E  C7 22              DEFW SETDOT         $8E SETDOT
0E60  CF 22              DEFW CLRDOT         $8F CLRDOT
0E62  B1 1D              DEFW GET            $90 GET
0E64  3F 0D              DEFW CHAIN          $91 CHAIN
0E66  70 01              DEFW L0170          $92 KILL: tilläggs-ROM:en
0E68  75 01              DEFW L0175          $93 NAME: tilläggs-ROM:en
0E6A  25 23              DEFW TRACE          $94 TRACE
0E6C  2A 23              DEFW NOTRAC         $95 NOTRACE
```

3.26 PRELNK: förlänkning innan programmet körs $0E6E

RUN går igenom hela programmet en gång innan det startar.

- Radreferenser ($BF/$BE) får adressen till målraden (fel 6).
- Variabler i uttryckskoden (token $C0 och uppåt) får plats på heapen och får adressen till sitt värde (LNKEXP).
- FOR och NEXT paras ihop redan här. BUF1 används som pekare till innersta öppna FOR (LNKFOR/LNKNXT). En FOR utan NEXT ger fel 24, och CURLIN pekar in i FOR-raden (ERRMSG letar upp raden).
- DEF FN: antalet parametrar läggs i $FE42 och namnen från $FE43, så att de känns igen i funktionskroppen (LNKDEF).
- ON ERROR GOTO 0 får adressen 0.
- Token $8D ger fel 8. Den finns inte i STKW1 och kan bara komma från en fil eller POKE.

DIRECT sätts till 0 först, så att felen visar raden. Inget körs förrän hela programmet är länkat: ett fel i sista raden stoppar programmet innan första raden har körts.

LNKOK = 1 när allt har gått bra.

```
0E6E  2A 1C FE     PRELNK: LD   HL,(BOFA)
0E71  AF                   XOR  A
0E72  FD 77 0E             LD   (IY+$0E),A     programläge: felen visar raden
0E75  FD 77 2B             LD   (IY+$2B),A     FOR-kedjan tom
0E78  FD 77 2C             LD   (IY+$2C),A     inga DEF FN-parametrar
0E7B  7E           L0E7B:  LD   A,(HL)         längdbyten
0E7C  D6 01                SUB  $01            $01: programmets slut
0E7E  28 0B                JR   Z,L0E8B
0E80  22 2C FE             LD   (CURLIN),HL    för felen
0E83  23                   INC  HL             förbi längd och radnummer
0E84  23                   INC  HL
0E85  23                   INC  HL
0E86  CD AA 0E             CALL LNKELM         hela raden, till CR
0E89  18 F0                JR   L0E7B

0E8B  FD B6 2B     L0E8B:  OR   (IY+$2B)       A = 0; NZ: en FOR saknar NEXT
0E8E  20 05                JR   NZ,L0E95
0E90  FD 36 10 01          LD   (IY+$10),$01   förlänkat
0E94  C9                   RET

0E95  67           L0E95:  LD   H,A            HL = innersta FOR-blocket
0E96  FD 6E 2A             LD   L,(IY+$2A)
0E99  23                   INC  HL
0E9A  23                   INC  HL
0E9B  23                   INC  HL
0E9C  CD 1C 10             CALL RDDE1          DE = +4, slingans början
0E9F  ED 53 2C FE          LD   (CURLIN),DE    CURLIN = en adress i FOR-raden
0EA3  C3 15 07             JP   L0715          fel 24, "NEXT" SAKNAS
```

```
0EA6  CD FB 0E     L0EA6:  CALL SKPELM
0EA9  D8                   RET  C              CR: raden slut, tillbaka till anroparen
```

LNKELM: *länka ett element i satsen vid HL. Returadressen* $0EAA *läggs på stacken, så rutinerna för* $C0-, $8B *och* $8C *kommer tillbaka hit och tar nästa element. In: HL = elementet. Ut: HL efter radens CR.*

```
0EAA  7E           LNKELM: LD   A,(HL)         elementets token
0EAB  E5                   PUSH HL
0EAC  21 AA 0E             LD   HL,$0EAA       LNKELM som returadress
0EAF  E3                   EX   (SP),HL        HL orört, $0EAA vid SP
0EB0  FE C0                CP   $C0            uttryckskod
0EB2  D2 32 10             JP   NC,LNKEXP      RET går till LNKELM
0EB5  FE 8B                CP   $8B            FOR
0EB7  CA 7C 0F             JP   Z,LNKFOR
0EBA  FE 8C                CP   $8C            NEXT
0EBC  CA DB 0F             JP   Z,LNKNXT
0EBF  FE 8D                CP   $8D            $8D: fel 8
0EC1  CA 09 07             JP   Z,L0709
0EC4  E3                   EX   (SP),HL        ta bort returadressen
0EC5  E1                   POP  HL             HL på elementet igen
0EC6  06 01                LD   B,$01          en radreferens
0EC8  FE 94                CP   $94            ON ERROR GOTO
0ECA  28 25                JR   Z,L0EF1
0ECC  FE 96                CP   $96            DEF FN
0ECE  CA 59 0F             JP   Z,LNKDEF
0ED1  FE BF                CP   $BF            $BF: radreferens
0ED3  28 06                JR   Z,L0EDB
0ED5  FE BE                CP   $BE            $BE: B = antal
0ED7  20 CD                JR   NZ,L0EA6
0ED9  23                   INC  HL
0EDA  46                   LD   B,(HL)
0EDB  23           L0EDB:  INC  HL
0EDC  CD 1D 10     L0EDC:  CALL RDDE           DE = radnumret
0EDF  C5           L0EDF:  PUSH BC
0EE0  E5                   PUSH HL
0EE1  CD 39 0F             CALL FINDLN
0EE4  C2 05 07             JP   NZ,L0705       fel 6
0EE7  EB                   EX   DE,HL          DE = radens adress
0EE8  E1                   POP  HL
0EE9  C1                   POP  BC
0EEA  CD 23 10     L0EEA:  CALL WRDE           efter radnumret
0EED  10 ED                DJNZ L0EDC
0EEF  18 B9                JR   LNKELM

0EF1  23           L0EF1:  INC  HL
0EF2  CD 1C 10             CALL RDDE1          förbi $BF, DE = radnumret
0EF5  7B                   LD   A,E
0EF6  B2                   OR   D
0EF7  20 E6                JR   NZ,L0EDF
0EF9  18 EF                JR   L0EEA          GOTO 0: adress 0
```

Korsreferenstabell för REN, RUN, körningen, SAVE och förlänkningen. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är DIRECT vid IY+$0E, FLAGS vid IY+$0F (bit 4 = TRACE), LNKOK vid IY+$10, felkoden vid IY+$22, kedjan av öppna FOR vid IY+$2A,+$2B (i BUF1) och antalet DEF FN-parametrar vid IY+$2C. Det som en anropad rutin gör räknas med, men inte det som satserna gör: RUNSTM, EXESTM och EXEST2 hoppar till satsens rutin i STEXE1 eller STEXE2. CMDRUN gör CLEAR (tabellen efter $0C88) och fyller också slumptalsfröet $FE39–$FE3D med RNDINI; vid programmets slut går RUNLIN till ENDPRG. LNKELM och PRELNK lägger variablerna och FOR-blocken på heapen med ALLOC (fel 3) genom LNKEXP, LNKFOR och LNKNXT, och LNKDEF skriver

parametrarnas namn från $FE43. CMDSAVE öppnar en FCB på 15 byte på stacken med OPNSTR och tar bort den igen genom slutet av CMDLOAD ($0C0D). CHAIN, CMDRUN och RUNLIN går inte tillbaka; CMDRUN sätter SP till STKTOP och lägger $071B (fel 29) i botten av stacken. LNKELM lämnar alltid CY = 1, EXESKP och NODIR alltid CY = 0. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler											IY+					
Rutin	A	CY	BC	DE	HL	IX	stack	BOFA	EOFA	HEAP	VARLST	FILLST	STKTOP	CURLIN	STMSP	DATPTR	DATLIN	ONERR	BRKFLG	$0E	$0F	$10	$22	$2B	$2C
CMDREN	X	X	X	X	I	X		I						U						U					
LNKREF	X	X	X	X	X			I						U						U					
REFADR	X	X		IU				I																	
CHAIN*				I			U	I	IU	IU	IU	IU	I	U	U	U	U	U	IU	U	IU	U	U	U	U
CMDRUN*					I		U	I	IU	IU	IU	IU	I	U	U	U	U	U	IU	U	IU	U	U	U	U
RUNLIN*				I			I							U	U				IU		I				
RUNSTM	X	X	X	IU	X														IU						
EXESTM	X	X	X	IU	X																				
EXEST2	X	X	X	IU	X																				
EXESKP		X		IU	X																				
NODIR	X	X																		I					
CMDSAVE	X	X	X	X	I	X	X	I																	
PRELNK	X	X	X	X	X	X		I		IU	IU			U						U		U		U	U
LNKELM	X	U	X	X	IU	X		I		IU	IU									I				IU	IU

Rutin	**Anropar**	**Anropas från**
CMDREN	GETLN, LNKREF, SKPELM, RDDE1; fel 31	CMDTAB (REN)
LNKREF	SKPELM, RDDE, REFADR, WRDE	CMDREN
REFADR	FINDLN; fel 6	LNKREF
CHAIN	RST38, NEW; går vidare till CMDRUN	STEXE2 (CHAIN)
CMDRUN	CMDLOAD, NEWLIN, CLEAR, PRELNK, RSTDAT, RNDINI; går vidare till RUNLIN	CMDTAB (RUN); CHAIN går in i den
RUNLIN	TRCPRT, RUNSTM; vid slutet ENDPRG	CMDRUN (går in i den), ERROR (ON ERROR), GOTO, GOSUB
RUNSTM	TSTBRK, EXESTM; BREAK till BRKSTP	CMDLP, RUNLIN
EXESTM	satsens rutin ur STEXE1 (genom $284D)	RUNSTM
EXEST2	satsens rutin ur STEXE2	STEXE1 ($86)
EXESKP		STEXE1 ($87)
NODIR	fel 2	LNKFOR, DEF, DIM, GOTO, GOSUB, ON, ONERRG, DATA, RETURN, STOP
CMDSAVE	OPNSTR, DVPUT, CLOSE; fel 52 till ERRSTP	CMDTAB (SAVE)
PRELNK	LNKELM; fel 24	CMDRUN
LNKELM	LNKEXP, LNKFOR, LNKNXT, LNKDEF, SKPELM, RDDE, FINDLN, WRDE; fel 6, 8	PRELNK, CMDLP, LNKDEF

3.27 SKPELM: hoppa över ett element i tokeniserad text $0EFB

Visar hur satserna lagras:

```
$0D                 slut på raden (CY = 1)
$86 k               sats k i tabell 2
$87 n               hopp framåt n byte  (IF/ELSE)
$B9 a b             två byte  data
$BF nl nh al ah     radreferens: radnummer, sedan adress
$BE k + k*4 byte    lista med k radreferenser (ON ... GOTO)
$96 ..              DEF FN: 2 byte, antal, 2 byte/parameter
$C0-$FF             uttryckskod, hoppas över med SKPEXP
```

In: HL = elementet. Ut: HL efter elementet och CY = 0; vid CR CY = 1. För $96 stannar HL på =. Förstör A och B, och för uttryckskod också C och DE.

```
0EFB  7E          SKPELM: LD   A,(HL)
0EFC  23                  INC  HL              förbi token
0EFD  FE 0D               CP   $0D             CR: slut, CY = 1
0EFF  37                  SCF
0F00  C8                  RET  Z
0F01  FE 96               CP   $96
0F03  28 23               JR   Z,L0F28         DEF FN
0F05  FE C0               CP   $C0
0F07  30 28               JR   NC,L0F31        uttryckskod
0F09  06 01               LD   B,$01           B = 1: ett varv i kedjan
0F0B  FE 87               CP   $87             $87 och $86: en byte
0F0D  28 14               JR   Z,L0F23
0F0F  FE 86               CP   $86
0F11  28 10               JR   Z,L0F23
0F13  FE B9               CP   $B9             $B9: två byte
0F15  28 0B               JR   Z,L0F22
0F17  FE BF               CP   $BF             $BF: fyra byte
0F19  28 05               JR   Z,L0F20
0F1B  EE BE               XOR  $BE             $BE: B gånger fyra
0F1D  C0                  RET  NZ              annat: bara token, CY = 0
0F1E  46                  LD   B,(HL)          B = antalet radnummer
0F1F  23                  INC  HL              förbi antalet
0F20  23          L0F20:  INC  HL              fyra byte per varv
0F21  23                  INC  HL
0F22  23          L0F22:  INC  HL
0F23  23          L0F23:  INC  HL
0F24  10 FA               DJNZ L0F20           ett varv utom för $BE
0F26  A7                  AND  A               CY = 0: inte radens slut
0F27  C9                  RET

0F28  23          L0F28:  INC  HL              DEF FN: förbi namnbyten
0F29  23                  INC  HL              och bokstaven
0F2A  7E                  LD   A,(HL)          antalet parametrar
0F2B  87                  ADD  A,A             två byte per parameter
0F2C  23          L0F2C:  INC  HL              2n + 1 byte fram
0F2D  3D                  DEC  A
0F2E  F8                  RET  M               HL på =, CY = 0
0F2F  18 FB               JR   L0F2C

0F31  2B          L0F31:  DEC  HL              tillbaka till token
0F32  CD C3 10    L0F32:  CALL SKPEXP          ett element i taget
0F35  30 FB               JR   NC,L0F32        tills sluttoken
0F37  A7                  AND  A               CY = 0: inte radens slut
0F38  C9                  RET
```

FINDLN: *leta efter raden med nummer DE, från* BOFA *och med längdbyten från rad till rad. In: DE = numret. Ut: Z = funnen, HL = raden. NZ = saknas, och HL pekar då på första raden med högre nummer eller på slutets* $01*, där en ny rad med numret ska in. CY = 0 och BC = numret i båda fallen. Förstör A och DE. Anropas av* INSLIN, ED, LIST *och* REFADR.

```
0F39  2A 1C FE     FINDLN: LD   HL,(BOFA)
0F3C  42                   LD   B,D            BC = det sökta numret
0F3D  4B                   LD   C,E
0F3E  7E           L0F3E:  LD   A,(HL)         radens längdbyte
0F3F  FE 01                CP   $01            $01: programmets slut, NZ
0F41  28 14                JR   Z,L0F57
0F43  23                   INC  HL
0F44  5E                   LD   E,(HL)         DE = radens nummer
0F45  23                   INC  HL
0F46  56                   LD   D,(HL)
0F47  2B                   DEC  HL
0F48  2B                   DEC  HL             HL på längdbyten igen
0F49  EB                   EX   DE,HL
0F4A  A7                   AND  A
0F4B  ED 42                SBC  HL,BC          radens nummer - det sökta
0F4D  EB                   EX   DE,HL          DE = 0 om lika
0F4E  C8                   RET  Z              Z: hittad
0F4F  30 06                JR   NC,L0F57       större: NZ
0F51  5F                   LD   E,A            DE = längden
0F52  16 00                LD   D,$00
0F54  19                   ADD  HL,DE          nästa rad
0F55  18 E7                JR   L0F3E

0F57  B7           L0F57:  OR   A              NZ, CY = 0
0F58  C9                   RET
```

LNKDEF: DEF FN, *nådd från* LNKELM *utan dess returadress. Antalet parametrar läggs i* $FE42 (*IY*+$2C) *och namnen, två byte var, från* $FE43 (*IY*+$2D). *Sedan länkas resten av raden med* LNKELM*, så att parametrarna känns igen i uttrycket* (LNKPAR)*, och* $FE42 *nollas. In: HL =* $96*. Ut: HL efter radens CR. Förstör A, BC, DE och IX.*

```
0F59  23           LNKDEF: INC  HL              förbi $96
0F5A  23                   INC  HL              förbi namnbyten
0F5B  23                   INC  HL              och bokstaven
0F5C  46                   LD   B,(HL)          B = antal parametrar
0F5D  23                   INC  HL
0F5E  FD 70 2C             LD   (IY+$2C),B      $FE42 = antalet
0F61  78                   LD   A,B
0F62  A7                   AND  A
0F63  28 0F                JR   Z,L0F74         inga parametrar
0F65  FD E5                PUSH IY              IY som pekare
0F67  CB 10                RL   B               B = 2n (CY = 0 efter AND A)
0F69  7E           L0F69:  LD   A,(HL)
0F6A  23                   INC  HL
0F6B  FD 77 2D             LD   (IY+$2D),A      namnbyte och bokstav, från $FE43
0F6E  FD 23                INC  IY
0F70  10 F7                DJNZ L0F69
0F72  FD E1                POP  IY              HL på =
0F74  CD AA 0E     L0F74:  CALL LNKELM          uttrycket, HL efter radens CR
0F77  FD 36 2C 00          LD   (IY+$2C),$00    $FE42 = 0: inga parametrar
0F7B  C9                   RET                  tillbaka till anroparen av LNKELM
```

LNKFOR: FOR *v* = *a* TO *b* [STEP *c*]. *Fel 2 i direktläge. Ett block reserveras på heapen: 16 byte för flyttalsvariabel, 10 byte för heltal (token* $C0 = *flyttal). Blocket länkas in först i* FOR*-kedjan* (BUF1)*, och dess adress skrivs efter* $B9 *i* FOR*-satsen. Blocket:*

```
+0  länken i FOR-kedjan, sedan adressen efter NEXT (LNKNXT)
+2  variabelns adress
+4  slingans början = efter FOR-satsens $B9-fält
+6  gränsen (TO) och steget: heltal 2 + 2 byte, flyttal 5 + 5
```

Allt utom gränsen och steget skrivs här; steget får värdet 1 (1,0 = $10 ... $81), och FOR skriver resten vid körning. Ut: tillbaka till $0EAA med HL efter satsen.

```
0F7C  CD BF 0D     LNKFOR: CALL NODIR          fel 2 i direktläge
0F7F  23                   INC  HL             HL på variabeln
0F80  E5                   PUSH HL             variabelns token vid SP
0F81  CD 32 10             CALL LNKEXP         v = a, till och med $B6+t
0F84  E3                   EX   (SP),HL        HL = variabelns token, vid SP: $BA
0F85  7E                   LD   A,(HL)         $C0 flyttal, $C1 heltal
0F86  23                   INC  HL
0F87  23                   INC  HL
0F88  23                   INC  HL             förbi namnbyten och bokstaven
0F89  4E                   LD   C,(HL)         BC = variabelns adress
0F8A  23                   INC  HL
0F8B  46                   LD   B,(HL)
0F8C  11 10 00             LD   DE,$0010
0F8F  FE C0                CP   $C0            flyttal: 16 byte
0F91  28 0F                JR   Z,L0FA2
0F93  11 0A 00             LD   DE,$000A       heltal: 10 byte
0F96  CD 74 33             CALL ALLOC          HL = blocket, nollställt
0F99  EB                   EX   DE,HL          DE = blocket
0F9A  19                   ADD  HL,DE          HL = blockets slut
0F9B  2B                   DEC  HL
0F9C  2B                   DEC  HL
0F9D  36 01                LD   (HL),$01       +8: steget 1
0F9F  EB                   EX   DE,HL          HL = blocket
0FA0  18 0F                JR   L0FB1

0FA2  CD 74 33     L0FA2:  CALL ALLOC
0FA5  EB                   EX   DE,HL
0FA6  19                   ADD  HL,DE          HL = blockets slut
0FA7  2B                   DEC  HL
0FA8  36 81                LD   (HL),$81       +15: exponenten $81
0FAA  21 0B 00             LD   HL,$000B
0FAD  19                   ADD  HL,DE          HL = blocket + 11
0FAE  36 10                LD   (HL),$10       $10 00 00 00 $81 = 1,0
0FB0  EB                   EX   DE,HL          HL = blocket
0FB1  ED 5B 40 FE  L0FB1:  LD   DE,(BUF1)      DE = förra innersta FOR
0FB5  22 40 FE             LD   (BUF1),HL      blocket först i kedjan
0FB8  E5                   PUSH HL             blocket vid SP
0FB9  CD 23 10             CALL WRDE           +0 länken, HL = +2
0FBC  CD 29 10             CALL WRBC           +2 variabeln, HL = +4
0FBF  C1                   POP  BC             BC = blocket
0FC0  E3                   EX   (SP),HL        HL = $BA, blocket + 4 vid SP
0FC1  C5                   PUSH BC             blocket vid SP
0FC2  23                   INC  HL             förbi $BA (TO)
0FC3  CD 32 10             CALL LNKEXP         gränsen, till och med $B9+t
0FC6  7E                   LD   A,(HL)         STEP eller satsens $B9
0FC7  FE BB                CP   $BB            STEP?
0FC9  20 04                JR   NZ,L0FCF
0FCB  23                   INC  HL             förbi $BB
0FCC  CD 32 10             CALL LNKEXP         steget, till och med $B9+t
0FCF  C1           L0FCF:  POP  BC             BC = blocket
0FD0  D1                   POP  DE             DE = blocket + 4
0FD1  23                   INC  HL             HL på fältet efter $B9
0FD2  CD 29 10             CALL WRBC           blockets adress i FOR, HL efter satsen
0FD5  EB                   EX   DE,HL          HL = blocket + 4, DE = efter satsen
```

```
0FD6  73              LD   (HL),E        +4 = slingans början
0FD7  23              INC  HL
0FD8  72              LD   (HL),D
0FD9  EB              EX   DE,HL         HL efter satsen
0FDA  C9              RET                till LNKELM ($0EAA)
```

LNKNXT: NEXT [v]. Plockar innersta FOR från kedjan (fel 27 om den är tom). Hör NEXT till en variabel ska den vara samma som i FOR (fel 28). NEXT och FOR får varandras adresser. NEXT utan variabel jämförs med 0 och ger därför alltid fel 28 (samma i 9913). I direktläge är BUF1 den inskrivna raden, så NEXT tar två tecken ur texten som blockets adress och skriver dit; utan minne där blir det fel 28. Ut: tillbaka till $0EAA med HL efter satsen.

```
0FDB  23        LNKNXT: INC  HL           efter NEXT (fel)
0FDC  7E                LD   A,(HL)       variabel eller $B9
0FDD  11 00 00          LD   DE,$0000     utan variabel: 0
0FE0  FE C0             CP   $C0
0FE2  38 0B             JR   C,L0FEF      ingen variabel
0FE4  E5                PUSH HL           variabelns token vid SP
0FE5  CD 32 10          CALL LNKEXP       v $BC+t, HL på $B9
0FE8  E3                EX   (SP),HL      HL = variabelns token
0FE9  23                INC  HL           förbi namnbyten
0FEA  23                INC  HL
0FEB  CD 1C 10          CALL RDDE1        förbi bokstaven, DE = variabelns adress
0FEE  E1                POP  HL           HL på $B9
0FEF  D5        L0FEF:  PUSH DE           variabeln (eller 0) vid SP
0FF0  ED 4B 40 FE       LD   BC,(BUF1)    BC = innersta FOR
0FF4  78                LD   A,B          bara höga byten prövas
0FF5  A7                AND  A
0FF6  CA 19 07          JP   Z,L0719      fel 27, "NEXT" UTAN "FOR"
0FF9  CD 28 10          CALL WRBC1        förbi $B9, HL efter satsen
0FFC  0A                LD   A,(BC)       +0: länken till yttre FOR
0FFD  FD 77 2A          LD   (IY+$2A),A   BUF1 = länken: kedjan ett steg bakåt
1000  7D                LD   A,L
1001  02                LD   (BC),A       +0 = efter NEXT
1002  03                INC  BC
1003  0A                LD   A,(BC)
1004  FD 77 2B          LD   (IY+$2B),A   BUF1:s höga byte
1007  7C                LD   A,H
1008  02                LD   (BC),A
1009  03                INC  BC           BC = blocket + 2
100A  D1                POP  DE           DE = variabeln i NEXT
100B  0A                LD   A,(BC)       +2: variabeln i FOR
100C  03                INC  BC
100D  AB                XOR  E            samma variabel? 0 aldrig
100E  5F                LD   E,A
100F  0A                LD   A,(BC)
1010  AA                XOR  D
1011  B3                OR   E
1012  C2 17 07          JP   NZ,L0717     fel 28, FEL VARIABEL EFTER "NEXT"
1015  C9                RET               till LNKELM ($0EAA)
```

Hjälprutiner för ett ord vid HL: RDBC och RDDE läser till BC och DE, WRBC och WRDE skriver. Varianterna med 1 i namnet hoppar först över en byte, oftast en token. Ut: HL efter ordet. Inget annat ändras.

```
1016  23        RDBC1:  INC  HL
1017  4E        RDBC:   LD   C,(HL)
1018  23                INC  HL
1019  46                LD   B,(HL)
101A  23                INC  HL
101B  C9                RET
```

```
101C  23          RDDE1:  INC  HL
101D  5E          RDDE:   LD   E,(HL)
101E  23                  INC  HL
101F  56                  LD   D,(HL)
1020  23                  INC  HL
1021  C9                  RET

1022  23          WRDE1:  INC  HL
1023  73          WRDE:   LD   (HL),E
1024  23                  INC  HL
1025  72                  LD   (HL),D
1026  23                  INC  HL
1027  C9                  RET

1028  23          WRBC1:  INC  HL
1029  71          WRBC:   LD   (HL),C
102A  23                  INC  HL
102B  70                  LD   (HL),B
102C  23                  INC  HL
102D  C9                  RET
```

LNKSKP: *elementet är ingen variabel.* SKPEXP *hoppar över det, och en sluttoken ger CY och tillbaka till anroparen av* LNKEXP.

```
102E  CD C3 10    LNKSKP: CALL SKPEXP
1031  D8                  RET  C
```

LNKEXP: *länka uttryckskoden vid HL, element för element. Ett funktionsanrop* ($CD–$D1) *går till* LNKFN. *En variabel känns igen i* LNKVAR *på att token finns bland de nio första paren i* TOKLEN. *Inne i ett* DEF FN *jämförs den först med parametrarna från* $FE43 (LNKPAR)*; en träff får adressen* $00nn, *där nn är antalet parametrar efter den här, och* FNPAR *tolkar den vid körningen. Annars söker* LNKLST *i variabellistan från* VARLST, *och* LNKNEW *lägger till en post om variabeln saknas. Adressen till värdet skrivs i de två byte efter variabelns namn. Allt annat hoppar* LNKSKP *över. In: HL = koden. Ut: HL efter sluttoken, CY = 1. Förstör A, BC, DE och IX.*

```
1032  7E          LNKEXP: LD   A,(HL)          token
1033  FE CD               CP   $CD             funktionsanrop?
1035  38 05               JR   C,LNKVAR
1037  FE D2               CP   $D2             $CD–$D1: funktionsanrop
1039  DA 1D 11            JP   C,LNKFN
103C  EB          LNKVAR: EX   DE,HL
103D  21 05 11            LD   HL,TOKLEN       de nio variabeltoken
1040  01 12 00            LD   BC,$0012        9 par, 18 byte
1043  ED B1               CPIR                 token bland dem?
1045  EB                  EX   DE,HL
1046  20 E6               JR   NZ,LNKSKP       nej: hoppa över elementet
1048  CD 16 10            CALL RDBC1           C = namnbyten, B = bokstaven, HL = adressfältet
104B  FD 5E 2C            LD   E,(IY+$2C)      E = antalet DEF-parametrar
104E  FD E5               PUSH IY              IY som pekare i parametrarna
1050  1D          LNKPAR: DEC  E               en parameter till?
1051  FA 6B 10            JP   M,LNKLST        inga fler: sök i listan
1054  FD 7E 2D            LD   A,(IY+$2D)      parameterns namnbyte
1057  A9                  XOR  C
1058  57                  LD   D,A             D = 0 om lika
1059  FD 23               INC  IY
105B  FD 7E 2D            LD   A,(IY+$2D)      parameterns bokstav
105E  FD 23               INC  IY
1060  A8                  XOR  B
1061  B2                  OR   D               0 om båda lika
1062  20 EC               JR   NZ,LNKPAR       nästa parameter
1064  FD E1               POP  IY              IY tillbaka
1066  CD 23 10            CALL WRDE            DE = $00nn, nn = parametrar efter den här
```

```
1069  18 C7              JR   LNKEXP          nästa element
```

LNKLST: *sök i variabellistan. Varje post:*

```
+0 namnbyte  +1 bokstav  +2/3 länk till nästa post (0 = sist)
+4 värdet (flyttal 5 byte, heltal 2, sträng- eller vektorpost)
```

Namnbyten (bit 4-7 siffran, bit 2-3 antal index, bit 0-1 typen) skiljer A, A%, A$, A(..) och A(..,..) åt. Samma namn med ett index och med två ger fel 1 (bara bit 2-3 skiljer: XOR = $0C*); A och A(1), eller A$ och A$(1), går bra.*

Adressen som skrivs in i koden är +4, för flyttal utan index +8 (exponenten).

```
106B  FD E1        LNKLST: POP  IY             IY tillbaka
106D  FD E5                PUSH IY             och dess värde vid SP
106F  E3                   EX   (SP),HL        HL = $FE16, koden vid SP
1070  11 12 00             LD   DE,$0012       $FE16 + $12 = $FE28
1073  19                   ADD  HL,DE
1074  23           LNKSRC: INC  HL             länken, första gången VARLST
1075  5E                   LD   E,(HL)
1076  23                   INC  HL
1077  56                   LD   D,(HL)
1078  7B                   LD   A,E            0: sist i listan
1079  B2                   OR   D
107A  28 12                JR   Z,LNKNEW       HL på den sista länkens höga byte
107C  EB                   EX   DE,HL          HL = posten
107D  5E                   LD   E,(HL)         E = namnbyten
107E  23                   INC  HL
107F  7E                   LD   A,(HL)         A = bokstaven
1080  A8                   XOR  B              B = bokstaven i koden
1081  20 F1                JR   NZ,LNKSRC      annan bokstav: HL på +1, nästa länk
1083  7B                   LD   A,E            namnbyten
1084  A9                   XOR  C              C = namnbyten i koden
1085  28 1D                JR   Z,LNKADR       funnen, HL på +1
1087  FE 0C                CP   $0C            bara antalet index skiljer (01 mot 10)?
1089  20 E9                JR   NZ,LNKSRC      nej: nästa post
108B  C3 FB 06             JP   L06FB          fel 1
```

LNKNEW: *variabeln finns inte. Storleken ur* VARSIZ *(namnbyten AND* $0F*), en nollställd plats på heapen från* ALLOC *(fel 3 om minnet inte räcker), och posten länkas in sist i listan. Sedan samma väg som en funnen post:* LNKADR *räknar fram värdets adress, +4, eller +8 för ett flyttal utan index (exponenten), och* LNKWR *skriver den i koden.*

```
108E  E5           LNKNEW: PUSH HL             sista länkens höga byte vid SP
108F  79                   LD   A,C            namnbyte AND $0F
1090  E6 0F                AND  $0F
1092  5F                   LD   E,A            DE = index i VARSIZ (D = 0: länken var 0)
1093  21 B8 10             LD   HL,VARSIZ
1096  19                   ADD  HL,DE
1097  5E                   LD   E,(HL)         E = postens storlek
1098  CD 74 33             CALL ALLOC          HL = posten, nollställd
109B  EB                   EX   DE,HL          DE = posten
109C  E1                   POP  HL             HL = sista länkens höga byte
109D  72                   LD   (HL),D         länka in posten sist
109E  2B                   DEC  HL
109F  73                   LD   (HL),E
10A0  EB                   EX   DE,HL          HL = posten
10A1  71                   LD   (HL),C         +0 namnbyten
10A2  23                   INC  HL
10A3  70                   LD   (HL),B         +1 bokstaven
10A4  23           LNKADR: INC  HL             från +1 till +4
10A5  23                   INC  HL
```

```
10A6  23                  INC  HL
10A7  79                  LD   A,C             flyttal utan index?
10A8  E6 0F               AND  $0F
10AA  20 04               JR   NZ,LNKWR        nej: värdet på +4
10AC  11 04 00            LD   DE,$0004        flyttal: +8, exponenten
10AF  19                  ADD  HL,DE
10B0  EB            LNKWR: EX   DE,HL          DE = värdets adress
10B1  E1                  POP  HL              HL = adressfältet i koden
10B2  CD 23 10            CALL WRDE            skriv in adressen
10B5  C3 32 10            JP   LNKEXP          nästa element
```

VARSIZ: *postens storlek för namnbyte* AND $0F *(4 byte huvud + värdet): flyttal 9, heltal 6, sträng 10 (strängpost 6 byte), med ett index 10 (vektorpost 6 byte), med två index 14. Typ 3 används inte.*

```
10B8  09 06 0A 00… VARSIZ: DEFB $09,$06,$0A,$00,$0A,$0A,$0A,$0A
                                               storlek per variabeltyp
10C0  0E 0E 0E             DEFB $0E,$0E,$0E    med två index: flyttal, heltal, sträng
```

SKPEXP: *hoppa över ett element i uttryckskoden. In: HL = token. Ut: HL efter elementet; CY = 1 om det var en sluttoken (*NODATA*), som avslutar uttrycket. Strängkonstanter (*SKPSTR*) och funktionsanrop (*SKPFN*) har egen längd, de andra token med data står i* TOKLEN*, och resten är en byte. Förstör A, BC och DE.*

```
10C3  7E            SKPEXP: LD   A,(HL)          token
10C4  23                    INC  HL              förbi token
10C5  01 0A 00               LD   BC,$000A        10 token
10C8  EB                     EX   DE,HL           DE = efter token
10C9  21 FB 10               LD   HL,NODATA       de tio sluttoken
10CC  ED B1                  CPIR                 bland dem?
10CE  37                     SCF                  CY = 1: uttrycket slut
10CF  28 17                  JR   Z,SKPRET        ja
10D1  FE CB                  CP   $CB             $CB: strängkonstant
10D3  28 1F                  JR   Z,SKPSTR
10D5  FE CD                  CP   $CD             funktionsanrop?
10D7  38 04                  JR   C,SKPTOK
10D9  FE D2                  CP   $D2             $CD–$D1
10DB  38 0D                  JR   C,SKPFN
10DD  21 05 11      SKPTOK:  LD   HL,TOKLEN       token med data?
10E0  0E 18                  LD   C,$18           12 par, B = 0
10E2  ED B1                  CPIR
10E4  4E                     LD   C,(HL)          C = antalet byte
10E5  28 11                  JR   Z,SKPLEN        hoppa över dem
10E7  A7                     AND  A               token utan data: CY = 0
10E8  EB            SKPRET:  EX   DE,HL           HL efter elementet
10E9  C9                     RET
```

SKPFN: *funktionsanrop* $CD–$D1. *Efter token står 2 × funktionens kod; en inbyggd funktion har inget mer.* FN *(kod 0) har fem byte till: namnbyten, bokstaven, adressen till = och avståndet till den sista argumentposten. XOR ger CY = 0.*

```
10EA  EB            SKPFN:   EX   DE,HL          HL = funktionens kod
10EB  7E                     LD   A,(HL)          2 × koden, 0 för FN
10EC  23                     INC  HL
10ED  EE 00                  XOR  $00             Z om FN, CY = 0
10EF  C0                     RET  NZ              inbyggd: klar
10F0  0E 05                  LD   C,$05           FN: namnbyte, bokstav, adress, avstånd
10F2  18 05                  JR   SKPADD
```

SKPSTR: $CB, *citattecknet, längden och texten. DE pekar efter* $CB.

```
10F4  13            SKPSTR:  INC  DE              förbi citattecknet
```

```
10F5  1A              LD   A,(DE)        längden
10F6  4F              LD   C,A
10F7  13              INC  DE            förbi längden
10F8  EB     SKPLEN:  EX   DE,HL         HL efter längden
10F9  09     SKPADD:  ADD  HL,BC         B = 0, HL efter datat
10FA  C9              RET
```

NODATA: *de tio sluttoken* $B6–$BF. *De avslutar ett uttryck och ger CY i* SKPEXP; $B6–$B8 *är tilldelningen,* $B9+*t lämnar ett värde av typen t.*

```
10FB  B6 B7 B8 B9… NODATA: DEFB $B6,$B7,$B8,$B9,$BA,$BB,$BC,$BD
                                          token utan data
1103  BE BF               DEFB $BE,$BF
```

TOKLEN: *token och antalet byte efter den. De nio första paren är variablerna* ($C0 + 4 × *antal index + typ: namnbyte, bokstav och adress*)*, och* LNKEXP *söker bara bland dem. Sedan argumentposten* $DB (4)*, flyttalskonstanten* $C3 (5) *och heltalskonstanten* $C7 (2). SKPEXP *söker i alla tolv.*

```
1105  C0 04 C4 04… TOKLEN: DEFB $C0,$04,$C4,$04,$C8,$04,$C1,$04
                                          token + längd
110D  C5 04 C9 04…        DEFB $C5,$04,$C9,$04,$C2,$04,$C6,$04
                                          heltal med ett och två index; sträng och sträng med ett
1115  CA 04 DB 04…        DEFB $CA,$04,$DB,$04,$C3,$05,$C7,$02
                                          sträng med två index; argumentposten, flyttalet,
                                          heltalet
```

LNKFN: *länka ett funktionsanrop* $CD+*k, kod. Inbyggda funktioner (kod <> 0) har inget att länka. För* FN *(kod 0) söks programmet från* BOFA *efter en rad med* $96 *och samma namn (fel 57, FUNKTIONEN EJ DEFINIERAD, om den saknas). Adressen till* DEF*-radens = (radens början + 7 + 2 * antal parametrar) skrivs in i anropet.*

Sedan gås argumentposterna igenom bakåt från anropet, och samtidigt parameternamnen bakåt från = (DE). I varje post skrivs parameterns typ (bit 0-1 i namnbyten), och TYPCNV *skriver över* $D3 *med* $D9 *eller* $DA *om argumentet måste omvandlas. Går typerna inte ihop blir det fel 13 (tal till strängparameter) eller 17 (sträng till talparameter). Olika antal argument och parametrar ger fel 13.*

```
111D  23           LNKFN:  INC  HL          förbi $CD+n
111E  7E                   LD   A,(HL)      funktionens kod
111F  23                   INC  HL          förbi koden
1120  FE 00                 CP   $00         kod 0 = FN?
1122  C2 32 10              JP   NZ,LNKEXP   inbyggd: inget att länka
1125  CD 17 10              CALL RDBC        C = namnbyten, B = bokstaven, HL = adressfältet
1128  DD 2A 1C FE          LD   IX,(BOFA)   IX = raden
112C  DD 7E 00     L112C:  LD   A,(IX+$00)
112F  FE 01                 CP   $01         programmets slut: fel 57
1131  CA 29 07              JP   Z,L0729
1134  5F                    LD   E,A         E = radens längd
1135  DD 7E 03              LD   A,(IX+$03)  +3: radens första element
1138  EE 96                 XOR  $96         $96?
113A  57                    LD   D,A
113B  DD 7E 04              LD   A,(IX+$04)  +4: namnbyten
113E  A9                    XOR  C
113F  B2                    OR   D
1140  57                    LD   D,A
1141  DD 7E 05              LD   A,(IX+$05)  +5: bokstaven
1144  A8                    XOR  B
1145  B2                    OR   D
1146  28 06                 JR   Z,L114E     funnen
1148  16 00                 LD   D,$00       DE = längden
114A  DD 19                 ADD  IX,DE       nästa rad
114C  18 DE                 JR   L112C
```

```
114E  DD 4E 06     L114E:  LD   C,(IX+$06)      +6: antalet parametrar
1151  3E 07               LD   A,$07           7 + 2n
1153  81                  ADD  A,C
1154  81                  ADD  A,C
1155  4F                  LD   C,A
1156  42                  LD   B,D             BC = 7 + 2n (D = 0)
1157  EB                  EX   DE,HL           DE = adressfältet i anropet
1158  DD E5               PUSH IX
115A  E1                  POP  HL
115B  09                  ADD  HL,BC           HL = "=" på DEF-raden
115C  EB                  EX   DE,HL           HL = adressfältet, DE = "="
115D  CD 23 10            CALL WRDE            adressen till = i anropet
1160  DD 46 06            LD   B,(IX+$06)      B = parametrar kvar
1163  23                  INC  HL              förbi avståndsbyten
1164  E5                  PUSH HL              efter anropet, vid SP
1165  2B                  DEC  HL              HL på avståndsbyten
1166  7E          L1166:  LD   A,(HL)          avståndet bakåt, negativt
1167  A7                  AND  A
1168  28 20               JR   Z,L118A         0: alla poster gjorda
116A  05                  DEC  B               en parameter till
116B  1B                  DEC  DE              förra parametern
116C  1B                  DEC  DE              DE på dess namnbyte
116D  85                  ADD  A,L             HL + A, A negativt
116E  6F                  LD   L,A
116F  38 01               JR   C,L1172         minnessiffra: H oförändrad
1171  25                  DEC  H               annars H - 1
1172  E5          L1172:  PUSH HL              posten vid SP
1173  C5                  PUSH BC              parametrarna kvar vid SP
1174  23                  INC  HL              +2: argumentets typ
1175  46                  LD   B,(HL)          B = argumentets typ
1176  23                  INC  HL              +3
1177  1A                  LD   A,(DE)          parameterns namnbyte
1178  E6 03               AND  $03             typen, bit 0–1
117A  77                  LD   (HL),A          i posten
117B  23                  INC  HL              +4
117C  36 D3               LD   (HL),$D3        $D3: ingen omvandling
117E  EB                  EX   DE,HL           DE = +4, HL = parametern
117F  CD 51 25            CALL TYPCNV          $D9 eller $DA vid DE om det behövs
1182  DA 2E 06            JP   C,ERROR         fel 13 eller 17
1185  EB                  EX   DE,HL           DE = parametern
1186  C1                  POP  BC              B = parametrar kvar
1187  E1                  POP  HL              HL = postens avståndsbyte
1188  18 DC               JR   L1166           nästa post

118A  78          L118A:  LD   A,B             alla parametrar använda?
118B  A7                  AND  A
118C  3E 8D               LD   A,$8D           fel 13: antalet stämmer inte
118E  C2 2E 06            JP   NZ,ERROR
1191  E1                  POP  HL              HL efter anropet
1192  C3 32 10            JP   LNKEXP          nästa element
```

Korsreferenstabell för länkningen av satserna och uttrycken. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är DIRECT vid IY+$0E, kedjan av öppna FOR vid IY+$2A,+$2B (BUF1), antalet DEF FN-parametrar vid IY+$2C och parametrarnas namn från IY+$2D ($FE43). CY-kolumnen gäller flaggorna: FINDLN svarar med Z (funnen) och SKPELM och SKPEXP med CY (radens eller uttryckets slut). Det som en anropad rutin gör räknas med: LNKEXP och LNKFN läser BOFA när ett FN söks, och ALLOC flyttar HEAP och ger fel 3 när minnet inte räcker. LNKFOR och LNKNXT nås med JP från LNKELM, som har lagt $0EAA vid SP, så deras RET går dit; LNKDEF nås sedan returadressen tagits bort och går tillbaka till anroparen av LNKELM. LNKFN

nås med JP från LNKEXP och går vidare dit, och LNKEXP:s ingångar LNKSKP, LNKVAR, LNKPAR, LNKLST, LNKNEW, LNKADR och LNKWR har inga egna rader. Tabellerna VARSIZ, NODATA och TOKLEN läses med CPIR eller som index. Felen: LNKEXP 1 och 3, LNKFN 13, 17 och 57, LNKFOR 2 och 3, LNKNXT 27 och 28. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler			IY+			
Rutin	A	CY	BC	DE	HL	IX	stack	BOFA	HEAP	VARLST	$0E	$2B	$2C	$2D
SKPELM	X	U	X	X	IU									
FINDLN	X	U	U	IX	U			I						
LNKDEF	X	U	X	X	IU	X		I	IU	IU	I	IU	U	U
LNKFOR	X	X	X	X	IU	X		I	IU	IU	I	IU	I	I
LNKNXT	X	X	X	X	IU	X		I	IU	IU		IU	I	I
RDBC, RDBC1			U		IU									
RDDE, RDDE1				U	IU									
WRBC, WRBC1			I		IU									
WRDE, WRDE1				I	IU									
LNKEXP	X	U	X	X	IU	X		I	IU	IU			I	I
SKPEXP	X	U	X	X	IU									
LNKFN	X	U	X	X	IU	X		I	IU	IU			I	I

Rutin	**Anropar**	**Anropas från**
SKPELM	SKPEXP	CMDREN, LNKREF, PRELNK
FINDLN		INSLIN, CMDED, CMDLIST, REFADR, LNKELM
LNKDEF	LNKELM	LNKELM ($96, med JP)
LNKFOR	NODIR, LNKEXP, ALLOC, WRDE, WRBC	LNKELM ($8B, med JP)
LNKNXT	LNKEXP, RDDE1, WRBC1; fel 27, 28	LNKELM ($8C, med JP)
RDBC, RDBC1		RDBC1: LNKEXP (LNKVAR), FOR; RDBC: LNKFN, DETEXP, PREPST, ASCARG, ARGWLK
RDDE, RDDE1		RDDE1: CMDREN, PRELNK, LNKELM, LNKNXT, FN, DIMELM, GOTO, GOSUB, ON, ONERRG, RESTOR, VAR0, VARIDX; RDDE: OPEN, LNKREF, LNKELM, DETOK (två ställen), PREPST (två ställen), CONSTI
WRBC, WRBC1		WRBC1: LNKNXT, CMPREF, CMPNUM; WRBC: LNKFOR (två ställen), CMPDEF (två ställen), CMPFN
WRDE, WRDE1		WRDE1: TOKLIN; WRDE: LNKREF, LNKELM, LNKFOR, LNKEXP (LNKPAR, LNKWR), LNKFN, CMPON
LNKEXP	SKPEXP, RDBC1, ALLOC, WRDE; LNKFN; fel 1	LNKELM (med JP), LNKFOR (tre ställen), LNKNXT; LNKFN går tillbaka in i den
SKPEXP		SKPELM, LNKEXP (LNKSKP)
LNKFN	RDBC, WRDE, TYPCNV; fel 13, 17 och 57 till ERROR; går vidare till LNKEXP	LNKEXP ($CD–$D1)

3.28 DETOK: gör om en programrad till text (för LIST och ED) $1195

In: HL = radens längdbyte, DE = buffert för texten. Ut: HL och DE efter CR. Förstör A, BC och IX.

Ut: texten `nnn sats...` slutar med `$0D`; DE pekar efter `$0D`. Radnumret skrivs med PRNUM. Sedan går slingan igenom raden:

```
':'       blir " : " (mellanslag före och efter)
$87 n     hopp i IF/ELSE; skrivs inte ut
$B9 x x   FOR-post (se FOR); skrivs inte ut
$BF n n a a    en radreferens: numret skrivs
$BE k ...      k radreferenser, skrivs med komma emellan
$96       DEF FN: "DEFFN", namnet och (param,param,...)
$80-$BF   nyckelord: texten hämtas ur STKW1 (eller ur STKW2 om
          token är $86, då nästa byte är token i STKW2);
          mellanslag före och efter
$C0-$FF   uttryckskod: DETEXP skriver om den till infix
$00-$7F   vanligt tecken, kopieras
$0D       radslut, kopieras, klart
```

Mellanslagen skrivs av SPACE, som inte skriver ett mellanslag till om det senaste tecknet redan är ett. Mellanslag mellan uttryckets delar sparas inte, så `PRINT A; B` listas `PRINT A;B` och `A=1:B=2` listas `A=1 : B=2`. Vanliga tecken (`$00`–`$7F`) kopieras däremot som de står: REM-texten (utan de inledande blanka, som inte sparas) och strängarna behåller sina mellanslag. Ett `:` i REM-texten får ändå mellanslag runt sig, så `REM A:B` listas `REM A : B`. `DEF FN` listas `DEFFNA(P,Q)=...` utan mellanslag.

```
1195  23           DETOK:  INC  HL          förbi längdbyten
1196  D5                   PUSH DE          bufferten vid SP
1197  CD 1D 10             CALL RDDE        DE = radnumret, HL efter det
119A  E3                   EX   (SP),HL     HL = bufferten, koden vid SP
119B  EB                   EX   DE,HL       HL = numret, DE = bufferten
119C  CD 62 18             CALL PRNUM       radnumret som text
119F  E1             L119F: POP  HL          HL = koden
11A0  CD EA 11       L11A0: CALL SPACE       efter numret, nyckelord och :
11A3  7E             L11A3: LD   A,(HL)
11A4  FE 3A                 CP   $3A         :
11A6  28 4D                 JR   Z,L11F5
11A8  FE 87                 CP   $87         IF/ELSE-hopp
11AA  28 59                 JR   Z,L1205
11AC  FE B9                 CP   $B9         FOR-post
11AE  28 54                 JR   Z,L1204
11B0  FE BF                 CP   $BF         radreferens
11B2  CA 3D 12              JP   Z,L123D
11B5  FE BE                 CP   $BE         lista med radreferenser
11B7  28 7C                 JR   Z,L1235
11B9  FE 0D                 CP   $0D         radslut
11BB  28 3F                 JR   Z,L11FC
11BD  FE C0                 CP   $C0         uttryckskod
11BF  30 3E                 JR   NC,L11FF
11C1  FE 96                 CP   $96         DEF FN (fel)
11C3  28 44                 JR   Z,L1209
11C5  FE 80                 CP   $80         vanligt tecken
11C7  38 70                 JR   C,L1239
11C9  CD EA 11              CALL SPACE       före nyckelordet
11CC  7E                    LD   A,(HL)      token
11CD  23                    INC  HL          förbi den
11CE  01 11 19              LD   BC,STKW1    i den första tabellen
11D1  FE 86                 CP   $86         $86: token i STKW2
11D3  20 05                 JR   NZ,L11DA    nej
```

```
11D5  01 9C 19           LD   BC,STKW2        i den andra tabellen
11D8  7E                 LD   A,(HL)          A = token i STKW2
11D9  23                 INC  HL              förbi den
```

Nyckelordstabellerna är token följd av text, post efter post. `CPIR` *letar upp token, och texten kopieras tills nästa byte med bit 7 satt.* `$81` *och* `$83`, `GOTO` *efter* `THEN` *och* `LET` *utan ordet, har ingen text och blir ingenting.*

```
11DA  E5           L11DA:  PUSH HL              koden vid SP
11DB  60                   LD   H,B             HL = tabellen
11DC  69                   LD   L,C
11DD  01 00 00             LD   BC,$0000        BC = 0: utan gräns
11E0  ED B1                CPIR                 HL efter token i tabellen
11E2  7E           L11E2:  LD   A,(HL)
11E3  87                   ADD  A,A             bit 7: nästa token, texten slut
11E4  38 B9                JR   C,L119F
11E6  ED A0                LDI                  texten till bufferten
11E8  18 F8                JR   L11E2
```

`SPACE`*: skriv ett mellanslag vid DE, om inte tecknet före DE redan är ett. Ut: A =* `$20`*, DE efter mellanslaget.*

```
11EA  3E 20        SPACE:  LD   A,$20
11EC  EB                   EX   DE,HL
11ED  2B                   DEC  HL
11EE  BE                   CP   (HL)            förra tecknet ett mellanslag?
11EF  23                   INC  HL
11F0  EB                   EX   DE,HL
11F1  C8                   RET  Z
11F2  12                   LD   (DE),A
11F3  13                   INC  DE
11F4  C9                   RET

11F5  CD EA 11     L11F5:  CALL SPACE           före :
11F8  ED A0                LDI                  : (mellanslaget efter vid $11A0)
11FA  18 A4                JR   L11A0

11FC  ED A0        L11FC:  LDI                  CR med, DE efter
11FE  C9                   RET

11FF  CD 58 12     L11FF:  CALL DETEXP          HL efter uttrycket
1202  18 9F                JR   L11A3

1204  23           L1204:  INC  HL               $B9: 3 byte
1205  23           L1205:  INC  HL               $87 n: 2 byte
1206  23                   INC  HL
1207  18 9A        L1207:  JR   L11A3
```

`DEF FN`*:* `DEFFN` *kopieras från* `KWDEFN`*, utan mellanslag, och sedan funktionens namn med* `DETVAR`*. Efter namnet står antalet parametrar och två byte per parameter, namnbyten och bokstaven.* `DETVAR` *går alltid fyra byte fram, förbi en adress som inte finns här, så HL backas två byte efter varje anrop. Parametrarna skrivs inom parentes med komma emellan.*

```
1209  23           L1209:  INC  HL               förbi $96
120A  E5                   PUSH HL              koden vid SP
120B  21 6D 19             LD   HL,KWDEFN       DEFFN
120E  ED A0        L120E:  LDI                  till bufferten
1210  CB 7E                BIT  7,(HL)          nästa byte en token?
1212  28 FA                JR   Z,L120E
1214  E1                   POP  HL              HL på namnbyten
1215  CD 22 14             CALL DETVAR          funktionens namn
1218  2B                   DEC  HL              tillbaka två byte till antalet
1219  2B                   DEC  HL
121A  7E                   LD   A,(HL)          antal parametrar
```

```
121B  23              INC  HL
121C  A7              AND  A
121D  28 84           JR   Z,L11A3       inga parametrar: inga parenteser
121F  47              LD   B,A           B = antalet
1220  3E 28           LD   A,$28         (
1222  12        L1222: LD   (DE),A
1223  13              INC  DE
1224  C5              PUSH BC
1225  CD 22 14        CALL DETVAR        parameterns namn, HL fyra byte fram
1228  2B              DEC  HL            och två tillbaka
1229  2B              DEC  HL
122A  C1              POP  BC
122B  3E 2C           LD   A,$2C         ,
122D  10 F3           DJNZ L1222         nästa parameter
122F  3E 29           LD   A,$29         )
1231  12              LD   (DE),A
1232  13              INC  DE
1233  18 D2           JR   L1207
```

Radreferenser: `$BE` *med antalet i B eller* `$BF` *med B = 1. Varje referens är fyra byte, numret och adressen, och numret skrivs med* `PRNUM`*, med* `,` *emellan.*

```
1235  23        L1235: INC  HL
1236  46               LD   B,(HL)        B = antal referenser
1237  18 06            JR   L123F

1239  ED A0     L1239: LDI                vanligt tecken, som det står
123B  18 CA            JR   L1207

123D  06 01     L123D: LD   B,$01         $BF: en radreferens
123F  23        L123F: INC  HL            förbi $BF eller antalet
1240  D5        L1240: PUSH DE            bufferten vid SP
1241  CD 1D 10         CALL RDDE          radnumret
1244  23               INC  HL            förbi adressen
1245  23               INC  HL
1246  E3               EX   (SP),HL       HL = bufferten, koden vid SP
1247  EB               EX   DE,HL         HL = numret, DE = bufferten
1248  C5               PUSH BC
1249  CD 62 18         CALL PRNUM         radnumret som text
124C  C1               POP  BC
124D  E1               POP  HL            HL = nästa referens
124E  10 02            DJNZ L1252         fler: , emellan
1250  18 B5            JR   L1207

1252  3E 2C     L1252: LD   A,$2C         ,
1254  12               LD   (DE),A
1255  13               INC  DE
1256  18 E8            JR   L1240
```

3.29 DETEXP: skriv ett kompilerat uttryck (omvänd polsk notation) som infix $1258

Uttryckskoden har operanderna först och operatorn efter (se CMPEXP). Texten byggs i samma ordning som koden läses:

```
* En operand skrivs direkt vid DE, och adressen där den börjar
  läggs på stacken.
* En operator skriver sin text sist i bufferten och flyttar den
  sedan bakåt till rätt plats med ROTTXT. För "A+B" står i koden
  A, B, +. Texten blir först "A" "B" "+" och efter rotationen
  "A" "+" "B". Startadressen för B tas bort från stacken, så att
  "A+B" nu räknas som en operand som börjar vid A.
* Parenteserna i originaltexten finns kvar som token $D3. Den
  sätter "(" framför den senaste operanden och ")" efter.
```

Så blir uttrycket skrivet med exakt de parenteser som användaren skrev, trots att koden i sig inte behöver några.

Slut: $B6–$B8 skrivs som "=" framför den senaste operanden (tilldelning i LET/FOR, typ 0–2). Andra token under $C0 (till exempel $B9+typ, som kompilatorn lägger sist i ett uttryck) avslutar uttrycket och skrivs inte ut. Adressen vid SP, den sista operandens start, tas bort vid slutet. Ut: HL efter sluttoken, DE efter texten. Förstör A, BC och IX.

```
1258  7E          DETEXP: LD   A,(HL)          nästa token
1259  23                  INC  HL
125A  FE B6               CP   $B6             tilldelning, typ 0-2
125C  28 3C       L125C:  JR   Z,L129A
125E  FE B7               CP   $B7
1260  28 FA               JR   Z,L125C
1262  FE B8               CP   $B8
1264  28 F6               JR   Z,L125C
1266  FE C0               CP   $C0             annan token < $C0: slut
1268  38 35               JR   C,L129F
126A  FE E1               CP   $E1             NOT
126C  CA 88 13            JP   Z,L1388
126F  FE E2               CP   $E2             jämförelser och aritmetik
1271  D2 94 13            JP   NC,L1394
1274  FE DC               CP   $DC             logiska operatorer
1276  D2 90 13            JP   NC,L1390
1279  FE D7               CP   $D7             unärt minus
127B  CA 1B 14    L127B:  JP   Z,L141B
127E  FE D8               CP   $D8
1280  28 F9               JR   Z,L127B
1282  FE DB               CP   $DB             FN-argumentpost, 4 byte, skrivs inte
1284  28 1B               JR   Z,L12A1
1286  FE D3               CP   $D3             parentes
1288  28 1D               JR   Z,L12A7
128A  30 CC               JR   NC,DETEXP       $D4-$D6 (hämta värde) skrivs inte
```

Token $C0-$D2 *går via hopptabellen* DETTAB *(2 byte per token):*

```
$C0-$C2  variabel utan index        -> DETVAR
$C3      flyttalskonstant           -> $12FD
$C4-$C6  variabel med ett index     -> namn + "(" index ")"
$C7      heltalskonstant            -> tal + "%"
$C8-$CA  variabel med två index     -> index sätts ihop med ","
$CB      strängkonstant             -> $131E
$CC      variabel i DIM             -> $1337
$D2      DIM av sträng utan längd   -> $137E
$CD-$D1  funktion med 0-4 argument  -> argumenten sätts ihop med ","
         och namnet sätts framför
```

```
128C  D6 C0              SUB  $C0          index i DETTAB
128E  87                 ADD  A,A          JR-instruktionerna är 2 byte
128F  4F                 LD   C,A
1290  06 00              LD   B,$00
1292  DD 21 B6 12        LD   IX,DETTAB
1296  DD 09              ADD  IX,BC        IX = instruktionen för token
1298  DD E9              JP   (IX)

129A  3E 3D        L129A: LD   A,$3D       =
129C  CD 0C 14           CALL JOINA        = framför den senaste operanden
129F  C1           L129F: POP  BC          uttryckets start bort
12A0  C9                 RET               HL efter sluttoken

12A1  23           L12A1: INC  HL          förbi de 4 bytena
12A2  23                  INC  HL
12A3  23                  INC  HL
12A4  23                  INC  HL
12A5  18 B1               JR   DETEXP       nästa token

12A7  D5           L12A7: PUSH DE          p2 = där ( skrivs
12A8  3E 28        L12A8: LD   A,$28       ( framför operanden
12AA  12                  LD   (DE),A
12AB  13                  INC  DE
12AC  CD 49 14            CALL ROTTXT       ( (eller namnet och () framför operanden
12AF  3E 29               LD   A,$29        )
12B1  12                  LD   (DE),A
12B2  13                  INC  DE
12B3  C1                  POP  BC           p2 bort igen
12B4  18 A2        L12B4: JR   DETEXP

12B6  18 28        DETTAB: JR  L12E0       $C0 flyttalsvariabel

12B8  18 26        L12B8: JR   L12E0       $C1 heltalsvariabel

12BA  18 24        L12BA: JR   L12E0       $C2 strängvariabel

12BC  18 3F        L12BC: JR   L12FD       $C3 flyttalskonstant

12BE  18 20        L12BE: JR   L12E0       $C4 flyttal, ett index

12C0  18 1E        L12C0: JR   L12E0       $C5 heltal, ett index

12C2  18 1C        L12C2: JR   L12E0       $C6 sträng, ett index

12C4  18 46        L12C4: JR   L130C       $C7 heltalskonstant

12C6  18 15        L12C6: JR   L12DD       $C8 flyttal, två index

12C8  18 13        L12C8: JR   L12DD       $C9 heltal, två index

12CA  18 11        L12CA: JR   L12DD       $CA sträng, två index

12CC  18 50        L12CC: JR   L131E       $CB strängkonstant

12CE  18 67        L12CE: JR   L1337       $CC variabel i DIM

12D0  18 25        L12D0: JR   L12F7       $CD funktion utan argument

12D2  18 1D        L12D2: JR   L12F1       $CE ett argument

12D4  18 18        L12D4: JR   L12EE       $CF två argument

12D6  18 13        L12D6: JR   L12EB       $D0 tre argument

12D8  18 0E        L12D8: JR   L12E8       $D1 fyra argument

12DA  C3 7E 13     L12DA: JP   L137E       $D2 sträng utan längd i DIM

12DD  CD 0A 14     L12DD: CALL JOINC       två index: ihop med ,
12E0  D5           L12E0: PUSH DE          operandens start
12E1  CD 22 14            CALL DETVAR
```

```
12E4  28 CE              JR   Z,L12B4        utan index: klar
12E6  18 C0              JR   L12A8          namn och ( framför indexet

12E8  CD 0A 14    L12E8: CALL JOINC          4 argument: tre komman
12EB  CD 0A 14    L12EB: CALL JOINC          3 argument: två komman
12EE  CD 0A 14    L12EE: CALL JOINC          2 argument
12F1  D5          L12F1: PUSH DE             funktion med argument
12F2  CD E9 13           CALL FNTEXT         namnet, HL efter anropet
12F5  18 B1              JR   L12A8          namnet och ( framför argumenten

12F7  D5          L12F7: PUSH DE             funktion utan argument (RND)
12F8  CD E9 13           CALL FNTEXT
12FB  18 B7       L12FB: JR   L12B4

12FD  D5          L12FD: PUSH DE             operandens start
12FE  01 04 00           LD   BC,$0004       4 byte mantissa
1301  09                  ADD  HL,BC
1302  E5                  PUSH HL
1303  3E 2C               LD   A,$2C          format $2C: 6 siffror, inget mellanslag
1305  CD 79 16            CALL FLTASC         skriv flyttalet
1308  E1                  POP  HL
1309  23                  INC  HL             förbi exponenten
130A  18 EF       L130A: JR   L12FB          klar

130C  D5          L130C: PUSH DE             operandens start
130D  D5                 PUSH DE             bufferten
130E  5E                 LD   E,(HL)         heltalet
130F  23                 INC  HL
1310  56                 LD   D,(HL)
1311  23                 INC  HL
1312  E3                 EX   (SP),HL        HL = bufferten, koden vid SP
1313  EB                 EX   DE,HL          HL = talet, DE = bufferten
1314  CD 62 18           CALL PRNUM          talet som text
1317  3E 25              LD   A,$25          %
1319  12                 LD   (DE),A
131A  13                 INC  DE
131B  E1                 POP  HL             HL = koden
131C  18 EC       L131C: JR   L130A
```

Strängkonstant: `$CB`*, citattecknet (" eller '), längden och texten. Ett citattecken inne i texten skrivs två gånger, så att raden kan läsas in igen.*

```
131E  D5          L131E: PUSH DE
131F  CD 17 10           CALL RDBC           C = citattecknet, B = längden
1322  79          L1322: LD   A,C           citattecknet
1323  12                 LD   (DE),A
1324  13                 INC  DE
1325  05          L1325: DEC  B              tecken kvar?
1326  FA 32 13           JP   M,L1332
1329  7E                 LD   A,(HL)
132A  23                 INC  HL
132B  12                 LD   (DE),A
132C  13                 INC  DE
132D  B9                 CP   C              citattecken i texten skrivs dubbelt
132E  28 F2              JR   Z,L1322
1330  18 F3              JR   L1325

1332  79          L1332: LD   A,C           och det sista
1333  12                 LD   (DE),A
1334  13                 INC  DE
1335  18 E5       L1335: JR   L131C
```

DIM-*post. I koden:* [*index ...*] [*längd*] $CC [*variabeltoken*] [*namnbyte, bokstav, 2 byte*]. *En sträng utan längd har* $D2 *i stället för längden;* $D2 *kommer då före* $CC*, och* $137E *hoppar över* $CC *och variabeltoken. Utskrift: tal* C(i,j)*, sträng med längd* A$=n *eller* A$(i,j)=n*, sträng utan längd* A$(i,j)*. Efter posten kommer* , *eller* $BF*, som avslutar* DIM*-listan.*

DIM *A$ utan både index och längd (går att skriva in i 11273, 9913 ger fel 15) blir* $D2 $CC *... utan någon operand före. Då får* ROTTXT *en adress på stacken som inte hör till texten, och* LIST *skriver sönder minnet. Körningen går bra.*

```
1337  23          L1337:  INC  HL          förbi variabeltoken, HL på namnbyten (fel)
1338  3E 02               LD   A,$02
133A  A6                  AND  (HL)        strängvariabel?
133B  20 1D               JR   NZ,L135A
133D  AF          L133D:  XOR  A           CY = 0: ) sist
133E  08          L133E:  EX   AF,AF'      CY sparas i F'
133F  CB 5E               BIT  3,(HL)      två index?
1341  C4 0A 14           CALL NZ,JOINC
1344  D5                  PUSH DE          p2 = namnets start
1345  3E 28               LD   A,$28       ( efter namnet
1347  F5          L1347:  PUSH AF          ( eller = vid SP
1348  CD 22 14            CALL DETVAR      namnet
134B  F1                  POP  AF
134C  12                  LD   (DE),A      ( eller =
134D  13                  INC  DE
134E  CD 49 14            CALL ROTTXT      namn och tecknet framför operanden
1351  08                  EX   AF,AF'      CY tillbaka
1352  38 1C               JR   C,L1370     CY: inget )
1354  3E 29               LD   A,$29       )
1356  12                  LD   (DE),A
1357  13                  INC  DE
1358  18 16               JR   L1370

135A  D5          L135A:  PUSH DE          sträng: p2 = där )= skrivs
135B  3E 0C               LD   A,$0C
135D  A6                  AND  (HL)
135E  28 22               JR   Z,L1382     utan index: A$=n
1360  3E 29               LD   A,$29       )
1362  12                  LD   (DE),A
1363  13                  INC  DE
1364  3E 3D               LD   A,$3D       =
1366  12                  LD   (DE),A
1367  13                  INC  DE
1368  CD 49 14            CALL ROTTXT      )= framför längden
136B  C1                  POP  BC          båda adresserna bort
136C  C1                  POP  BC
136D  37                  SCF              CY: inget ) sist
136E  18 CE               JR   L133E

1370  C1          L1370:  POP  BC          operandens adresser bort
1371  C1                  POP  BC
1372  7E                  LD   A,(HL)      , eller $BF
1373  23                  INC  HL
1374  FE BF               CP   $BF         $BF: DIM-listan slut
1376  C8                  RET  Z
1377  2B                  DEC  HL          HL tillbaka på ,
1378  3E 2C               LD   A,$2C       ,
137A  12                  LD   (DE),A
137B  13                  INC  DE
137C  18 B7       L137C:  JR   L1335       nästa element

137E  23          L137E:  INC  HL          $D2: förbi $CC och variabeltoken
137F  23                  INC  HL
1380  18 BB               JR   L133D       vidare som för ett tal
```

```
1382  37           L1382:  SCF                 CY: inget )
1383  08                   EX   AF,AF'
1384  3E 3D                LD   A,$3D          =
1386  18 BF                JR   L1347          namnet och = framför längden
```

Operatorer: CALL DETOP *följs av en flaggbyte.*

```
bit 0 = mellanslag runt operatorn (EQV, IMP, OR, XOR, AND, NOT)
bit 1 = unär operator (bara en operand)
$E1 NOT: 3,  $DC-$E0: 1,  $E2-$FF (jämförelser, aritmetik): 0.
```

Ingången $138C *(flagga 2, unär utan mellanslag) används inte i denna ROM.* DETOP *återvänder aldrig till flaggbyten utan hoppar tillbaka in i* DETEXP*.*

```
1388  CD 98 13     L1388:  CALL DETOP          NOT: mellanslag, unär
138B  03                   DEFB $03

138C  CD 98 13     L138C:  CALL DETOP          används inte
138F  02                   DEFB $02

1390  CD 98 13     L1390:  CALL DETOP          EQV, IMP, OR, XOR, AND: mellanslag
1393  01                   DEFB $01

1394  CD 98 13     L1394:  CALL DETOP          jämförelser och räknesätt
1397  00                   DEFB $00
```

DETOP*: operatorn. Anropas med* CALL *följt av flaggbyten, som läses via returadressen i IX;* DETOP *återvänder aldrig dit utan hoppar tillbaka in i* DETEXP*. Vid SP ligger den högra operandens start och över den den vänstras. Operatorns text skrivs sist, med mellanslag före och efter om flaggans bit 0 är satt och där tecknet intill inte redan är ett mellanslag.* ROTTXT *flyttar den framför den högra operanden (unär: framför den enda), och sedan räknas allt som en operand.*

```
1398  DD E1        DETOP:  POP  IX              IX = flaggbyten
139A  C1                   POP  BC              BC = högra operandens start
139B  C5                   PUSH BC              tillbaka vid SP
139C  D5                   PUSH DE              p = där operatorn skrivs
139D  DD CB 00 46          BIT  0,(IX+$00)      mellanslag kring operatorn?
13A1  28 0B                JR   Z,L13AE
13A3  C5                   PUSH BC              sparas till mellanslaget efter
13A4  0B                   DEC  BC              tecknet före operatorn
13A5  0A                   LD   A,(BC)
13A6  FE 20                CP   $20             redan ett mellanslag?
13A8  28 04                JR   Z,L13AE         ja
13AA  3E 20                LD   A,$20           mellanslag före
13AC  12                   LD   (DE),A
13AD  13                   INC  DE
13AE  2B           L13AE:  DEC  HL              token
13AF  7E                   LD   A,(HL)
13B0  23                   INC  HL
13B1  CD D6 13             CALL OPTEXT          operatorns text
13B4  DD CB 00 46          BIT  0,(IX+$00)      mellanslag efter?
13B8  28 0A                JR   Z,L13C4
13BA  C1                   POP  BC              BC = högra operandens start
13BB  0A                   LD   A,(BC)          dess första tecken
13BC  FE 20                CP   $20             börjar den med ett mellanslag?
13BE  28 04                JR   Z,L13C4         ja
13C0  3E 20                LD   A,$20           mellanslag efter
13C2  12                   LD   (DE),A
13C3  13                   INC  DE
13C4  DD CB 00 4E  L13C4:  BIT  1,(IX+$00)      unär?
13C8  20 06                JR   NZ,L13D0
13CA  CD 49 14             CALL ROTTXT          binär: framför högra operanden
13CD  C1                   POP  BC              p bort
```

```
13CE  18 03              JR   L13D3

13D0  CD 49 14     L13D0: CALL ROTTXT      unär: framför operanden
13D3  C1           L13D3: POP  BC          binär: högra operandens start bort; unär: p bort
13D4  18 A6               JR   L137C       tillbaka till DETEXP
```

OPTEXT: *skriv texten för operatortoken A vid DE. Jämförelserna och räknesätten har en token per typ, men bara den första står i tabellen från* KWEQV*, så A, A - 1, A - 2 ... prövas tills token hittas.* $FF *(flyttal ^ heltal) börjar på* $FE*, eftersom* $FF *också skiljer tabellerna åt. Ut: DE efter texten. Förstör A och BC.*

```
13D6  E5           OPTEXT: PUSH HL          koden sparas
13D7  FE FF               CP   $FF         $FF: börja på $FE
13D9  28 01               JR   Z,L13DC     ja
13DB  3C                  INC  A           så att DEC ger A igen
13DC  3D           L13DC: DEC  A           token, token - 1 ...
13DD  21 F5 23            LD   HL,KWEQV    operatorernas text
13E0  01 3C 00            LD   BC,$003C    $3C byte
13E3  ED B1               CPIR
13E5  20 F5               JR   NZ,L13DC    inte funnen: token - 1
13E7  18 11               JR   L13FA       kopiera texten
```

FNTEXT: *skriv funktionens namn. Byten i koden är 2 × funktionens nummer, och* SCF *och* RRA *gör om den till token* $80 *+ numret i* FNKW*. Numret 0 är* FN*, och då följer namnet med* DETVAR*. Ut: HL efter anropet, DE efter texten. Förstör A och BC.*

```
13E9  7E           FNTEXT: LD   A,(HL)      2 × funktionens nummer
13EA  37                   SCF
13EB  1F                   RRA              A = $80 + numret
13EC  23                   INC  HL          förbi koden
13ED  FE 80                CP   $80         kod 0: FN
13EF  28 11                JR   Z,L1402
13F1  E5           FNKWTX: PUSH HL
13F2  21 31 24             LD   HL,FNKW     tabellen med funktionernas namn
13F5  01 00 00             LD   BC,$0000    BC = 0: utan gräns
13F8  ED B1                CPIR             HL efter token
13FA  ED A0        L13FA:  LDI              texten till DE
13FC  7E                   LD   A,(HL)
13FD  87                   ADD  A,A         bit 7: slut på texten
13FE  30 FA                JR   NC,L13FA
1400  E1                   POP  HL          koden
1401  C9                   RET

1402  CD F1 13     L1402:  CALL FNKWTX      FN
1405  CD 22 14             CALL DETVAR      namnet, HL fyra byte fram
1408  23                   INC  HL          förbi avståndsbyten
1409  C9                   RET
```

JOINC *och* JOINA*: skriv , eller tecknet i A framför den senaste operanden, vars start ligger vid SP+2, över returadressen. De två senaste operanderna blir en. Förstör BC och IX.*

```
140A  3E 2C        JOINC:  LD   A,$2C
140C  DD E1        JOINA:  POP  IX          IX = returadressen
140E  C1                   POP  BC          BC = senaste operandens start
140F  DD E5                PUSH IX          returadressen tillbaka
1411  C5                   PUSH BC          p1 = senaste operandens start
1412  D5                   PUSH DE          p2 = där tecknet skrivs
1413  12                   LD   (DE),A      tecknet sist i texten
1414  13                   INC  DE
1415  CD 49 14             CALL ROTTXT      tecknet framför operanden
1418  C1                   POP  BC          p2 bort
1419  C1                   POP  BC          p1 bort: två operander har blivit en
```

```
141A  C9                RET
```

Unärt minus (`$D7` och `$D8`): `-` framför operanden, utan flaggbyte, med slutet av `DETOP`.

```
141B  D5          L141B: PUSH DE             p = där - skrivs
141C  3E 2D              LD   A,$2D          -
141E  12                 LD   (DE),A
141F  13                 INC  DE
1420  18 AE               JR   L13D0
```

`DETVAR`: skriv variabelnamnet. HL pekar på namnbyten (siffran i bit 4–7, `$F` om namnet inte har någon, antalet index i bit 2–3 och typen i bit 0–1), sedan bokstaven och två byte för adressen. Skrivs: bokstaven, siffran och `%` (typ 1) eller `$` (typ 2). Ut: HL fyra byte fram, DE efter namnet, A = namnbyten AND `$0C` och Z om variabeln inte har något index. Förstör C.

```
1422  7E          DETVAR: LD   A,(HL)        namnbyten
1423  23                  INC  HL            förbi namnbyten
1424  ED A0               LDI                bokstaven
1426  23                  INC  HL            förbi adressen
1427  23                  INC  HL
1428  4F                  LD   C,A           C = namnbyten
1429  E6 F0               AND  $F0           siffra i namnet?
142B  FE F0               CP   $F0           $F: ingen siffra
142D  28 08               JR   Z,L1437       ingen siffra
142F  0F                  RRCA               siffran till bit 0–3
1430  0F                  RRCA
1431  0F                  RRCA
1432  0F                  RRCA
1433  F6 30               OR   $30           som ASCII
1435  12                  LD   (DE),A
1436  13                  INC  DE
1437  3E 03       L1437:  LD   A,$03         typen
1439  A1                  AND  C
143A  28 09               JR   Z,L1445       0: flyttal
143C  3D                  DEC  A             typen - 1
143D  3E 25               LD   A,$25         %
143F  28 02               JR   Z,L1443       1: heltal
1441  3E 24               LD   A,$24         $
1443  12          L1443: LD   (DE),A
1444  13                  INC  DE
1445  3E 0C       L1445:  LD   A,$0C         Z: inga index
1447  A1                  AND  C
1448  C9                  RET
```

3.30 ROTTXT: flytta den senast skrivna texten bakåt $1449

På stacken (ovanför returadressen) ligger två adresser: p2, som lades senast, vid SP+2 och p1 över den. Texten mellan p1 och DE ser ut så här:

```
p1         p2         DE
[ text 1   ][ text 2   ]
```

Efter `ROTTXT`:

```
[ text 2   ][ text 1   ]
```

Först flyttas hela texten från p1 till DE uppåt med längden av text 2 (`LDDR`), och sedan flyttas text 2, som nu ligger efter DE, ner till p1. HL, DE och stacken är oförändrade. Förstör BC och IX. Byten efter DE används som mellanlager.

```
1449  E5          ROTTXT: PUSH HL            HL och DE sparas
```

```
144A  D5                PUSH DE
144B  DD 21 00 00       LD   IX,$0000      IX = stacken
144F  DD 39             ADD  IX,SP
1451  DD 4E 06          LD   C,(IX+$06)    BC = p2 (+6)
1454  DD 46 07          LD   B,(IX+$07)
1457  62                LD   H,D           HL = DE
1458  6B                LD   L,E
1459  ED 42             SBC  HL,BC         längden av text 2 (CY = 0 efter ADD IX,SP)
145B  E5                PUSH HL            längden av text 2 vid SP
145C  19                ADD  HL,DE         HL = nya slutet
145D  E5                PUSH HL            nya slutet vid SP
145E  62                LD   H,D           HL = DE
145F  6B                LD   L,E
1460  DD 4E 08          LD   C,(IX+$08)    BC = p1 (+8)
1463  DD 46 09          LD   B,(IX+$09)
1466  ED 42             SBC  HL,BC         längden av text 1 + text 2 (CY = 0 efter ADD HL,DE)
1468  44                LD   B,H           BC = längden av båda
1469  4D                LD   C,L
146A  E1                POP  HL            HL = nya slutet
146B  2B                DEC  HL
146C  1B                DEC  DE
146D  EB                EX   DE,HL         HL = gamla slutet - 1, DE = nya - 1
146E  D5                PUSH DE            spara nya slutet - 1
146F  ED B8             LDDR               BC = båda texterna: allt uppåt
1471  E1                POP  HL            HL = nya slutet - 1
1472  C1                POP  BC            BC = längden av text 2
1473  ED B8             LDDR               text 2 ner till p1
1475  D1                POP  DE            DE och HL tillbaka
1476  E1                POP  HL
1477  C9                RET
```

Korsreferenstabell för programraden som text. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. DE är hela tiden pekaren i bufferten där texten skrivs, och HL pekaren i den kompilerade raden. DETEXP använder stacken för operandernas startadresser: en operand lägger sin start vid SP, en operator eller JOINC/JOINA gör de två senaste till en, och den sista tas bort vid slutet, så stacken är som förut när DETEXP kommer tillbaka. ROTTXT läser de två senaste (p1 och p2) utan att ta bort dem. DETOP* anropas med CALL följt av en flaggbyte, tar returadressen (till IX) och den högra operandens start från stacken och hoppar tillbaka in i DETEXP; för den visas bara stacken. DETEXP skriver tal med FLTASC (förstör IX och använder stacken som arbetsyta) och PRNUM, och hoppar till operandernas rutiner genom DETTAB med JP (IX). SPACE och DETVAR svarar med Z (mellanslaget fanns redan, variabeln har inget index). Rutinerna rör inga systemvariabler. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
DETOK	X	X	X	IU	IU	X	
SPACE	U	U		IU			
DETEXP	X	X	X	IU	IU	X	
DETOP*							I
OPTEXT	IX	X	X	IU			
FNTEXT, FNKWTX	X	X	X	IU	IU		
JOINC, JOINA	I		X	IU		X	I
DETVAR	U	U	X	IU	IU		
ROTTXT		X	X	I		X	I

Rutin	Anropar	Anropas från
DETOK	RDDE, PRNUM, SPACE, DETEXP, DETVAR; tabellerna STKW1, STKW2 och KWDEFN	CMDED, CMDLIST
SPACE		DETOK (tre ställen)
DETEXP	DETOP, JOINC, JOINA, DETVAR, FNTEXT, RDBC, FLTASC, PRNUM, ROTTXT; hopptabellen DETTAB	DETOK; DETOP och rutinerna i DETTAB hoppar tillbaka in i den
DETOP	OPTEXT, ROTTXT; tillbaka in i DETEXP	DETEXP ($1388, $1390, $1394; $138C används inte)
OPTEXT	tabellen KWEQV	DETOP
FNTEXT, FNKWTX	FNTEXT: FNKWTX, DETVAR; tabellen FNKW	FNTEXT: DETEXP (två ställen); FNKWTX: FNTEXT
JOINC, JOINA	ROTTXT	JOINC: DETEXP (fem ställen); JOINA: DETEXP (=)
DETVAR		DETOK (två ställen), DETEXP (två ställen), FNTEXT
ROTTXT		DETEXP (tre ställen), DETOP (två ställen), JOINA

3.31 FNAME: tolka ett filnamn [enhet:]namn[.typ] $1478

In: HL = text, BC = längd (1–127), DE = 14 byte för resultatet.

Ut:

```
+0  namn, 8 tecken
+8  typ (filändelse), 3 tecken
+11 enhet, 3 tecken
```

Det som inte anges blir mellanslag. CY = felaktigt namn (OPEN ger då fel 21); resultatet kan då vara delvis ifyllt. Förstör A, BC, DE och HL.

Texten måste börja med en bokstav (kod $41 eller högre). Om det finns ett ":" någonstans i texten är det som står före enhetens namn (1–3 tecken). Namnet får vara tomt (PR:, PR:.BAC), och efter enheten får det börja med en siffra. Mellanslag hoppas över var de än står (AB C = ABC). Tillåtna tecken är siffror och A–Z samt [\ ] (ÄÖÅ i den svenska 7-bitarskoden). Små bokstäver görs om till stora (AND $5F); samma AND stryker bit 7, så $C1 blir "A". Varje del har ett högsta antal tecken; fler tecken ger CY, liksom varje annat tecken som blir kvar (AB$, A.B:C). Provat: cas:ab c.bac ger ABC, BAC och CAS.

```
1478  78          FNAME:  LD   A,B
1479  A7                  AND  A
147A  37                  SCF
147B  C0                  RET  NZ              längden måste vara 1-127
147C  B1                  OR   C
147D  37                  SCF
147E  C8                  RET  Z
147F  F8                  RET  M               C >= $80: CY
1480  7E                  LD   A,(HL)
1481  FE 41               CP   $41             måste börja med en bokstav
1483  D8                  RET  C
1484  C5                  PUSH BC
1485  E5                  PUSH HL
1486  D5                  PUSH DE
1487  0E 0D               LD   C,$0D           fyll 14 byte med mellanslag
1489  6B                  LD   L,E
148A  62                  LD   H,D
148B  13                  INC  DE
```

```
148C  36 20              LD   (HL),$20
148E  ED B0              LDIR
1490  D1                 POP  DE
1491  E1                 POP  HL
1492  C1                 POP  BC
1493  C5                 PUSH BC
1494  E5                 PUSH HL
1495  3E 3A              LD   A,$3A          finns ett :?
1497  ED B1              CPIR                i hela texten
1499  E1                 POP  HL
149A  C1                 POP  BC
149B  20 18              JR   NZ,L14B5
149D  D5                 PUSH DE
149E  3E 0B              LD   A,$0B          enhet på +11
14A0  83                 ADD  A,E
14A1  5F                 LD   E,A
14A2  30 01              JR   NC,L14A5
14A4  14                 INC  D
14A5  06 03        L14A5: LD  B,$03          högst 3 tecken
14A7  CD DA 14           CALL FNPART
14AA  D1                 POP  DE
14AB  D8                 RET  C              fel i enheten
14AC  7E                 LD   A,(HL)
14AD  FE 3A              CP   $3A            : måste följa
14AF  37                 SCF
14B0  C0                 RET  NZ
14B1  23                 INC  HL
14B2  A7                 AND  A
14B3  0D                 DEC  C
14B4  C8                 RET  Z              bara enhet: klart
14B5  06 08        L14B5: LD  B,$08          namn, högst 8 tecken
14B7  CD DA 14           CALL FNPART
14BA  D8                 RET  C
14BB  79                 LD   A,C
14BC  A7                 AND  A
14BD  C8                 RET  Z              slut efter namnet
14BE  7E                 LD   A,(HL)
14BF  FE 2E              CP   $2E            .
14C1  37                 SCF
14C2  C0                 RET  NZ
14C3  78                 LD   A,B            hoppa till +8
14C4  83                 ADD  A,E
14C5  5F                 LD   E,A
14C6  30 01              JR   NC,L14C9
14C8  14                 INC  D
14C9  23           L14C9: INC  HL
14CA  A7                 AND  A
14CB  0D                 DEC  C
14CC  C8                 RET  Z
14CD  06 03              LD   B,$03          typ, högst 3 tecken
14CF  CD DA 14           CALL FNPART
14D2  79                 LD   A,C            något kvar efter typen: CY
14D3  A7                 AND  A
14D4  C8                 RET  Z
14D5  37                 SCF
14D6  C9                 RET

14D7  23           L14D7: INC  HL
14D8  0D                 DEC  C
14D9  C8                 RET  Z
```

FNPART: kopiera högst B tillåtna tecken från HL till DE. Slutar vid första otillåtna tecken (NC) eller när texten är slut (C = 0, Z). CY om delen blir för lång.

```
14DA  7E          FNPART: LD   A,(HL)
14DB  FE 20               CP   $20          mellanslag hoppas över
14DD  28 F8               JR   Z,L14D7
14DF  FE 30               CP   $30
14E1  3F                  CCF               under 0: slut (NC)
14E2  D0                  RET  NC
14E3  FE 3A               CP   $3A
14E5  38 09               JR   C,L14F0
14E7  E6 5F               AND  $5F          små bokstäver -> stora
14E9  FE 41               CP   $41
14EB  3F                  CCF               under A: slut
14EC  D0                  RET  NC
14ED  FE 5E               CP   $5E          A-Z och [\] (ÄÖÅ)
14EF  D0                  RET  NC
14F0  05          L14F0:  DEC  B            för många tecken?
14F1  37                  SCF
14F2  F8                  RET  M
14F3  23                  INC  HL
14F4  12                  LD   (DE),A
14F5  13                  INC  DE
14F6  0D                  DEC  C
14F7  20 E1               JR   NZ,FNPART
14F9  A7                  AND  A            texten slut: NC
14FA  C9                  RET
```

GETSFL: som GETFLT, men blanka och ett + eller − före talet är tillåtna. Vid − sätts teckenbyten (HL-1) till 1 efteråt, även vid CY (DE pekar då efter tecknet) och även för talet 0: −0 blir 00 00 00 01 00, som räknas som 0 (provat: <0 falskt, =0 sant, SGN 0).

Ut vid −: HL = teckenbyten, annars som GETFLT.

```
14FB  EB          GETSFL: EX   DE,HL         (gränsfall)
14FC  E7                  RST  $20          blanka före tecknet
14FD  EB                  EX   DE,HL
14FE  1A                  LD   A,(DE)
14FF  13                  INC  DE
1500  FE 2B               CP   $2B          +
1502  28 0F               JR   Z,GETFLT
1504  1B                  DEC  DE
1505  FE 2D               CP   $2D          -
1507  20 0A               JR   NZ,GETFLT
1509  13                  INC  DE
150A  E5                  PUSH HL
150B  CD 13 15            CALL GETFLT
150E  E1                  POP  HL           även vid CY
150F  2B                  DEC  HL
1510  36 01               LD   (HL),$01     teckenbyte = negativt
1512  C9                  RET
```

Korsreferenstabell för filnamnet och talet med tecken. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. FNAME tar texten i HL med längden i BC och fyller de 14 byten vid DE (namn, typ och enhet) med hjälp av FNPART; CY = felaktigt namn. GETSFL läser ett tal med + eller − före genom GETFLT, som använder 8 byte på stacken och IX och tar bort dem igen; vid − blir HL teckenbyten. Rutinerna rör inga systemvariabler. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
FNAME	X	U	IX	IX	IX		
FNPART	X	U	IU	IU	IU		
GETSFL	X	U	X	IU	IU	X	

Rutin	**Anropar**	**Anropas från**
FNAME	FNPART	den fasta ingången $0041, OPEN
FNPART		FNAME
GETSFL	RST20, GETFLT	RDVAL, EXVAL

3.32 GETFLT: läs ett flyttal från texten $1513

In: DE = texten, HL = talets sista byte (exponenten).

Ut: NC: talet är lagrat, HL = mantissans början och DE = ett steg efter första tecknet efter talet (anroparna gör DEC DE).

CY: inget giltigt tal, eller spill (DE oförändrad, HL = mantissans början).

Förstör A, BC och IX.

Flyttalsformatet (5 byte, BCD):

```
+0 +1 +2   mantissa, 6 BCD-siffror, första siffran i bit 4-7 av +0
+3         tecken: 0 = positivt, 1 = negativt
+4         exponent + $80; $00 betyder att talet är 0
Värdet är 0.d1d2d3d4d5d6 * 10^(exponent-$80).
Exempel ur ROM:en: 1 = 10 00 00 00 81, PI/2 = 15 70 80 00 81,
2/PI = 63 66 20 00 80.
```

Rutinerna pekar alltid på den sista byten (exponenten).

Inläsningen använder 8 byte på stacken, IX pekar på dem:

```
IX+0,1  textpekaren
IX+2,3  var nästa siffra ska skrivas i mantissan
IX+4    bit 4/5: minst en siffra har lästs
IX+6    E: bit 7 = decimalpunkt läst, bit 6 = punkten kom före
        första siffran som inte är 0; bit 0-5 = antal siffror
        före punkten, eller antal nollor efter punkten
IX+7    1 = negativ exponent efter "E"
```

D räknar de gällande siffrorna (bit 7 = halv byte). Bara de sex första sparas; resten räknas bara in i exponenten. Inledande nollor sparas inte. Den sjunde siffran och de följande avrundas inte utan huggs av (9999999 blir 9.99999E+6). Tecknen läses med RST $18, så blanka inne i talet hoppas över (1 2 . 5 = 12.5). En andra decimalpunkt ändrar inget (1.2.3 = 1.23). Högst 63 siffror före punkten.

Efter "E" (eller "e") följer ett heltal med tecken, 0–127, som läses med GETLN. Följer ingen siffra direkt efter "E" räknas "E" inte till talet (talet slutar före "E", VAL("1E") ger fel 12). Men efter "E+" och "E-" utan siffror slutar talet efter tecknet: 1E+ = 1.

Omfånget (provat): största tal .999999E127 (9.99999E126), minsta .1E-127 (.1E-127 går, 1E-128 ger CY).

Rutinen på $1544 läggs som returadress på stacken, så varje teckenrutin som gör RET går tillbaka till slingan.

```
1513  06 05        GETFLT: LD   B,$05          nollställ de 5 bytena (egenhet)
1515  36 00        L1515:  LD   (HL),$00
1517  2B                   DEC  HL
1518  10 FB                DJNZ L1515
151A  23                   INC  HL             HL = mantissans början
151B  E5                   PUSH HL             IX+$0A/$0B
151C  D5                   PUSH DE             IX+8/9: texten
151D  DD 21 F8 FF          LD   IX,$FFF8       8 byte arbetsyta
1521  DD 39                ADD  IX,SP
1523  DD F9                LD   SP,IX
1525  1B                   DEC  DE             RST $18 börjar på nästa
1526  EB                   EX   DE,HL          HL = texten - 1
1527  CD 70 16             CALL FLTSAV         IX+0/1 = textpekaren
152A  21 04 00             LD   HL,$0004       IX+4..7 = 0
152D  06 04                LD   B,$04
152F  39                   ADD  HL,SP
1530  36 00        L1530:  LD   (HL),$00
1532  23                   INC  HL
1533  10 FB                DJNZ L1530
1535  11 00 00             LD   DE,$0000       D = gällande siffror, E = punkten och räknaren
1538  DD 6E 0A             LD   L,(IX+$0A)     IX+2,3 = mantissans början
153B  DD 66 0B             LD   H,(IX+$0B)
153E  DD 75 02     L153E:  LD   (IX+$02),L     platsen för nästa siffra
1541  DD 74 03             LD   (IX+$03),H
1544  21 44 15             LD   HL,$1544       returadress = slingan
1547  E5                   PUSH HL
1548  DD 36 07 00          LD   (IX+$07),$00   E-tecknet nollas i varje varv
154C  CD C4 15             CALL FLTCH          nästa tecken, CY för en siffra
154F  38 25                JR   C,L1576        siffra
1551  FE 2E                CP   $2E            .
1553  28 18                JR   Z,L156D        punkt
1555  E6 DF                AND  $DF            E eller e
1557  FE 45                CP   $45
1559  CA 00 16             JP   Z,L1600        exponenten
155C  47           L155C:  LD   B,A            B = tecknet efter talet
155D  DD 7E 04             LD   A,(IX+$04)
1560  E6 10                AND  $10            har någon siffra lästs?
1562  20 71                JR   NZ,L15D5
1564  21 0A 00   L1564:  LD   HL,$000A       SP = IX+8
1567  39                   ADD  HL,SP
1568  F9                   LD   SP,HL
1569  D1                   POP  DE             DE = texten som vid start
156A  E1                   POP  HL             HL = mantissans början
156B  37                   SCF
156C  C9                   RET

156D  AF           L156D:  XOR  A              decimalpunkt
156E  B2                   OR   D
156F  20 02                JR   NZ,L1573
1571  CB F3                SET  6,E            punkt före första gällande siffran
1573  CB FB        L1573:  SET  7,E            en andra punkt ändrar inget
1575  C9                   RET

1576  E6 0F        L1576:  AND  $0F             siffra
1578  47                   LD   B,A            B = siffran
1579  3E 30                LD   A,$30
157B  DD B6 04             OR   (IX+$04)       minst en siffra har lästs
157E  DD 77 04             LD   (IX+$04),A
1581  AF                   XOR  A
1582  B0                   OR   B              inledande nolla?
1583  20 0D                JR   NZ,L1592
1585  B2                   OR   D              redan en gällande siffra?
```

```
1586  20 0A              JR   NZ,L1592
1588  B3                 OR   E
1589  5F                 LD   E,A            ingen punkt än: nollan hoppas över
158A  C8                 RET  Z
158B  1C                 INC  E              nolla efter punkten: räkna
158C  3E 3F              LD   A,$3F          högst 63 nollor
158E  A3                 AND  E
158F  C0                 RET  NZ
1590  18 D2              JR   L1564          64 nollor: inget tal

1592  7B           L1592: LD   A,E            punkt läst?
1593  17                 RLA
1594  38 06              JR   C,L159C        efter punkten räknas inget
1596  1C                 INC  E              siffra före punkten: räkna
1597  3E 3F              LD   A,$3F          högst 63 siffror före punkten
1599  A3                 AND  E
159A  28 C8              JR   Z,L1564
159C  DD 73 06     L159C: LD   (IX+$06),E     spara E
159F  14                 INC  D              en gällande siffra till
15A0  7A                 LD   A,D
15A1  E6 7F              AND  $7F
15A3  28 BF              JR   Z,L1564        fler än 127 siffror
15A5  FE 07              CP   $07            mer än 6 siffror: sparas inte
15A7  D0                 RET  NC
15A8  CB 7A              BIT  7,D            första eller andra halvan av byten?
15AA  78                 LD   A,B
15AB  DD 6E 02           LD   L,(IX+$02)     HL = platsen för siffran
15AE  DD 66 03           LD   H,(IX+$03)
15B1  20 08              JR   NZ,L15BB
15B3  CB FA              SET  7,D
15B5  07                 RLCA                till bit 4-7
15B6  07                 RLCA
15B7  07                 RLCA
15B8  07                 RLCA
15B9  77                 LD   (HL),A         första halvan skriven
15BA  C9                 RET

15BB  CB BA        L15BB: RES  7,D
15BD  B6                 OR   (HL)           bit 0-3
15BE  77                 LD   (HL),A
15BF  23                 INC  HL             nästa byte
15C0  C1                 POP  BC             släng returen, spara HL
15C1  C3 3E 15           JP   L153E

15C4  DD 6E 00     FLTCH: LD   L,(IX+$00)     HL = textpekaren
15C7  DD 66 01           LD   H,(IX+$01)
15CA  DF                 RST  $18            nästa tecken
15CB  CD 70 16           CALL FLTSAV         spara den nya
15CE  FE 3A              CP   $3A            över 9?
15D0  D0                 RET  NC             NC: inte siffra
15D1  FE 30              CP   $30
15D3  3F                 CCF                 CY = siffra
15D4  C9                 RET
```

Talet är färdigt. Om mantissan är 0 blir exponenten 0 (talet 0), annars räknar `FLTEXP` *ut exponenten +* `$80`*.*

```
15D5  11 00 00     L15D5: LD   DE,$0000       L = 0: ingen exponent efter E
15D8  C5           L15D8: PUSH BC             spara tecknet (B)
15D9  EB                  EX   DE,HL          L = exponenten efter E
15DA  DD 4E 0A            LD   C,(IX+$0A)     BC = mantissans början
15DD  DD 46 0B            LD   B,(IX+$0B)
15E0  0A                  LD   A,(BC)         första byten
```

```
15E1  C5                PUSH BC
15E2  B7                OR   A              mantissan 0?
15E3  28 05             JR   Z,L15EA
15E5  CD 42 16          CALL FLTEXP
15E8  C6 80             ADD  A,$80          exponent + $80
15EA  DD E3       L15EA: EX   (SP),IX       IX = mantissan: skriv exponenten
15EC  DD 77 04          LD   (IX+$04),A
15EF  DD E1             POP  IX             IX = arbetsytan igen
15F1  C1                POP  BC             B = tecknet
15F2  D1                POP  DE             returadressen $1544
15F3  E1                POP  HL             HL = IX+0, det sista tecknet
15F4  11 0A 00          LD   DE,$000A       SP = IX+$0A
15F7  DD 19             ADD  IX,DE
15F9  DD F9             LD   SP,IX
15FB  D1                POP  DE             DE = mantissans början
15FC  EB                EX   DE,HL
15FD  13                INC  DE             DE = ett efter det
15FE  B7                OR   A              NC: talet är klart
15FF  C9                RET

1600  DD 7E 04     L1600: LD   A,(IX+$04)
1603  E6 10             AND  $10            E utan siffror före
1605  CA 64 15          JP   Z,L1564
1608  DD 6E 00          LD   L,(IX+$00)     HL = textpekaren vid E
160B  DD 66 01          LD   H,(IX+$01)
160E  E5                PUSH HL             spara den
160F  CD C4 15          CALL FLTCH
1612  38 1B             JR   C,L162F        siffra direkt efter E
1614  E1                POP  HL             tillbaka till E
1615  FE 2B             CP   $2B            +
1617  28 10             JR   Z,L1629
1619  FE 2D             CP   $2D            -
161B  28 08             JR   Z,L1625
161D  CD 70 16     L161D: CALL FLTSAV         utan tal: slutar före E (efter E+)
1620  3E 45             LD   A,$45          E blir tecknet efter talet
1622  C3 5C 15          JP   L155C

1625  DD 36 07 01 L1625: LD   (IX+$07),$01  negativ exponent
1629  CD C4 15    L1629: CALL FLTCH         E+ eller E−
162C  30 EF             JR   NC,L161D
162E  D5                PUSH DE             så att POP DE går jämnt
162F  D1            L162F: POP  DE
1630  CD 1E 18          CALL GETLN          exponenten
1633  46                LD   B,(HL)         slutade vid HL
1634  CD 70 16          CALL FLTSAV
1637  7A                LD   A,D            över 255?
1638  B7                OR   A
1639  20 04             JR   NZ,L163F       högst 127
163B  B3                OR   E              över 127?
163C  F2 D8 15          JP   P,L15D8
163F  C3 64 15     L163F: JP   L1564
```

FLTEXP: *räkna ut talets exponent (utan $80). In: IX = arbetsytan, L = talet efter E (0 om inget). Med E = IX+6 och B = antalet i bit 0-5: utan punkt före första gällande siffran är exponenten B + L (B siffror före punkten), med punkten först L - B (B nollor efter punkten). Vid negativ exponent (IX+7 = 1) räknas med -L. Utanför -127..127 tas tre poster bort från stacken och GETFLT slutar med CY. Ut: A. Förstör B och E.*

```
1642  DD 7E 06  FLTEXP: LD   A,(IX+$06)     E = punkten och räknaren
1645  5F                LD   E,A
1646  E6 3F             AND  $3F
1648  47                LD   B,A            B = antalet (bit 0-5)
```

```
1649  DD 7E 07          LD   A,(IX+$07)     negativ exponent?
164C  B7                OR   A
164D  28 10             JR   Z,L165F
164F  CB 73             BIT  6,E            punkt före första siffran?
1651  28 08             JR   Z,L165B
1653  7D                LD   A,L            nollor + exponent
1654  80                ADD  A,B
1655  FA 6B 16          JP   M,L166B        för litet: fel
1658  2F                CPL                 -(nollor + exponent)
1659  3C                INC  A
165A  C9                RET

165B  AF          L165B: XOR  A             siffror före punkten - exponent
165C  95                 SUB  L
165D  80                 ADD  A,B
165E  C9                 RET

165F  3E 40       L165F: LD   A,$40         punkt före första siffran?
1661  A3                 AND  E
1662  28 04              JR   Z,L1668
1664  AF                 XOR  A             exponent - nollor
1665  90                 SUB  B
1666  85                 ADD  A,L
1667  C9                 RET

1668  78          L1668: LD   A,B           siffror före punkten + exponent
1669  85                 ADD  A,L
166A  F0                 RET  P
166B  E1          L166B: POP  HL            utanför: släng tre poster, fel
166C  E1                 POP  HL
166D  E1                 POP  HL
166E  18 CF              JR   L163F
```

FLTSAV: *spara textpekaren HL i IX+0/1. Förstör inget.*

```
1670  DD 75 00    FLTSAV: LD   (IX+$00),L
1673  DD 74 01            LD   (IX+$01),H
1676  C9                  RET
```

Korsreferenstabell för inläsningen av flyttal. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. GETFLT har sin arbetsyta på 8 byte vid IX på stacken och tar bort den igen; FLTCH, FLTEXP och FLTSAV tar IX = arbetsytan. GETFLT och FLTCH svarar i CY: GETFLT med CY om det inte blev något tal, FLTCH med CY för en siffra (A = tecknet). FLTEXP lämnar exponenten i A; är den utanför tar den bort tre poster från stacken och går till GETFLT:s utgång med CY ($163F). Rutinerna rör inga systemvariabler och ger inga fel; anroparna gör fel av CY. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
GETFLT	X	U	X	IU	IU	X	
FLTCH	U	U			X	I	
FLTEXP	U	X	X	X	I	I	
FLTSAV					I	I	

Rutin	Anropar	Anropas från
GETFLT	FLTSAV, FLTCH, FLTEXP, GETLN	GETSFL (två ställen med JR, ett med CALL), CMPNUM
FLTCH	RST $18, FLTSAV	GETFLT (tre ställen)
FLTEXP		GETFLT ($15E5)
FLTSAV		GETFLT (tre ställen), FLTCH

FLTAS6: FLTASC *med format* $0C (*6 siffror, mellanslag före positivt tal*).

```
1677  3E 0C        FLTAS6: LD   A,$0C
```

3.33 FLTASC: skriv flyttalet vid HL som text vid DE $1679

In: HL = talets exponentbyte, DE = buffert, A = format:

```
        bit 0    alltid E-format (d.dddddE+nn)
        bit 1-4  n = antal decimaler (ROM:en använder 6). Fast format:
                 n decimaler, avrundat, men högst talets 6 siffror.
                 E-format: n decimaler efter första siffran, högst 5
                 (provat: 12.3456 med n = 3 ger 12.346 och 1.235E+1)
        bit 5    inget mellanslag före ett positivt tal
        bit 6    E-format om antalet heltalssiffror (eller nollor
                 efter punkten) + n >= 7. Med n = 6 blir det
                 E-format för allt utom .1 <= x < 1 (provat)
        bit 7    för små tal (minst n nollor efter punkten) skrivs
                 "." och n nollor i stället för " 0" (provat: .0005
                 med n = 3 ger ".000")
      ROM:en använder bara $0C (FLTAS6: n = 6, PRINT med flera) och
      $2C (n = 6 utan mellanslag, när konstanter listas).
```

Ut: DE efter texten. Förstör A, BC, HL och IX.

Mantissan packas upp till en siffra per byte på stacken, med en nolla före (för spill vid avrundning) och en efter. Sedan avrundas talet till det antal siffror som ska skrivas (en siffra från 5 och uppåt ger +1 på siffran före, med minnessiffra), och nollor i slutet tas bort. Skrivsätt:

- Minustecken för negativa tal, annars ett mellanslag (om inte bit 5 är satt).
- Exponent 1..6: siffrorna, med "." där heltalsdelen slutar, till exempel `12.5`.
- Exponent 0 eller mindre: "." och nollor först, som `.05` (ingen nolla före punkten).
- Annars E-format: första siffran, ".", resten, "E", tecken och exponenten (med PRNUM).
- Talet 0 skrivs som "0".

Stackutrymme: 14 byte. IX+0 = format, +1 = teckenbyten, +2 = antal siffror före punkten (eller nollor efter), +$0D = 1 om exponenten är 0 eller negativ, +5.. = siffrorna.

Med 6 siffror (provat):

```
      x >= 100000          E-format: 123456 -> 1.23456E+5
      .1 <= x < 100000     alla siffror: 12345.6, .123456
      1E-6 <= x < .1       högst 6 decimaler, avrundat: .0123456 ->
                           .012346, .0000015 -> .000002
      x < 1E-6             E-format: 5E-7, 1.2E-7
```

Fel: när avrundningen går upp till nästa tiopotens (x i [.0999995, .1), [.00999995, .01) och så vidare) blir det en nolla för mycket: .0999999 skrivs ".001" och .00999999 ".0001". Se $1787.

Rättat i version 9913: där står `LD E,A` i stället för `INC E` (där blir .0999999 ".1").

```
1679  DD 21 F2 FF  FLTASC: LD   IX,$FFF2        14 byte arbetsyta, IX = SP (gränsfall)
167D  DD 39               ADD  IX,SP
```

```
167F  DD F9              LD   SP,IX
1681  DD 77 00           LD   (IX+$00),A      IX+0 = formatet
1684  D5                 PUSH DE              bufferten
1685  01 FC FF           LD   BC,$FFFC        HL till mantissans början
1688  09                 ADD  HL,BC
1689  DD E5              PUSH IX              IX vid SP
168B  06 03              LD   B,$03           3 byte, 6 siffror
168D  DD 36 05 00        LD   (IX+$05),$00    IX+5 = 0, plats för minnet
1691  DD 23        L1691: INC  IX              packa upp 6 siffror
1693  7E                 LD   A,(HL)
1694  57                 LD   D,A
1695  1F                 RRA                  övre siffran
1696  1F                 RRA
1697  1F                 RRA
1698  1F                 RRA
1699  E6 0F              AND  $0F
169B  DD 77 05           LD   (IX+$05),A
169E  DD 23              INC  IX
16A0  7A                 LD   A,D             undre siffran
16A1  E6 0F              AND  $0F
16A3  DD 77 05           LD   (IX+$05),A
16A6  23                 INC  HL
16A7  10 E8              DJNZ L1691
16A9  DD 70 06           LD   (IX+$06),B      IX+12 = 0 efter d6
16AC  DD E1              POP  IX              IX = arbetsytan
16AE  7E                 LD   A,(HL)          teckenbyten
16AF  DD 77 01           LD   (IX+$01),A
16B2  DD 70 0D           LD   (IX+$0D),B      IX+$0D = 0
16B5  23                 INC  HL              HL = exponenten
```

Exponenten ger IX+2 och IX+$0D:

```
e > $80    IX+2 = e - $80 (heltalssiffror), IX+$0D = 0
e <= $80   IX+2 = $80 - e (nollor efter punkten), IX+$0D = 1
x = 0      IX+2 = 1, IX+$0D = 0
```

E = IX+2. 6 eller fler (heltalssiffror eller nollor) ger E-format.

```
16B6  7E                 LD   A,(HL)          exponenten
16B7  B7                 OR   A
16B8  28 0E              JR   Z,FAZERS        talet 0
16BA  D6 80              SUB  $80             - $80
16BC  20 03              JR   NZ,FAEXP        inte $80
16BE  DD 34 0D           INC  (IX+$0D)        e = $80: .1 <= x < 1
16C1  F2 C9 16     FAEXP: JP   P,FAPOS        x >= .1
16C4  2F                 CPL                  nollor - 1
16C5  DD 34 0D           INC  (IX+$0D)        x < 1
16C8  3C           FAZERS: INC  A              nollor (x = 0: 1)
16C9  DD 77 02     FAPOS: LD   (IX+$02),A     IX+2 = heltalssiffror/nollor
16CC  5F                 LD   E,A             E = IX+2
16CD  FE 06              CP   $06             6 eller fler siffror före punkten: E-format
16CF  38 04              JR   C,FAFMT
16D1  DD CB 00 C6  FAEFMT: SET  0,(IX+$00)    E-format
```

FAFMT: C = antal siffror som behålls, räknat från d1; siffran efter dem avrundar (FAROUN). n är antalet decimaler. E-format: C = n+1, högst 6 (provat: n = 3 ger 1.235E+0). Fast format: C = E + n för x >= 1 och n - E (nollorna) för x < 1. C är högst 7, och plats 7 är IX+$0D, som är 0 när x >= 1: då avrundas inget, och talets 6 siffror är taket (provat: 12.3456 med n = 3 ger 12.346, 1234.56 ger 1234.56). Med $0C (n = 6) avrundas alltså bara tal i [1E-6, .1).

```
16D5  DD 7E 00     FAFMT: LD   A,(IX+$00)
16D8  1F                  RRA                 CY = bit 0
```

```
16D9  30 0C              JR   NC,FAFIX
16DB  E6 0F              AND  $0F            n = antal siffror
16DD  FE 06              CP   $06
16DF  38 02              JR   C,FAEDIG
16E1  3E 05              LD   A,$05          högst 5
16E3  57          FAEDIG: LD   D,A         D = n
16E4  3C                 INC  A              C = D + 1
16E5  18 32              JR   FAROUN

16E7  E6 0F       FAFIX:  AND  $0F            D = n
16E9  57                 LD   D,A
16EA  83                 ADD  A,E            + E
16EB  FE 07              CP   $07            >= 7?
16ED  38 06              JR   C,FAFIX1
16EF  DD CB 00 76        BIT  6,(IX+$00)     bit 6: E-format
16F3  20 DC              JR   NZ,FAEFMT
16F5  DD CB 0D 46 FAFIX1: BIT 0,(IX+$0D)     x < 1?
16F9  20 02              JR   NZ,FASMAL
16FB  18 1C              JR   FAROUN         x >= 1: C = n + E

16FD  7A          FASMAL: LD   A,D           n - nollor
16FE  93                 SUB  E
16FF  38 02              JR   C,FATINY
1701  20 16              JR   NZ,FAROUN      > 0: C = n - nollor
```

FATINY: *x < 1 och minst n nollor efter punkten, så ingen siffra syns med n decimaler. Utan bit 7:* `0`*, även för negativa tal. Med bit 7: tecknet,* `.` *och n nollor (provat med n = 3: .0005 ger* `.000`*, -.0005 ger* `-.000`*). Med n = 6 kommer inget tal hit, för 6 nollor ger E-format.*

```
1703  DD 7E 00    FATINY: LD   A,(IX+$00)
1706  B7                  OR   A
1707  F2 51 17            JP   P,FAZERO       bit 7 = 0:  0
170A  E6 0E               AND  $0E            n (bit 1-3)
170C  28 43               JR   Z,FAZERO
170E  0F                  RRCA                E = n - 1 nollor
170F  5F                  LD   E,A
1710  1D                  DEC  E
1711  0E 01               LD   C,$01          och en nolla från IX+5
1713  21 06 00            LD   HL,$0006       HL+1 = IX+5
1716  39                  ADD  HL,SP
1717  18 71               JR   FAOUT
```

FAROUN: *HL = siffran efter de C som behålls (IX+3/4 sparar den). >= 5: +1 på siffran före, med minnessiffra åt vänster. Minnet kan gå ända in i platsen före d1 (IX+5).*

```
1719  E6 1F       FAROUN: AND  $1F            C = högst 7
171B  FE 07               CP   $07
171D  38 02               JR   C,FAROU1
171F  3E 07               LD   A,$07
1721  4F          FAROU1: LD   C,A            C = antal siffror som behålls
1722  06 00               LD   B,$00
1724  21 08 00            LD   HL,$0008       IX+6 + C (SP = IX-2)
1727  39                  ADD  HL,SP
1728  09                  ADD  HL,BC
1729  DD 75 03            LD   (IX+$03),L     spara platsen
172C  DD 74 04            LD   (IX+$04),H
172F  7E                  LD   A,(HL)
1730  FE 05               CP   $05            avrunda
1732  38 13               JR   C,FASTRP       < 5: ingen avrundning
1734  2B          FACARY: DEC  HL
1735  34                  INC  (HL)           +1 med minnessiffra
```

```
1736  7E              LD   A,(HL)
1737  B7              OR   A             kan inte bli 0: siffrorna är 0–9, platsen före d1 är 0
1738  28 0E           JR   Z,FASTR1
173A  D6 0A           SUB  $0A           10?
173C  20 03           JR   NZ,FARND1
173E  77              LD   (HL),A        0 och minne
173F  18 F3           JR   FACARY

1741  DD 6E 03  FARND1: LD  L,(IX+$03)   HL = avrundningsplatsen
1744  DD 66 04        LD   H,(IX+$04)
```

FASTRP: *gå bakåt över nollor i slutet. C blir antalet siffror som ska skrivas. Är alla noll (C < 0) skrivs* `0`.

```
1747  2B        FASTRP: DEC  HL
1748  7E        FASTR1: LD   A,(HL)        ta bort nollor i slutet
1749  B7                OR   A
174A  20 1A             JR   NZ,FAOVFL
174C  2B                DEC  HL           förra siffran
174D  0D                DEC  C            C = antal siffror
174E  F2 48 17          JP   P,FASTR1
1751  D1        FAZERO: POP  DE           bufferten
1752  DD CB 00 6E       BIT  5,(IX+$00)   bit 5: inget mellanslag
1756  20 04             JR   NZ,FAZER1
1758  3E 20             LD   A,$20         0
175A  12                LD   (DE),A
175B  13                INC  DE
175C  CD 61 17  FAZER1: CALL WRZERO       0
175F  18 6D             JR   FAEND

1761  3E 30    WRZERO: LD   A,$30
1763  12              LD   (DE),A
1764  13              INC  DE
1765  C9              RET
```

FAOVFL: *har avrundningen gett en etta i platsen före d1 (IX+5) börjar siffrorna där. För x >= 1 blir det en heltalssiffra till (B = 1), för x < 1 en nolla mindre (B = -1). Var nollorna 0 (.999999...) blir talet >= 1. IX+2 ändras rätt med B, men E (som styr utskriften) ökas alltid med 1, även för x < 1. Därför skrivs .0999999 som* `.001` *(provat). Version 9913 har* `LD E,A` *(E = IX+2) här.*

```
1766  21 07 00  FAOVFL: LD   HL,$0007    platsen före d1
1769  39              ADD  HL,SP
176A  7E              LD   A,(HL)
176B  B7              OR   A             minne dit?
176C  28 1C           JR   Z,FAOUT       nej: från d1
176E  06 01           LD   B,$01         x >= 1: en heltalssiffra till
1770  DD 7E 0D        LD   A,(IX+$0D)    1 om x < 1
1773  B7              OR   A
1774  28 02           JR   Z,FAOVF1
1776  06 FF           LD   B,$FF         x < 1: en nolla mindre
1778  DD 7E 02  FAOVF1: LD  A,(IX+$02)   nollor = 0?
177B  B7              OR   A
177C  20 05           JR   NZ,FAOVF2
177E  DD 77 0D        LD   (IX+$0D),A    nu x >= 1
1781  06 01           LD   B,$01         en heltalssiffra
1783  80        FAOVF2: ADD  A,B         IX+2 + B
1784  DD 77 02        LD   (IX+$02),A    rätt exponent för E-format
1787  1C              INC  E             fel: E + 1 även när B = -1 (ny ROM: LD E,A) (fel)
1788  0C              INC  C             en siffra till
1789  2B              DEC  HL            börja vid IX+5
178A  23        FAOUT:  INC  HL          första siffran
178B  79              LD   A,C           högst 6 siffror
```

```
178C  FE 07              CP   $07
178E  20 01              JR   NZ,FASIGN
1790  0D                 DEC  C
```

FASIGN: *B = E (heltalssiffror eller nollor), DE = bufferten. -, " " eller inget (bit 5).*

```
1791  DD 7E 01     FASIGN: LD   A,(IX+$01)      tecknet
1794  43                   LD   B,E             B = heltalssiffror/nollor
1795  D1                   POP  DE              bufferten
1796  1F                   RRA                  CY = negativt
1797  3E 2D                LD   A,$2D           -
1799  38 09                JR   C,FASGN1
179B  DD 7E 00             LD   A,(IX+$00)      bit 5?
179E  E6 20                AND  $20
17A0  20 04                JR   NZ,FAFORM
17A2  3E 20                LD   A,$20           " "
17A4  12           FASGN1: LD   (DE),A
17A5  13                   INC  DE
17A6  DD CB 00 46  FAFORM: BIT  0,(IX+$00)      E-format?
17AA  20 37                JR   NZ,FAEOUT
17AC  DD CB 0D 46          BIT  0,(IX+$0D)      x < 1?
17B0  28 22                JR   Z,FAINT
17B2  79                   LD   A,C             inga siffror: 0
17B3  B7                   OR   A
17B4  20 05                JR   NZ,FADOT
17B6  CD 61 17             CALL WRZERO
17B9  18 13                JR   FAEND

17BB  3E 2E        FADOT:  LD   A,$2E           .
17BD  12                   LD   (DE),A
17BE  13                   INC  DE
17BF  AF           FALEAD: XOR  A               B nollor
17C0  B0                   OR   B
17C1  28 06                JR   Z,FAFRAC
17C3  CD 61 17             CALL WRZERO          nollor efter punkten
17C6  05                   DEC  B
17C7  20 F6                JR   NZ,FALEAD
17C9  CD 15 18     FAFRAC: CALL WRDIG           C siffror
17CC  20 FB                JR   NZ,FAFRAC
17CE  21 0E 00     FAEND:  LD   HL,$000E        ta bort arbetsytan
17D1  39                   ADD  HL,SP
17D2  F9                   LD   SP,HL
17D3  C9                   RET
```

FAINT: *x >= 1. B heltalssiffror (nollor om siffrorna tar slut), sedan . och resten om det finns fler (C > 0).*

```
17D4  CD 15 18     FAINT:  CALL WRDIG           heltalsdelen
17D7  05                   DEC  B
17D8  20 FA                JR   NZ,FAINT
17DA  79                   LD   A,C             fler siffror?
17DB  B7                   OR   A
17DC  28 F0                JR   Z,FAEND
17DE  FA CE 17             JP   M,FAEND
17E1  18 D8                JR   FADOT
```

FAEOUT: *E-format. d1, . och resten om det finns fler,* E *och exponenten med tecken: x >= 1 ger +(IX+2 - 1), x < 1 ger -(IX+2 + 1).*

```
17E3  CD 15 18     FAEOUT: CALL WRDIG           d1
17E6  28 09                JR   Z,FAEXPN        bara en siffra
17E8  3E 2E                LD   A,$2E           .
```

```
17EA  12              LD   (DE),A
17EB  13              INC  DE
17EC  CD 15 18  FAEDGT: CALL WRDIG          resten
17EF  20 FB           JR   NZ,FAEDGT
17F1  3E 45     FAEXPN: LD   A,$45          E
17F3  12              LD   (DE),A
17F4  13              INC  DE
17F5  DD 7E 0D        LD   A,(IX+$0D)       x < 1?
17F8  B7              OR   A
17F9  28 0A           JR   Z,FAEPOS
17FB  3E 2D           LD   A,$2D            -
17FD  12              LD   (DE),A
17FE  13              INC  DE
17FF  DD 7E 02        LD   A,(IX+$02)       nollor + 1
1802  3C              INC  A
1803  18 08           JR   FAEPRN

1805  3E 2B     FAEPOS: LD   A,$2B          +
1807  12              LD   (DE),A
1808  13              INC  DE
1809  DD 7E 02        LD   A,(IX+$02)       heltalssiffror - 1
180C  3D              DEC  A
180D  6F        FAEPRN: LD   L,A
180E  26 00           LD   H,$00
1810  CD 62 18        CALL PRNUM            exponenten
1813  18 B9           JR   FAEND
```

`WRDIG`: *skriv siffran vid HL som ASCII vid DE, HL+1, DE+1, C-1 (Z när C = 0). Förstör A.*

```
1815  7E        WRDIG:  LD   A,(HL)
1816  C6 30             ADD  A,$30
1818  12                LD   (DE),A
1819  13                INC  DE
181A  23                INC  HL
181B  0D                DEC  C
181C  C9                RET
```

`GETLN1`: *HL+1 och hoppa över mellanslag, sedan* `GETLN`.

```
181D  DF        GETLN1: RST  $18
```

`GETLN`: *läs ett decimalt heltal (radnummer) vid HL till DE.*

Ut: CY = 1 om det inte börjar med en siffra eller om talet går över 65535. HL står då kvar. Annars CY = 0 och HL efter talet.

Mellan siffrorna hoppas mellanslag över (`RST $18`*), så* `1 0` *= 10, och efter talet står HL på nästa tecken som inte är blankt. Vid spill har DE värdet före den siffra som gav spillet (*`MUL10A` *ändrar inte DE då). Förstör A.*

```
181E  7E        GETLN:  LD   A,(HL)          första tecknet
181F  11 00 00          LD   DE,$0000       DE = 0
1822  FE 30            CP   $30
1824  D8                RET  C              inte en siffra: CY
1825  FE 3A             CP   $3A
1827  3F                CCF
1828  D8                RET  C              över 9: CY
1829  E5                PUSH HL             spara HL för spill
182A  7E        GETLP:  LD   A,(HL)
182B  D6 30             SUB  $30
182D  38 04             JR   C,GETEND       inte en siffra: klart
182F  FE 0A             CP   $0A            0-9?
1831  38 04             JR   C,GETDIG
```

```
1833  33           GETEND: INC  SP              släng sparat HL
1834  33                   INC  SP
1835  A7                   AND  A               CY = 0
1836  C9                   RET

1837  CD 00 3F     GETDIG: CALL MUL10A          DE = DE*10 + siffra, CY vid spill
183A  38 03                 JR   C,GETOVF       spill
183C  DF                    RST  $18            nästa, mellanslag hoppas över
183D  18 EB                 JR   GETLP

183F  E1           GETOVF: POP  HL              HL tillbaka, CY
1840  C9                   RET
```

GETSIN: heltal med tecken: mellanslag hoppas över, sedan + eller - och GETLN. In: HL = texten. Ut: DE, HL efter talet, CY = 1 om det inte är ett tal. - ger DE = -DE, men intervallet kontrolleras bara som 0..65535, så `40000` *blir -25536 och* `-40000` *blir 25536. Blanka får stå före och efter tecknet. Kommer det ingen siffra efter tecknet står HL efter det. Förstör A.*

```
1841  E7           GETSIN: RST  $20             hoppa över mellanslag
1842  FE 2B                CP   $2B             +
1844  28 D7                JR   Z,GETLN1        +: som utan tecken
1846  FE 2D                CP   $2D             -
1848  20 D4                JR   NZ,GETLN
184A  CD 1D 18             CALL GETLN1          talet
184D  D8                   RET  C               ingen siffra: HL efter tecknet
184E  EB                   EX   DE,HL
184F  CD EE 2A             CALL NEGHL           DE = -DE
1852  EB                   EX   DE,HL
1853  A7                   AND  A               CY = 0
1854  C9                   RET
```

PRSIN: skriv heltalet HL med tecken vid DE: mellanslag eller -, sedan siffrorna (PRNUM). -32768 blir `-32768` *(NEGHL ger samma tal, och PRNUM räknar utan tecken). Ut: DE efter texten. Förstör A, BC och HL.*

```
1855  CB 7C        PRSIN:  BIT  7,H             negativt?
1857  3E 20                LD   A,$20           mellanslag
1859  28 05                JR   Z,PRSGN
185B  CD EE 2A             CALL NEGHL           HL = -HL
185E  3E 2D                LD   A,$2D           -
1860  12           PRSGN:  LD   (DE),A          tecknet först
1861  13                   INC  DE
```

PRNUM: skriv HL decimalt som ASCII vid DE, utan inledande nollor och utan tecken (0..65535). Dividerar med 10000, 1000, 100 och 10 genom upprepad subtraktion. Ut: DE efter texten. Förstör A, BC och HL.

```
1862  AF           PRNUM:  XOR  A               inga siffror ännu
1863  01 F0 D8             LD   BC,$D8F0        -10000
1866  CD 7D 18             CALL PRDIG
1869  01 18 FC             LD   BC,$FC18        -1000
186C  CD 7D 18             CALL PRDIG
186F  01 9C FF             LD   BC,$FF9C        -100
1872  CD 7D 18             CALL PRDIG
1875  0E F6                LD   C,$F6           BC = $FFF6 = -10 (B är redan $FF)
1877  CD 7D 18             CALL PRDIG
187A  7D                   LD   A,L             entalssiffran skrivs alltid
187B  18 0C                JR   PRDIG1
```

PRDIG: E = HL / -BC genom upprepad addition av BC (negativt), och HL = resten. A är 0 så länge ingen siffra har skrivits, så inledande nollor hoppas över (`RET Z`*). Efter en skriven siffra är A =* `$30`*, och* `OR E` *ger*

siffran även när E = 0. Ut: HL = resten, DE efter siffran (oförändrat om den hoppades över), A = $30 efter en skriven siffra, annars 0. Förstör inte BC.

```
187D  D5          PRDIG:  PUSH DE          pekaren i texten
187E  1E FF               LD   E,$FF       E = -1
1880  1C          L1880:  INC  E           räkna
1881  09                  ADD  HL,BC       HL - divisorn
1882  38 FC               JR   C,L1880     tills HL blir negativt
1884  ED 42               SBC  HL,BC       en gång för mycket: lägg tillbaka
1886  B3                  OR   E           inledande nolla?
1887  D1                  POP  DE
1888  C8                  RET  Z           hoppa över
1889  F6 30       PRDIG1: OR   $30         ASCII
188B  12                  LD   (DE),A      skriv siffran
188C  13                  INC  DE
188D  3E 30               LD   A,$30       nu skrivs även nollor
188F  C9                  RET
```

Korsreferenstabell för talen som text. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. FLTASC tar HL = talets exponent, DE = bufferten och A = formatet (FLTAS6 sätter A = $0C), har sin arbetsyta på 14 byte vid IX på stacken och tar bort den igen; etiketterna inne i den (FAEXP till FAEPRN) är delar av den och får inga egna rader. WRDIG svarar med Z när C blir 0. GETLN, GETLN1 och GETSIN läser vid HL och lämnar talet i DE, med CY om det inte börjar med en siffra eller om det går över 65535. PRSIN, PRNUM och PRDIG skriver vid DE och lämnar DE efter texten; PRDIG tar BC = −divisorn och A = 0 så länge ingen siffra har skrivits. Rutinerna rör inga systemvariabler och ger inga fel. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
FLTAS6	X	X	X	IU	IX	X	
FLTASC	IX	X	X	IU	IX	X	
WRZERO	X			IU			
WRDIG	X		IU	IU	IU		
GETLN1	X	U		U	IU		
GETLN	X	U		U	IU		
GETSIN	X	U		U	IU		
PRSIN	X	X	X	IU	IX		
PRNUM	X	X	X	IU	IX		
PRDIG	IU	X	I	IU	IU		

Rutin	**Anropar**	**Anropas från**
FLTAS6	går in i FLTASC	PRTEXP, EXNUM
FLTASC	WRZERO, WRDIG, PRNUM	FLTAS6 (går in i den), DETEXP ($1305)
WRZERO		FLTASC (tre ställen)
WRDIG		FLTASC (fyra ställen)
GETLN1	RST $18; går in i GETLN	GETSIN (med JR och CALL)
GETLN	MUL10A, RST $18	CMDED, CMDLIST (två ställen), CMDREN (två ställen), GETFLT, GETSIN (med JR), TOKLIN, CMPREF, CMPON, CMPNUM
GETSIN	RST $20, GETLN1, GETLN (med JR), NEGHL	RDVAL
PRSIN	NEGHL; går in i PRNUM	PRTEXP ($220B)

Rutin	Anropar	Anropas från
PRNUM	PRDIG (också $1889, med JR)	ERRMSG (två ställen), DETOK, DETEXP ($1314), FLTASC, TRCPRT, PRSIN (går in i den)
PRDIG		PRNUM (fyra ställen, och till $1889 med JR)

3.34 TOKLIN: gör om en inskriven rad till intern kod $1890

In: HL = texten (slutar med $0D).

Ut: IX = $FEB8, där raden ligger i samma format som i programmet: [längd][nr lo][nr hi][kod ...][$0D]. HL = längden, utom när raden bara är ett nummer: då står HL vid CR och längden 0 i IX+0. Båda anroparna läser raden genom IX.

CY + A = felkod vid fel.

Utan radnummer blir numret 0 (direktkommando). Om det bara finns ett radnummer är längden 0 (raden ska tas bort).

Numret läses direkt vid HL, utan blanka före (kommandoslingan har redan hoppat över dem). Numret 0 är detsamma som inget nummer, så `0 PRINT 7` körs direkt. Ett nummer över 65535 räknas inte som nummer: texten kompileras som en sats och ger fel 11.

Satserna, åtskilda av ":", kompileras en i taget av TOKSTM. ":" följer med i koden. Efter den sista måste $0D komma, annars fel 14.

Förstör A, BC och DE.

```
1890  CD 1E 18     TOKLIN: CALL GETLN          radnumret vid HL till DE; CY om inget
1893  30 03                JR   NC,L1898       ett nummer
1895  11 00 00             LD   DE,$0000       inget nummer: 0
1898  E5           L1898:  PUSH HL             texten efter numret
1899  21 B8 FE             LD   HL,$FEB8       bygg raden vid $FEB8
189C  E5                   PUSH HL
189D  36 00                LD   (HL),$00       längden 0 tills vidare
189F  CD 22 10             CALL WRDE1          radnumret efter längden
18A2  EB                   EX   DE,HL          DE = $FEBB, där koden börjar
18A3  E1                   POP  HL
18A4  E3                   EX   (SP),HL        HL = texten, $FEB8 på stacken
18A5  7E                   LD   A,(HL)
18A6  FE 0D                CP   $0D            bara ett radnummer?
18A8  20 03                JR   NZ,L18AD
18AA  DD E1                POP  IX             IX = $FEB8, längd 0
18AC  C9                   RET                 NC: raden ska tas bort

18AD  CD C7 18     L18AD:  CALL TOKSTM         satserna, DE efter koden
18B0  DD E1                POP  IX             IX = $FEB8
18B2  D8                   RET  C              fel i en sats: CY, A = felkoden
18B3  7E                   LD   A,(HL)
18B4  FE 0D                CP   $0D            måste sluta med $0D
18B6  3E 8E                LD   A,$8E          fel 14
18B8  37                   SCF
18B9  C0                   RET  NZ
18BA  DD E5                PUSH IX             radens början
18BC  ED A0                LDI                 $0D följer med
18BE  EB                   EX   DE,HL          HL efter CR i raden
18BF  A7                   AND  A              CY = 0
18C0  C1                   POP  BC             BC = $FEB8
```

```
18C1  ED 42              SBC  HL,BC          längden
18C3  DD 75 00           LD   (IX+$00),L     längden i första byten, och i HL
18C6  C9                 RET                 NC
```

TOKSTM: kompilera satserna med TOKKW tills det efter en sats inte står :. Kolonet kopieras till koden, och blanka efter det hoppas över. In: HL = texten, DE = var koden skrivs. Ut: NC, HL på tecknet efter den sista satsen och DE efter koden; CY och A = felkoden om en sats var fel.

```
18C7  CD D4 18     TOKSTM: CALL TOKKW          en sats
18CA  D8                   RET  C              fel
18CB  7E                   LD   A,(HL)
18CC  EE 3A                XOR  $3A            :?
18CE  C0                   RET  NZ             inget :: klart, NC
18CF  ED A0                LDI                 : följer med i koden
18D1  E7                   RST  $20            blanka efter :
18D2  18 F3                JR   TOKSTM         nästa sats
```

3.35 TOKKW: kompilera en sats $18D4

Nyckelordet letas upp med KWSRCH, först i STKW1 och sedan i STKW2.

```
STKW1:  token $80-$96 skrivs, rutinen tas ur STTAB1
STKW2:  $86 och sedan $80+k (k = plats i STKW2), rutinen tas ur STTAB2
inget:  $83 (underförstådd LET) och LET-rutinen. Om det inte
        heller är en tilldelning blir felet 11, FÖRSTÅR EJ, i
        stället för 15.
```

Rutinen hämtas ur tabellen med 2 × token; för token $80–$96 blir det 2 × (token - $80), eftersom biten som skiftas ut försvinner. Rutinen anropas med HL = texten efter nyckelordet och DE = var koden ska skrivas. Den ger CY och felkod om satsen är fel.

In: HL = satsens första tecken, DE = var koden skrivs. Ut: det som rutinen ger, NC med HL på tecknet efter satsen och DE efter koden, eller CY och A = felkoden. Rutinens RET går direkt till anroparen av TOKKW. Förstör A och BC, och oftast IX.

```
18D4  D5           TOKKW:  PUSH DE             var koden ska skrivas
18D5  11 11 19             LD   DE,STKW1       sök först i tabell 1
18D8  CD F5 09             CALL KWSRCH         Z: A = token, HL efter ordet
18DB  01 08 1A             LD   BC,STTAB1      rutinerna till tabell 1 (LD ändrar inte Z)
18DE  28 22                JR   Z,L1902        funnen i STKW1
18E0  11 9C 19             LD   DE,STKW2       sedan i tabell 2
18E3  CD F5 09             CALL KWSRCH         Z: A = $80 + k
18E6  01 36 1A             LD   BC,STTAB2      rutinerna till tabell 2
18E9  D1                   POP  DE             DE = koden
18EA  EB                   EX   DE,HL          HL = koden, DE = texten
18EB  28 0F                JR   Z,L18FC        funnen i STKW2
18ED  36 83                LD   (HL),$83       $83: underförstådd LET
18EF  EB                   EX   DE,HL          HL = texten igen
18F0  13                   INC  DE             efter $83
18F1  CD 89 1B             CALL CMPLET         en tilldelning?
18F4  D0                   RET  NC             en tilldelning: klart
18F5  FE 8F                CP   $8F            fel 15 blir fel 11
18F7  37                   SCF
18F8  C0                   RET  NZ             andra fel som de är
18F9  3E 8B                LD   A,$8B          fel 11, FÖRSTÅR EJ
18FB  C9                   RET

18FC  36 86        L18FC:  LD   (HL),$86       $86 + token i STKW2
18FE  23                   INC  HL
18FF  EB                   EX   DE,HL          DE = koden, HL = texten
1900  18 01                JR   L1903          skriv $80 + k
```

```
1902  D1          L1902:  POP  DE          DE = koden
1903  12          L1903:  LD   (DE),A      token i koden
1904  13                  INC  DE
1905  E5                  PUSH HL          spara texten
1906  87                  ADD  A,A         2 × token; bit 7 faller bort
1907  6F                  LD   L,A
1908  26 00               LD   H,$00       HL = 2k
190A  09                  ADD  HL,BC       platsen i tabellen
190B  4E                  LD   C,(HL)      BC = rutinen
190C  23                  INC  HL
190D  46                  LD   B,(HL)
190E  E1                  POP  HL          HL = texten efter nyckelordet
190F  C5                  PUSH BC          hoppa till rutinen
1910  C9                  RET              rutinens RET går till TOKSTM
```

Korsreferenstabell för kompileringen av en rad. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken; inga systemvariabler används. TOKLIN bygger raden i bufferten vid $FEB8 och lämnar IX där. Det som satsens kompileringsrutin gör räknas med: TOKKW hoppar till den genom STTAB1 eller STTAB2 med PUSH och RET, och rutinens RET går till anroparen av TOKKW. CMD och NAME går till tilläggs-ROM:en ($0147, $0151), vars verkan inte räknas med. Tabellerna STKW1, STKW2, KWLIST, KWTHEN, KWELSE, KWSTEP, KWTO, KWASF och KWAS läses av KWSRCH och DETOK och har inga egna rader. Felen: TOKLIN 14, TOKKW 11 (i stället för 15 när ingen tilldelning stämmer) och de fel som satsens rutin ger. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
TOKLIN	U	U	X	X	IU	U	
TOKSTM	U	U	X	IU	IU	X	
TOKKW	U	U	X	IU	IU	X	

Rutin	**Anropar**	**Anropas från**
TOKLIN	GETLN, WRDE1, TOKSTM; fel 14	CMDLP, CMDED
TOKSTM	TOKKW	TOKLIN, CMPIF (THEN- och ELSE-delen)
TOKKW	KWSRCH, CMPLET; satsens rutin ur STTAB1 eller STTAB2 (med RET); fel 11	TOKSTM

STKW1: *satsernas nyckelord, tabell 1.*

Varje post är token (bit 7 satt) följd av ordet, och tabellen slutar med $FF. *Tabellen används på två sätt:*

```
    * KWSRCH (inmatning) jämför texten med orden i tur och ordning och
      tar det första som stämmer. Därför står INPUTLINE före INPUT
      och ONERRORGOTO före ON. KWSRCH hoppar över
      blanktecken, så "ON ERROR GOTO" och "INPUT LINE" stämmer.
      Ordet behöver bara vara början av texten: i "LETTER=1"
      hittas LET, variabeln blir T och E står där "=" ska stå: fel 15.
    * DETOK (listning) letar upp token med CPIR från STKW1 och framåt,
      och texten slutar vid nästa byte med bit 7. Därför kan tokens
      efter $FF (KWLIST, THEN, ELSE ...) också listas.
```

; *ger samma token som* PRINT, *men listas som* ; ($85).

```
1911  82 4C 45 54  STKW1:  DEFB $82,"LET"
1915  80 47 4F 54…         DEFB $80,"GOTO"       GOTO
191A  84 50 52 49…         DEFB $84,"PRINT"
```

```
1920  85 3B              DEFB $85,";"          ; = PRINT
1922  88 49 46           DEFB $88,"IF"
1925  8A 49 4E 50…       DEFB $8A,"INPUTLINE"  före INPUT
192F  89 49 4E 50…       DEFB $89,"INPUT"
1935  8B 46 4F 52        DEFB $8B,"FOR"
1939  8C 4E 45 58…       DEFB $8C,"NEXT"
193E  8E 52 45 41…       DEFB $8E,"READ"
1943  8F 52 45 53…       DEFB $8F,"RESTORE"
194B  90 47 4F 53…       DEFB $90,"GOSUB"
1951  91 52 45 54…       DEFB $91,"RETURN"
1958  94 4F 4E 45…       DEFB $94,"ONERRORGOTO"
                                               före ON
1964  92 4F 4E           DEFB $92,"ON"
1967  93 44 41 54…       DEFB $93,"DATA"
196C  96                 DEFB $96              DEF FN
196D  44 45 46 46… KWDEFN: DEFB "DEFFN"        listas via LD HL,KWDEFN ($120B)
1972  FF                 DEFB $FF              slut på sökningen
```

KWLIST: *tokens som bara listas.* $81 (GOTO *efter* THEN/ELSE) *och* $83 (LET *utan ordet*) *har ingen text.* $95 *är en rad som inte gick att tokenisera* ($0BEB)*: den sparas som* $95 *och den råa texten, och listas som* ??? *följt av texten.*

```
1973  81 83 95 3F… KWLIST: DEFB $81,$83,$95,"??? "
                                               $81, $83: ingen text; $95: felaktig rad
```

Tabeller med ett ord var, som KWSRCH *söker i under kompileringen:* THEN ($1E00), ELSE ($1E22, $2196), STEP ($1C9F), TO ($1C89) *och ASFILE* ($2062, OPEN/PREPARE)*. Tokens* $BA-$BD *och* $B8 *står sedan i koden och listas genom* STKW1*-tabellen.*

```
197A  BD 54 48 45… KWTHEN: DEFB $BD,"THEN"
197F  FF                   DEFB $FF
1980  BC 45 4C 53… KWELSE: DEFB $BC,"ELSE"
1985  FF                   DEFB $FF
1986  BB 53 54 45… KWSTEP: DEFB $BB,"STEP"
198B  FF                   DEFB $FF
198C  BA 54 4F     KWTO:   DEFB $BA,"TO"
198F  FF                   DEFB $FF
1990  B8 41 53 46… KWASF:  DEFB $B8,"ASFILE"
1997  FF                   DEFB $FF
```

AS ($B7)*: ingen söker i den här tabellen, och* $B7 *i uttryckskoden betyder tilldelning (typ 1), inte* AS*. Posten verkar oanvänd (?).*

```
1998  B7 41 53     KWAS:   DEFB $B7,"AS"
199B  FF                   DEFB $FF
```

STKW2*: satsernas nyckelord, tabell 2. Koden* $80 + *k skrivs efter token* $86*, och k är index i* STTAB2 *och* STEXE2*. Koderna* $83*,* $8C *och* $8D *saknar ord.*

```
199C  80 44 49 4D  STKW2:  DEFB $80,"DIM"
19A0  81 50 4F 4B…         DEFB $81,"POKE"
19A5  82 4F 55 54          DEFB $82,"OUT"
19A9  84 52 45 4D          DEFB $84,"REM"
19AD  85 4F 50 45…         DEFB $85,"OPEN"
19B2  86 50 52 45…         DEFB $86,"PREPARE"
19BA  87 43 4C 4F…         DEFB $87,"CLOSE"
19C0  88 52 41 4E…         DEFB $88,"RANDOMIZE"
19CA  89 53 54 4F…         DEFB $89,"STOP"
19CF  8A 45 4E 44          DEFB $8A,"END"
19D3  8B 43 4D 44          DEFB $8B,"CMD"
```

```
19D7  8E 53 45 54…        DEFB $8E,"SETDOT"
19DE  8F 43 4C 52…        DEFB $8F,"CLRDOT"
19E5  90 47 45 54         DEFB $90,"GET"
19E9  91 43 48 41…        DEFB $91,"CHAIN"
19EF  92 4B 49 4C…        DEFB $92,"KILL"
19F4  93 4E 41 4D…        DEFB $93,"NAME"
19F9  94 54 52 41…        DEFB $94,"TRACE"
19FF  95 4E 4F 54…        DEFB $95,"NOTRACE"
1A07  FF                  DEFB $FF              slut
```

3.36 STTAB1/STTAB2: satsernas kompileringsrutiner $1A08

Index = token - $80 (STKW1) eller byten efter $86 - $80 (STKW2). Körrutinerna står i STEXE1/STEXE2 ($0E14) i samma ordning.

token	sats	kompilering	körning
$80	GOTO	CMPREF	GOTO
$81	(GOTO efter THEN/ELSE)	CMPREF	GOTO
$82	LET	CMPLET	LET
$83	(LET utan ordet)	CMPLET	LET
$84	PRINT	CMPPRT	PRINTS
$85	; (= PRINT)	CMPPRT	PRINTS
$86	tabell 2	-	EXEST2
$87	hopp (IF)	-	EXESKP
$88	IF	CMPIF	IF
$89	INPUT	CMPINP	INPT
$8A	INPUT LINE	CMPINL	INPLIN
$8B	FOR	CMPFOR	FOR
$8C	NEXT	CMPNXT	NEXT
$8D	(inget ord)	fel 8	fel 8
$8E	READ	CMPRD	READ
$8F	RESTORE	CMPRES	RESTOR
$90	GOSUB	CMPREF	GOSUB
$91	RETURN	NOCODE	RETURN
$92	ON	CMPON	ON
$93	DATA	CMPREM	DATA
$94	ON ERROR GOTO	CMPREF	ONERRG
$95	(felaktig rad)	-	fel 53
$96	DEF FN	CMPDEF	DEF

Tabell 2 ($86 k):

0	DIM	CMPDIM	DIM	1	POKE	CMPPOK	POKE
2	OUT	CMPOUT	OUTST	3	(inget ord)	CMPINX	fel 8
4	REM	CMPREM	REM	5	OPEN	CMPOPN	OPENST
6	PREPARE	CMPOPN	PREPST	7	CLOSE	CMPCLS	CLOSST
8	RANDOMIZE	CMPNOP	RANDOM	9	STOP	NOCODE	STOP
A	END	NOCODE	ENDPRG	B	CMD	tillägg	tillägg
C, D	(inget ord)	fel 8		E	SETDOT	CMPDOT	SETDOT
F	CLRDOT	CMPDOT	CLRDOT	10	GET	CMPGET	GET
11	CHAIN	CMPSTX	CHAIN	12	KILL	CMPSTX	tillägg
13	NAME	tillägg	tillägg	14	TRACE	NOCODE	TRACE
15	NOTRACE	NOCODE	NOTRAC				

”tillägg” = ingång till tilläggs-ROM:en (från $0147), fel 8 om den saknas. Kod 3, C och D har inga nyckelord och kan inte skrivas in; de är nog platser för satser som inte finns i denna version (?).

Alla kompileringsrutiner tar HL = texten efter nyckelordet och DE = var koden skrivs. De lämnar NC, HL på första tecknet efter satsen som inte är blankt (TOKSTM jämför det direkt med ”:”) och DE efter koden, eller CY och A = felkoden.

```
1A08  C7 1D     STTAB1: DEFW CMPREF     $80 GOTO
1A0A  C7 1D             DEFW CMPREF     $81 GOTO efter THEN/ELSE
1A0C  89 1B             DEFW CMPLET     $82 LET
1A0E  89 1B             DEFW CMPLET     $83 LET utan ordet
1A10  54 21             DEFW CMPPRT     $84 PRINT
1A12  54 21             DEFW CMPPRT     $85 ; (= PRINT)
1A14  00 00             DEFW RESET      $86 tabell 2: kompileras inte härifrån
1A16  00 00             DEFW RESET      $87 hoppet i IF: kompileras inte härifrån
1A18  FB 1D             DEFW CMPIF      $88 IF
1A1A  58 1E             DEFW CMPINP     $89 INPUT
1A1C  6B 1F             DEFW CMPINL     $8A INPUT LINE
1A1E  83 1C             DEFW CMPFOR     $8B FOR
1A20  B7 1C             DEFW CMPNXT     $8C NEXT
1A22  09 07             DEFW L0709      $8D inget ord: fel 8
1A24  6E 1E             DEFW CMPRD      $8E READ
1A26  81 22             DEFW CMPRES     $8F RESTORE
1A28  C7 1D             DEFW CMPREF     $90 GOSUB
1A2A  B5 22             DEFW NOCODE     $91 RETURN
1A2C  BF 1F             DEFW CMPON      $92 ON
1A2E  6B 22             DEFW CMPREM     $93 DATA
1A30  C7 1D             DEFW CMPREF     $94 ON ERROR GOTO
1A32  00 00             DEFW RESET      $95 felaktig rad: kompileras inte härifrån
1A34  62 1A             DEFW CMPDEF     $96 DEF FN
1A36  AA 1B     STTAB2: DEFW CMPDIM     0 DIM
1A38  04 21             DEFW CMPPOK     1 POKE
1A3A  FF 20             DEFW CMPOUT     2 OUT
1A3C  70 23             DEFW CMPINX     3 inget ord: ett heltalsuttryck, körs som fel 8
1A3E  6B 22             DEFW CMPREM     4 REM
1A40  5D 20             DEFW CMPOPN     5 OPEN
1A42  5D 20             DEFW CMPOPN     6 PREPARE
1A44  71 20             DEFW CMPCLS     7 CLOSE
1A46  6A 22             DEFW CMPNOP     8 RANDOMIZE
1A48  B5 22             DEFW NOCODE     9 STOP
1A4A  B5 22             DEFW NOCODE     $0A END
1A4C  47 01             DEFW L0147      $0B CMD: tilläggskretsen ($7000)
1A4E  09 07             DEFW L0709      $0C inget ord: fel 8
1A50  09 07             DEFW L0709      $0D inget ord: fel 8
1A52  B6 22             DEFW CMPDOT     $0E SETDOT
1A54  B6 22             DEFW CMPDOT     $0F CLRDOT
1A56  A8 23             DEFW CMPGET     $10 GET
1A58  6C 23             DEFW CMPSTX     $11 CHAIN
1A5A  6C 23             DEFW CMPSTX     $12 KILL
1A5C  51 01             DEFW L0151      $13 NAME: tilläggskretsen ($6054)
1A5E  B5 22             DEFW NOCODE     $14 TRACE
1A60  B5 22             DEFW NOCODE     $15 NOTRACE
```

3.37 CMPDEF: kompilera DEF FNx(p1,p2,...)=uttryck $1A62

Koden blir:

```
$96 [namnbyte][bokstav] [antal param] [2 byte  per param] "=" uttryck $B9+typ
```

Högst fyra parametrar (annars fel 13). Uttrycket omvandlas till funktionens typ (efter namnet: A = flyttal, A% = heltal). Parametrarnas namn kopplas vid förlänkningen (LNKDEF) till funktionens argument. En parameter döljer variabeln med samma namn bara i uttrycket; andra variabler är de vanliga. Finns två DEF för samma namn gäller den första. Ett heltalsargument avrundas nedåt som vid tilldelning (2.7 blir 2, -2.5 blir -3). En strängfunktion (FNA$) går att skriva in men ger fel 8 vid anropet (FN).

In: HL vid funktionens namn (efter DEFFN), DE = koden efter $96. Ut: som CMPTYX, NC med HL efter uttrycket och DE efter $B9 + typ. Förstör A, BC och IX.

```
1A62  CD CB 26     CMPDEF: CALL VARNAM          funktionens namn
1A65  D8                   RET  C               HL = koden, DE = texten
1A66  EB                   EX   DE,HL
1A67  CD 29 10             CALL WRBC            namnbyten och bokstaven efter $96
1A6A  36 00                LD   (HL),$00        antal parametrar = 0
1A6C  E5                   PUSH HL
1A6D  DD E1                POP  IX              IX = antalet
1A6F  23                   INC  HL              första parameterns plats
1A70  EB                   EX   DE,HL           DE = koden, HL = texten
1A71  7E                   LD   A,(HL)
1A72  FE 28                CP   $28             (?
1A74  20 22                JR   NZ,L1A98        inga parametrar
1A76  DF           L1A76:  RST  $18             förbi ( eller ,
1A77  CD CB 26             CALL VARNAM          parameterns namn
1A7A  D8                   RET  C
1A7B  EB                   EX   DE,HL
1A7C  CD 29 10             CALL WRBC            två byte i koden
1A7F  EB                   EX   DE,HL
1A80  DD 7E 00             LD   A,(IX+$00)      antalet hittills
1A83  FE 04                CP   $04             redan fyra?
1A85  3E 8D                LD   A,$8D           fel 13
1A87  37                   SCF
1A88  C8                   RET  Z
1A89  DD 34 00             INC  (IX+$00)        en parameter till
1A8C  7E                   LD   A,(HL)
1A8D  FE 2C                CP   $2C             ,
1A8F  28 E5                JR   Z,L1A76         nästa parameter
1A91  FE 29                CP   $29             )
1A93  3E 92                LD   A,$92           fel 18
1A95  37                   SCF
1A96  C0                   RET  NZ
1A97  DF                   RST  $18             förbi )
1A98  FE 3D        L1A98:  CP   $3D             =
1A9A  3E 8F                LD   A,$8F           fel 15
1A9C  37                   SCF
1A9D  C0                   RET  NZ
1A9E  ED A0                LDI                  = följer med
1AA0  E7                   RST  $20             uttryckets första tecken
1AA1  DD 7E FE             LD   A,(IX-$02)      (IX-2) = funktionens namnbyte
1AA4  E6 03                AND  $03             typen
1AA6  47                   LD   B,A
1AA7  C3 72 23             JP   CMPTYX          uttrycket, omvandlat till typen
```

DEF vid körning: hoppa över resten av raden. DE-4 är radens längd-byte om DEF står först på raden, och DE sätts på radens $0D. Resten av raden körs alltså inte (`10 DEF FNB(X)=X:PRINT 5` skriver inget). DEF FN får bara stå först på en rad: annars hittar förlänkningen den inte (anropet ger fel 57), och DEF hoppar till en felaktig adress (till exempel fel 4). I direktläge fel 2 (NODIR).

```
1AAA  CD BF 0D     DEF:    CALL NODIR           (egenhet)
1AAD  21 FC FF             LD   HL,$FFFC        DE-4 = radens längdbyte
1AB0  19                   ADD  HL,DE
1AB1  5E                   LD   E,(HL)          E = längden
1AB2  16 00                LD   D,$00
1AB4  19                   ADD  HL,DE           HL efter radens CR
1AB5  EB                   EX   DE,HL
1AB6  1B                   DEC  DE              DE på $0D
1AB7  C9                   RET
```

3.38 CMPFN: kompilera ett anrop FNx(a1,a2,...) i ett uttryck $1AB8

Varje argument kompileras som ett uttryck och följs av en fem byte lång post:

```
$DB [avstånd bakåt till förra $DB-posten, 0 för den första]
    [argumentets typ] [parameterns typ] [omvandling]
```

De två sista fylls i vid förlänkningen (LNKFN, $111D): parameterns typ, och $D3 (gör inget) eller $D9/$DA som omvandlar argumentet. Efter argumenten:

```
$CD+antal  $00 (funktionskod 0 = användarfunktion)
[namnbyte][bokstav] [2 byte: adress till DEF, fylls i vid
förlänkningen] [avstånd bakåt till sista $DB-posten]
```

Högst fyra argument (fel 13).

Avstånden gör att argumentposterna kan hittas bakåt från anropet. Ett avstånd är en byte, skillnaden mellan två adressers låga byte.

In: HL vid namnet efter FN, DE = koden. Medan argumenten kompileras ligger en ram vid SP (IX+0 antalet, IX+1 den låga byten av förra avståndets adress) och namnet ovanför. Ut: NC, HL efter anropet, DE efter koden och B = typen. Förstör A, C och IX.

```
1AB8  CD CB 26     CMPFN:  CALL VARNAM          funktionens namn i BC
1ABB  D8                   RET  C
1ABC  DD 21 00 00          LD   IX,$0000        ramen: antal 0
1AC0  DD E5                PUSH IX              IX+0 = antal arg, IX+1 = senaste $DB-post
1AC2  DD 39                ADD  IX,SP           IX = ramen, vid SP
1AC4  C5                   PUSH BC              BC = funktionens namn
1AC5  7E                   LD   A,(HL)
1AC6  FE 28                CP   $28             (?
1AC8  20 37                JR   NZ,L1B01        inga argument
1ACA  DF           L1ACA:  RST  $18             förbi ( eller ,
1ACB  DD E5                PUSH IX              CMPEXP ändrar IX
1ACD  CD 01 27             CALL CMPEXP          argumentet
1AD0  DD E1                POP  IX
1AD2  38 52                JR   C,L1B26         fel: släng namn och ram
1AD4  EB                   EX   DE,HL           HL = koden
1AD5  36 DB                LD   (HL),$DB        argumentpost $DB
1AD7  23                   INC  HL              avståndets plats
1AD8  DD 7E 00             LD   A,(IX+$00)      antal hittills
1ADB  A7                   AND  A
1ADC  28 0A                JR   Z,L1AE8         första: avståndet 0
1ADE  FE 04                CP   $04             redan fyra?
1AE0  3E 8D                LD   A,$8D           fel 13
1AE2  28 42                JR   Z,L1B26
1AE4  DD 7E 01             LD   A,(IX+$01)      förra avståndsbytens låga byte
1AE7  95                   SUB  L               avståndet bakåt, negativt
1AE8  77           L1AE8:  LD   (HL),A          avståndet, 0 först
1AE9  DD 75 01             LD   (IX+$01),L      kom ihåg den här
1AEC  DD 34 00             INC  (IX+$00)        ett argument till
1AEF  23                   INC  HL
1AF0  70                   LD   (HL),B          argumentets typ (B från CMPEXP)
1AF1  23                   INC  HL
1AF2  23                   INC  HL              parameterns typ och omvandlingen: LNKFN
1AF3  23                   INC  HL
1AF4  EB                   EX   DE,HL           DE efter posten
1AF5  7E                   LD   A,(HL)
1AF6  FE 2C                CP   $2C             ,?
1AF8  28 D0                JR   Z,L1ACA         nästa argument
1AFA  FE 29                CP   $29             )?
1AFC  3E 92                LD   A,$92           fel 18
1AFE  20 26                JR   NZ,L1B26
1B00  DF                   RST  $18             förbi )
```

```
1B01  EB           L1B01:  EX   DE,HL          HL = koden
1B02  3E CD                LD   A,$CD          $CD + antal argument
1B04  DD 86 00             ADD  A,(IX+$00)
1B07  77                   LD   (HL),A
1B08  23                   INC  HL
1B09  36 00                LD   (HL),$00       funktionskod 0
1B0B  23                   INC  HL
1B0C  C1                   POP  BC             namnet
1B0D  CD 29 10             CALL WRBC           namnet
1B10  23                   INC  HL             plats för DEF-adressen
1B11  23                   INC  HL
1B12  DD 7E 00             LD   A,(IX+$00)     antalet
1B15  A7                   AND  A
1B16  28 04                JR   Z,L1B1C        inga argument: 0
1B18  DD 7E 01             LD   A,(IX+$01)     sista avståndsbyten
1B1B  95                   SUB  L              avståndet bakåt
1B1C  77           L1B1C:  LD   (HL),A         sist i anropet
1B1D  23                   INC  HL
1B1E  EB                   EX   DE,HL          DE efter anropet
1B1F  DD E1                POP  IX             släng ramen
1B21  3E 03                LD   A,$03          B = funktionens typ, ur namnbyten
1B23  A1                   AND  C
1B24  47                   LD   B,A
1B25  C9                   RET                 NC (AND)

1B26  C1           L1B26:  POP  BC             släng namn och ram
1B27  C1                   POP  BC
1B28  37                   SCF
1B29  C9                   RET
```

3.39 FN vid körning (funktionskod 0 i EXPTAB2) $1B2A

1. STKCHK, sedan läses adressen till DEF-raden ur anropet.
2. Den gamla ramen (FNFRAM, $FE3E) sparas på stacken och FNFRAM pekar på den nya. Genom FNFRAM hittar parametrarna i DEF-uttrycket argumenten.
3. RST $38 räknar ut DEF-uttrycket. Därefter återställs FNFRAM.
4. Resultatet läggs på stacken som uttryckets värde: flyttal (5 byte) eller heltal. En strängfunktion ger fel 8, FINNS EJ I DETTA SYSTEM, alltså finns det inga strängfunktioner med DEF FN.

In: DE vid namnbyten i anropet, efter funktionskoden 0 (EXFUNC). Argumenten ligger på stacken, det sista vid SP. Under körningen ligger kodpekaren och den gamla ramen under argumenten, vid FNFRAM-2 och -4. Ut: värdet vid SP där argumenten låg, DE efter anropet, och RST $30 hämtar nästa token.

```
1B2A  CD 89 33     FN:     CALL STKCHK         fel 3 vid djup rekursion
1B2D  EB                   EX   DE,HL          HL = koden
1B2E  23                   INC  HL             förbi namnbyten
1B2F  CD 1C 10             CALL RDDE1          förbi bokstaven, DE = DEF-radens =
1B32  E5                   PUSH HL             kodpekaren, vid avståndsbyten
1B33  2A 3E FE             LD   HL,(FNFRAM)    spara ramen
1B36  E5                   PUSH HL
1B37  21 04 00             LD   HL,$0004       FNFRAM = det sista argumentet
1B3A  39                   ADD  HL,SP
1B3B  22 3E FE             LD   (FNFRAM),HL
1B3E  13                   INC  DE             förbi =: uttrycket
1B3F  FF                   RST  $38            räkna ut DEF-uttrycket, B = typen
```

```
1B40  78                LD   A,B            A = typen
1B41  ED 5B 3E FE       LD   DE,(FNFRAM)    DE = ramen
1B45  D5                PUSH DE             IX = ramen
1B46  DD E1             POP  IX
1B48  DD 5E FC          LD   E,(IX-$04)     IX-4: den gamla ramen
1B4B  DD 56 FD          LD   D,(IX-$03)
1B4E  ED 53 3E FE       LD   (FNFRAM),DE    återställ ramen
1B52  DD 5E FE          LD   E,(IX-$02)     DE = kodpekaren
1B55  DD 56 FF          LD   D,(IX-$01)
1B58  A7                AND  A              flyttal?
1B59  28 0C             JR   Z,L1B67
1B5B  3D                DEC  A              heltal?
1B5C  C2 09 07          JP   NZ,L0709       sträng: fel 8
1B5F  E5                PUSH HL             heltalet (i HL från $BA)
1B60  CD 7B 1B          CALL FNRES          HL ovanför argumenten, DE efter anropet
1B63  C1                POP  BC             BC = heltalet
1B64  F9                LD   SP,HL          bort med argument och ram
1B65  C5                PUSH BC             heltalet vid SP
1B66  F7                RST  $30            nästa token

1B67  CD 7B 1B    L1B67: CALL FNRES          HL ovanför argumenten, DE efter anropet
1B6A  D5                PUSH DE             kodpekaren undan
1B6B  EB                EX   DE,HL         DE = målet
1B6C  1B                DEC  DE             målets högsta byte
1B6D  21 06 00          LD   HL,$0006       HL = flyttalets högsta byte (exponenten)
1B70  39                ADD  HL,SP
1B71  01 05 00          LD   BC,$0005
1B74  ED B8             LDDR                flyttalet uppåt, över argumenten
1B76  EB                EX   DE,HL
1B77  23                INC  HL             HL = flyttalets nya plats
1B78  D1                POP  DE             kodpekaren
1B79  F9                LD   SP,HL          resultatet vid SP, argumenten borta
1B7A  F7                RST  $30            nästa token
```

FNRES: *hitta slutet på argumenten. HL = IX = ramen* (FNFRAM), *E =* $FF: ARGWLK *går igenom alla argumentposter och ger HL = första byten ovanför det första argumentet, där resultatet ska ligga.*

In: DE = kodpekaren vid avståndsbyten, IX = ramen. Ut: HL som ovan och DE = koden + 1 (efter avståndsbyten). Förstör A, BC och IX.

```
1B7B  13          FNRES: INC  DE            efter avståndsbyten
1B7C  D5                 PUSH DE
1B7D  1E FF              LD   E,$FF         gå igenom alla
1B7F  DD E5              PUSH IX
1B81  E1                 POP  HL
1B82  C3 B0 33           JP   ARGWLK
```

SKPARG ($DB): *en argumentpost i FN-anropet, vid körning. Argumentets värde ligger redan på stacken. Avståndet och de två typerna hoppas över, och omvandlingen* ($D3, $D9 *eller* $DA) *körs som nästa token. In: DE efter* $DB.

```
1B85  13          SKPARG: INC  DE           förbi avståndet
1B86  13                  INC  DE           argumentets typ
1B87  13                  INC  DE           parameterns typ
1B88  F7                  RST  $30          omvandlingen är nästa token
```

Korsreferenstabell för DEF FN och FN. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnen är DIRECT vid IY+$0E. CY-kolumnen gäller flaggorna. FN och SKPARG går inte tillbaka utan hämtar nästa token med RST $30; för dem visas bara stacken, variablerna och det de tar emot. FN räknar ut DEF-uttrycket med RST $38, och det som uttrycket gör räknas inte med; FNFRAM

sparas på stacken och återställs, och STKCHK läser HEAP (fel 3). Argumenten ligger på stacken med det sista vid FNFRAM, och resultatet hamnar där argumenten låg. CMPFN lägger en ram med antalet argument och namnet på stacken medan argumenten kompileras och tar bort den igen. Felen: CMPDEF 13, 15 och 18, CMPFN 13 och 18, FN 3 och 8 (en strängfunktion), DEF 2. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler		IY+
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	FNFRAM	$0E
CMPDEF	U	U	X	IU	IU	X				
DEF	X	X		IU	X					I
CMPFN	U	U	U	IU	IU	X				
FN*				I			IU	I	IU	
FNRES	X	X	X	IU	U	IX	I			
SKPARG*				I						

Rutin	**Anropar**	**Anropas från**
CMPDEF	VARNAM, WRBC, CMPTYX (med JP); fel 13, 15, 18	STTAB1 ($96)
DEF	NODIR	STEXE1 ($96)
CMPFN	VARNAM, CMPEXP, WRBC; fel 13, 18	CMPFNC
FN	STKCHK, RDDE1, RST38 (DEF-uttrycket), FNRES; fel 8; går vidare med RST $30	EXPTAB2 (kod 0)
FNRES	ARGWLK (med JP)	FN (två ställen)
SKPARG	går vidare med RST $30	EXPTAB1 ($DB)

3.40 LET: kompilering $1B89

[LET] v = uttryck (LET kan utelämnas, se TOKKW)

Koden blir ett enda uttryck som slutar med tilldelningen:

```
[index] [variabel, 5 byte] [uttryck] [$B6 + typ]
```

Uttrycket omvandlas till variabelns typ (TYPCNV): flyttal till heltal avrundas nedåt (FTOINT), A$=5 ger fel 13 och A%="Q" fel 17. Utan "=" blir det fel 15.

Tilldelningen ($B6–$B8) avslutar också körningen av uttrycket: den gör RET i stället för att hämta nästa token.

In: HL vid variabeln, DE = koden. Ut: NC, HL efter uttrycket, DE efter $B6 + typ och B = variabelns typ; vid fel CY och A. Förstör A, C och IX.

```
1B89  CD EE 25     CMPLET: CALL CMPVAR          variabeln, B = typen
1B8C  D8           L1B8C:  RET  C
1B8D  7E                   LD   A,(HL)
1B8E  FE 3D                CP   $3D             =?
1B90  3E 8F                LD   A,$8F           fel 15
1B92  37                   SCF
1B93  C0                   RET  NZ
1B94  C5                   PUSH BC              variabelns typ
1B95  DF                   RST  $18             förbi =
1B96  CD 01 27             CALL CMPEXP          uttrycket
1B99  38 5C                JR   C,CMPERR        fel: BC bort
1B9B  F1                   POP  AF              A = variabelns typ
```

```
1B9C  CD 51 25            CALL TYPCNV           till variabelns typ
1B9F  D8                  RET  C
1BA0  3E B6               LD   A,$B6            $B6 + typ: tilldelning
1BA2  C3 68 23            JP   PUTTOK
```

CMPFLV: som CMPLET, men variabeln får inte vara en sträng (fel 17) och inte ha index (då står (där = ska stå: fel 15). Används av FOR. In och ut som CMPLET.

```
1BA5  CD 7F 23     CMPFLV: CALL CMPSV
1BA8  18 E2                JR   L1B8C
```

3.41 DIM: kompilering $1BAA

DIM post [, post ...]

Varje post blir:

```
[index] [längd eller $D2] $CC [variabel, 5 byte]
```

Hela listan slutar med $BF. Tal ska ha 1 eller 2 index. En sträng kan ha 0–2 index och =längd efter (DIM A$(5)=20, DIM B$=40); utan längd skrivs $D2, som ger längden 0 vid körning. Indexen och längden är heltal. Ett tal utan index eller med tre index är tänkt att ge fel 13, men CMPDMI lämnar BC och AF på stacken och CMPERR tar bort bara det ena, så RET hoppar till variabelns namn som adress ($41F0 för A) och det som händer beror på minnet där (ofta fel 4, eftersom tomt minne är $FF = RST $38; 9913 ger 13). En längd efter ett tal eller något annat än "," mellan posterna ger fel 14.

In: HL efter DIM, DE = koden. Ut: NC, HL efter listan och DE efter $BF. Förstör A, BC och IX.

```
1BAA  CD BD 1B     CMPDIM: CALL CMPDMI           en post
1BAD  D8                   RET  C
1BAE  7E                   LD   A,(HL)           ,?
1BAF  EE 2C                XOR  $2C
1BB1  06 00                LD   B,$00
1BB3  20 03                JR   NZ,CMPBF         inget ,: slut på listan
1BB5  DF                   RST  $18              förbi ,
1BB6  18 F2                JR   CMPDIM           nästa post
```

Skriv $BF, slut på DIM-listan.

```
1BB8  3E BF        CMPBF:  LD   A,$BF
1BBA  12                   LD   (DE),A
1BBB  13                   INC  DE
1BBC  C9                   RET
```

CMPDMI: en post i DIM-listan. Först indexen med CMPARG och typlistan $2548 (sträng, 0 till 2 index) eller $254D (tal, 1 eller 2), för en sträng sedan längden efter = eller $D2, och sist $CC och variabeln (via slutet av CMPVAR, $25FC). In: HL vid namnet, DE = koden. Ut: NC, HL efter posten och DE efter koden; vid fel CY och A, men ett fel i indexen går fel väg (se CMPERR).

```
1BBD  CD CB 26     CMPDMI: CALL VARNAM           namnet i BC (fel)
1BC0  D8                   RET  C
1BC1  3E 03                LD   A,$03
1BC3  A1                   AND  C
1BC4  EE 02                XOR  $02              Z = sträng
1BC6  C5                   PUSH BC               variabelns namn
1BC7  DD 21 48 25          LD   IX,$2548         sträng: 0-2 index
1BCB  28 04                JR   Z,L1BD1          Z: sträng
1BCD  DD 21 4D 25          LD   IX,$254D         tal: 1-2 index
1BD1  F5           L1BD1:  PUSH AF               flaggan för sträng
```

```
1BD2  CD 4E 26            CALL  CMPARG        indexen, C = antalet
1BD5  38 20               JR    C,CMPERR      fel: BC och AF på stacken
1BD7  F1                  POP   AF            Z = sträng
1BD8  20 16               JR    NZ,L1BF0      tal: ingen längd
1BDA  7E                  LD    A,(HL)
1BDB  FE 3D               CP    $3D           = längd?
1BDD  28 06               JR    Z,L1BE5
1BDF  3E D2               LD    A,$D2         $D2: längd 0
1BE1  12                  LD    (DE),A
1BE2  13                  INC   DE
1BE3  18 0B               JR    L1BF0

1BE5  DF          L1BE5:  RST   $18           förbi =
1BE6  C5                  PUSH  BC            C = antal index
1BE7  CD 01 27            CALL  CMPEXP        längden
1BEA  D4 61 25            CALL  NC,TOINT      längden är ett heltal
1BED  C1                  POP   BC
1BEE  38 07               JR    C,CMPERR      fel: namnet bort
1BF0  3E CC       L1BF0:  LD    A,$CC         $CC: DIM-post
1BF2  12                  LD    (DE),A
1BF3  13                  INC   DE
1BF4  C3 FC 25            JP    L25FC         variabeln med C index, namnet från stacken
```

CMPERR: fel under kompileringen. Tar bort ett sparat ord och sätter CY; A är felkoden. Det stämmer för CMPLET och för längden i CMPDMI, där bara variabelns namn ligger vid SP. Efter indexen i CMPDMI ligger också flaggorna där: bara de tas bort, och RET hoppar till namnet som adress ($41F0 för A).

```
1BF7  C1          CMPERR: POP   BC            flaggorna eller namnet
1BF8  37                  SCF
1BF9  C9                  RET                 till namnet i BC som adress: $41F0 för DIM A
```

DIM vid körning: fel 2 i direktläge. Sedan körs koden som ett uttryck: indexen och längden hamnar på stacken, och $CC (DIMELM) dimensionerar variabeln. $BF avslutar.

```
1BFA  CD BF 0D    DIM:    CALL  NODIR
```

LET vid körning: RST $30 kastar returadressen från RST och kör koden vid DE som ett uttryck (EXEXP). Tilldelningen sist gör RET till satsens anropare, med DE efter koden.

```
1BFD  F7          LET:    RST   $30
```

3.42 DIMELM: $CC i uttryckskoden, dimensionera en variabel $1BFE

Efter $CC kommer variabeln (5 byte); IX = dess post. Värdena på stacken, från SP och uppåt: [längd (strängar)] [index 2] [index 1]. In: DE efter $CC. Ut: värdena borta från stacken, och RST $30 hämtar nästa token med DE efter variabeln.

Bit 2–3 i namnbyten (antal index) väljer fall:

```
0 index   sträng: DIMSTR med längden
1 index   DIM1, sedan varje element (6 byte) som en sträng
2 index   DIM2, sedan likadant
```

Tal behöver inte någon längd. En strängvektor med längden 0 får bara sina element, och texten reserveras när något tilldelas: 80 byte, eller textens längd om den är längre (ASGNS). Högsta index 0 lämnar +2/3 = 0, det vill säga odimensionerad, och första användningen ger DIM (10) (VAR1). Högsta index -1 ger 0 byte, och då nollar ALLOC hela minnet (BC = 0 - 1): återhoppet går till $0000, kallstart.

En post för en vektor:

```
+0/1 adress till elementen     +2/3 antal element - 1
+4   elementets storlek        +5   offset till värdet
+6/7 första indexets max       +8/9 andra indexets max (DIM2)
```

En post för en sträng (även varje element i en strängvektor):

```
+0/1 reserverad längd   +2/3 adress till texten   +4/5 längd
```

```
1BFE  EB          DIMELM: EX   DE,HL          HL = koden (fel)
1BFF  23                  INC  HL             förbi variabelns token
1C00  4E                  LD   C,(HL)         namnbyte
1C01  23                  INC  HL
1C02  CD 1C 10            CALL RDDE1          förbi bokstaven, DE = posten
1C05  D5                  PUSH DE
1C06  DD E1               POP  IX
1C08  E5                  PUSH HL             kodpekaren efter variabeln
1C09  3E 0C               LD   A,$0C          antal index
1C0B  A1                  AND  C
1C0C  28 56               JR   Z,L1C64        inget index
1C0E  FE 04               CP   $04
1C10  28 3C               JR   Z,L1C4E        ett index
1C12  CD 6C 1C            CALL ELMSIZ         två index
1C15  C1                  POP  BC             BC = koden
1C16  E1                  POP  HL             tal: HL = andra index (sträng: längden)
1C17  D1                  POP  DE             tal: DE = första index (sträng: andra)
1C18  20 06               JR   NZ,L1C20       sträng?
1C1A  C5                  PUSH BC             koden undan
1C1B  CD 11 33            CALL DIM2           tal: DIM2
1C1E  D1                  POP  DE             DE = koden
1C1F  F7                  RST  $30            nästa token

1C20  E3          L1C20:  EX   (SP),HL        HL = första index, längden vid SP
1C21  C5                  PUSH BC             koden
1C22  EB                  EX   DE,HL          DE = första, HL = andra
1C23  CD 11 33            CALL DIM2
```

Strängvektor: gå igenom alla (+2/3 plus 1) element, 6 byte var, och reservera längden DE för varje (DIMSTR).

```
1C26  C1          L1C26:  POP  BC             BC = koden
1C27  D1                  POP  DE             DE = längden
1C28  7A                  LD   A,D
1C29  B3                  OR   E
1C2A  28 3D               JR   Z,L1C69        längden 0: inget att reservera
1C2C  C5                  PUSH BC             koden
1C2D  DD 4E 02            LD   C,(IX+$02)     antal element - 1
1C30  DD 46 03            LD   B,(IX+$03)
1C33  03                  INC  BC             antal element
1C34  DD 6E 00            LD   L,(IX+$00)     elementen
1C37  DD 66 01            LD   H,(IX+$01)
1C3A  E5                  PUSH HL
1C3B  DD E1               POP  IX             IX = första elementet
1C3D  CD 38 33    L1C3D:  CALL DIMSTR         reservera strängen
1C40  0B                  DEC  BC             ett element mindre
1C41  EB                  EX   DE,HL          längden undan i HL
1C42  11 06 00            LD   DE,$0006       nästa element
1C45  DD 19               ADD  IX,DE
1C47  EB                  EX   DE,HL          DE = längden
1C48  78                  LD   A,B
1C49  B1                  OR   C
1C4A  20 F1               JR   NZ,L1C3D       fler element
1C4C  D1                  POP  DE             DE = koden
1C4D  F7                  RST  $30            nästa token
```

```
1C4E  CD 6C 1C     L1C4E:  CALL ELMSIZ          ett index
1C51  C1                   POP  BC              BC = koden
1C52  20 07                JR   NZ,L1C5B        sträng
1C54  D1                   POP  DE              DE = index
1C55  C5                   PUSH BC
1C56  CD E6 32             CALL DIM1            tal: DIM1
1C59  D1                   POP  DE              DE = koden
1C5A  F7                   RST  $30             nästa token

1C5B  E1           L1C5B:  POP  HL              sträng: HL = längden
1C5C  D1                   POP  DE              DE = index
1C5D  E5                   PUSH HL              längden
1C5E  C5                   PUSH BC              koden
1C5F  CD E6 32             CALL DIM1
1C62  18 C2                JR   L1C26           sedan elementen

1C64  C1           L1C64:  POP  BC              inget index: sträng; BC = koden
1C65  D1                   POP  DE              DE = längden
1C66  CD 38 33             CALL DIMSTR          reservera strängen
1C69  50           L1C69:  LD   D,B             DE = koden
1C6A  59                   LD   E,C
1C6B  F7                   RST  $30             nästa token
```

ELMSIZ*: sätt elementets storlek i IX+4: flyttal 5, heltal 2, sträng 6. In: C = namnbyten, IX = posten. Ut: Z för tal, NZ för sträng. Förstör A. RET sist är också* ENDDIM ($BF*, slut på DIM-listan).*

```
1C6C  DD 36 04 05  ELMSIZ: LD   (IX+$04),$05    flyttal: 5 byte
1C70  3E 03                LD   A,$03
1C72  A1                   AND  C
1C73  C8                   RET  Z
1C74  DD 36 04 02          LD   (IX+$04),$02    heltal: 2 byte
1C78  3D                   DEC  A
1C79  C8                   RET  Z
1C7A  DD 36 04 06          LD   (IX+$04),$06    sträng: 6 byte
1C7E  C9           ENDDIM: RET                  även ENDDIM ($BF): slut på DIM-listan
```

PUSH0 ($D2)*: värdet 0 på stacken, som längd för en sträng utan = i DIM.*

```
1C7F  21 00 00     PUSH0:  LD   HL,$0000
1C82  EF                   RST  $28             0 på stacken, nästa token
```

Korsreferenstabell för LET och DIM. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnen är DIRECT vid IY+$0E. CY-kolumnen gäller flaggorna: ELMSIZ svarar med Z för tal. LET och DIM kör koden som ett uttryck (RST $30), och tilldelningen eller $BF (ENDDIM) gör RET till satsens anropare; det som uttrycket gör räknas inte med, utom DIMELM, som har en egen rad. DIMELM och PUSH0 går inte tillbaka utan hämtar nästa token; för dem visas bara stacken, variablerna och det de tar emot. DIMELM tar indexen och längden från stacken och reserverar elementen och texten på heapen (DIM1, DIM2 och DIMSTR med ALLOC). CMPERR tar bort ett ord från stacken; efter indexen i CMPDMI ligger två där, så att RET hoppar till variabelns namn som adress. Felen: CMPLET 15, CMPDMI 13 och 14, DIMELM 0 och 3, DIM 2, och de fel som CMPEXP och TYPCNV ger. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler	IY+
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	$0E
CMPLET, CMPFLV	U	U	U	IU	IU	X			
CMPDIM	U	U	X	IU	IU	X			

	register och stack							variabler	IY+
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	$0E
CMPBF	X			IU					
CMPDMI	U	U	X	IU	IU	X			
CMPERR	I	U	X				I		
DIM	X	X	X	IU	X	X			I
LET	X	X	X	IU	X	X			
DIMELM*				I			I	IU	
ELMSIZ, ENDDIM	X	U	I			I			
PUSH0*							U		

Rutin	**Anropar**	**Anropas från**
CMPLET, CMPFLV	CMPVAR (CMPFLV: CMPSV), CMPEXP, TYPCNV, PUTTOK (med JP), CMPERR; fel 15	CMPLET: TOKKW, STTAB1 ($82, $83); CMPFLV: CMPFOR
CMPDIM	CMPDMI, CMPBF	STTAB2 (DIM)
CMPBF		CMPDIM
CMPDMI	VARNAM, CMPARG, CMPEXP, TOINT, CMPERR; slutet av CMPVAR ($25FC, med JP)	CMPDIM
CMPERR		INSLIN (med JP), CMPLET, CMPDMI (två ställen), CMPTYX, CMPFNC, CMPARG
DIM	NODIR; går in i LET	STEXE2 (DIM)
LET	koden körs som ett uttryck (RST $30)	STEXE1 ($82, $83); DIM går in i den
DIMELM	RDDE1, ELMSIZ, DIM1, DIM2, DIMSTR; går vidare med RST $30	EXPTAB1 ($CC)
ELMSIZ, ENDDIM		ELMSIZ: DIMELM (två ställen); ENDDIM: EXPTAB1 ($BF)
PUSH0	går vidare med RST $28	EXPTAB1 ($D2)

3.43 FOR: kompilering $1C83

```
FOR v = a TO b [STEP c]
      v a $B6+t  $BA  b $B9+t  [$BB c $B9+t]  $B9 [2 byte]
```

v får inte ha index och inte vara en sträng. b och c omvandlas till v:s typ. $B9+t avslutar ett uttryck och lämnar värdet av typ t (se EXPTAB1). De två sista byten får adressen till FOR-blocket vid förlänkningen (LNKFOR). Fel 23 om TO saknas. In: HL vid variabeln, DE = koden. Ut: NC, HL efter satsen och DE efter de två byten för blocket. Förstör A, BC och IX.

```
1C83  CD A5 1B     CMPFOR: CALL CMPFLV          v = a, B = v:s typ
1C86  D8                   RET  C
1C87  C5                   PUSH BC              typen
1C88  D5                   PUSH DE              koden
1C89  11 8C 19             LD   DE,KWTO         tabellen med TO
1C8C  CD F5 09             CALL KWSRCH          Z: A = $BA
1C8F  D1                   POP  DE
1C90  C1                   POP  BC
1C91  4F                   LD   C,A             token undan
1C92  3E 97                LD   A,$97           fel 23
1C94  37                   SCF
1C95  C0                   RET  NZ              inget TO
1C96  79                   LD   A,C             $BA
1C97  12                   LD   (DE),A
```

```
1C98  13                INC  DE
1C99  CD 72 23          CALL CMPTYX          gränsen, i v:s typ
1C9C  D8                RET  C
1C9D  D5                PUSH DE
1C9E  C5                PUSH BC
1C9F  11 86 19          LD   DE,KWSTEP       tabellen med STEP
1CA2  CD F5 09          CALL KWSRCH          Z: A = $BB
1CA5  C1                POP  BC
1CA6  D1                POP  DE
1CA7  20 06             JR   NZ,CMPB9        inget STEP
1CA9  12                LD   (DE),A          $BB
1CAA  13                INC  DE
1CAB  CD 72 23          CALL CMPTYX          steget, i v:s typ
1CAE  D8                RET  C
```

CMPB9: *skriv* $B9 *och lämna plats för två byte (adressen till* FOR*-blocket, som fylls i vid förlänkningen). Ut: CY = 0.*

```
1CAF  3E B9       CMPB9: LD   A,$B9
1CB1  12                 LD   (DE),A
1CB2  13                 INC  DE
1CB3  13                 INC  DE
1CB4  13                 INC  DE
1CB5  A7                 AND  A
1CB6  C9                 RET
```

NEXT: kompilering. NEXT [v]
[v $BC+t] $B9 [2 byte]

Utan variabel slutar satsen vid :*, CR eller* ELSE *(S218D).*

Obs: LNKNXT *jämför variabeln med den i* FOR*, och utan variabel blir det alltid fel 28 vid* RUN*.* NEXT *ska alltså ha en variabel.*

In: HL efter NEXT*, DE = koden. Ut: NC, HL efter satsen och DE efter de två byten för blocket. Förstör A, BC och IX.*

```
1CB7  CD 8D 21    CMPNXT: CALL STMEND          slut på satsen?
1CBA  28 F3               JR   Z,CMPB9         utan variabel: bara $B9
1CBC  CD 92 23            CALL CMPSVR          variabeln + $BC + typ
1CBF  D8                  RET  C
1CC0  18 ED               JR   CMPB9           sedan $B9
```

3.44 FOR vid körning $1CC2

1. Första RST $38: v = a (tilldelningen avslutar).
2. Andra: gränsen b. Ett heltal läggs på stacken, ett flyttal ligger redan där.
3. Om STEP: steget c.
4. BC = FOR-blocket (från $B9-byten). Gränsen och steget skrivs in i blocket. Utan STEP gäller steget som LNKFOR lade in (1).
5. Testa direkt om slingan ska köras (FORTSI/FORTSF). Är a redan förbi b körs den noll gånger och körningen fortsätter efter NEXT (v = a).

Gränsen och steget räknas bara här. STEP 0 räknas som positivt, och slingan tar aldrig slut. Ett heltal slår runt: FOR I%=1 TO 32767 tar inte heller slut (v blir -32768, som är mindre än gränsen). FOR-blocket (reserverat på heapen av LNKFOR):

```
+0/1  adress efter NEXT (under förlänkningen länk till yttre FOR)
+2/3  variabelns adress
```

```
+4/5  adress efter FOR-satsen (slingans början)
heltal:  +6/7 gränsen    +8/9 steget
flyttal: +6..+10 gränsen +11..+15 steget
```

In: DE efter token $8B. Ut: DE = var körningen ska fortsätta. Med `FOR I=5 TO 1` körs slingan inte alls och I blir 5; med `STEP -1` blir det 5, 4, 3, 2, 1 och sedan I = 0 (prövat).

```
1CC2  FF          FOR:    RST  $38          v = a (gränsfall)
1CC3  13                  INC  DE           hoppa över $BA
1CC4  FF                  RST  $38          gränsen, B = typen
1CC5  78                  LD   A,B
1CC6  FE 00               CP   $00          flyttal (redan på stacken)?
1CC8  28 01               JR   Z,L1CCB
1CCA  E5                  PUSH HL           heltalet till stacken
1CCB  1A          L1CCB:  LD   A,(DE)
1CCC  FE BB               CP   $BB          STEP?
1CCE  13                  INC  DE           förbi $BB eller $B9
1CCF  20 25               JR   NZ,L1CF6     inget STEP
1CD1  FF                  RST  $38          steget, B = typen
1CD2  78                  LD   A,B          A = typen
1CD3  EB                  EX   DE,HL        DE = heltalssteget, HL = koden vid $B9
1CD4  CD 16 10            CALL RDBC1        förbi $B9, BC = FOR-blocket
1CD7  C5                  PUSH BC
1CD8  DD E1               POP  IX           IX = blocket
1CDA  FE 00               CP   $00          flyttal?
1CDC  28 3D               JR   Z,L1D1B      flyttal: steget vid SP
1CDE  DD 73 08            LD   (IX+$08),E   heltal: steget
1CE1  DD 72 09            LD   (IX+$09),D
1CE4  D1          L1CE4:  POP  DE           gränsen
1CE5  DD 73 06            LD   (IX+$06),E   +6/7
1CE8  DD 72 07            LD   (IX+$07),D
1CEB  DD 6E 02            LD   L,(IX+$02)   variabelns adress
1CEE  DD 66 03            LD   H,(IX+$03)
1CF1  5E                  LD   E,(HL)       DE = v
1CF2  23                  INC  HL
1CF3  56                  LD   D,(HL)
1CF4  18 63               JR   FORTSI       ska slingan köras?
```

Ingen `STEP`. `$B9` *är redan överhoppad; BC =* `FOR`*-blocket.*

```
1CF6  78          L1CF6:  LD   A,B          A = gränsens typ
1CF7  EB                  EX   DE,HL        HL = koden efter $B9
1CF8  4E                  LD   C,(HL)       BC = FOR-blocket
1CF9  23                  INC  HL
1CFA  46                  LD   B,(HL)
1CFB  C5                  PUSH BC
1CFC  DD E1               POP  IX
1CFE  FE 01               CP   $01          heltal?
1D00  28 E2               JR   Z,L1CE4      heltal: gränsen vid SP
```

Flytta 5 byte (ett flyttal) från stacken till blocket: +11 för steget (CY = 1, sedan en gång till för gränsen), +6 för gränsen.

```
1D02  21 06 00    L1D02:  LD   HL,$0006     +6: gränsen
1D05  AF                  XOR  A            CY = 0: sista flytten
1D06  08          L1D06:  EX   AF,AF'       flaggan undan
1D07  09                  ADD  HL,BC        HL = platsen i blocket
1D08  EB                  EX   DE,HL        DE = platsen
1D09  21 00 00            LD   HL,$0000
1D0C  39                  ADD  HL,SP        HL = SP
1D0D  01 05 00            LD   BC,$0005
```

```
1D10  ED B0              LDIR              flyttalet från stacken
1D12  F9                 LD   SP,HL        ta bort det
1D13  08                 EX   AF,AF'       flaggan tillbaka
1D14  30 72              JR   NC,FORTSF    båda flyttade: pröva
1D16  DD E5              PUSH IX
1D18  C1                 POP  BC           BC = blocket
1D19  18 E7              JR   L1D02        sedan gränsen

1D1B  21 0B 00     L1D1B: LD   HL,$000B     steget till +11
1D1E  37                 SCF               CY = 1: gränsen kommer sedan
1D1F  18 E5              JR   L1D06
```

3.45 NEXT vid körning $1D21

IX = FOR-blocket, vars adress står 7 byte in (efter v:s 5 byte, $BC+t och $B9). In: DE efter token $8C. Heltal: v = v + steg (ingen kontroll av spill). Flyttal: flyttalsaddition med steget på blocket+15 ($33F7).

Sedan samma test som i FOR:

```
positivt steg: fortsätt slingan om v <= gränsen
negativt steg: fortsätt slingan om v >= gränsen
```

Ut: DE = blocket+4 (slingans början) eller blocket+0 (efter NEXT). Hoppet sker bara genom DE, och CURLIN sätts inte om: står FOR mitt i en rad får ett fel i resten av den raden NEXT-radens nummer, tills nästa rad börjar.

```
1D21  1A           NEXT: LD   A,(DE)         variabelns token, eller $B9 (egenhet)
1D22  FE B9              CP   $B9            ingen variabel?
1D24  20 10              JR   NZ,L1D36       med variabel
```

Utan variabel ($B9 först): blocket+4 används som uttryckskod och körs; ger det 0 fortsätter körningen vid ordet som blocket pekar på. Grenen passar inte med FOR-blocket, och LNKNXT tillåter inte NEXT utan variabel. Den nås aldrig: NEXT utan variabel ger fel 28 redan vid länkningen (provat).

```
1D26  13                 INC  DE
1D27  EB                 EX   DE,HL
1D28  5E                 LD   E,(HL)
1D29  23                 INC  HL
1D2A  56                 LD   D,(HL)
1D2B  EB                 EX   DE,HL
1D2C  E5                 PUSH HL
1D2D  23                 INC  HL
1D2E  23                 INC  HL
1D2F  23                 INC  HL
1D30  23                 INC  HL
1D31  5E                 LD   E,(HL)         DE = (blocket+4)
1D32  23                 INC  HL
1D33  56                 LD   D,(HL)
1D34  18 72              JR   L1DA8

1D36  21 07 00     L1D36: LD   HL,$0007      adressen till FOR-blocket
1D39  19                 ADD  HL,DE
1D3A  5E                 LD   E,(HL)
1D3B  23                 INC  HL
1D3C  56                 LD   D,(HL)
1D3D  D5                 PUSH DE
1D3E  DD E1              POP  IX             IX = blocket
1D40  DD 6E 02           LD   L,(IX+$02)     variabelns adress
1D43  DD 66 03           LD   H,(IX+$03)
1D46  FE C0              CP   $C0            flyttal?
```

```
1D48  28 2F              JR   Z,L1D79
1D4A  DD 4E 08           LD   C,(IX+$08)      steget
1D4D  DD 46 09           LD   B,(IX+$09)
1D50  5E                 LD   E,(HL)          DE = v
1D51  23                 INC  HL
1D52  56                 LD   D,(HL)
1D53  EB                 EX   DE,HL           v + steget
1D54  09                 ADD  HL,BC
1D55  EB                 EX   DE,HL           DE = v + steget, HL = adressen + 1
1D56  72                 LD   (HL),D          skriv tillbaka v
1D57  2B                 DEC  HL
1D58  73                 LD   (HL),E
```

FORTSI: *heltal, för FOR och NEXT. In: IX = blocket, DE = v. HL = gränsen; med negativt steg byts HL och DE. Är skillnaden HL - DE negativ (rättad för spill med P/V) är slingan klar. STEP 0 räknas som positivt. Ut: DE = slingans början (blocket +4/5) eller platsen efter NEXT (+0/1). Förstör A och HL.*

```
1D59  DD 6E 06     FORTSI: LD   L,(IX+$06)      HL = gränsen, DE = v
1D5C  DD 66 07             LD   H,(IX+$07)
1D5F  DD 7E 09             LD   A,(IX+$09)      stegets tecken
1D62  A7                   AND  A               CY = 0 för SBC
1D63  F2 67 1D             JP   P,L1D67         positivt?
1D66  EB                   EX   DE,HL           negativt: byt
1D67  ED 52        L1D67:  SBC  HL,DE           gränsen - v
1D69  E2 6F 1D             JP   PO,L1D6F        inget spill?
1D6C  3E 80                LD   A,$80           spill: vänd tecknet
1D6E  AC                   XOR  H               S = rätt tecken
1D6F  FA A1 1D     L1D6F:  JP   M,L1DA1          negativt: klar
1D72  DD 5E 04     L1D72:  LD   E,(IX+$04)      slingans början
1D75  DD 56 05             LD   D,(IX+$05)
1D78  C9                   RET                  DE = där körningen fortsätter

1D79  44           L1D79:  LD   B,H             BC = variabelns adress
1D7A  4D                   LD   C,L
1D7B  21 0F 00             LD   HL,$000F        steget, blocket+15
1D7E  19                   ADD  HL,DE           HL = blocket + 15
1D7F  50                   LD   D,B             DE = variabeln
1D80  59                   LD   E,C
1D81  DD E5                PUSH IX              FADD ändrar IX
1D83  CD F7 33             CALL FADD            v = v + steget
1D86  DD E1                POP  IX
```

FORTSF: *flyttal, för FOR och NEXT. Jämför gränsen (blocket+10) med v med FCMP, i omvänd ordning om steget är negativt (tecknet i blocket+14). In: IX = blocket. Ut: som FORTSI.*

```
1D88  DD E5        FORTSF: PUSH IX
1D8A  C1                   POP  BC              BC = blocket
1D8B  21 0A 00             LD   HL,$000A        gränsen, blocket+10
1D8E  09                   ADD  HL,BC
1D8F  DD 5E 02             LD   E,(IX+$02)      DE = v
1D92  DD 56 03             LD   D,(IX+$03)
1D95  DD 7E 0E             LD   A,(IX+$0E)      stegets tecken
1D98  A7                   AND  A
1D99  28 01                JR   Z,L1D9C         positivt
1D9B  EB                   EX   DE,HL           negativt: byt
1D9C  CD 93 2A     L1D9C:  CALL FCMP            CY: talet vid HL mindre
1D9F  30 D1                JR   NC,L1D72        NC: slingan igen
1DA1  DD 5E 00     L1DA1:  LD   E,(IX+$00)      efter NEXT
1DA4  DD 56 01             LD   D,(IX+$01)
1DA7  C9                   RET
```

```
1DA8  FF          L1DA8:  RST  $38
1DA9  7C                  LD   A,H
1DAA  B5                  OR   L
1DAB  E1                  POP  HL
1DAC  C0                  RET  NZ
1DAD  5E                  LD   E,(HL)
1DAE  23                  INC  HL
1DAF  56                  LD   D,(HL)
1DB0  C9                  RET
```

GET *v$: vänta på en tangent* (GETKEY) *och tilldela v$ en sträng med det tecknet. Går även i direktläge;* GET *till en talvariabel ger fel 17 vid kompileringen. Tecknet läggs på stacken som en tillfällig sträng:*

```
[antal tillfälliga byte  = 1] [adress] [längd = 1] [tecknet]
```

och sedan hoppar det till ASGNS*, som har variabelns adress från* RST $38 *ovanför strängen. In: DE efter token* $86 $90*. Ut: som* ASGNS*,* RET *till satsens anropare.*

```
1DB1  FF          GET:    RST  $38            variabeln: HL = adressen
1DB2  E5                  PUSH HL             adressen till ASGNS
1DB3  D5                  PUSH DE             koden
1DB4  CD EA 02            CALL GETKEY         vänta på en tangent
1DB7  D1                  POP  DE
1DB8  21 FF FF            LD   HL,$FFFF
1DBB  39                  ADD  HL,SP
1DBC  F9                  LD   SP,HL          en byte på stacken
1DBD  01 01 00            LD   BC,$0001
1DC0  77                  LD   (HL),A         tecknet på stacken
1DC1  C5                  PUSH BC             längd 1
1DC2  E5                  PUSH HL             adressen till tecknet
1DC3  C5                  PUSH BC             1 tillfällig byte
1DC4  C3 C3 29            JP   ASGNS          tilldela
```

CMPREF*: kompilera en radreferens (GOTO, GOSUB, ON ERROR GOTO, ett nummer efter THEN eller ELSE och RESTORE; ON har en egen slinga). Koden blir* $BF*, radnumret och två byte för adressen, som* LNKREF *fyller i vid förlänkningen. Inget nummer, eller ett över 65535, ger CY och A =* $90 *(fel 16). Blanka inne i numret hoppas över (*GOTO 1 0 *= GOTO 10). In: HL vid numret, DE = koden. Ut: NC, HL efter numret och blanka och DE efter de fem byten. Förstör A och BC.*

```
1DC7  D5          CMPREF: PUSH DE             koden
1DC8  CD 1E 18            CALL GETLN          radnumret till DE
1DCB  42                  LD   B,D            BC = numret
1DCC  4B                  LD   C,E
1DCD  D1                  POP  DE
1DCE  3E 90               LD   A,$90          fel 16, RADNUMMER SAKNAS
1DD0  D8                  RET  C              inget nummer
1DD1  EB                  EX   DE,HL          HL = koden
1DD2  36 BF               LD   (HL),$BF       radreferens
1DD4  CD 28 10            CALL WRBC1          radnumret
1DD7  23                  INC  HL             plats för adressen
1DD8  23                  INC  HL
1DD9  EB                  EX   DE,HL          DE efter referensen
1DDA  C9                  RET                 NC från GETLN
```

GOTO*: fel 2 i direktläge. Adressen hämtas ur* $BF*-elementet och* RUNLIN *fortsätter där. De två POP tar bort returadresserna till* RUNSTM *och* RUNLIN*. En rad som saknas ger fel 6 redan vid länkningen. In: DE efter token, vid* $BF*.*

```
1DDB  CD BF 0D    GOTO:   CALL NODIR
1DDE  13                  INC  DE             förbi $BF
1DDF  13                  INC  DE             och radnumrets låga byte
1DE0  EB                  EX   DE,HL
```

```
1DE1  CD 1C 10          CALL  RDDE1     förbi den höga, DE = målradens adress
1DE4  E1                POP   HL       bort med RUNSTM:s och
1DE5  E1                POP   HL       RUNLIN:s returadresser
1DE6  C3 71 0D          JP    RUNLIN    fortsätt på målraden
```

GOSUB: som GOTO, men RUNLIN anropas med CALL. RETURN går tillbaka hit, och körningen fortsätter efter GOSUB. Varje GOSUB-nivå kostar alltså stack, och STKCHK kontrollerar att den räcker: med 16 kilobyte RAM och ett kort program blir det fel 3 efter omkring 1870 nivåer.

```
1DE9  CD BF 0D    GOSUB:  CALL  NODIR
1DEC  13                  INC   DE        förbi $BF
1DED  13                  INC   DE        och radnumrets låga byte
1DEE  EB                  EX    DE,HL
1DEF  CD 1C 10            CALL  RDDE1     DE = målradens adress
1DF2  CD 89 33            CALL  STKCHK    fel 3 om stacken inte räcker
1DF5  E5                  PUSH  HL        fortsättningen i raden
1DF6  CD 71 0D            CALL  RUNLIN    kör från målraden till RETURN
1DF9  D1                  POP   DE        DE efter referensen
1DFA  C9                  RET             satsen klar
```

Korsreferenstabell för FOR, NEXT, GET, radreferenserna, GOTO och GOSUB. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är DIRECT vid IY+$0E och FLAGS vid IY+$0F (bit 4 = TRACE). CMPB9 lämnar alltid CY = 0. FOR, NEXT och GET räknar ut uttryck med RST $38, och det som uttrycken gör räknas inte med; FOR-blocket på heapen (från LNKFOR) får gränsen och steget av FOR och läses av FORTSI och FORTSF. GET väntar med GETKEY, som läser markörens plats och tangentbordets flaggor vid $FDF3, och ASGNS kan reservera texten på heapen. GOTO går inte tillbaka; för den visas bara stacken, variablerna och det den tar emot. GOTO och GOSUB fortsätter med RUNLIN, vars verkan räknas med, men inte det som satserna på målraderna gör; efter GOSUB pekar CURLIN på RETURN-raden. Felen: CMPFOR 23, CMPREF 16, GOTO och GOSUB 2, GOSUB 3 och GET 3. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler				IY+	
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	CURLIN	STMSP	BRKFLG	$0E	$0F
CMPFOR	U	U	X	IU	IU	X							
CMPB9	X	U		IU									
CMPNXT	U	U	X	IU	IU	X							
FOR	X	X	X	IU	X	U							
NEXT	X	X	X	IU	X	U							
FORTSI	X	X		IU	X	I							
FORTSF	X	X	X	U	X	I							
GET	X	X	X	IU	X	X		IU					
CMPREF	U	U	X	IU	IU								
GOTO*				I			I		U	U	IU	I	I
GOSUB	X	X	X	IU	X	X		I	U	U	IU	I	I

Rutin	**Anropar**	**Anropas från**
CMPFOR	CMPFLV, KWSRCH (KWTO, KWSTEP), CMPTYX, CMPB9; fel 23	STTAB1 ($8B)
CMPB9		CMPFOR (går in i den), CMPNXT (två ställen)
CMPNXT	STMEND, CMPSVR, CMPB9	STTAB1 ($8C)
FOR	RST38, RDBC1, FORTSI, FORTSF (med JR)	STEXE1 ($8B)

Rutin	Anropar	Anropas från
NEXT	FADD, FORTSI, FORTSF (går in i dem); RST38 i grenen utan variabel	STEXE1 ($8C)
FORTSI		FOR (med JR), NEXT (går in i den)
FORTSF	FCMP	FOR (med JR), NEXT (går in i den)
GET	RST38, GETKEY, ASGNS (med JP)	STEXE2 (GET)
CMPREF	GETLN, WRBC1; fel 16	STTAB1 (GOTO, GOSUB, ON ERROR GOTO, $81), CMPIF (två ställen), CMPRES
GOTO	NODIR, RDDE1; går vidare till RUNLIN	STEXE1 ($80, $81)
GOSUB	NODIR, RDDE1, STKCHK, RUNLIN	STEXE1 ($90)

3.46 IF: kompilering $1DFB

IF villkor [THEN] satser [ELSE satser]
villkoret (kod) [$BD] $87 n THEN-del [$87 m $BC ELSE-del]

Ett radnummer direkt efter THEN eller ELSE blir en GOTO: token $81 och en radreferens. n och m är längderna att hoppa, och de fylls i när delen är kompilerad.

En sats efter THEN eller ELSE tar resten av raden (TOKSTM), utom att THEN-delen slutar vid ELSE. Ett radnummer däremot är hela delen: det som följer efter ":" är en vanlig sats efter IF. I IF v THEN 100 : S körs S alltså när v är falskt, och i IF v THEN T ELSE 100 : S när v är sant. ELSE hör till det närmaste IF:et. Villkoret är ett heltalsuttryck (CMPINX): ett strängvillkor ger fel 17, och utan THEN-del eller med tom THEN- eller ELSE-del blir det fel 11. REM i THEN-delen tar även ELSE-delen. In: HL efter IF, DE = koden. Ut: NC, HL efter satsen och DE efter koden; vid fel CY och A = felkoden. Förstör A, BC och IX.

```
1DFB  CD 70 23     CMPIF:  CALL CMPINX          villkoret, heltal + $BA (egenhet)
1DFE  D8                   RET  C               fel i villkoret (sträng: 17)
1DFF  D5                   PUSH DE              koden undan
1E00  11 7A 19             LD   DE,KWTHEN       THEN
1E03  CD F5 09             CALL KWSRCH          Z: THEN finns, A = $BD
1E06  D1                   POP  DE
1E07  20 02                JR   NZ,L1E0B        THEN kan saknas
1E09  12                   LD   (DE),A          $BD i koden, så att LIST visar THEN
1E0A  13                   INC  DE
1E0B  3E 87        L1E0B:  LD   A,$87           $87 n: hopp förbi THEN-delen
1E0D  12                   LD   (DE),A
1E0E  13                   INC  DE
1E0F  D5                   PUSH DE              platsen för n, ifylls sist
1E10  13                   INC  DE              över n
1E11  3E 81                LD   A,$81           gissa ett radnummer: $81 = GOTO
1E13  12                   LD   (DE),A
1E14  13                   INC  DE
1E15  CD C7 1D             CALL CMPREF          ett nummer ger $BF nr adress
1E18  30 04                JR   NC,L1E1E        radnummer: klart
1E1A  1B                   DEC  DE              inget nummer: skriv över $81
1E1B  CD C7 18             CALL TOKSTM          annars satserna, till ELSE eller CR
1E1E  C1           L1E1E:  POP  BC              BC = platsen för n
1E1F  D8                   RET  C               fel i THEN-delen
1E20  C5                   PUSH BC              n ligger kvar
1E21  D5                   PUSH DE
1E22  11 80 19             LD   DE,KWELSE       ELSE
1E25  CD F5 09             CALL KWSRCH          Z: ELSE finns, A = $BC
1E28  D1                   POP  DE
1E29  C1                   POP  BC
1E2A  20 1A                JR   NZ,L1E46        inget ELSE: fyll i n
```

```
1E2C  EB                EX   DE,HL          HL = koden
1E2D  36 87             LD   (HL),$87       $87 m: hopp förbi ELSE-delen
1E2F  23                INC  HL
1E30  E5                PUSH HL             platsen för m
1E31  23                INC  HL             över m
1E32  77                LD   (HL),A         $BC i koden (ELSE)
1E33  23                INC  HL
1E34  7D                LD   A,L            n räknas från n:s byte
1E35  91                SUB  C
1E36  02                LD   (BC),A         n = THEN-delens längd
1E37  36 81             LD   (HL),$81       gissa radnummer här också
1E39  23                INC  HL
1E3A  EB                EX   DE,HL          DE = koden, HL = texten
1E3B  CD C7 1D          CALL CMPREF         ELSE nr
1E3E  30 04             JR   NC,L1E44
1E40  1B                DEC  DE             skriv över $81
1E41  CD C7 18          CALL TOKSTM         ELSE-delens satser
1E44  C1          L1E44: POP  BC            BC = platsen för m
1E45  D8                RET  C              fel i ELSE-delen
1E46  7B          L1E46: LD   A,E           längd = slutet - platsen
1E47  91                SUB  C
1E48  02                LD   (BC),A         n eller m
1E49  A7                AND  A              NC: klart
1E4A  C9                RET
```

IF*: beräkna villkoret* (RST $38)*. HL = 0 betyder falskt. Då pekar DE på* $87*, och hoppet tar körningen till* ELSE*-delen. Annars hoppar DE över* $87 *n och* THEN*-delen körs. Ett flyttal avrundas nedåt till heltal:* IF *.5 är falskt,* IF *-.5 sant, och från 65536 blir det fel 7. In: DE efter token* $88*. Ut: DE vid* $87 *(falskt) eller vid THEN-delen (sant).*

```
1E4B  FF          IF:   RST  $38            villkoret som heltal i HL
1E4C  1A                LD   A,(DE)         token efter villkoret
1E4D  FE BD             CP   $BD            THEN?
1E4F  20 01             JR   NZ,L1E52       ordet kan saknas
1E51  13                INC  DE             över THEN
1E52  7D          L1E52: LD   A,L           HL = 0: falskt?
1E53  B4                OR   H
1E54  C8                RET  Z              DE kvar på $87
1E55  13                INC  DE             över $87 n: THEN-delen
1E56  13                INC  DE
1E57  C9                RET
```

3.47 INPUT: kompilering $1E58

```
INPUT [#n,] v [, v ...]
      ["#" n $BA ","]  v $BC+t  ["," v $BC+t ...]
```

Tecknen ”#” och ”,” kopieras direkt till koden. Filnumret är ett heltal. Fel 13 om ”,” saknas efter filnumret. Variablerna kan ha index och vara av alla typer.

CMPRD (READ) är samma sak utan filnumret. In: HL efter INPUT eller READ, DE = koden. Ut: NC, HL efter listan och DE efter koden; vid fel CY och A = felkoden. Förstör A, BC och IX.

```
1E58  7E          CMPINP: LD   A,(HL)
1E59  FE 23             CP   $23            #?
1E5B  20 11             JR   NZ,CMPRD
1E5D  ED A0             LDI                 kopiera #
1E5F  E7                RST  $20            mellanslag efter
1E60  CD 70 23          CALL CMPINX         filnumret
1E63  D8                RET  C              fel i filnumret
```

```
1E64  7E                LD   A,(HL)
1E65  FE 2C             CP   $2C            ,?
1E67  3E 8D             LD   A,$8D          annars fel 13
1E69  37                SCF                 CY + fel
1E6A  C0                RET  NZ             inget ,: fel 13
1E6B  ED A0       L1E6B: LDI                 kopiera ,
1E6D  E7                RST  $20            mellanslag efter
1E6E  CD 97 23    CMPRD: CALL CMPVR          variabel + $BC + typ
1E71  D8                RET  C             fel i variabeln
1E72  7E                LD   A,(HL)
1E73  EE 2C             XOR  $2C            ,?
1E75  C0                RET  NZ             NZ: listan slut, och CY = 0
1E76  18 F3             JR   L1E6B          nästa variabel
```

3.48 INPUT vid körning $1E78

Raden läses in i BUF1 ($FE40, högst 120 tecken). Filnumret (0 = tangentbordet) sparas i IY+$0D. För varje variabel:

- Står det ”,” i bufferten tas nästa värde efter det.
- Annars läses en ny rad. Från tangentbordet skrivs först ”?”, och efter raden en ny rad (NEWLIN).

Värdet läses och tilldelas av RDVAL. Kommer det för få värden frågar datorn alltså igen med ”?”. Värden som blir över glöms. Står det något annat än ”,” efter ett värde (5X,6) glöms också resten, och nästa variabel får en ny rad. INPUT LINE går till INPLIN ($1F82). In: DE efter token $89. Ut: DE efter satsen.

```
1E78  21 40 FE    INPT:  LD   HL,$FE40      BUF1, där raden läses in
1E7B  E5                 PUSH HL            BUF1 vid SP under satsen
1E7C  36 0D              LD   (HL),$0D      tom buffert
1E7E  FD 36 0D 00        LD   (IY+$0D),$00  fil 0: tangentbordet
1E82  1A                 LD   A,(DE)        första byten i koden
1E83  FE 23              CP   $23           #?
1E85  20 06              JR   NZ,L1E8D
1E87  13                 INC  DE            över #
1E88  FF                 RST  $38           filnumret
1E89  13                 INC  DE            över ,
1E8A  FD 75 0D           LD   (IY+$0D),L    IY+$0D = filnumret
1E8D  E1          L1E8D: POP  HL            HL = BUF1
1E8E  E5                 PUSH HL
1E8F  7E          L1E8F: LD   A,(HL)        tecknet vid läspekaren
1E90  23                 INC  HL
1E91  FE 2C              CP   $2C           ,: nästa värde på raden
1E93  28 24              JR   Z,L1EB9
1E95  FD 7E 0D           LD   A,(IY+$0D)    slut på värden: ny rad
1E98  A7                 AND  A             tangentbordet?
1E99  20 0B              JR   NZ,L1EA6
1E9B  D5                 PUSH DE            kodpekaren undan
1E9C  21 6A 1F           LD   HL,D1F6A      ?
1E9F  01 01 00           LD   BC,$0001
1EA2  CD A8 01           CALL PRTXT         skriv ?
1EA5  D1                 POP  DE
1EA6  E1          L1EA6: POP  HL            HL = BUF1
1EA7  E5                 PUSH HL
1EA8  D5                 PUSH DE            kodpekaren undan
1EA9  01 78 00           LD   BC,$0078      högst 120 tecken
1EAC  CD DD 08           CALL INPUT         läs en rad
1EAF  FD 7E 0D           LD   A,(IY+$0D)
```

```
1EB2  A7                AND  A
1EB3  CC 42 07          CALL Z,NEWLIN        tangentbordet: ny rad
1EB6  D1                POP  DE
1EB7  E1                POP  HL
1EB8  E5                PUSH HL              HL = läspekaren
1EB9  CD FE 1E  L1EB9:  CALL RDVAL           läs värdet och tilldela
1EBC  1A                LD   A,(DE)          byten efter variabelns $BC+t
1EBD  13                INC  DE
1EBE  FE 2C             CP   $2C             fler variabler?
1EC0  28 CD             JR   Z,L1E8F
1EC2  1B                DEC  DE              DE på byten efter satsen
1EC3  E1                POP  HL              ta bort BUF1 från stacken
1EC4  C9                RET
```

3.49 READ vid körning $1EC5

DATPTR pekar på nästa värde i en DATA-rad, och DATLIN på raden. Står DATPTR på CR letas nästa rad upp som börjar med DATA ($93 direkt efter radnumret); finns ingen blir det fel 30, DATA SLUT. DATA räknas alltså bara när det står först på raden. Värdena läses med RDVAL, precis som för INPUT, och ”,” mellan dem hoppas över. Till skillnad från INPUT krävs inget ”,”: det som står efter ett värde blir nästa värde. I DATA "AB"C är C ett eget värde, och i DATA 2 : REM blir : REM nästa värde (fel 12 för ett tal). In: DE efter token $8E. Ut: DE efter satsen.

```
1EC5  2A 2E FE  READ:   LD   HL,(DATPTR)     (egenhet)
1EC8  7E        L1EC8:  LD   A,(HL)          tecknet vid DATPTR
1EC9  FE 0D             CP   $0D             slut på DATA-raden?
1ECB  20 1D             JR   NZ,L1EEA
1ECD  2A 30 FE          LD   HL,(DATLIN)
1ED0  4E        L1ED0:  LD   C,(HL)          C = radens längd
1ED1  79                LD   A,C
1ED2  FE 01             CP   $01             slut på programmet?
1ED4  CA 1F 07          JP   Z,L071F         fel 30, DATA SLUT
1ED7  06 00             LD   B,$00           BC = längden
1ED9  09                ADD  HL,BC           nästa rad
1EDA  44                LD   B,H             BC = radens början
1EDB  4D                LD   C,L
1EDC  03                INC  BC
1EDD  03                INC  BC
1EDE  03                INC  BC
1EDF  0A                LD   A,(BC)          första token
1EE0  FE 93             CP   $93             DATA?
1EE2  20 EC             JR   NZ,L1ED0        nej: nästa rad
1EE4  22 30 FE          LD   (DATLIN),HL     raden blir DATLIN
1EE7  60                LD   H,B             HL = DATA-token
1EE8  69                LD   L,C
1EE9  23                INC  HL              efter DATA-token
1EEA  CD FE 1E  L1EEA:  CALL RDVAL
1EED  7E                LD   A,(HL)
1EEE  FE 2C             CP   $2C             ,?
1EF0  CC BE 25          CALL Z,SKIPSP        över kommat
1EF3  1A                LD   A,(DE)          byten efter variabelns $BC+t
1EF4  13                INC  DE
1EF5  FE 2C             CP   $2C             fler variabler?
1EF7  28 CF             JR   Z,L1EC8
1EF9  1B                DEC  DE              DE på byten efter satsen
1EFA  22 2E FE          LD   (DATPTR),HL     spara pekaren
1EFD  C9                RET
```

3.50 RDVAL: läs ett värde ur texten vid HL och tilldela det $1EFE

DE = koden v $BC+t. RST $38 ger variabelns adress och typ.

```
flyttal  GETSFL (tecken tillåtet), fel 12 om det inte är ett tal
         (tom rad, "." eller "-"). Text efter talet stoppar bara
         läsningen: "5X" = 5, och "1E" = 1 med HL på "E".
heltal   GETSIN (tecken tillåtet), fel 12, även över 65535;
         65535 = -1 och -40000 = 25536 (modulo 65536)
sträng   inom " " eller ' ': till samma citattecken eller CR, två
         citattecken efter varandra blir ett. Annars till "," eller
         CR, och alla mellanslag hoppas över (RST $20 före värdet,
         RST $18 i slingan), så att "A B" blir "AB". Saknas det
         avslutande citattecknet tas texten till CR. Texten kopieras
         till BUF1 ($FE40) och tilldelas med ASGNS.
```

Samma sak gäller READ av DATA-strängar utan citattecken. INPUT LINE tar raden som den är. Christensen klagar på att INPUT tar bort blanktecken (Christensen 1979); det stämmer, i båda versionerna.

INPUT läser också in raden till BUF1, men kopian hamnar aldrig efter originalet, så det går bra.

In: HL vid texten. Ut: HL efter värdet (och mellanslag), DE efter koden. Förstör A, BC och IX.

```
1EFE  E7          RDVAL:  RST  $20          mellanslag före värdet (egenhet)
1EFF  E5                  PUSH HL           texten undan
1F00  FF                  RST  $38          HL = variabeln, B = typ
1F01  78                  LD   A,B          A = typen
1F02  EB                  EX   DE,HL        HL = koden, DE = variabeln
1F03  E3                  EX   (SP),HL      (SP) = koden, HL = texten
1F04  FE 02               CP   $02          sträng?
1F06  28 1D               JR   Z,L1F25
1F08  FE 01               CP   $01          heltal?
1F0A  28 0B               JR   Z,L1F17
1F0C  EB                  EX   DE,HL        HL = variabeln, DE = texten
1F0D  CD FB 14            CALL GETSFL       flyttal
1F10  38 09               JR   C,L1F1B      inte ett tal: fel 12
1F12  1B                  DEC  DE           GETFLT lämnar DE ett steg efter
1F13  EB                  EX   DE,HL
1F14  D1          L1F14:  POP  DE           DE = koden
1F15  E7                  RST  $20          mellanslag efter värdet
1F16  C9                  RET

1F17  D5          L1F17:  PUSH DE           variabeln undan
1F18  CD 41 18            CALL GETSIN       heltal
1F1B  DA 13 07    L1F1B:  JP   C,L0713      fel 12
1F1E  E3                  EX   (SP),HL      HL = variabeln, (SP) = texten
1F1F  73                  LD   (HL),E       skriv heltalet
1F20  23                  INC  HL
1F21  72                  LD   (HL),D
1F22  E1                  POP  HL           HL = texten
1F23  18 EF               JR   L1F14
```

Sträng. $1F13 läggs som returadress, så att ASGNS kommer tillbaka dit. Längden räknas i B (högst 255).

```
1F25  01 13 1F    L1F25:  LD   BC,$1F13     retur till $1F13, efter DEC DE
1F28  C5                  PUSH BC
1F29  D5                  PUSH DE           variabelns adress, under strängen
1F2A  11 40 FE            LD   DE,$FE40     kopian i BUF1
1F2D  06 00               LD   B,$00        B = längden
1F2F  D5                  PUSH DE           BUF1 undan
1F30  7E                  LD   A,(HL)       första tecknet
1F31  FE 22               CP   $22          citattecken?
```

```
1F33  28 12              JR   Z,L1F47
1F35  FE 27              CP   $27            eller apostrof?
1F37  28 0E              JR   Z,L1F47
1F39  FE 2C     L1F39:   CP   $2C            tecknet i A
1F3B  28 20              JR   Z,L1F5D
1F3D  FE 0D              CP   $0D            CR?
1F3F  28 1C              JR   Z,L1F5D
1F41  04                 INC  B              räkna
1F42  12                 LD   (DE),A         kopiera
1F43  13                 INC  DE
1F44  DF                 RST  $18            nästa tecken, utan mellanslag
1F45  18 F2              JR   L1F39

1F47  4F        L1F47:   LD   C,A            C = citattecknet som avslutar
1F48  23                 INC  HL             över det
1F49  7E        L1F49:   LD   A,(HL)         nästa tecken
1F4A  FE 0D              CP   $0D
1F4C  28 0F              JR   Z,L1F5D        CR: slut, även utan citattecken
1F4E  23                 INC  HL             HL efter tecknet
1F4F  B9                 CP   C              samma citattecken?
1F50  28 05              JR   Z,L1F57
1F52  12        L1F52:   LD   (DE),A         kopiera
1F53  13                 INC  DE
1F54  04                 INC  B              räkna
1F55  18 F2              JR   L1F49

1F57  BE        L1F57:   CP   (HL)           ett till efter citattecknet?
1F58  20 03              JR   NZ,L1F5D       nej: slut, HL efter citattecknet
1F5A  23                 INC  HL             två blir ett, som kopieras
1F5B  18 F5              JR   L1F52

1F5D  EB        L1F5D:   EX   DE,HL          DE = texten
1F5E  26 00              LD   H,$00          HL = längden
1F60  68                 LD   L,B
1F61  E3                 EX   (SP),HL        (SP) = längden, HL = BUF1
1F62  E5                 PUSH HL             strängens adress
1F63  21 00 00           LD   HL,$0000       inget tillfälligt område
1F66  E5                 PUSH HL
1F67  C3 C3 29           JP   ASGNS          strängen till variabeln, RET till $1F13

1F6A  3F        D1F6A:   DEFB "?"            när INPUT läser en ny rad från tangentbordet
```

INPUT LINE: kompilering. INPUT LINE [#n[,]] v$
`["#" n $BA [","]] v$ $BE`

Kommat efter filnumret kan utelämnas. In: HL efter INPUT LINE*, DE = koden. Ut: som* CMPGET*.*

```
1F6B  7E        CMPINL:  LD   A,(HL)
1F6C  FE 23              CP   $23            #?
1F6E  20 0F              JR   NZ,L1F7F
1F70  ED A0              LDI                 kopiera #
1F72  E7                 RST  $20            mellanslag efter
1F73  CD 70 23           CALL CMPINX         filnumret
1F76  D8                 RET  C              fel i filnumret
1F77  7E                 LD   A,(HL)
1F78  FE 2C              CP   $2C            ,?
1F7A  20 03              JR   NZ,L1F7F
1F7C  ED A0              LDI                 kopiera kommat
1F7E  E7                 RST  $20
1F7F  C3 A8 23  L1F7F:   JP   CMPGET         strängvariabel + $BE
```

INPUT LINE *vid körning: en hel rad till strängvariabeln, utan* ? *och utan ny rad efteråt på skärmen (inget* NEWLIN *som i* INPT*).*

120 byte reserveras på stacken och INPUT läser högst 119 tecken dit. Efter CR läggs LF, och strängen blir hela raden med CR LF sist. Sedan ASGNS med de 120 byten som tillfälligt område. In: DE efter token $8A. Ut: DE efter satsen.

```
1F82  FD 36 0D 00  INPLIN: LD   (IY+$0D),$00     (egenhet)
1F86  1A                   LD   A,(DE)           första byten i koden
1F87  FE 23                CP   $23              #?
1F89  20 0B                JR   NZ,L1F96
1F8B  13                   INC  DE               över #
1F8C  FF                   RST  $38              filnumret
1F8D  FD 75 0D             LD   (IY+$0D),L       IY+$0D = filnumret
1F90  1A                   LD   A,(DE)           kommat efter filnumret?
1F91  FE 2C                CP   $2C
1F93  20 01                JR   NZ,L1F96
1F95  13                   INC  DE               över det
1F96  FF           L1F96:  RST  $38              variabelns adress
1F97  E5                   PUSH HL               variabeln undan
1F98  21 88 FF             LD   HL,$FF88         120 byte på stacken
1F9B  39                   ADD  HL,SP
1F9C  F9                   LD   SP,HL            stacken flyttas ner
1F9D  01 77 00             LD   BC,$0077         högst 119 tecken
1FA0  D5                   PUSH DE               kodpekaren undan
1FA1  E5                   PUSH HL               områdets adress
1FA2  CD DD 08             CALL INPUT
1FA5  E1                   POP  HL               HL = området
1FA6  D1                   POP  DE
1FA7  E5                   PUSH HL               områdets adress
1FA8  01 00 00             LD   BC,$0000         BC = 0, CPIR räknar nedåt
1FAB  3E 0D                LD   A,$0D
1FAD  ED B1                CPIR                  leta upp CR
1FAF  36 0A                LD   (HL),$0A         HL efter CR
1FB1  AF                   XOR  A                A = 0
1FB2  67                   LD   H,A              H = 0
1FB3  91                   SUB  C                A = antal byte till och med CR
1FB4  3C                   INC  A                + LF
1FB5  6F                   LD   L,A              HL = längden
1FB6  E3                   EX   (SP),HL          (SP) = längden, HL = adressen
1FB7  E5                   PUSH HL               adressen
1FB8  21 78 00             LD   HL,$0078         120 byte tillfälligt
1FBB  E5                   PUSH HL
1FBC  C3 C3 29             JP   ASGNS            strängen till variabeln
```

Korsreferenstabell för `IF`, `INPUT`, `READ` och `INPUT LINE`. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnen är det valda filnumret vid IY+$0D. IF, INPT, RDVAL och INPLIN räknar ut uttryck med `RST $38`, och det som uttrycken gör räknas inte med, utom att A, flaggorna, BC, HL och IX förstörs. INPT och INPLIN läser med INPUT och skriver frågetecknet och radslutet med PRTXT och NEWLIN; vad de gör med markören och FCB:n står i tabellen efter $09EC. ASGNS kan reservera texten på heapen. Felen: CMPIF 11 och 17, CMPINP 13, READ 30, RDVAL 12, ASGNS 3 och 10 och INPUT 32. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler			IY+
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	DATPTR	DATLIN	$0D
CMPIF	U	U	X	IU	IU	X					
IF	X	X	X	IU	X	X					
CMPINP	U	U	X	IU	IU	X					
CMPRD	U	U	X	IU	IU	X					

	register och stack							variabler			IY+
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	DATPTR	DATLIN	$0D
INPT	X	X	X	IU	X	X		IU			U
READ	X	X	X	IU	X	X		IU	IU	IU	
RDVAL	X	X	X	IU	IU	X		IU			
CMPINL	U	U	X	IU	IU	X					
INPLIN	X	X	X	IU	X	X		IU			U

Rutin	**Anropar**	**Anropas från**
CMPIF	CMPINX, KWSRCH (KWTHEN, KWELSE), CMPREF, TOKSTM	STTAB1 ($88)
IF	RST38	STEXE1 ($88)
CMPINP	CMPINX, CMPVR (går in i CMPRD); fel 13	STTAB1 ($89)
CMPRD	CMPVR	CMPINP (går in i den), STTAB1 ($8E)
INPT	RST38, PRTXT, INPUT, NEWLIN, RDVAL	STEXE1 ($89)
READ	RDVAL, SKIPSP; fel 30	STEXE1 ($8E)
RDVAL	RST38, GETSFL, GETSIN, ASGNS (med JP); fel 12	INPT, READ
CMPINL	CMPINX, CMPGET (med JP)	STTAB1 ($8A)
INPLIN	RST38, INPUT, ASGNS (med JP)	STEXE1 ($8A)

3.51 ON: kompilering

$1FBF

```
ON uttryck GOTO|GOSUB|RESTORE nr [, nr ...]
      n $BA  token  $BE k  k * [nr lo][nr hi][adress lo][adress hi]
```

Token är $80 (GOTO), $90 (GOSUB) eller $8F (RESTORE), annars fel 25. Uttrycket är ett heltal. Adresserna fylls i vid förlänkningen, och REN ändrar numren. Saknas ett radnummer blir det fel 16. In: HL efter ON, DE = koden. Ut: NC, HL efter listan och DE efter koden; vid fel CY och A = felkoden. Förstör A, BC och IX.

```
1FBF  CD 70 23     CMPON:  CALL CMPINX          n, heltal + $BA
1FC2  D8                   RET  C               fel i uttrycket
1FC3  D5                   PUSH DE              koden undan
1FC4  11 11 19             LD   DE,STKW1        GOTO, GOSUB ...
1FC7  CD F5 09             CALL KWSRCH
1FCA  D1                   POP  DE              DE = koden
1FCB  20 0C                JR   NZ,L1FD9        inget satsord: fel 25
1FCD  12                   LD   (DE),A          token i koden
1FCE  13                   INC  DE
1FCF  FE 80                CP   $80             GOTO?
1FD1  28 06                JR   Z,L1FD9
1FD3  FE 90                CP   $90             GOSUB?
1FD5  28 02                JR   Z,L1FD9
1FD7  FE 8F                CP   $8F             RESTORE?
1FD9  3E 99        L1FD9:  LD   A,$99           fel 25
1FDB  37                   SCF                  CY + fel
1FDC  C0                   RET  NZ              NZ: fel 25
1FDD  3E BE                LD   A,$BE           $BE: lista med radreferenser
1FDF  12                   LD   (DE),A
1FE0  13                   INC  DE
1FE1  D5                   PUSH DE              plats för antalet
1FE2  13                   INC  DE              över antalet
1FE3  06 00                LD   B,$00           B = antalet
```

```
1FE5  C5          L1FE5:  PUSH BC          antalet undan
1FE6  D5                  PUSH DE          kodpekaren undan
1FE7  CD 1E 18            CALL GETLN       radnumret
1FEA  38 12               JR   C,L1FFE     inget radnummer: fel 16
1FEC  E3                  EX   (SP),HL     HL = kodpekaren
1FED  CD 23 10            CALL WRDE        radnumret i koden
1FF0  23                  INC  HL          två byte, ifylls vid förlänkningen
1FF1  23                  INC  HL
1FF2  D1                  POP  DE          DE = texten
1FF3  EB                  EX   DE,HL       HL = texten, DE = koden
1FF4  C1                  POP  BC
1FF5  04                  INC  B           räkna
1FF6  7E                  LD   A,(HL)      tecknet efter numret
1FF7  EE 2C               XOR  $2C         ,?
1FF9  20 0A               JR   NZ,L2005    inget ,: listan klar
1FFB  DF                  RST  $18         över kommat
1FFC  18 E7               JR   L1FE5

1FFE  C1          L1FFE:  POP  BC          kodpekaren
1FFF  C1                  POP  BC          antalet
2000  C1                  POP  BC          platsen för antalet
2001  3E 90               LD   A,$90       fel 16
2003  37                  SCF              CY + fel
2004  C9                  RET

2005  78          L2005:  LD   A,B         A = antalet
2006  C1                  POP  BC          BC = platsen för antalet
2007  02                  LD   (BC),A      antalet
2008  C9                  RET              NC från XOR
```

3.52 ON vid körning $2009

Värdet n (1–k) väljer referens nr n; annars fel 26, även för negativt n och n över 255. Ett flyttal avrundas nedåt (`ON 2.9 = ON 2`), och från 65536 blir det fel 7. Efter `ON GOSUB` fortsätter `RETURN` efter hela listan. DE = målradens adress, HL = efter listan. Hoppet görs genom att `RET` går till en del av `GOTO` (`$1DE4`), `GOSUB` (`$1DF2`) eller `RESTORE` (`$2297`), som har lagts på stacken. In: DE efter token $92.

```
2009  CD BF 0D    ON:     CALL NODIR       fel 2 i direktläge
200C  FF                  RST  $38         n i HL, avrundat nedåt
200D  1A                  LD   A,(DE)      A = token: GOTO, GOSUB eller RESTORE
200E  13                  INC  DE          över token
200F  13                  INC  DE          hoppa över $BE
2010  01 E4 1D            LD   BC,$1DE4    GOTO
2013  FE 80               CP   $80
2015  28 0A               JR   Z,L2021
2017  01 F2 1D            LD   BC,$1DF2    GOSUB
201A  FE 90               CP   $90
201C  28 03               JR   Z,L2021
201E  01 97 22            LD   BC,$2297    RESTORE
2021  C5          L2021:  PUSH BC          RET går dit
2022  AF                  XOR  A           A = 0
2023  B4                  OR   H           A = hög byte
2024  C2 1D 07            JP   NZ,L071D    n > 255: fel 26
2027  B5                  OR   L           låg byte
2028  CA 1D 07            JP   Z,L071D     n = 0: fel 26
202B  1A                  LD   A,(DE)      A = antalet k
202C  13                  INC  DE          DE = första posten
202D  95                  SUB  L           antal - n
```

```
202E  DA 1D 07          JP   C,L071D        n > antal: fel 26
2031  4F                LD   C,A            C = referenser efter nr n
2032  2D                DEC  L              n-1
2033  29                ADD  HL,HL          4*(n-1)
2034  29                ADD  HL,HL
2035  19                ADD  HL,DE          + första posten
2036  23                INC  HL             HL = posten + 1
2037  CD 1C 10          CALL RDDE1          DE = målradens adress
203A  06 00             LD   B,$00          BC = C
203C  09                ADD  HL,BC          4 byte per post kvar
203D  09                ADD  HL,BC
203E  09                ADD  HL,BC
203F  09                ADD  HL,BC
2040  C9                RET                 till GOTO, GOSUB eller RESTORE
```

ON ERROR GOTO *nr: spara målradens adress i* ONERR. *Nummer 0 ger adressen 0 (se* PRELNK*), och då är felhanteringen avstängd. In: DE vid* $BF. *Ut: DE efter satsen, HL = radens adress.*

```
2041  CD BF 0D    ONERRG: CALL NODIR          fel 2 i direktläge
2044  EB                  EX   DE,HL         HL vid $BF
2045  23                  INC  HL            hoppa över $BF och numret
2046  23                  INC  HL
2047  CD 1C 10            CALL RDDE1         DE = adressen, 0 för nummer 0
204A  EB                  EX   DE,HL         HL = adressen, DE = efter satsen
204B  22 34 FE            LD   (ONERR),HL    radens adress
204E  C9                  RET
```

ERRCODE*: koden för det senaste felet som* ON ERROR *fångade (IY+*$22*, sparas bara på* $0648*) utan bit 7. Fel i direktläge och fel som inte fångas ändrar den inte.* CLEAR *(och därmed* RUN*) nollar den, och 0 ger värdet -1; fel 0 finns inte i* ERRTRP*. Ut: värdet i HL, via* RST $28.

```
204F  FD 7E 22    EXERR:  LD   A,(IY+$22)     IY+$22: felkoden
2052  21 FF FF            LD   HL,$FFFF       -1: inget fångat fel
2055  A7                  AND  A
2056  28 04               JR   Z,L205C        0: inget fångat fel
2058  E6 7F               AND  $7F
205A  6F                  LD   L,A
205B  24                  INC  H              H = 0
205C  EF          L205C:  RST  $28             HL är funktionens värde, ett heltal
```

Korsreferenstabell för ON, ON ERROR GOTO och ERRCODE. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är DIRECT vid IY+$0E, FLAGS vid IY+$0F (bit 4 = TRACE) och felkoden vid IY+$22. ONERR är adressen till ON ERROR-raden, med höga byten 0 när ON ERROR är av. * = kommer inte tillbaka; för dem visas bara stacken, variablerna och det de tar emot. ON räknar ut index med RST $38 och går med RET till GOTO ($1DE4), GOSUB ($1DF2) eller RESTORE ($2297); deras verkan räknas med, men inte det som satserna på målraden gör. EXERR lämnar felkoden som heltal med RST $28. Felen: CMPON 25 och 16, ON 2 och 26 och ONERRG 2. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler							IY+		
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	CURLIN	STMSP	BRKFLG	DATPTR	DATLIN	ONERR	$0E	$0F	$22
CMPON	U	U	X	IU	IU	X											
ON*				I			I	I	U	U	IU	U	U		I	I	
ONERRG	X	X		IU	X									U	I		
EXERR*				I			U										I

Rutin	Anropar	Anropas från
CMPON	CMPINX, KWSRCH (STKW1), GETLN, WRDE; fel 25 och 16	STTAB1 ($92)
ON	NODIR, RST38, RDDE1; fel 2 och 26	STEXE1 ($92)
ONERRG	NODIR, RDDE1; fel 2	STEXE1 ($94)
EXERR	RST $28	EXPTAB2 (kod $1E)

3.53 OPEN / PREPARE: kompilering $205D

```
OPEN namn$ AS FILE n   (PREPARE likadant)
      namn$ $BB  $B8  n $BA
```

Namnet är ett stränguttryck, n ett heltal. Fel 33 om AS FILE saknas. CLOSE n: bara n $BA. In: HL efter OPEN eller PREPARE (CMPCLS: efter CLOSE), DE = koden. Ut: NC, HL efter satsen och DE efter koden; vid fel CY och A = felkoden. Förstör A, BC och IX.

```
205D  CD 6C 23     CMPOPN: CALL CMPSTX          namnet, sträng + $BB
2060  D8                  RET  C                fel i namnet
2061  D5                  PUSH DE               koden undan
2062  11 90 19            LD   DE,KWASF         tabellen med AS FILE
2065  CD F5 09            CALL KWSRCH           Z: AS FILE funnen, A = $B8
2068  47                  LD   B,A              B = $B8
2069  D1                  POP  DE               DE = koden
206A  3E A1               LD   A,$A1            fel 33
206C  37                  SCF                   CY + fel
206D  C0                  RET  NZ               NZ: fel 33
206E  78                  LD   A,B              $B8
206F  12                  LD   (DE),A           AS FILE
2070  13                  INC  DE               efter $B8
2071  C3 70 23    CMPCLS: JP   CMPINX           filnumret, heltal + $BA
```

3.54 OPEN / PREPARE vid körning $2074

A = 0 (OPEN) eller 1 (PREPARE) läggs på stacken, sedan namnet (en sträng på stacken) och filnumret. Fil 0 gör ingenting.

Filerna har var sin FCB på 15 byte, i en kedja från FILLST:

```
      +0/1 nästa FCB   +2 filnummer   +3- se OPEN ($07E2)
```

Finns filnumret redan stängs den filen först och FCB:n används igen. Annars reserveras en ny FCB och läggs sist i kedjan.

Sedan OPEN med HL = namnet, BC = längden och A = typen. A hämtas från stacken ovanför strängens tillfälliga område.

Till sist tas strängen och A bort från stacken.

Vid fil 0 tolkas inte ens namnet: OPEN "XYZ:" AS FILE 0 ger inget fel. Filen står öppen efter STOP, men programslutet (ENDPRG via CLEAR till CLSALL) stänger den, och en senare INPUT #n ger fel 32. In: DE efter token. Ut: DE efter satsen.

```
2074  97          OPENST: SUB  A                A = 0: OPEN
2075  18 02               JR   L2079            hoppa över LD A,$01

2077  3E 01       PREPST: LD   A,$01            A = 1: PREPARE
```

```
2079  F5           L2079:  PUSH AF
207A  FF                   RST  $38          namnet, en sträng vid SP
207B  13                   INC  DE           hoppa över $B8
207C  FF                   RST  $38          filnumret i HL, ett heltal + $BA
207D  D5                   PUSH DE           kodpekaren undan
207E  7D                   LD   A,L          A = filnumret
207F  A7                   AND  A
2080  28 35                JR   Z,L20B7      fil 0: ingenting
2082  CD E1 20             CALL FNDFIL       finns filen?
2085  28 16                JR   Z,L209D      Z: filen finns
2087  F5                   PUSH AF           filnumret
2088  11 0F 00             LD   DE,$000F     ny FCB, 15 byte
208B  CD 74 33             CALL ALLOC
208E  DD 75 00             LD   (IX+$00),L   sist i kedjan
2091  DD 74 01             LD   (IX+$01),H
2094  E5                   PUSH HL
2095  DD E1                POP  IX
2097  F1                   POP  AF
2098  DD 77 02             LD   (IX+$02),A   filnumret
209B  18 03                JR   L20A0

209D  CD 7F 08     L209D:  CALL CLOSE        stäng den gamla
20A0  21 02 00     L20A0:  LD   HL,$0002     HL = SP + 2, förbi kodpekaren
20A3  39                   ADD  HL,SP
20A4  CD 1D 10             CALL RDDE         DE = antal tillfälliga byte
20A7  D5                   PUSH DE           antalet
20A8  CD 1D 10             CALL RDDE         DE = namnets adress
20AB  CD 17 10             CALL RDBC         BC = namnets längd
20AE  EB                   EX   DE,HL        HL = adressen, DE efter längden
20AF  E3                   EX   (SP),HL      (SP) = adressen, HL = antalet
20B0  19                   ADD  HL,DE        förbi det tillfälliga området
20B1  23                   INC  HL           A:s byte i AF
20B2  7E                   LD   A,(HL)       A (OPEN/PREPARE)
20B3  E1                   POP  HL           HL = namnet
20B4  CD E2 07             CALL OPEN         öppna
20B7  D1           L20B7:  POP  DE           DE = koden
20B8  E1                   POP  HL           HL = antal tillfälliga byte
20B9  C1                   POP  BC           adressen
20BA  C1                   POP  BC           längden
20BB  39                   ADD  HL,SP        ta bort strängen
20BC  F9                   LD   SP,HL
20BD  E1                   POP  HL           och A
20BE  C9                   RET               ta bort AF
```

CLOSE *n: stäng filen om den finns.* CLOSE *0 och* CLOSE *av en fil som inte är öppen gör ingenting. In: DE efter token. Ut: DE efter satsen.*

```
20BF  FF           CLOSST: RST  $38          filnumret
20C0  7D                   LD   A,L
20C1  A7                   AND  A
20C2  C8                   RET  Z            0: ingenting
20C3  CD E1 20             CALL FNDFIL       finns filen?
20C6  C0                   RET  NZ           finns inte
20C7  D5                   PUSH DE           kodpekaren undan
20C8  CD 7F 08             CALL CLOSE        IX = FCB:n
20CB  D1                   POP  DE           DE = koden
20CC  C9                   RET
```

CLSALL*: stäng alla filer i* FILLST*-kedjan (från* CLEAR*).*

```
20CD  2A 32 FE     CLSALL: LD   HL,(FILLST)  första FCB
```

```
20D0  7C          L20D0:  LD   A,H
20D1  A7                  AND  A              hög byte 0: slut på kedjan
20D2  C8                  RET  Z
20D3  E5                  PUSH HL             IX = FCB:n
20D4  DD E1               POP  IX
20D6  CD 7F 08            CALL CLOSE
20D9  DD 6E 00            LD   L,(IX+$00)
20DC  DD 66 01            LD   H,(IX+$01)     nästa FCB
20DF  18 EF               JR   L20D0
```

FNDFIL: leta efter FCB för fil A. Ut: Z och IX = FCB om den finns, annars NZ och IX = sista FCB i kedjan (eller FILLST om kedjan är tom, så att IX+0/1 alltid är länken som ska ändras). Förstör BC.

```
20E1  FD E5       FNDFIL: PUSH IY             IX = IY
20E3  DD E1               POP  IX
20E5  01 1C 00            LD   BC,$001C       IX = FILLST ($FE32)
20E8  DD 09               ADD  IX,BC
20EA  DD 4E 00    L20EA:  LD   C,(IX+$00)     BC = länken
20ED  DD 46 01            LD   B,(IX+$01)
20F0  04                  INC  B              hög byte 0: slut på kedjan
20F1  05                  DEC  B
20F2  20 02               JR   NZ,L20F6
20F4  04                  INC  B              NZ: hittades inte
20F5  C9                  RET                 IX = sista FCB

20F6  C5          L20F6:  PUSH BC             IX = nästa FCB
20F7  DD E1               POP  IX
20F9  DD BE 02            CP   (IX+$02)       rätt filnummer?
20FC  20 EC               JR   NZ,L20EA
20FE  C9                  RET                 Z: funnen
```

3.55 OUT / POKE: kompilering $20FF

OUT p, v [, p, v ...] **POKE a, v [, v ...]**

Alla är heltal, och kommatecknen kopieras till koden:

```
p $BA "," v $BA ["," p $BA "," v $BA ...]
a $BA "," v $BA ["," v $BA ...]
```

Slingan görs med en returadress i BC: $2107 för OUT (nästa par), $2119 för POKE (nästa värde). Fel 13 om "," saknas efter p eller a (och därmed för ett udda antal tal i OUT); ett "," sist ger fel 11. In: HL efter OUT eller POKE, DE = koden. Ut: NC, HL efter satsen och DE efter koden; vid fel CY och A = felkoden. Förstör A, BC och IX.

```
20FF  01 07 21    CMPOUT: LD   BC,$2107       OUT: nästa par
2102  18 03                JR   L2107          hoppa över LD BC

2104  01 19 21    CMPPOK: LD   BC,$2119       POKE: nästa värde
2107  C5          L2107:  PUSH BC             returadressen undan
2108  CD 70 23              CALL CMPINX         p eller a
210B  C1                    POP  BC             BC tillbaka
210C  D8                    RET  C              fel i uttrycket
210D  7E                    LD   A,(HL)         tecknet efter
210E  EE 2C                 XOR  $2C            ,?
2110  3E 8D                 LD   A,$8D          fel 13
2112  37                    SCF                 CY + fel
2113  C0                    RET  NZ             NZ: fel 13
2114  C5                    PUSH BC
2115  ED A0                 LDI                 kopiera ,
2117  E7                    RST  $20            mellanslag efter
2118  C1                    POP  BC             BC tillbaka
```

```
2119  C5                PUSH BC           returadressen undan
211A  CD 70 23          CALL CMPINX       v
211D  C1                POP  BC
211E  D8                RET  C            fel i uttrycket
211F  7E                LD   A,(HL)       tecknet efter
2120  EE 2C             XOR  $2C
2122  C0                RET  NZ           inget ,: klart
2123  C5                PUSH BC           en gång för RET
2124  C5                PUSH BC           och en för POP BC
2125  ED A0             LDI               kopiera kommat
2127  E7                RST  $20          mellanslag efter
2128  C1                POP  BC           BC tillbaka
2129  C9                RET               till $2107 eller $2119
```

OUT *vid körning: för varje par skrivs först det sparade värdet för port 1 (IY+$15) till port 1 och sedan v till port p. Ett värde som skrivs till port 1 sparas där. Port 1 väljer enhet på ABC-bussen (abc80sim), och valet gäller för alla senare* OUT *och* INP.

Både p och v tas modulo 256, så även OUT *257,v räknas som port 1. Det sparade värdet är* $FF *från start;* CLEAR *och* RUN *ändrar det inte. In: DE efter token. Ut: DE efter satsen.*

```
212A  FF          OUTST:  RST  $38           p, heltal i HL
212B  13                  INC  DE           hoppa över ,
212C  E5                  PUSH HL           p undan
212D  FF                  RST  $38          v, heltal i HL
212E  C1                  POP  BC           BC = p
212F  79                  LD   A,C          A = porten (modulo 256)
2130  3D                  DEC  A            port 1 = val av enhet?
2131  20 03               JR   NZ,L2136     annan port
2133  FD 75 15            LD   (IY+$15),L   spara värdet
2136  FD 7E 15    L2136:  LD   A,(IY+$15)
2139  D3 01               OUT  ($01),A      port 1 först
213B  ED 69               OUT  (C),L        sedan v till port p
213D  1A                  LD   A,(DE)       tecknet efter v
213E  FE 2C               CP   $2C          ,?
2140  C0                  RET  NZ           nej: satsen klar
2141  13                  INC  DE           över kommat
2142  18 E6               JR   OUTST        nästa par
```

POKE *vid körning: värdena skrivs till a, a+1, a+2 ... med avbrotten avstängda, modulo 256 (*POKE *a,258 skriver 2). In: DE efter token. Ut: DE efter satsen.*

```
2144  FF          POKE:   RST  $38          a, heltal i HL
2145  13          L2145:  INC  DE           över kommat
2146  E5                  PUSH HL           a undan
2147  FF                  RST  $38          v
2148  7D                  LD   A,L          A = v modulo 256
2149  E1                  POP  HL           HL = a
214A  F3                  DI
214B  77                  LD   (HL),A       skriv
214C  FB                  EI
214D  23                  INC  HL           nästa adress
214E  1A                  LD   A,(DE)       tecknet efter v
214F  FE 2C               CP   $2C          ,?
2151  28 F2               JR   Z,L2145      ja: nästa värde
2153  C9                  RET
```

Korsreferenstabell för OPEN, PREPARE, CLOSE, OUT och POKE. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken, i systemvariablerna och i FCB:erna; IY-kolumnen är porten för OUT vid IY+$15. FCB-kolumnerna: +0 länken till nästa FCB (+0,+1) och +2 filnumret. FILLST pekar på första FCB:n. CY-kolumnen gäller flaggorna:

FNDFIL svarar med Z. OPENST, PREPST, OUTST och POKE räknar ut uttryck med RST $38, och det som uttrycken gör räknas inte med, utom att A, flaggorna, BC, HL och IX förstörs. OPENST och PREPST reserverar FCB:n med ALLOC och öppnar med OPEN; CLOSST och CLSALL stänger med CLOSE. Vad de gör med enheten och FCB:ns övriga fält står i tabellen efter $09EC. Felen: CMPOPN 33, CMPOUT och CMPPOK 13 och 11. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler			FCB		IY+
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	FILLST	DEVHEAD	+0	+2	$15
CMPOPN	U	U	X	IU	IU	X							
CMPCLS	U	U	X	IU	IU	X							
OPENST	X	X	X	IU	X	X		IU	IU	I	IU	IU	
PREPST	X	X	X	IU	X	X		IU	IU	I	IU	IU	
CLOSST	X	X	X	IU	X	X			I		I	I	
CLSALL	X	X	X	X	X	X			I		I		
FNDFIL	I	U	X			U			I		I	I	
CMPOUT	U	U	X	IU	IU	X							
CMPPOK	U	U	X	IU	IU	X							
OUTST	X	X	X	IU	X	X							IU
POKE	X	X	X	IU	X	X							

Rutin	**Anropar**	**Anropas från**
CMPOPN	CMPSTX, KWSRCH (KWASF), CMPINX (med JP); fel 33	STTAB2 (OPEN, PREPARE)
CMPCLS	CMPINX	STTAB2 (CLOSE)
OPENST, PREPST	RST38, FNDFIL, ALLOC, CLOSE, RDDE, RDBC, OPEN	STEXE2 (OPEN, PREPARE)
CLOSST	RST38, FNDFIL, CLOSE	STEXE2 (CLOSE)
CLSALL	CLOSE	CLEAR ($0A82)
FNDFIL		CURFIL ($0902), OPENST, CLOSST
CMPOUT, CMPPOK	CMPINX; fel 13 och 11	STTAB2 (OUT, POKE)
OUTST	RST38	STEXE2 (OUT)
POKE	RST38	STEXE2 (POKE)

3.56 PRINT: kompilering $2154

```
PRINT [#n,] [uttryck] [, eller ;] [uttryck] ...   (";" = PRINT)
      ["#" n $BA] ... uttryck $B9+t ... "," ";" ...
```

Tecknen ”#”, ”,” och ”;” kopieras till koden, och varje uttryck följs av $B9+typ. Två uttryck får stå direkt efter varandra (PRINT AB), utom när det första är ett flyttal och det andra börjar med en bokstav (även gemen och ÄÖÅ): då blir det fel 11 (?: så att till exempel PRINT A B inte godtas). PRINT A%B och PRINT 1(2) går, och PRINT 1 2 skriver 12, eftersom talet läses förbi blanka (GETFLT).

Satsen slutar vid ”:”, CR eller ELSE (STMEND). In: HL efter PRINT eller ”;”, DE = koden. Ut: HL vid satsens slut och DE efter koden; vid fel CY och A = felkoden. Förstör A, BC och IX.

```
2154  7E          CMPPRT: LD   A,(HL)          (egenhet)
2155  FE 23               CP   $23             #?
2157  20 07               JR   NZ,L2160
```

```
2159  ED A0              LDI                 kopiera #
215B  E7                 RST  $20            mellanslag efter
215C  CD 70 23           CALL CMPINX         filnumret
215F  D8                 RET  C              fel i filnumret
2160  7E           L2160: LD   A,(HL)         nästa tecken i texten
2161  FE 2C              CP   $2C            ,?
2163  28 23              JR   Z,L2188
2165  FE 3B              CP   $3B            ;?
2167  28 1F              JR   Z,L2188
2169  CD 8D 21           CALL STMEND         slut på satsen?
216C  C8                 RET  Z              Z: satsen klar
216D  CD 62 23           CALL CMPEXB         uttryck + $B9 + typ
2170  D8                 RET  C              fel i uttrycket
2171  3E 00              LD   A,$00          0 = flyttal
2173  B8                 CP   B              inte flyttal: vidare
2174  20 EA              JR   NZ,L2160
2176  CD 8D 21           CALL STMEND         slut efter talet?
2179  C8                 RET  Z              ja: klart
217A  7E                 LD   A,(HL)         tecknet efter flyttalet
217B  E6 5F              AND  $5F            stora bokstäver
217D  FE 41              CP   $41            bokstav, A-Z ÄÖÅ?
217F  38 DF              JR   C,L2160        under A: vidare
2181  FE 5E              CP   $5E
2183  30 DB              JR   NC,L2160       över Å: vidare
2185  3E 8B              LD   A,$8B          fel 11
2187  C9                 RET                 CY från CP

2188  ED A0        L2188: LDI                 kopiera , eller ;
218A  E7                 RST  $20            mellanslag efter
218B  18 D3              JR   L2160          nästa del
```

STMEND: *Z om HL står på slutet av en sats:* :, *CR eller* ELSE. *In: HL = texten. Ut: Z vid slutet, A = tecknet. Förstör A.*

```
218D  7E           STMEND: LD   A,(HL)
218E  FE 3A                CP   $3A            :?
2190  C8                  RET  Z
2191  FE 0D                CP   $0D            CR?
2193  C8                  RET  Z
2194  D5                  PUSH DE              DE och HL undan
2195  E5                  PUSH HL
2196  11 80 19            LD   DE,KWELSE       ELSE?
2199  CD F5 09            CALL KWSRCH          Z: ELSE
219C  E1                  POP  HL              HL tillbaka, inte över ordet
219D  D1                  POP  DE
219E  C9                  RET                  Z: slutet
```

3.57 PRINT vid körning $219F

Filnumret väljs med IY+$0D (0 = skärmen). Sedan, i ordning:

```
","     nästa tabulatorstopp (var 14:e kolumn), eller ny rad om
        det inte får plats
";"     ingenting
uttryck skrivs ut (PRTEXP)
slut    CR LF, utom när satsen slutar med "," eller ";"
```

Ett ”,” direkt efter #n räknas som ”;”. In: DE efter token $84 eller $85. Ut: DE efter satsen.

```
219F  FD 36 0D 00  PRINTS: LD   (IY+$0D),$00
21A3  1A                   LD   A,(DE)          första byten
21A4  FE 23                CP   $23             #?
21A6  20 0A                JR   NZ,L21B2
21A8  13                   INC  DE              över #
21A9  FF                   RST  $38             filnumret
21AA  FD 75 0D             LD   (IY+$0D),L      välj filen
21AD  1A                   LD   A,(DE)
21AE  FE 2C                CP   $2C             , efter #n: som ;
21B0  28 07                JR   Z,L21B9
21B2  1A           L21B2:  LD   A,(DE)          nästa byte i koden
21B3  FE 2C                CP   $2C             ,?
21B5  28 65                JR   Z,PRTCOM        tabulatorstopp
21B7  FE 3B                CP   $3B             ;?
21B9  CA 3A 22     L21B9:  JP   Z,PRTSEP        hoppa över separatorn
21BC  FE C0                CP   $C0             uttryck?
21BE  38 36                JR   C,PRCRLF        nej: CR LF och klart
```

PRTEXP: *räkna ut uttrycket och skriv det.*

```
flyttal  FLTAS6 i ett område på 16 byte  på stacken
heltal   S1855 i bufferten $FDD5: mellanslag eller "-", sedan
         siffrorna
sträng   texten skrivs direkt från stacken
```

Tal som inte får plats på raden (PRTFIT) börjar på en ny rad. Strängar kontrolleras inte. Efteråt tas stackområdet bort. In: DE vid uttrycket.

```
21C0  FF           PRTEXP: RST  $38              värdet: HL, eller på stacken
21C1  78                   LD   A,B              A = typen
21C2  FE 01                CP   $01              heltal?
21C4  28 40                JR   Z,L2206
21C6  FE 02                CP   $02              sträng?
21C8  28 49                JR   Z,L2213
21CA  21 F0 FF             LD   HL,$FFF0         flyttal: 16 byte på stacken
21CD  39                   ADD  HL,SP
21CE  F9                   LD   SP,HL
21CF  D5                   PUSH DE
21D0  EB                   EX   DE,HL            DE = området, HL = koden
21D1  21 14 00             LD   HL,$0014         flyttalet (sista byten)
21D4  19                   ADD  HL,DE
21D5  D5                   PUSH DE               områdets början
21D6  CD 77 16             CALL FLTAS6           till text
21D9  21 15 00             LD   HL,$0015         16 + 5 byte ska bort
21DC  E3           L21DC:  EX   (SP),HL          HL = början, (SP) = byte som ska bort
21DD  EB                   EX   DE,HL            HL = slutet, DE = början
21DE  A7                   AND  A
21DF  ED 52                SBC  HL,DE            HL = längden
21E1  44                   LD   B,H              BC = längden
21E2  4D                   LD   C,L
21E3  EB                   EX   DE,HL            HL = texten
21E4  CD F0 21             CALL PRTFIT           ny rad om det inte får plats
```

```
21E7  CD D4 08     L21E7:  CALL PRINT          skriv
21EA  E1                   POP  HL             HL = byte som ska bort
21EB  D1                   POP  DE             DE = koden
21EC  39                   ADD  HL,SP          ta bort stackområdet
21ED  F9                   LD   SP,HL
21EE  18 C2                JR   L21B2          nästa del
```

PRTFIT: *BC = längden. Om kolumn + längd inte är mindre än radbredden, skriv CR LF först* (GETPOS: *A = kolumn, E = bredd*). *Ett tal som skulle sluta i sista kolumnen börjar alltså på nästa rad.*

```
21F0  CD EF 08     PRTFIT: CALL GETPOS         A = kolumn, E = radbredd
21F3  81                   ADD  A,C            kolumn + längd
21F4  93                   SUB  E              - bredd
21F5  D8                   RET  C              får plats; annars vidare i PRCRLF
```

PRCRLF: *skriv CR LF* ($072D). *Bevarar BC, DE och HL.*

```
21F6  D5           PRCRLF: PUSH DE
21F7  C5                   PUSH BC
21F8  E5                   PUSH HL
21F9  21 2D 07             LD   HL,$072D       CR LF
21FC  01 02 00             LD   BC,$0002
21FF  CD D4 08             CALL PRINT          skriv
2202  E1                   POP  HL
2203  C1                   POP  BC
2204  D1                   POP  DE
2205  C9                   RET

2206  D5           L2206:  PUSH DE             kodpekaren undan
2207  11 D5 FD             LD   DE,$FDD5       texten i $FDD5
220A  D5                   PUSH DE             början
220B  CD 55 18             CALL PRSIN          tecken och siffror
220E  21 00 00             LD   HL,$0000       inget stackområde
2211  18 C9                JR   L21DC          räkna längden och skriv

2213  DD E1       L2213:  POP  IX             antal tillfälliga byte
2215  E1                   POP  HL             adress
2216  C1                   POP  BC             längd
2217  D5                   PUSH DE             kodpekaren undan
2218  DD E5                PUSH IX             antalet tillfälliga byte överst
221A  18 CB                JR   L21E7          skriv, sedan bort från stacken
```

PRTCOM: *,. C = 14 - (kolumn mod 14) mellanslag ur* SPACES. *Går det över radbredden blir det ny rad i stället. På 40 tecken finns alltså stoppen 14 och 28, och från kolumn 28 blir det ny rad. In: DE vid ","*.

```
221C  D5           PRTCOM: PUSH DE             kodpekaren undan
221D  CD EF 08             CALL GETPOS         A = kolumn, E = bredd
2220  47                   LD   B,A            B = kolumnen
2221  D6 0E        L2221:  SUB  $0E             kolumn mod 14
2223  30 FC                JR   NC,L2221       tills det slår runt
2225  3D                   DEC  A              DEC och CPL byter tecken:
2226  2F                   CPL                 A = 14 − kolumn mod 14
2227  4F                   LD   C,A            C = mellanslag till nästa stopp
2228  80                   ADD  A,B            + kolumnen B: nästa stopp
2229  BB                   CP   E              före radens bredd E?
222A  38 05                JR   C,L2231        ja: mellanslagen, annars CR LF
222C  CD F6 21             CALL PRCRLF         nej: ny rad
222F  18 08                JR   L2239

2231  06 00        L2231:  LD   B,$00           BC = C
2233  21 4A 22             LD   HL,SPACES      mellanslag
2236  CD D4 08             CALL PRINT          skriv C mellanslag
```

```
2239  D1          L2239:  POP  DE              DE = koden, vid ,
```

PRTSEP: *efter* , *eller* ;. *Kommer det inget uttryck efter slutar* PRINT *utan CR LF. In: DE vid "," eller ";".*

```
223A  13          PRTSEP: INC  DE              över separatorn
223B  1A                  LD   A,(DE)
223C  FE 2C               CP   $2C             ,?
223E  28 DC               JR   Z,PRTCOM
2240  FE 3B               CP   $3B             ;?
2242  28 F6               JR   Z,PRTSEP        fler ;
2244  FE C0               CP   $C0             uttryck?
2246  D2 C0 21            JP   NC,PRTEXP       nästa uttryck
2249  C9                  RET                  slut, utan CR LF

224A  20 20 20 20… SPACES: DEFB "              " 14 mellanslag, en kolumnbredd för ,
```

Korsreferenstabell för PRINT. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnen är det valda filnumret vid IY+$0D. CY-kolumnen gäller flaggorna: STMEND svarar med Z. PRINTS och PRTEXP räknar ut uttryck med RST $38, och det som uttrycken gör räknas inte med, utom att A, flaggorna, BC, HL och IX förstörs. Alla skriver med PRINT och läser kolumnen med GETPOS; vad de gör med enheten och FCB:n står i tabellen efter $09EC. PRTEXP gör om talen till text med FLTAS6 i 16 byte under SP och med PRSIN i $FDD5. Felen: CMPPRT 11. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler		IY+
Rutin	A	CY	BC	DE	HL	IX	stack	CURROW	CURCOL	$0D
CMPPRT	U	U	X	IU	IU	X				
STMEND	U	U			I					
PRINTS	X	X	X	IU	X	X		IU	IU	U
PRTEXP	X	X	X	IU	X	X		IU	IU	I
PRTFIT	X	X	I	X		X		IU	IU	I
PRCRLF	X	X				X		IU	IU	I
PRTCOM	X	X	X	IU	X	X		IU	IU	I
PRTSEP	X	X	X	IU	X	X		IU	IU	I

Rutin	**Anropar**	**Anropas från**
CMPPRT	CMPINX, STMEND, CMPEXB; fel 11	STTAB1 ($84, $85)
STMEND	KWSRCH (KWELSE)	CMPNXT, CMPPRT (två ställen)
PRINTS	RST38, PRCRLF (med JR), PRTCOM (med JR), PRTSEP (med JP); går in i PRTEXP	STEXE1 ($84, $85)
PRTEXP	RST38, FLTAS6, PRSIN, PRTFIT, PRINT	PRINTS (går in i den), PRTSEP (med JP)
PRTFIT	GETPOS; går in i PRCRLF	PRTEXP
PRCRLF	PRINT	PRINTS (med JR), PRTFIT (går in i den), PRTCOM
PRTCOM, PRTSEP	GETPOS, PRCRLF, PRINT; PRTEXP (med JP)	PRINTS (med JR och JP)

RANDOMIZE: *slumptalsfröet* $FE39-$FE3D *fylls med R-registret.*

RNDINI (*från* RUN) *använder masken C = 0, så fröet blir noll och* RND *ger samma följd varje gång programmet körs.*

R ökar med 6 per varv i slingan (sex M1-cykler), så de fem byten blir R, R+6, R+12 ... ROM:en skriver aldrig till R, och bit 7 är 0 efter RESET. RANDOMIZE *kan därför bara ge 128 olika frön. Förstör A, BC och HL.*

```
2258  01 FF 05     RANDOM: LD   BC,$05FF        C = $FF: hela R; B = 5 byte (gränsfall)
225B  18 03                 JR   L2260
225D  01 00 05     RNDINI: LD   BC,$0500
2260  21 39 FE     L2260:  LD   HL,$FE39        slumptalsfröet
2263  ED 5F        L2263:  LD   A,R              R AND mask
2265  A1                   AND  C
2266  77                   LD   (HL),A          R, R+6, R+12 ...
2267  23                   INC  HL
2268  10 F9                DJNZ L2263           fem byte
```

CMPNOP: RANDOMIZE *m.fl. har ingen kod efter token.*

```
226A  C9           CMPNOP: RET
```

CMPREM: DATA *och* REM. *Resten av raden kopieras oförändrad till koden. (Därför måste* DATA *stå sist, och* READ *läser värdena direkt ur texten med* RDVAL.*) In: HL = texten, DE = koden. Ut: NC, HL vid CR och DE efter koden. Förstör A och BC.*

```
226B  3E 0D        CMPREM: LD   A,$0D           CR
226D  BE           L226D:  CP   (HL)             till CR
226E  C8                   RET  Z
226F  ED A0                LDI                  kopiera ett tecken
2271  18 FA                JR   L226D           nästa tecken
```

DATA *vid körning: fel 2 i direktläge, annars som* REM*: hoppa till CR (DE pekar på CR efteråt). Förstör A och BC.*

```
2273  CD BF 0D     DATA:   CALL NODIR           fel 2 i direktläge
2276  3E 0D        REM:    LD   A,$0D           CR
2278  EB                   EX   DE,HL           HL = koden
2279  01 00 00             LD   BC,$0000        BC = 0: ingen gräns
227C  ED B1                CPIR                 leta upp CR
227E  EB                   EX   DE,HL
227F  1B                   DEC  DE              DE på CR
2280  C9                   RET
```

RESTORE [*nr*]*: kompilering. Med radnummer en radreferens ($BF), annars ingenting efter token (*CMPREF*:s fel ignoreras). In: HL efter* RESTORE*, DE = koden. Ut: NC, HL efter satsen och DE efter koden. Förstör A och BC.*

```
2281  CD C7 1D     CMPRES: CALL CMPREF          $BF nr adress, eller fel
2284  A7                   AND  A               CY = 0 även utan radnummer
2285  C9                   RET
```

RESTORE *vid körning: med radnummer börjar* READ *på den raden om den börjar med* DATA*, annars på nästa* DATA*-rad efter den. Utan radnummer från programmets början (*RSTDAT*). En rad som saknas ger fel 6 vid länkningen. In: DE efter token* $8F*. Ut: DE efter satsen.*

```
2286  1A           RESTOR: LD   A,(DE)          byten efter token
2287  FE BF                CP   $BF             radnummer?
2289  20 08                JR   NZ,RSTDAT       nej: från början
228B  EB                   EX   DE,HL           HL vid $BF
228C  23                   INC  HL
228D  23                   INC  HL
228E  CD 1C 10             CALL RDDE1           DE = radens adress, HL efter satsen
2291  18 04                JR   L2297           DE till HL i $2297
```

RSTDAT: DATA-pekaren. DATLIN får första raden, och om den börjar med DATA ($93) pekar DATPTR på dess data. Annars pekar DATPTR på $1ECA, en byte $0D inuti ROM-koden, så att nästa READ letar vidare. DE behålls. Förstör A och HL.

```
2293  2A 1C FE    RSTDAT: LD   HL,(BOFA)       första raden
2296  EB                  EX   DE,HL           två EX: DE oförändrad
2297  EB          L2297:  EX   DE,HL           HL = raden, DE = koden
2298  22 30 FE            LD   (DATLIN),HL     nästa READ börjar här
229B  7E                  LD   A,(HL)          radens längd
229C  FE 01               CP   $01             1: programmet slut
229E  28 09               JR   Z,L22A9
22A0  23                  INC  HL              över längden och numret
22A1  23                  INC  HL
22A2  23                  INC  HL
22A3  7E                  LD   A,(HL)          första token
22A4  23                  INC  HL              HL efter token
22A5  FE 93               CP   $93             DATA?
22A7  28 03               JR   Z,L22AC         ja: data efter token
22A9  21 CA 1E    L22A9:  LD   HL,$1ECA        $0D i READ: nästa DATA-rad
22AC  22 2E FE    L22AC:  LD   (DATPTR),HL     nästa READ läser här
22AF  C9                  RET
```

3.58 RETURN $22B0

Stacken när en sats körs: [till RUNSTM] [till RUNLIN] [RUNLIN:s anropare]. De två POP tar bort de två första, och RET går då till den som anropade RUNLIN:

- GOSUB ($1DF9): DE = koden efter GOSUB, och den gamla raden fortsätter.
- RUN: $071B, fel 29 (”RETURN” UTAN ”GOSUB”).

Resten av RETURN-raden körs aldrig. CURLIN pekar på RETURN-raden tills nästa rad börjar: ett fel eller STOP i resten av GOSUB-raden får RETURN-radens nummer. I direktläge fel 2 först när satsen körs.

```
22B0  CD BF 0D    RETURN: CALL NODIR           fel 2 i direktläge
22B3  E1                  POP  HL              ta bort RUNSTM och RUNLIN
22B4  E1                  POP  HL
```

NOCODE: satser utan argument (RETURN, STOP, END, TRACE, NOTRACE) kompileras till bara token.

```
22B5  C9          NOCODE: RET
```

Korsreferenstabell för RANDOMIZE, DATA, REM, RESTORE och RETURN. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; $FE39 är slumptalsfröet (fem byte) och IY-kolumnen DIRECT vid IY+$0E. CMPNOP och NOCODE gör ingenting. * = kommer inte tillbaka; för den visas bara stacken, variablerna och det den tar emot. RETURN tar bort returadresserna till RUNSTM och RUNLIN och går med RET tillbaka efter GOSUB ($1DF9), eller till $071B (fel 29) när det inte finns någon GOSUB. Felen: DATA 2, RETURN 2 och 29. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler				IY+
Rutin	A	CY	BC	DE	HL	IX	stack	BOFA	DATPTR	DATLIN	$FE39	$0E
RANDOM	X	X	X		X						U	
RNDINI	X	X	X		X						U	

	register och stack							variabler				IY+
Rutin	A	CY	BC	DE	HL	IX	stack	BOFA	DATPTR	DATLIN	$FE39	$0E
CMPNOP												
NOCODE												
CMPREM	X	U	X	IU	IU							
DATA	X	X	X	IU								I
REM	X	X	X	IU								
CMPRES	X	U	X	IU	IU							
RESTOR	X	X		IU	X			I	U	U		
RSTDAT	X	X			X			I	U	U		
RETURN*							I					I

Rutin	**Anropar**	**Anropas från**
RANDOM		STEXE2 (RANDOMIZE)
RNDINI		CMDRUN ($0D5E)
CMPNOP		STTAB2 (RANDOMIZE)
NOCODE		STTAB1 ($91), STTAB2 (STOP, END, TRACE, NOTRACE)
CMPREM		STTAB1 ($93), STTAB2 (REM)
DATA	NODIR; fel 2; går in i REM	STEXE1 ($93)
REM		DATA (går in i den), STEXE2 (REM)
CMPRES	CMPREF	STTAB1 ($8F)
RESTOR	RDDE1, RSTDAT (med JR)	STEXE1 ($8F)
RSTDAT		CMDRUN ($0D5B), RESTOR (med JR); ON går med RET till $2297
RETURN	NODIR; fel 2 och 29	STEXE1 ($91)

CMPDOT: SETDOT *r,k* / CLRDOT *r,k: r* $BA , *k* $BA.

Kontrollen av kommat fungerar inte: SCF *före* RET *NC gör att den aldrig slår till, och tecknet efter r kopieras vad det än är:* SETDOT 5;6 *och* SETDOT 5X6 *sätter punkten 5,6. Fel 61 (KOMMA SAKNAS) kommer alltså inte härifrån. Utan k beror utfallet på det som står efter raden: fel 11 eller 14, eller i direktläge en punkt. In: HL efter* SETDOT *eller* CLRDOT, *DE = koden. Ut: som* CMPINX.

```
22B6  CD 70 23     CMPDOT: CALL CMPINX          r (fel)
22B9  D8                   RET  C               fel i r
22BA  7E                   LD   A,(HL)          tecknet efter r
22BB  EE 2C                XOR  $2C             ,? (svaret i Z)
22BD  3E BD                LD   A,$BD           fel 61, KOMMA SAKNAS
22BF  37                   SCF                  (CY = 1: tas aldrig)
22C0  D0                   RET  NC              borde ha varit RET NZ
22C1  ED A0                LDI                  kopiera tecknet
22C3  E7                   RST  $20             över mellanslag
22C4  C3 70 23             JP   CMPINX          k
```

SETDOT / CLRDOT: *sätt eller släck en grafikpunkt i skärmens grafik.*

*Bit 5 sätts alltid, så att tecknet blir ett grafiktecken. Står det text där ORas biten in i tecknet (*A *blir* a*). CLRDOT lämnar bit 5, så ett tomt grafiktecken blir* $20*. r och k avrundas nedåt. In: DE efter token. Ut: DE efter satsen.*

```
22C7  CD E3 22     SETDOT: CALL DOTARG          HL = adressen, A = biten
22CA  B6                   OR   (HL)            sätt biten
22CB  F6 20        L22CB:  OR   $20             grafiktecken
22CD  77                   LD   (HL),A
22CE  C9                   RET
```

```
22CF  CD E3 22     CLRDOT: CALL DOTARG          HL = adressen, A = biten
22D2  2F                   CPL                  släck biten
22D3  A6                   AND  (HL)            släck den
22D4  18 F5                JR   L22CB
```

DOT(r,k): -1 om grafikpunkten är satt, annars 0. CY från DOTADR *prövas inte: utanför skärmen blir det inget fel 62, utan A* AND *en ROM-byte (på adressen k, eller* $0372 *när r >= 72). Även texttecken ger -1 när biten råkar vara satt.*

```
22D6  E1           EXDOT:  POP  HL               k (fel)
22D7  C1                   POP  BC               r
22D8  CD EE 22             CALL DOTADR           HL = adressen, A = biten
22DB  A6                   AND  (HL)             satt?
22DC  21 00 00             LD   HL,$0000         0: inte satt
22DF  28 01                JR   Z,L22E2
22E1  2B                   DEC  HL               ja: -1
22E2  EF           L22E2:  RST  $28              HL är funktionens värde
```

DOTARG*: räkna ut r och k (koden* `r $BA , k $BA`*) och ta fram adress och bit med* DOTADR*. Fel 62 om grafikpunkten är utanför skärmen. In: DE vid r. Ut: som* DOTADR*, och DE efter k.*

```
22E3  FF           DOTARG: RST  $38              r
22E4  E5                   PUSH HL               r undan
22E5  13                   INC  DE               hoppa över ,
22E6  FF                   RST  $38              k
22E7  C1                   POP  BC               BC = r
22E8  CD EE 22             CALL DOTADR           HL = adressen, A = biten
22EB  D0                   RET  NC               NC: på skärmen
22EC  D7                   RST  $10
22ED  BE                   DEFB $BE              fel 62: DOT-ADRESS UTANFÖR SKÄRMEN
```

3.59 DOTADR: skärmadress och bit för en grafikpunkt $22EE

In: BC = r (rad, 0–71), HL = k (kolumn, 0–79).

Ut: HL = tecknets adress på skärmen, A = biten. CY = utanför. Varje tecken är 2 × 3 grafikpunkter:

```
k jämn  k udda
$01     $02       r mod 3 = 0
$04     $08       r mod 3 = 1
$10     $40       r mod 3 = 2
```

Teckenraden = r/3 (radtabellen på $0374), teckenkolumnen = k/2. Biten räknas fram genom att rotera $40 eller $80 två steg per rad; bit 5 hoppas över med ADD $20 / AND $DF, så $20 blir $40. Förstör BC.

```
22EE  7C           DOTADR: LD   A,H
22EF  B0                   OR   B                over 255?
22F0  37                   SCF
22F1  C0                   RET  NZ               NZ: utanför, CY
22F2  B5                   OR   L                A = k
22F3  FE 50                CP   $50              k < 80?
22F5  3F                   CCF                   CY om k >= 80
22F6  D8                   RET  C                utanför
22F7  06 40                LD   B,$40            jämn kolumn: $40
22F9  1F                   RRA                   k/2
22FA  30 02                JR   NC,L22FE
22FC  06 80                LD   B,$80            udda: $80
22FE  6F           L22FE:  LD   L,A              L = k/2
```

```
22FF  79              LD   A,C          A = r
2300  4D              LD   C,L          C = k/2
2301  21 72 03        LD   HL,$0372     radtabellen - 2
2304  FE 48           CP   $48          r < 72?
2306  3F              CCF               CY: r >= 72
2307  D8              RET  C            utanför
2308  23        L2308: INC  HL          +2 per teckenrad
2309  23              INC  HL
230A  D6 03           SUB  $03          r/3
230C  30 FA           JR   NC,L2308     tills r/3 är räknad
230E  CB 08       L230E: RRC  B         två steg per rad
2310  CB 08              RRC  B
2312  3C              INC  A            A = -3, -2 eller -1 från början
2313  20 F9           JR   NZ,L230E
2315  7E              LD   A,(HL)       radens adress, låg byte
2316  81              ADD  A,C
2317  23              INC  HL
2318  66              LD   H,(HL)       hög byte
2319  6F              LD   L,A
231A  78              LD   A,B          A = biten
231B  C6 20           ADD  A,$20        hoppa över bit 5
231D  E6 DF           AND  $DF
231F  C9              RET
```

STOP*: fel 2 i direktläge. Annars "fel" 0, som skrivs som* STOP LINE *nn. Resten av raden körs inte, och* ON ERROR *fångar inte* STOP *(fel 0 finns inte i* ERRTRP*).*

```
2320  CD BF 0D    STOP:   CALL NODIR
```

BREAK under körning: "fel" med kod 0, som ERRMSG *skriver som* STOP*.*

```
2323  D7          BRKSTP: RST  $10           fångas inte av ON ERROR
2324  00                  DEFB $00           kod 0: skrivs som STOP
```

TRACE / NOTRAC*: bit 4 i IY+*$0F*, som* RUNLIN *prövar före varje rad (*$0D7B*). Förstör ingenting.*

```
2325  FD CB 0F E6 TRACE:  SET  4,(IY+$0F)    bit 4: TRACE på
2329  C9                  RET

232A  FD CB 0F A6 NOTRAC: RES  4,(IY+$0F)    bit 4: TRACE av
232E  C9                  RET
```

TRACE*: skriv radnumret för raden som ska köras (från* RUNLIN*, en gång per rad, inte per sats). DE = raden. Numret skrivs med* PRNUM *i 10 byte på stacken, följt av* $1F*, som i bildminnet syns som blankt. Ny rad när kolumnen + 7 >= 40: 11 tvåsiffriga nummer per rad, 6 femsiffriga. Direktrader skrivs inte.* TRACE *mitt på en rad gäller från nästa rad.* CLEAR *och* RUN *behåller* TRACE*,* NEW *slår av det. In: DE = raden. Ut: DE oförändrad. Förstör A, BC, HL och IX.*

```
232F  D5          TRCPRT: PUSH DE
2330  CD 38 03           CALL SCRPOS        A = kolumnen, E = 40
2333  C6 07               ADD  A,$07         plats för numret?
2335  BB                  CP   E
2336  D4 42 07            CALL NC,NEWLIN     ryms inte: ny rad
2339  DD E1               POP  IX            IX = raden
233B  21 F6 FF            LD   HL,$FFF6      10 byte på stacken
233E  39                  ADD  HL,SP
233F  F9                  LD   SP,HL
2340  EB                  EX   DE,HL         DE = området
2341  DD 6E 01            LD   L,(IX+$01)    radnumret
2344  DD 66 02            LD   H,(IX+$02)
2347  DD E5               PUSH IX            raden undan
```

```
2349  D5              PUSH DE          början undan
234A  CD 62 18        CALL PRNUM       numret, utan tecken
234D  EB              EX   DE,HL       HL efter siffrorna
234E  36 1F           LD   (HL),$1F    syns som blankt
2350  23              INC  HL
2351  D1              POP  DE          DE = början
2352  A7              AND  A
2353  ED 52           SBC  HL,DE       HL = längden
2355  EB              EX   DE,HL       DE = längden, HL = texten
2356  42              LD   B,D         BC = längden
2357  4B              LD   C,E
2358  CD A8 01        CALL PRTXT       direkt i bildminnet
235B  D1              POP  DE          DE = raden
235C  21 0A 00        LD   HL,$000A    ta bort området
235F  39              ADD  HL,SP
2360  F9              LD   SP,HL       SP tillbaka
2361  C9              RET
```

Korsreferenstabell för SETDOT, CLRDOT, DOT, STOP, TRACE och NOTRACE. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är DIRECT vid IY+$0E och FLAGS vid IY+$0F (bit 4 = TRACE). CY-kolumnen gäller flaggorna: DOTADR svarar med CY = utanför skärmen. DOTARG räknar ut r och k med RST $38, och det som uttrycken gör räknas inte med, utom att A, flaggorna, BC, HL och IX förstörs. SETDOT och CLRDOT ändrar tecknet i bildminnet. * = kommer inte tillbaka; för dem visas bara stacken, variablerna och det de tar emot. EXDOT lämnar värdet med RST $28, och STOP och BRKSTP går till felhanteringen med RST $10 och kod 0, som skrivs som STOP och inte fångas av ON ERROR. TRCPRT skriver radnumret direkt i bildminnet med PRTXT och gör 10 byte under SP till text med PRNUM. Felen: DOTARG 62, STOP 2; fel 61 i CMPDOT tas aldrig. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler		IY+	
Rutin	A	CY	BC	DE	HL	IX	stack	CURROW	CURCOL	$0E	$0F
CMPDOT	U	U	X	IU	IU	X					
SETDOT	X	X	X	IU	X	X					
CLRDOT	X	X	X	IU	X	X					
EXDOT*				I			IU				
DOTARG	U	X	X	IU	U	X					
DOTADR	U	U	IX		IU						
STOP*										I	
BRKSTP*											
TRACE											U
NOTRAC											U
TRCPRT	X	X	X	I	X	X		IU	IU		

Rutin	**Anropar**	**Anropas från**
CMPDOT	CMPINX (två gånger, en med JP)	STTAB2 (SETDOT, CLRDOT)
SETDOT, CLRDOT	DOTARG	STEXE2 (SETDOT, CLRDOT)
EXDOT	DOTADR, RST $28	EXPTAB2 (kod $23)
DOTARG	RST38, DOTADR; fel 62	SETDOT, CLRDOT
DOTADR		EXDOT, DOTARG
STOP	NODIR; fel 2; går in i BRKSTP	STEXE2 (STOP)
BRKSTP	RST $10	STOP (går in i den), RUNSTM ($0D96)
TRACE, NOTRAC		STEXE2 (TRACE, NOTRACE)

Rutin	Anropar	Anropas från
TRCPRT	SCRPOS, NEWLIN, PRNUM, PRTXT	RUNLIN ($0D7F)

CMPEXB: *kompilera ett uttryck och skriv* $B9 + *typ. In: HL vid uttrycket, DE = koden. Ut: NC, B = typen och DE efter koden, eller CY och A = felkoden.*

```
2362  CD 01 27     CMPEXB: CALL CMPEXP          B = typen
2365  D8           L2365:  RET  C               fel i uttrycket
2366  3E B9                LD   A,$B9           $B9 + typ
```

PUTTOK: *skriv A + B (token + typ). Ut: DE efter byten.*

```
2368  80           PUTTOK: ADD  A,B
2369  12                   LD   (DE),A
236A  13                   INC  DE
236B  C9                   RET
```

CMPSTX / CMPINX / CMPTYX: *kompilera ett uttryck, omvandla det till sträng / heltal / typen B, och skriv* $B9 + *typ.* CMPSTX *anropas också från tilläggs-ROM:en via* $003B*. In: HL vid uttrycket, DE = koden. Ut: NC, B = typen och DE efter koden, eller CY och A = felkoden (fel i uttrycket via* CMPERR*, fel 17 från* TYPCNV*).*

```
236C  06 02        CMPSTX: LD   B,$02           sträng
236E  18 02                JR   CMPTYX

2370  06 01        CMPINX: LD   B,$01           heltal
2372  C5          CMPTYX: PUSH BC              typen som krävs undan
2373  CD 01 27            CALL CMPEXP
2376  DA F7 1B            JP   C,CMPERR        fel: CMPERR tar bort BC, CY till anroparen
2379  F1                  POP  AF              A = typen som krävs
237A  CD 51 25            CALL TYPCNV          omvandla till typen A, fel 17 om det inte går
237D  18 E6                JR   L2365
```

CMPSV: *enkel variabel (utan index) som inte är sträng. Fel 17 för sträng. Koden blir variabelns 5 byte. In: HL vid namnet, DE = koden.*

```
237F  CD CB 26     CMPSV:  CALL VARNAM          C = namnbyten med typen
2382  D8                   RET  C               ingen variabel
2383  79                   LD   A,C             typen
2384  E6 03                AND  $03
2386  FE 02                CP   $02             sträng?
2388  3E 91                LD   A,$91           fel 17
238A  37                   SCF
238B  C8                   RET  Z               sträng: fel 17
238C  C5                   PUSH BC              namnet
238D  0E 00                LD   C,$00           inga index
238F  C3 FC 25             JP   L25FC           variabelns 5 byte, som i CMPVAR
```

CMPSVR / CMPVR: *variabel följd av* $BC + *typ. Vid körning avslutar* $BC-$BE *uttrycket med variabelns adress i HL och typen i B (se* EXPTAB1*).* CMPSVR*: enkel talvariabel,* CMPVR*: vilken variabel som helst, med index. Används där satsen ska skriva till variabeln. In: HL vid variabeln, DE = koden. Ut: NC, HL efter variabeln och DE efter* $BC+*t; vid fel CY och A = felkoden.*

```
2392  CD 7F 23    CMPSVR: CALL CMPSV           enkel talvariabel
2395  18 03               JR   L239A

2397  CD EE 25    CMPVR:  CALL CMPVAR          vilken variabel som helst
239A  D8          L239A:  RET  C               fel i variabeln
239B  3E BC       PUTBC:  LD   A,$BC           $BC + typ: variabelns adress
239D  18 C9               JR   PUTTOK
```

CMPIVR / CMPGET: *variabel med index + $BC + typ.* CMPIVR *kräver heltal (anropas inte i denna ROM),* CMPGET *kräver sträng (*GET, INPUT LINE*). Annars fel 17. In och ut: som* CMPVR.

```
239F  CD EE 25     CMPIVR: CALL CMPVAR        variabeln, B = typen
23A2  D8                   RET  C             fel i variabeln
23A3  78                   LD   A,B
23A4  FE 01                CP   $01           heltal?
23A6  18 07                JR   L23AF

23A8  CD EE 25     CMPGET: CALL CMPVAR        variabeln, B = typen
23AB  D8                   RET  C             fel i variabeln
23AC  78                   LD   A,B
23AD  FE 02                CP   $02           sträng?
23AF  3E 91        L23AF:  LD   A,$91         annars fel 17
23B1  37                   SCF                CY + fel
23B2  C0                   RET  NZ            fel typ
23B3  18 E6                JR   PUTBC         rätt typ: $BC+t
```

Korsreferenstabell för hjälprutinerna för kompileringen. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X) i registren. Alla utom PUTTOK och PUTBC kompilerar med CMPEXP eller CMPVAR och lämnar som de NC, B = typen, HL efter texten och DE efter koden, eller vid fel CY och A = felkoden. CMPSTX, CMPINX och CMPTYX tar bort den sparade typen genom CMPERR vid fel och omvandlar annars med TYPCNV. Felen: TYPCNV, CMPSV, CMPIVR och CMPGET 17. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register					
Rutin	A	CY	BC	DE	HL	IX
CMPEXB	U	U	U	IU	IU	X
PUTTOK	IX	X	I	IU		
CMPSTX	U	U	U	IU	IU	X
CMPINX	U	U	U	IU	IU	X
CMPTYX	U	U	IU	IU	IU	X
CMPSV	U	U	U	IU	IU	X
CMPSVR	U	U	U	IU	IU	X
CMPVR	U	U	U	IU	IU	X
PUTBC	X	X	I	IU		
CMPIVR	U	U	U	IU	IU	X
CMPGET	U	U	U	IU	IU	X

Rutin	**Anropar**	**Anropas från**
CMPEXB	CMPEXP; går in i PUTTOK	CMPPRT
PUTTOK		CMPLET ($1BA2, med JP), CMPEXB (går in i den), PUTBC (med JR)
CMPSTX	CMPTYX (med JR)	J003B, STTAB2 (CHAIN, KILL), CMPOPN
CMPINX	går in i CMPTYX	STTAB2 (kod 3), CMPIF, CMPINP, CMPINL, CMPON, CMPOPN, CMPCLS, CMPOUT, CMPPOK, CMPPRT, CMPDOT
CMPTYX	CMPEXP, CMPERR, TYPCNV, PUTTOK; fel 17	CMPSTX, CMPINX (går in i den), CMPDEF ($1AA7), CMPFOR (två ställen)
CMPSV	VARNAM, CMPVAR (med JP till $25FC); fel 17	CMPFLV, CMPSVR
CMPSVR	CMPSV; går in i PUTBC	CMPNXT
CMPVR	CMPVAR; går in i PUTBC	CMPRD
PUTBC	PUTTOK (med JR)	CMPSVR, CMPVR (går in i den), CMPIVR, CMPGET (med JR)
CMPIVR	CMPVAR, PUTBC; fel 17	anropas inte

Rutin	Anropar	Anropas från
CMPGET	CMPVAR, PUTBC; fel 17	STTAB2 (GET), CMPINL (med JP)

3.60 Uttryckskompilatorn: en post per prioritetsnivå $23B5

Varje post är [rutin][nästa nivå][operatortabell][klass]. Nivåerna, från lägst till högst prioritet:

```
PREC1   EQV             klass 3 (logisk)
PREC2   IMP             klass 3
PREC3   OR  XOR         klass 3
PREC4   AND             klass 3
PREC5   NOT             egen rutin (CMPNOT), unär
PREC6   = <> < <= > >=  klass 2 (jämförelse)
PREC7   +  -            klass 1, egen rutin (CMPADD) för unärt -/+
PREC8   *  /            klass 1 (aritmetik)
PREC9   ^  **           klass 4 (upphöjt till)
PREC10  primärt uttryck (PRIMARY)
```

PREC5 har ingen klassbyte; den första byten i PREC6 hamnar där, men CMPNOT läser den inte.

Alla binära operatorer är vänsterassociativa: A-B-C = (A-B)-C.

```
23B5  05 27 BC 23… PREC1:  DEFW PRECLV,PREC2,KWEQV
                                                  nivå 1: EQV
23BB  03                   DEFB $03               klass 3: logisk
23BC  05 27 C3 23… PREC2:  DEFW PRECLV,PREC3,KWIMP
                                                  nivå 2: IMP
23C2  03                   DEFB $03               klass 3
23C3  05 27 CA 23… PREC3:  DEFW PRECLV,PREC4,KWOR
                                                  nivå 3: OR och XOR
23C9  03                   DEFB $03               klass 3
23CA  05 27 D1 23… PREC4:  DEFW PRECLV,PREC5,KWAND
                                                  nivå 4: AND
23D0  03                   DEFB $03               klass 3
23D1  26 28 D7 23… PREC5:  DEFW CMPNOT,PREC6,KWNOT
                                                  nivå 5: NOT, egen rutin och ingen klass
23D7  05 27 DE 23… PREC6:  DEFW PRECLV,PREC7,KWCMP
                                                  nivå 6: jämförelserna
23DD  02                   DEFB $02               klass 2: jämförelse
23DE  FB 27 E5 23… PREC7:  DEFW CMPADD,PREC8,KWADD
                                                  nivå 7: + och -, egen rutin för tecknet först
23E4  01                   DEFB $01               klass 1: aritmetik
23E5  05 27 EC 23… PREC8:  DEFW PRECLV,PREC9,KWMUL
                                                  nivå 8: * och /
23EB  01                   DEFB $01               klass 1
23EC  05 27 F3 23… PREC9:  DEFW PRECLV,PREC10,KWPOW
                                                  nivå 9: potens
23F2  04                   DEFB $04               klass 4: potens
23F3  7F 25        PREC10: DEFW PRIMARY           nivå 10: bara rutinen
```

*Operatortabellerna har samma form som nyckelordstabellerna: [token][text] … $FF. Jämförelser och aritmetik har en token per typ: bas + 0 flyttal, + 1 heltal, + 2 sträng. I tabellen står bara bastoken. <= och >= står före < och >, annars skulle KWSRCH hitta < först. ** är en annan stavning av ^.*

```
23F5  DC 45 51 56  KWEQV:  DEFB $DC,"EQV"        OPEQV vid körning
23F9  FF                   DEFB $FF
23FA  DD 49 4D 50  KWIMP:  DEFB $DD,"IMP"        OPIMP
23FE  FF                   DEFB $FF
23FF  DE 4F 52     KWOR:   DEFB $DE,"OR"         OPOR
```

```
2402  DF 58 4F 52          DEFB $DF,"XOR"        OPXOR
2406  FF                   DEFB $FF
2407  E0 41 4E 44  KWAND:  DEFB $E0,"AND"        OPAND
240B  FF                   DEFB $FF
240C  E1 4E 4F 54  KWNOT:  DEFB $E1,"NOT"        OPNOT
2410  FF                   DEFB $FF
2411  F1 3C 3D     KWCMP:  DEFB $F1,"<="         $F1-$F3: LEF LEI LES
2414  E5 3C 3E             DEFB $E5,"<>"         $E5-$E7: NEF NEI NES
2417  E8 3C                DEFB $E8,"<"          $E8-$EA: LTF LTI LTS
2419  EB 3E 3D             DEFB $EB,">="         $EB-$ED: GEF GEI GES
241C  EE 3E                DEFB $EE,">"          $EE-$F0: GTF GTI GTS
241E  E2 3D                DEFB $E2,"="          $E2-$E4: EQF EQI EQS
2420  FF                   DEFB $FF              =< finns inte: fel 11
2421  F4 2B        KWADD:  DEFB $F4,"+"          $F4 $F5: ADDF ADDI; två strängar $F6
2423  F7 2D                DEFB $F7,"-"          $F7 $F8: SUBF SUBI, ingen sträng
2425  FF                   DEFB $FF
2426  F9 2A        KWMUL:  DEFB $F9,"*"          $F9 $FA: MULF MULI
2428  FB 2F                DEFB $FB,"/"          $FB $FC: DIVF DIVI
242A  FF                   DEFB $FF
242B  FD 5E        KWPOW:  DEFB $FD,"^"          $FD $FE: POWF POWI
242D  FD 2A 2A             DEFB $FD,"**"         samma token; flyttal^heltal $FF
2430  FF                   DEFB $FF
```

Funktionernas namn. Token `$80`*+kod; koden (*0*-*`$34`*) blir 2*kod i den kompilerade koden och används som index i* `EXPTAB2`*.* `LOG10` *står före* `LOG` *av samma skäl som* `<=` *före* `<`*.*

```
2431  80 46 4E     FNKW:   DEFB $80,"FN"         kod 0: kompileras av CMPFN
2434  81 41 42 53         DEFB $81,"ABS"
2438  82 41 54 4E         DEFB $82,"ATN"
243C  83 43 4F 53         DEFB $83,"COS"
2440  84 45 58 50         DEFB $84,"EXP"
2444  85 46 49 58         DEFB $85,"FIX"
2448  86 49 4E 54         DEFB $86,"INT"
244C  88 4C 4F 47…        DEFB $88,"LOG10"      kod 8, före LOG
2452  87 4C 4F 47         DEFB $87,"LOG"        kod 7
2456  89 50 49           DEFB $89,"PI"
2459  8A 52 4E 44         DEFB $8A,"RND"
245D  8B 53 47 4E         DEFB $8B,"SGN"
2461  8C 53 49 4E         DEFB $8C,"SIN"
2465  8D 53 51 52         DEFB $8D,"SQR"
2469  8E 54 41 4E         DEFB $8E,"TAN"
246D  8F 41 53 43         DEFB $8F,"ASC"
2471  90 43 48 52…        DEFB $90,"CHR$"
2476  91 4C 45 46…        DEFB $91,"LEFT$"
247C  92 52 49 47…        DEFB $92,"RIGHT$"
2483  93 4D 49 44…        DEFB $93,"MID$"
2488  94 4C 45 4E         DEFB $94,"LEN"
248C  95 49 4E 53…        DEFB $95,"INSTR"
2492  96 53 50 41…        DEFB $96,"SPACE$"
2499  97 53 54 52…        DEFB $97,"STRING$"
24A1  98 4E 55 4D…        DEFB $98,"NUM$"
24A6  99 56 41 4C         DEFB $99,"VAL"
24AA  9A 53 57 41…        DEFB $9A,"SWAP%"
24B0  9B 50 45 45…        DEFB $9B,"PEEK"
24B5  9C 49 4E 50         DEFB $9C,"INP"
24B9  9D 43 41 4C…        DEFB $9D,"CALL"
24BE  9E 45 52 52…        DEFB $9E,"ERRCODE"
24C6  9F 49 45 43…        DEFB $9F,"IEC$"
24CB  A0 54 41 42         DEFB $A0,"TAB"
24CF  A1 43 55 52         DEFB $A1,"CUR"        kod $21
24D3  A3 44 4F 54         DEFB $A3,"DOT"        kod $23; $22 saknas
```

```
24D7  B0 41 44 44…        DEFB $B0,"ADD$"       kod $30; $24-$2F saknas
24DC  B1 53 55 42…        DEFB $B1,"SUB$"
24E1  B2 4D 55 4C…        DEFB $B2,"MUL$"
24E6  B3 44 49 56…        DEFB $B3,"DIV$"
24EB  B4 43 4F 4D…        DEFB $B4,"COMP%"      kod $34, den sista
24F1  FF                  DEFB $FF
```

3.61 FNTYPE: typer för funktionernas resultat och argument $24F2

Varje post:

```
[en eller flera token][resultat][argument ...]
```

Posten slutar vid nästa byte med bit 7 satt (nästa posts token, eller $FF). Resultat- och argumentbyten:

```
bit 0-1  typ: 0 = flyttal, 1 = heltal, 2 = sträng
bit 6    listan får sluta här (i resultatbyten: funktionen får
         skrivas utan argument)
```

Argumenten omvandlas till den typ som står (se TYPCNV); omvandling mellan heltal och flyttal görs automatiskt, mellan sträng och tal ger den fel.

Ur tabellen:

```
ABS ATN COS EXP FIX INT LOG LOG10 SIN SQR TAN (x)   -> flyttal
PI                                                  -> flyttal
RND  eller  RND(x)                                  -> flyttal
SGN(x)                                              -> heltal
ASC(s$) LEN(s$)                                     -> heltal
CHR$(i[,i[,i[,i]]])  1-4 tecken                     -> sträng
LEFT$(s$,i) RIGHT$(s$,i)                            -> sträng
MID$(s$,i,i)                                        -> sträng
INSTR(i,s$,s$)                                      -> heltal
TAB(i) SPACE$(i) IEC$(i)                            -> sträng
CUR(i,i) STRING$(i,i)                               -> sträng
NUM$(x)                                             -> sträng
VAL(s$)                                             -> flyttal
PEEK(i) SWAP%(i) INP(i)                             -> heltal
ERRCODE                                             -> heltal
DOT(i,i)                                            -> heltal
ADD$ SUB$ MUL$ DIV$ (s$,s$,i)                       -> sträng
COMP%(s$,s$)                                        -> heltal
```

Efter $FF följer två listor som används direkt med adress:

```
$2548  $41 $41 $41 $FF   DIM av sträng: 0-2 index
$254D  $01 $41 $41 $FF   DIM av tal: 1-2 index. Samma byte  är
                         också posten för CALL(i[,i]) -> heltal
                         (token $9D står före).
```

```
24F2  81 82 83 84… FNTYPE: DEFB $81,$82,$83,$84,$85,$86,$87,$88
                                          ABS ATN COS EXP FIX INT LOG LOG10,
24FA  8C 8D 8E 00…         DEFB $8C,$8D,$8E,$00,$40,$89,$40,$8A
                                          SIN SQR TAN: flyttal(x); PI; RND
2502  40 40 8B 01…        DEFB $40,$40,$8B,$01,$40,$8F,$94,$01
                                          RND, x får saknas; SGN: heltal(x); ASC LEN:
250A  42 90 02             DEFB $42,$90,$02       heltal(s$); CHR$: sträng
250D  41 41 41 41          DEFM "AAAA"            1-4 heltal ($41 är A)
2511  91 92 02 02…        DEFB $91,$92,$02,$02,$41,$93,$02,$02
                                          LEFT$ RIGHT$: sträng(s$,i); MID$:
2519  01 41 95 01…        DEFB $01,$41,$95,$01,$01,$02,$42,$A0
                                          sträng(s$,i,i); INSTR: heltal(i,s$,s$); TAB
```

```
2521  96 9F 02 41…        DEFB $96,$9F,$02,$41,$A1,$97,$02,$01
                                 SPACE$ IEC$: sträng(i); CUR STRING$:
2529  41 98 02 40…        DEFB $41,$98,$02,$40,$99,$00,$42,$9B
                                 sträng(i,i); NUM$(x); VAL: flyttal(s$); PEEK
2531  9A 9C 01 41…        DEFB $9A,$9C,$01,$41,$9E,$41,$A3,$01
                                 SWAP% INP: heltal(i); ERRCODE; DOT:
2539  01 41 B0 B1…        DEFB $01,$41,$B0,$B1,$B2,$B3,$02,$02
                                 heltal(i,i); ADD$ SUB$ MUL$ DIV$:
2541  02 41 B4 01…        DEFB $02,$41,$B4,$01,$02,$42,$FF,$41
                                 sträng(s$,s$,i); COMP%: heltal(s$,s$); slut; $2548:
2549  41 41 FF 9D…        DEFB $41,$41,$FF,$9D,$01,$41,$41,$FF
                                 DIM av sträng; $254C: CALL, och $254D: DIM av tal
```

3.62 TYPCNV: omvandla värdet till typen A (bit 0–1) $2551

B = typen på det kompilerade värdet. Heltal till flyttal skriver $DA, flyttal till heltal $D9. Sträng och tal blandade ger fel 17 (OTILLÅTEN BLANDNING AV TAL OCH STRÄNGAR); ett tal där en sträng ska vara ger fel 13.

TOINT/TOFLT: samma, men till heltal/flyttal.

In: A = typen som ska bli (bara för TYPCNV), B = värdets typ, DE efter värdets kod. Ut: NC, B = den nya typen och DE efter omvandlingsbyten, om en skrevs; vid fel CY och A = felkoden. Förstör A.

```
2551  E6 03     TYPCNV: AND  $03          bit 0-1: typen som ska bli
2553  B8                CP   B           redan rätt typ
2554  C8                RET  Z
2555  FE 01             CP   $01         heltal?
2557  28 08             JR   Z,TOINT
2559  FE 00             CP   $00         flyttal?
255B  28 11             JR   Z,TOFLT
255D  3E 8D             LD   A,$8D       sträng, men värdet är ett tal: fel 13
255F  37                SCF
2560  C9                RET

2561  78        TOINT:  LD   A,B         A = värdets typ
2562  06 01             LD   B,$01       B = heltal efteråt
2564  B8                CP   B           redan heltal
2565  C8                RET  Z
2566  FE 00             CP   $00         flyttal? annars sträng
2568  20 0B             JR   NZ,L2575
256A  3E D9             LD   A,$D9       $D9: flyttal -> heltal
256C  18 0D             JR   L257B

256E  78        TOFLT:  LD   A,B         A = värdets typ
256F  06 00             LD   B,$00       B = flyttal efteråt
2571  B8                CP   B           redan flyttal
2572  C8                RET  Z
2573  FE 01             CP   $01         heltal? annars sträng
2575  3E 91   L2575:   LD   A,$91       fel 17
2577  37                SCF
2578  C0                RET  NZ          sträng: fel 17
2579  3E DA             LD   A,$DA       $DA: heltal -> flyttal
257B  12        L257B:  LD   (DE),A      omvandlingen efter värdets kod
257C  13                INC  DE
257D  A7                AND  A           NC
257E  C9                RET
```

3.63 PRIMARY: kompilera ett primärt uttryck $257F

```
siffra eller "."    tal (CMPNUM)
"(" uttryck ")"     uttrycket följt av $D3 (för LIST)
" eller '           strängkonstant (CMPSTR)
funktionsnamn       CMPFNC (FN.. går till CMPFN)
annars              variabel (CMPVAR) och $D4+typ (hämta värdet)
```

In: HL = texten, DE = koden. Ut: NC, B = typen, DE efter koden och HL efter texten (blanktecken hoppas över); vid fel CY och A = felkoden. Förstör A, C och IX.

```
                PRIMARY:
257F  7E              LD   A,(HL)          första tecknet
2580  FE 2E           CP   $2E             .
2582  CA 89 26        JP   Z,CMPNUM
2585  FE 30           CP   $30             siffra?
2587  38 05           JR   C,L258E
2589  FE 3A           CP   $3A
258B  DA 89 26        JP   C,CMPNUM
258E  FE 28     L258E:  CP   $28            (
2590  28 1C           JR   Z,L25AE
2592  FE 22           CP   $22             citattecken "
2594  28 30           JR   Z,CMPSTR
2596  FE 27           CP   $27             apostrof '
2598  28 2C           JR   Z,CMPSTR
259A  D5              PUSH DE              KWSRCH tar tabellen i DE
259B  11 31 24        LD   DE,FNKW         funktionsnamn?
259E  CD F5 09        CALL KWSRCH
25A1  D1              POP  DE
25A2  28 78           JR   Z,CMPFNC        A = $80 + kod, HL efter namnet
25A4  CD EE 25        CALL CMPVAR          annars en variabel
25A7  D8              RET  C
25A8  3E D4           LD   A,$D4           $D4+typ: hämta värdet
25AA  80              ADD  A,B
25AB  12              LD   (DE),A
25AC  13              INC  DE
25AD  C9              RET

25AE  DF        L25AE:  RST  $18           över (
25AF  CD 01 27        CALL CMPEXP          från PREC1 igen
25B2  D8              RET  C
25B3  7E              LD   A,(HL)          CMPEXP har hoppat över mellanslag
25B4  FE 29           CP   $29             )
25B6  3E 92           LD   A,$92           fel 18
25B8  37              SCF
25B9  C0              RET  NZ
25BA  3E D3           LD   A,$D3           $D3: parentesen, för LIST
25BC  12              LD   (DE),A
25BD  13              INC  DE              faller in i SKIPSP: över ) och mellanslag
```

SKIPSP (RST $18): *HL+1, hoppa över mellanslag.* SKIPS0 (RST $20) *utan HL+1. Ut: NC, HL på det första tecknet som inte är ett mellanslag och A = tecknet; Z om det är 0. Förstör A.*

```
25BE  23        SKIPSP: INC  HL
25BF  7E        SKIPS0: LD   A,(HL)
25C0  FE 20           CP   $20
25C2  28 FA           JR   Z,SKIPSP        mellanslag: ett till
25C4  A7              AND  A               NC; Z om tecknet är 0
25C5  C9              RET
```

CMPSTR: $CB, *citattecknet, längden, texten. Två citattecken efter varandra blir ett tecken i strängen. Strängen får sluta med radslutet utan ett avslutande citattecken.*

In: A = citattecknet, HL på det, DE = koden. Ut: NC, B = 2 (sträng), HL efter strängen och mellanslagen och DE efter texten. Förstör A och C.

```
25C6  23           CMPSTR: INC  HL              över citattecknet
25C7  EB                   EX   DE,HL
25C8  36 CB                LD   (HL),$CB        $CB
25CA  23                   INC  HL
25CB  EB                   EX   DE,HL
25CC  4F                   LD   C,A             C = citattecknet, B = längden
25CD  06 00                LD   B,$00
25CF  12                   LD   (DE),A          citattecknet sparas för LIST
25D0  13                   INC  DE
25D1  D5                   PUSH DE              plats för längden
25D2  13                   INC  DE
25D3  7E           L25D3:  LD   A,(HL)
25D4  FE 0D                CP   $0D             radslut?
25D6  28 0F                JR   Z,L25E7
25D8  23                   INC  HL              tecknet läst
25D9  B9                   CP   C               samma citattecken?
25DA  28 05                JR   Z,L25E1
25DC  12           L25DC:  LD   (DE),A          tecken till strängen
25DD  13                   INC  DE
25DE  04                   INC  B
25DF  18 F2                JR   L25D3

25E1  BE           L25E1:  CP   (HL)            dubbelt citattecken?
25E2  20 03                JR   NZ,L25E7        nej: strängen är slut
25E4  23                   INC  HL              två citattecken: ett i strängen
25E5  18 F5                JR   L25DC

25E7  78           L25E7:  LD   A,B             A = längden
25E8  C1                   POP  BC              BC = platsen för längden
25E9  02                   LD   (BC),A
25EA  06 02                LD   B,$02           typ sträng
25EC  18 D1                JR   SKIPS0          mellanslag efter strängen, NC
```

3.64 CMPVAR: kompilera en variabel, med index $25EE

Koden blir fem byte:

```
[$C0 + 4*antal index + typ] [namnbyte] [bokstav] [adress lo, hi]
```

Namnbyten: bit 4–7 = siffran ($F om ingen), bit 2–3 = antal index, bit 0–1 = typen. Adressen fylls i vid förlänkningen (LNKEXP). Indexen (högst två, heltal) kompileras först, före variabeln, med typlistan D2618.

In: HL = namnet, DE = koden. Ut: NC, B = variabelns typ, HL efter namnet och indexen och DE efter de fem byten; vid fel CY och A = felkoden. Förstör A, C och IX.

```
25EE  CD CB 26     CMPVAR: CALL VARNAM          B = bokstaven, C = namnbyten
25F1  D8                   RET  C
25F2  C5                   PUSH BC              namnet sparas medan indexen kompileras
25F3  DD 21 18 26          LD   IX,D2618        högst 2 index
25F7  CD 4E 26             CALL CMPARG          C = antal index
25FA  38 45                JR   C,L2641         fel: CMPERR tar bort namnet
25FC  79           L25FC:  LD   A,C             $C0 + 4*antal index
25FD  87                   ADD  A,A
25FE  87                   ADD  A,A
25FF  C6 C0                ADD  A,$C0
```

```
2601  12                LD   (DE),A          tokens första del
2602  C1                POP  BC             B = bokstaven, C = namnbyten
2603  3E 03             LD   A,$03          typen ur namnbyten
2605  A1                AND  C
2606  EB                EX   DE,HL          HL = koden
2607  B6                OR   (HL)           + typ
2608  77                LD   (HL),A         token $C0 + 4n + typ
2609  23                INC  HL
260A  E6 0C             AND  $0C            4n: antal index i bit 2-3
260C  B1                OR   C              ihop med siffran och typen
260D  77                LD   (HL),A
260E  23                INC  HL
260F  70                LD   (HL),B         bokstaven
2610  23                INC  HL
2611  23                INC  HL             över adressen, som förlänkningen fyller i
2612  23                INC  HL
2613  E6 03             AND  $03            typen
2615  47                LD   B,A            B = typen
2616  EB                EX   DE,HL          DE efter variabeln, HL i texten igen
2617  C9                RET

2618  41 41 41 FF  D2618: DEFB $41,$41,$41,$FF  som $2548: 0-2 heltal
```

CMPFNC: *A = funktionens token. Posten letas upp i* FNTYPE*, och argumenten kompileras med* CMPARG*. Koden blir*

```
argument ... [$CD + antal argument] [2*kod]
```

Token $80 *är* FN *och går till* CMPFN*. In: A = token, HL efter namnet, DE = koden. Ut: NC, B = resultatets typ, HL efter argumenten och DE efter koden; vid fel CY och A = felkoden. Förstör A, C och IX.*

```
261C  FE 80       CMPFNC: CP   $80             A = funktionens token
261E  CA B8 1A            JP   Z,CMPFN
2621  E5                  PUSH HL              textpekaren
2622  21 F2 24            LD   HL,FNTYPE
2625  01 00 00            LD   BC,$0000        BC = 0: sök hela minnet
2628  ED B1               CPIR                 hitta token i FNTYPE
262A  2B                  DEC  HL              HL på token som hittades
262B  23          L262B:  INC  HL              nästa byte
262C  4E                  LD   C,(HL)
262D  0C                  INC  C               hoppa över token
262E  FA 2B 26            JP   M,L262B         fler token i samma post
2631  E5                  PUSH HL              IX = resultatbyten
2632  DD E1               POP  IX
2634  E1                  POP  HL              HL = texten igen
2635  87                  ADD  A,A             2*kod
2636  4F                  LD   C,A
2637  DD 7E 00            LD   A,(IX+$00)      resultatbyten
263A  E6 03               AND  $03
263C  47                  LD   B,A             B = resultatets typ
263D  C5                  PUSH BC              B och 2*kod sparas
263E  CD 4E 26            CALL CMPARG
2641  DA F7 1B    L2641:  JP   C,CMPERR        fel: CMPERR tar bort BC
2644  79                  LD   A,C             C = antal argument
2645  C6 CD               ADD  A,$CD           $CD + antal
2647  12                  LD   (DE),A
2648  13                  INC  DE
2649  C1                  POP  BC              B = resultatets typ, C = 2*kod
264A  79                  LD   A,C
264B  12                  LD   (DE),A          2*kod
264C  13                  INC  DE
264D  C9                  RET
```

CMPARG: *kompilera argumentlistan* (a,b,...). *IX pekar på resultat-byten i en typlista* (FNTYPE)*; för varje argument flyttas IX ett steg och argumentet omvandlas till den typen. En byte med bit 7 betyder att det inte får komma fler argument* (*fel 13*)*, och listan får bara sluta där bit 6 är satt* (*annars fel 13*)*. Utan* (*är listan tom.*

In: IX = typlistans resultatbyte, HL = texten, DE = koden. Ut: NC, C = antal argument, HL efter) *och mellanslagen, DE efter argumentens kod och IX på den sista typbyten som användes; vid fel CY och A = felkoden. Förstör A.*

```
264E  0E 00       CMPARG: LD   C,$00          C = antal argument
2650  3E 28               LD   A,$28          (?
2652  BE                  CP   (HL)
2653  20 2A               JR   NZ,L267F       (?
2655  DF          L2655:  RST  $18            över ( eller ,
2656  DD 23               INC  IX             nästa typbyte
2658  0C                  INC  C
2659  C5                  PUSH BC             antal och typbyten sparas
265A  DD E5               PUSH IX
265C  CD 01 27            CALL CMPEXP
265F  DD E1               POP  IX
2661  38 DE               JR   C,L2641        fel: CMPERR tar bort BC
2663  DD 7E 00            LD   A,(IX+$00)     argumentets typ
2666  4F                  LD   C,A            C = typbyten
2667  A7                  AND  A              bit 7?
2668  3E 8D               LD   A,$8D          bit 7: för många argument, fel 13
266A  FA F7 1B            JP   M,CMPERR
266D  79                  LD   A,C            A = typen
266E  CD 51 25            CALL TYPCNV         omvandla argumentet
2671  C1                  POP  BC             C = antal argument
2672  D8                  RET  C              fel från TYPCNV
2673  7E                  LD   A,(HL)
2674  FE 2C               CP   $2C            ,
2676  28 DD               JR   Z,L2655
2678  FE 29               CP   $29            )?
267A  3E 92               LD   A,$92          annars fel 18
267C  37                  SCF
267D  C0                  RET  NZ
267E  DF                  RST  $18            över )
267F  DD 7E 00    L267F:  LD   A,(IX+$00)     typbyten, där listan slutar
2682  E6 40               AND  $40            får listan sluta här?
2684  C0                  RET  NZ
2685  3E 8D               LD   A,$8D          listan får inte sluta: fel 13
2687  37                  SCF
2688  C9                  RET
```

CMPNUM: *siffror följda av* % *är ett heltal:* $C7 *lo hi* (*fel 12 om talet är större än 65535; 65535% = -1*)*. Blanka mellan siffrorna hoppas över:* 1 2% *= 12,* 1 2 3 *= 123, och* LIST *skriver dem ihop.* 1.5% *ger fel 11* (*punkten gör det till ett flyttal, och* % *blir kvar*)*. Annars ett flyttal:* $C3 *och 5 byte från* GETFLT (*fel 12 om talet är fel*)*.*

In: HL = talets första tecken, DE = koden. Ut: NC, B = typen (*0 eller 1*)*, HL efter talet och mellanslagen och DE efter konstanten; vid fel CY och A = felkoden. Förstör A, C och IX.*

```
2689  E5          CMPNUM: PUSH HL             talets början
268A  2B                  DEC  HL             SKIPSP börjar med HL+1
268B  DF          L268B:  RST  $18            blanka hoppas över
268C  FE 25               CP   $25            % efter siffrorna?
268E  28 1E               JR   Z,L26AE
2690  FE 30               CP   $30
2692  38 04               JR   C,L2698
2694  FE 3A               CP   $3A
2696  38 F3               JR   C,L268B
2698  E1          L2698:  POP  HL             HL = talets början
```

```
2699  EB              EX   DE,HL          HL = koden, DE = texten
269A  36 C3           LD   (HL),$C3       $C3 flyttal
269C  01 05 00        LD   BC,$0005
269F  09              ADD  HL,BC          HL = exponentens plats
26A0  E5              PUSH HL             sparas
26A1  CD 13 15        CALL GETFLT         Ut: DE ett tecken efter talet
26A4  EB              EX   DE,HL          HL = texten
26A5  2B              DEC  HL             HL efter talet
26A6  D1              POP  DE             DE = exponentens plats
26A7  13              INC  DE             DE efter konstanten
26A8  38 0C           JR   C,L26B6        CY: inget tal, fel 12
26AA  E7              RST  $20            mellanslag efter talet, NC
26AB  06 00           LD   B,$00          typ flyttal
26AD  C9              RET

26AE  E1        L26AE: POP  HL            HL = talets början
26AF  D5               PUSH DE            koden sparas
26B0  CD 1E 18         CALL GETLN         DE = talet, CY om det är över 65535
26B3  42               LD   B,D           BC = talet
26B4  4B               LD   C,E
26B5  D1               POP  DE            DE = koden
26B6  3E 8C     L26B6: LD   A,$8C         fel 12
26B8  D8               RET  C
26B9  EB               EX   DE,HL         HL = koden, DE = texten
26BA  36 C7            LD   (HL),$C7      $C7 heltal
26BC  CD 28 10         CALL WRBC1         två byte, låg först; HL efter
26BF  EB               EX   DE,HL         HL = texten, DE efter konstanten
26C0  7E               LD   A,(HL)
26C1  FE 25            CP   $25           alltid: förläsningen hittade det
26C3  37               SCF
26C4  20 F0            JR   NZ,L26B6
26C6  06 01            LD   B,$01         typ heltal
26C8  C3 BE 25         JP   SKIPSP        över % och mellanslag
```

3.65 VARNAM: läs ett variabelnamn $26CB

Namnet är en bokstav (A–Z eller ÄÖÅ, små görs till stora), sedan en siffra om det finns, och sedan ”%” (heltal) eller ”$” (sträng).

In: HL = namnet. Ut: NC, B = bokstaven, C = namnbyte (siffra i bit 4–7, `$F0` utan siffra, typ i bit 0–1) och HL efter namnet och mellanslagen; CY och A = fel 11 om det inte börjar med en bokstav. Förstör A. Eftersom mellanslag hoppas över är `A 1` samma som `A1`.

```
26CB  7E        VARNAM: LD   A,(HL)
26CC  E6 DF             AND  $DF          små bokstäver -> stora
26CE  47                LD   B,A          B = bokstaven, stor
26CF  FE 41             CP   $41          under A?
26D1  0E 8B             LD   C,$8B        C = fel 11
26D3  79                LD   A,C
26D4  D8                RET  C
26D5  78                LD   A,B
26D6  FE 5E             CP   $5E          över Å?
26D8  3F                CCF               CY om $5E eller mer
26D9  79                LD   A,C
26DA  D8                RET  C            fel 11
26DB  DF                RST  $18          över bokstaven
26DC  0E F0             LD   C,$F0        ingen siffra
26DE  7E                LD   A,(HL)
26DF  D6 30             SUB  $30          siffra?
26E1  38 0A             JR   C,L26ED      nej
```

```
26E3  FE 0A              CP   $0A          över 9?
26E5  30 06              JR   NC,L26ED     nej
26E7  87                 ADD  A,A          siffran i bit 4-7
26E8  87                 ADD  A,A
26E9  87                 ADD  A,A
26EA  87                 ADD  A,A
26EB  4F                 LD   C,A          C = siffran i bit 4-7
26EC  DF                 RST  $18          över siffran
26ED  7E           L26ED: LD   A,(HL)
26EE  FE 24              CP   $24          $?
26F0  28 0B              JR   Z,L26FD
26F2  D6 25              SUB  $25          %?
26F4  A7                 AND  A            NC
26F5  C0                 RET  NZ           nej: flyttal, C har typ 0
26F6  3E 01              LD   A,$01        typ heltal
26F8  B1           L26F8: OR   C
26F9  4F                 LD   C,A          C = namnbyten
26FA  C3 BE 25           JP   SKIPSP       över tecknet och mellanslagen

26FD  3E 02        L26FD: LD   A,$02       typ sträng
26FF  18 F7              JR   L26F8
```

3.66 CMPEXP: kompilera ett uttryck till omvänd polsk notation

`$2701`

Kompilatorn går rekursivt genom prioritetsnivåerna PREC1–PREC10. PRECLV är den gemensamma rutinen för en nivå med binära operatorer:

1. vänster operand = nästa nivå (NXTLEV)
2. finns en operator ur nivåns tabell (KWSRCH)? nej: klart
3. höger operand = nästa nivå
4. typerna kontrolleras efter klassen, och operatorns token skrivs efter båda operanderna; tillbaka till 2.

Klasser:

```
1  aritmetik: typerna görs lika (OPTYPE), token = bas + typ.
   Strängar bara för "+" (sammanfogning, $F6).
2  jämförelse: typerna görs lika, token = bas + typ, resultat heltal
3  logisk: båda operanderna görs till heltal, resultat heltal
4  "^": flyttal^heltal får egen token ($FF), annars som klass 1
```

Ett tal utan ”%” är ett flyttal, så `7%/2` ger 3.5 men `7%/2%` ger 3, och `2^3%` ger exakt 8 men `2^3` ger 8.00005 (via flyttal); `(-2)^3` ger fel 4. Heltalen slår runt utan fel: `32767%+1%` ger -32768.

Om en operand måste omvandlas och det är den vänstra, skjuts omvandlingstoken in direkt efter den (INSCNV); den högra står sist och får token efter sig.

Stacken efter steg 3, med IX = SP: +0/1 = nivåns post, +2 = flaggorna, +3 = token, +4 = klass, +5 = vänster typ, +6/7 = var den högra operandens kod börjar. Klassen räknas ned med `DEC (IX+4)` tills fallet är funnet.

In: HL = texten, DE = var koden ska skrivas. Ut: NC, B = uttryckets typ, HL efter uttrycket och mellanslagen och DE efter koden; vid fel CY och A = felkoden. Förstör A, C och IX.

```
2701  DD 21 B5 23  CMPEXP: LD   IX,PREC1     nivå 1
2705  DD E5        PRECLV: PUSH IX           IX = nivåns post
2707  CD E8 27             CALL NXTLEV       vänster operand
270A  DD E1                POP  IX           IX = nivån igen
270C  D8                   RET  C            fel
270D  D5           L270D:  PUSH DE           DE = koden, sparas
270E  DD 5E 04             LD   E,(IX+$04)   nivåns operatortabell
```

```
2711  DD 56 05           LD   D,(IX+$05)
2714  CD F5 09           CALL KWSRCH          operator?
2717  D1                 POP  DE              DE = koden
2718  C0                 RET  NZ              ingen operator: klart, B = typen, NC
2719  DD 4E 06           LD   C,(IX+$06)      A = operatorns bastoken
271C  D5                 PUSH DE              IX+6/7: var den högra operanden börjar
271D  C5                 PUSH BC              IX+4 = klassen, IX+5 = vänster typ
271E  F5                 PUSH AF              IX+3 = token
271F  DD E5              PUSH IX              IX+0/1 = nivåns post
2721  CD E8 27           CALL NXTLEV          höger operand
2724  38 5B              JR   C,L2781         fel
2726  DD 21 00 00        LD   IX,$0000        IX = stacken
272A  DD 39              ADD  IX,SP           IX+3 = token, IX+4 = klass ...
272C  DD 7E 03           LD   A,(IX+$03)
272F  FE F4              CP   $F4             + (tillåter strängar)
2731  20 0F              JR   NZ,L2742
2733  CD A0 27           CALL OPTYPE          strängar, tal eller blandat
2736  38 49        L2736: JR   C,L2781         fel
2738  79           L2738: LD   A,C            C = operatorns token
2739  12                  LD   (DE),A
273A  13                  INC  DE
273B  DD E1               POP  IX              IX = nivån igen
273D  F1                  POP  AF              ta bort token
273E  F1                  POP  AF              klassen och vänster typ
273F  F1                  POP  AF              var koden började
2740  18 CB               JR   L270D           nästa operator på samma nivå

2742  DD 35 04     L2742: DEC  (IX+$04)        klass 1
2745  20 0B              JR   NZ,L2752
2747  78           L2747: LD   A,B             B = höger typ
2748  FE 02              CP   $02             sträng?
274A  3E 91              LD   A,$91           annars fel 17
274C  37                 SCF
274D  C4 A0 27           CALL NZ,OPTYPE       en sträng till höger går inte
2750  18 E4              JR   L2736           CY: fel

2752  DD 35 04     L2752: DEC  (IX+$04)        klass 2
2755  20 07              JR   NZ,L275E
2757  CD A0 27           CALL OPTYPE          strängar går också
275A  06 01              LD   B,$01           jämförelsen ger ett heltal
275C  18 D8              JR   L2736

275E  DD 35 04     L275E: DEC  (IX+$04)        klass 3
2761  20 24              JR   NZ,L2787
2763  CD 61 25           CALL TOINT           höger till heltal
2766  38 19              JR   C,L2781         fel 17 om en sträng
2768  DD 7E 05           LD   A,(IX+$05)      vänster typ
276B  FE 01              CP   $01
276D  28 0D              JR   Z,L277C
276F  FE 00              CP   $00
2771  3E 91              LD   A,$91           annars sträng: fel 17
2773  20 0C              JR   NZ,L2781        sträng
2775  3E D9              LD   A,$D9           vänster till heltal
2777  CD CF 27           CALL INSCNV          skjut in $D9 efter vänster operand
277A  06 01              LD   B,$01           resultat heltal
277C  DD 4E 03     L277C: LD   C,(IX+$03)      C = token
277F  18 B7              JR   L2738           skriv den

2781  C1           L2781: POP  BC              ta bort de fyra orden
2782  C1                  POP  BC
2783  C1                  POP  BC
2784  C1                  POP  BC
2785  37                  SCF                  CY, A = felkoden
```

```
2786  C9                RET
2787  DD 35 04    L2787: DEC  (IX+$04)        klass 4
```

Klass 5 och högre finns inte; här skulle datorn stå still.

```
278A  20 FE       L278A: JR   NZ,L278A        klass över 4 finns inte: evig slinga
278C  78                 LD   A,B             höger heltal och vänster flyttal?
278D  FE 01              CP   $01
278F  20 B6              JR   NZ,L2747        nej: som klass 1
2791  DD 7E 05          LD   A,(IX+$05)      vänstra typen
2794  FE 00              CP   $00
2796  20 AF              JR   NZ,L2747        nej: som klass 1
2798  DD 4E 03           LD   C,(IX+$03)      $FF: flyttal^heltal
279B  0C                 INC  C               bastoken + 2
279C  0C                 INC  C
279D  47                 LD   B,A             resultatet flyttal (A = 0)
279E  18 96              JR   L2736           NC från CP $00: skriv token
```

OPTYPE: gör typerna lika. B = höger typ, IX+5 = vänster typ. Heltal och flyttal blir flyttal. Fel 17 om en sträng blandas med ett tal, utom i ett fall: är den högra ett heltal prövas den vänstra med flaggorna från CP $01 *(*$27AE*), så en sträng till vänster släpps igenom och heltalet blir ett flyttal.* "A"+1% *ger fel 4 vid körningen och* "A"=1% *fel 50 i stället för fel 17. Version 9913 har rättat det.*

In: B = höger typ, IX = stackramen från PRECLV*, DE efter den högra operandens kod. Ut: NC, B = den gemensamma typen, C = token + typ och DE efter en omvandlingsbyte, om en skrevs; vid fel CY och A = felkoden. Förstör A.*

```
27A0  78          OPTYPE: LD   A,B             B = höger typ
27A1  DD BE 05            CP   (IX+$05)        samma typ?
27A4  28 11               JR   Z,L27B7
27A6  FE 00               CP   $00             höger flyttal?
27A8  28 13               JR   Z,L27BD
27AA  FE 01               CP   $01             höger heltal?
27AC  20 14               JR   NZ,L27C2        nej: höger sträng, fel 17
27AE  DD 7E 05            LD   A,(IX+$05)      vänster typ, men flaggorna är kvar (fel)
27B1  20 0F               JR   NZ,L27C2        Z från CP $01: hoppet tas aldrig
27B3  CD 6E 25            CALL TOFLT           höger till flyttal, vad vänster än är
27B6  D8                  RET  C               kan inte hända: höger är ett heltal
27B7  78          L27B7:  LD   A,B             B = den gemensamma typen
27B8  DD 86 03            ADD  A,(IX+$03)      token + typ
27BB  4F                  LD   C,A             C = token
27BC  C9                  RET                  NC

27BD  DD 7E 05    L27BD:  LD   A,(IX+$05)      vänster typ
27C0  FE 01               CP   $01             heltal?
27C2  3E 91       L27C2:  LD   A,$91           annars fel 17
27C4  37                  SCF
27C5  C0                  RET  NZ              vänster sträng och höger flyttal
27C6  3E DA               LD   A,$DA           vänster till flyttal
27C8  CD CF 27            CALL INSCNV
27CB  06 00               LD   B,$00           typ flyttal
27CD  18 E8               JR   L27B7
```

INSCNV: skjut in byten A vid början av den högra operandens kod (IX+6/7), det vill säga direkt efter den vänstra operanden. Koden efter flyttas en byte uppåt.

In: A = byten, IX = stackramen, DE efter koden. Ut: DE en byte längre fram. Förstör BC.

```
27CF  E5          INSCNV: PUSH HL              HL = texten, sparas
27D0  62                  LD   H,D             HL = slutet av koden
27D1  6B                  LD   L,E
```

```
27D2  DD 4E 06           LD   C,(IX+$06)      var den högra operandens kod börjar
27D5  DD 46 07           LD   B,(IX+$07)
27D8  A7                 AND  A
27D9  ED 42              SBC  HL,BC            längden av högra operandens kod
27DB  44                 LD   B,H              BC = antal byte att flytta
27DC  4D                 LD   C,L
27DD  62                 LD   H,D
27DE  6B                 LD   L,E
27DF  2B                 DEC  HL               HL = den sista byten
27E0  D5                 PUSH DE               slutet sparas
27E1  ED B8              LDDR                  en byte uppåt, från slutet
27E3  12                 LD   (DE),A           DE = den högra operandens början
27E4  D1                 POP  DE               DE = slutet
27E5  13                 INC  DE               en byte längre
27E6  E1                 POP  HL               HL = texten
27E7  C9                 RET
```

NXTLEV: anropa nästa nivå. IX = aktuell post; IX sätts till posten för nästa nivå, och dess rutin anropas. Rutinen gör RET *till den som anropade* NXTLEV*, och IX står då kvar på en annan nivås post; anroparna sparar IX själva. HL, DE och B går in och ut som till* CMPEXP*.*

```
27E8  E5           NXTLEV: PUSH HL               textpekaren
27E9  DD 6E 02             LD   L,(IX+$02)       nästa nivås post
27EC  DD 66 03             LD   H,(IX+$03)
27EF  E3                   EX   (SP),HL          posten på stacken, HL tillbaka
27F0  DD E1                POP  IX               IX = nästa nivås post
27F2  E5                   PUSH HL               textpekaren igen
27F3  DD 6E 00             LD   L,(IX+$00)       nivåns rutin
27F6  DD 66 01             LD   H,(IX+$01)
27F9  E3                   EX   (SP),HL          rutinen på stacken, HL tillbaka
27FA  C9                   RET                   hoppa till rutinen
```

*CMPADD: nivån för + och -. Ett − först (unärt) gäller hela den följande termen: -A*B = -(A*B). Koden blir termen följd av* $D7 *(flyttal) eller* $D8 *(heltal). Ett + först hoppas över. Sedan fortsätter det som en vanlig binär nivå.*

*Ett unärt tecken går bara först på nivån: efter * / ^ + - och efter ett annat unärt tecken är nästa nivå* PREC8/PRIMARY*, och det ger fel 11 (2*-3, 2+-3, 2^-1, - -2; skriv 2*(-3)). Efter* (*,* =*,* NOT*,* AND *... går det (1=-1).*

In: IX = PREC7*, HL = texten, DE = koden. Ut som* PRECLV*: NC, B = typen, HL efter termerna och DE efter koden; vid fel CY och A = felkoden (fel 17 för -A$). Förstör A, C och IX.*

```
27FB  7E           CMPADD: LD   A,(HL)          (egenhet)
27FC  FE 2D                LD   CP   $2D         unärt minus?
27FE  28 08                JR   Z,L2808          ja
2800  FE 2B                CP   $2B              unärt plus hoppas över
2802  CC BE 25             CALL Z,SKIPSP
2805  C3 05 27             JP   PRECLV           som en binär nivå, med IX = PREC7

2808  DF           L2808:  RST  $18              över - och mellanslag
2809  DD E5                PUSH IX               IX = PREC7
280B  CD E8 27             CALL NXTLEV           PREC8: termen
280E  DD E1                POP  IX               IX = PREC7 igen
2810  D8                   RET  C                fel
2811  3E 01                LD   A,$01            heltal?
2813  B8                   CP   B
2814  3E D8                LD   A,$D8            $D8: heltal
2816  28 09                JR   Z,L2821
2818  3E 00                LD   A,$00            flyttal?
281A  B8                   CP   B
281B  3E 91                LD   A,$91            sträng: fel 17
```

```
281D  37                SCF
281E  C0                RET   NZ
281F  3E D7             LD    A,$D7           $D7: flyttal
2821  12          L2821: LD   (DE),A         efter termen
2822  13                INC   DE
2823  C3 0D 27          JP    L270D           vidare som en binär nivå
```

CMPNOT: NOT *följt av nästa nivå (jämförelser), omvandlat till heltal, och* $E1. *Utan* NOT *bara nästa nivå.* NOT *binder alltså svagare än jämförelser:* NOT *A=B* = NOT *(A=B),* NOT *0+1 = -2. Efter* NOT *kommer* PREC6*, så* NOT NOT *5 och 1+*NOT *0 ger fel 11.*

In: IX = nivåns post (tabellen KWNOT *i +4/5), HL = texten, DE = koden. Ut: NC, B = typen (heltal efter* NOT*), HL efter texten och DE efter koden; vid fel CY och A = felkoden (fel 17 för* NOT *A$). Förstör A, C och IX.*

```
2826  D5          CMPNOT: PUSH DE             koden sparas
2827  DD 5E 04            LD   E,(IX+$04)    KWNOT
282A  DD 56 05            LD   D,(IX+$05)
282D  CD F5 09            CALL KWSRCH        NOT?
2830  D1                  POP  DE
2831  C2 E8 27            JP   NZ,NXTLEV     inte NOT: bara nästa nivå
2834  CD E8 27            CALL NXTLEV        PREC6: inget NOT till
2837  D8                  RET  C
2838  CD 61 25            CALL TOINT         värdet görs till heltal
283B  D8                  RET  C             fel 17 om en sträng
283C  3E E1               LD   A,$E1         OPNOT vid körning
283E  12                  LD   (DE),A
283F  13                  INC  DE
2840  C9                  RET                NC, B = heltal
```

Korsreferenstabell för kompileringen av uttrycken. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X) i registren. Nivåerna PREC1–PREC10 är data: varje post har nivåns rutin, nästa nivås post, operatortabellen och klassen, och NXTLEV går från en nivå till nästa. Nivårutinerna (PRECLV, CMPNOT, CMPADD och PRIMARY) tar IX = nivåns post, HL = texten och DE = koden och lämnar som CMPEXP NC, B = typen, HL efter texten och DE efter koden, eller vid fel CY och A = felkoden. SKIPSP och SKIPS0 är RST $18 och RST $20. Felen: VARNAM 11, CMPNUM 12, TYPCNV 13 och 17, TOINT och TOFLT 17, CMPARG 13 och 18, PRIMARY 18, och PRECLV, OPTYPE, CMPADD och CMPNOT 17. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register					
Rutin	A	CY	BC	DE	HL	IX
TYPCNV	IU	U	IU	IU		
TOINT	U	U	IU	IU		
TOFLT	U	U	IU	IU		
PRIMARY	U	U	U	IU	IU	X
SKIPSP	U	U			IU	
SKIPS0	U	U			IU	
CMPSTR	IX	U	U	IU	IU	
CMPVAR	U	U	U	IU	IU	X
CMPFNC	IU	U	U	IU	IU	X
CMPARG	U	U	U	IU	IU	IU
CMPNUM	U	U	U	IU	IU	X
VARNAM	U	U	U		IU	
CMPEXP	U	U	U	IU	IU	X
PRECLV	U	U	U	IU	IU	IX
OPTYPE	U	U	IU	IU		I
INSCNV	I	X	X	IU		I

	register					
Rutin	A	CY	BC	DE	HL	IX
NXTLEV	U	U	U	IU	IU	IX
CMPADD	U	U	U	IU	IU	IX
CMPNOT	U	U	U	IU	IU	IX

Rutin	**Anropar**	**Anropas från**
TYPCNV	TOINT, TOFLT (med JR); fel 13	LNKFN ($117F), CMPLET, CMPTYX, CMPARG
TOINT	fel 17	TYPCNV (med JR), CMPDMI, PRECLV, CMPNOT
TOFLT	fel 17	TYPCNV (med JR), OPTYPE
PRIMARY	CMPNUM (med JP), CMPSTR, CMPFNC (med JR), KWSRCH (FNKW), CMPVAR, CMPEXP (inom parentes); går in i SKIPSP; fel 18	NXTLEV (nivå 10, PREC10)
SKIPSP	går in i SKIPS0	RST $18 från många ställen, KWSRCH, READ, SKIPS0 (med JR), CMPNUM, VARNAM (med JP), CMPADD
SKIPS0	SKIPSP (med JR)	RST $20 från många ställen, SKIPSP (går in i den), CMPSTR (med JR)
CMPSTR	SKIPS0 (med JR)	PRIMARY (med JR)
CMPVAR	VARNAM, CMPARG (typlistan D2618), CMPERR (med JP)	CMPLET, CMPVR, CMPIVR, CMPGET, PRIMARY; slutet ($25FC) från CMPDMI och CMPSV (med JP)
CMPFNC	CMPFN (med JP, för FN), CMPARG (typlistan i FNTYPE), CMPERR (med JP)	PRIMARY (med JR)
CMPARG	CMPEXP, TYPCNV, CMPERR (med JP); fel 13 och 18	CMPDMI, CMPVAR, CMPFNC
CMPNUM	GETFLT, GETLN, WRBC1, SKIPSP (med JP); fel 12	PRIMARY (med JP)
VARNAM	SKIPSP (med JP); fel 11	CMPDEF, CMPFN, CMPDMI, CMPSV, CMPVAR
CMPEXP	går in i PRECLV med PREC1	CMPFN, CMPLET, CMPDMI, CMPEXB, CMPTYX, PRIMARY, CMPARG
PRECLV	NXTLEV, KWSRCH (nivåns operatortabell), OPTYPE, TOINT, INSCNV; fel 17	CMPEXP (går in i den), NXTLEV (nivå 1–4, 6, 8 och 9), CMPADD (med JP)
OPTYPE	TOFLT, INSCNV; fel 17	PRECLV
INSCNV		PRECLV, OPTYPE
NXTLEV	nästa nivås rutin (med RET): PRECLV, CMPNOT, CMPADD eller PRIMARY	PRECLV, CMPADD, CMPNOT (också med JP)
CMPADD	SKIPSP, NXTLEV, PRECLV (med JP); fel 17	NXTLEV (nivå 7, PREC7)
CMPNOT	KWSRCH (KWNOT), NXTLEV (också med JP), TOINT	NXTLEV (nivå 5, PREC5)

3.67 EXEXP: utför kompilerad uttryckskod (RST $38) $2841

DE pekar på koden. Varje token $B6–$FF hämtas och rutinen tas ur EXPTAB1 (2 × (token - $B6)). Rutinerna arbetar med värden på Z80-stacken: ett heltal är 2 byte, ett flyttal 5 byte, och de flesta slutar med RST $30 (hämta nästa token) eller RST $28 (lägg HL på stacken och hämta nästa token). En sluttoken (EXEND) avslutar. EXEXPH ($2841, RST $28): PUSH HL, sedan som EXEXP.

```
2841  E5          EXEXPH: PUSH HL             HL på stacken
```

3.68 Värdestacken när uttryckskoden körs $2842

DE pekar på koden och flyttas fram av varje rutin. Värdena ligger på Z80-stacken:

```
heltal   2 byte
flyttal  5 byte: SP+0..2 mantissa (BCD, mest signifikanta byten
         först), SP+3 tecken (0 = +, 1 = -), SP+4 exponent
         (0 = talet är noll). Flyttalsrutinerna får adressen till
         exponentbyten (den högsta), och en flyttalsvariabels
         adress pekar också på exponenten.
sträng   tre ord, från SP och uppåt:
           [n = antal tillfälliga byte] [adress] [längd]
         följda av n byte. För en variabel är n = 0; en ny
         sträng (som från CHR$) ligger i de n byten, direkt
         ovanför beskrivningen.
variabeladress 2 byte  (för tilldelning).
```

När en rutin startar är CY = 0 och B = 0 (SUBI och EXFUNC räknar med det). Typen i B vid slutet: 0 = flyttal, 1 = heltal, 2 = sträng.

In: DE = koden. Ut efter sluttoken: B = typen och DE efter koden; ett heltal eller en variabels adress i HL, ett flyttal eller en sträng vid SP (se EXPTAB1). Fel vid körningen går inte tillbaka hit utan till felhanteringen. Förstör A, C och HL.

```
2842  1A          EXEXP:  LD   A,(DE)          token
2843  13                  INC  DE
2844  D6 B6               SUB  $B6             index i EXPTAB1
2846  21 8B 28            LD   HL,EXPTAB1
2849  4F                  LD   C,A
284A  06 00               LD   B,$00           BC = index
284C  09                  ADD  HL,BC           2*index (B = 0)
284D  09          L284D:  ADD  HL,BC           HL på rutinens adress
284E  7E          L284E:  LD   A,(HL)          läs rutinens adress
284F  23                  INC  HL
2850  66                  LD   H,(HL)
2851  6F                  LD   L,A             HL = rutinen
2852  E9          JPHL:   JP   (HL)            också för EXCALL ($31DB)
```

EXFUNC: $CD-$D1, *nästa byte = 2*kod, rutinen tas ur* EXPTAB2.

```
2853  21 1F 29    EXFUNC: LD   HL,EXPTAB2      HL = tabellen
2856  1A                  LD   A,(DE)          2*kod
2857  13                  INC  DE
2858  4F                  LD   C,A             B är redan 0
2859  18 F2               JR   L284D           B är redan 0
```

EXEND ($BA *och* $BD)*: slut, heltal. HL = heltalet* ($BA) *eller heltalsvariabelns adress* ($BD)*, B = 1.*

```
285B  E1          EXEND:  POP  HL              värdet, eller heltalsvariabelns adress
285C  06 01               LD   B,$01           B = heltal
285E  C9                  RET                  till satsen som gjorde RST $38
```

ENDF ($B9): *slut, flyttal. Flyttalet ska bli kvar på stacken, och returadressen ovanför det måste bort: de 9 byten (flyttalet och två sparade ord) flyttas 2 byte uppåt, över den, och returadressen hamnar vid SP.*

```
285F  21 05 00     ENDF:  LD   HL,$0005      returadressen (SP+5/6)
2862  39                 ADD  HL,SP
2863  4E                 LD   C,(HL)
2864  23                 INC  HL
2865  46                 LD   B,(HL)
2866  C5                 PUSH BC              kopia av returadressen
2867  D5                 PUSH DE              kodpekaren
2868  EB                 EX   DE,HL           DE = returadressens höga byte
2869  21 FE FF           LD   HL,$FFFE
286C  19                 ADD  HL,DE           HL = flyttalets exponent
286D  01 09 00           LD   BC,$0009        flyttal + 2 ord
2870  ED B8              LDDR                 9 byte flyttas 2 byte uppåt
2872  D1                 POP  DE              kodpekaren
2873  D1                 POP  DE              dess kopia; SP på returadressen
2874  06 00        L2874: LD   B,$00           B = 0: flyttal
2876  C9                 RET                  till satsen, flyttalet vid SP
```

ENDS ($BB): *slut, sträng. Returadressen under strängen räknas in bland de tillfälliga byten ($n + 2$), och det hoppas till den med* JP *(HL). Strängen blir kvar på stacken.*

```
2877  E1           ENDS:  POP  HL               HL = n
2878  23                  INC  HL               returadressen räknas som tillfällig
2879  23                  INC  HL
287A  E5                  PUSH HL               n + 2
287B  39                  ADD  HL,SP            HL = SP + n + 2
287C  23                  INC  HL
287D  23                  INC  HL
287E  23                  INC  HL
287F  23                  INC  HL               HL = SP + n + 6: returadressen
2880  06 02               LD   B,$02            B = sträng, hämta adressen och hoppa
2882  18 CA               JR   L284E
```

ENDVF ($BC): *HL = variabelns adress, B = 0 (flyttal).*

```
2884  E1          ENDVF:  POP  HL               adressen
2885  18 ED               JR   L2874            B = flyttal
```

ENDVS ($BE): *HL = variabelns adress, B = 2 (sträng).*

```
2887  E1          ENDVS:  POP  HL               adressen
2888  06 02               LD   B,$02
288A  C9                  RET                   till satsen
```

3.69 EXPTAB1: rutiner för token $B6–$FF vid körning $288B

```
$B6-$B8  tilldelning flyttal/heltal/sträng    ASGNF ASGNI ASGNS
$B9-$BB  slut, värde av typ token-$B9:        $285F EXEND $2877
         flyttal och sträng blir kvar på
         stacken, heltal i HL
$BC-$BE  slut, HL = variabelns adress och    $2884 EXEND $2887
         B = typ token-$BC (efter CMPVR)
         $BD avslutar också IF-villkoret
$BF      slut på DIM-listan                   RET
$C0-$C2  variabel utan index                  VAR0
$C3      flyttalskonstant                     CONSTF
$C4-$C6  variabel med ett index               VAR1F VAR1I VAR1S
$C7      heltalskonstant                      CONSTI
```

```
$C8-$CA  variabel med två index               VAR2F VAR2I VAR2S
$CB      strängkonstant                       CONSTS
$CC      DIM-post                             $1BFE
$CD-$D1  funktion, 0-4 argument               EXFUNC
$D2      DIM av sträng utan längd: värdet 0   $1C7F
$D3      parentes: gör inget                  EXEXP
$D4-$D6  hämta variabelns värde               GETF GETI GETS
$D7 $D8  unärt minus                          NEGF NEGI
$D9 $DA  flyttal -> heltal, heltal -> flyttal FTOI ITOF
$DB      FN-argumentpost: hoppas över         $1B85
$DC-$E1  EQV IMP OR XOR AND NOT               OPEQV ...
$E2-$F3  = <> < >= > <=, tre typer var        EQF EQI EQS ...
$F4-$FC  + - * /, $F6 = sammanfogning         ADDF ... CONCAT
$FD-$FF  ^ flyttal, heltal, flyttal^heltal    POWF POWI POWFI
```

Namnen i tabellen är satta efter vilken token de hör till.

```
                EXPTAB1:
288B  89 29             DEFW ASGNF          $B6 tilldelning
288D  B4 29             DEFW ASGNI          $B7
288F  C3 29             DEFW ASGNS          $B8
2891  5F 28             DEFW ENDF           $B9 slut, värdet
2893  5B 28             DEFW EXEND          $BA
2895  77 28             DEFW ENDS           $BB
2897  84 28             DEFW ENDVF          $BC slut, variabelns adress
2899  5B 28             DEFW EXEND          $BD, också IF-villkoret
289B  87 28             DEFW ENDVS          $BE
289D  7E 1C             DEFW ENDDIM         $BF slut på DIM-listan
289F  CD 2C             DEFW VAR0           $C0 variabel utan index
28A1  CD 2C             DEFW VAR0           $C1
28A3  CD 2C             DEFW VAR0           $C2
28A5  36 2A             DEFW CONSTF         $C3 konstant, flyttal
28A7  D9 2C             DEFW VAR1F          $C4 variabel med ett index
28A9  DF 2C             DEFW VAR1I          $C5
28AB  E4 2C             DEFW VAR1S          $C6
28AD  E3 2A             DEFW CONSTI         $C7 konstant, heltal
28AF  2B 2D             DEFW VAR2F          $C8 variabel med två index
28B1  31 2D             DEFW VAR2I          $C9
28B3  36 2D             DEFW VAR2S          $CA
28B5  D7 2E             DEFW CONSTS         $CB konstant, sträng
28B7  FE 1B             DEFW DIMELM         $CC DIM-post
28B9  53 28             DEFW EXFUNC         $CD funktion utan argument
28BB  53 28             DEFW EXFUNC         $CE ett argument
28BD  53 28             DEFW EXFUNC         $CF två
28BF  53 28             DEFW EXFUNC         $D0 tre
28C1  53 28             DEFW EXFUNC         $D1 fyra
28C3  7F 1C             DEFW PUSH0          $D2 DIM av sträng utan längd
28C5  42 28             DEFW EXEXP          $D3 parentes: nästa token
28C7  A1 29             DEFW GETF           $D4 hämta värdet
28C9  BD 29             DEFW GETI           $D5
28CB  22 2A             DEFW GETS           $D6
28CD  43 2A             DEFW NEGF           $D7 unärt minus
28CF  E9 2A             DEFW NEGI           $D8
28D1  E2 2D             DEFW FTOI           $D9 flyttal till heltal
28D3  F0 2D             DEFW ITOF           $DA heltal till flyttal
28D5  85 1B             DEFW SKPARG         $DB FN-argumentpost
28D7  F5 2A             DEFW OPEQV          $DC
28D9  00 2B             DEFW OPIMP          $DD
28DB  0B 2B             DEFW OPOR           $DE
28DD  21 2B             DEFW OPXOR          $DF
28DF  14 2B             DEFW OPAND          $E0
28E1  1D 2B             DEFW OPNOT          $E1
```

```
28E3  4C 2A             DEFW EQF          $E2 =
28E5  2A 2B             DEFW EQI          $E3
28E7  E4 2F             DEFW EQS          $E4
28E9  55 2A             DEFW NEF          $E5 <>
28EB  34 2B             DEFW NEI          $E6
28ED  ED 2F             DEFW NES          $E7
28EF  70 2A             DEFW LTF          $E8 <
28F1  3E 2B             DEFW LTI          $E9
28F3  08 30             DEFW LTS          $EA
28F5  75 2A             DEFW GEF          $EB >=
28F7  4A 2B             DEFW GEI          $EC
28F9  0D 30             DEFW GES          $ED
28FB  5E 2A             DEFW GTF          $EE >
28FD  5A 2B             DEFW GTI          $EF
28FF  F6 2F             DEFW GTS          $F0
2901  67 2A             DEFW LEF          $F1 <=
2903  56 2B             DEFW LEI          $F2
2905  FF 2F             DEFW LES          $F3
2907  B7 2A             DEFW ADDF         $F4 +
2909  5E 2B             DEFW ADDI         $F5
290B  47 2F             DEFW CONCAT       $F6 två strängar
290D  BD 2A             DEFW SUBF         $F7 -
290F  62 2B             DEFW SUBI         $F8
2911  C3 2A             DEFW MULF         $F9 *
2913  67 2B             DEFW MULI         $FA
2915  C9 2A             DEFW DIVF         $FB /
2917  73 2B             DEFW DIVI         $FC
2919  00 2E             DEFW POWF         $FD ^
291B  18 2E             DEFW POWI         $FE
291D  5E 2E             DEFW POWFI        $FF flyttal^heltal
```

3.70 EXPTAB2: funktionerna, index = kod (token i FNKW - $80) $291F

```
 0 FN      1 ABS     2 ATN     3 COS     4 EXP     5 FIX
 6 INT     7 LOG     8 LOG10   9 PI      A RND     B SGN
 C SIN     D SQR     E TAN     F ASC    10 CHR$   11 LEFT$
12 RIGHT$ 13 MID$   14 LEN    15 INSTR  16 SPACE$ 17 STRING$
18 NUM$   19 VAL    1A SWAP%  1B PEEK   1C INP    1D CALL
1E ERRCODE 1F IEC$  20 TAB    21 CUR    23 DOT
30 ADD$   31 SUB$   32 MUL$   33 DIV$   34 COMP%
```

Kod $22 och $24–$2F har inget namn i FNKW och går till fel 8, FINNS EJ I DETTA SYSTEM (platser för fler funktioner (?)). IEC$ ($016B) ligger bland ingångarna till tilläggs-ROM:en.

```
                EXPTAB2:
291F  2A 1B             DEFW FN           kod 0
2921  7F 2D             DEFW EXABS        kod 1
2923  AF 2D             DEFW EXATN        kod 2
2925  A5 2D             DEFW EXCOS        kod 3
2927  B4 2D             DEFW EXEXP1       kod 4
2929  96 2D             DEFW EXFIX        kod 5
292B  9B 2D             DEFW EXINT        kod 6
292D  B9 2D             DEFW EXLOG        kod 7
292F  BE 2D             DEFW EXLG10       kod 8
2931  6E 2D             DEFW EXPI         kod 9
2933  E0 31             DEFW EXRND        kod $0A
2935  86 2D             DEFW EXSGN        kod $0B
2937  A0 2D             DEFW EXSIN        kod $0C
2939  C3 2D             DEFW EXSQR        kod $0D
293B  AA 2D             DEFW EXTAN        kod $0E
```

```
293D  EC 2E           DEFW EXASC        kod $0F
293F  C1 2F           DEFW EXCHR        kod $10
2941  07 2F           DEFW EXLEFT       kod $11
2943  1C 2F           DEFW EXRGHT       kod $12
2945  FD 2E           DEFW EXMID        kod $13
2947  E3 2E           DEFW EXLEN        kod $14
2949  75 30           DEFW EXINST       kod $15
294B  F7 30           DEFW EXSPC        kod $16
294D  FB 30           DEFW EXSTRG       kod $17
294F  52 31           DEFW EXNUM        kod $18
2951  72 31           DEFW EXVAL        kod $19
2953  79 2B           DEFW EXSWAP       kod $1A
2955  7E 2B           DEFW EXPEEK       kod $1B
2957  C1 31           DEFW EXINP        kod $1C
2959  CC 31           DEFW EXCALL       kod $1D
295B  4F 20           DEFW EXERR        kod $1E
295D  6B 01           DEFW EXIEC        kod $1F, i tilläggskretsen
295F  26 31           DEFW EXTAB        kod $20
2961  3B 31           DEFW EXCUR        kod $21
2963  09 07           DEFW L0709        kod $22 utan namn: fel 8
2965  D6 22           DEFW EXDOT        kod $23
2967  09 07           DEFW L0709        kod $24-$2F utan namn: fel 8
2969  09 07           DEFW L0709
296B  09 07           DEFW L0709
296D  09 07           DEFW L0709
296F  09 07           DEFW L0709
2971  09 07           DEFW L0709
2973  09 07           DEFW L0709
2975  09 07           DEFW L0709
2977  09 07           DEFW L0709
2979  09 07           DEFW L0709
297B  09 07           DEFW L0709
297D  09 07           DEFW L0709
297F  9F 2B           DEFW EXADD        kod $30
2981  A4 2B           DEFW EXSUB        kod $31
2983  A9 2B           DEFW EXMUL        kod $32
2985  AE 2B           DEFW EXDIV        kod $33
2987  B3 2B           DEFW EXCOMP       kod $34
```

3.71 ASGNF / ASGNI / ASGNS: tilldelning ($B6–$B8) $2989

Stacken: [värde] [variabelns adress]. Värdet skrivs till variabeln, båda tas bort, och RET går direkt till den som anropade RST $38.

ASGNS: variabelns adress ligger under strängens tillfälliga byte.

- Tom sträng: bara längden (+4/5) sätts till 0.
- Är strängen inte reserverad (+0/1 = 0) reserveras 80 byte, eller strängens längd om den är större (DIMSTR).
- Räcker inte minnet till reservationen blir det fel 3 i DIMSTR.
- Annars fel 10, TEXTEN FÅR EJ PLATS I STRÄNGEN, om texten är längre än den reserverade längden.
- Fel 10 även om variabelns post ligger ovanför SP, det vill säga på stacken. Där ligger bara argumenten till en DEF-funktion (FNPAR), och ett DEF-uttryck kan inte tilldela något, så kontrollen slår aldrig till.

In: DE efter tilldelningens token. Ut: B = typen, HL = variabelns adress (för en sträng dess post) och DE efter koden. Förstör C, ASGNS också A och IX.

```
2989  21 06 00     ASGNF:  LD   HL,$0006        variabelns adress (SP+5/6)
298C  39                   ADD  HL,SP
298D  D5                   PUSH DE               kodpekaren
298E  56                   LD   D,(HL)           DE = variabelns adress
298F  2B                   DEC  HL
2990  5E                   LD   E,(HL)
2991  2B                   DEC  HL               HL = flyttalets exponent
2992  D5                   PUSH DE               variabelns adress sparas
2993  01 05 00             LD   BC,$0005         5 byte, bakifrån
2996  ED B8                LDDR                  exponenten först
2998  E1                   POP  HL               HL = variabelns adress
2999  D1                   POP  DE               DE = koden
299A  C1                   POP  BC               ta bort 7 byte
299B  C1                   POP  BC
299C  C1                   POP  BC
299D  33                   INC  SP               ta bort 7 byte
299E  06 00                LD   B,$00            B = flyttal
29A0  C9                   RET                   till satsen som gjorde RST $38
```

GETF ($D4): *ersätt adressen på stacken med flyttalet (5 byte).*

```
29A1  C1           GETF:   POP  BC               adressen, vid exponenten
29A2  21 FB FF             LD   HL,$FFFB         plats för flyttalet
29A5  39                   ADD  HL,SP
29A6  F9                   LD   SP,HL            SP på platsen
29A7  D5                   PUSH DE               kodpekaren
29A8  50                   LD   D,B              DE = variabelns exponent
29A9  59                   LD   E,C
29AA  01 04 00             LD   BC,$0004
29AD  09                   ADD  HL,BC            HL = platsens högsta byte
29AE  EB                   EX   DE,HL            HL = variabeln, DE = platsen
29AF  03                   INC  BC               5 byte
29B0  ED B8                LDDR                  kopiera 5 byte
29B2  D1                   POP  DE               DE = koden
29B3  F7                   RST  $30              nästa token

29B4  C1           ASGNI:  POP  BC               värdet
29B5  E1                   POP  HL               adressen
29B6  71                   LD   (HL),C
29B7  23                   INC  HL
29B8  70                   LD   (HL),B
29B9  2B                   DEC  HL               HL = variabelns adress
29BA  06 01                LD   B,$01            B = heltal
29BC  C9                   RET                   till satsen som gjorde RST $38
```

GETI ($D5): *ersätt adressen med heltalet.*

```
29BD  E1           GETI:   POP  HL               variabelns adress
29BE  7E                   LD   A,(HL)
29BF  23                   INC  HL
29C0  66                   LD   H,(HL)
29C1  6F                   LD   L,A
29C2  EF                   RST  $28              heltalet på stacken

29C3  DD E1        ASGNS:  POP  IX               IX = n
29C5  E1                   POP  HL               textens adress
29C6  C1                   POP  BC               längd
29C7  DD 39                ADD  IX,SP            IX = platsen för variabelns adress
29C9  79                   LD   A,C              längden 0?
29CA  B0                   OR   B
29CB  28 46                JR   Z,L2A13          tom sträng
29CD  DD E5                PUSH IX               platsen för variabelns adress sparas
```

```
29CF  D5              PUSH DE          kodpekaren
29D0  E5              PUSH HL          textens adress
29D1  DD 6E 00        LD   L,(IX+$00)  variabelns adress
29D4  DD 66 01        LD   H,(IX+$01)
29D7  E5              PUSH HL          sparas
29D8  ED 72           SBC  HL,SP       på stacken?
29DA  D2 0D 07        JP   NC,L070D    ovanför SP: fel 10
29DD  DD E1           POP  IX          IX = variabelns post
29DF  DD 6E 00        LD   L,(IX+$00)  reserverad längd
29E2  DD 66 01        LD   H,(IX+$01)
29E5  7D              LD   A,L
29E6  B4              OR   H
29E7  20 10           JR   NZ,L29F9    reserverad
29E9  21 50 00        LD   HL,$0050    inte reserverad: 80 byte
29EC  54              LD   D,H         DE = 80
29ED  5D              LD   E,L
29EE  ED 42           SBC  HL,BC       längden större?
29F0  30 02           JR   NC,L29F4
29F2  50              LD   D,B         nej: DE = längden
29F3  59              LD   E,C
29F4  CD 38 33  L29F4: CALL DIMSTR     fel 3 om minnet inte räcker
29F7  18 05           JR   L29FE

29F9  ED 42     L29F9: SBC  HL,BC      får plats?
29FB  DA 0D 07        JP   C,L070D     fel 10
29FE  DD 71 04  L29FE: LD   (IX+$04),C nuvarande längd = BC
2A01  DD 70 05        LD   (IX+$05),B
2A04  DD 5E 02        LD   E,(IX+$02)  textens plats
2A07  DD 56 03        LD   D,(IX+$03)
2A0A  E1              POP  HL          HL = texten
2A0B  ED B0           LDIR             kopiera texten
2A0D  D1              POP  DE          DE = koden
2A0E  E1              POP  HL          HL = platsen för variabelns adress
2A0F  F9              LD   SP,HL       ta bort strängen
2A10  E1              POP  HL          HL = variabelns post
2A11  18 0C           JR   L2A1F

2A13  DD F9     L2A13: LD   SP,IX      SP på variabelns adress
2A15  E1              POP  HL          HL = variabelns post
2A16  E5              PUSH HL
2A17  DD E1           POP  IX          IX = posten
2A19  DD 71 04        LD   (IX+$04),C  längd 0 (BC = 0)
2A1C  DD 70 05        LD   (IX+$05),B
2A1F  06 02     L2A1F: LD   B,$02      B = sträng
2A21  C9              RET              till satsen som gjorde RST $38
```

GETS ($D6): *ersätt adressen med en strängbeskrivning [0] [adress] [längd] som pekar på variabelns text. Förstör också IX.*

```
2A22  DD E1     GETS:  POP  IX          variabelns post
2A24  DD 4E 04         LD   C,(IX+$04)  längd
2A27  DD 46 05         LD   B,(IX+$05)
2A2A  DD 6E 02         LD   L,(IX+$02)  adress
2A2D  DD 66 03         LD   H,(IX+$03)
2A30  C5               PUSH BC
2A31  E5               PUSH HL
2A32  21 00 00         LD   HL,$0000    n = 0
2A35  EF               RST  $28         beskrivningen klar, nästa token
```

CONSTF ($C3): *kopiera flyttalet (5 byte) ur koden till stacken.*

```
2A36  21 FB FF  CONSTF: LD   HL,$FFFB   plats för 5 byte
```

```
2A39  39                ADD   HL,SP
2A3A  F9                LD    SP,HL
2A3B  EB                EX    DE,HL          HL = koden, DE = SP
2A3C  01 05 00          LD    BC,$0005       mantissan först, exponenten sist
2A3F  ED B0             LDIR
2A41  EB                EX    DE,HL          DE efter konstanten
2A42  F7                RST   $30            nästa token
```

NEGF ($D7): *byt tecken på flyttalet (tecknet i SP+3).*

```
2A43  21 03 00    NEGF: LD    HL,$0003       tecknet
2A46  39                ADD   HL,SP
2A47  3E 01             LD    A,$01          bit 0 vänds
2A49  AE                XOR   (HL)
2A4A  77                LD    (HL),A
2A4B  F7                RST   $30            nästa token
```

Korsreferenstabell för körningen av uttryckskoden. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna. EXEXP (RST $38) räknar ut ett helt uttryck och står för det som satserna ser: B = typen och DE efter koden, ett heltal eller en variabels adress i HL och ett flyttal eller en sträng vid SP. Kodrutinerna tar DE efter sin token och värdena vid SP och lämnar resultatet där (stacken IU). De flesta slutar med RST $30 eller, med HL på stacken, RST $28 (EXEXPH), som tar nästa token genom EXEXP; det som nästa rutin gör räknas inte, men EXEXP förstör A, flaggorna, BC och HL. Sluttoken och tilldelningarna går med RET tillbaka till satsen. ASGNS kan reservera texten på heapen med DIMSTR. Felen: ASGNS 3 och 10. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP
EXEXPH	X	X	X	IU	IX		U	
EXEXP	X	X	U	IU	U	X	U	
JPHL					I			
EXFUNC	X	X	IX	IU	X			
EXEND			U		U		IU	
ENDF		X	U		X		IU	
ENDS	X	X	U		X		IU	
ENDVF			U		U		IU	
ENDVS			U		U		IU	
ASGNF		X	U		U		IU	
GETF	X	X	X	IU	X		IU	
ASGNI			U		U		IU	
GETI	X	X	X	IU	X		IU	
ASGNS	X	X	U		U	X	IU	IU
GETS	X	X	X	IU	X	X	IU	
CONSTF	X	X	X	IU	X		U	
NEGF	X	X	X	IU	X		IU	

Rutin	**Anropar**	**Anropas från**
EXEXPH	går in i EXEXP	RST $28 (RST28, med JP)
EXEXP	rutinen för token i EXPTAB1 (genom JPHL)	RST $38 (RST38, med JP), RST $30, EXEXPH (går in i den), EXPTAB1 ($D3)
JPHL	rutinen i HL	EXEXP och EXFUNC (går in i den), EXESTM (till $284D, med JP), EXCALL ($31DB)

Rutin	Anropar	Anropas från
EXFUNC	rutinen för koden i EXPTAB2 (genom JPHL)	EXPTAB1 ($CD–$D1)
EXEND		EXPTAB1 ($BA och $BD)
ENDF		EXPTAB1 ($B9), ENDVF (till $2874, med JR)
ENDS	hoppar till returadressen (genom JPHL)	EXPTAB1 ($BB)
ENDVF	ENDF ($2874, med JR)	EXPTAB1 ($BC)
ENDVS		EXPTAB1 ($BE)
ASGNF		EXPTAB1 ($B6)
GETF	RST $30	EXPTAB1 ($D4)
ASGNI		EXPTAB1 ($B7)
GETI	RST $28	EXPTAB1 ($D5)
ASGNS	DIMSTR; fel 10	EXPTAB1 ($B8), GET, RDVAL och INPLIN (med JP)
GETS	RST $28	EXPTAB1 ($D6)
CONSTF	RST $30	EXPTAB1 ($C3)
NEGF	RST $30	EXPTAB1 ($D7)

3.72 Jämförelse av flyttal, $E2–$F3 $2A4C

FCMPST tar bort båda flyttalen och ger flaggorna: Z = lika, CY = vänster mindre än höger. Resultatet är ett heltal: -1 (sant) eller 0.

Token: EQF $E2, NEF $E5, LTF $E8, GEF $EB, GTF $EE och LEF $F1. LTF och GEF har ingen egen kod efter anropet utan hoppar in i LEF och GTF förbi provet av Z.

In: DE efter token, de två flyttalen vid SP (det högra närmast). Ut: heltalet i deras ställe (RST $28). Förstör A, BC, HL och IX.

```
2A4C  CD 7A 2A     EQF:     CALL  FCMPST          Z: lika, CY: vänster mindre
2A4F  28 09                 JR    Z,L2A5A         lika: sant
2A51  21 00 00     L2A51:   LD    HL,$0000        falskt
2A54  EF                    RST   $28             heltalet på stacken, nästa token

2A55  CD 7A 2A     NEF:     CALL  FCMPST
2A58  28 F7                 JR    Z,L2A51         lika: falskt
2A5A  21 FF FF     L2A5A:   LD    HL,$FFFF        sant
2A5D  EF                    RST   $28             heltalet på stacken, nästa token

2A5E  CD 7A 2A     GTF:     CALL  FCMPST
2A61  28 EE                 JR    Z,L2A51         lika: falskt
2A63  38 EC        L2A63:   JR    C,L2A51         mindre: falskt
2A65  18 F3                 JR    L2A5A           större: sant

2A67  CD 7A 2A     LEF:     CALL  FCMPST
2A6A  28 EE                 JR    Z,L2A5A         lika: sant
2A6C  38 EC        L2A6C:   JR    C,L2A5A         mindre: sant
2A6E  18 E1                 JR    L2A51           större: falskt

2A70  CD 7A 2A     LTF:     CALL  FCMPST
2A73  18 F7                 JR    L2A6C           som LEF, men lika ger falskt

2A75  CD 7A 2A     GEF:     CALL  FCMPST
2A78  18 E9                 JR    L2A63           som GTF, men lika ger sant
```

FCMPST: jämför de två flyttalen närmast SP (FCMP) och ta bort dem. Returadressen tas först bort och hoppet tillbaka görs med JP (IX). Flaggorna sparas över SP-justeringen med RLA/RRA.

In: flyttalen ovanför returadressen, DE = kodpekaren. Ut: Z om lika, CY om det vänstra är mindre, SP förbi båda flyttalen och DE orörd. Förstör A, BC, HL och IX (returadressen).

```
2A7A  DD E1        FCMPST: POP   IX          returadressen
2A7C  21 04 00             LD    HL,$0004    höger operands exponent
2A7F  39                   ADD   HL,SP       HL = SP + 4
2A80  D5                   PUSH  DE          kodpekaren
2A81  11 05 00             LD    DE,$0005    fem byte till den vänstra
2A84  EB                   EX    DE,HL       DE = b:s exponent
2A85  19                   ADD   HL,DE       HL = a:s exponent
2A86  CD 93 2A             CALL  FCMP        Z: lika, CY: a < b
2A89  17                   RLA               CY i bit 0 av A; Z orörd
2A8A  D1                   POP   DE          kodpekaren tillbaka
2A8B  21 0A 00             LD    HL,$000A    tio byte: båda flyttalen
2A8E  39                   ADD   HL,SP       ändrar CY, inte Z
2A8F  F9                   LD    SP,HL       SP förbi båda flyttalen
2A90  1F                   RRA               CY tillbaka
2A91  DD E9        JPIX:   JP    (IX)        tillbaka till anroparen
```

3.73 FCMP: jämför flyttal $2A93

In: HL och DE pekar på exponenterna för a och b.

Ut: Z om a = b, annars CY om a är mindre än b.

Båda exponenterna 0 räknas som lika. Olika tecken: tecknet avgör. Samma tecken: exponenten, sedan mantissans 3 byte från den mest signifikanta. För negativa tal byts a och b först. Tecknets byte är 0 för plus och 1 för minus, och -0 (exponenten 0, tecknet 1) är lika med 0 men större än -1.

Förstör A, BC, DE och HL; HL och DE pekar efteråt in i talen.

Används också av FOR/NEXT (FORTSF).

```
2A93  1A           FCMP:   LD    A,(DE)      b:s exponent
2A94  47                   LD    B,A         B = b:s exponent
2A95  B6                   OR    (HL)        båda exponenterna 0?
2A96  C8                   RET   Z           båda 0: lika (Z)
2A97  4E                   LD    C,(HL)      C = a:s exponent
2A98  1B                   DEC   DE          DE på b:s tecken
2A99  2B                   DEC   HL          HL på a:s tecken
2A9A  1A                   LD    A,(DE)      b:s tecken (0 = +, 1 = -)
2A9B  BE                   CP    (HL)        b - a: CY om bara a är negativt
2A9C  C0                   RET   NZ          olika tecken: svaret klart
2A9D  B7                   OR    A           samma tecken: positiva?
2A9E  79                   LD    A,C         A = a:s exponent (flaggorna kvar)
2A9F  28 03                JR    Z,L2AA4     positiva: a mot b
2AA1  EB                   EX    DE,HL       negativa: byt a och b
2AA2  78                   LD    A,B         A = b:s exponent
2AA3  41                   LD    B,C         B = a:s exponent
2AA4  B8           L2AA4:  CP    B           exponenterna, CY om den i A är mindre
2AA5  C0                   RET   NZ          olika: svaret klart
2AA6  01 03 00             LD    BC,$0003    tre byte bakåt till mantissan
2AA9  ED 42                SBC   HL,BC       HL på mantissan (CY = 0)
2AAB  EB                   EX    DE,HL       DE = a:s mantissa (b:s om negativa)
2AAC  ED 42                SBC   HL,BC       HL = b:s mantissa (a:s om negativa)
2AAE  41                   LD    B,C         B = 3 byte
2AAF  1A           L2AAF:  LD    A,(DE)      3 byte, mest signifikant först
2AB0  BE                   CP    (HL)        CY om byten i DE är mindre
2AB1  C0                   RET   NZ          olika: svaret klart
2AB2  23                   INC   HL
```

```
2AB3  13                INC  DE
2AB4  10 F9             DJNZ L2AAF          nästa byte
2AB6  C9                RET                 alla lika: Z
```

3.74 ADDF / SUBF / MULF / DIVF $2AB7

IX = flyttalsrutinen. FARITH anropar den med DE = BC = vänster operands exponent och HL = höger operands. Resultatet hamnar i vänster operand, och höger (5 byte) tas bort från stacken.

Token: ADDF $F4, SUBF $F7, MULF $F9 och DIVF $FB. Ingen RST $28: resultatet ligger redan där det ska, och RST $30 går till nästa token.

In: DE efter token, de två flyttalen vid SP (det högra närmast). Ut: resultatet vid SP, DE orörd. Förstör A, BC, HL, IX och det som flyttalsrutinen förstör.

```
2AB7  DD 21 F7 33  ADDF:   LD   IX,FADD
2ABB  18 10                JR   FARITH

2ABD  DD 21 60 34  SUBF:   LD   IX,FSUB
2AC1  18 0A                JR   FARITH

2AC3  DD 21 DF 34  MULF:   LD   IX,FMUL
2AC7  18 04                JR   FARITH

2AC9  DD 21 EE 35  DIVF:   LD   IX,FDIV
2ACD  21 04 00     FARITH: LD   HL,$0004      höger operands exponent
2AD0  39                   ADD  HL,SP
2AD1  D5                   PUSH DE             kodpekaren
2AD2  EB                   EX   DE,HL          DE = höger
2AD3  21 05 00             LD   HL,$0005       5 byte att ta bort
2AD6  E5                   PUSH HL             sparas till slutet
2AD7  19                   ADD  HL,DE          HL = vänster operands exponent
2AD8  EB                   EX   DE,HL          DE = vänster, HL = höger
2AD9  42                   LD   B,D            BC = DE = vänster
2ADA  4B                   LD   C,E
2ADB  CD 91 2A             CALL JPIX           rutinen (IX); resultatet i vänster
2ADE  E1                   POP  HL             HL = 5
2ADF  D1                   POP  DE             kodpekaren tillbaka
2AE0  39                   ADD  HL,SP          ta bort höger operand
2AE1  F9                   LD   SP,HL          SP förbi höger operand
2AE2  F7                   RST  $30            resultatet ligger kvar, nästa token
```

Korsreferenstabell för flyttalens jämförelser och aritmetik. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. Kodrutinerna tar DE efter sin token och operanderna vid SP och lämnar resultatet där (stacken IU). Jämförelserna slutar med RST $28 (heltalet −1 eller 0 på stacken) och aritmetiken med RST $30; som i tabellen efter $2A4B räknas det som nästa rutin gör inte, men EXEXP förstör A, flaggorna, BC och HL. FCMP och FCMPST svarar i flaggorna: Z om lika, annars CY om det vänstra talet är mindre. FADD, FSUB, FMUL och FDIV bevarar BC, DE och HL och förstör A, flaggorna och IX. Felen: ADDF, SUBF, MULF och DIVF 4 (genom flyttalsrutinen, också för division med 0). Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
EQF	X	X	X	IU	X	X	IU
NEF	X	X	X	IU	X	X	IU
GTF	X	X	X	IU	X	X	IU

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
LEF	X	X	X	IU	X	X	IU
LTF	X	X	X	IU	X	X	IU
GEF	X	X	X	IU	X	X	IU
FCMPST	X	U	X		X	X	IU
FCMP	X	U	X	IX	IX		
JPIX						I	
ADDF	X	X	X	IU	X	X	IU
SUBF	X	X	X	IU	X	X	IU
MULF	X	X	X	IU	X	X	IU
DIVF	X	X	X	IU	X	X	IU
FARITH	X	X	X	IU	X	IX	IU

Rutin	**Anropar**	**Anropas från**
EQF	FCMPST, RST $28	EXPTAB1 ($E2)
NEF	FCMPST, RST $28	EXPTAB1 ($E5)
GTF	FCMPST, RST $28	EXPTAB1 ($EE), GEF (till $2A63, med JR)
LEF	FCMPST, RST $28	EXPTAB1 ($F1), LTF (till $2A6C, med JR)
LTF	FCMPST, LEF ($2A6C, med JR)	EXPTAB1 ($E8)
GEF	FCMPST, GTF ($2A63, med JR)	EXPTAB1 ($EB)
FCMPST	FCMP; går in i JPIX (tillbaka till anroparen)	EQF, NEF, GTF, LEF, LTF, GEF
FCMP		FORTSF, FCMPST
JPIX	rutinen i IX	FCMPST (går in i den), FARITH, SMULDV, FFUNC
ADDF	FARITH (med JR) med FADD	EXPTAB1 ($F4)
SUBF	FARITH (med JR) med FSUB	EXPTAB1 ($F7)
MULF	FARITH (med JR) med FMUL	EXPTAB1 ($F9)
DIVF	går in i FARITH med FDIV	EXPTAB1 ($FB)
FARITH	JPIX (rutinen i IX), RST $30	ADDF, SUBF och MULF (med JR), DIVF (går in i den), POWF (med JP)

CONSTI ($C7): *lägg heltalet ur koden på stacken.*

```
2AE3  EB          CONSTI: EX   DE,HL      HL = koden efter token
2AE4  CD 1D 10            CALL RDDE       heltalet ur koden
2AE7  EB                  EX   DE,HL      HL = heltalet, DE efter det
2AE8  EF                  RST  $28        heltalet på stacken, nästa token
```

NEGI ($D8): *byt tecken på heltalet. Ingen kontroll av -32768: -(-32768%) = -32768.*

```
2AE9  E1          NEGI:   POP  HL         heltalet
2AEA  CD EE 2A            CALL NEGHL      HL = -HL
2AED  EF                  RST  $28        på stacken, nästa token
```

NEGHL: *HL = -HL, räknat som 0 - HL en byte i taget. -32768 blir sig själv. Förstör A.*

```
2AEE  AF          NEGHL:  XOR  A          A = 0
2AEF  95                  SUB  L          A = -L, CY om L inte är 0
2AF0  6F                  LD   L,A        den låga byten klar
2AF1  9F                  SBC  A,A        A = -CY, lånet
2AF2  94                  SUB  H          A = -H - lånet
2AF3  67                  LD   H,A        den höga byten klar
```

```
2AF4  C9                  RET
```

3.75 Logiska operatorer, $DC–$E1

$2AF5

Bitvis på 16-bitars heltal (sant = -1, falskt = 0):

```
EQV: NOT (a XOR b)   IMP: (NOT a) OR b   OR, AND, XOR
NOT: lägger -1 på stacken och gör XOR.
```

Token: OPEQV $DC, OPIMP $DD, OPOR $DE, OPXOR $DF, OPAND $E0 och OPNOT $E1. Kompilatorn har redan gjort operanderna till heltal. Med vanliga tal är de bitoperationer: `6 AND 3` = 2, `5 EQV 3` = -7, `-5% IMP 3%` = 7.

In: DE efter token, a och b vid SP (b närmast). Ut: resultatet i deras ställe (`RST $28`). Förstör A, BC och HL.

```
2AF5  E1          OPEQV:  POP   HL        HL = b, den högra
2AF6  C1                  POP   BC        BC = a, den vänstra
2AF7  78                  LD    A,B       den höga byten
2AF8  AC                  XOR   H
2AF9  2F                  CPL             NOT (a XOR b)
2AFA  67                  LD    H,A
2AFB  79                  LD    A,C       den låga byten
2AFC  AD                  XOR   L
2AFD  2F                  CPL
2AFE  6F                  LD    L,A
2AFF  EF                  RST   $28       resultatet på stacken

2B00  E1          OPIMP:  POP   HL        HL = b, den högra
2B01  C1                  POP   BC        BC = a, den vänstra
2B02  78                  LD    A,B       den höga byten
2B03  2F                  CPL             NOT a
2B04  B4                  OR    H         OR b
2B05  67                  LD    H,A
2B06  79                  LD    A,C       den låga byten
2B07  2F                  CPL
2B08  B5                  OR    L
2B09  6F                  LD    L,A
2B0A  EF                  RST   $28       resultatet på stacken

2B0B  E1          OPOR:   POP   HL        HL = b, den högra
2B0C  C1                  POP   BC        BC = a, den vänstra
2B0D  79                  LD    A,C       den låga byten
2B0E  B5                  OR    L
2B0F  6F                  LD    L,A
2B10  78                  LD    A,B       den höga byten
2B11  B4                  OR    H
2B12  67                  LD    H,A
2B13  EF                  RST   $28       resultatet på stacken

2B14  E1          OPAND:  POP   HL        HL = b, den högra
2B15  C1                  POP   BC        BC = a, den vänstra
2B16  79                  LD    A,C       den låga byten
2B17  A5                  AND   L
2B18  6F                  LD    L,A
2B19  78                  LD    A,B       den höga byten
2B1A  A4                  AND   H
2B1B  67                  LD    H,A
2B1C  EF                  RST   $28       resultatet på stacken

2B1D  21 FF FF    OPNOT:  LD    HL,$FFFF  NOT x = x XOR -1
2B20  E5                  PUSH  HL        -1 som den högra operanden
```

```
2B21  E1          OPXOR:  POP   HL        HL = b, den högra
2B22  C1                  POP   BC        BC = a, den vänstra
2B23  79                  LD    A,C       den låga byten
2B24  AD                  XOR   L
2B25  6F                  LD    L,A
2B26  78                  LD    A,B       den höga byten
2B27  AC                  XOR   H
2B28  67                  LD    H,A
2B29  EF                  RST   $28       resultatet på stacken
```

3.76 Jämförelse av heltal, $E2–$F3 $2B2A

Skillnaden med `SBC HL,BC`, där CY = 0 från `EXEXP`. `EQI` och `NEI` prövar Z; `NEI` lägger vid lika skillnaden själv, 0, på stacken. För mindre och större avgör teckenflaggan, och vid spill (P/V) gäller den omvända, så att också `32767%>-1%` blir sant. Resultatet är -1 (sant) eller 0. `LEI` och `GTI` byter operanderna och använder samma kod som `GEI` och `LTI`.

Token: `EQI $E3`, `NEI $E6`, `LTI $E9`, `GEI $EC`, `GTI $EF` och `LEI $F2`.

In: DE efter token, a och b vid SP (b närmast). Ut: heltalet i deras ställe (`RST $28`). Förstör BC och HL.

```
2B2A  E1          EQI:    POP   HL        HL = b, den högra
2B2B  C1                  POP   BC        BC = a, den vänstra
2B2C  ED 42               SBC   HL,BC     b - a (CY = 0)
2B2E  28 0A               JR    Z,L2B3A   lika: sant
2B30  21 00 00    L2B30:  LD    HL,$0000  falskt
2B33  EF                  RST   $28       heltalet på stacken

2B34  E1          NEI:    POP   HL        HL = b, den högra
2B35  C1                  POP   BC        BC = a, den vänstra
2B36  ED 42               SBC   HL,BC     b - a (CY = 0)
2B38  28 03               JR    Z,L2B3D   lika: HL = 0 är falskt
2B3A  21 FF FF    L2B3A:  LD    HL,$FFFF  sant
2B3D  EF          L2B3D:  RST   $28       heltalet på stacken

2B3E  C1          LTI:    POP   BC        BC = b, den högra
2B3F  E1                  POP   HL        HL = a, den vänstra
2B40  ED 42       L2B40:  SBC   HL,BC     a - b; från GTI b - a
2B42  EA 51 2B            JP    PE,L2B51  spill: tecknet omvänt
2B45  F2 30 2B    L2B45:  JP    P,L2B30   inte negativt: falskt
2B48  18 F0               JR    L2B3A     negativt: sant

2B4A  C1          GEI:    POP   BC        BC = b, den högra
2B4B  E1                  POP   HL        HL = a, den vänstra
2B4C  ED 42       L2B4C:  SBC   HL,BC     a - b; från LEI b - a
2B4E  EA 45 2B            JP    PE,L2B45  spill: tecknet omvänt
2B51  FA 30 2B    L2B51:  JP    M,L2B30   negativt: falskt
2B54  18 E4               JR    L2B3A     inte negativt: sant

2B56  E1          LEI:    POP   HL        HL = b: operanderna byts
2B57  C1                  POP   BC        BC = a
2B58  18 F2               JR    L2B4C     b >= a, som GEI

2B5A  E1          GTI:    POP   HL        HL = b: operanderna byts
2B5B  C1                  POP   BC        BC = a
2B5C  18 E2               JR    L2B40     b < a, som LTI
```

`ADDI` / `SUBI`: heltalsaddition och -subtraktion utan kontroll av spill (resultatet blir modulo 65536: `32767%+1%` = -32768). Token `$F5` och `$F8`. `SUBI` kan använda `SBC`, eftersom CY = 0 från `EXEXP`.

In: DE efter token, a och b vid SP (b närmast). Ut: resultatet i deras ställe (`RST $28`). Förstör BC och HL.

```
2B5E  C1           ADDI:  POP  BC          BC = b, den högra
2B5F  E1                  POP  HL          HL = a, den vänstra
2B60  09                  ADD  HL,BC       inget spilltest
2B61  EF                  RST  $28         summan på stacken

2B62  C1           SUBI:  POP  BC          BC = b, den högra
2B63  E1                  POP  HL          HL = a, den vänstra
2B64  ED 42               SBC  HL,BC       a - b (CY = 0), inget spilltest
2B66  EF                  RST  $28         skillnaden på stacken
```

MULI / DIVI: *via* SMULDV *med* IMUL *resp.* IDIV. *Division med 0 ger fel 7, FÖR STORT HELTAL, och division avrundar mot 0. Beloppen multipliceras utan tecken, så fel 7 kommer först när produkten i belopp är över 65535; 32768-65535 slår runt (200%*200% = -25536, -128%*256% = -32768). Kvoten -32768 / -1 är 32768 i belopp, ryms och blir åter -32768 utan fel. Token* $FA *och* $FC.

*In: DE efter token, a och b vid SP (b närmast). Ut: resultatet i deras ställe (*RST $28*). Förstör A, BC, HL och IX.*

```
2B67  DD 21 74 37  MULI:   LD   IX,IMUL        (gränsfall)
2B6B  E1           IMULDV: POP  HL             HL = b, den högra
2B6C  EB                   EX   DE,HL          DE = b, HL = kodpekaren
2B6D  E3                   EX   (SP),HL        HL = a, kodpekaren vid SP
2B6E  CD 87 2B             CALL SMULDV         HL = a × b eller a / b
2B71  D1                   POP  DE             kodpekaren
2B72  EF                   RST  $28            resultatet på stacken

2B73  DD 21 88 37  DIVI:   LD   IX,IDIV
2B77  18 F2                JR   IMULDV         samma väg
```

EXSWAP: *SWAP%(x) byter plats på den höga och den låga byten:* SWAP%(1%) = *256,* SWAP%(-256%) = *255. Funktion med kod* $1A *i* EXPTAB2. *Förstör A och HL.*

```
2B79  E1           EXSWAP: POP  HL          x
2B7A  7C                   LD   A,H
2B7B  65                   LD   H,L         byt den höga och den låga byten
2B7C  6F                   LD   L,A
2B7D  EF                   RST  $28         resultatet på stacken
```

EXPEEK: PEEK(a) *läser byten på adressen a och ger den som ett heltal 0-255. Kod* $1B *i* EXPTAB2. *Förstör HL.*

```
2B7E  E1           EXPEEK: POP  HL          HL = adressen
2B7F  6E                   LD   L,(HL)      byten
2B80  26 00                LD   H,$00       som heltal 0-255
2B82  EF                   RST  $28         resultatet på stacken
```

SMUL / SMULDV: *multiplikation/division med tecken. HL = a, DE = b, IX =* IMUL *eller* IDIV *(utan tecken). Båda görs positiva, och resultatet byter tecken om a och b hade olika tecken. CY från rutinen (resultat över 65535, eller division med 0) ger fel 7; 16-bitarsspill prövas inte.* SMUL *anropas av potensen med heltal (*$2E2B, $2E38*).*

Ut: HL = resultatet. Förstör A, BC och DE (och det som IMUL *eller* IDIV *förstör).*

```
2B83  DD 21 74 37  SMUL:   LD   IX,IMUL
2B87  06 02        SMULDV: LD   B,$02          två varv
2B89  EB           L2B89:  EX   DE,HL          varv 1: HL = b; varv 2: HL = a
2B8A  E5                   PUSH HL             talet med sitt tecken, b först
2B8B  7C                   LD   A,H            den höga byten
2B8C  A7                   AND  A              S = teckenbiten
2B8D  FC EE 2A             CALL M,NEGHL        negativt: gör positivt
2B90  10 F7                DJNZ L2B89          för a och b
2B92  CD 91 2A             CALL JPIX           beloppen: IMUL eller IDIV
2B95  DA 07 07             JP   C,L0707        fel 7
```

```
2B98  C1           POP  BC        BC = a, B = a:s höga byte
2B99  F1           POP  AF        A = b:s höga byte (F = den låga)
2B9A  A8           XOR  B         S om tecknen är olika
2B9B  FC EE 2A     CALL M,NEGHL   olika: byt tecken
2B9E  C9           RET            HL = resultatet
```

Korsreferenstabell för heltalens operatorer. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. Kodrutinerna tar DE efter sin token och operanderna vid SP och lägger resultatet i deras ställe med `RST $28`; som i tabellen efter `$2A4B` räknas det som nästa rutin gör inte, men `EXEXP` förstör A, flaggorna, BC och HL. Jämförelserna ger −1 (sant) eller 0. `EXSWAP` och `EXPEEK` är funktionerna SWAP% och PEEK och nås genom `EXFUNC`. `IMUL` förstör A, BC och DE, och `IDIV` A och BC och lämnar resten i DE. Felen: `MULI`, `DIVI`, `IMULDV`, `SMUL` och `SMULDV` 7 (produkten över 65535 i belopp eller division med 0). Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
CONSTI	X	X	X	IU	X		U
NEGI	X	X	X	IU	X		IU
NEGHL	X	X			IU		
OPEQV	X	X	X	IU	X		IU
OPIMP	X	X	X	IU	X		IU
OPOR	X	X	X	IU	X		IU
OPAND	X	X	X	IU	X		IU
OPNOT	X	X	X	IU	X		IU
OPXOR	X	X	X	IU	X		IU
EQI	X	X	X	IU	X		IU
NEI	X	X	X	IU	X		IU
LTI	X	X	X	IU	X		IU
GEI	X	X	X	IU	X		IU
LEI	X	X	X	IU	X		IU
GTI	X	X	X	IU	X		IU
ADDI	X	X	X	IU	X		IU
SUBI	X	X	X	IU	X		IU
MULI	X	X	X	IU	X	X	IU
IMULDV	X	X	X	IU	X	IX	IU
DIVI	X	X	X	IU	X	X	IU
EXSWAP	X	X	X	IU	X		IU
EXPEEK	X	X	X	IU	X		IU
SMUL	X	X	X	IX	IU	X	
SMULDV	X	X	X	IX	IU	IX	

Rutin	**Anropar**	**Anropas från**
CONSTI	RDDE, `RST $28`	EXPTAB1 (`$C7`)
NEGI	NEGHL, `RST $28`	EXPTAB1 (`$D8`)
NEGHL		GETSIN, PRSIN, NEGI, SMULDV (två ställen), FTISGN, INTTOF
OPEQV	`RST $28`	EXPTAB1 (`$DC`)
OPIMP	`RST $28`	EXPTAB1 (`$DD`)
OPOR	`RST $28`	EXPTAB1 (`$DE`)
OPAND	`RST $28`	EXPTAB1 (`$E0`)
OPNOT	går in i OPXOR med −1 som höger operand	EXPTAB1 (`$E1`)
OPXOR	`RST $28`	EXPTAB1 (`$DF`), OPNOT (går in i den)

Rutin	Anropar	Anropas från
EQI	RST $28	EXPTAB1 ($E3)
NEI	RST $28	EXPTAB1 ($E6)
LTI	RST $28	EXPTAB1 ($E9), GTI (till $2B40, med JR)
GEI	RST $28	EXPTAB1 ($EC), LEI (till $2B4C, med JR)
LEI	GEI ($2B4C, med JR)	EXPTAB1 ($F2)
GTI	LTI ($2B40, med JR)	EXPTAB1 ($EF)
ADDI	RST $28	EXPTAB1 ($F5)
SUBI	RST $28	EXPTAB1 ($F8)
MULI	går in i IMULDV med IMUL	EXPTAB1 ($FA)
IMULDV	SMULDV, RST $28	MULI (går in i den), DIVI (med JR)
DIVI	IMULDV (med JR) med IDIV	EXPTAB1 ($FC)
EXSWAP	RST $28	EXPTAB2 (kod $1A)
EXPEEK	RST $28	EXPTAB2 (kod $1B)
SMUL	går in i SMULDV med IMUL	POWI (två ställen)
SMULDV	NEGHL, JPIX (IMUL eller IDIV); fel 7	SMUL (går in i den), IMULDV

3.77 ADD$ SUB$ MUL$ DIV$ COMP%: aritmetik på tal skrivna som text

$2B9F

ADD$(a$,b$,n) och så vidare ger en sträng med n decimaler; COMP%(a$,b$) ger -1, 0 eller 1. För COMP% läggs n = 0 på stacken. BC = rutinen: AADD, ASUB, AMUL, ADIV (från $37A6) eller ASCCMP.

1. ROOM: det ska finnas 185 + 100 byte ledigt över HEAP.
2. n till ASDEC, DE (koden) till ASCODE. 87 byte buffert på stacken.
3. ASCARG kopierar b$ (vid SP) till bufferten och sedan a$, som fylls ut till n+1 decimaler och högerjusteras med mellanslag (ASARG1 = a$ i bufferten). Varje kopia slutar med $83. Tal längre än 29 tecken ger fel 65, SPILL I ASCII-ARITMETIK, och en tom sträng fel 66, STRÄNG EJ NUMERISK. Utfylld får a$ högst 29 tecken (fel 65): med punkt 28 positioner, utan punkt 29, eftersom punkten läggs till. Resultatets heltalsdel får därför högst 27 - n siffror om a$ har en punkt och 28 - n om inte; ADD$("1",b$,0) med 27 nior ger 28 siffror, med "1.0" fel 65. Slutar a$ med punkten ("12.") läggs en punkt till och talet ger fel 66.
4. Rutinen anropas med HL = a$ (ASARG1) och DE = b$. Resultatet hamnar i a$:s plats, med a$:s bredd och n+1 decimaler.
5. n över 26 ger fel 66. Under ASEND, där argumenten började, skrivs 29 nollor med en punkt före de n sista, och $83. Resultatet adderas till det med AADD, som avrundar till n decimaler (exakt hälft uppåt för positiva, nedåt i belopp för negativa resultat: DIV$("-7","2",0) ger -3). Se $37A6. Har b$ fler än n+1 decimaler avrundas det alltså två gånger, först till n+1 i rutinen och sedan till n: ADD$("1","0.049",1) ger 1.1.
6. Inledande mellanslag hoppas över, och texten blir en tillfällig sträng på stacken: [längd+1] [adress] [längd] [text $83].

CY från en rutin: fel 66 om bit 3 i A är satt, annars fel 65.

In: DE = koden; vid SP n (inte för COMP%), b$ och a$ som strängar på stacken. Ut: resultatet på stacken i stället för argumenten, sträng eller heltal, och körningen fortsätter med koden (RST30). Förstör alla register utom IY.

```
2B9F  01 A6 37     EXADD:  LD   BC,AADD          ADD$: a + b
2BA2  18 16                JR   ASCAR
```

```
2BA4  01 AE 37     EXSUB:  LD   BC,ASUB        SUB$: a - b
2BA7  18 11                JR   ASCAR

2BA9  01 B8 38     EXMUL:  LD   BC,AMUL        MUL$: a × b
2BAC  18 0C                JR   ASCAR

2BAE  01 3B 38     EXDIV:  LD   BC,ADIV        DIV$: a / b
2BB1  18 07                JR   ASCAR

2BB3  01 51 2C     EXCOMP: LD   BC,ASCCMP      COMP%: jämför
2BB6  21 00 00             LD   HL,$0000       n = 0
2BB9  E5                   PUSH HL
2BBA  21 47 FF     ASCAR:  LD   HL,$FF47       185 byte
2BBD  39                   ADD  HL,SP
2BBE  CD 9B 33             CALL ROOM           + 100 över HEAP?
2BC1  E1                   POP  HL             n
2BC2  22 EB FD             LD   (ASDEC),HL
2BC5  ED 53 ED FD          LD   (ASCODE),DE
2BC9  21 A9 FF             LD   HL,$FFA9       87 byte buffert
2BCC  39                   ADD  HL,SP
2BCD  F9                   LD   SP,HL
2BCE  E5                   PUSH HL             buffertens början
2BCF  EB                   EX   DE,HL          DE = bufferten
2BD0  21 57 00             LD   HL,$0057       b$:s beskrivning
2BD3  19                   ADD  HL,DE
2BD4  C5                   PUSH BC             rutinen
2BD5  AF                   XOR  A              Z: kopiera b$
2BD6  CD 6C 2C             CALL ASCARG
2BD9  D5                   PUSH DE             a$:s början i bufferten
2BDA  B2                   OR   D              NZ: fyll ut a$
2BDB  CD 6C 2C             CALL ASCARG
2BDE  22 E9 FD             LD   (ASEND),HL     där argumenten började
2BE1  E1                   POP  HL
2BE2  22 E7 FD             LD   (ASARG1),HL
2BE5  C1                   POP  BC             rutinen
2BE6  D1                   POP  DE             b$
2BE7  FD E5                PUSH IY
2BE9  CD 47 2C             CALL CALLBC         kör rutinen
2BEC  FD E1                POP  IY
2BEE  38 5B                JR   C,ASCERR
2BF0  2A EB FD             LD   HL,(ASDEC)     n
2BF3  7C                   LD   A,H
2BF4  A7                   AND  A
2BF5  20 52                JR   NZ,ERR66
2BF7  7D                   LD   A,L
2BF8  FE 1B                CP   $1B
2BFA  30 4D                JR   NC,ERR66       n > 26: fel 66
2BFC  2A E9 FD             LD   HL,(ASEND)
2BFF  E5                   PUSH HL
2C00  2B                   DEC  HL
2C01  36 83                LD   (HL),$83       slutmarkering
2C03  2B                   DEC  HL
2C04  36 30                LD   (HL),$30
2C06  5D                   LD   E,L
2C07  54                   LD   D,H
2C08  1B                   DEC  DE             DE = HL - 1
2C09  01 1C 00             LD   BC,$001C       28 till: 29 nollor
2C0C  ED B8                LDDR                fyll med nollor
2C0E  E3                   EX   (SP),HL        HL = ASEND, fältets början vid SP
2C0F  ED 5B E7 FD          LD   DE,(ASARG1)    DE = resultatet (a$ i bufferten)
2C13  2B                   DEC  HL             HL = sista nollan
2C14  2B                   DEC  HL
```

```
2C15  ED 4B EB FD          LD   BC,(ASDEC)
2C19  78                   LD   A,B
2C1A  B1                   OR   C
2C1B  28 04                JR   Z,L2C21          n = 0: ingen punkt
2C1D  ED 42                SBC  HL,BC            n decimaler från slutet
2C1F  36 2E                LD   (HL),$2E         punkt före n decimaler
2C21  E1           L2C21:  POP  HL               fältets början
2C22  FD E5                PUSH IY
2C24  CD A6 37             CALL AADD             addera resultatet
2C27  FD E1                POP  IY
2C29  38 20                JR   C,ASCERR
2C2B  2A E9 FD             LD   HL,(ASEND)
2C2E  2B                   DEC  HL
2C2F  01 1D 00             LD   BC,$001D         29 positioner, HL = fältets början
2C32  ED 42                SBC  HL,BC
2C34  7E           L2C34:  LD   A,(HL)           hoppa över mellanslag
2C35  FE 20                CP   $20
2C37  20 04                JR   NZ,L2C3D
2C39  23                   INC  HL
2C3A  0B                   DEC  BC               längden minskar
2C3B  18 F7                JR   L2C34

2C3D  F9           L2C3D:  LD   SP,HL            texten ligger på stacken
2C3E  C5                   PUSH BC               längd
2C3F  E5                   PUSH HL               adress
2C40  03                   INC  BC               n = längd + 1 ($83)
2C41  C5           L2C41:  PUSH BC
2C42  ED 5B ED FD          LD   DE,(ASCODE)      DE = koden
2C46  F7                   RST  $30              fortsätt med koden (RST30)
```

CALLBC: *anropa rutinen i BC.*

```
2C47  C5           CALLBC: PUSH BC
2C48  C9                   RET
```

ERR66 och ERR65: felen 66 och 65. ASCERR: CY från rutinen; bit 3 i A (A = 8, inte ett tal) ger fel 66, annars fel 65. Kommer inte tillbaka.

```
2C49  D7           ERR66:  RST  $10
2C4A  C2                   DEFB $C2              fel 66: STRÄNG EJ NUMERISK

2C4B  CB 5F        ASCERR: BIT  3,A              bit 3: ej numerisk
2C4D  20 FA                JR   NZ,ERR66
2C4F  D7           ERR65:  RST  $10
2C50  C1                   DEFB $C1              fel 65: SPILL I ASCII-ARITMETIK
```

ASCCMP: COMP%. *Returadressen från* CALLBC *tas bort, och* ACOMP *jämför. A bit 6 = 0: lika (0). Annars bit 7 = 1 ger -1, bit 7 = 0 ger 1.*

```
2C51  F1           ASCCMP: POP  AF               ta bort returadressen
2C52  CD F6 37             CALL ACOMP
2C55  FD E1                POP  IY               IY från $2BE7
2C57  38 F2                JR   C,ASCERR
2C59  2A E9 FD             LD   HL,(ASEND)       ta bort argumenten
2C5C  F9                   LD   SP,HL
2C5D  01 00 00             LD   BC,$0000         BC = 0
2C60  CB 77                BIT  6,A              lika: 0
2C62  28 DD                JR   Z,L2C41
2C64  87                   ADD  A,A              bit 7
2C65  38 02                JR   C,L2C69
2C67  03                   INC  BC               +1
2C68  03                   INC  BC
```

```
2C69  0B          L2C69:  DEC  BC            -1
2C6A  18 D5               JR   L2C41
```

3.78 ASCARG: kopiera ett strängargument till bufferten (DE) $2C6C

In: HL = strängbeskrivningen på stacken, Z = kopiera som den är (b$), NZ = fyll ut (a$).

Ut: HL = nästa beskrivning, DE efter kopian och $83.

Utfyllnad: k = antal tecken efter ".", eller 0 om punkt saknas. Det behövs z = n+1-k nollor (0 om negativt). Först 29-längd-z mellanslag, sedan texten, sedan "." om punkt saknas, och z nollor.

```
2C6C  08          ASCARG: EX   AF,AF'         (fel)
2C6D  3E 03               LD   A,$03          läs n, adress, längd
2C6F  CD 17 10    L2C6F:  CALL RDBC
2C72  C5                  PUSH BC
2C73  3D                  DEC  A              tre ord
2C74  20 F9               JR   NZ,L2C6F
2C76  C1                  POP  BC             längd
2C77  E5                  PUSH HL
2C78  21 1D 00            LD   HL,$001D       högst 29 tecken
2C7B  AF                  XOR  A
2C7C  ED 42               SBC  HL,BC
2C7E  38 CF               JR   C,ERR65        fel 65
2C80  B1                  OR   C
2C81  28 C6               JR   Z,ERR66        tom: fel 66
2C83  7D                  LD   A,L            A = 29 - längd
2C84  E1                  POP  HL             HL efter beskrivningen
2C85  E3                  EX   (SP),HL        HL = texten
2C86  08                  EX   AF,AF'
2C87  28 22               JR   Z,L2CAB        b$: ingen utfyllnad
2C89  08                  EX   AF,AF'         A = 29 - längd
2C8A  F5                  PUSH AF
2C8B  C5                  PUSH BC
2C8C  E5                  PUSH HL
2C8D  3E 2E               LD   A,$2E
2C8F  ED B1               CPIR                sök .
2C91  3A EB FD            LD   A,(ASDEC)      n + 1 - decimaler
2C94  3C                  INC  A
2C95  91                  SUB  C
2C96  E1                  POP  HL
2C97  C1                  POP  BC
2C98  30 01               JR   NC,L2C9B       negativt: 0
2C9A  AF                  XOR  A
2C9B  47          L2C9B:  LD   B,A            B = z nollor
2C9C  F1                  POP  AF
2C9D  90                  SUB  B              mellanslag = 29 - längd - nollor
2C9E  38 AF               JR   C,ERR65        fel 65
2CA0  28 09               JR   Z,L2CAB
2CA2  C5                  PUSH BC
2CA3  47                  LD   B,A
2CA4  3E 20               LD   A,$20
2CA6  12          L2CA6:  LD   (DE),A         mellanslag
2CA7  13                  INC  DE
2CA8  10 FC               DJNZ L2CA6
2CAA  C1                  POP  BC
2CAB  78          L2CAB:  LD   A,B            A = z
2CAC  06 00               LD   B,$00          BC = längden
2CAE  ED B0               LDIR                kopiera texten
2CB0  A7                  AND  A              z = 0: inga nollor
```

```
2CB1  28 12              JR   Z,L2CC5
2CB3  47                 LD   B,A            B = z
2CB4  3A EB FD           LD   A,(ASDEC)
2CB7  3C                 INC  A
2CB8  B8                 CP   B              z = n+1: ingen punkt (eller punkten sist)
2CB9  20 04              JR   NZ,L2CBF
2CBB  3E 2E              LD   A,$2E
2CBD  12                 LD   (DE),A         .
2CBE  13                 INC  DE
2CBF  3E 30        L2CBF: LD   A,$30
2CC1  12           L2CC1: LD   (DE),A        nollor
2CC2  13                 INC  DE
2CC3  10 FC              DJNZ L2CC1
2CC5  3E 83        L2CC5: LD   A,$83         slutmarkering
2CC7  12                 LD   (DE),A
2CC8  13                 INC  DE
2CC9  E1                 POP  HL             HL efter beskrivningen
2CCA  C1                 POP  BC             BC = längd+1
2CCB  09                 ADD  HL,BC          nästa beskrivning
2CCC  C9                 RET
```

Korsreferenstabell för ADD$, SUB$, MUL$, DIV$ och COMP%. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; $FDE7 är ASARG1 (a$ i bufferten), $FDE9 ASEND (där argumenten började), $FDEB ASDEC (n) och $FDED ASCODE (koden). Funktionerna nås genom EXPTAB2, tar DE efter sin kod och argumenten vid SP och lägger resultatet i deras ställe, med RST $28 (ett heltal) eller RST $30; som i tabellen efter $2A4B räknas det som nästa rutin gör inte, men EXEXP förstör A, flaggorna, BC och HL. EXADD till EXDIV lägger rutinen i BC och går till ASCAR, EXCOMP lägger också n = 0 på stacken. ASCAR sparar IY runt rutinen, som också använder $FDD0–$FDD6 (se tabellen efter $3B76). ASCCMP tar bort returadressen från CALLBC och går direkt vidare till nästa token. ASCARG tar Z = kopiera som den är (b$) och NZ = fyll ut (a$), läser ASDEC och använder A'. Felen: ASCAR 3 (genom ROOM), 65 och 66; ASCARG 65 och 66. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack									variabler				
Rutin	A	CY	BC	DE	HL	IX	IY	skugg.	stack	HEAP	$FDE7	$FDE9	$FDEB	$FDED
EXADD	X	X	X	IU	X	X		X	IU	I	X	X	X	X
EXSUB	X	X	X	IU	X	X		X	IU	I	X	X	X	X
EXMUL	X	X	X	IU	X	X		X	IU	I	X	X	X	X
EXDIV	X	X	X	IU	X	X		X	IU	I	X	X	X	X
EXCOMP	X	X	X	IU	X	X		X	IU	I	X	X	X	X
ASCAR	X	X	IX	IU	X	X		X	IU	I	X	X	X	X
CALLBC			I											
ERR66*														
ASCERR*	I													
ERR65*														
ASCCMP	X	X	X	U	X	X		X	IU			I		I
ASCARG	IX	X	X	IU	IU			X					I	

Rutin	**Anropar**	**Anropas från**
EXADD	ASCAR (med JR), rutinen AADD	EXPTAB2 (kod $30)
EXSUB	ASCAR (med JR), rutinen ASUB	EXPTAB2 (kod $31)
EXMUL	ASCAR (med JR), rutinen AMUL	EXPTAB2 (kod $32)
EXDIV	ASCAR (med JR), rutinen ADIV	EXPTAB2 (kod $33)

Rutin	Anropar	Anropas från
EXCOMP	går in i ASCAR, rutinen ASCCMP	EXPTAB2 (kod $34)
ASCAR	ROOM, ASCARG, CALLBC (rutinen), AADD, ASCERR och ERR66 (med JR), RST $30; fel 3, 65 och 66	EXADD, EXSUB, EXMUL och EXDIV (med JR), EXCOMP (går in i den)
CALLBC	rutinen i BC	ASCAR ($2BE9)
ERR66	RST $10; fel 66	ASCAR (två ställen), ASCERR, ASCARG (alla med JR)
ASCERR	ERR66 (med JR); går in i ERR65	ASCAR (två ställen), ASCCMP (alla med JR)
ERR65	RST $10; fel 65	ASCERR (går in i den), ASCARG (två ställen, med JR)
ASCCMP	ACOMP, ASCERR (med JR), RST $30 (vid $2C46)	EXCOMP (genom CALLBC)
ASCARG	RDBC, ERR65 och ERR66 (med JR)	ASCAR (två ställen)

3.79 VAR0 ($C0–$C2): variabel utan index $2CCD

Koden: token, namnbyte, bokstav, adress (2 byte). Adressen läggs på stacken. Är adressen under $100 är det en DEF FN-parameter, och FNPAR letar upp argumentet genom ramen FNFRAM.

Adressen pekar för ett flyttal på exponenten, för ett heltal på den låga byten och för en sträng på posten. Den följande token avgör om värdet hämtas ($D4-$D6) eller om adressen står kvar för en tilldelning.

In: DE efter token. Ut: adressen på stacken (RST $28), DE efter variabeln. Förstör A och HL, för en parameter också BC och IX.

```
2CCD  EB          VAR0:  EX   DE,HL      HL = koden efter token
2CCE  23                 INC  HL         hoppa över namnbyten
2CCF  CD 1C 10           CALL RDDE1      och bokstaven: DE = adressen
2CD2  EB                 EX   DE,HL      HL = adressen, DE efter variabeln
2CD3  7C                 LD   A,H        hög byte 0?
2CD4  A7                 AND  A
2CD5  CC AB 33           CALL Z,FNPAR    < $100: DEF FN-parameter
2CD8  EF                 RST  $28        adressen på stacken, nästa token
```

3.80 VAR1x / VAR2x: element i vektor eller matris $2CD9

BC = elementets storlek (C) och offset till värdet (B): flyttal 5/4 (exponenten), heltal 2/0, sträng 6/0. De skrivs in i posten (+4, +5). En variabel som inte är dimensionerad (+2/3 = 0) dimensioneras med 10 som högsta index (DIM1 / DIM2).

Index utanför ger fel 5, FÖR STORT INDEX. Ett negativt index blir ett stort tal utan tecken och ger också fel 5.

Adressen = +0/1 + elementnummer × storlek + offset. För en matris är elementnumret i × (max2 + 1) + j.

En matris prövar båda indexen var för sig, så D(1,4) ger fel 5 med DIM D(2,3), fast elementnumret 8 skulle rymmas.

Token: VAR1F $C4, VAR1I $C5, VAR1S $C6, VAR2F $C8, VAR2I $C9 och VAR2S $CA.

In: DE efter token, indexen vid SP som heltal (för en matris j närmast och i ovanför). Ut: elementets adress i deras ställe (RST $28), DE efter variabeln. Förstör A, BC, HL och IX, vid den första användningen också det som DIM1 eller DIM2 förstör.

```
2CD9  06 04        VAR1F:  LD   B,$04          offset 4: exponenten
2CDB  0E 05                LD   C,$05          storlek 5
2CDD  18 08                JR   VAR1

2CDF  01 02 00     VAR1I:  LD   BC,$0002       storlek 2, offset 0
2CE2  18 03                JR   VAR1

2CE4  01 06 00     VAR1S:  LD   BC,$0006       storlek 6, offset 0
2CE7  CD 11 2D     VAR1:   CALL VARIDX         IX = posten; Z: odimensionerad
2CEA  CC E6 32             CALL Z,DIM1         odimensionerad: DIM (10)
2CED  D1                   POP  DE             kodpekaren
2CEE  C1                   POP  BC             index
2CEF  DD 6E 02             LD   L,(IX+$02)     högsta index
2CF2  DD 66 03             LD   H,(IX+$03)
2CF5  A7                   AND  A              CY = 0
2CF6  ED 42                SBC  HL,BC          högsta index - index
2CF8  38 55                JR   C,ERRIDX       för stort: fel 5
2CFA  DD 7E 04     ELMADR: LD   A,(IX+$04)     A = storleken
2CFD  DD 6E 00             LD   L,(IX+$00)     elementens början
2D00  DD 66 01             LD   H,(IX+$01)
2D03  09           L2D03:  ADD  HL,BC          + index, storleken gånger
2D04  3D                   DEC  A
2D05  20 FC                JR   NZ,L2D03
2D07  DD B6 05             OR   (IX+$05)       A = 0, så A = offset
2D0A  28 04                JR   Z,L2D10        offset 0: klart
2D0C  4F                   LD   C,A            BC = offset
2D0D  06 00                LD   B,$00
2D0F  09                   ADD  HL,BC          + offset
2D10  EF           L2D10:  RST  $28            adressen på stacken, nästa token
```

VARIDX: läs postens adress ur koden, sätt IX på den och skriv in storleken (C) och offset (B) i +4 och +5. Det görs vid varje användning, så att DIM1 har storleken när fältet dimensioneras första gången. Kodpekaren läggs ovanför returadressen och är efter RET den som anroparen hämtar först.

In: DE efter token, BC = offset och storlek. Ut: IX = posten, DE = 10 (det förvalda högsta indexet) och Z om fältet inte är dimensionerat (+2/3 = 0). Förstör A och HL.

```
2D11  EB           VARIDX: EX   DE,HL          HL = koden efter token
2D12  23                   INC  HL             hoppa över namnbyten
2D13  CD 1C 10             CALL RDDE1          och bokstaven: DE = posten
2D16  D5                   PUSH DE             IX = posten
2D17  DD E1                POP  IX
2D19  E3                   EX   (SP),HL        HL = returadressen, kodpekaren vid SP
2D1A  E5                   PUSH HL             returadressen vid SP, kodpekaren ovanför
2D1B  11 0A 00             LD   DE,$000A       förvalt högsta index
2D1E  DD 71 04             LD   (IX+$04),C     storlek
2D21  DD 70 05             LD   (IX+$05),B     offset
2D24  DD 7E 02             LD   A,(IX+$02)     dimensionerad?
2D27  DD B6 03             OR   (IX+$03)
2D2A  C9                   RET                 Z: inte dimensionerad

2D2B  06 04        VAR2F:  LD   B,$04          offset 4: exponenten
2D2D  0E 05                LD   C,$05          storlek 5
2D2F  18 08                JR   VAR2

2D31  01 02 00     VAR2I:  LD   BC,$0002       storlek 2, offset 0
2D34  18 03                JR   VAR2

2D36  01 06 00     VAR2S:  LD   BC,$0006       storlek 6, offset 0
```

```
2D39  CD 11 2D     VAR2:   CALL VARIDX          IX = posten; Z: odimensionerad
2D3C  6B                   LD   L,E             HL = DE = 10
2D3D  62                   LD   H,D
2D3E  CC 11 33             CALL Z,DIM2          odimensionerad: DIM (10,10)
2D41  C1                   POP  BC              kodpekaren
2D42  E1                   POP  HL              j
2D43  D1                   POP  DE              i
2D44  C5                   PUSH BC              kodpekaren tillbaka
2D45  E5                   PUSH HL              j
2D46  DD 6E 06             LD   L,(IX+$06)      första indexets max
2D49  DD 66 07             LD   H,(IX+$07)
2D4C  A7                   AND  A               CY = 0
2D4D  ED 52                SBC  HL,DE           första indexets max - i
2D4F  DA 03 07     ERRIDX: JP   C,L0703         för stort index: fel 5
2D52  DD 6E 08             LD   L,(IX+$08)      andra indexets max
2D55  DD 66 09             LD   H,(IX+$09)
2D58  E5                   PUSH HL              andra indexets max sparas
2D59  23                   INC  HL              + 1: elementen i en rad
2D5A  CD 74 37             CALL IMUL            i * (max2 + 1)
2D5D  38 F0                JR   C,ERRIDX        spill: fel 5
2D5F  EB                   EX   DE,HL           DE = i × (max2 + 1)
2D60  E1                   POP  HL              HL = andra indexets max
2D61  C1                   POP  BC              BC = j
2D62  ED 42                SBC  HL,BC           (CY = 0) j för stort?
2D64  38 E9                JR   C,ERRIDX
2D66  60                   LD   H,B             HL = j
2D67  69                   LD   L,C
2D68  19                   ADD  HL,DE           + j
2D69  44                   LD   B,H             BC = elementets nummer
2D6A  4D                   LD   C,L
2D6B  D1                   POP  DE              kodpekaren
2D6C  18 8C                JR   ELMADR          adressen som för en vektor
```

EXPI: PI (*kod 9*): *gör plats för 5 byte på stacken och kopiera dit flyttalet* FPI. *In: DE efter koden. Ut: flyttalet vid SP, DE orörd. Förstör BC och HL.*

```
2D6E  21 FB FF     EXPI:   LD   HL,$FFFB        plats för 5 byte
2D71  39                   ADD  HL,SP
2D72  F9                   LD   SP,HL           SP = platsen
2D73  D5                   PUSH DE              kodpekaren
2D74  EB                   EX   DE,HL           DE = platsen
2D75  21 A6 3F             LD   HL,FPI          PI
2D78  01 05 00             LD   BC,$0005        5 byte
2D7B  ED B0                LDIR
2D7D  D1                   POP  DE              kodpekaren tillbaka
2D7E  F7                   RST  $30             flyttalet ligger kvar, nästa token
```

EXABS: ABS(x) (*kod 1*) *sätter tecknets byte i flyttalet vid SP till 0, plus. Förstör HL.*

```
2D7F  21 03 00     EXABS:  LD   HL,$0003        tecknet i SP+3
2D82  39                   ADD  HL,SP
2D83  36 00                LD   (HL),$00        tecknet = +
2D85  F7                   RST  $30             nästa token
```

EXSGN: SGN(x) (*kod* $0B) *ger ett heltal: 0 om exponenten är 0, annars -1 eller 1 efter tecknet. Därför ger också -0 värdet 0. Flyttalet tas bort och heltalet läggs i dess ställe* (RST $28). *Förstör A, BC och HL.*

```
2D86  C1           EXSGN:  POP  BC              två av mantissans tre byte
2D87  33                   INC  SP              den tredje
2D88  C1                   POP  BC              C = tecken, B = exponent
2D89  AF                   XOR  A               A = 0
```

```
2D8A  6F              LD   L,A          HL = 0
2D8B  67              LD   H,A
2D8C  B0              OR   B            exponenten 0: talet är 0
2D8D  28 06           JR   Z,L2D95      SGN = 0
2D8F  2B              DEC  HL           -1
2D90  0D              DEC  C            tecknet 1: negativt
2D91  28 02           JR   Z,L2D95      negativt: -1
2D93  23              INC  HL           +1
2D94  23              INC  HL
2D95  EF        L2D95: RST  $28         heltalet på stacken, nästa token
```

Korsreferenstabell för variablerna, fältelementen, PI, ABS och SGN. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna. Kodrutinerna tar DE efter sin token och lägger variabelns eller elementets adress på stacken med `RST $28`, i indexens ställe; `EXPI`, `EXABS` och `EXSGN` är funktionerna PI, ABS och SGN och nås genom `EXFUNC`. Som i tabellen efter `$2A4B` räknas det som nästa rutin gör inte, men `EXEXP` förstör A, flaggorna, BC och HL. `VAR0` slår upp en DEF FN-parameter med `FNPAR` genom `FNFRAM`. Ett fält som inte är dimensionerat får 10 som högsta index med `DIM1` eller `DIM2` vid den första användningen, som reserverar elementen på heapen. `VARIDX` svarar i flaggorna: Z om fältet inte är dimensionerat. Felen: `VAR1` och `VAR2` 5 (genom `ERRIDX`), och vid den första användningen 3. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler	
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	FNFRAM
VAR0	X	X	X	IU	X	X	U		I
VAR1F	X	X	X	IU	X	X	IU	IU	
VAR1I	X	X	X	IU	X	X	IU	IU	
VAR1S	X	X	X	IU	X	X	IU	IU	
VAR1	X	X	IX	IU	X	X	IU	IU	
ELMADR	X	X	IX	IU	X	I	U		
VARIDX	X	U	I	IU	X	U	IU		
VAR2F	X	X	X	IU	X	X	IU	IU	
VAR2I	X	X	X	IU	X	X	IU	IU	
VAR2S	X	X	X	IU	X	X	IU	IU	
VAR2	X	X	IX	IU	X	X	IU	IU	
EXPI	X	X	X	IU	X		U		
EXABS	X	X	X	IU	X		IU		
EXSGN	X	X	X	IU	X		IU		

Rutin	**Anropar**	**Anropas från**
VAR0	RDDE1, FNPAR, `RST $28`	EXPTAB1 (`$C0–$C2`)
VAR1F	VAR1 (med JR)	EXPTAB1 (`$C4`)
VAR1I	VAR1 (med JR)	EXPTAB1 (`$C5`)
VAR1S	går in i VAR1	EXPTAB1 (`$C6`)
VAR1	VARIDX, DIM1; går in i ELMADR; fel 5	VAR1F och VAR1I (med JR), VAR1S (går in i den)
ELMADR	`RST $28`	VAR1 (går in i den), VAR2 (med JR)
VARIDX	RDDE1	VAR1, VAR2
VAR2F	VAR2 (med JR)	EXPTAB1 (`$C8`)
VAR2I	VAR2 (med JR)	EXPTAB1 (`$C9`)
VAR2S	går in i VAR2	EXPTAB1 (`$CA`)
VAR2	VARIDX, DIM2, IMUL, ELMADR (med JR); fel 5	VAR2F och VAR2I (med JR), VAR2S (går in i den)
EXPI	`RST $30`	EXPTAB2 (kod 9)

Rutin	Anropar	Anropas från
EXABS	RST $30	EXPTAB2 (kod 1)
EXSGN	RST $28	EXPTAB2 (kod $0B)

3.81 Flyttalsfunktioner med ett argument $2D96

BC = rutinen, som FFUNC anropar med HL = BC = argumentets exponent. Resultatet ersätter argumentet på stacken.

```
      FIX FFIX   INT FINT   SIN FSIN   COS FCOS   TAN FTAN
      ATN FATN   EXP FEXP   LOG FLOG   LOG10 FLOG10   SQR FSQR

2D96  01 D5 2D     EXFIX:  LD   BC,FFIX          BC = rutinen för FIX
2D99  18 2B                JR   FFUNC

2D9B  01 7F 3C     EXINT:  LD   BC,FINT          BC = rutinen för INT
2D9E  18 26                JR   FFUNC

2DA0  01 77 3D     EXSIN:  LD   BC,FSIN          BC = rutinen för SIN
2DA3  18 21                JR   FFUNC

2DA5  01 70 3D     EXCOS:  LD   BC,FCOS          BC = rutinen för COS
2DA8  18 1C                JR   FFUNC

2DAA  01 AD 3E     EXTAN:  LD   BC,FTAN          BC = rutinen för TAN
2DAD  18 17                JR   FFUNC

2DAF  01 77 3B     EXATN:  LD   BC,FATN          BC = rutinen för ATN
2DB2  18 12                JR   FFUNC

2DB4  01 03 3C     EXEXP1: LD   BC,FEXP          BC = rutinen för EXP
2DB7  18 0D                JR   FFUNC

2DB9  01 C4 3C     EXLOG:  LD   BC,FLOG          BC = rutinen för LOG
2DBC  18 08                JR   FFUNC

2DBE  01 B7 3C     EXLG10: LD   BC,FLOG10        BC = rutinen för LOG10
2DC1  18 03                JR   FFUNC

2DC3  01 F5 3D     EXSQR:  LD   BC,FSQR          BC = rutinen för SQR
```

FFUNC: anropa flyttalsrutinen i BC för argumentet vid SP. In: BC = rutinen, DE = koden. Rutinen får HL = BC = argumentets exponent och skriver resultatet över argumentet. Ut: DE = koden, nästa token med RST $30. Rutinen får förstöra alla register utom IY; DE sparas här.

```
2DC6  21 04 00     FFUNC:  LD   HL,$0004         argumentet vid SP, exponenten 4 byte upp
2DC9  39                   ADD  HL,SP            HL = argumentets exponent
2DCA  C5                   PUSH BC               rutinens adress
2DCB  DD E1                POP  IX               IX = rutinen
2DCD  44                   LD   B,H
2DCE  4D                   LD   C,L              BC = exponenten också
2DCF  D5                   PUSH DE               spara pekaren i koden
2DD0  CD 91 2A             CALL JPIX             som CALL (IX); resultatet över argumentet
2DD3  D1                   POP  DE               DE = pekaren i koden
2DD4  F7                   RST  $30              nästa token
```

FFIX: $FIX(x) = INT(ABS(x))$ med x:s tecken, det vill säga heltalsdelen mot 0. In: HL = BC = talets exponent. Talet ändras på plats. Tecknet sätts tillbaka också på en nolla, så $FIX(-.5)$ blir 00 00 00 01 00, en negativ nolla, som skrivs 0 och är lika med 0. Förstör A, BC, DE och IX (FINT).

```
2DD5  2B           FFIX:   DEC  HL               HL = tecknet
2DD6  7E                   LD   A,(HL)           A = tecknet
```

```
2DD7  36 00              LD   (HL),$00       gör positivt
2DD9  E5                 PUSH HL             spara tecknets adress
2DDA  F5                 PUSH AF             och tecknet
2DDB  CD 7F 3C           CALL FINT           INT av det positiva talet
2DDE  F1                 POP  AF
2DDF  E1                 POP  HL
2DE0  77                 LD   (HL),A         tecknet tillbaka
2DE1  C9                 RET
```

FTOI (`$D9`)*: flyttal till heltal (FTOINT, resultat i DE). Flyttalet vid SP tas bort och heltalet läggs på stacken. Fel 7 om talet efter INT inte ryms i 0..65535 (se FTOINT).*

```
2DE2  21 04 00     FTOI: LD   HL,$0004      exponenten 4 byte upp
2DE5  39                 ADD  HL,SP          HL = flyttalets exponent
2DE6  D5                 PUSH DE             spara koden
2DE7  CD CA 3E           CALL FTOINT         DE = heltalet
2DEA  E1                 POP  HL             HL = koden
2DEB  F1                 POP  AF             ta bort 5 byte
2DEC  F1                 POP  AF
2DED  33                 INC  SP             den femte
2DEE  EB                 EX   DE,HL          HL = heltalet, DE = koden
2DEF  EF                 RST  $28            heltalet på stacken, nästa token
```

ITOF (`$DA`)*: heltal till flyttal. Heltalet vid SP tas bort, plats för 5 byte görs på stacken och INTTOF skriver flyttalet där.*

```
2DF0  E1           ITOF: POP  HL             HL = heltalet
2DF1  F5                 PUSH AF             plats för 5 byte
2DF2  F5                 PUSH AF
2DF3  3B                 DEC  SP
2DF4  D5                 PUSH DE             spara koden
2DF5  EB                 EX   DE,HL          DE = heltalet
2DF6  21 06 00           LD   HL,$0006       platsens exponent: 6 byte upp
2DF9  39                 ADD  HL,SP
2DFA  EB                 EX   DE,HL          DE = exponenten, HL = heltalet
2DFB  CD 18 3F           CALL INTTOF         skriv flyttalet
2DFE  D1                 POP  DE             DE = koden
2DFF  F7                 RST  $30            nästa token
```

3.82 Potens: POWF ($FD), POWI ($FE), POWFI ($FF) $2E00

`POWF`: `a^b` med båda flyttal. `FPOW` anropas via `FARITH` (BC = DE = a, HL = b; resultatet hamnar i a) och räknar `EXP(b*LOG(a))`. Om b = 0 hoppas `LOG` över, så `a^0` ger 1 för alla a, även 0 och negativa tal. Annars går a <= 0 till `FLOG`: `0^2` och `(-2)^2` ger fel 4 (men `(-2)^3%` går via `POWFI`).

`POWI`: `a^b` med båda heltal, med upprepad kvadrering. För varje bit i b, den lägsta först, multipliceras resultatet med a om biten är satt, och a kvadreras. `a^0` ger 1 (även `0^0`). Spill som i `SMUL`: fel 7 först över 65535, annars slår talet runt (`200%^2%` ger -25536, `2%^15%` ger -32768, `256%^2%` ger fel 7). Med negativt b ger a = 0 fel 4, a = 1 ger 1, a = -1 ger -1 om b är udda och annars 1, och alla andra a ger 0 (`1/a^|b|` huggs av till 0).

`POWFI`: `a^n` med a flyttal och n heltal. Med negativt n blir först a = 1/a och n = -n. Sedan upprepad kvadrering som i `POWI`, med `FMUL`. `FMUL` ger fel 4 redan när exponenternas summa blir för stor, före normaliseringen: `10^125%` går och `10^126%` ger fel 4, fast 1E126 går att skriva in (likaså `1E100*1E26` och `1E126*1`; se `FMUL`, `$34DF`). EXP och LOG avrundar: `4^.5` = 2.00001 (prövat).

In: POWF och POWFI har b eller n vid SP och a (5 byte) ovanför, POWI b och a som heltal. Ut: resultatet på stacken, nästa token. DE (koden) bevaras, de andra registren förstörs.

```
2E00  DD 21 07 2E  POWF:   LD   IX,FPOW        FPOW räknas via FARITH (gränsfall)
2E04  C3 CD 2A             JP   FARITH         BC = DE = a, HL = b

2E07  7E           FPOW:   LD   A,(HL)         b:s exponent
2E08  A7                   AND  A              b = 0?
2E09  E5                   PUSH HL             spara b
2E0A  C5                   PUSH BC             och a
2E0B  C4 C4 3C             CALL NZ,FLOG        a = LOG a (inte om b = 0)
2E0E  C1                   POP  BC
2E0F  E1                   POP  HL
2E10  59                   LD   E,C            DE = a
2E11  50                   LD   D,B
2E12  CD DF 34             CALL FMUL           a = LOG(a) * b
2E15  C3 03 3C             JP   FEXP           a = EXP(a), och RET

2E18  C1           POWI:   POP  BC             BC = b (vid SP)
2E19  E1                   POP  HL             a
2E1A  D5                   PUSH DE             spara koden
2E1B  78                   LD   A,B
2E1C  87                   ADD  A,A            CY = bit 15
2E1D  38 21                JR   C,L2E40
2E1F  11 01 00             LD   DE,$0001       resultatet = 1
2E22  EB                   EX   DE,HL          HL = 1, DE = a
2E23  CB 38        L2E23:  SRL  B              b >>= 1, biten i CY
2E25  CB 19                RR   C
2E27  30 07                JR   NC,L2E30       biten 0: ingen multiplikation
2E29  D5                   PUSH DE             spara a
2E2A  C5                   PUSH BC             och b
2E2B  CD 83 2B             CALL SMUL           HL = HL * DE, fel 7 vid spill
2E2E  C1                   POP  BC
2E2F  D1                   POP  DE
2E30  78           L2E30:  LD   A,B            b = 0: klart
2E31  B1                   OR   C
2E32  28 1A                JR   Z,L2E4E
2E34  C5                   PUSH BC             spara b
2E35  E5                   PUSH HL             och resultatet
2E36  62                   LD   H,D            HL = a
2E37  6B                   LD   L,E
2E38  CD 83 2B             CALL SMUL           a * a
2E3B  D1                   POP  DE             DE = resultatet
2E3C  C1                   POP  BC             BC = b
2E3D  EB                   EX   DE,HL          HL = resultatet, DE = a
2E3E  18 E3                JR   L2E23          nästa bit

2E40  7C           L2E40:  LD   A,H            b < 0: a:s höga byte
2E41  B7                   OR   A
2E42  20 0C                JR   NZ,L2E50       a < 0 eller a > 255
2E44  B5                   OR   L              a = 0?
2E45  CA 01 07             JP   Z,L0701        0^-n: fel 4
2E48  3D                   DEC  A              a = 1?
2E49  28 03                JR   Z,L2E4E        1^-n = 1
2E4B  21 00 00     L2E4B:  LD   HL,$0000       annars 0
2E4E  D1           L2E4E:  POP  DE             DE = koden, resultatet i HL
2E4F  EF                   RST  $28            på stacken, nästa token

2E50  A5           L2E50:  AND  L              A = H AND L
2E51  3C                   INC  A              0 om a = $FFFF (-1)
2E52  20 F7                JR   NZ,L2E4B       annars 0
2E54  3E 01                LD   A,$01          b udda?
2E56  A1                   AND  C
```

```
2E57  20 F5              JR   NZ,L2E4E        -1^udda = -1
2E59  21 01 00           LD   HL,$0001       -1^jämnt = 1
2E5C  18 F0              JR   L2E4E
```

POWFI: *ett resultat r = 1 läggs på stacken under koden. För varje bit i n (n skiftas på sin plats på stacken) blir r = r * a om biten är satt, och a = a * a. Till sist kopieras r över a, och r och n tas bort. Stacken i slingan, IX = r:*

```
      IX+0..4  r        IX+5,6  n        IX+7..11  a
```

Koden ligger under r, vid IX-2. Med n = -32768 blir -n = $8000, som skiftas utan tecken och blir rätt.

```
2E5E  E1           POWFI:  POP  HL             HL = n
2E5F  E5                   PUSH HL
2E60  7C                   LD   A,H            A = n:s höga byte
2E61  87                   ADD  A,A            CY = bit 15
2E62  30 16                JR   NC,L2E7A       n >= 0
2E64  44                   LD   B,H            BC = n
2E65  4D                   LD   C,L
2E66  21 01 00             LD   HL,$0001
2E69  ED 42                SBC  HL,BC          1 - n - CY = -n
2E6B  E3                   EX   (SP),HL        ersätter n
2E6C  D5                   PUSH DE             spara koden
2E6D  21 08 00             LD   HL,$0008       a:s exponent
2E70  39                   ADD  HL,SP          HL = a
2E71  11 BB 3F             LD   DE,FONE        1
2E74  44                   LD   B,H            BC = a
2E75  4D                   LD   C,L
2E76  CD EE 35             CALL FDIV           a = 1 / a
2E79  D1                   POP  DE             DE = koden
2E7A  21 FB FF     L2E7A:  LD   HL,$FFFB       r: 5 byte under n
2E7D  39                   ADD  HL,SP          SP = r
2E7E  F9                   LD   SP,HL
2E7F  D5                   PUSH DE             spara koden
2E80  E5                   PUSH HL             r:s adress, till IX
2E81  01 05 00             LD   BC,$0005
2E84  09                   ADD  HL,BC
2E85  2B                   DEC  HL             HL = r:s exponent
2E86  11 BB 3F             LD   DE,FONE
2E89  EB                   EX   DE,HL          HL = FONE, DE = r:s exponent
2E8A  ED B8                LDDR                r = 1
2E8C  DD E1        L2E8C:  POP  IX             IX = r
2E8E  DD CB 06 3E          SRL  (IX+$06)       n >>= 1
2E92  DD CB 05 1E          RR   (IX+$05)
2E96  30 13                JR   NC,L2EAB       biten 0?
2E98  DD E5                PUSH IX             spara IX
2E9A  E1                   POP  HL
2E9B  E5                   PUSH HL             och tillbaka på stacken
2E9C  11 04 00             LD   DE,$0004
2E9F  19                   ADD  HL,DE          HL = r:s exponent
2EA0  1E 07                LD   E,$07          DE = 7
2EA2  EB                   EX   DE,HL          DE = r:s exponent, HL = 7
2EA3  19                   ADD  HL,DE          HL = a:s exponent (r + 11)
2EA4  42                   LD   B,D            BC = DE = r
2EA5  4B                   LD   C,E
2EA6  CD DF 34             CALL FMUL           r = r * a
2EA9  DD E1                POP  IX             IX = r
2EAB  DD 7E 06     L2EAB:  LD   A,(IX+$06)     n = 0?
2EAE  DD B6 05             OR   (IX+$05)
2EB1  28 11                JR   Z,L2EC4        klart
2EB3  DD E5                PUSH IX             spara IX igen
2EB5  E1                   POP  HL
```

```
2EB6  E5                PUSH HL             och IX på stacken
2EB7  11 0B 00          LD   DE,$000B
2EBA  19                ADD  HL,DE          HL = a:s exponent (r + 11)
2EBB  54                LD   D,H            DE = a
2EBC  5D                LD   E,L
2EBD  44                LD   B,H            BC = a
2EBE  4D                LD   C,L
2EBF  CD DF 34          CALL FMUL           a = a * a
2EC2  18 C8             JR   L2E8C          IX tas från stacken

2EC4  DD E5       L2EC4: PUSH IX             DE = r
2EC6  D1               POP  DE             DE = r:s första byte
2EC7  21 07 00         LD   HL,$0007       a ligger 7 byte efter r
2ECA  E5               PUSH HL             spara 7
2ECB  19               ADD  HL,DE          HL = a:s första byte
2ECC  EB               EX   DE,HL          DE = a, HL = r
2ECD  01 05 00         LD   BC,$0005
2ED0  ED B0            LDIR                a = r
2ED2  E1               POP  HL             HL = 7
2ED3  D1               POP  DE             koden
2ED4  39               ADD  HL,SP          SP = a, flyttalet kvar på stacken
2ED5  F9               LD   SP,HL
2ED6  F7               RST  $30            nästa token
```

CONSTS ($CB): *strängkonstant i koden:* $CB, *citattecken, längd, text. Beskrivningen* [*n = 0*][*adress*][*längd*] *läggs på stacken; texten ligger kvar i programmet.*

```
2ED7  EB          CONSTS: EX   DE,HL          HL = koden, vid citattecknet
2ED8  23                  INC  HL             hoppa över citattecknet
2ED9  5E                  LD   E,(HL)         längd
2EDA  16 00               LD   D,$00
2EDC  23                  INC  HL
2EDD  D5                  PUSH DE             längd
2EDE  E5                  PUSH HL             adress
2EDF  19                  ADD  HL,DE          HL = koden efter texten
2EE0  5A                  LD   E,D            n = 0
2EE1  EB                  EX   DE,HL          DE = koden, HL = 0
2EE2  EF                  RST  $28            n på stacken, nästa token
```

Korsreferenstabell för funktionerna med ett argument, typbytena, potenserna och CONSTS. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. EXFIX till EXSQR nås genom EXPTAB2 och de andra genom EXPTAB1; de tar DE efter sin kod och argumenten vid SP och lägger resultatet i deras ställe, med RST $28 (ett heltal) eller RST $30. Som i tabellen efter $2A4B räknas det som nästa rutin gör inte, men EXEXP förstör A, flaggorna, BC och HL. EXFIX till EXSQR lägger rutinen i BC, och FFUNC anropar den med HL = BC = argumentets exponent; rutinen får förstöra allt utom IY och DE sparas i FFUNC. FFIX är rutinen för FIX. POWF räknar a^b genom FARITH med FPOW, som får BC = DE = a och HL = b. Etiketterna inne i POWI och POWFI får inga egna rader. Felen: FTOI 7, POWI 4 och 7 (genom SMUL), POWF, FPOW och POWFI 4 (genom FLOG, FEXP, FMUL och FDIV), FPOW också 7 (genom FEXP). Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
EXFIX	X	X	X	IU	X	X	IU
EXINT	X	X	X	IU	X	X	IU
EXSIN	X	X	X	IU	X	X	IU
EXCOS	X	X	X	IU	X	X	IU

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
EXTAN	X	X	X	IU	X	X	IU
EXATN	X	X	X	IU	X	X	IU
EXEXP1	X	X	X	IU	X	X	IU
EXLOG	X	X	X	IU	X	X	IU
EXLG10	X	X	X	IU	X	X	IU
EXSQR	X	X	X	IU	X	X	IU
FFUNC	X	X	IX	IU	X	X	IU
FFIX	X	X	IX	X	IX	X	
FTOI	X	X	X	IU	X		IU
ITOF	X	X	X	IU	X		IU
POWF	X	X	X	IU	X	X	IU
FPOW	X	X	IX	IX	IX	X	
POWI	X	X	X	IU	X	X	IU
POWFI	X	X	X	IU	X	X	IU
CONSTS		X		IU	X		IU

Rutin	**Anropar**	**Anropas från**
EXFIX	FFUNC (med JR) och FFIX	EXPTAB2 (kod $05)
EXINT	FFUNC (med JR) och FINT	EXPTAB2 (kod $06)
EXSIN	FFUNC (med JR) och FSIN	EXPTAB2 (kod $0C)
EXCOS	FFUNC (med JR) och FCOS	EXPTAB2 (kod $03)
EXTAN	FFUNC (med JR) och FTAN	EXPTAB2 (kod $0E)
EXATN	FFUNC (med JR) och FATN	EXPTAB2 (kod $02)
EXEXP1	FFUNC (med JR) och FEXP	EXPTAB2 (kod $04)
EXLOG	FFUNC (med JR) och FLOG	EXPTAB2 (kod $07)
EXLG10	FFUNC (med JR) och FLOG10	EXPTAB2 (kod $08)
EXSQR	går in i FFUNC, med FSQR	EXPTAB2 (kod $0D)
FFUNC	JPIX (rutinen i BC), RST $30	EXFIX, EXINT, EXSIN, EXCOS, EXTAN, EXATN, EXEXP1, EXLOG och EXLG10 (med JR), EXSQR (går in i den)
FFIX	FINT	EXFIX (genom FFUNC)
FTOI	FTOINT, RST $28; fel 7	EXPTAB1 ($D9)
ITOF	INTTOF, RST $30	EXPTAB1 ($DA)
POWF	FARITH (med JP) och FPOW	EXPTAB1 ($FD)
FPOW	FLOG, FMUL, FEXP (med JP)	POWF (genom IX och FARITH)
POWI	SMUL, RST $28; fel 4	EXPTAB1 ($FE)
POWFI	FDIV, FMUL, RST $30	EXPTAB1 ($FF)
CONSTS	RST $28	EXPTAB1 ($CB)

3.83 Strängfunktionerna $2EE3

Argumenten ligger på stacken, sista argumentet vid SP. En sträng är beskrivningen [n][adress][längd] följd av n tillfälliga byte (se $2842). En ny sträng byggs direkt på stacken och blir en tillfällig sträng med n = längden.

```
LEN(s$)        längden
ASC(s$)        första tecknets kod, 0 för en tom sträng
LEFT$(s$,j)    de j första tecknen; fel 9 om j > längden
RIGHT$(s$,i)   från tecken i till slutet (i = 1 ger hela
               strängen); fel 9 om i = 0 eller i > längden + 1
MID$(s$,i,j)   j tecken från tecken i: först som RIGHT$, sedan
               som LEFT$ på det som blev kvar
```

Negativa j och i blir stora tal och ger fel 9.

LEFT$, RIGHT$ och MID$ kopierar inget: bara adress och längd i beskrivningen ändras, och de tillfälliga byten ligger kvar.

Koderna i EXPTAB2: LEN $14, ASC $0F, LEFT$ $11, RIGHT$ $12, MID$ $13. In: DE = koden, argumenten vid SP. Ut: EXLEN och EXASC tar bort strängen och lämnar ett heltal (RST $28); de andra lämnar den ändrade beskrivningen (RST $30). Förstör A, BC, HL och IX.

```
2EE3  DD E1        EXLEN:  POP  IX          n (s$ tas bort)
2EE5  E1                   POP  HL          adress
2EE6  E1                   POP  HL          längd = värdet
2EE7  DD 39                ADD  IX,SP
2EE9  DD F9                LD   SP,IX       ta bort de tillfälliga byten
2EEB  EF                   RST  $28         längden som heltal

2EEC  DD E1        EXASC:  POP  IX          n
2EEE  E1                   POP  HL          adress
2EEF  4E                   LD   C,(HL)      första tecknet
2EF0  E1                   POP  HL          längd
2EF1  DD 39                ADD  IX,SP
2EF3  DD F9                LD   SP,IX       ta bort strängen
2EF5  7C                   LD   A,H         tom sträng?
2EF6  B5                   OR   L
2EF7  28 03                JR   Z,L2EFC     tom: HL = 0
2EF9  26 00                LD   H,$00       HL = tecknets kod
2EFB  69                   LD   L,C
2EFC  EF           L2EFC:  RST  $28          heltalet på stacken, nästa token

2EFD  E1           EXMID:  POP  HL          j
2EFE  C1                   POP  BC          i
2EFF  E5                   PUSH HL          j tillbaka, vid SP
2F00  DD 21 04 00          LD   IX,$0004    beskrivningen över returadressen och j
2F04  CD 25 2F             CALL STRSKP      som RIGHT$, sedan LEFT$
2F07  C1           EXLEFT: POP  BC          j
2F08  21 04 00             LD   HL,$0004    beskrivningen: n, adress, längd
2F0B  39                   ADD  HL,SP
2F0C  7E                   LD   A,(HL)      längd
2F0D  23                   INC  HL
2F0E  66                   LD   H,(HL)
2F0F  6F                   LD   L,A
2F10  ED 42                SBC  HL,BC       längd - j
2F12  38 1C                JR   C,L2F30     j > längden: fel 9
2F14  E1                   POP  HL          n
2F15  DD E1                POP  IX          adress
2F17  F1                   POP  AF          gamla längden
2F18  C5                   PUSH BC          ny längd = j
2F19  DD E5                PUSH IX
2F1B  EF                   RST  $28         n tillbaka (RST28 lägger HL), nästa token
```

```
2F1C  C1           EXRGHT: POP  BC              i
2F1D  DD 21 02 00          LD   IX,$0002       beskrivningen över returadressen
2F21  CD 25 2F             CALL STRSKP
2F24  F7                   RST  $30            nästa token
```

STRSKP: *hoppa över de i-1 första tecknen i strängen. In: BC = i, IX = avståndet från SP (efter* CALL*) till beskrivningen: 4 från* MID$ *(j ligger emellan), 2 från* RIGHT$*. Adressen ökas med i-1 och längden minskas lika mycket. Fel 9 om i-1 är större än längden (också för i = 0, som blir* $FFFF*). Ut: IX = beskrivningen, BC = i-1. Förstör HL.*

```
2F25  0B           STRSKP: DEC  BC              i - 1
2F26  DD 39                ADD  IX,SP          IX = beskrivningen
2F28  DD 6E 04             LD   L,(IX+$04)     längden
2F2B  DD 66 05             LD   H,(IX+$05)
2F2E  ED 42                SBC  HL,BC          längd - (i - 1)
2F30  DA 0B 07     L2F30:  JP   C,L070B        fel 9
2F33  DD 75 04             LD   (IX+$04),L     ny längd
2F36  DD 74 05             LD   (IX+$05),H
2F39  DD 6E 02             LD   L,(IX+$02)     adressen
2F3C  DD 66 03             LD   H,(IX+$03)
2F3F  09                   ADD  HL,BC          adress + i - 1
2F40  DD 75 02             LD   (IX+$02),L     ny adress
2F43  DD 74 03             LD   (IX+$03),H
2F46  C9                   RET
```

3.84 CONCAT ($F6): a$ + b$ $2F47

b$ ligger vid SP, a$ ovanför. Den nya strängen byggs först under stacken (ROOM kontrollerar att det finns plats): a$ och sedan b$ kopieras dit. Sedan flyttas den med LDDR uppåt så att den slutar där a$:s tillfälliga byte slutade, och ersätter båda strängarna. Den blir en tillfällig sträng [n = längd][adress][längd].

STRPSH ($2FBB) är gemensamt slut för CONCAT och CHR$: HL = strängen (och ny SP), BC = längden; beskrivningen läggs vid SP. CONCAT ger fel 3 om summan inte får plats. In: DE = koden. Förstör A, BC, HL och IX.

```
2F47  DD 21 06 00  CONCAT: LD   IX,$0006       IX-2 = b$:s längd
2F4B  DD 39                ADD  IX,SP
2F4D  DD 6E FE             LD   L,(IX-$02)     b$:s längd
2F50  DD 66 FF             LD   H,(IX-$01)
2F53  C1                   POP  BC             b$:s n
2F54  C5                   PUSH BC             n kvar vid SP
2F55  DD 09                ADD  IX,BC          IX = a$:s beskrivning
2F57  DD 4E 04             LD   C,(IX+$04)     a$:s längd
2F5A  DD 46 05             LD   B,(IX+$05)
2F5D  09                   ADD  HL,BC          summan
2F5E  44                   LD   B,H            BC = summan
2F5F  4D                   LD   C,L
2F60  21 00 00             LD   HL,$0000       HL = SP
2F63  39                   ADD  HL,SP
2F64  ED 42                SBC  HL,BC          SP - summan
2F66  DA FF 06             JP   C,L06FF        fel 3
2F69  CD 9B 33             CALL ROOM           minst 100 byte över HEAP?
2F6C  F9                   LD   SP,HL          plats för den nya strängen
2F6D  D5                   PUSH DE             koden
2F6E  C5                   PUSH BC             längd
2F6F  EB                   EX   DE,HL          DE = nya strängen
2F70  DD 46 05             LD   B,(IX+$05)     BC = a$:s längd
2F73  DD 4E 04             LD   C,(IX+$04)
```

```
2F76  79              LD   A,C          tom a$?
2F77  B0              OR   B
2F78  28 08           JR   Z,L2F82
2F7A  DD 66 03        LD   H,(IX+$03)   HL = a$:s adress
2F7D  DD 6E 02        LD   L,(IX+$02)
2F80  ED B0           LDIR              kopiera a$
2F82  DD E1     L2F82: POP  IX          IX = summan
2F84  DD E5           PUSH IX           summan kvar vid SP
2F86  DD 39           ADD  IX,SP        IX+4 = b$:s beskrivning
2F88  DD 46 09        LD   B,(IX+$09)
2F8B  DD 4E 08        LD   C,(IX+$08)   BC = b$:s längd
2F8E  79              LD   A,C          tom b$?
2F8F  B0              OR   B
2F90  28 08           JR   Z,L2F9A
2F92  DD 66 07        LD   H,(IX+$07)   HL = b$:s adress
2F95  DD 6E 06        LD   L,(IX+$06)
2F98  ED B0           LDIR              kopiera b$
2F9A  1B        L2F9A: DEC  DE          sista byten
2F9B  C1              POP  BC           BC = summan
2F9C  21 02 00        LD   HL,$0002
2F9F  7D              LD   A,L          A = 2 strängar
2FA0  39              ADD  HL,SP        SP+2 = nya strängen
2FA1  09              ADD  HL,BC        HL = b$:s beskrivning
2FA2  C5              PUSH BC           summan
2FA3  4E        L2FA3: LD   C,(HL)      n
2FA4  23              INC  HL
2FA5  46              LD   B,(HL)
2FA6  09              ADD  HL,BC        + n
2FA7  01 05 00        LD   BC,$0005
2FAA  09              ADD  HL,BC        + 6: nästa värde
2FAB  3D              DEC  A
2FAC  20 F5           JR   NZ,L2FA3
2FAE  2B              DEC  HL           sista byten i a$:s plats
2FAF  C1              POP  BC
2FB0  C5              PUSH BC
2FB1  EB              EX   DE,HL        DE = målet, HL = nya strängens sista byte
2FB2  78              LD   A,B          tom summa?
2FB3  B1              OR   C
2FB4  28 02           JR   Z,L2FB8
2FB6  ED B8           LDDR              flytta upp strängen
2FB8  C1        L2FB8: POP  BC          BC = summan
2FB9  13              INC  DE           DE = strängens nya början
2FBA  EB              EX   DE,HL        HL = strängen
2FBB  D1        STRPSH: POP  DE         koden
2FBC  F9              LD   SP,HL        strängen vid SP
2FBD  C5              PUSH BC           längd
2FBE  E5              PUSH HL           adress
2FBF  C5              PUSH BC           n = längd
2FC0  F7              RST  $30
```

EXCHR: CHR$(i[,i[,i[,i]]]) (*kod* $10) *ger en sträng med 1-4 tecken. Antalet k tas ur token före funktionskoden* ($CD + *k*). *Heltalen ligger på stacken, det sista vid SP. Deras låga byte packas ihop i början av området (sista argumentet först) och skrivs sedan baklänges efter dem, så att tecknen i rätt ordning hamnar i den övre halvan av de 2k byte som heltalen tog. Den halvan blir strängen* (STRPSH). *Bara den låga byten räknas* (CHR$(321) = "A")*; fem argument ger fel 13 redan när raden skrivs in. Förstör A, BC och HL.*

```
2FC1  D5        EXCHR: PUSH DE          koden
2FC2  1B               DEC  DE          tillbaka till token före funktionskoden
2FC3  1B               DEC  DE
2FC4  1A               LD   A,(DE)      token $CD + k
```

```
2FC5  D6 CD              SUB   $CD          k = antal argument
2FC7  4F                 LD    C,A
2FC8  21 02 00           LD    HL,$0002     HL = SP + 2, B = 0
2FCB  44                 LD    B,H
2FCC  39                 ADD   HL,SP        sista argumentet
2FCD  54                 LD    D,H
2FCE  5D                 LD    E,L
2FCF  ED A0        L2FCF: LDI                packa de låga byten
2FD1  23                 INC   HL           hoppa över höga byten
2FD2  EA CF 2F           JP    PE,L2FCF
2FD5  4F                 LD    C,A          BC = k igen
2FD6  D5                 PUSH  DE           strängens början
2FD7  62                 LD    H,D          HL = DE
2FD8  6B                 LD    L,E
2FD9  2B           L2FD9: DEC   HL           skriv dem baklänges
2FDA  ED A0              LDI
2FDC  2B                 DEC   HL
2FDD  EA D9 2F           JP    PE,L2FD9
2FE0  E1                 POP   HL           HL = strängen
2FE1  4F                 LD    C,A          BC = k: längden
2FE2  18 D7              JR    STRPSH       beskrivningen och nästa token
```

Korsreferenstabell för strängfunktionerna och CONCAT. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna. Kodrutinerna tar DE efter sin token och strängarna och argumenten vid SP och lägger resultatet i deras ställe, med `RST $28` (ett heltal) eller `RST $30`; som i tabellen efter `$2A4B` räknas det som nästa rutin gör inte, men `EXEXP` förstör A, flaggorna, BC och HL. `EXLEN`, `EXASC`, `EXLEFT`, `EXRGHT`, `EXMID` och `EXCHR` är funktionerna LEN, ASC, LEFT$, RIGHT$, MID$ och CHR$ och nås genom `EXFUNC`. `STRSKP` ändrar beskrivningen på stacken och lämnar IX = beskrivningen och BC = $i - 1$. `STRPSH` tar koden från stacken och lägger beskrivningen för strängen vid HL. Felen: `EXLEFT`, `EXRGHT`, `EXMID` och `STRSKP` 9, `CONCAT` 3. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP
EXLEN	X	X	X	IU	X	X	IU	
EXASC	X	X	X	IU	X	X	IU	
EXMID	X	X	X	IU	X	X	IU	
EXLEFT	X	X	X	IU	X	X	IU	
EXRGHT	X	X	X	IU	X	X	IU	
STRSKP		X	IU		X	IU	IU	
CONCAT	X	X	X	IU	X	X	IU	I
STRPSH	X	X	IX	U	IX		IU	
EXCHR	X	X	X	IU	X		IU	

Rutin	**Anropar**	**Anropas från**
EXLEN	`RST $28`	EXPTAB2 (kod $14)
EXASC	`RST $28`	EXPTAB2 (kod $0F)
EXMID	STRSKP; går in i EXLEFT	EXPTAB2 (kod $13)
EXLEFT	`RST $28`; fel 9 (genom $2F30 i STRSKP)	EXPTAB2 (kod $11), EXMID (går in i den)
EXRGHT	STRSKP, `RST $30`	EXPTAB2 (kod $12)
STRSKP	fel 9	EXMID, EXRGHT
CONCAT	ROOM; går in i STRPSH; fel 3	EXPTAB1 ($F6)
STRPSH	`RST $30`	CONCAT (går in i den), EXCHR (med JR)

Rutin	Anropar	Anropas från
EXCHR	STRPSH (med JR)	EXPTAB2 (kod $10)

3.85 Strängjämförelser: EQS NES GTS LES LTS GES $2FE4

STRCMP ger Z = lika och CY = a$ mindre än b$, och tar bort båda strängarna. Resultatet är ett heltal: -1 (STRUE) = sant, 0 (SFALSE) = falskt. Teckenkoderna jämförs som de är, så ”a” är större än ”A”, och den tomma strängen är mindre än ”A”. Token: = $E4, <> $E7, < $EA, >= $ED, > $F0, <= $F3. Värdet läggs med RST $28. Förstör A, BC och HL.

```
2FE4  CD 12 30     EQS:     CALL STRCMP
2FE7  28 09                 JR   Z,STRUE          lika: sant
2FE9  21 00 00     SFALSE:  LD   HL,$0000        falskt
2FEC  EF                    RST  $28

2FED  CD 12 30     NES:     CALL STRCMP
2FF0  28 F7                 JR   Z,SFALSE        lika: falskt
2FF2  21 FF FF     STRUE:   LD   HL,$FFFF        sant
2FF5  EF                    RST  $28

2FF6  CD 12 30     GTS:     CALL STRCMP
2FF9  28 EE                 JR   Z,SFALSE        lika: falskt
2FFB  38 EC        L2FFB:   JR   C,SFALSE        mindre: falskt
2FFD  18 F3                 JR   STRUE           större: sant

2FFF  CD 12 30     LES:     CALL STRCMP
3002  28 EE                 JR   Z,STRUE         lika: sant
3004  38 EC        L3004:   JR   C,STRUE         mindre: sant
3006  18 E1                 JR   SFALSE          större: falskt

3008  CD 12 30     LTS:     CALL STRCMP
300B  18 F7                 JR   L3004           mindre: sant

300D  CD 12 30     GES:     CALL STRCMP
3010  18 E9                 JR   L2FFB           mindre: falskt, annars sant
```

STRCMP: jämför a$ och b$ (b$ närmast ovanför returadressen) tecken för tecken. Ut: Z = lika, CY = a$ < b$. Om den ena strängen är början av den andra är den kortare minst. Båda strängarna tas bort, och återhoppet görs med JP (IX)*. CY sparas i bit 7 av A* (RRA ... RLA) *medan* ADD HL,SP *ändrar CY. Stacken efter* PUSH DE*: SP+2 returadress, SP+4 b$:s beskrivning, sedan b$:s tillfälliga byte och a$:s beskrivning (IX+*$0A/$0C/$0E*). DE bevaras; förstör A, BC, HL och IX.*

```
3012  D5           STRCMP:  PUSH DE              koden
3013  21 09 00               LD   HL,$0009       b$:s längd
3016  39                     ADD  HL,SP
3017  46                     LD   B,(HL)
3018  2B                     DEC  HL
3019  4E                     LD   C,(HL)
301A  2B                     DEC  HL
301B  56                     LD   D,(HL)         b$:s adress
301C  2B                     DEC  HL
301D  5E                     LD   E,(HL)
301E  2B                     DEC  HL
301F  7E                     LD   A,(HL)         b$:s n
3020  2B                     DEC  HL
3021  6E                     LD   L,(HL)         HL = b$:s n
3022  67                     LD   H,A
3023  39                     ADD  HL,SP          hoppa över b$:s byte
3024  E5                     PUSH HL             IX = SP + n
```

```
3025  DD E1              POP   IX
3027  DD 6E 0E           LD    L,(IX+$0E)      a$:s längd
302A  DD 66 0F           LD    H,(IX+$0F)
302D  ED 42              SBC   HL,BC           a$:s längd - b$:s
302F  DD 6E 0C           LD    L,(IX+$0C)      a$:s adress
3032  DD 66 0D           LD    H,(IX+$0D)      HL = a$:s adress
3035  EB                 EX    DE,HL           DE = a$, HL = b$
3036  38 23              JR    C,L305B         a$ kortare
3038  20 1C              JR    NZ,L3056        a$ längre
303A  78           L303A:  LD    A,B           lika långa: jämför allt
303B  B1                   OR    C
303C  28 07                JR    Z,L3045       alla lika: Z, NC
303E  1A                   LD    A,(DE)        tecknet i a$
303F  13                   INC   DE
3040  BE                   CP    (HL)          mot b$
3041  23                   INC   HL
3042  0B                   DEC   BC
3043  28 F5                JR    Z,L303A       lika: nästa tecken
3045  1F           L3045:  RRA                 spara CY
3046  D1                   POP   DE            koden
3047  DD E1                POP   IX            returadressen
3049  E1                   POP   HL            b$:s n
304A  C1                   POP   BC
304B  C1                   POP   BC
304C  39                   ADD   HL,SP         ta bort b$
304D  F9                   LD    SP,HL
304E  E1                   POP   HL            a$:s n
304F  C1                   POP   BC
3050  C1                   POP   BC
3051  39                   ADD   HL,SP         ta bort a$
3052  F9                   LD    SP,HL
3053  17                   RLA                 CY tillbaka
3054  DD E9                JP    (IX)          tillbaka till jämförelsen

3056  CD 68 30     L3056:  CALL  STRCPN        jämför lika många tecken som b$ har
3059  18 0A                JR    L3065         lika början: a$ större

305B  DD 4E 0E     L305B:  LD    C,(IX+$0E)    jämför lika många tecken som a$ har
305E  DD 46 0F             LD    B,(IX+$0F)
3061  CD 68 30             CALL  STRCPN
3064  37                   SCF                 lika början: a$ mindre
3065  3C           L3065:  INC   A             NZ, CY behålls
3066  18 DD                JR    L3045         ta bort strängarna
```

STRCPN: *jämför BC tecken, DE = a$, HL = b$. Lika:* RET *med Z.*

Olika: returadressen tas bort och STRCMP *slutar med flaggorna från* CP*: Z = 0, CY = tecknet i a$ mindre. Förstör A och BC; DE och HL efter tecknen.*

```
3068  78           STRCPN: LD    A,B           tecken kvar?
3069  B1                   OR    C
306A  C8                   RET   Z             alla lika: Z
306B  1A                   LD    A,(DE)        tecknet i a$
306C  BE                   CP    (HL)          mot b$: Z och CY
306D  13                   INC   DE            flaggorna står kvar
306E  23                   INC   HL
306F  0B                   DEC   BC            ett tecken färre
3070  28 F6                JR    Z,STRCPN      lika: nästa tecken
3072  E1                   POP   HL            olika: ta bort returadressen
3073  18 D0                JR    L3045         ta bort strängarna
```

3.86 EXINST: INSTR(i, a$, b$) $3075

Ger positionen (1 = första tecknet) där b$ finns i a$, sökt från tecken i. 0 om b$ inte finns, om i = 0, eller om i är större än längden + 1. Första tecknet i b$ söks med CPIR, resten jämförs med CPI. Hittas det inte går sökningen vidare efter det första tecknet. Med tom b$ hoppar koden till $30DB med HL = antal tecken kvar i a$ i stället för en adress, så resultatet blir (längd - i + 1) minus a$:s adress. Det är ett fel i ROM:en: INSTR(1,A$,"") ger ett tal som beror på var A$ ligger i minnet, inte 1.

Kod $15 i EXPTAB2. Prövat: med A$="HELLOABC" ger INSTR(1,A$,"L") 3, INSTR(4,A$,"L") 4, och i = 0, i = 9 och i = 10 ger 0. In: DE = koden. Ut: heltalet (RST $28), i och båda strängarna borttagna. Förstör A, BC, HL och IX.

```
3075  DD E1        EXINST: POP  IX              b$:s n (fel)
3077  E1                   POP  HL              b$:s adress
3078  C1                   POP  BC              b$:s längd
3079  DD 39                ADD  IX,SP           IX = a$:s beskrivning
307B  D5                   PUSH DE              koden
307C  E5                   PUSH HL              b$:s adress
307D  C5                   PUSH BC              b$:s längd
307E  DD 4E 00             LD   C,(IX+$00)      a$:s n
3081  DD 46 01             LD   B,(IX+$01)
3084  DD 6E 02             LD   L,(IX+$02)      a$:s adress
3087  DD 66 03             LD   H,(IX+$03)
308A  DD 5E 04             LD   E,(IX+$04)      a$:s längd
308D  DD 56 05             LD   D,(IX+$05)
3090  DD E5                 PUSH IX
3092  DD 09                ADD  IX,BC           hoppa över a$:s byte
3094  DD 4E 06             LD   C,(IX+$06)      i
3097  DD 46 07             LD   B,(IX+$07)
309A  DD E1                POP  IX
309C  0B                   DEC  BC              i - 1
309D  09                   ADD  HL,BC           sökningens början
309E  EB                   EX   DE,HL
309F  ED 42                SBC  HL,BC           tecken kvar = längd - (i - 1)
30A1  38 4E                JR   C,L30F1         i för stort: 0
30A3  C1                   POP  BC              b$:s längd
30A4  78                   LD   A,B
30A5  B1                   OR   C
30A6  28 33                JR   Z,L30DB         tom b$: fel resultat, se blocket
30A8  C5                   PUSH BC
30A9  ED 42                SBC  HL,BC           - b$:s längd
30AB  38 44                JR   C,L30F1         b$ längre än resten: 0
30AD  23                   INC  HL              antal möjliga startlägen
30AE  44                   LD   B,H
30AF  4D                   LD   C,L
30B0  E1                   POP  HL              b$:s längd
30B1  E3                   EX   (SP),HL         HL = b$:s adress
30B2  EB                   EX   DE,HL           DE = b$, HL = a$
30B3  78             L30B3: LD   A,B           inga lägen kvar: 0
30B4  B1                   OR   C
30B5  28 3B                JR   Z,L30F2
30B7  1A                   LD   A,(DE)          b$:s första tecken
30B8  ED B1                CPIR                 sök det
30BA  20 36                JR   NZ,L30F2        finns inte: 0
30BC  C5                   PUSH BC              lägen kvar
30BD  D5                   PUSH DE              b$
30BE  E5                   PUSH HL              a$ efter det funna tecknet
30BF  21 06 00             LD   HL,$0006
30C2  39                   ADD  HL,SP
```

```
30C3  4E           LD   C,(HL)          b$:s längd
30C4  23           INC  HL
30C5  46           LD   B,(HL)
30C6  0B           DEC  BC              resten av b$
30C7  78           LD   A,B
30C8  B1           OR   C
30C9  28 0B        JR   Z,L30D6         ett tecken: hittat
30CB  E1           POP  HL              HL = a$ efter det funna tecknet
30CC  E5           PUSH HL
30CD  13     L30CD: INC  DE              nästa tecken i b$
30CE  1A           LD   A,(DE)
30CF  ED A1        CPI                  jämför nästa tecken
30D1  20 03        JR   NZ,L30D6
30D3  EA CD 30     JP   PE,L30CD        fler tecken i b$
30D6  E1     L30D6: POP  HL
30D7  D1           POP  DE
30D8  C1           POP  BC              BC = lägen kvar
30D9  20 D8        JR   NZ,L30B3        olika: sök vidare
30DB  DD 5E 02 L30DB: LD  E,(IX+$02)    a$:s adress
30DE  DD 56 03     LD   D,(IX+$03)
30E1  ED 52        SBC  HL,DE           position = efter första tecknet - a$
30E3  D1     L30E3: POP  DE              b$:s längd (b$:s adress vid tom b$)
30E4  D1           POP  DE              koden
30E5  DD F9        LD   SP,IX           a$:s beskrivning
30E7  DD E1        POP  IX              a$:s n
30E9  C1           POP  BC
30EA  C1           POP  BC
30EB  DD 39        ADD  IX,SP
30ED  DD F9        LD   SP,IX           ta bort a$ och b$
30EF  C1           POP  BC              ta bort i
30F0  EF           RST  $28

30F1  E1     L30F1: POP  HL              b$:s längd
30F2  21 00 00 L30F2: LD  HL,$0000      0
30F5  18 EC        JR   L30E3
```

Korsreferenstabell för strängjämförelserna och INSTR. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. Kodrutinerna tar DE efter sin token och strängarna vid SP och lägger heltalet (−1 för sant, 0 för falskt, eller positionen) i deras ställe med `RST $28`; som i tabellen efter `$2A4B` räknas det som nästa rutin gör inte, men `EXEXP` förstör A, flaggorna, BC och HL. `STRCMP` svarar i flaggorna: Z om lika, annars CY om a$ är mindre; den tar bort båda strängarna och hoppar tillbaka med `JP (IX)`. `STRCPN` svarar med Z när alla BC tecken är lika; annars hoppar den tillbaka in i `STRCMP` med flaggorna från `CP`. `EXINST` är funktionen INSTR och nås genom `EXFUNC`. Inga fel. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
EQS	X	X	X	IU	X	X	IU
NES	X	X	X	IU	X	X	IU
GTS	X	X	X	IU	X	X	IU
LES	X	X	X	IU	X	X	IU
LTS	X	X	X	IU	X	X	IU
GES	X	X	X	IU	X	X	IU
STRCMP	X	U	X		X	X	IU
STRCPN	X	U	IX	IX	IX		
EXINST	X	X	X	IU	X	X	IU

Rutin	Anropar	Anropas från
EQS	STRCMP, STRUE (med JR), RST $28	EXPTAB1 ($E4), NES, GTS och LES (till SFALSE, med JR)
NES	STRCMP, SFALSE (med JR), RST $28	EXPTAB1 ($E7), EQS, GTS och LES (till STRUE, med JR)
GTS	STRCMP, SFALSE och STRUE (med JR)	EXPTAB1 ($F0), GES (till $2FFB, med JR)
LES	STRCMP, SFALSE och STRUE (med JR)	EXPTAB1 ($F3), LTS (till $3004, med JR)
LTS	STRCMP, LES ($3004, med JR)	EXPTAB1 ($EA)
GES	STRCMP, GTS ($2FFB, med JR)	EXPTAB1 ($ED)
STRCMP	STRCPN; JP (IX) tillbaka	EQS, NES, GTS, LES, LTS, GES, STRCPN (till $3045, med JR)
STRCPN	olika: STRCMP ($3045, med JR)	STRCMP (två ställen)
EXINST	RST $28	EXPTAB2 (kod $15)

EXSPC: SPACE$(n) (*kod* $16). EXSTRG: STRING$(n,c) (*kod* $17): *n tecken med koden c; bara den låga byten av c räknas* (STRING$(2,321) = "AA"), *och c som sträng ger fel 17.* STRFIL ($30FE): *BC = längden, A = tecknet, DE = koden. Strängen byggs på stacken och blir tillfällig* (*n = längden*), *och* STRFIL *slutar med* RST $30. *Fel 3 om den inte får plats* (ROOM).

```
30F7  3E 20        EXSPC:  LD   A,$20          mellanslag
30F9  18 02                JR   L30FD

30FB  E1           EXSTRG: POP  HL             c (bara låga byten)
30FC  7D                   LD   A,L
30FD  C1           L30FD:  POP  BC             n
30FE  21 00 00     STRFIL: LD   HL,$0000       HL = SP
3101  39                   ADD  HL,SP
3102  ED 42                SBC  HL,BC          SP - n
3104  DA FF 06             JP   C,L06FF        fel 3
3107  F5                   PUSH AF             tecknet
3108  CD 9B 33             CALL ROOM           minst 100 byte över HEAP?
310B  F1                   POP  AF             tecknet
310C  F9                   LD   SP,HL          plats för strängen
310D  C5                   PUSH BC             längd
310E  E5                   PUSH HL             adress
310F  C5                   PUSH BC             n = längd
3110  F5                   PUSH AF             tecknet igen
3111  78                   LD   A,B
3112  B1                   OR   C
3113  28 0F                JR   Z,L3124        tom sträng
3115  F1                   POP  AF             tecknet
3116  77                   LD   (HL),A         första tecknet
3117  0B                   DEC  BC
3118  78                   LD   A,B
3119  B1                   OR   C
311A  28 07                JR   Z,L3123
311C  D5                   PUSH DE             koden
311D  54                   LD   D,H            DE = HL + 1
311E  5D                   LD   E,L
311F  13                   INC  DE
3120  ED B0                LDIR                fyll resten
3122  D1                   POP  DE             koden
3123  F7           L3123:  RST  $30            nästa token
```

```
3124  C1          L3124:  POP   BC          ta bort AF
3125  F7                  RST   $30         nästa token (tom sträng)
```

EXTAB: TAB(n) (*kod* $20) *ger en sträng med mellanslag från den aktuella kolumnen* (GETPOS) *fram till kolumn n, tom om man redan är där eller längre. Bara den låga byten av skillnaden används:* TAB(261) *i kolumn 0 ger 5 mellanslag. Förstör A, BC och HL.*

```
3126  D5          EXTAB:  PUSH  DE          koden (egenhet)
3127  CD EF 08            CALL  GETPOS      A = kolumnen
312A  D1                  POP   DE
312B  E1                  POP   HL          n
312C  4F                  LD    C,A         BC = kolumnen
312D  06 00               LD    B,$00
312F  A7                  AND   A           CY = 0
3130  ED 42               SBC   HL,BC       n - kolumnen
3132  4D                  LD    C,L         längd = låga byten
3133  3E 20               LD    A,$20       mellanslag
3135  30 C7               JR    NC,STRFIL   n >= kolumnen
3137  0E 00               LD    C,$00       redan förbi: tom
3139  18 C3               JR    STRFIL
```

EXCUR: CUR(r,k) (*kod* $21) *ger en sträng på 4 byte: ESC = r+32 k+32, som flyttar markören när den skrivs ut* (*se* $0202). *Det är samma följd som terminalen ADM-3A använder.* $3D1B *är konstanten ESC =* (*byte* $1B $3D). *Bara de låga byten av r och k används, och inga gränser prövas* (*LEN*(CUR(300,2)) = 4). *Strängen läggs direkt på stacken utan* ROOM. *Förstör BC och HL.*

```
313B  C1          EXCUR:  POP   BC          k
313C  E1                  POP   HL          r
313D  61                  LD    H,C         H = k, L = r
313E  01 20 20            LD    BC,$2020
3141  09                  ADD   HL,BC       + 32
3142  E5                  PUSH  HL
3143  21 1B 3D            LD    HL,$3D1B    ESC =
3146  E5                  PUSH  HL
3147  21 00 00            LD    HL,$0000
314A  39                  ADD   HL,SP       strängen ligger på SP
314B  01 04 00            LD    BC,$0004
314E  C5                  PUSH  BC          längd 4
314F  E5                  PUSH  HL          adress
3150  C5                  PUSH  BC          n = 4
3151  F7                  RST   $30         nästa token
```

EXNUM: NUM$(x) (*kod* $18). *Talet skrivs med* FLTAS6 (*6 siffror, som* PRINT) *i en buffert på 15 byte under x. Ett positivt tal får ett mellanslag först, som i* PRINT: NUM$(1.5) = " 1.5", NUM$(-2) = "-2", LEN(NUM$(1)) = 2. *Resultatet blir en tillfällig sträng med n = 20 (bufferten + x:s 5 byte), så x tas bort med den. Förstör A, BC och HL.*

```
3152  21 F1 FF    EXNUM:  LD    HL,$FFF1    15 byte buffert
3155  39                  ADD   HL,SP       HL = SP - 15
3156  CD 9B 33            CALL  ROOM        minst 100 byte över HEAP?
3159  F9                  LD    SP,HL
315A  D5                  PUSH  DE          koden
315B  EB                  EX    DE,HL       DE = bufferten
315C  21 13 00            LD    HL,$0013
315F  19                  ADD   HL,DE       HL = x:s exponent
3160  D5                  PUSH  DE
3161  CD 77 16            CALL  FLTAS6      skriv talet
3164  E1                  POP   HL          buffertens början
3165  7B                  LD    A,E         längd = slut - början
3166  95                  SUB   L
```

```
3167  4F                LD   C,A
3168  06 00             LD   B,$00
316A  D1                POP  DE             koden
316B  C5                PUSH BC             längd
316C  E5                PUSH HL             adress
316D  01 14 00          LD   BC,$0014       n = 15 + 5
3170  C5                PUSH BC
3171  F7                RST  $30            nästa token
```

3.87 EXVAL: VAL(s$) $3172

Texten kopieras till en buffert på 80 byte på stacken, avslutas med CR och läses med GETSFL ("+" eller "-" först är tillåtet).

```
    längd 0, eller inget tal              -> fel 12
    längd 78 eller mer                    -> fel 3
    något annat än blanka efter talet     -> fel 12 ("12X")
```

Blanka före, efter och inne i talet hoppas över (GETFLT): VAL(" 1 2 ") ger 12. RST $20 på $31B3 läser från HL och gör ingenting som används; blanka efter talet har GETSFL redan hoppat över.

Resultatet skrivs i de översta fem byten av strängens plats på stacken (beskrivningen + n byte, minst 6), och allt under tas bort.

Stacken: IX = bufferten, IX+$50 adress, IX+$52 längd, IX+$54 och uppåt s$:s tillfälliga byte.

Kod $19 i EXPTAB2. Prövat: VAL(" 2 ") = 2, VAL("+4") = 4. In: DE = koden. Ut: flyttalet vid SP (RST $30). Förstör A, BC, HL och IX.

```
3172  C1          EXVAL:  POP  BC             n
3173  21 B0 FF            LD   HL,$FFB0       80 byte buffert
3176  39                  ADD  HL,SP
3177  CD 9B 33            CALL ROOM           minst 100 byte över HEAP?
317A  F9                  LD   SP,HL
317B  D5                  PUSH DE             koden
317C  C5                  PUSH BC
317D  E5                  PUSH HL             IX = bufferten
317E  DD E1               POP  IX
3180  EB                  EX   DE,HL          DE = bufferten
3181  DD 7E 53            LD   A,(IX+$53)     längd, hög byte
3184  A7                  AND  A
3185  20 08               JR   NZ,L318F       >= 256: fel 3
3187  DD B6 52            OR   (IX+$52)       längd, låg byte
318A  CA 13 07            JP   Z,L0713        tom: fel 12
318D  FE 4E               CP   $4E            >= 78?
318F  D2 FF 06    L318F:  JP   NC,L06FF       fel 3
3192  4F                  LD   C,A
3193  06 00               LD   B,$00          BC = längden
3195  DD 6E 50            LD   L,(IX+$50)     adress
3198  DD 66 51            LD   H,(IX+$51)
319B  ED B0               LDIR                kopiera texten
319D  3E 0D               LD   A,$0D
319F  12                  LD   (DE),A         CR sist
31A0  7B                  LD   A,E            CR:s adress, låg byte (egenhet)
31A1  DD E5               PUSH IX             DE = bufferten
31A3  D1                  POP  DE
31A4  E1                  POP  HL             n
31A5  19                  ADD  HL,DE
31A6  01 53 00            LD   BC,$0053
31A9  09                  ADD  HL,BC          översta byten i s$:s plats
31AA  E5                  PUSH HL             = resultatets exponent
```

```
31AB  F5                PUSH AF
31AC  CD FB 14          CALL GETSFL          läs talet
31AF  DA 13 07          JP   C,L0713         inget tal: fel 12
31B2  1B                DEC  DE
31B3  E7                RST  $20             verkningslös: HL används inte sedan
31B4  F1                POP  AF
31B5  BB                CP   E               slutade talet vid CR?
31B6  C2 13 07          JP   NZ,L0713        nej: fel 12
31B9  D1                POP  DE              resultatets exponent
31BA  21 FC FF          LD   HL,$FFFC
31BD  19                ADD  HL,DE           resultatets början
31BE  D1                POP  DE              koden
31BF  F9                LD   SP,HL           ta bort resten
31C0  F7                RST  $30             nästa token
```

EXINP: INP(p) (*kod* $1C). *Först skrivs det sparade värdet för port 1* ($FE2B, *från* OUT 1,*v*) *till port 1, sedan läses port p.* OUT *gör likadant* (OUTST). *Port 1 väljer enhet på ABC-bussen, så läsningen går till den enhet som programmet senast valde. Förstör A, BC och HL.*

```
31C1  C1          EXINP: POP  BC             p
31C2  FD 7E 15           LD   A,(IY+$15)     sparat värde
31C5  D3 01              OUT  ($01),A        till port 1
31C7  ED 68              IN   L,(C)          läs port p
31C9  26 00              LD   H,$00          HL = värdet 0-255
31CB  EF                 RST  $28            heltalet på stacken, nästa token
```

EXCALL: CALL(a[,d]) (*kod* $1D) *anropar maskinkod på adress a med DE = d (0 om d saknas). HL vid återhoppet blir funktionens värde (heltal):* CALL(796,7) = *7 (*EX DE,HL *och* RET *på* $031C), CALL(796) = *0. Rutinen får ändra alla register utom SP och IY* ($FE16, *som tolken räknar med); koden sparas på stacken.*

```
31CC  D5          EXCALL: PUSH DE            koden
31CD  1B                  DEC  DE            tillbaka till token före funktionskoden
31CE  1B                  DEC  DE
31CF  1A                  LD   A,(DE)        token $CD + antal argument
31D0  FE CF               CP   $CF           två argument?
31D2  C1                  POP  BC            koden
31D3  11 00 00            LD   DE,$0000
31D6  20 01               JR   NZ,L31D9
31D8  D1                  POP  DE            d
31D9  E1          L31D9:  POP  HL            a
31DA  C5                  PUSH BC            koden vid SP
31DB  CD 52 28            CALL JPHL          JP (HL)
31DE  D1                  POP  DE            koden
31DF  EF                  RST  $28           resultatet i HL
```

3.88 EXRND: RND och RND(x) $31E0

Argumentet x används inte; dess fem byte blir en del av arbetsytan (13 byte reserveras med argument, 18 utan, så ytan är alltid 18 byte). Slumptalsfröet RNDSED ($FE39–$FE3D) är ett heltal på 40 bitar med den minst signifikanta byten på $FE3D.

1. Nytt frö (linjär kongruens, modulo 2^40):

   ```
   frö = frö * $F73B40E64D + $00A205B065
   ```

   Multiplikationen görs bit för bit. Faktorns byte tas från RNDMUL ($32E0) och nedåt, lägsta biten först. Är biten satt adderas fröet till summan på IX+0–4, och fröet skiftas ett steg åt vänster (på sin plats; det är 0 efteråt). Efter varje byte blir additionen en byte kortare, eftersom fröets lägsta byte då är 0. Faktorns högsta byte, $F7, är RST $30 på $32DC, den sista

instruktionen i rutinen. Att den används visar följden: från fröet 0 ger RND .079112, .106671, .452764 och så vidare, som med faktorn $F73B40E64D (med $3B40E64D blir andra talet .731671).

Sedan skrivs summan + konstanten (RNDADD, $32E5 och nedåt) tillbaka till fröet.

2. De 35 lägsta bitarna av fröet blir ett decimalt bråk 0.dddddd: för varje bit, lägsta först, blir bråket (bråk + bit) / 2, räknat i BCD: skift höger, +5 i översta siffran för en inskiftad bit, och -3 för en siffra som fick bit 3 från siffran över. Siffrorna efter den sjätte faller bort (avhuggning, inte avrundning: .452764 där bråket är .4527646...).
3. Bråket normaliseras (inledande nollor bort, exponenten minskas) och blir ett flyttal från och med 0 och mindre än 1. Om alla sex siffror är 0 (bråket under 0.000001, ungefär en gång på en miljon) slutar normaliseringen efter tre hela nollbyte med exponenten $7A i stället för 0. Talet skrivs ut som 0 men är ingen riktig nolla: X=0 är falskt, X>0 sant, SGN(X) ger 1 och X<1E-100 falskt.

Arbetsytan: IX+0–4 summan, IX+$0A/$0B pekare till fröet, IX+$0C antal byte kvar i faktorn, IX+$0D–$11 resultatet.

Kod $0A i EXPTAB2; token $CD före koden betyder RND utan argument. Prövat i ett program efter RUN: .079112 och .106671. In: DE = koden. Ut: flyttalet vid SP (RST $30). Förstör alla register utom DE och IY.

```
31E0  1B           EXRND:  DEC  DE              (fel)
31E1  1B                   DEC  DE
31E2  1A                   LD   A,(DE)          token före funktionskoden
31E3  13                   INC  DE
31E4  13                   INC  DE
31E5  DD 21 EE FF          LD   IX,$FFEE        18 byte
31E9  FE CD                CP   $CD             RND utan argument?
31EB  28 04                JR   Z,L31F1
31ED  DD 21 F3 FF          LD   IX,$FFF3        13 byte + x:s 5
31F1  DD 39        L31F1:  ADD  IX,SP
31F3  DD F9                LD   SP,IX           arbetsytan
31F5  D5                   PUSH DE              koden
31F6  DD E5                PUSH IX              HL = arbetsytan
31F8  E1                   POP  HL
31F9  01 00 12             LD   BC,$1200        18 byte, C = 0
31FC  71           L31FC:  LD   (HL),C          nollställ
31FD  23                   INC  HL
31FE  10 FC                DJNZ L31FC
3200  11 27 00             LD   DE,$0027        IY + $27 = $FE3D
3203  FD E5                PUSH IY
3205  E1                   POP  HL
3206  19                   ADD  HL,DE
3207  EB                   EX   DE,HL           fröets lägsta byte
3208  DD 73 0A             LD   (IX+$0A),E      pekare till fröet
320B  DD 72 0B             LD   (IX+$0B),D
320E  21 E0 32             LD   HL,RNDMUL       faktorns lägsta byte
3211  DD 36 0C 05          LD   (IX+$0C),$05    5 byte
3215  0E 08        L3215:  LD   C,$08           8 bitar
3217  7E                   LD   A,(HL)          nästa byte i faktorn
3218  E5                   PUSH HL              faktorns pekare
3219  D5          L3219:   PUSH DE              fröets pekare för additionen
321A  0F                   RRCA                 nästa bit
321B  F5                   PUSH AF
321C  30 1A                JR   NC,L3238        0: ingen addition
321E  DD E5                PUSH IX              summan
3220  26 00                LD   H,$00           HL = antal byte
3222  DD 6E 0C             LD   L,(IX+$0C)      antal byte
```

```
3225  45              LD   B,L          B = antal byte
3226  EB              EX   DE,HL        HL = fröet, DE = antal
3227  DD 19           ADD  IX,DE        IX = summan + antal
3229  7E        L3229: LD  A,(HL)       summa += frö
322A  DD 8E FF        ADC  A,(IX-$01)
322D  DD 77 FF        LD   (IX-$01),A
3230  2B              DEC  HL
3231  DD 2B           DEC  IX
3233  10 F4           DJNZ L3229
3235  DD E1           POP  IX           IX = arbetsytan
3237  AF              XOR  A            CY = 0
3238  DD 6E 0A  L3238: LD  L,(IX+$0A)   fröet
323B  DD 66 0B        LD   H,(IX+$0B)
323E  06 05           LD   B,$05
3240  CB 16     L3240: RL   (HL)        frö <<= 1
3242  2B              DEC  HL
3243  10 FB           DJNZ L3240
3245  F1              POP  AF           faktorns byte och CY
3246  D1              POP  DE           fröets pekare
3247  0D              DEC  C
3248  20 CF           JR   NZ,L3219
324A  E1              POP  HL           faktorns pekare
324B  1B              DEC  DE           nästa byte i fröet
324C  2B              DEC  HL           nästa byte i faktorn
324D  DD 35 0C        DEC  (IX+$0C)     en byte kortare
3250  20 C3           JR   NZ,L3215
3252  06 05           LD   B,$05        5 byte
3254  11 E5 32        LD   DE,RNDADD    konstantens lägsta byte
3257  DD 6E 0A        LD   L,(IX+$0A)   fröet
325A  DD 66 0B        LD   H,(IX+$0B)
325D  AF              XOR  A            CY = 0
325E  DD E5           PUSH IX           arbetsytan
3260  1A        L3260: LD  A,(DE)       frö = summa + konstant
3261  DD 8E 04        ADC  A,(IX+$04)
3264  77              LD   (HL),A
3265  2B              DEC  HL
3266  1B              DEC  DE
3267  DD 2B           DEC  IX
3269  10 F5           DJNZ L3260
326B  DD E1           POP  IX           arbetsytan
326D  FD E5           PUSH IY           IY = $FE16
326F  11 0D 00        LD   DE,$000D
3272  DD 19           ADD  IX,DE        IX = resultatet
3274  DD E3           EX   (SP),IX      IX = IY, resultatet på stacken
3276  1E 23           LD   E,$23        35 bitar
3278  DD 56 27  L3278: LD  D,(IX+$27)   nästa byte i fröet
327B  CB 1A     L327B: RR  D            nästa bit, lägsta först
327D  E1              POP  HL           resultatets mantissa
327E  E5              PUSH HL
327F  06 03           LD   B,$03        3 byte
3281  7E        L3281: LD  A,(HL)       två BCD-siffror
3282  30 02           JR   NC,L3286     ingen bit in
3284  C6 A0           ADD  A,$A0        +5 i översta siffran efter RRA
3286  1F        L3286: RRA              halvera
3287  CB 11           RL   C            spara utskiftad bit
3289  CB 5F           BIT  3,A          fick låga siffran bit 3?
328B  28 02           JR   Z,L328F
328D  C6 FD           ADD  A,$FD        -3
328F  CB 19     L328F: RR  C            biten tillbaka i CY
3291  77              LD   (HL),A
3292  23              INC  HL           nästa byte, lägre siffror
```

```
3293  10 EC              DJNZ L3281          tre byte, sex siffror
3295  1D                 DEC  E              alla bitar?
3296  28 0A              JR   Z,L32A2
3298  0C                 INC  C              bitar i byten
3299  79                 LD   A,C
329A  E6 07              AND  $07
329C  20 DD              JR   NZ,L327B
329E  DD 2B              DEC  IX             nästa byte i fröet
32A0  18 D6              JR   L3278

32A2  E1           L32A2: POP  HL            mantissan
32A3  0E 80              LD   C,$80          exponent 0 ($80)
32A5  7E           L32A5: LD   A,(HL)
32A6  A7                 AND  A
32A7  28 13              JR   Z,L32BC        hel byte 0
32A9  E6 F0              AND  $F0            första siffran 0?
32AB  20 26              JR   NZ,L32D3       nej: klart
32AD  11 02 00           LD   DE,$0002
32B0  06 03              LD   B,$03
32B2  19                 ADD  HL,DE
32B3  ED 6F        L32B3: RLD                 skifta en siffra
32B5  2B                 DEC  HL
32B6  10 FB              DJNZ L32B3
32B8  0D                 DEC  C              exponent - 1
32B9  23                 INC  HL
32BA  18 E9              JR   L32A5

32BC  79           L32BC: LD   A,C
32BD  FE 7A              CP   $7A            tre nollbyte: exponent $7A, ingen riktig nolla
32BF  28 12              JR   Z,L32D3
32C1  11 02 00           LD   DE,$0002
32C4  06 03              LD   B,$03
32C6  19                 ADD  HL,DE
32C7  AF                 XOR  A
32C8  5E           L32C8: LD   E,(HL)         skifta en byte
32C9  77                 LD   (HL),A
32CA  7B                 LD   A,E
32CB  2B                 DEC  HL
32CC  10 FA              DJNZ L32C8
32CE  0D                 DEC  C              exponent - 2
32CF  0D                 DEC  C
32D0  23                 INC  HL
32D1  18 D2              JR   L32A5

32D3  11 04 00     L32D3: LD   DE,$0004
32D6  EB                 EX   DE,HL
32D7  19                 ADD  HL,DE
32D8  71                 LD   (HL),C         exponenten
32D9  EB                 EX   DE,HL
32DA  D1                 POP  DE             koden
32DB  F9                 LD   SP,HL          resultatet vid SP
32DC  F7                 RST  $30

32DD  3B 40 E6           DEFB $3B,$40,$E6    faktorns byte 3-1; byte 4 = $F7 på $32DC
32E0  4D           RNDMUL: DEFB $4D            RNDMUL: lägsta byten i $F73B40E64D
32E1  00 A2 05 B0        DEFB $00,$A2,$05,$B0 konstanten $00A205B065, baklänges
32E5  65           RNDADD: DEFB $65            lägsta byten
```

Korsreferenstabell för SPACE$, STRING$, TAB, CUR, NUM$, VAL, INP, CALL och RND. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; $FE39 är slumptalsfröet (fem byte) och IY-kolumnerna är filnumret vid IY+$0D och det sparade värdet för port 1 vid IY+$15. Funktionerna nås genom

EXFUNC, tar DE efter sin kod och argumenten vid SP och lägger resultatet i deras ställe, med RST $28 (ett heltal) eller RST $30; som i tabellen efter $2A4B räknas det som nästa rutin gör inte, men EXEXP förstör A, flaggorna, BC och HL. STRFIL bygger en sträng med BC gånger tecknet i A. EXTAB läser kolumnen med GETPOS. EXCALL anropar maskinkod som får ändra allt utom SP och IY, också systemvariablerna. RNDMUL och RNDADD är faktorns och konstantens lägsta byte i EXRND. Felen: EXSPC, EXSTRG, STRFIL, EXTAB, EXNUM och EXVAL 3 (genom ROOM), EXVAL också 12. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler		IY+	
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	$FE39	$0D	$15
EXSPC	X	X	X	IU	X		IU	I			
EXSTRG	X	X	X	IU	X		IU	I			
STRFIL	IX	X	IX	IU	X		U	I			
EXTAB	X	X	X	IU	X	X	IU	I		I	
EXCUR	X	X	X	IU	X		IU				
EXNUM	X	X	X	IU	X	X	IU	I			
EXVAL	X	X	X	IU	X	X	IU	I			
EXINP	X	X	X	IU	X		IU				I
EXCALL	X	X	X	IU	X	X	IU				
EXRND	X	X	X	IU	X	X	IU		IU		

Rutin	**Anropar**	**Anropas från**
EXSPC	STRFIL (till $30FD, med JR)	EXPTAB2 (kod $16)
EXSTRG	går in i STRFIL	EXPTAB2 (kod $17)
STRFIL	ROOM, RST $30; fel 3	EXSTRG (går in i den), EXSPC (med JR), EXTAB (med JR, två ställen)
EXTAB	GETPOS, STRFIL (med JR)	EXPTAB2 (kod $20)
EXCUR	RST $30	EXPTAB2 (kod $21)
EXNUM	ROOM, FLTAS6, RST $30	EXPTAB2 (kod $18)
EXVAL	ROOM, GETSFL, RST $20, RST $30; fel 3 och 12	EXPTAB2 (kod $19)
EXINP	RST $28	EXPTAB2 (kod $1C)
EXCALL	JPHL (maskinkoden), RST $28	EXPTAB2 (kod $1D)
EXRND	RST $30	EXPTAB2 (kod $0A)

DIM1: *dimensionera en vektor med högsta index DE (IX = posten: +0/+1 adressen, +2/+3 högsta index, +4 elementets storlek). Finns den redan är det bara tillåtet att minska (annars fel 0), och inget händer: de gamla elementen går fortfarande att nå. Annars reserveras (DE+1) * storleken byte, nollställda (ALLOC). Högsta index 0 sparas som 0, så* DIM *A(0) räknas som odimensionerad: en andra* DIM *A(0) reserverar 5 byte till (prövat).* DIM *A(-1) ger 0 byte, och* ALLOC*:s* LDIR *med BC = $FFFF nollställer då hela minnet. DE bevaras; förstör A, B och HL.*

```
32E6  DD 6E 02     DIM1:   LD   L,(IX+$02)      +2/+3: högsta index, 0 = inte dimensionerad
32E9  DD 66 03             LD   H,(IX+$03)
32EC  7D                   LD   A,L
32ED  B4                   OR   H
32EE  28 05                JR   Z,L32F5         inte dimensionerad än
32F0  ED 52                SBC  HL,DE           gamla - nya (CY = 0 från OR)
32F2  38 50                JR   C,L3344         större: fel 0
32F4  C9                   RET                  lika eller mindre: inget händer

32F5  D5           L32F5:  PUSH DE              högsta index
32F6  13                   INC  DE              antal element
```

```
32F7  DD 46 04           LD   B,(IX+$04)      B = elementets storlek
32FA  19          L32FA: ADD  HL,DE           (DE+1) * storleken
32FB  38 24              JR   C,L3321         spill: fel 3
32FD  10 FB              DJNZ L32FA
32FF  EB                 EX   DE,HL           DE = antal byte
3300  CD 74 33           CALL ALLOC           reservera, nollställt
3303  D1                 POP  DE              högsta index
3304  DD 73 02           LD   (IX+$02),E      +2/+3: högsta index
3307  DD 72 03           LD   (IX+$03),D
330A  DD 75 00           LD   (IX+$00),L      +0/+1: adressen
330D  DD 74 01           LD   (IX+$01),H
3310  C9                 RET
```

DIM2: *två index, DE = första, HL = andra (IX = posten). Antal element = (DE+1)*(HL+1); för stort ger fel 3, liksom index -1. Sedan* DIM1 *och +6..+9.* DIM1 *jämför bara antalet element, och +6..+9 skrivs även när fältet redan finns:* DIM *A(3,2) efter* DIM *A(2,3) (12 element båda) ger inget fel utan byter formen, och elementen ligger kvar radvis (A(1,2) blir A(2,0)).* DIM *A(1,1) krymper formen på samma sätt. DE och HL bevaras; förstör A och BC.*

```
3311  E5           DIM2:  PUSH HL              andra index (fel)
3312  D5                  PUSH DE              första index
3313  23                  INC  HL              antal i varje led
3314  13                  INC  DE
3315  7D                  LD   A,L
3316  B4                  OR   H
3317  28 02               JR   Z,L331B         index -1: fel 3
3319  7B                  LD   A,E             DE+1 = 0?
331A  B2                  OR   D
331B  CA FF 06     L331B: JP   Z,L06FF         fel 3
331E  CD 74 37           CALL IMUL            (DE+1) * (HL+1)
3321  DA FF 06     L3321: JP   C,L06FF
3324  2B                 DEC  HL              DE = antal - 1
3325  EB                 EX   DE,HL
3326  CD E6 32           CALL DIM1            DE = antal - 1 som högsta index
3329  D1                 POP  DE              första index
332A  E1                 POP  HL              andra index
332B  DD 75 08           LD   (IX+$08),L      formen, även när fältet fanns
332E  DD 74 09           LD   (IX+$09),H
3331  DD 73 06           LD   (IX+$06),E      första index i +6/+7
3334  DD 72 07           LD   (IX+$07),D
3337  C9                 RET
```

DIMSTR: *reservera DE byte för strängen (IX = posten: +0/+1 reserverad längd, +2/+3 adressen). Finns den redan får den bara minska (annars fel 0).* DIM *A$=0 reserverar ingenting och lämnar +0/+1 = 0: strängen räknas som odimensionerad och får 80 byte vid första tilldelningen, som en sträng utan* DIM *(prövat). Minnesbrist ger fel 3. DE bevaras; förstör A och HL.*

```
3338  DD 6E 00   DIMSTR: LD   L,(IX+$00)      +0/+1: reserverad längd, 0 = ingen
333B  DD 66 01           LD   H,(IX+$01)
333E  7D                 LD   A,L
333F  B4                 OR   H
3340  28 06              JR   Z,L3348
3342  ED 52              SBC  HL,DE           gamla - nya (CY = 0 från OR)
3344  DA F9 06   L3344:  JP   C,L06F9         större: fel 0
3347  C9                 RET                  lika eller mindre: inget händer

3348  CD 5A 33   L3348:  CALL MEMRES          reservera
334B  38 D4              JR   C,L3321         ryms inte: fel 3
334D  DD 73 00           LD   (IX+$00),E      +0/+1: längden
3350  DD 72 01           LD   (IX+$01),D
3353  DD 75 02           LD   (IX+$02),L      +2/+3: adressen
```

```
3356  DD 74 03            LD   (IX+$03),H
3359  C9                  RET
```

MEMRES: *reservera DE byte på heapen. HEAP + DE + 100 måste vara under SP, annars CY = 1. Ut: HL = det reserverade området, och HEAP har flyttats fram. DE och BC bevaras; vid CY är HL ändrad men inte HEAP.*

ALLOC: *samma sak men området nollställs. Minnesbrist ger fel 3.* STKCHK: *fel 3 om SP före anropet ligger högst 100 byte över HEAP (94 + returadressen och de två* PUSH*), samma marginal som i MEMRES och ROOM. Den anropas av FN och GOSUB. ALLOC och STKCHK ändrar inga register utom flaggorna (ALLOC lämnar HL = området).*

```
335A  2A 20 FE     MEMRES: LD   HL,(HEAP)      HL = HEAP + DE
335D  19                   ADD  HL,DE
335E  D8                   RET  C              över 64 KB: CY
335F  D5                   PUSH DE
3360  11 64 00             LD   DE,$0064
3363  19                   ADD  HL,DE          minst 100 byte marginal mot stacken
3364  D1                   POP  DE
3365  D8                   RET  C              över 64 KB: CY
3366  ED 72                SBC  HL,SP          jämför med SP
3368  3F                   CCF                 CY: ryms inte
3369  D8                   RET  C
336A  2A 20 FE             LD   HL,(HEAP)      HL = det reserverade området
336D  E5                   PUSH HL
336E  19                   ADD  HL,DE          HEAP flyttas fram
336F  22 20 FE             LD   (HEAP),HL
3372  E1                   POP  HL
3373  C9                   RET

3374  CD 5A 33     ALLOC:  CALL MEMRES
3377  38 1F                JR   C,MEMFUL       ryms inte: fel 3
3379  E5                   PUSH HL             området
337A  D5                   PUSH DE             antal byte
337B  C5                   PUSH BC
337C  42                   LD   B,D            BC = antal
337D  4B                   LD   C,E
337E  54                   LD   D,H            DE = HL
337F  5D                   LD   E,L
3380  0B                   DEC  BC             BC = antal - 1
3381  13                   INC  DE
3382  36 00                LD   (HL),$00       första byten, LDIR sprider nollan
3384  ED B0                LDIR
3386  C1                   POP  BC
3387  18 0C                JR   L3395          återställ och RET (NC)

3389  E5           STKCHK: PUSH HL             SP - 2
338A  D5                   PUSH DE             SP - 4
338B  2A 20 FE             LD   HL,(HEAP)
338E  11 5E 00             LD   DE,$005E       94 + 6 byte på stacken = 100
3391  19                   ADD  HL,DE
3392  ED 72                SBC  HL,SP          HEAP + 94 - SP
3394  3F                   CCF                 CY: för nära
3395  D1           L3395:  POP  DE
3396  E1                   POP  HL
3397  D0                   RET  NC             det räcker
3398  C3 FF 06     MEMFUL: JP   L06FF          fel 3
```

ROOM: *fel 3 om HL (den nya SP) ligger mindre än 100 byte över HEAP. Används innan en sträng eller buffert byggs på stacken (CONCAT, STRFIL, EXNUM, EXVAL och ASCAR). HL och DE bevaras; förstör A.*

```
339B  E5           ROOM:   PUSH HL             den nya SP
339C  D5                   PUSH DE
```

```
339D  11 9C FF        LD   DE,$FF9C      DE = -100
33A0  19              ADD  HL,DE         HL - 100
33A1  7D              LD   A,L
33A2  FD 96 0A        SUB  (IY+$0A)      under HEAP?
33A5  7C              LD   A,H
33A6  FD 9E 0B        SBC  A,(IY+$0B)
33A9  18 EA           JR   L3395         återställ HL och DE, fel 3 vid CY
```

3.89 FNPAR: adressen till en DEF FN-parameter $33AB

In: HL = $00nn från koden (LNKEXP), nn = antalet parametrar efter den här. Ut: HL = adressen till motsvarande argument på stacken. Argumenten ligger på stacken med det sista överst, vid FNFRAM. Under FNFRAM ligger kodpekaren från FN, som pekar på avståndsbyten sist i anropet. Därifrån nås argumentposterna bakåt (se CMPFN):

```
IX+0  avstånd till förra posten (0 = den första)
IX+2  parameterns typ
```

ARGWLK hoppar över nn argument nedifrån, från SP-sidan, med storleken enligt typen: flyttal 5, heltal 2, sträng 6 + n. För ett flyttal pekar HL till sist på exponenten (+4), som för vanliga variabler.

FNRES använder ARGWLK med nn = $FF för att hoppa över alla. Båda bevarar DE (FNPAR kodpekaren, FNRES koden + 1, som den lade på stacken) och förstör A, BC och IX.

```
33AB  D5           FNPAR:  PUSH DE            koden
33AC  EB                   EX   DE,HL         E = nn
33AD  2A 3E FE             LD   HL,(FNFRAM)   ramen
33B0  2B           ARGWLK: DEC  HL
33B1  2B                   DEC  HL
33B2  CD 17 10             CALL RDBC          kodpekaren från FN
33B5  C5                   PUSH BC            IX = avståndsbyten
33B6  DD E1                POP  IX
33B8  7B           L33B8:  LD   A,E           framme?
33B9  A7                   AND  A
33BA  06 FF                LD   B,$FF         avstånden är negativa
33BC  DD 4E 00             LD   C,(IX+$00)    avståndet till förra posten
33BF  28 23                JR   Z,L33E4       framme: peka ut argumentet
33C1  79                   LD   A,C
33C2  A7                   AND  A
33C3  28 30                JR   Z,L33F5       ingen post till: klart
33C5  DD 09                ADD  IX,BC         förra posten
33C7  1D                   DEC  E
33C8  DD 7E 02             LD   A,(IX+$02)    parameterns typ
33CB  FE 02                CP   $02
33CD  28 0D                JR   Z,L33DC       sträng
33CF  0E 02                LD   C,$02         heltal: 2 byte
33D1  FE 01                CP   $01
33D3  28 02                JR   Z,L33D7
33D5  0E 05                LD   C,$05         flyttal: 5 byte
33D7  06 00        L33D7:  LD   B,$00
33D9  09                   ADD  HL,BC         hoppa över argumentet
33DA  18 DC                JR   L33B8

33DC  4E           L33DC:  LD   C,(HL)        n
33DD  23                   INC  HL
33DE  46                   LD   B,(HL)
33DF  09                   ADD  HL,BC         + n
33E0  0E 05                LD   C,$05         + 1 + 5 = 6 + n
33E2  18 F3                JR   L33D7
```

```
33E4  79           L33E4:  LD   A,C
33E5  A7                   AND  A
33E6  28 0D                JR   Z,L33F5          inga argument
33E8  DD 09                ADD  IX,BC            argumentets post
33EA  DD 7E 02             LD   A,(IX+$02)       argumentets typ
33ED  FE 00                CP   $00              flyttal?
33EF  20 04                JR   NZ,L33F5
33F1  01 04 00             LD   BC,$0004         peka på exponenten
33F4  09                   ADD  HL,BC
33F5  D1           L33F5:  POP  DE               koden
33F6  C9                   RET
```

Korsreferenstabell för DIM-rutinerna, minnesreserveringen och DEF FN-parametrarna. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna. DIM1, DIM2 och DIMSTR tar IX = variabelns post och skriver dess fält. MEMRES svarar med CY när det inte ryms; ALLOC, STKCHK och ROOM ger då fel 3. STKCHK och ROOM jämför SP och den nya SP (HL) med HEAP. FNPAR och ARGWLK går genom DEF FN-argumenten från FNFRAM; ARGWLK tar E = antalet argument att hoppa över och sist DE från stacken, som FNPAR eller FNRES lade där. * = kommer inte tillbaka; för den visas bara stacken, variablerna och det den tar emot. Felen: DIM1, DIM2 och DIMSTR 0 och 3, ALLOC, STKCHK, MEMFUL och ROOM 3. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler	
Rutin	A	CY	BC	DE	HL	IX	stack	HEAP	FNFRAM
DIM1	X	X	X	I	X	I		IU	
DIM2	X	X	X	I	I	I		IU	
DIMSTR	X	X		I	X	I		IU	
MEMRES		U		I	U			IU	
ALLOC		X		I	U			IU	
STKCHK		X					I	I	
MEMFUL*									
ROOM	X	X			I			I	
FNPAR	X	X	X		IU	X			I
ARGWLK	X	X	X	IU	IU	X	I		

Rutin	**Anropar**	**Anropas från**
DIM1	ALLOC; fel 0 (genom $3344 i DIMSTR) och 3 (genom $3321 i DIM2)	DIMELM (två ställen), VAR1, DIM2
DIM2	IMUL, DIM1; fel 3	DIMELM (två ställen), VAR2
DIMSTR	MEMRES; fel 0 och 3 (genom $3321 i DIM2)	DIMELM (två ställen), ASGNS
MEMRES		INSLIN, CMDMERG, DIMSTR, ALLOC
ALLOC	MEMRES, MEMFUL (med JR)	LNKFOR (två ställen), LNKNEW, PREPST, DIM1
STKCHK	går in i MEMFUL	FN, GOSUB
MEMFUL	fel 3	ALLOC (med JR), STKCHK (går in i den), ROOM (till $3395, med JR)
ROOM	STKCHK ($3395) och MEMFUL (med JR)	ASCAR, CONCAT, STRFIL, EXNUM, EXVAL
FNPAR	går in i ARGWLK	VAR0
ARGWLK	RDBC	FNPAR (går in i den), FNRES (med JP)

3.90 Flyttalsaritmetik: FADD FSUB FMUL FDIV $33F7

Talformat, fem byte. Adressen pekar på exponenten e:

```
e-4..e-2  mantissan, 6 BCD-siffror d1..d6 (d1 högst i e-4)
e-1       tecken, 0 = +, 1 = -
e         exponent med förskjutningen $80; 0 = talet är noll
värdet = 0,d1d2d3d4d5d6 * 10^(e - $80), normaliserat (d1 <> 0)
Exempel: 2 = $20 $00 $00 $00 $81,  PI = $31 $41 $59 $00 $81
```

In: DE = BC = a (vänster), HL = b (höger). Resultatet skrivs över a (BC). HL, DE och BC bevaras; AF och IX förstörs.

FENTER ($3766) sparar HL, DE och BC och ger IX = SP - $27. Rutinen sätter sedan SP = IX, så att arbetsytan ligger ovanför stacken:

```
IX+0..4      FDIV: kvoten (mantissa, tecken, exponent)
IX+0..$1F    FMUL: a gånger 1, 2, 4, 8, 10, 20, 40, 80, 4 byte  var
IX+$20       FMUL: räknare (annars 0)
IX+$21..$24  mantissan, med en extra byte (IX+$21) överst
IX+$25       tecken
IX+$26       exponent
IX+$27       FADD: <> 0 om förskjutningen är ett udda antal siffror
IX+$28       FADD: byten med den första siffran som skiftas ut
```

FSTORE ($3434) kopierar IX+$22–$26 till resultatet, tar bort arbetsytan och återställer registren. FDIV går in på FSTOR1 med HL = 4, så att kvoten på IX+0–4 kopieras i stället.

FOVFL ($3758): fel 4, FÖR STORT FLYTTAL (också för division med 0). FZERO ($375B): resultatet blir 0. Det går via FMULE, som nollställer mantissan och sätter exponenten till 0.

```
33F7  CD 66 37     FADD:   CALL FENTER          spara registren, IX = arbetsytan
33FA  DD F9                LD   SP,IX           SP under arbetsytan
33FC  C5                   PUSH BC              resultatets adress
33FD  CD BB 36             CALL FALIGN          DE = störst exponent
3400  0E 00                LD   C,$00           addition: tecknet byts inte
```

FADD / FSUB. FALIGN *har låtit DE peka på det tal som har störst exponent. Det andra talet ligger i IX+$22..$25, skiftat åt höger med skillnaden i exponent (högst 6 siffror), och C = 1 om talen bytte plats.* FSUB *har bytt tecken på kopian. Har talen bytt plats (C = 1) byts också resultatets tecken, eftersom b - a = -(a - b). Det gäller bara* FSUB*;* FADD *sätter C = 0.*

Den första siffran som skiftades ut avrundar: ADD *A,*$B0 *ger CY om den är >= 5, och den CY:n går in i additionen (*ADC*) eller subtraktionen (*SBC*) som en etta i sista siffran.* RRA/RLA *tar bit 0 av B (olika tecken?) utan att tappa CY.*

Samma tecken: MADD3 *adderar. Spill ger en etta i IX+$21; mantissan skiftas en siffra åt höger och exponenten ökar (fel 4 vid* $FF*). Siffran som skiftas ut då avrundar inte. Egenheter (provat):*

- *Skillnaden begränsas till 6, så ett hur litet tal som helst kan avrunda:* `1 + 5E-51 = 1.00001`*, men* `1 + 4E-51 = 1`*.*
- *Bara den mindre operanden avrundas:* `1 - .999999 = 0`*, eftersom* `.999999` *blir* `1.00000` *före subtraktionen.* `1 - 5E-6 = .99999`*.*
- *Vid spill kapas summan:* `.999999 + .999999 = 1.99999`*, inte 2.*

```
3402  1B           FADDC:  DEC  DE              DE = teckenbyten (gränsfall)
3403  EB                   EX   DE,HL           HL = det större talets tecken
3404  DD 7E 25             LD   A,(IX+$25)      kopians tecken
3407  AE                   XOR  (HL)            XOR det större talets tecken
3408  47                   LD   B,A             B bit 0 = olika tecken
3409  EB                   EX   DE,HL           DE = teckenbyten igen
340A  1A                   LD   A,(DE)          tecknet för talet vid DE
340B  1B                   DEC  DE              DE = lägsta mantissabyten
340C  A9                   XOR  C               byt om talen bytte plats
```

```
340D  DD 77 25           LD   (IX+$25),A      resultatets tecken
3410  DD 7E 27           LD   A,(IX+$27)      udda förskjutning?
3413  B7                 OR   A               Z = jämn förskjutning
3414  DD 7E 28           LD   A,(IX+$28)      byten med utskiftad siffra
3417  28 04              JR   Z,L341D         jämn: siffran står i den höga halvan
3419  07                 RLCA                 den låga halvan upp
341A  07                 RLCA
341B  07                 RLCA
341C  07                 RLCA
341D  C6 B0        L341D: ADD  A,$B0          CY = första utskiftade siffran >= 5
341F  78                 LD   A,B             bit 0 = olika tecken
3420  1F                 RRA                  CY = olika tecken, A bit 7 = avrundning
3421  38 50              JR   C,FSUBM         olika tecken: subtrahera
3423  17                 RLA                  CY = avrundning
3424  CD 47 34           CALL MADD3           addera
3427  30 0B              JR   NC,FSTORE       inget spill
3429  06 04              LD   B,$04           en siffra
342B  CD 14 37           CALL MSHR            skifta åt höger; siffran som faller ut avrundar inte
342E  DD 34 26           INC  (IX+$26)        exponent + 1
3431  CA 58 37           JP   Z,FOVFL         $FF + 1: fel 4
3434  21 26 00   FSTORE: LD   HL,$0026        mantissa, tecken, exponent vid IX+$22..$26
3437  D1         FSTOR1: POP  DE              resultatets adress
3438  39                 ADD  HL,SP           HL = IX+$26 (SP = IX efter POP)
3439  01 05 00           LD   BC,$0005        5 byte
343C  ED B8                LDDR                 exponenten först, sedan nedåt
343E  21 29 00           LD   HL,$0029        förbi arbetsytan
3441  39                 ADD  HL,SP
3442  F9                 LD   SP,HL           SP vid det sparade BC
3443  C1                 POP  BC              återställ registren
3444  D1                 POP  DE
3445  E1                 POP  HL
3446  C9                 RET
```

MADD3: *IX+$22..$24 += de 3 byten vid DE (DE pekar på den lägsta), BCD med* ADC *och* DAA. *CY in läggs till i sista siffran. Spill går in i IX+$21. Ut: CY = spill, DE 3 byte lägre. Förstör A och B.*

```
3447  DD E5        MADD3: PUSH IX
3449  06 03               LD   B,$03           3 byte
344B  1A           L344B: LD   A,(DE)          nästa byte, den lägsta först
344C  DD 8E 24             ADC  A,(IX+$24)     + CY
344F  27                   DAA
3450  DD 77 24             LD   (IX+$24),A
3453  DD 2B                DEC  IX              nästa högre byte
3455  1B                   DEC  DE
3456  10 F3                DJNZ L344B
3458  30 03                JR   NC,L345D        inget spill
345A  DD 34 24             INC  (IX+$24)        spill till IX+$21
345D  DD E1        L345D:  POP  IX
345F  C9                   RET
```

FSUB: a - b. Som FADD, men tecknet på kopian av det mindre talet byts innan additionen. In och ut som FADD; C från FALIGN står kvar och byter resultatets tecken om talen bytte plats.

```
3460  CD 66 37     FSUB:  CALL FENTER          spara registren, IX = arbetsytan
3463  DD F9               LD   SP,IX           SP under arbetsytan
3465  C5                  PUSH BC              resultatets adress
3466  CD BB 36            CALL FALIGN          DE = störst exponent
3469  DD 7E 25            LD   A,(IX+$25)      byt tecken på kopian
346C  EE 01               XOR  $01
346E  DD 77 25            LD   (IX+$25),A
3471  18 8F               JR   FADDC           C = 1 om talen bytte plats
```

FSUBM: *olika tecken, subtrahera. Blir det lån var b större än a (bara möjligt vid samma exponent): tecknet byts och mantissan byts mot 0 - mantissan (tiokomplement; XOR nollställer CY först).*

FNORM ($3494): *räkna inledande nollor i IX+$22..$24 (C = antal siffror). Är alla noll blir resultatet 0. Annars minskas exponenten med C (blir den <= 0 blir resultatet 0) och mantissan skiftas åt vänster. Ingen siffra skiftas in, så en avrundad siffra från den mindre operanden kommer aldrig tillbaka.*

```
3473  17           FSUBM:  RLA             CY = avrundning
3474  CD CB 34             CALL MSUB3      subtrahera
3477  30 1B                JR   NC,FNORM   inget lån
3479  DD 7E 25             LD   A,(IX+$25) byt tecken
347C  EE 01                XOR  $01
347E  DD 77 25             LD   (IX+$25),A
3481  DD E5                PUSH IX
3483  06 03                LD   B,$03      3 byte
3485  3E 00          L3485:  LD   A,$00      0 - mantissan (XOR gav CY = 0)
3487  DD 9E 24             SBC  A,(IX+$24)
348A  27                   DAA
348B  DD 77 24             LD   (IX+$24),A
348E  DD 2B                DEC  IX          nästa högre byte
3490  10 F3                DJNZ L3485
3492  DD E1                POP  IX
3494  DD E5          FNORM:  PUSH IX         (fel)
3496  01 00 03             LD   BC,$0300   B = 3 byte, C = 0 nollor
3499  DD 7E 22       L3499:  LD   A,(IX+$22) första siffran?
349C  B7                   OR   A
349D  20 0E                JR   NZ,L34AD
349F  DD 23                INC  IX          nästa byte
34A1  0C                   INC  C           två nollor till
34A2  0C                   INC  C
34A3  10 F4                DJNZ L3499       nästa byte
34A5  DD E1                POP  IX
34A7  AF                   XOR  A           alla noll: resultatet 0
34A8  DD 77 26             LD   (IX+$26),A
34AB  18 87                JR   FSTORE

34AD  DD E1          L34AD:  POP  IX
34AF  FE 10                CP   $10         första siffran också 0?
34B1  30 01                JR   NC,L34B4
34B3  0C                   INC  C
34B4  DD 7E 26       L34B4:  LD   A,(IX+$26) exponent - antal nollor
34B7  91                   SUB  C
34B8  CA 5B 37             JP   Z,FZERO     <= 0: resultatet 0
34BB  DA 5B 37             JP   C,FZERO
34BE  DD 77 26             LD   (IX+$26),A
34C1  79                   LD   A,C         4 bitar per siffra
34C2  07                   RLCA
34C3  07                   RLCA
34C4  47                   LD   B,A         B = 4 * antalet nollor
34C5  CD 36 37             CALL MSHL        skifta bort nollorna
34C8  C3 34 34             JP   FSTORE
```

MSUB3: *IX+$22..$24 = de 3 byten vid DE minus IX+$22..$24, BCD med SBC och DAA. CY in dras också av. Ut: CY = lån, DE 3 byte lägre. Förstör A och B.*

```
34CB  DD E5          MSUB3:  PUSH IX
34CD  06 03                LD   B,$03      3 byte
34CF  1A             L34CF:  LD   A,(DE)
34D0  DD 9E 24             SBC  A,(IX+$24) - CY
34D3  27                   DAA
34D4  DD 77 24             LD   (IX+$24),A
34D7  DD 2B                DEC  IX
```

```
34D9  1B                DEC  DE
34DA  10 F3             DJNZ L34CF
34DC  DD E1             POP  IX
34DE  C9                RET
```

FMUL: *exponenten = ea + eb -* $80 *(fel 4 om den blir >* $FF*, 0 om den blir < 1). Är något tal 0 blir resultatet 0. Tecknet =* XOR*.*

En tabell med a gånger 1, 2, 4, 8, 10, 20, 40 och 80 byggs på IX+0.. $1F*, med BCD-dubblering (*ADC *A,A och* DAA*) och 10a = 8a + 2a. b tas en byte (två siffror) i taget, den lägsta först. Varje bit i byten, den högsta först, motsvarar ett tabellvärde (80, 40 ... 2, 1), och* MADD3 *lägger till det i IX+*$21..$24*. Efter varje byte skiftas summan en byte åt höger (*MSHR*); den utskiftade byten finns kvar i D.*

*Efter tre byte är mantissan klar (*FMRND*).*

*Exponenten prövas före multiplikationen, så fel 4 kommer också när produkten efter normaliseringen hade fått plats (provat: .1E127 * .1E1 = fel 4, fast .1E127 finns). Åt andra hållet går det rätt: .1E-127 * .1 blir 0.*

```
34DF  CD 66 37     FMUL:   CALL FENTER          spara registren, IX = arbetsytan (gränsfall)
34E2  DD F9                LD   SP,IX           SP under arbetsytan
34E4  C5                   PUSH BC              resultatets adress
34E5  7E                   LD   A,(HL)          eb
34E6  B7                   OR   A
34E7  28 11                JR   Z,FMULE         b = 0: resultatet 0
34E9  1A                   LD   A,(DE)          ea
34EA  B7                   OR   A
34EB  28 0D                JR   Z,FMULE         a = 0: resultatet 0
34ED  86                   ADD  A,(HL)          ea + eb
34EE  38 05                JR   C,L34F5         ea + eb >= $100
34F0  F2 5B 37             JP   P,FZERO         < $80: resultatet 0
34F3  18 03                JR   L34F8           ea + eb = $80..$FF

34F5  FA 58 37     L34F5:  JP   M,FOVFL         för stort: fel 4
34F8  D6 80        L34F8:  SUB  $80             - $80
34FA  DD 77 26     FMULE:  LD   (IX+$26),A      exponenten
34FD  1B                   DEC  DE              DE = a:s tecken
34FE  2B                   DEC  HL              HL = b:s tecken
34FF  1A                   LD   A,(DE)          teckenbyten
3500  AE                   XOR  (HL)            tecknet = XOR
3501  2B                   DEC  HL              HL = b:s lägsta mantissabyte
3502  1B                   DEC  DE              DE = a:s lägsta mantissabyte
3503  DD 77 25             LD   (IX+$25),A
3506  DD E5                PUSH IX
3508  AF                   XOR  A               nollställ IX+$20..$24
3509  06 05                LD   B,$05
350B  DD 77 24     L350B:  LD   (IX+$24),A
350E  DD 2B                DEC  IX
3510  10 F9                DJNZ L350B
3512  DD E1                POP  IX
3514  DD 7E 26             LD   A,(IX+$26)      exponent 0?
3517  B7                   OR   A               ja: resultatet 0
3518  CA 34 34             JP   Z,FSTORE
351B  01 03 00             LD   BC,$0003        3 byte
351E  DD E5                PUSH IX              (SP) = b:s mantissa
3520  E3                   EX   (SP),HL
3521  09                   ADD  HL,BC           HL = IX+3
3522  1A           L3522:  LD   A,(DE)          a:s mantissa till IX+1..3
3523  77                   LD   (HL),A
3524  2B                   DEC  HL
3525  1B                   DEC  DE
3526  0D                   DEC  C
3527  20 F9                JR   NZ,L3522
3529  71                   LD   (HL),C          IX+0 = 0
```

```
352A  2B              DEC   HL
352B  06 FA           LD    B,$FA          -6: 2a, 4a, 8a
352D  11 04 00  L352D: LD   DE,$0004       4 byte per tabellvärde
3530  4B              LD    C,E            C = 4 byte
3531  19              ADD   HL,DE
3532  EB              EX    DE,HL
3533  19              ADD   HL,DE          DE = förra värdet, HL = nästa
3534  04              INC   B              nästa värde
3535  F2 68 35        JP    P,L3568        färdig
3538  1A        L3538: LD   A,(DE)         dubbla: ADC A,A och DAA
3539  8F              ADC   A,A
353A  27              DAA
353B  77              LD    (HL),A
353C  1B              DEC   DE
353D  2B              DEC   HL
353E  0D              DEC   C
353F  20 F7           JR    NZ,L3538
3541  04              INC   B              nästa värde
3542  20 E9           JR    NZ,L352D
3544  23              INC   HL
3545  EB              EX    DE,HL
3546  21 14 00        LD    HL,$0014       IX+$10
3549  39              ADD   HL,SP
354A  EB              EX    DE,HL
354B  0E 04           LD    C,$04          kopiera 8a till IX+$10
354D  41              LD    B,C
354E  7E        L354E: LD   A,(HL)
354F  12              LD    (DE),A
3550  23              INC   HL
3551  13              INC   DE
3552  0D              DEC   C
3553  20 F9           JR    NZ,L354E
3555  21 0B 00        LD    HL,$000B       2a, lägsta byten (IX+7)
3558  39              ADD   HL,SP
3559  1B              DEC   DE             DE = IX+$13, kopians lägsta byte
355A  1A        L355A: LD   A,(DE)         10a = 8a + 2a
355B  8E              ADC   A,(HL)
355C  27              DAA
355D  12              LD    (DE),A
355E  1B              DEC   DE
355F  2B              DEC   HL
3560  05              DEC   B
3561  20 F7           JR    NZ,L355A
3563  06 F9           LD    B,$F9          -7: 20a, 40a, 80a
3565  EB              EX    DE,HL
3566  18 C5           JR    L352D          vidare från 10a

3568  EB        L3568: EX   DE,HL
3569  23              INC   HL
356A  36 04           LD    (HL),$04       IX+$20: räknare, 3 byte
356C  C1        FMLOOP: POP BC             BC = b:s mantissa
356D  DD 35 20        DEC   (IX+$20)       alla byte?
3570  28 2B           JR    Z,FMRND        alla tre: avrunda
3572  0A              LD    A,(BC)         nästa byte i b, lägsta först
3573  0B              DEC   BC
3574  21 21 00        LD    HL,$0021       80a, lägsta byten: IX+$1F (SP = IX-2)
3577  39              ADD   HL,SP
3578  C5              PUSH  BC
3579  EB        L3579: EX   DE,HL          DE = tabellvärdets lägsta byte
357A  87        L357A: ADD  A,A            nästa bit, högsta först
357B  38 08           JR    C,L3585        biten satt
```

```
357D  28 17              JR   Z,L3596          inga bitar kvar
357F  21 FC FF           LD   HL,$FFFC        nästa tabellvärde
3582  19                 ADD  HL,DE           fyra byte nedåt
3583  18 F4              JR   L3579

3585  4F           L3585: LD   C,A             spara bitarna
3586  B7                 OR   A               CY = 0
3587  CD 47 34           CALL MADD3           summan += tabellvärdet
358A  1A                 LD   A,(DE)          översta byten
358B  DD 86 21           ADD  A,(IX+$21)      + summans översta byte
358E  27                 DAA
358F  DD 77 21           LD   (IX+$21),A
3592  79                 LD   A,C             bitarna
3593  1B                 DEC  DE              nästa tabellvärde
3594  18 E4              JR   L357A           nästa bit

3596  06 08        L3596: LD   B,$08          en byte
3598  CD 14 37           CALL MSHR            skifta åt höger, D = utskiftad byte
359B  18 CF              JR   FMLOOP
```

FMRND: *avrunda med de två siffrorna i D. Är d1 <> 0 avrundar D:s höga siffra. Är d1 = 0 skiftas mantissan en siffra åt vänster (exponenten minskar) och A = D med halvorna bytta: den låga halvan (D:s höga siffra) fyller den tomma sista siffran, och den höga halvan (D:s låga siffra) avrundar. Avrundningen: >= $50 ger* INC *A (+1), sedan läggs A:s låga halva till sista siffran. Spill ger mantissan .100000 och exponenten + 1. Provat: .111111 * .111111 = .123457E-1.*

```
359D  DD 7E 22     FMRND: LD   A,(IX+$22)      d1
35A0  E6 F0               AND  $F0
35A2  28 05               JR   Z,L35A9         d1 = 0: normalisera
35A4  7A                  LD   A,D             avrundning: D:s höga siffra
35A5  E6 F0               AND  $F0
35A7  18 10               JR   L35B9

35A9  06 04        L35A9: LD   B,$04
35AB  DD 35 26            DEC  (IX+$26)        exponent - 1
35AE  CA 5B 37            JP   Z,FZERO         0: resultatet 0
35B1  CD 36 37            CALL MSHL            skifta åt vänster
35B4  7A                  LD   A,D             låg halva: D:s höga siffra, hög: den låga
35B5  0F                  RRCA
35B6  0F                  RRCA
35B7  0F                  RRCA
35B8  0F                  RRCA
35B9  FE 50        L35B9: CP   $50             avrunda uppåt?
35BB  38 26               JR   C,L35E3
35BD  3C                  INC  A               +1
35BE  E6 0F               AND  $0F             siffran till sista platsen
35C0  06 03               LD   B,$03           3 byte
35C2  DD E5               PUSH IX
35C4  DD 8E 24     L35C4: ADC  A,(IX+$24)      lägg till
35C7  27                  DAA
35C8  DD 77 24            LD   (IX+$24),A
35CB  3E 00               LD   A,$00           nästa byte: bara CY
35CD  DD 2B               DEC  IX
35CF  10 F3               DJNZ L35C4
35D1  DD E1               POP  IX
35D3  D2 34 34            JP   NC,FSTORE       inget spill
35D6  DD 36 22 10         LD   (IX+$22),$10    mantissan .100000
35DA  DD 34 26            INC  (IX+$26)        exponent + 1
35DD  C2 34 34            JP   NZ,FSTORE       exponenten inte $FF + 1
35E0  C3 58 37            JP   FOVFL           fel 4

35E3  E6 0F        L35E3: AND  $0F             bara siffran till sista platsen
```

```
35E5  DD 86 24           ADD  A,(IX+$24)
35E8  DD 77 24           LD   (IX+$24),A
35EB  C3 34 34           JP   FSTORE
```

FDIV: *division med 0 ger fel 4, a = 0 ger 0. Exponenten blir ea - eb + $81 (fel 4 eller 0 om den hamnar utanför). a:s mantissa är resten i IX+$21..$24 (FCOPY), och tecknet = XOR läggs i IX+3.* FDLOOP ($3622)*: för varje siffra dras b från resten tills det blir lån. Då läggs b tillbaka (MADD3), och antalet lyckade subtraktioner är siffran. Resten skiftas en siffra åt vänster. Blir den första siffran 0 minskas exponenten och siffran räknas om.*

L räknar siffrorna från 7 och ned. Siffrorna packas två och två i IX+0..2 (udda L: låg halva, jämnt L: föregående siffra flyttas upp). Den sjunde siffran avrundar (FDRND).

*Egenhet (provat): exponenten prövas bara före $81 läggs till. Är ea - eb = $7F blir den $00 utan fel. Blir första siffran 0 räknas den ned till $FF och kvoten blir rätt (1E126 / .2 = 5E126). Annars står exponenten kvar som 0 med mantissan ifylld: 1E126 / .1 ger ett tal som räknas som 0 (X = 0 är sant, SGN 0, X * 2 = 0) men skrivs ut som 1 (9E126 / .1 som 9), i st. f. fel 4. Likadant i 9913.*

```
35EE  CD 66 37   FDIV:    CALL FENTER          spara registren, IX = arbetsytan (fel)
35F1  DD F9              LD   SP,IX           SP under arbetsytan
35F3  C5                 PUSH BC              resultatets adress
35F4  7E                 LD   A,(HL)          eb
35F5  B7                 OR   A
35F6  CA 58 37           JP   Z,FOVFL         division med 0: fel 4
35F9  1A                 LD   A,(DE)          ea
35FA  B7                 OR   A
35FB  CA FA 34           JP   Z,FMULE         a = 0: resultatet 0
35FE  96                 SUB  (HL)            ea - eb
35FF  38 05              JR   C,L3606
3601  FA 58 37           JP   M,FOVFL         för stort: fel 4
3604  18 03              JR   L3609           ea - eb = $00..$7F

3606  F2 5B 37   L3606:  JP   P,FZERO         för litet: 0
3609  C6 81      L3609:  ADD  A,$81           + $81
360B  DD 77 04           LD   (IX+$04),A      kvotens exponent
360E  EB                 EX   DE,HL           HL = a, DE = b
360F  D5                 PUSH DE              spara b
3610  CD FC 36           CALL FCOPY           a:s mantissa blir resten
3613  D1                 POP  DE              DE = b
3614  EB                 EX   DE,HL           HL = b, DE = a:s exponent - 5
3615  DD 7E 25           LD   A,(IX+$25)      a:s tecken
3618  2B                 DEC  HL              HL = b:s tecken
3619  AE                 XOR  (HL)            XOR b:s tecken
361A  DD 77 03           LD   (IX+$03),A      kvotens tecken
361D  EB                 EX   DE,HL           DE = b:s tecken
361E  1B                 DEC  DE              DE = b:s lägsta byte
361F  DD E5              PUSH IX
3621  C1                 POP  BC              BC = IX: kvotens första byte
3622  2E 07      FDLOOP: LD   L,$07           7 siffror
3624  C5         L3624:  PUSH BC              kvotens byte
3625  E5                 PUSH HL              L = siffror kvar
3626  0E 00              LD   C,$00           C = siffran
3628  A7         L3628:  AND  A               CY = 0
3629  DD E5              PUSH IX
362B  06 03              LD   B,$03
362D  DD 7E 24   L362D:  LD   A,(IX+$24)      resten - b
3630  EB                 EX   DE,HL           HL = b
3631  9E                 SBC  A,(HL)
3632  EB                 EX   DE,HL
3633  27                 DAA
3634  DD 77 24           LD   (IX+$24),A
3637  DD 2B              DEC  IX
```

```
3639  1B              DEC  DE
363A  10 F1           DJNZ L362D
363C  DD E1           POP  IX
363E  DD 7E 21        LD   A,(IX+$21)      lån från översta byten
3641  DE 00           SBC  A,$00
3643  DD 77 21        LD   (IX+$21),A
3646  1F              RRA                   spara CY
3647  21 03 00        LD   HL,$0003         DE tillbaka
364A  19              ADD  HL,DE
364B  EB              EX   DE,HL
364C  0C              INC  C                en subtraktion till
364D  17              RLA                   CY = lån
364E  30 D8           JR   NC,L3628         inget lån: igen
3650  B7              OR   A                CY = 0
3651  CD 47 34        CALL MADD3            lägg tillbaka b
3654  21 03 00        LD   HL,$0003
3657  19              ADD  HL,DE
3658  EB              EX   DE,HL
3659  C5              PUSH BC
365A  06 04           LD   B,$04            en siffra
365C  CD 36 37        CALL MSHL             resten * 10
365F  C1              POP  BC
3660  0D              DEC  C                siffran
3661  E1              POP  HL
3662  61              LD   H,C              H = siffran, L = siffror kvar
3663  C1              POP  BC
3664  7D              LD   A,L              första siffran och 0?
3665  20 0C           JR   NZ,L3673         siffran inte 0
3667  FE 07           CP   $07
3669  20 08           JR   NZ,L3673         inte den första siffran
366B  DD 35 04        DEC  (IX+$04)         exponent - 1
366E  CA 5B 37        JP   Z,FZERO          0: resultatet 0
3671  18 AF           JR   FDLOOP           börja om

3673  FE 01     L3673: CP   $01              sjunde siffran?
3675  28 12           JR   Z,FDRND
3677  1F              RRA                   CY = udda L
3678  7C              LD   A,H              A = siffran
3679  38 0A           JR   C,L3685          udda: ny byte
367B  0A              LD   A,(BC)           förra siffran
367C  07              RLCA
367D  07              RLCA
367E  07              RLCA
367F  07              RLCA                  upp till höga halvan
3680  84              ADD  A,H              + den nya
3681  02              LD   (BC),A
3682  03              INC  BC               nästa byte i kvoten
3683  18 01           JR   L3686

3685  02        L3685: LD   (BC),A          siffran i låga halvan
3686  2D        L3686: DEC  L               siffror kvar
3687  18 9B           JR   L3624
```

FDRND: *sjunde siffran > 5 avrundar uppåt, < 5 nedåt. Exakt 5 avrundar uppåt om inte bit 7 i IY+3* ($FE19) *är satt. Inget annat i ROM:en läser eller skriver* $FE19, *och* INIT *nollställer den inte, så det som RAM-kretsarna råkar innehålla efter start avgör. Värdet står kvar genom* RUN, CLEAR *och* NEW (*provat*). *Spill ger .100000 och exponent + 1.*

Provat: .246913 / 2 = .123456 med $FE19 = $FF, *.123457 med* $7F (POKE *65049,0 i BASIC*).

```
3689  7C        FDRND: LD   A,H              sjunde siffran (fel)
368A  FE 05           CP   $05
```

```
368C  38 27              JR   C,L36B5        < 5: klart
368E  20 06              JR   NZ,L3696       > 5: avrunda
3690  FD B6 03           OR   (IY+$03)       = 5: bit 7 i $FE19 hindrar
3693  FA B5 36           JP   M,L36B5        satt: ingen avrundning
3696  DD E5        L3696: PUSH IX
3698  11 03 00           LD   DE,$0003       IX+3
369B  DD 19              ADD  IX,DE
369D  DD E3              EX   (SP),IX
369F  E1                 POP  HL             HL = IX+3
36A0  06 03              LD   B,$03          3 byte
36A2  37                 SCF                 +1
36A3  2B           L36A3: DEC  HL            från IX+2, den lägsta byten
36A4  7E                 LD   A,(HL)
36A5  CE 00              ADC  A,$00          + CY
36A7  27                 DAA
36A8  77                 LD   (HL),A
36A9  10 F8              DJNZ L36A3
36AB  30 08              JR   NC,L36B5       inget spill
36AD  DD 34 04           INC  (IX+$04)       exponent + 1
36B0  CA 58 37           JP   Z,FOVFL        fel 4
36B3  36 10              LD   (HL),$10       IX+0 = $10
36B5  21 04 00     L36B5: LD   HL,$0004      kvoten vid IX+0..4
36B8  C3 37 34           JP   FSTOR1
```

FALIGN: *DE och HL pekar på två tals exponenter. Ut: DE = talet med störst exponent (C = 1 om DE och HL bytte plats), IX+$26 = dess exponent. Det andra talet kopieras (FCOPY) och skiftas åt höger med skillnaden, högst 6 siffror. B = 4 * skillnaden (bitar). IX+$28 får byten där den första utskiftade siffran låg, och IX+$27 säger om den är den låga halvan (udda skillnad). Vid skillnaden 0 är IX+$28 = 0. In: IX = arbetsytan. Förstör A och HL.*

```
36BB  1A           FALIGN: LD   A,(DE)        skillnad i exponent
36BC  96                   SUB  (HL)
36BD  0E 00                LD   C,$00         C = 0: ingen byte
36BF  30 04                JR   NC,L36C5
36C1  0C                   INC  C             DE mindre: byt
36C2  EB                   EX   DE,HL
36C3  2F                   CPL                skillnaden positiv
36C4  3C                   INC  A
36C5  47           L36C5:  LD   B,A           B = skillnaden
36C6  1A                   LD   A,(DE)        den största exponenten
36C7  DD 77 26             LD   (IX+$26),A
36CA  78                   LD   A,B
36CB  FE 06                CP   $06           högst 6 siffror
36CD  38 02                JR   C,L36D1
36CF  3E 06                LD   A,$06
36D1  07           L36D1:  RLCA               4 bitar per siffra
36D2  07                   RLCA
36D3  47                   LD   B,A
36D4  E6 04                AND  $04           udda antal siffror?
36D6  DD 77 27             LD   (IX+$27),A
36D9  D5                   PUSH DE            det större talet
36DA  C5                   PUSH BC            B = bitar, C = 1 om bytt
36DB  CD FC 36             CALL FCOPY         kopiera det mindre talet
36DE  C1                   POP  BC            B = bitar
36DF  C5                   PUSH BC
36E0  3E 28                LD   A,$28         vilken byte skiftas ut först
36E2  90                   SUB  B
36E3  FE 28                CP   $28           ingen förskjutning
36E5  28 0F                JR   Z,L36F6
36E7  E6 F8                AND  $F8
```

```
36E9  1F                  RRA                 byte = (40 - B) / 8
36EA  1F                  RRA
36EB  1F                  RRA
36EC  83                  ADD  A,E            DE = IX+$20 + byte
36ED  5F                  LD   E,A
36EE  7A                  LD   A,D
36EF  CE 00               ADC  A,$00
36F1  57                  LD   D,A
36F2  1A                  LD   A,(DE)         byten med första utskiftade siffran
36F3  DD 77 28            LD   (IX+$28),A
36F6  CD 14 37    L36F6:  CALL MSHR           skifta åt höger
36F9  C1                  POP  BC
36FA  D1                  POP  DE
36FB  C9                  RET
```

FCOPY: *kopiera tecken och mantissa för talet vid HL till IX+$22..$25. IX+$20, IX+$21 och IX+$28 nollställs. In: HL = talets exponent. Ut: DE = IX+$20, HL = exponenten - 5, BC = 0, A = 0.*

```
36FC  DD E5       FCOPY:  PUSH IX
36FE  11 25 00            LD   DE,$0025       DE = IX+$25
3701  DD 19               ADD  IX,DE
3703  DD E3               EX   (SP),IX
3705  D1                  POP  DE
3706  01 04 00            LD   BC,$0004       tecken och mantissa
3709  2B                  DEC  HL             HL = teckenbyten
370A  ED B8               LDDR
370C  AF                  XOR  A              IX+$21 = 0
370D  12                  LD   (DE),A
370E  1B                  DEC  DE
370F  12                  LD   (DE),A         IX+$20 = 0
3710  DD 77 28            LD   (IX+$28),A     IX+$28 = 0
3713  C9                  RET
```

MSHR: *skifta IX+$21..$24 åt höger, B/4 siffror (B = 4 per siffra).*

Hela byte flyttas först; den sista utskiftade byten ligger sedan kvar i D. En siffra till skiftas med RRD. *Utskiftade siffror försvinner. Förstör A, BC, D och HL.*

```
3714  0E 04       MSHR:   LD   C,$04          4 byte
3716  21 21 00            LD   HL,$0021       IX+$21
3719  CD 5F 37            CALL IXOFF
371C  78                  LD   A,B            minst en hel byte?
371D  D6 08               SUB  $08
371F  30 0A               JR   NC,L372B
3721  05                  DEC  B              B = 0: klart
3722  F8                  RET  M
3723  AF                  XOR  A              en siffra, 0 in
3724  ED 67       L3724:  RRD                 halv byte åt höger
3726  23                  INC  HL
3727  0D                  DEC  C
3728  20 FA               JR   NZ,L3724
372A  C9                  RET

372B  47          L372B:  LD   B,A            B - 8
372C  AF                  XOR  A              0 in
372D  56          L372D:  LD   D,(HL)          hel byte åt höger
372E  77                  LD   (HL),A
372F  7A                  LD   A,D
3730  23                  INC  HL
3731  0D                  DEC  C
3732  20 F9               JR   NZ,L372D
3734  18 DE               JR   MSHR
```

MSHL: *skifta IX+$21..$24 åt vänster på samma sätt, med* RLD. *Nollor skiftas in. Förstör A, BC, D och HL.*

```
3736  0E 04        MSHL:   LD   C,$04          4 byte
3738  21 24 00             LD   HL,$0024       IX+$24
373B  CD 5F 37           CALL  IXOFF
373E  78                 LD   A,B            minst en hel byte?
373F  D6 08              SUB  $08
3741  30 0A              JR   NC,L374D
3743  05                 DEC  B              B = 0: klart
3744  F8                 RET  M
3745  AF                 XOR  A              en siffra, 0 in
3746  ED 6F        L3746:  RLD                halv byte åt vänster
3748  2B                 DEC  HL
3749  0D                 DEC  C
374A  20 FA              JR   NZ,L3746
374C  C9                 RET

374D  47           L374D:  LD   B,A            B - 8
374E  AF                 XOR  A
374F  56           L374F:  LD   D,(HL)         hel byte åt vänster
3750  77                 LD   (HL),A
3751  7A                 LD   A,D
3752  2B                 DEC  HL
3753  0D                 DEC  C
3754  20 F9              JR   NZ,L374F
3756  18 DE              JR   MSHL

3758  C3 01 07     FOVFL:  JP   L0701          fel 4, FÖR STORT FLYTTAL

375B  AF           FZERO:  XOR  A              exponent 0
375C  C3 FA 34             JP   FMULE          via FMUL: resultatet 0
```

IXOFF: HL = IX + HL. DE bevaras.

```
375F  D5           IXOFF:  PUSH DE
3760  DD E5                PUSH IX
3762  D1                  POP  DE
3763  19                  ADD  HL,DE          HL = IX + HL
3764  D1                  POP  DE
3765  C9                  RET
```

FENTER: *början på flyttalsrutinerna. Returadressen tas bort och läggs tillbaka efter HL, DE och BC. Ut: IX = SP -* $27 (*före* RET)*, det vill säga arbetsytan IX+0..$28 slutar precis under de sparade registren. Förstör IX.*

```
3766  DD E1        FENTER: POP  IX             returadressen
3768  E5                   PUSH HL             spara registren
3769  D5                   PUSH DE
376A  C5                   PUSH BC
376B  DD E5                PUSH IX             returadressen tillbaka
376D  DD 21 D9 FF          LD   IX,$FFD9       IX = arbetsytan
3771  DD 39                ADD  IX,SP
3773  C9                   RET
```

3.91 IMUL / IDIV: 16-bitars heltal utan tecken $3774

IMUL: HL = HL × DE. Skift och addition, högsta biten först. CY = 1 om produkten inte ryms i 16 bitar. Förstör A, BC (= DE in) och DE. Anropas via SMULDV och direkt från $2D5A och $331E.

IDIV: HL = HL / DE, DE = resten. Återställande division, 16 varv (A = 17 räknar också det sista skiftet av kvoten). CY = 1 om DE = 0, annars CY = 0. Förstör A och BC (= DE in). Anropas via SMULDV ($2B87), som sköter tecknen, och direkt bara från INTDIG ($3F44).

```
3774  42           IMUL:   LD   B,D            BC = b
3775  4B                   LD   C,E
3776  11 00 00             LD   DE,$0000       DE = produkten
3779  3E 10               LD   A,$10          16 bitar
377B  29           L377B:  ADD  HL,HL          nästa bit i a
377C  EB                   EX   DE,HL
377D  30 02                JR   NC,L3781       0: ingen addition
377F  09                   ADD  HL,BC          produkten += b
3780  D8                   RET  C              spill
3781  3D           L3781:  DEC  A
3782  C8                   RET  Z              klart, HL = produkten
3783  29                   ADD  HL,HL          produkten * 2
3784  EB                   EX   DE,HL          HL = a, DE = produkten
3785  30 F4                JR   NC,L377B       inget spill
3787  C9                   RET                 spill
```

IDIV: *HL / DE. Kvoten skiftas in i HL och resten byggs upp i DE (växlas med* EX).

```
3788  7A           IDIV:   LD   A,D            b = 0?
3789  B3                   OR   E
378A  37                   SCF                 CY: division med 0
378B  C8                   RET  Z
378C  42                   LD   B,D            BC = b
378D  4B                   LD   C,E
378E  11 00 00             LD   DE,$0000
3791  EB                   EX   DE,HL          DE = a, HL = resten = 0
3792  3E 11                LD   A,$11          16 bitar + sista skiftet
3794  37                   SCF                 CCF ger CY = 0 första varvet
3795  3F           L3795:  CCF                 kvotbiten = NOT CY
3796  EB                   EX   DE,HL          HL = a, med kvoten inskiftad
3797  ED 6A                ADC  HL,HL          in i kvoten, ut ur a
3799  3D                   DEC  A
379A  C8                   RET  Z              klart, HL = kvoten
379B  EB                   EX   DE,HL          HL = resten
379C  ED 6A                ADC  HL,HL          in i resten
379E  A7                   AND  A              CY = 0
379F  ED 42                SBC  HL,BC          resten - b
37A1  30 F2                JR   NC,L3795       gick: kvotbit 1
37A3  09                   ADD  HL,BC          lägg tillbaka: kvotbit 0
37A4  18 EF                JR   L3795
```

Korsreferenstabell för flyttalsaritmetiken och IMUL / IDIV. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och vid IY; IY-kolumnen är byten vid IY+$03 ($FE19), som bara FDIV läser. FADD, FSUB, FMUL och FDIV tar DE = BC = a och HL = b och skriver resultatet över a; HL, DE och BC bevaras. FENTER ger dem arbetsytan på stacken vid IX och FSTORE (eller FSTOR1) tar bort den igen, så den räknas inte; etiketterna inne i rutinerna (FADDC, FSUBM, FNORM, FSTORE, FMULE, FMRND, FDLOOP, FDRND) får inga egna rader. MADD3, MSUB3, FALIGN, FCOPY, MSHR och MSHL arbetar på arbetsytan vid IX. IMUL och IDIV räknar med 16 bitar utan tecken och svarar med CY vid spill eller division med 0. Felen: fel 4 vid FOVFL. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							IY+
Rutin	A	CY	BC	DE	HL	IX	stack	$03
FADD	X	X	I	I	I	X		
FSUB	X	X	I	I	I	X		
FMUL	X	X	I	I	I	X		
FDIV	X	X	I	I	I	X		I
MADD3	X	IU	X	IU		I		
MSUB3	X	IU	X	IU		I		
FALIGN	X	X	U	IU	IX	I		
FCOPY	U	X	U	U	IU	I		
MSHR	X	X	IX	X	X	I		
MSHL	X	X	IX	X	X	I		
FOVFL*								
FZERO*	X					I		
IXOFF		X			IU	I		
FENTER			I	I	I	U	U	
IMUL	X	U	X	IX	IU			
IDIV	X	U	X	IU	IU			

Rutin	**Anropar**	**Anropas från**
FADD	FENTER, FALIGN; går in i FADDC: MADD3, MSHR, FOVFL, FSUBM, FSTORE	ADDF (via IX), FORTSI ($1D83), FATN (två ställen), FLOG (tre ställen), FPOLY ($3F82)
FSUB	FENTER, FALIGN; till FADDC med JR	SUBF (via IX), FATN ($3B97), FEXP ($3C18), FINT ($3CB4, med JP), FLOG ($3CED), FSIN ($3DAB)
FMUL	FENTER, MADD3, MSHR, MSHL, FOVFL, FZERO, FSTORE	MULF (via IX), FPOW ($2E12), POWFI (två ställen), FATN (två ställen), FEXP ($3C08), FLOG10 (med JP), FLOG ($3D22), FSIN (fyra ställen), FPOLY ($3F7E)
FDIV	FENTER, FCOPY, MADD3, MSHL, FOVFL, FZERO, FMULE, FSTOR1	DIVF (via IX), POWFI ($2E76), FATN ($3B9C), FTAN (med JP)
MADD3		FADD ($3424), FMUL ($3587), FDIV ($3651)
MSUB3		FSUB och FADD ($3474, i FSUBM)
FALIGN	FCOPY, MSHR	FADD, FSUB
FCOPY		FDIV ($3610), FALIGN
MSHR	IXOFF	FADD ($342B), FMUL ($3598), FALIGN
MSHL	IXOFF	FADD ($34C5, i FNORM), FMUL ($35B1), FDIV ($365C)
FOVFL	fel 4	FADD ($3431), FMUL (två ställen), FDIV (tre ställen)
FZERO	till FMULE med JP	FADD (två ställen, i FNORM), FMUL (två ställen), FDIV (två ställen)
IXOFF		MSHR, MSHL
FENTER		FADD, FSUB, FMUL, FDIV
IMUL		MULI och SMUL (via IX och SMULDV), ERRIDX ($2D5A), DIM2 ($331E)
IDIV		DIVI (via IX och SMULDV), INTDIG ($3F44)

3.92 ASCII-aritmetik: AADD ASUB ACOMP ADIV AMUL $37A6

Talen är text: mellanslag, tecken (+ eller -), siffror och högst en punkt, avslutade med $83 (samma format som ASCARG lägger i bufferten). Mellanslag, tecken och punkt räknas som siffran 0 när siffrorna läses, och punkten hoppas över.

In: HL = a, DE = b.

Ut: resultatet skrivs över a, i a:s format: lika många positioner och lika många decimaler. Ledande nollor blir mellanslag och ett minustecken skrivs närmast före första siffran.

A: bit 7 = negativt, bit 6 = inte noll, bit 5 = spill.

CY = 1 vid fel. A = 8 (bit 3): texten är inte ett tal, annars spill. (ASCERR, $2C4B, gör fel 66 respektive 65 av det.)

ACOMP skriver också över a (med skräp) och ger bara A.

ASENTR ($396F) ger en arbetsyta under stacken (SP0 = SP vid anropet av rutinen, HL' = SP0 för felutgång):

```
IY = SP0-20: IY+4 a:s längd  IY+5 a:s decimaler  IY+6 flaggor
             (bit 7 tecken, 6 inte noll, 5 spill)
IX = SP0-10: IX+4 b:s längd  IX+5 b:s decimaler  IX+6 b:s tecken
             IX+7 räknare    IX+8 se ASWIDE
ASBBEG/ASBEND ($FDD0/$FDD2) = b:s början och $83,
ASABEG/ASAEND ($FDD4/$FDD6) = a:s början och $83.
```

”Längd” är antalet positioner utom punkten, alltså med ledande mellanslag och tecken. Högst 31, annars fel (A = 8).

Z från ASENTR: a och b har samma tecken.

Felutgången ASBAD återställer SP = HL' och returnerar direkt till den som anropade rutinen, så `RET C` efter `CALL ASENTR` tas aldrig.

Addition och subtraktion görs på beloppen: AADD med samma tecken och ASUB med olika tecken adderar (ASMADD), annars subtraheras (ASMSUB). Resultatet får a:s tecken; blir det lån tiokomplementeras a och tecknet vänds (ASNEG).

Avrundning: har b fler decimaler än a kapas b, och första kapade siffran avgör. Över 5, eller 5 följd av någon siffra skild från 0: beloppet avrundas uppåt. Exakt 5 (resten nollor): uppåt bara om b är positivt. AADD avrundar därför exakt hälft mot +oändligheten, ASUB mot -oändligheten. Till exempel blir 1.00 + 0.005 = 1.01, -1.00 + -0.005 = -1.00 och 1.0 - 1.05 = -.1.

```
37A6  CD 6F 39     AADD:   CALL ASENTR          Z: samma tecken
37A9  D8                   RET  C               (tas aldrig)
37AA  28 08                JR   Z,ASMADD        addera beloppen
37AC  18 1B                JR   ASMSUB          subtrahera beloppen

37AE  CD 6F 39     ASUB:   CALL ASENTR          Z: samma tecken
37B1  D8                   RET  C               (tas aldrig)
37B2  28 15                JR   Z,ASMSUB        subtrahera beloppen
37B4  CD 12 3A     ASMADD: CALL ASALGN          ställ upp, B = avrundningen
37B7  C4 0D 39             CALL NZ,ASADDN       a += b
37BA  CD CD 3A             CALL ASREST          a:s övriga siffror
37BD  C4 1A 39             CALL NZ,ASCARN       för minnet vidare
37C0  30 04                JR   NC,L37C6        minne ut ur a
37C2  FD CB 06 EE          SET  5,(IY+$06)      spill
37C6  C3 00 3B     L37C6:  JP   ASFIN
```

ASMSUB: |a| - |b| med avrundningen i B. Lån betyder |a| < |b|: a tiokomplementeras och tecknet vänds (ASNEG). Har b en ledande 1 utöver a:s bredd (ASWIDE, IX+8) och blir det inget lån, får resultatet ändå plats: spillet tas bort, och a tiokomplementeras som vid lån.

```
37C9  CD 12 3A     ASMSUB: CALL ASALGN          ställ upp, B = avrundningen
37CC  C4 3B 39             CALL NZ,ASSUBN       a -= b
37CF  CD CD 3A             CALL ASREST          a:s övriga siffror
37D2  C4 48 39             CALL NZ,ASBORN       för lånet vidare
37D5  38 0A                JR   C,L37E1         lån: a < b
37D7  DD 7E 08             LD   A,(IX+$08)      b:s ledande 1 (ASWIDE)?
37DA  A7                   AND  A
37DB  28 E9                JR   Z,L37C6
37DD  FD CB 06 AE          RES  5,(IY+$06)      får plats ändå
37E1  CD D6 3A     L37E1:  CALL ASNEG           tiokomplement, vänd tecknet
37E4  18 E0                JR   L37C6
```

ASZERA: a:s siffror och mellanslag blir `0` (punkten står kvar), och ASMADD lägger in b, avrundat till a:s decimaler. Används av ADIV och AMUL när kvoten eller produkten blivit b (ASSETB).

```
37E6  FD 46 04     ASZERA: LD   B,(IY+$04)      a:s längd
37E9  CD 72 3B             CALL ASALST          HL = a:s sista siffra
37EC  CD 4D 3B     L37EC:  CALL ASDIG           fyll a med 0
37EF  36 30                LD   (HL),$30
37F1  2B                   DEC  HL
37F2  10 F8                DJNZ L37EC
37F4  18 BE                JR   ASMADD          a += b (produkten/kvoten)
```

ACOMP: jämför a och b. Olika tecken: beloppen adderas bara för att se om båda är noll; resultatet får a:s tecken. Samma tecken: beloppen subtraheras (ASCMPS). Ut: A bit 6 = 0 lika, annars bit 7 = 1 för a < b. CY = 0 om texterna är tal.

Fel i ROM:en: vid samma tecken och inget lån vänds tecknet också när a är negativt (`ADD A,A` lägger tecknet i CY); rätt vore att vända bara vid lån. Två olika negativa tal ger därför alltid `a > b`: `COMP%("-2","-1")` = 1 och `COMP%("-10","-5")` = 1 (men `COMP%("-1","-2")` = 1 är rätt). Positiva tal och olika tecken jämförs rätt. Lika i 9913.

```
37F6  CD 6F 39     ACOMP:  CALL ASENTR
37F9  D8                   RET  C               (tas aldrig)
37FA  28 1B                JR   Z,ASCMPS        samma tecken
37FC  CD 12 3A             CALL ASALGN          olika tecken: addera
37FF  C4 0D 39             CALL NZ,ASADDN
3802  CD CD 3A             CALL ASREST
3805  C4 1A 39             CALL NZ,ASCARN
3808  FD 4E 06             LD   C,(IY+$06)      flaggorna
380B  CB 69                BIT  5,C             spill: inte noll
380D  20 04                JR   NZ,L3813
380F  CB 18                RR   B               minne ut: inte noll
3811  30 1D                JR   NC,L3830
3813  CB F1     L3813:  SET  6,C             inte noll
3815  18 19                JR   L3830

3817  CD 12 3A     ASCMPS: CALL ASALGN          subtrahera beloppen
381A  C4 3B 39             CALL NZ,ASSUBN       a -= b
381D  CD CD 3A             CALL ASREST          a:s övriga siffror
3820  C4 48 39             CALL NZ,ASBORN       för lånet vidare, CY = lånet
3823  FD 7E 06             LD   A,(IY+$06)      flaggorna, bit 7 = a:s tecken
3826  4F                   LD   C,A
3827  38 03                JR   C,L382C         lån: |a| < |b|
3829  87                   ADD  A,A             CY = a:s tecken: vänder fel (fel)
382A  30 04                JR   NC,L3830        a positivt: tecknet står
382C  3E 80     L382C:  LD   A,$80           vänd tecknet
382E  A9                   XOR  C
382F  4F                   LD   C,A
3830  CB A9     L3830:  RES  5,C             inget spill
3832  CB 71                BIT  6,C             noll: inget tecken
```

```
3834  20 02              JR   NZ,L3838
3836  CB B9              RES  7,C
3838  C3 3E 3B    L3838: JP   ASRET          A = flaggorna
```

3.93 ADIV: a = a / b $383B

ASWORK lägger 62 nollor + $83 (W) under arbetsytan och ger kvotens tecken i IY+6. a kopieras till W, förskjuten b:s decimaler åt vänster. Sedan vanlig återställande lång division: b dras från W så länge det går, antalet blir kvotsiffran och skrivs där fönstret började. Kvoten får a:s längd + b:s decimaler siffror med a:s + b:s decimaler. Blir en kvotsiffra 10 (till exempel b = 0) sätts spill. Blir a:s längd + b:s längd + 2 × b:s decimaler 63 eller mer ger det A = 8.

Kvoten blir sedan b (ASSETB), a nollställs och ASMADD lägger in den, med avrundning till a:s decimaler. Har b inga decimaler finns inga extra siffror, så kvoten kapas: 2.00/3 = .66 men 2.00/3.0 = .67.

```
383B  CD 6F 39    ADIV:  CALL ASENTR         (fel)
383E  D8                 RET  C
383F  CD 9D 3A           CALL ASWORK         W, kvotens tecken
3842  DD 7E 05           LD   A,(IX+$05)     b:s decimaler
3845  4F                 LD   C,A
3846  FD 86 04           ADD  A,(IY+$04)     + a:s längd = antal kvotsiffror
3849  CD FC 38           CALL ASLEN          + b:s längd
384C  81                 ADD  A,C            + b:s decimaler; HL = W - b:s decimaler
384D  CD 02 39           CALL ASLEN1
3850  EB                 EX   DE,HL          DE = W
3851  FD 4E 04           LD   C,(IY+$04)     a:s längd
3854  CD 72 3B           CALL ASALST
3857  CD 0D 39           CALL ASADDN         kopiera a till W
385A  DD 7E 04           LD   A,(IX+$04)     b:s heltalssiffror
385D  DD 96 05           SUB  (IX+$05)
3860  4F                 LD   C,A
3861  AF                 XOR  A
3862  47                 LD   B,A
3863  EB                 EX   DE,HL          DE = fönstrets början
3864  ED 42              SBC  HL,BC
3866  EB                 EX   DE,HL
3867  13                 INC  DE
3868  3E 30      ASDLP:  LD   A,$30          kvotsiffran = "0"
386A  08         L386A:  EX   AF,AF'
386B  DD 4E 04           LD   C,(IX+$04)     b:s längd
386E  AF                 XOR  A
386F  47                 LD   B,A
3870  EB                 EX   DE,HL          DE = fönstrets slut
3871  09                 ADD  HL,BC
3872  EB                 EX   DE,HL
3873  CD 6D 3B           CALL ASBLST
3876  CD 3B 39           CALL ASSUBN         W -= b
3879  1A                 LD   A,(DE)         lånet i siffran till vänster
387A  90                 SUB  B
387B  12                 LD   (DE),A
387C  FE 30              CP   $30
387E  38 0B              JR   C,L388B        negativt: klart
3880  08                 EX   AF,AF'         kvotsiffran + 1
3881  3C                 INC  A
3882  FE 3A              CP   $3A            högst 9
3884  38 E4              JR   C,L386A
3886  3D                 DEC  A              9
3887  FD CB 06 EE        SET  5,(IY+$06)     spill
```

```
388B  CD BA 3A    L388B:  CALL  ASADDB          lägg tillbaka b
388E  12                  LD    (DE),A          kvotsiffran
388F  13                  INC   DE              nästa fönster
3890  DD 35 07            DEC   (IX+$07)        fler kvotsiffror?
3893  20 D3               JR    NZ,ASDLP
3895  EB                  EX    DE,HL
3896  36 83               LD    (HL),$83        $83 efter kvoten
3898  DD 7E 05            LD    A,(IX+$05)      kvotens längd
389B  FD 86 04            ADD   A,(IY+$04)
389E  DD 77 04            LD    (IX+$04),A
38A1  4F                  LD    C,A
38A2  FD 7E 05            LD    A,(IY+$05)      kvotens decimaler
38A5  DD 86 05            ADD   A,(IX+$05)
38A8  DD 77 05            LD    (IX+$05),A
38AB  22 D2 FD    ASSETB: LD    (ASBEND),HL     b = W
38AE  AF                  XOR   A
38AF  47                  LD    B,A
38B0  ED 42               SBC   HL,BC
38B2  22 D0 FD            LD    (ASBBEG),HL
38B5  C3 E6 37            JP    ASZERA          a = 0, a += b
```

3.94 AMUL: a = a × b $38B8

Skolmultiplikation i W (se ADIV): för varje siffra k i a, från höger, adderas b k gånger (ASADDB) på rätt plats. Produkten har a:s + b:s längd och a:s + b:s decimaler; 63 positioner eller fler ger A = 8. Sedan som i ADIV: produkten blir b, a nollställs och ASMADD avrundar in den. Exakt hälft avrundas uppåt om det ursprungliga b är positivt:

1.00 × 0.125 = .13 men 1.00 × -0.125 = -.12.

```
38B8  CD 6F 39    AMUL:   CALL  ASENTR
38BB  D8                  RET   C
38BC  CD 9D 3A            CALL  ASWORK          W, produktens tecken
38BF  FD 7E 04            LD    A,(IY+$04)      a:s längd
38C2  DD 4E 04            LD    C,(IX+$04)      b:s längd
38C5  CD FC 38            CALL  ASLEN           IX+7 = a:s längd; HL = W - b:s längd
38C8  EB                  EX    DE,HL           DE = W
38C9  CD 72 3B            CALL  ASALST          a:s sista siffra
38CC  CD 4D 3B    ASMLP:  CALL  ASDIG           siffran k
38CF  E5                  PUSH  HL
38D0  D6 30               SUB   $30             k = 0?
38D2  28 07               JR    Z,L38DB
38D4  08          L38D4:  EX    AF,AF'          W += b, k gånger
38D5  CD BA 3A            CALL  ASADDB
38D8  3D                  DEC   A
38D9  20 F9               JR    NZ,L38D4
38DB  E1          L38DB:  POP   HL              nästa siffra i a
38DC  2B                  DEC   HL
38DD  1B                  DEC   DE              W en plats åt vänster
38DE  DD 35 07            DEC   (IX+$07)        fler siffror i a?
38E1  20 E9               JR    NZ,ASMLP
38E3  FD 7E 05            LD    A,(IY+$05)      produktens decimaler
38E6  DD 86 05            ADD   A,(IX+$05)
38E9  DD 77 05            LD    (IX+$05),A
38EC  FD 7E 04            LD    A,(IY+$04)      produktens längd
38EF  DD 86 04            ADD   A,(IX+$04)
38F2  DD 77 04            LD    (IX+$04),A
38F5  21 3E 00            LD    HL,$003E        HL = W:s $83
38F8  39                  ADD   HL,SP
```

```
38F9  4F                LD   C,A            C = produktens längd
38FA  18 AF             JR   ASSETB
```

ASLEN: *IX+7 = A, A += b:s längd.* ASLEN1: *A >= 63 ger* ASBAD, *annars HL -= C (B = 0).*

```
38FC  DD 77 07    ASLEN:  LD   (IX+$07),A
38FF  DD 86 04            ADD  A,(IX+$04)
3902  FE 3F      ASLEN1: CP   $3F
3904  D2 BF 39           JP   NC,ASBAD       för långt: A = 8
3907  A7                  AND  A
3908  06 00               LD   B,$00
390A  ED 42               SBC  HL,BC
390C  C9                  RET
```

Siffer-slingor. HL = x, DE = y pekar på sista siffran. Resultatet skrivs i y, från höger. C = antal siffror, B = minnessiffran (0/1). ASADDN: *y = y + x + B, C siffror. Ut: HL = y, DE = x (nästa).* ASCARN: *för minnet vidare genom HL, C siffror. CY = minnessiffran.* ASSUBN / ASBORN: *samma med subtraktion (y - x - B) och lån.*

ASPUT / ASPUTB: *skriv siffran och ge nytt minne/lån i B; en siffra*
`<> 0 sätter bit 6 i IY+6. Mellanslag m.m. i y blir alltså "0".`

ASDIG2: *A = y-siffran, B = x-siffran + B.*

```
390D  CD 5E 39    ASADDN: CALL ASDIG2         B = b:s siffra + minnet, A = a:s
3910  80                  ADD  A,B            y + x + minne
3911  D6 30               SUB  $30            summan som siffertecken
3913  CD 26 39            CALL ASPUT          skriv, B = minnet
3916  EB                  EX   DE,HL          HL = b igen
3917  C8                  RET  Z              C = 0: klart
3918  18 F3               JR   ASADDN         nästa siffra

391A  CD 4D 3B    ASCARN: CALL ASDIG          siffran
391D  80                  ADD  A,B            + minnet
391E  CD 26 39            CALL ASPUT
3921  20 F7               JR   NZ,ASCARN
3923  78                  LD   A,B            CY = minnet
3924  0F                  RRCA
3925  C9                  RET

3926  06 00       ASPUT:  LD   B,$00          inget minne
3928  FE 3A               CP   $3A            > 9?
392A  38 03               JR   C,L392F        0 till 9: som den är
392C  D6 0A               SUB  $0A            - 10
392E  04          L392E:  INC  B              minne/lån
392F  77          L392F:  LD   (HL),A         skriv siffran
3930  FE 30               CP   $30
3932  28 04               JR   Z,L3938        noll: bit 6 orörd
3934  FD CB 06 F6         SET  6,(IY+$06)     inte noll
3938  2B          L3938:  DEC  HL             nästa plats till vänster
3939  0D                  DEC  C              Z: alla C siffror klara
393A  C9                  RET

393B  CD 5E 39    ASSUBN: CALL ASDIG2
393E  90                  SUB  B              y - x - lån
393F  C6 30               ADD  A,$30
3941  CD 54 39            CALL ASPUTB
3944  EB                  EX   DE,HL
3945  C8                  RET  Z
3946  18 F3               JR   ASSUBN

3948  CD 4D 3B    ASBORN: CALL ASDIG
394B  90                  SUB  B
```

```
394C  CD 54 39            CALL ASPUTB
394F  20 F7               JR   NZ,ASBORN
3951  78                  LD   A,B            CY = lånet
3952  0F                  RRCA
3953  C9                  RET

3954  06 00       ASPUTB: LD   B,$00
3956  FE 30               CP   $30            < 0?
3958  30 D5               JR   NC,L392F
395A  C6 0A               ADD  A,$0A          + 10
395C  18 D0               JR   L392E

395E  7E          ASDIG2: LD   A,(HL)         x-siffran
395F  FE 30               CP   $30
3961  DC 47 3B            CALL C,ASDOT        punkt m.m.
3964  80                  ADD  A,B            + minnet
3965  47                  LD   B,A
3966  2B                  DEC  HL
3967  EB                  EX   DE,HL
3968  7E                  LD   A,(HL)         y-siffran
3969  FE 30               CP   $30
396B  D0                  RET  NC
396C  C3 47 3B            JP   ASDOT
```

ASENTR: gemensam ingång för AADD, ASUB, ACOMP, ADIV och AMUL. In: HL = a, DE = b, returadressen till rutinen vid SP. Lägger arbetsytan (IY, IX) under SP0, läser a och b med ASPARS och sparar början, slut, längd, decimaler och tecken. Ut: Z = samma tecken, IX+8 = 0, CY = 0; SP under arbetsytan, HL′ = SP0. Inte ett tal: ASBAD, som returnerar direkt till rutinens anropare. Förstör alla register.

```
396F  C1          ASENTR: POP  BC              returadressen
3970  D9                  EXX                  den andra uppsättningen
3971  21 00 00            LD   HL,$0000        HL′ = SP0
3974  39                  ADD  HL,SP
3975  EB                  EX   DE,HL           DE′ = SP0
3976  DD 21 F6 FF         LD   IX,$FFF6        IX = SP0-10
397A  FD 21 EC FF         LD   IY,$FFEC        IY = SP0-20
397E  DD 19               ADD  IX,DE
3980  FD 19               ADD  IY,DE
3982  FD F9               LD   SP,IY           SP under arbetsytan
3984  EB                  EX   DE,HL           HL′ = SP0 till ASRET
3985  D9                  EXX                  HL = a, DE = b
3986  C5                  PUSH BC              returadressen under arbetsytan
3987  ED 53 D0 FD         LD   (ASBBEG),DE     b:s början
398B  22 D4 FD            LD   (ASABEG),HL     a:s början
398E  D5                  PUSH DE
398F  CD C5 39            CALL ASPARS          läs a
3992  22 D6 FD            LD   (ASAEND),HL     a:s $83
3995  FD 70 06            LD   (IY+$06),B      a:s tecken
3998  FD 73 04            LD   (IY+$04),E      a:s längd
399B  FD 72 05            LD   (IY+$05),D      a:s decimaler
399E  E1                  POP  HL              HL = b
399F  38 1E               JR   C,ASBAD
39A1  CD C5 39            CALL ASPARS          läs b
39A4  38 19               JR   C,ASBAD
39A6  DD 70 06            LD   (IX+$06),B      b:s tecken
39A9  22 D2 FD            LD   (ASBEND),HL     b:s $83
39AC  DD 72 05            LD   (IX+$05),D
39AF  DD 73 04            LD   (IX+$04),E
39B2  AF                  XOR  A               IX+8 = 0
39B3  DD 77 08            LD   (IX+$08),A
39B6  DD 7E 06            LD   A,(IX+$06)      Z: samma tecken
```

```
39B9  FD AE 06            XOR  (IY+$06)
39BC  E6 80               AND  $80
39BE  C9                  RET
```

ASBAD: *fel, A = 8 och CY. SP = HL' (SP0) och* RET *direkt till den som anropade* AADD *osv.*

```
39BF  3E 08       ASBAD:  LD   A,$08          inte ett tal
39C1  37                  SCF
39C2  D9                  EXX
39C3  F9                  LD   SP,HL          SP = SP0
39C4  C9                  RET
```

3.95 ASPARS: läs ett tal i text $39C5

In: HL = texten. Ut: HL på $83, E = antal positioner (mellanslag, tecken och siffror, inte punkten), D = antal decimaler, B bit 7 = minustecken. CY = inte ett tal: annat tecken än mellanslag före talet, en andra punkt, tom text eller fler än 31 positioner (ASCNT). Tecknet måste stå direkt före första siffran eller punkten.

```
39C5  11 00 00    ASPARS: LD   DE,$0000
39C8  43                  LD   B,E
39C9  48                  LD   C,B
39CA  7E          L39CA:  LD   A,(HL)
39CB  FE 20               CP   $20            mellanslag
39CD  20 08               JR   NZ,L39D7
39CF  CD 68 3B            CALL ASCNT          räknas
39D2  28 34               JR   Z,L3A08
39D4  23                  INC  HL
39D5  18 F3               JR   L39CA

39D7  FE 2B       L39D7:  CP   $2B            +
39D9  28 06               JR   Z,L39E1
39DB  FE 2D               CP   $2D            -
39DD  20 09               JR   NZ,L39E8
39DF  CB F8               SET  7,B            minustecken
39E1  CD 68 3B    L39E1:  CALL ASCNT          tecknet räknas
39E4  28 22               JR   Z,L3A08
39E6  23                  INC  HL
39E7  7E                  LD   A,(HL)
39E8  FE 2E       L39E8:  CP   $2E            punkt
39EA  28 1E               JR   Z,L3A0A
39EC  FE 30               CP   $30            inte en siffra: fel
39EE  38 18               JR   C,L3A08
39F0  FE 3A               CP   $3A
39F2  30 14               JR   NC,L3A08
39F4  CD 68 3B            CALL ASCNT          siffran räknas
39F7  28 0F               JR   Z,L3A08
39F9  23          L39F9:  INC  HL
39FA  7E                  LD   A,(HL)
39FB  FE 83               CP   $83            slut?
39FD  20 E9               JR   NZ,L39E8
39FF  AF                  XOR  A              E = 0: tom
3A00  B3                  OR   E
3A01  28 05               JR   Z,L3A08
3A03  92                  SUB  D              positioner efter punkten
3A04  0D                  DEC  C              ingen punkt: D = 0 kvar
3A05  C0                  RET  NZ
3A06  57                  LD   D,A            D = decimalerna
3A07  C9                  RET
```

```
3A08  37          L3A08:  SCF
3A09  C9                  RET

3A0A  AF          L3A0A:  XOR  A              andra punkten: fel
3A0B  B1                  OR   C
3A0C  20 FA               JR   NZ,L3A08
3A0E  0C                  INC  C
3A0F  53                  LD   D,E            D = positioner före punkten
3A10  18 E7               JR   L39F9
```

3.96 ASALGN: ställ upp a och b så att decimalerna står under varandra $3A12

a har minst lika många decimaler: a:s extra decimaler gås igenom med B = 0 (de står kvar, men mellanslag blir ”0”). b har fler decimaler (ASALG1): b:s extra decimaler kapas och ger avrundningen i B (se ovan), och b:s längd minskas.

ASWIDE: jämför de återstående längderna. IX+7 = hur många siffror a har utöver b. Är b längre sätts IX+7 = 0, och b:s extra ledande siffror måste vara 0; annars spill. Undantag: är den enda en ”1” närmast a:s bredd sätts IX+8, för en subtraktion kan ändå få plats (ASMSUB). Ut: HL = b, DE = a (sista siffran), C = antal par, Z om 0.

```
3A12  FD 7E 05    ASALGN: LD   A,(IY+$05)     a:s decimaler - b:s (fel)
3A15  DD 96 05            SUB  (IX+$05)
3A18  38 18               JR   C,ASALG1       b har fler
3A1A  4F                  LD   C,A
3A1B  FD 7E 04            LD   A,(IY+$04)     a:s längd - skillnaden
3A1E  91                  SUB  C
3A1F  08                  EX   AF,AF'
3A20  CD 72 3B            CALL ASALST         a:s sista siffra
3A23  AF                  XOR  A
3A24  47                  LD   B,A
3A25  B1                  OR   C
3A26  C4 1A 39            CALL NZ,ASCARN      gå förbi a:s extra decimaler
3A29  23                  INC  HL
3A2A  E5                  PUSH HL             a:s plats
3A2B  CD 6D 3B            CALL ASBLST         b:s sista siffra
3A2E  5D                  LD   E,L
3A2F  54                  LD   D,H
3A30  18 35               JR   ASWIDE

3A32  ED 44       ASALG1: NEG                 C = b:s extra decimaler
3A34  4F                  LD   C,A
3A35  AF                  XOR  A
3A36  47                  LD   B,A
3A37  CD 6D 3B            CALL ASBLST         b:s sista siffra
3A3A  23                  INC  HL
3A3B  ED 42               SBC  HL,BC          HL = första kapade siffran
3A3D  54                  LD   D,H
3A3E  5D                  LD   E,L
3A3F  DD 7E 04            LD   A,(IX+$04)     b:s längd - C
3A42  91                  SUB  C
3A43  DD 77 04            LD   (IX+$04),A
3A46  0D                  DEC  C
3A47  7E                  LD   A,(HL)         kapad siffra - 5
3A48  D6 35               SUB  $35
3A4A  38 11               JR   C,L3A5D        < 5: B = 0
3A4C  20 0D               JR   NZ,L3A5B       > 5: B = 1
3A4E  B1                  OR   C
```

```
3A4F  23                  INC  HL
3A50  C4 53 3B            CALL NZ,ASNZ          fler siffror <> 0?
3A53  20 06               JR   NZ,L3A5B
3A55  DD CB 06 7E         BIT  7,(IX+$06)       exakt hälft: b negativt?
3A59  20 02               JR   NZ,L3A5D
3A5B  06 01       L3A5B:  LD   B,$01            avrunda uppåt
3A5D  1B          L3A5D:  DEC  DE               DE = b:s sista kvarvarande siffra
3A5E  FD 7E 04            LD   A,(IY+$04)       a:s längd
3A61  08                  EX   AF,AF'
3A62  CD 72 3B            CALL ASALST           a:s sista siffra
3A65  23                  INC  HL
3A66  E5                  PUSH HL               a:s plats
3A67  08          ASWIDE: EX   AF,AF'
3A68  4F                  LD   C,A              C = a:s återstående längd
3A69  DD 96 04            SUB  (IX+$04)         - b:s
3A6C  38 08               JR   C,L3A76          b längre
3A6E  DD 77 07            LD   (IX+$07),A       a:s siffror utöver b
3A71  DD 4E 04            LD   C,(IX+$04)       par = b:s längd
3A74  18 21               JR   L3A97

3A76  ED 44       L3A76:  NEG                   b:s extra siffror
3A78  C5                  PUSH BC
3A79  4F                  LD   C,A
3A7A  DD 36 07 00         LD   (IX+$07),$00     inga utöver b i a
3A7E  2A D0 FD            LD   HL,(ASBBEG)      b:s början
3A81  CD 53 3B            CALL ASNZ             någon <> 0?
3A84  28 10               JR   Z,L3A96
3A86  FE 31               CP   $31              en 1...
3A88  20 08               JR   NZ,L3A92
3A8A  79                  LD   A,C
3A8B  FE 01               CP   $01              ...närmast a:s bredd?
3A8D  20 03               JR   NZ,L3A92
3A8F  DD 77 08            LD   (IX+$08),A       IX+8 = 1
3A92  FD CB 06 EE L3A92:  SET  5,(IY+$06)       spill (kanske)
3A96  C1          L3A96:  POP  BC               C = par, B = avrundningen
3A97  E1          L3A97:  POP  HL               a:s plats
3A98  EB                  EX   DE,HL            HL = b, DE = a
3A99  1B                  DEC  DE
3A9A  AF                  XOR  A
3A9B  B1                  OR   C
3A9C  C9                  RET
```

ASWORK: *IY+6 = produktens/kvotens tecken (a:s* XOR *b:s). Flyttar SP 63 byte nedåt och fyller W med 62* 0 *och* $83. *HL = W:s sista siffra.*

```
3A9D  FD CB 06 BE ASWORK: RES  7,(IY+$06)       a:s tecken bort
3AA1  FD B6 06            OR   (IY+$06)         XOR b:s
3AA4  FD 77 06            LD   (IY+$06),A
3AA7  C1                  POP  BC               returadressen
3AA8  21 C1 FF            LD   HL,$FFC1         63 byte
3AAB  39                  ADD  HL,SP
3AAC  F9                  LD   SP,HL
3AAD  C5                  PUSH BC
3AAE  06 3E               LD   B,$3E            62 0
3AB0  36 30       L3AB0:  LD   (HL),$30
3AB2  23                  INC  HL
3AB3  10 FB               DJNZ L3AB0
3AB5  36 83               LD   (HL),$83         $83
3AB7  2B                  DEC  HL               W:s sista siffra
3AB8  AF                  XOR  A
3AB9  C9                  RET
```

ASADDB: *W(DE) += b. DE flyttas b:s längd åt höger och b adderas med* ASADDN*; minnet läggs i siffran till vänster. A' tillbaka till A (räknaren i* AMUL/ADIV).

```
3ABA  AF           ASADDB: XOR  A
3ABB  47                   LD   B,A
3ABC  DD 4E 04             LD   C,(IX+$04)      b:s längd
3ABF  EB                   EX   DE,HL
3AC0  09                   ADD  HL,BC
3AC1  EB                   EX   DE,HL
3AC2  CD 6D 3B             CALL ASBLST          b:s sista siffra
3AC5  CD 0D 39             CALL ASADDN          W += b
3AC8  1A                   LD   A,(DE)          minnet till vänster
3AC9  80                   ADD  A,B
3ACA  12                   LD   (DE),A
3ACB  08                   EX   AF,AF'          A = räknaren
3ACC  C9                   RET
```

ASREST: *C = IX+7 (a:s siffror utöver b), Z om inga; HL = a; CY = minnet i B.*

```
3ACD  DD 4E 07     ASREST: LD   C,(IX+$07)
3AD0  79                   LD   A,C
3AD1  A7                   AND  A
3AD2  EB                   EX   DE,HL
3AD3  78                   LD   A,B
3AD4  0F                   RRCA                 CY = minnet
3AD5  C9                   RET
```

ASNEG: *tiokomplement av a och vänt tecken i IY+6. Nollor längst till höger står kvar, den första siffran c <> 0 blir 10-c* ($6A - *c*) *och resten 9-c* ($69 - *c*).

```
3AD6  FD 46 04     ASNEG:  LD   B,(IY+$04)
3AD9  3E 80                LD   A,$80           vänd tecknet
3ADB  FD AE 06             XOR  (IY+$06)
3ADE  FD 77 06             LD   (IY+$06),A
3AE1  CD 72 3B             CALL ASALST          a:s sista siffra
3AE4  CD 4D 3B     L3AE4:  CALL ASDIG
3AE7  FE 30                CP   $30             nollor till höger står kvar
3AE9  20 04                JR   NZ,L3AEF
3AEB  2B                   DEC  HL
3AEC  10 F6                DJNZ L3AE4
3AEE  C9                   RET

3AEF  4F           L3AEF:  LD   C,A
3AF0  3E 6A                LD   A,$6A           10 - c
3AF2  91           L3AF2:  SUB  C
3AF3  77                   LD   (HL),A
3AF4  10 01                DJNZ L3AF7
3AF6  C9                   RET

3AF7  2B           L3AF7:  DEC  HL
3AF8  CD 4D 3B             CALL ASDIG
3AFB  4F                   LD   C,A
3AFC  3E 69                LD   A,$69           9 - c
3AFE  18 F2                JR   L3AF2
```

3.97 ASFIN: gör resultatet i a läsbart $3B00

Blev resultatet 0 (bit 6 = 0) tas minustecknet bort. Ledande ”0” blir mellanslag. Är allt noll, eller står bara ”.” och $83 kvar, sätts en ”0” tillbaka före $83 eller punkten (heltal: ”0”, annars ”.00”). Negativt: ”-” skrivs direkt före första siffran; saknas plats blir det spill (bit 5).

ASRET: A = flaggorna, SP = SP0, CY = bit 5 (spill).

```
3B00  FD 4E 06     ASFIN:  LD   C,(IY+$06)
3B03  CB 71                BIT  6,C            noll?
3B05  20 02                JR   NZ,L3B09
3B07  CB B9                RES  7,C            inget minustecken
3B09  2A D4 FD     L3B09:  LD   HL,(ASABEG)    a:s början
3B0C  7E           L3B0C:  LD   A,(HL)
3B0D  FE 30                CP   $30
3B0F  20 05                JR   NZ,L3B16       ledande nolla
3B11  36 20                LD   (HL),$20       blir mellanslag
3B13  23                   INC  HL
3B14  18 F6                JR   L3B0C

3B16  FE 83        L3B16:  CP   $83            allt noll
3B18  28 0B                JR   Z,L3B25
3B1A  FE 2E                CP   $2E            punkt...
3B1C  20 0A                JR   NZ,L3B28
3B1E  23                   INC  HL
3B1F  7E                   LD   A,(HL)
3B20  2B                   DEC  HL
3B21  FE 83                CP   $83            ...sist?
3B23  20 03                JR   NZ,L3B28
3B25  2B           L3B25:  DEC  HL
3B26  36 30                LD   (HL),$30       en 0 tillbaka
3B28  CB 79        L3B28:  BIT  7,C            negativt?
3B2A  28 12                JR   Z,ASRET
3B2C  2B                   DEC  HL             platsen före
3B2D  E5                   PUSH HL
3B2E  ED 5B D4 FD          LD   DE,(ASABEG)
3B32  A7                   AND  A
3B33  ED 52                SBC  HL,DE
3B35  E1                   POP  HL
3B36  38 04                JR   C,L3B3C        före a: ingen plats
3B38  36 2D                LD   (HL),$2D       minustecken
3B3A  18 02                JR   ASRET

3B3C  CB E9        L3B3C:  SET  5,C            spill
3B3E  AF           ASRET:  XOR  A
3B3F  79                   LD   A,C            A = flaggorna
3B40  D9                   EXX
3B41  F9                   LD   SP,HL          SP = SP0
3B42  CB 6F                BIT  5,A            spill?
3B44  C8                   RET  Z
3B45  37                   SCF
3B46  C9                   RET
```

ASDOT / ASDIG: *A = siffran vid HL. Punkten hoppas över* (DEC *HL), och allt under* 0 *(mellanslag, tecken) läses som* 0.

```
3B47  FE 2E        ASDOT:  CP   $2E            punkt?
3B49  3E 30                LD   A,$30          annars 0
3B4B  C0                   RET  NZ
3B4C  2B                   DEC  HL             hoppa över punkten
3B4D  7E           ASDIG:  LD   A,(HL)
3B4E  FE 30                CP   $30
```

```
3B50  D0                RET  NC
3B51  18 F4             JR   ASDOT
```

ASNZ: *sök framåt efter en siffra <> 0 bland C positioner (punkten hoppas över). NZ och A = siffran om den finns; Z vid* $83 *eller slut.*

```
3B53  7E          ASNZ:  LD   A,(HL)          siffra?
3B54  FE 30              CP   $30
3B56  38 06              JR   C,L3B5E
3B58  C0                 RET  NZ              <> 0: NZ
3B59  23          L3B59: INC  HL
3B5A  0D                 DEC  C
3B5B  20 F6              JR   NZ,ASNZ
3B5D  C9                 RET

3B5E  FE 83       L3B5E: CP   $83             slut: Z
3B60  C8                 RET  Z
3B61  FE 2E              CP   $2E
3B63  20 F4              JR   NZ,L3B59
3B65  23                 INC  HL
3B66  18 EB              JR   ASNZ
```

ASCNT: *E += 1; Z om E blir 32 (för långt).*

```
3B68  1C          ASCNT: INC  E
3B69  3E 1F              LD   A,$1F
3B6B  A3                 AND  E              E = 32: Z
3B6C  C9                 RET
```

ASBLST / ASALST: *HL = b:s resp. a:s sista siffra (före* $83*).*

```
3B6D  2A D2 FD    ASBLST: LD   HL,(ASBEND)
3B70  2B                  DEC  HL
3B71  C9                  RET

3B72  2A D6 FD    ASALST: LD   HL,(ASAEND)
3B75  2B                  DEC  HL
3B76  C9                  RET
```

Korsreferenstabell för ASCII-aritmetiken. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; $FDD0 är ASBBEG och $FDD2 ASBEND (b:s början och $83), $FDD4 ASABEG och $FDD6 ASAEND (a:s). AADD, ASUB, ACOMP, ADIV och AMUL tar HL = a och DE = b, skriver resultatet över a och svarar med A = flaggorna och CY vid fel (A = 8: inte ett tal); arbetsytan som ASENTR ger dem vid IX och IY under stacken tas bort igen och räknas inte för dem. ASMADD, ASMSUB, ASZERA och ASSETB är delar som flera av rutinerna går till, och de, ASFIN, ASRET och ASBAD slutar med att SP = HL' (det SP som rutinen anropades med) och returnerar direkt till den som anropade rutinen. ASCMPS, ASDLP, ASMLP, ASALG1 och ASWIDE är delar av ACOMP, ADIV, AMUL och ASALGN och får inga egna rader. Skuggregistren: ASENTR lägger SP0 i HL', och A' används som räknare. Rutinerna ger inga fel själva; ASCAR gör fel 65 och 66 av CY. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack									variabler			
Rutin	A	CY	BC	DE	HL	IX	IY	skugg.	stack	$FDD0	$FDD2	$FDD4	$FDD6
AADD	U	U	X	IX	IX	X	X	X		X	X	X	X
ASUB	U	U	X	IX	IX	X	X	X		X	X	X	X
ACOMP	U	U	X	IX	IX	X	X	X		X	X	X	X

	register och stack									variabler			
Rutin	A	CY	BC	DE	HL	IX	IY	skugg.	stack	$FDD0	$FDD2	$FDD4	$FDD6
ADIV	U	U	X	IX	IX	X	X	X		X	X	X	X
AMUL	U	U	X	IX	IX	X	X	X		X	X	X	X
ASMADD*	U	U	X	X	X	I	I	IX	X	I	I	I	I
ASMSUB*	U	U	X	X	X	I	I	IX	X	I	I	I	I
ASZERA*	U	U	X	X	X	I	I	IX	X	I	I	I	I
ASSETB*	U	U	IX	X	IX	I	I	IX	X	U	U	I	I
ASENTR	U	U	X	IX	IX	U	U	U	U	U	U	U	U
ASBAD*	U	U						IX	X				
ASPARS	X	U	U	U	IU								
ASALGN	U	X	U	U	U	I	I	X		I	I		I
ASWORK	IX	X	X		U		I		U				
ASFIN*	U	U	X	X	X		I	IX	X			I	
ASRET*	U	U	I					IX	X				

Rutin	**Anropar**	**Anropas från**
AADD	ASENTR, ASMADD och ASMSUB (med JR)	EXADD (genom CALLBC), ASCAR ($2C24)
ASUB	ASENTR, ASMSUB (med JR); går in i ASMADD	EXSUB (genom CALLBC)
ASMADD	ASALGN, ASADDN, ASREST, ASCARN, ASFIN (med JP)	AADD och ASZERA (med JR), ASUB (går in i den)
ASMSUB	ASALGN, ASSUBN, ASREST, ASBORN, ASNEG, ASFIN (genom $37C6)	AADD och ASUB (med JR)
ASZERA	ASALST, ASDIG, ASMADD (med JR)	ASSETB (med JP)
ACOMP	ASENTR, ASALGN, ASADDN, ASREST, ASCARN, ASSUBN, ASBORN, ASRET (med JP)	ASCCMP ($2C52)
ADIV	ASENTR, ASWORK, ASLEN, ASLEN1, ASALST, ASADDN, ASBLST, ASSUBN, ASADDB; går in i ASSETB	EXDIV (genom CALLBC)
AMUL	ASENTR, ASWORK, ASLEN, ASALST, ASDIG, ASADDB, ASSETB (med JR)	EXMUL (genom CALLBC)
ASSETB	ASZERA (med JP)	ADIV (går in i den), AMUL (med JR)
ASENTR	ASPARS, ASBAD (med JR)	AADD, ASUB, ACOMP, ADIV, AMUL
ASBAD		ASENTR (två ställen, med JR), ASLEN1 (med JP)
ASPARS	ASCNT	ASENTR (två ställen)
ASALGN	ASALST, ASCARN, ASBLST, ASNZ	ASMADD, ASMSUB, ACOMP (två ställen)
ASWORK		ADIV, AMUL
ASFIN	går in i ASRET	ASMADD (med JP, också för ASMSUB)
ASRET		ACOMP (med JP), ASFIN (två ställen med JR, och går in i den)

Korsreferenstabell för ASCII-aritmetikens hjälprutiner. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och i systemvariablerna; $FDD2 är ASBEND och $FDD6 ASAEND. Sifferslingorna arbetar bakifrån med HL = x och DE = y på sista siffran, C = antalet siffror och B = minnessiffran eller lånet; ASPUT och ASPUTB sätter bit 6 i IY+6 för en siffra skild från 0. IX och IY är arbetsytan från ASENTR. ASADDB slutar med EX AF,AF' och lämnar räknaren från ADIV och AMUL i A. ASLEN1 går till ASBAD när längden blir 63 eller mer. Rutinerna ger inga fel själva. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register								variabler	
Rutin	A	CY	BC	DE	HL	IX	IY	skugg.	$FDD2	$FDD6
ASLEN	IU	X	IX		IU	I				
ASLEN1	I	X	IX		IU					
ASADDN	X	X	IU	IU	IU		I			
ASCARN	X	U	IU		IU		I			
ASPUT	IX	X	IU		IU		I			
ASSUBN	X	X	IU	IU	IU		I			
ASBORN	X	U	IU		IU		I			
ASPUTB	IX	X	IU		IU		I			
ASDIG2	U	X	IU	IU	IU					
ASADDB	U	X	X	IU	X	I	I	IU	I	
ASREST	X	U	IU	IU	IU	I				
ASNEG	X	X	X		X		I			I
ASDOT	IU	X			IU					
ASDIG	U	X			IU					
ASNZ	U	X	IU		IU					
ASCNT	X	X		IU						
ASBLST					U				I	
ASALST					U					I

Rutin	**Anropar**	**Anropas från**
ASLEN	går in i ASLEN1	ADIV, AMUL
ASLEN1	ASBAD (med JP)	ADIV, ASLEN (går in i den)
ASADDN	ASDIG2, ASPUT	ASMADD, ACOMP, ADIV, ASADDB
ASCARN	ASDIG, ASPUT	ASMADD, ACOMP, ASALGN
ASPUT		ASADDN, ASCARN, ASPUTB (till $392E och $392F, med JR)
ASSUBN	ASDIG2, ASPUTB	ASMSUB, ACOMP ($381A), ADIV ($3876)
ASBORN	ASDIG, ASPUTB	ASMSUB, ACOMP ($3820)
ASPUTB	ASPUT ($392E och $392F, med JR)	ASSUBN, ASBORN
ASDIG2	ASDOT (också med JP)	ASADDN, ASSUBN
ASADDB	ASBLST, ASADDN	ADIV ($388B), AMUL ($38D5)
ASREST		ASMADD, ASMSUB, ACOMP (två ställen)
ASNEG	ASALST, ASDIG	ASMSUB ($37E1)
ASDOT	går in i ASDIG	ASDIG2 (två ställen), ASDIG (med JR)
ASDIG	ASDOT (med JR)	ASZERA, AMUL ($38CC), ASCARN, ASBORN, ASNEG (två ställen), ASDOT (går in i den)
ASNZ		ASALGN (två ställen)
ASCNT		ASPARS (tre ställen)
ASBLST		ADIV ($3873), ASALGN (två ställen), ASADDB
ASALST		ASZERA, ADIV, AMUL, ASALGN (två ställen), ASNEG

3.98 Flyttalsfunktioner: ATN EXP INT LOG LOG10 SIN COS TAN SQR$3B77

FFUNC ($2DC6) anropar rutinen med HL = BC = argumentets exponent på värdestacken, och resultatet skrivs över argumentet. FPOW ($2E07) använder FLOG och FEXP. FTOI och ITOF använder FTOINT och INTTOF. Arbetsminnet är samma byte som ASCII-aritmetiken använder (därav namnen ASBBEG och ASEND):

```
$FDD0-$FDD4  x, kopia av argumentet (FLDX), exponenten i $FDD4
$FDD5-$FDD9  ett tal till: ATN x+1, LOG n*LN 10, SIN först
             q, sedan INT(q) och sist vinkeln v
$FDDA-$FDDE  ATN: t^2
$FDDF-$FDE3  TAN: kopia av argumentet, sedan COS
$FDE4        SINQ: SIN/COS-kvadranten
$FDE9        ASEND: adressen där resultatet ska skrivas (FLDX)
```

Serierna räknas med FPOLY ($3F61). Konstanterna har bara 6 siffror och varje steg avrundas, så den sista siffran är ofta fel.

Provat (rätt värde inom parentes):

```
ATN(.001) = .000999 (.001)      ATN(1E30) = 1.57079 (1.57080)
EXP(-1) = .367883 (.367879)     EXP(2.302585) = 9.99987 (10)
LOG(2.718282) = 1.00001 (1)     LOG10(1E-5) = -5.00003 (-5)
SIN(100) = -.50633 (-.506366)   COS(10000) = -.95106 (-.952155)
```

FATN: ATN(-x) = -ATN(x). Negativt x görs positivt (FNEGX), ATNPOS räknar och resultatets tecken byts sedan. x = 0 går till ATNPOS.

In: BC = HL = argumentets exponent. Ut: resultatet på argumentets plats. Rutinerna förstör alla register utom IY; FFUNC sparar DE.

```
3B77  CD C5 3B     FATN:   CALL FLDX           Z: x = 0, M: x < 0 (gränsfall)
3B7A  F2 8A 3B             JP   P,ATNPOS       x >= 0, också x = 0
3B7D  CD BD 3B             CALL FNEGX          x = -x
3B80  23                   INC  HL             HL = $FDD4, x
3B81  CD 8A 3B             CALL ATNPOS         ATN(-x)
3B84  2A E9 FD             LD   HL,(ASEND)     HL = resultatets exponent
3B87  2B                   DEC  HL             resultatets tecken
3B88  18 36                JR   FNEG           byt tecken, och RET
```

ATNPOS: x >= 0 i $FDD4. *Med t = (x-1)/(x+1), som ligger i [-1, 1), är* ATN(x) = PI/4 + ATN(t), *och* ATN(t) = *t * P(t^2) med serien* ATNSER *(.999996 -.333174 .198079 -.132335 .0796263 -.0336063 .00681241, en approximation för hela [-1, 1], inte Taylorserien). Nära 0 tar* PI/4 *och* ATN(t) *nästan ut varandra, så ATN av små tal tappar siffror:* ATN(.001) = *.000999.*

```
3B8A  EB           ATNPOS: EX   DE,HL          DE = x
3B8B  21 BB 3F             LD   HL,FONE        1
3B8E  01 D9 FD             LD   BC,$FDD9       BC = $FDD9
3B91  C5                   PUSH BC             spara $FDD9
3B92  D5                   PUSH DE             och x
3B93  CD F7 33             CALL FADD           $FDD9 = x + 1
3B96  C1                   POP  BC             BC = x ($FDD4)
3B97  CD 60 34             CALL FSUB           x = x - 1
3B9A  E1                   POP  HL             HL = x + 1
3B9B  C5                   PUSH BC             spara t:s plats
3B9C  CD EE 35             CALL FDIV           t = (x-1) / (x+1)
3B9F  E1                   POP  HL             HL = t
3BA0  01 DE FD             LD   BC,$FDDE       BC = $FDDE
3BA3  C5                   PUSH BC
3BA4  CD DF 34             CALL FMUL           $FDDE = t^2
3BA7  D1                   POP  DE             DE = t^2
3BA8  01 02 3C             LD   BC,ATNSER      serien
3BAB  CD 61 3F             CALL FPOLY          DE = P(t^2), HL = t
```

```
3BAE  44                LD   B,H            BC = t
3BAF  4D                LD   C,L
3BB0  CD DF 34          CALL FMUL           t = t * P(t^2)
3BB3  11 B4 3F          LD   DE,FPIQ        PI/4
3BB6  ED 4B E9 FD       LD   BC,(ASEND)     BC = argumentets plats
3BBA  C3 F7 33          JP   FADD           = PI/4 + ATN(t), och RET
```

FNEGX: *byt tecken på x* ($FDD3). FNEG: *byt tecken på talet vars teckenbyte HL pekar på (bit 0).*

```
3BBD  21 D3 FD    FNEGX: LD   HL,$FDD3     x:s tecken
3BC0  3E 01       FNEG:  LD   A,$01
3BC2  AE                 XOR  (HL)         byt bit 0
3BC3  77                 LD   (HL),A
3BC4  C9                 RET
```

FLDX: *HL = BC = argumentet.* ASEND *= argumentet (där resultatet ska skrivas) och argumentet kopieras till x* ($FDD0-$FDD4).

Ut: HL = $FDD4*. Z om x = 0. Annars NZ, och S = tecknet (M om x < 0, genom att* RRCA *lägger teckenbiten i bit 7). Förstör A, BC (= 0) och DE.*

```
3BC5  60          FLDX:  LD   H,B          HL = argumentet
3BC6  69                 LD   L,C
3BC7  22 E9 FD           LD   (ASEND),HL   ASEND = argumentet
3BCA  11 D4 FD           LD   DE,$FDD4     DE = x:s exponent
3BCD  01 05 00           LD   BC,$0005
3BD0  D5                 PUSH DE           spara $FDD4
3BD1  ED B8              LDDR              x = argumentet, 5 byte
3BD3  E1                 POP  HL           HL = $FDD4
3BD4  7E                 LD   A,(HL)       exponent
3BD5  A7                 AND  A
3BD6  C8                 RET  Z            Z: x = 0
3BD7  2B                 DEC  HL
3BD8  7E                 LD   A,(HL)       tecknet
3BD9  0F                 RRCA              bit 0 -> bit 7
3BDA  F6 01              OR   $01          NZ
3BDC  23                 INC  HL
3BDD  C9                 RET
```

ATN-serien, 7 koefficienter c0..c6. FPOLY *går baklänges från* ATNSER *(c6);* $FF *där nästa posts tecken skulle ligga avslutar.*

```
3BDE  FF FF             DEFB $FF,$FF           slutet för FPOLY: $FF där en term till skulle ha tecknet
3BE0  99 99 96 00…      DEFB $99,$99,$96,$00,$80
                                               0.999996
3BE5  33 31 74 01…      DEFB $33,$31,$74,$01,$80
                                               -0.333174
3BEA  19 80 79 00…      DEFB $19,$80,$79,$00,$80
                                               0.198079
3BEF  13 23 35 01…      DEFB $13,$23,$35,$01,$80
                                               -0.132335
3BF4  79 62 63 00…      DEFB $79,$62,$63,$00,$7F
                                               0.0796263
3BF9  33 60 63 01…      DEFB $33,$60,$63,$01,$7F
                                               -0.0336063
3BFE  68 12 41 00       DEFB $68,$12,$41,$00   0.00681241
3C02  7E          ATNSER: DEFB $7E             exponenten; ATNSER pekar på c6:s sista byte
```

FEXP: EXP(x) *= 10^y med y = x * LOG10(e) (FLGE *= .434294). y delas i n =* INT(y) *och f = y - n i [0, 1). 10^f räknas med serien* EXPSER *(Taylorserien för 10^f: LN(10)^k/k!) och blir 1..10. Sedan läggs n till*

exponenten. Över `$FF`*: fel 4. Under 1: resultatet 0. |y| >= 65536 ger fel 7 i* `FTOINT`*, så* `EXP(1E6)` *och* `EXP(-1E6)` *ger FÖR STORT HELTAL i stället för fel 4 resp. 0. Under den gränsen prövas exponenten plus n som ett 16-bitarstal med tecken, fast n är utan tecken upp till 65535, så stora |x| slår runt:* `EXP(75100)` *fel 4 men* `EXP(75200)` *= 0,* `EXP(-75700)` *= 0 men* `EXP(-75800)` *fel 4 och* `EXP(-150800)` *= 3.16228E44 (samma i 9913). Provat:* `EXP(292)` *= 6.51628E126,* `EXP(300)` *fel 4,* `EXP(-300)` *= 0.*

```
3C03  50           FEXP:  LD   D,B             DE = x (fel)
3C04  59                  LD   E,C
3C05  21 31 3D             LD   HL,FLGE         HL = .434294
3C08  CD DF 34             CALL FMUL            y = x * LOG10(e)
3C0B  C5                   PUSH BC              spara y:s plats
3C0C  CD C5 3B             CALL FLDX            x = y, ASEND = y:s plats
3C0F  E5                   PUSH HL              spara $FDD4
3C10  CD CA 3E             CALL FTOINT          $FDD4 = INT(y), DE = n
3C13  E1                   POP  HL              HL = $FDD4
3C14  C1                   POP  BC              BC = y
3C15  D5                   PUSH DE              n
3C16  50                   LD   D,B             DE = y
3C17  59                   LD   E,C
3C18  CD 60 34             CALL FSUB            f = y - INT(y), på y:s plats
3C1B  D5                   PUSH DE              f:s plats
3C1C  01 7E 3C             LD   BC,EXPSER       serien
3C1F  CD 61 3F             CALL FPOLY           10^f i f:s plats, 1..10
3C22  E1                   POP  HL              HL = resultatet
3C23  C1                   POP  BC              BC = n
3C24  5E                   LD   E,(HL)          E = 10^f:s exponent
3C25  AF                   XOR  A
3C26  57                   LD   D,A             DE = exponenten i 16 bitar
3C27  B3                   OR   E               0?
3C28  28 0F                JR   Z,FCLR5         10^f är 0: resultatet 0
3C2A  EB                   EX   DE,HL
3C2B  09                   ADD  HL,BC           exponenten + n
3C2C  EB                   EX   DE,HL           DE = summan, HL = resultatets exponent
3C2D  7A                   LD   A,D
3C2E  87                   ADD  A,A             CY = bit 15
3C2F  38 08                JR   C,FCLR5         < 0: resultatet 0
3C31  C2 01 07             JP   NZ,L0701        > $FF: fel 4
3C34  B3                   OR   E               D = 0: är E 0?
3C35  28 02                JR   Z,FCLR5         = 0: resultatet 0
3C37  73                   LD   (HL),E          ny exponent
3C38  C9                   RET
```

`FCLR5`*: nollställ de 5 byte som slutar vid HL (talet blir 0).*

```
3C39  06 05        FCLR5: LD   B,$05
3C3B  AF                  XOR  A
3C3C  77           L3C3C: LD   (HL),A
3C3D  2B                  DEC  HL
3C3E  10 FC               DJNZ L3C3C
3C40  C9                  RET
```

`EXP`*-serien, 12 koefficienter: 1 2.30259 2.65095 ... .000241667 (LN(10)^k/k!, k = 0..11), sista (k = 11) vid* `EXPSER`*.*

```
3C41  FF FF                DEFB $FF,$FF           slutet för FPOLY
3C43  10 00 00 00…         DEFB $10,$00,$00,$00,$81
                                                  1
3C48  23 02 59 00…         DEFB $23,$02,$59,$00,$81
                                                  2.30259
3C4D  26 50 95 00…         DEFB $26,$50,$95,$00,$81
                                                  2.65095
```

```
3C52  20 34 68 00…          DEFB $20,$34,$68,$00,$81
                                                     2.03468
3C57  11 71 25 00…          DEFB $11,$71,$25,$00,$81
                                                     1.17125
3C5C  53 93 83 00…          DEFB $53,$93,$83,$00,$80
                                                     0.539383
3C61  20 69 96 00…          DEFB $20,$69,$96,$00,$80
                                                     0.206996
3C66  68 08 94 00…          DEFB $68,$08,$94,$00,$7F
                                                     0.0680894
3C6B  19 59 77 00…          DEFB $19,$59,$77,$00,$7F
                                                     0.0195977
3C70  50 13 93 00…          DEFB $50,$13,$93,$00,$7E
                                                     0.00501393
3C75  11 54 50 00…          DEFB $11,$54,$50,$00,$7E
                                                     0.0011545
3C7A  24 16 67 00           DEFB $24,$16,$67,$00     0.000241667
3C7E  7D          EXPSER:   DEFB $7D                 exponenten; EXPSER pekar på c11
```

FINT: INT(x), *det största heltalet <= x (golv). BC = talets exponent, talet ändras på plats.*

Antalet siffror efter kommat är 6 - (e - $80). e >= $86: talet är redan ett heltal. e <= $80 (|x| < 1): exponenten sätts till 0 och alla 6 siffror tas bort. Siffrorna nollställs bakifrån, och E samlar dem (OR), så att E <> 0 om något togs bort. Var talet negativt och något togs bort dras 1 av: INT(*-2.5*) = *-3,* INT(-.5) = *-1.*

Provat: INT(2.5) = *2,* INT(*-3*) = *-3,* INT(*-12345.6*) = *-12346.*

```
3C7F  0A          FINT:     LD   A,(BC)          exponenten
3C80  D6 81                 SUB  $81             - $81
3C82  F2 8B 3C              JP   P,FINT1         e >= $81, eller e = 0 (x = 0)
3C85  AF                    XOR  A
3C86  02                    LD   (BC),A          |x| < 1: exponenten 0
3C87  3E FA                 LD   A,$FA           alla 6 siffror bort
3C89  18 03                 JR   FINT2

3C8B  D6 05       FINT1:    SUB  $05             A = -(siffror efter kommat)
3C8D  D0                    RET  NC              NC: e >= $86, inga siffror efter kommat
3C8E  57          FINT2:    LD   D,A             D = -antalet
3C8F  1E 00                 LD   E,$00           E = borttagna siffror
3C91  0B                    DEC  BC              tecknet
3C92  C5                    PUSH BC              spara tecknets adress
3C93  0B          FINTLP:   DEC  BC              sista kvarvarande byten
3C94  0A                    LD   A,(BC)
3C95  E6 0F                 AND  $0F             låga siffran
3C97  B3                    OR   E
3C98  5F                    LD   E,A
3C99  0A                    LD   A,(BC)
3C9A  E6 F0                 AND  $F0             behåll den höga
3C9C  02                    LD   (BC),A
3C9D  14                    INC  D               en siffra till borttagen
3C9E  28 07                 JR   Z,FINTFL        Z: udda antal, klart
3CA0  B3                    OR   E               och den höga
3CA1  5F                    LD   E,A
3CA2  AF                    XOR  A               hela byten bort
3CA3  02                    LD   (BC),A
3CA4  14                    INC  D
3CA5  20 EC                 JR   NZ,FINTLP
3CA7  C1          FINTFL:   POP  BC              BC = tecknet
3CA8  7B                    LD   A,E
3CA9  A7                    AND  A               inget borttaget
3CAA  C8                    RET  Z
3CAB  0A                    LD   A,(BC)          A = tecknet
```

```
3CAC  A7                AND  A
3CAD  C8                RET  Z              positivt: klart
3CAE  03                INC  BC             exponenten
3CAF  59                LD   E,C            DE = exponenten
3CB0  50                LD   D,B
3CB1  21 BB 3F          LD   HL,FONE        1
3CB4  C3 60 34          JP   FSUB           x = x - 1, och RET
```

FLOG10: LOG10(x) = LOG(x) * LOG10(e). *Provat:* LOG10(1000) = *3*, LOG10(2) = *.301031* (*.301030*).

```
3CB7  C5          FLOG10: PUSH BC             spara argumentet
3CB8  CD C4 3C            CALL FLOG
3CBB  C1                  POP  BC             BC = argumentet
3CBC  59                  LD   E,C            DE = LOG(x)
3CBD  50                  LD   D,B
3CBE  21 31 3D            LD   HL,FLGE        LOG10(e)
3CC1  C3 DF 34            JP   FMUL           = LOG(x) * LOG10(e), och RET
```

FLOG: LOG(x), *naturliga logaritmen. x <= 0 ger fel 4.*

*Reduktion: x dubblas (*FADD *med sig själv) k gånger tills första siffran är 1 (k = 0..3: 1 ger 0, 2 ger 3, 3-4 ger 2, 5-9 ger 1). Med exponenten d efter dubblingen sätts exponenten till* $81*, så att y = 1,d2d3... ligger i [1, 2). Då är*

```
LOG(x) = LOG(y) - k * LN 2 + (d - $81) * LN 10.
```

LOG(y) = *P(y - 1) med serien* LOGSER *(c0 = 0, .999981 -.499471 ...). -k * LN 2 hämtas ur tabellen* FMLN2 *(-LN 2, -2 LN 2, -3 LN 2). Nära 1 tappar LOG siffror, eftersom c1 = .999981:* LOG(1.001)*1000 = *.999482 (prövat).*

```
3CC4  CD C5 3B    FLOG:   CALL FLDX           x = argumentet
3CC7  CA 01 07            JP   Z,L0701        LOG 0: fel 4
3CCA  FA 01 07            JP   M,L0701        negativt: fel 4
3CCD  58                  LD   E,B            E = k = 0 (B = 0 efter FLDX)
3CCE  3A D0 FD    LOGDBL: LD   A,(ASBBEG)     första siffran
3CD1  E6 F0               AND  $F0
3CD3  FE 10               CP   $10            = 1?
3CD5  28 0C               JR   Z,LOGRED
3CD7  1C                  INC  E              k + 1
3CD8  D5                  PUSH DE
3CD9  54                  LD   D,H            BC = DE = HL = x
3CDA  44                  LD   B,H
3CDB  5D                  LD   E,L
3CDC  4D                  LD   C,L
3CDD  CD F7 33            CALL FADD           x = x + x
3CE0  D1                  POP  DE
3CE1  18 EB               JR   LOGDBL         tills första siffran är 1

3CE3  56          LOGRED: LD   D,(HL)         D = d, exponenten efter dubblingen
3CE4  36 81               LD   (HL),$81       y = x med exponent $81
3CE6  D5                  PUSH DE             spara d och k
3CE7  44                  LD   B,H            BC = y
3CE8  4D                  LD   C,L
3CE9  EB                  EX   DE,HL          DE = y
3CEA  21 BB 3F            LD   HL,FONE        1
3CED  CD 60 34            CALL FSUB           u = y - 1
3CF0  01 6F 3D            LD   BC,LOGSER      serien
3CF3  CD 61 3F            CALL FPOLY          LN(1+u)
3CF6  D1                  POP  DE             D = d, E = k
3CF7  1D                  DEC  E              k - 1
3CF8  FA 0D 3D            JP   M,LOGEXP       k = 0: inget k LN 2
3CFB  D5                  PUSH DE             spara d
3CFC  21 3B 3D            LD   HL,FMLN2       HL = -LN 2
```

```
3CFF  16 00              LD   D,$00
3D01  3E 05              LD   A,$05          HL + 5 * (k - 1)
3D03  19          LOGK:  ADD  HL,DE
3D04  3D                 DEC  A
3D05  20 FC              JR   NZ,LOGK
3D07  50                 LD   D,B            DE = LN(y)
3D08  59                 LD   E,C
3D09  CD F7 33           CALL FADD           $FDD4 = LN(y) - k LN 2
3D0C  D1                 POP  DE             D = d
```

LOGEXP: *(d -* $81*) blir ett flyttal (*INTTOF*) i* $FDD9*, gånger LN 10, och läggs till* LOG(y) *i* $FDD4*. Resultatet skrivs vid* ASEND*.*

```
3D0D  C5          LOGEXP: PUSH BC             spara $FDD4
3D0E  6A                  LD   L,D            HL = d
3D0F  26 00               LD   H,$00
3D11  11 7F FF            LD   DE,$FF7F       - $81
3D14  19                  ADD  HL,DE          HL = d - $81
3D15  11 D9 FD            LD   DE,$FDD9       $FDD9
3D18  D5                  PUSH DE
3D19  CD 18 3F            CALL INTTOF         $FDD9 = d - $81
3D1C  D1                  POP  DE
3D1D  42                  LD   B,D            BC = $FDD9
3D1E  4B                  LD   C,E
3D1F  21 36 3D            LD   HL,FLN10       LN 10
3D22  CD DF 34            CALL FMUL           $FDD9 = (d - $81) * LN 10
3D25  ED 4B E9 FD         LD   BC,(ASEND)     resultatet
3D29  E1                  POP  HL             HL = $FDD4
3D2A  C3 F7 33            JP   FADD           = LOG(x), och RET
```

Konstanter: LOG10(e) *= .434294 (*FLGE*), LN 10 (*FLN10*), och* FMLN2*: -LN 2, -2 LN 2, -3 LN 2 (steg om 5 byte).*

```
3D2D  43 42 94 00         DEFB $43,$42,$94,$00  0.434294
3D31  80          FLGE:   DEFB $80              exponenten; FLGE = talets adress
3D32  23 02 59 00         DEFB $23,$02,$59,$00  2.30259
3D36  81          FLN10:  DEFB $81              exponenten; FLN10
3D37  69 31 47 01         DEFB $69,$31,$47,$01  -0.693147
3D3B  80          FMLN2:  DEFB $80              exponenten; FMLN2 = -LN 2
3D3C  13 86 29 01…        DEFB $13,$86,$29,$01,$81
                                                -1.38629
3D41  20 79 44 01…        DEFB $20,$79,$44,$01,$81
                                                -2.07944
```

LOG*-serien för LN(1+u), u i [0, 1): 8 koefficienter, c0 = 0, c7 vid* LOGSER*.*

```
3D46  FF FF               DEFB $FF,$FF          slutet för FPOLY
3D48  00 00 00 00…        DEFB $00,$00,$00,$00,$00
                                                0
3D4D  99 99 81 00…        DEFB $99,$99,$81,$00,$80
                                                0.999981
3D52  49 94 71 01…        DEFB $49,$94,$71,$01,$80
                                                -0.499471
3D57  32 82 33 00…        DEFB $32,$82,$33,$00,$80
                                                0.328233
3D5C  22 58 73 01…        DEFB $22,$58,$73,$01,$80
                                                -0.225873
3D61  13 46 39 00…        DEFB $13,$46,$39,$00,$80
                                                0.134639
3D66  55 12 00 01…        DEFB $55,$12,$00,$01,$7F
                                                -0.05512
```

```
3D6B  10 75 74 00          DEFB $10,$75,$74,$00  0.0107574
3D6F  7F           LOGSER: DEFB $7F              exponenten; LOGSER pekar på c7
```

3.99 FSIN / FCOS $3D70

FCOS: COS(x) = SIN(x + PI/2): argumentets tecken nollställs (COS är jämn) och kvadranten börjar på 1. FSIN: kvadranten börjar på 0. Negativt x: kvadranten + 2, och x görs positivt (SIN(-x) = SIN(x + PI)). q = |x| × 2/PI, n = INT(q), kvadranten + n (bara de två lägsta bitarna används). Vinkeln v = (q - n) × PI/2 i [0, PI/2).

Udda kvadrant: COS(v) = C(v^2) (COSSER), annars SIN(v) = v × S(v^2) (SINSER). Kvadrant 2 och 3 (bit 1): tecknet byts. q räknas med 2/PI = .63662, och n måste rymmas i 16 bitar (FTOINT), så |x| över ungefär 102940 ger fel 7. Provat: SIN(103000) ger fel 7, SIN(102900) ger .309017. COS(1.570796) (läst som 1.5708) blir 00 00 00 01 00, en negativ nolla (rätt värde -.00000367), som uppför sig som 0: X=0 är sant och SGN(X) ger 0. TAN(1.570796) ger därför fel 4. In: BC = argumentets exponent. Ut: resultatet på argumentets plats. Använder $FDD0-$FDDE, SINQ och ASEND.

```
3D70  AF           FCOS:   XOR  A             FCOS: kvadrant 1
3D71  0B                   DEC  BC            BC = tecknet
3D72  02                   LD   (BC),A        COS(-x) = COS(x)
3D73  03                   INC  BC            BC = exponenten
3D74  3C                   INC  A             A = 1
3D75  18 01                JR   FSINC

3D77  AF           FSIN:   XOR  A             FSIN: kvadrant 0
3D78  F5           FSINC:  PUSH AF            spara kvadranten
3D79  CD C5 3B             CALL FLDX          x = argumentet
3D7C  F1                   POP  AF
3D7D  2B                   DEC  HL            tecknet
3D7E  86                   ADD  A,(HL)
3D7F  86                   ADD  A,(HL)        + 2 om negativt
3D80  32 E4 FD             LD   (SINQ),A      kvadranten
3D83  36 00                LD   (HL),$00      |x|
3D85  01 D4 FD             LD   BC,$FDD4
3D88  C5                   PUSH BC            spara $FDD4
3D89  50                   LD   D,B
3D8A  59                   LD   E,C           DE = $FDD4
3D8B  21 A5 3F             LD   HL,F2DPI      2/PI
3D8E  CD DF 34             CALL FMUL          q = |x| * 2/PI
3D91  EB                   EX   DE,HL         HL = q
3D92  11 D9 FD             LD   DE,$FDD9      $FDD9
3D95  D5                   PUSH DE
3D96  D5                   PUSH DE
3D97  01 05 00             LD   BC,$0005
3D9A  ED B8                LDDR               $FDD9 = q
3D9C  E1                   POP  HL            HL = $FDD9
3D9D  CD CA 3E             CALL FTOINT        $FDD9 = INT(q), DE = n
3DA0  21 E4 FD             LD   HL,$FDE4      HL = SINQ
3DA3  7E                   LD   A,(HL)
3DA4  83                   ADD  A,E           kvadranten + n
3DA5  77                   LD   (HL),A
3DA6  E1                   POP  HL            HL = INT(q)
3DA7  C1                   POP  BC            BC = q
3DA8  E5                   PUSH HL
3DA9  59                   LD   E,C
3DAA  50                   LD   D,B
3DAB  CD 60 34             CALL FSUB          $FDD4 = q - INT(q)
```

```
3DAE  21 A0 3F            LD   HL,FPIH          PI/2
3DB1  CD DF 34            CALL FMUL             $FDD4 = v = (q - n) * PI/2
3DB4  EB                  EX   DE,HL            HL = $FDD4
3DB5  D1                  POP  DE               DE = $FDD9
3DB6  E5                  PUSH HL               $FDD4 tre gånger
3DB7  E5                  PUSH HL
3DB8  E5                  PUSH HL
3DB9  01 05 00            LD   BC,$0005
3DBC  ED B8               LDDR                  $FDD9 = v
3DBE  E1                  POP  HL               HL = DE = BC = $FDD4
3DBF  D1                  POP  DE
3DC0  C1                  POP  BC
3DC1  CD DF 34            CALL FMUL             $FDD4 = v^2
3DC4  3A E4 FD            LD   A,(SINQ)
3DC7  0F                  RRCA                  udda kvadrant?
3DC8  38 10               JR   C,SINCOS
3DCA  01 D4 3F            LD   BC,SINSER        SIN-serien
3DCD  CD 61 3F            CALL FPOLY            $FDD4 = S(v^2)
3DD0  21 D9 FD            LD   HL,$FDD9         v
3DD3  42                  LD   B,D              BC = $FDD4
3DD4  4B                  LD   C,E
3DD5  CD DF 34            CALL FMUL             SIN(v) = v × S(v^2)
3DD8  18 06               JR   SINSGN

3DDA  01 F4 3F     SINCOS: LD   BC,COSSER       COS-serien
3DDD  CD 61 3F             CALL FPOLY           COS(v) = C(v^2)
3DE0  EB           SINSGN: EX   DE,HL           HL = $FDD4, resultatet
3DE1  3A E4 FD             LD   A,(SINQ)
3DE4  2B                   DEC  HL              tecknet
3DE5  CB 4F                BIT  1,A             kvadrant 2, 3:
3DE7  C4 C0 3B             CALL NZ,FNEG         byt tecken
3DEA  23                   INC  HL
3DEB  ED 5B E9 FD          LD   DE,(ASEND)      DE = argumentets exponent
3DEF  01 05 00             LD   BC,$0005
3DF2  ED B8                LDDR                 resultatet på argumentets plats
3DF4  C9                   RET
```

3.100 FSQR: kvadratrot, siffra för siffra med udda tal $3DF5

Arbetsyta IX+0..$0F (SP = IX):

```
IX+0         antal kvarvarande byte  i roten (3, 2, 1)
IX+3..       o, udda tal (BCD) som dras från resten
IX+8..IX+10  roten, 6 siffror
IX+$0C..$0F  resten; talets mantissa läggs i IX+$0D..$0F
```

Exponenten: e = (e-$80)/2 avrundat uppåt, + $80 (SRA och RL C ger C = 1 om e-$80 var udda). Negativt tal: fel 50.

För varje siffra: dra o från resten, lägg 1 till rotens siffra och 2 till o (o = 1, 3, 5 och så vidare, i siffrans position C = $10 eller $01) tills det blir lån. Då läggs o tillbaka, o minskas med 1 i positionen C och får en etta i positionen till höger (nästa byte när C = $01), och resten skiftas en siffra åt vänster (RLD över 4 byte). Udda exponent börjar med o = 1 i den höga halvan, så att talet läses som .0d1d2... Efter 6 siffror kopieras roten till mantissan. Sista siffran huggs av: SQR(10) ger 3.16227 (3.162278).

SQR(0): exponenten blir $40 med mantissan 0 (inte $00). Talet skrivs 0 och fungerar som 0 i FADD och FMUL (X+1 ger 1, X*5 ger 0), men X=0 är falskt och SGN(X) ger 1. 5/SQR(0) fastnar i FDIV, som aldrig får lån när den drar av 0, och CTRL-C bryter inte. Version 9913 ger 0 direkt.

In: BC = talets exponent. Ut: roten på talets plats. Använder 16 byte på stacken. Förstör A, BC, DE, HL och IX.

```
3DF5  21 FF FF     FSQR:   LD   HL,$FFFF        HL = SP0 - 1 = IX+$0F (fel)
3DF8  39                   ADD  HL,SP
3DF9  DD 21 F0 FF          LD   IX,$FFF0        IX = SP0 - 16
3DFD  DD 39                ADD  IX,SP
3DFF  DD F9                LD   SP,IX           SP = IX
3E01  DD 36 00 03          LD   (IX+$00),$03    3 byte = 6 siffror
3E05  EB                   EX   DE,HL           DE = IX+$0F
3E06  60                   LD   H,B             HL = talets exponent
3E07  69                   LD   L,C
3E08  7E                   LD   A,(HL)          exponenten
3E09  EE 80                XOR  $80             med tecken
3E0B  CB 2F                SRA  A               / 2
3E0D  0E 00                LD   C,$00
3E0F  CB 11                RL   C               C = udda
3E11  81                   ADD  A,C             avrunda uppåt
3E12  EE 80                XOR  $80             exponenten med $80 igen
3E14  77                   LD   (HL),A          ny exponent
3E15  79                   LD   A,C             A = 1 om e - $80 udda
3E16  2B                   DEC  HL              tecknet
3E17  CB 46                BIT  0,(HL)
3E19  C2 25 07             JP   NZ,L0725        negativt: fel 50
3E1C  2B                   DEC  HL              sista mantissabyten
3E1D  E5                   PUSH HL              resultatets plats
3E1E  D5                   PUSH DE              spara IX+$0F
3E1F  01 03 00             LD   BC,$0003
3E22  ED B8                LDDR                 mantissan till IX+$0D..$0F
3E24  13                   INC  DE              IX+$0D
3E25  D5                   PUSH DE              spara IX+$0D
3E26  1B                   DEC  DE              IX+$0C
3E27  EB                   EX   DE,HL           HL = IX+$0C, C = 0 efter LDDR
3E28  06 0A                LD   B,$0A           nollställ IX+3..IX+$0C
3E2A  71           SQRCLR: LD   (HL),C
3E2B  2B                   DEC  HL
3E2C  10 FC                DJNZ SQRCLR
3E2E  23                   INC  HL              HL = IX+4
3E2F  23                   INC  HL
3E30  D1                   POP  DE              DE = IX+$0D
3E31  1F                   RRA                  udda exponent?
3E32  30 59                JR   NC,SQRBYT
3E34  36 10                LD   (HL),$10        o = 1 i höga halvan
3E36  4E                   LD   C,(HL)          C = $10, höga siffran
3E37  3E 05        SQRLP:  LD   A,$05           antal byte
3E39  DD 96 00             SUB  (IX+$00)
3E3C  47                   LD   B,A             B = 5 - (IX+0) byte
3E3D  E5                   PUSH HL              spara o:s sista byte
3E3E  D5                   PUSH DE              och restens
3E3F  A7                   AND  A               CY = 0
3E40  1A           SQRSUB: LD   A,(DE)
3E41  9E                   SBC  A,(HL)          resten - o
3E42  27                   DAA
3E43  12                   LD   (DE),A
3E44  2B                   DEC  HL
3E45  1B                   DEC  DE
3E46  10 F8                DJNZ SQRSUB
3E48  D1                   POP  DE              DE = restens sista byte
3E49  38 1D                JR   C,SQRBOR        lån: siffran klar
3E4B  E1                   POP  HL              HL = o:s sista byte
3E4C  E5                   PUSH HL
```

```
3E4D  7E                LD   A,(HL)          o + C
3E4E  81                ADD  A,C
3E4F  27                DAA
3E50  47                LD   B,A            B = o + C
3E51  30 08             JR   NC,SQRDIG      ingen minnessiffra
3E53  2B          SQRCY: DEC  HL
3E54  7E                LD   A,(HL)
3E55  C6 01             ADD  A,$01          minnet till byte före
3E57  27                DAA
3E58  77                LD   (HL),A
3E59  38 F8             JR   C,SQRCY
3E5B  21 FB FF    SQRDIG: LD   HL,$FFFB     rotens byte (DE - 5)
3E5E  19                ADD  HL,DE
3E5F  79                LD   A,C            A = C
3E60  86                ADD  A,(HL)         roten + C
3E61  77                LD   (HL),A
3E62  E1                POP  HL             HL = o:s sista byte
3E63  78                LD   A,B            A = o + C
3E64  81                ADD  A,C            jämn siffra + C: ingen DAA behövs
3E65  77                LD   (HL),A         o = o + 2C
3E66  18 CF             JR   SQRLP          dra av igen

3E68  7D          SQRBOR: LD   A,L          A = L efter subtraktionen
3E69  E1                POP  HL             HL = o:s sista byte
3E6A  E5                PUSH HL             spara den
3E6B  D5                PUSH DE             och restens sista byte
3E6C  2F                CPL                 B = antal byte
3E6D  3C                INC  A
3E6E  85                ADD  A,L
3E6F  A7                AND  A
3E70  47                LD   B,A            B = byte i subtraktionen
3E71  1A          SQRADD: LD   A,(DE)
3E72  8E                ADC  A,(HL)         lägg tillbaka o
3E73  27                DAA
3E74  12                LD   (DE),A
3E75  2B                DEC  HL
3E76  1B                DEC  DE
3E77  10 F8             DJNZ SQRADD
3E79  D1                POP  DE
3E7A  E1                POP  HL             HL = o:s sista byte
3E7B  7E                LD   A,(HL)
3E7C  91                SUB  C              o - C
3E7D  77                LD   (HL),A
3E7E  0D                DEC  C              C = $01?
3E7F  28 05             JR   Z,SQRNXT       Z: siffran var den låga
3E81  0E 01             LD   C,$01          nu låga halvan
3E83  34                INC  (HL)           o + 1
3E84  18 0A             JR   SQRSHL

3E86  DD 35 00    SQRNXT: DEC  (IX+$00)     nästa byte
3E89  28 13             JR   Z,SQREND       alla 6 siffror klara
3E8B  23                INC  HL             o:s nästa byte
3E8C  13                INC  DE             restens nästa byte
3E8D  0E 10       SQRBYT: LD   C,$10        o + 1 i höga halvan
3E8F  71                LD   (HL),C
3E90  E3          SQRSHL: EX   (SP),HL      HL = IX+$0F
3E91  E5                PUSH HL             spara IX+$0F
3E92  06 04             LD   B,$04          4 byte
3E94  AF                XOR  A              A = 0: nolla in till höger
3E95  ED 6F       SQRRLD: RLD               resten en siffra åt vänster
3E97  2B                DEC  HL
3E98  10 FB             DJNZ SQRRLD
```

```
3E9A  E1                POP   HL              HL = IX+$0F
3E9B  E3                EX    (SP),HL         HL = o:s sista byte
3E9C  18 99             JR    SQRLP           nästa siffra

3E9E  01 04 00    SQREND: LD    BC,$0004        roten = HL + 4
3EA1  09                ADD   HL,BC
3EA2  D1                POP   DE              DE = IX+$0F
3EA3  D1                POP   DE              DE = resultatets mantissa
3EA4  0B                DEC   BC              BC = 3
3EA5  ED B8             LDDR                  kopiera 3 byte
3EA7  21 10 00          LD    HL,$0010        ta bort arbetsytan
3EAA  39                ADD   HL,SP
3EAB  F9                LD    SP,HL           SP = SP0, returadressen
3EAC  C9                RET                   till FFUNC
```

FTAN: TAN(x) = SIN(x) / COS(x). x kopieras till $FDDF-$FDE3, där COS räknas, och SIN räknas på argumentets plats. COS = 0 ger fel 4 (TAN(1.570796)). In: BC = argumentets exponent. Ut: resultatet på argumentets plats. Förstör alla register utom IY.

```
3EAD  60          FTAN: LD    H,B
3EAE  69                LD    L,C
3EAF  11 E3 FD          LD    DE,$FDE3        DE = $FDE3
3EB2  D5                PUSH  DE              spara $FDE3
3EB3  C5                PUSH  BC              och argumentet
3EB4  D5                PUSH  DE              och $FDE3 till BC
3EB5  01 05 00          LD    BC,$0005
3EB8  ED B8             LDDR                  $FDE3 = x
3EBA  C1                POP   BC              BC = $FDE3
3EBB  CD 70 3D          CALL  FCOS            $FDE3 = COS(x)
3EBE  C1                POP   BC              BC = argumentet
3EBF  C5                PUSH  BC
3EC0  CD 77 3D          CALL  FSIN            argumentet = SIN(x)
3EC3  C1                POP   BC              BC = argumentet, SIN(x)
3EC4  E1                POP   HL              HL = $FDE3, COS(x)
3EC5  50                LD    D,B             DE = SIN(x)
3EC6  59                LD    E,C
3EC7  C3 EE 35          JP    FDIV            argumentet = SIN / COS, och RET
```

Korsreferenstabell för flyttalsfunktionerna. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i arbetsminnet; kolumnerna är talen som slutar vid $FDD4 (x, $FDD0–$FDD4), $FDD9, $FDDE och $FDE3, kvadranten $FDE4 (SINQ) och resultatets adress $FDE9 (ASEND). Funktionerna tar BC = argumentets exponent (från FFUNC också HL) och skriver resultatet på argumentets plats; de får förstöra alla register utom IY. FSQR har sin arbetsyta på 16 byte vid IX på stacken och tar bort den igen. FPOLY ($3F61) står efter FTOINT men hör hit: den tar DE = z och BC = tabellens slut och lämnar BC = DE = P(z) och HL som förut. ATNPOS, LOGDBL, LOGRED, LOGK, LOGEXP, FSINC, SINCOS, SINSGN och etiketterna i FINT och FSQR är delar av rutinerna och får inga egna rader. Felen: FEXP 4 och 7, FLOG och FLOG10 4, FSIN och FCOS 7 (genom FTOINT), FTAN 4 och 7, FSQR 50. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack							variabler					
Rutin	A	CY	BC	DE	HL	IX	stack	$FDD4	$FDD9	$FDDE	$FDE3	$FDE4	$FDE9
FATN	X	X	IX	X	X	X		X	X	X			X
FNEGX	X	X			U			IU					
FNEG	X	X			I								
FLDX	X	X	IX	X	U			U					U
FEXP	X	X	IX	X	X	X		X					X

	register och stack							variabler					
Rutin	A	CY	BC	DE	HL	IX	stack	$FDD4	$FDD9	$FDDE	$FDE3	$FDE4	$FDE9
FCLR5	X	X	X		I	X							
FINT	X	X	I	X	X	X							
FLOG10	X	X	I	X	X	X		X	X				X
FLOG	X	X	I	X	X	X		X	X				X
FCOS	X	X	I	X	X	X		X	X			X	X
FSIN	X	X	I	X	X	X		X	X			X	X
FSQR	X	X	I	X	X	X							
FTAN	X	X	I	X	X	X		X	X		X	X	X
FPOLY	X	X	I	U	I	X							

Rutin	**Anropar**	**Anropas från**
FATN	FLDX, FNEGX, FNEG (med JR); i ATNPOS FADD, FSUB, FDIV, FMUL, FPOLY	EXATN (genom FFUNC)
FNEGX	går in i FNEG	FATN
FNEG		FATN (med JR), FNEGX (går in i den), FSIN ($3DE7)
FLDX		FATN, FEXP, FLOG, FSIN ($3D79)
FEXP	FMUL, FLDX, FTOINT, FSUB, FPOLY, FCLR5 (med JR); fel 4	EXEXP1 (genom FFUNC), FPOW (med JP)
FCLR5		FEXP (tre ställen, med JR)
FINT	FSUB (med JP)	EXINT (genom FFUNC), FFIX, FTOINT
FLOG10	FLOG, FMUL (med JP)	EXLG10 (genom FFUNC)
FLOG	FLDX, FADD, FSUB, FPOLY, INTTOF, FMUL; fel 4	EXLOG (genom FFUNC), FLOG10, FPOW
FCOS	till FSIN ($3D78) med JR	EXCOS (genom FFUNC), FTAN
FSIN	FLDX, FMUL, FTOINT, FSUB, FPOLY, FNEG	EXSIN (genom FFUNC), FTAN, FCOS (till $3D78, med JR)
FSQR	fel 50	EXSQR (genom FFUNC)
FTAN	FCOS, FSIN, FDIV (med JP)	EXTAN (genom FFUNC)
FPOLY	FMUL, FADD	FATN ($3BAB), FEXP, FLOG ($3CF3), FSIN (två ställen)

3.101 FTOINT: flyttalet vid HL till heltal i DE $3ECA

Talet ändras först till INT(x) på plats (FINT), så -2.7 ger -3. Sedan läses e - $80 siffror med MUL10A (DE = DE × 10 + siffran). C = e - 1 räknas ned; när den blir positiv (från $80 till $7F) är alla siffror lästa. Mer än 16 bitar utan tecken: fel 7. Negativt: DE = -DE. Intervallet kontrolleras bara som 0–65535 före tecknet, så 40000 ger DE = $9C40 (-25536) och -40000 ger $63C0, 65535 ger -1, -65535 ger 1 och 65536 fel 7 (provat).

In: HL = talets exponent; talet skrivs över med INT(x). Ut: DE = heltalet. Förstör A, BC och HL. Anropas av FTOI ($2DE7), FEXP ($3C10) och FSIN ($3D9D).

```
3ECA  44           FTOINT: LD   B,H          BC = talet, FINT:s argument
3ECB  4D                   LD   C,L
3ECC  E5                   PUSH HL
3ECD  CD 7F 3C             CALL FINT          INT(x) på plats
3ED0  E1                   POP  HL
3ED1  4E                   LD   C,(HL)        exponenten
3ED2  2B                   DEC  HL
3ED3  46                   LD   B,(HL)        tecknet
```

```
3ED4  11 FD FF            LD   DE,$FFFD        -3
3ED7  19                  ADD  HL,DE           HL = första mantissabyten
3ED8  AF                  XOR  A
3ED9  5F                  LD   E,A             DE = 0
3EDA  57                  LD   D,A
3EDB  B1                  OR   C               x = 0
3EDC  C8                  RET  Z
3EDD  0D                  DEC  C               C = e - 1
3EDE  7E           FTILP: LD   A,(HL)
3EDF  0F                  RRCA                 höga siffran
3EE0  0F                  RRCA
3EE1  0F                  RRCA
3EE2  0F                  RRCA
3EE3  CD 00 3F            CALL MUL10A          DE * 10 + siffran
3EE6  38 09               JR   C,FTIERR
3EE8  0D                  DEC  C               C < $80: alla siffror lästa
3EE9  F2 F8 3E            JP   P,FTISGN
3EEC  7E                  LD   A,(HL)          låga siffran
3EED  CD 00 3F            CALL MUL10A
3EF0  23                  INC  HL              nästa byte
3EF1  DA 07 07     FTIERR: JP  C,L0707         fel 7
3EF4  0D                  DEC  C
3EF5  FA DE 3E            JP   M,FTILP         fler siffror
3EF8  05           FTISGN: DEC B               negativt?
3EF9  C0                  RET  NZ              positivt: klart
3EFA  EB                  EX   DE,HL
3EFB  CD EE 2A            CALL NEGHL           DE = -DE
3EFE  EB                  EX   DE,HL
3EFF  C9                  RET
```

MUL10A: *DE = DE * 10 + (A* AND $0F*). CY = spill. Spill i multiplikationen lämnar DE oförändrat, spill när siffran läggs till lämnar den avkortade summan. Förstör A. Anropas också av* GETDIG ($1837).

```
3F00  E5           MUL10A: PUSH HL
3F01  62                   LD   H,D
3F02  6B                   LD   L,E
3F03  29                   ADD  HL,HL           * 2
3F04  38 10                JR   C,MUL10X
3F06  29                   ADD  HL,HL           * 4
3F07  38 0D                JR   C,MUL10X
3F09  19                   ADD  HL,DE           * 5
3F0A  38 0A                JR   C,MUL10X
3F0C  29                   ADD  HL,HL           * 10
3F0D  38 07                JR   C,MUL10X
3F0F  16 00                LD   D,$00
3F11  E6 0F                AND  $0F
3F13  5F                   LD   E,A
3F14  19                   ADD  HL,DE           + siffran
3F15  EB                   EX   DE,HL
3F16  E1           MUL10X: POP  HL
3F17  C9                   RET
```

3.102 INTTOF: heltalet i HL (med tecken) som flyttal vid DE (exponenten) $3F18

De fem byten nollställs. 0 blir 0. Negativt: HL = -HL och tecknet 1. INTDIG delar med 10 (IDIV) rekursivt, så att siffrorna kommer ut den högsta först. B räknar från $80 och blir exponenten: udda B lägger siffran i den höga halvan (RRD), jämnt B i den låga (OR), och går till nästa byte.

In: HL = heltalet, DE = exponentens adress. Ut: BC = exponentens adress. Förstör A, DE och HL. -32768 blir rätt: NEGHL ger $8000 igen, och det är 32768 utan tecken. Anropas av ITOF ($2DFB) och FLOG ($3D19).

```
3F18  D5           INTTOF: PUSH DE
3F19  E5                   PUSH HL
3F1A  62                   LD   H,D            HL = exponentens adress
3F1B  6B                   LD   L,E
3F1C  1B                   DEC  DE             DE = HL - 1
3F1D  36 00                LD   (HL),$00       exponenten 0
3F1F  01 04 00             LD   BC,$0004
3F22  ED B8                LDDR                nollställ 5 byte
3F24  EB                   EX   DE,HL          DE = första mantissabyten
3F25  E1                   POP  HL             HL = heltalet
3F26  C1                   POP  BC             BC = exponentens plats
3F27  7C                   LD   A,H
3F28  B5                   OR   L
3F29  C8                   RET  Z              0
3F2A  7C                   LD   A,H
3F2B  87                   ADD  A,A            CY = tecknet
3F2C  C5                   PUSH BC
3F2D  30 07                JR   NC,ITFPOS
3F2F  CD EE 2A             CALL NEGHL          HL = -HL
3F32  0B                   DEC  BC
3F33  3E 01                LD   A,$01
3F35  02                   LD   (BC),A         tecknet 1
3F36  06 80        ITFPOS: LD   B,$80          exponenten från $80
3F38  CD 3F 3F             CALL INTDIG
3F3B  78                   LD   A,B
3F3C  C1                   POP  BC
3F3D  02                   LD   (BC),A         exponenten
3F3E  C9                   RET
```

INTDIG: skriv HL:s decimala siffror vid DE, den högsta först. Siffrorna fås med IDIV (HL / 10), och rutinen anropar sig själv för kvoten innan den skriver sin egen siffra, högst fem nivåer. B ökar med 1 för varje siffra. Ut: B = $80 + antalet siffror, DE = byten där nästa siffra skulle stå. Förstör A och HL.

```
3F3F  D5           INTDIG: PUSH DE
3F40  C5                   PUSH BC
3F41  11 0A 00             LD   DE,$000A
3F44  CD 88 37             CALL IDIV           HL = HL / 10, E = resten
3F47  7C                   LD   A,H
3F48  B5                   OR   L              Z: kvoten 0
3F49  C1                   POP  BC
3F4A  7B                   LD   A,E            A = siffran
3F4B  D1                   POP  DE
3F4C  F5                   PUSH AF
3F4D  C4 3F 3F             CALL NZ,INTDIG      högre siffror först
3F50  04                   INC  B              exponenten + 1
3F51  78                   LD   A,B
3F52  0F                   RRCA                udda: höga halvan
3F53  EB                   EX   DE,HL          HL = byten, DE = heltalet
3F54  38 06                JR   C,INTDHI
```

```
3F56  F1              POP   AF          A = siffran
3F57  B6              OR    (HL)        låga halvan
3F58  77              LD    (HL),A
3F59  EB              EX    DE,HL
3F5A  13              INC   DE          nästa byte
3F5B  C9              RET

3F5C  F1        INTDHI: POP   AF
3F5D  ED 67           RRD               siffran till höga halvan
3F5F  EB              EX    DE,HL
3F60  C9              RET
```

Korsreferenstabell för omvandlingen mellan flyttal och heltal. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren och på stacken. FTOINT tar HL = flyttalets exponent, skriver INT(x) över talet och lämnar heltalet i DE; FTILP, FTIERR och FTISGN är delar av den. MUL10A ger DE = DE × 10 + siffran i A och svarar med CY vid spill. INTTOF tar HL = heltalet och DE = exponentens adress och lämnar adressen i BC; ITFPOS är en del av den. INTDIG skriver HL:s siffror vid DE och räknar upp B. Felen: fel 7 i FTOINT. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack						
Rutin	A	CY	BC	DE	HL	IX	stack
FTOINT	X	X	X	U	IX		
MUL10A	IX	U		IU			
INTTOF	X	X	U	IX	IX		
INTDIG	X	X	IU	IU	IX		

Rutin	**Anropar**	**Anropas från**
FTOINT	FINT, MUL10A, NEGHL, fel 7	FTOI ($2DE7), FEXP ($3C10), FSIN ($3D9D)
MUL10A		GETLN ($1837), FTOINT (två ställen)
INTTOF	NEGHL, INTDIG	ITOF ($2DFB), FLOG ($3D19)
INTDIG	IDIV, INTDIG	INTTOF ($3F38), sig själv ($3F4D)

3.103 FPOLY: P(z) = c0 + c1 z + ... + cn z^n med Horners metod

$3F61

In: DE = z (skrivs över med P(z)), BC = cn:s exponent (tabellens slut). HL bevaras. z kopieras till fem byte på stacken och cn till DE. Sedan

```
s = s * z + c(k-1)
```

för varje post bakåt i tabellen (fem byte per post). Tabellen slutar när byten där nästa posts tecken skulle ligga har bit 7 satt ($FF). Ut: BC = DE = P(z), HL som förut. Förstör A och IX (via FMUL och FADD).

```
3F61  E5        FPOLY:  PUSH  HL          spara HL
3F62  21 FB FF          LD    HL,$FFFB    HL = SP - 5
3F65  39                ADD   HL,SP
3F66  F9                LD    SP,HL       SP = HL, 5 byte för z
3F67  C5                PUSH  BC          spara tabellens slut
3F68  01 04 00          LD    BC,$0004
3F6B  09                ADD   HL,BC       stackplatsens exponent
```

```
3F6C  E5                PUSH HL             spara platsens exponent
3F6D  D5                PUSH DE             och z
3F6E  EB                EX   DE,HL          DE = platsen, HL = z
3F6F  03                INC  BC             BC = 5
3F70  ED B8             LDDR                kopiera z dit
3F72  D1                POP  DE             DE = z
3F73  E1                POP  HL             HL = platsens exponent
3F74  E3                EX   (SP),HL        HL = tabellen, (SP) = z
3F75  D5                PUSH DE
3F76  0E 05             LD   C,$05          BC = 5
3F78  ED B8             LDDR                s = cn, på z:s plats
3F7A  D1                POP  DE             DE = s (z:s plats)
3F7B  42                LD   B,D            BC = DE = s
3F7C  4B                LD   C,E
3F7D  E3          POLYLP: EX   (SP),HL      HL = z
3F7E  CD DF 34          CALL FMUL           s = s * z
3F81  E3                EX   (SP),HL        HL = tabellen
3F82  CD F7 33          CALL FADD           s = s + ck
3F85  C5                PUSH BC             spara s
3F86  01 FA FF          LD   BC,$FFFA       -6
3F89  09                ADD  HL,BC          HL - 6: tecknet i posten före
3F8A  C1                POP  BC             BC = s
3F8B  7E                LD   A,(HL)         A = tecknet
3F8C  23                INC  HL             HL - 5: postens exponent
3F8D  87                ADD  A,A            CY = bit 7, satt i $FF
3F8E  30 ED             JR   NC,POLYLP      0 eller 1: ett varv till
3F90  21 07 00          LD   HL,$0007       ta bort 7 byte
3F93  39                ADD  HL,SP
3F94  F9                LD   SP,HL
3F95  E1                POP  HL             HL tillbaka
3F96  C9                RET
```

Konstanter: 2 och LN 2 (.693147, $3FAB) används inte (ingen instruktion pekar dit). PI/2 (FPIH), 2/PI (F2DPI), PI (FPI; EXPI kopierar de 5 byten från $3FA6), PI/4 (FPIQ). $FF avslutar SIN-serien, och FONE (1) är dess c0.

```
3F97  20 00 00 00…      DEFB $20,$00,$00,$00,$81
                                            2
3F9C  15 70 80 00       DEFB $15,$70,$80,$00  1.5708
3FA0  81          FPIH: DEFB $81              exponenten; FPIH = PI/2
3FA1  63 66 20 00       DEFB $63,$66,$20,$00  0.63662
3FA5  80          F2DPI: DEFB $80             exponenten; F2DPI = 2/PI
3FA6  31 41 59 00… FPI: DEFB $31,$41,$59,$00,$81
                                            3.14159
3FAB  69 31 47 00…      DEFB $69,$31,$47,$00,$80
                                            0.693147
3FB0  78 53 98 00       DEFB $78,$53,$98,$00  0.785398
3FB4  80          FPIQ: DEFB $80              exponenten; FPIQ = PI/4
3FB5  FF 00             DEFB $FF,$00          slutet för FPOLY i SIN-serien
3FB7  10 00 00 00       DEFB $10,$00,$00,$00  1
3FBB  81          FONE: DEFB $81              exponenten; FONE = 1, SIN-seriens c0
```

SIN-serien för SIN(v)/v i v^2: 1 (FONE) -1/3! 1/5! ... -1/11!, sista vid SINSER.

```
3FBC  16 66 67 01…      DEFB $16,$66,$67,$01,$80
                                            -0.166667
3FC1  83 33 33 00…      DEFB $83,$33,$33,$00,$7E
                                            0.00833333
3FC6  19 84 13 01…      DEFB $19,$84,$13,$01,$7D
                                            -0.000198413
```

```
3FCB  27 55 73 00…          DEFB $27,$55,$73,$00,$7B
                                                     2.75573e-06
3FD0  25 05 21 01           DEFB $25,$05,$21,$01     -2.50521e-08
3FD4  79            SINSER: DEFB $79                 exponenten; SINSER pekar på sista termen
```

COS-serien i v^2: 1 -1/2! 1/4! ... -1/10!, sista vid COSSER.

```
3FD5  FF 00                 DEFB $FF,$00             slutet för FPOLY i COS-serien
3FD7  10 00 00 00…          DEFB $10,$00,$00,$00,$81
                                                     1
3FDC  50 00 00 01…          DEFB $50,$00,$00,$01,$80
                                                     -0.5
3FE1  41 66 67 00…          DEFB $41,$66,$67,$00,$7F
                                                     0.0416667
3FE6  13 88 89 01…          DEFB $13,$88,$89,$01,$7E
                                                     -0.00138889
3FEB  24 80 16 00…          DEFB $24,$80,$16,$00,$7C
                                                     2.48016e-05
3FF0  27 55 73 01           DEFB $27,$55,$73,$01     -2.75573e-07
3FF4  7A            COSSER: DEFB $7A                 exponenten; COSSER pekar på sista termen
```

Oanvänt ($FF) *till slutet av ROM:en.*

```
3FF5  FF FF FF FF…          DEFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
3FFD  FF FF FF              DEFB $FF,$FF,$FF
```

Fel och egenheter

Det i version 11273 som inte fungerar som man kan vänta sig, ur bilagan Fel och egenheter i tolken i band 1 (där finns hela resonemanget). Adressen är rutinen eller kodraden där det händer och är en länk dit; kodraden har i sin tur en länk hit. Exemplen är prövade i emulatorn med ROM:en 11273. Felen finns likadant i version 9913, utom de i det sista avsnittet och där posten säger något annat. Sex av felen står på Luxors errata i bruksanvisningen (ABC 80 Bruksanvisning, s. 60); alla sex är rättade i 9913.

Fel

Adress	Rutin	Vad som händer
$0FDB	LNKNXT	**NEXT utan variabel ger alltid fel 28.** LNKNXT jämför variabeln efter NEXT med den i FOR. Utan variabel jämförs FOR:s variabel med 0, och det ger fel 28 vid RUN. Formen är ändå tänkt att fungera: CMPNXT godtar NEXT utan variabel, och LNKNXT har en egen väg för den (DE = 0), men jämförelsen görs ändå. Varje NEXT måste därför ha sin variabel: NEXT I, inte NEXT.
$0FDB	LNKNXT	**NEXT i direktläge ger fel 28.** I direktläge finns ingen förlänkning, och BUF1 rymmer då den inskrivna raden. LNKNXT tar radens tecken för en kedja av FOR-block som inte är tom, och jämförelsen av variablerna ger fel 28.
$1BFE	DIMELM	**DIM A(-1) ger kallstart.** Högsta index −1 ger 0 byte. ALLOC räknar då med BC = 0 − 1, alltså $FFFF byte, och nollar hela minnet, också stacken. Återhoppet går därför till $0000, och maskinen startar om som vid strömpåslag.
$3311	DIM2	**DIM A(3,2) efter DIM A(2,3) byter fältets form.** DIM2 låter DIM1 pröva det nya fältet, och DIM1 ser bara antalet element. Har det nya fältet lika många eller färre element skrivs indexens gränser in utan fel, och fältet byter form. Elementen ligger kvar i samma ordning radvis, så A(1,2) blir A(2,0).
$22B6	CMPDOT	**SETDOT och CLRDOT godtar vilket tecken som helst i stället för komma.** SETDOT r,k och CLRDOT r,k ska ge fel 61 när kommat saknas, men kontrollen slår aldrig till: SCF sätter CY just före RET NC. Tecknet efter r kopieras vad det än är, så SETDOT 5;6 och SETDOT 5X6 sätter båda grafikpunkten (5, 6). Utan k beror utfallet på det som står efter raden.
$22D6	EXDOT	**DOT utanför skärmen ger inget fel.** EXDOT prövar inte CY från DOTADR. En grafikpunkt utanför skärmen ger därför inte fel 62 utan −1 eller 0, efter vad som står i en byte i ROM:en.
$35EE	FDIV	**1E126/.1 blir ett tal som räknas som 0.** FDIV prövar exponenten innan $81 läggs till. Är skillnaden mellan exponenterna $7F blir den $00 utan fel. När kvotens första siffra är 0 rättas det av normaliseringen (1E126/.2 = 5E126), men annars står exponenten kvar som 0 med mantissan ifylld. 1E126/.1 ger då ett tal som räknas som 0 (X=0 är sant, SGN(X) = 0) men skrivs ut som 1, i stället för fel 4.
$3689	FDRND	**Avrundningen i division beror på en oinitierad byte.** När sjunde siffran i en kvot är exakt 5 avgör bit 7 i $FE19 om FDRND avrundar uppåt eller nedåt. Inget annat i ROM:en läser eller skriver $FE19, och INIT nollställer den inte, så det som står i RAM-kretsarna efter strömpåslaget avgör: .246913/2 blir .123456 eller .123457. I emulatorn blir det .123456, och efter POKE 65049,0 .123457.

Adress	Rutin	Vad som händer
$31E0	EXRND	**RND kan ge en nolla som inte är 0.** Är alla sex siffror i slumptalet 0 (ungefär en gång på en miljon) slutar normaliseringen med exponenten $7A i stället för 0. Talet skrivs ut som 0, men X=0 är falskt, X>0 sant och SGN(X) = 1.
$0DC6	CMDSAVE	**SAVE av en rad som inte ryms i ett block.** SAVE skriver ut blocket med DVPUT när nästa rad inte får plats i det. En rad som är längre än ett helt block (252 byte på kassetten) får aldrig plats, och SAVE skriver då tomma block utan slut. SAVE prövar inte heller CY från DVPUT, så ett fel vid skrivningen märks inte.
$3829	ASCMPS	**COMP% med två negativa tal.** COMP% jämför två tal som står som text. När båda är negativa och olika blir svaret alltid 1 (a > b): COMP%("-2","-1") och COMP%("-10","-5") ger 1. Efter subtraktionen av beloppen vänds tecknet på resultatet om a är negativt, också när det inte blev något lån. Lika tal, positiva tal och olika tecken jämförs rätt.
$3A12	ASALGN	**COMP% avrundar b till a:s decimaler.** Jämförelsen ställer upp talen som en addition, och har b fler decimaler än a avrundas b först. COMP%("1","1.04") och COMP%("1","0.95") ger båda 0, men COMP%("1.04","1") ger 1. Svaret är bara säkert när a har minst lika många decimaler som b.
$383B	ADIV	**DIV$ hugger av och avrundar sedan.** Kvoten räknas med en decimal mer än den ska ha, huggs av där och avrundas sedan. Siffrorna som höggs av räknas inte, så svaret kan bli fel: DIV$("-11","20",0) är −.55, som ska avrundas till −1, men kvoten huggs av till −.5 och blir 0.
$2C6C	ASCARG	**En punkt sist i a$ ger fel 66.** ASCARG räknar decimalerna efter punkten. Står punkten sist blir antalet 0, som när punkten saknas, och då läggs en punkt till. ADD$("1.","1",0) och ADD$(".","1",0) ger därför fel 66, medan ADD$("1","1.",0) ger 2.
$3C03	FEXP	**EXP av mycket stora tal.** FEXP räknar EXP(x) som 10 upphöjt till y = x · .434294. Heltalsdelen av y läggs till talets exponent, och summan prövas som ett 16-bitarstal med tecken. Över 32767 ser summan negativ ut, och det blir 0 i stället för fel 4: EXP(75100) ger fel 4, men EXP(75200) ger 0. Åt andra hållet ger EXP(-75700) 0, EXP(-75800) fel 4 och EXP(-150800) 3.16228E44. Först när y är 65536 eller mer åt något håll kommer fel 7, som för EXP(1E6).
$3494	FNORM	**Subtraktionen tappar den avrundade siffran.** Det mindre talet skiftas och avrundas innan det dras av. Blir svaret kortare skiftar FNORM in en nolla där den bortskiftade siffran borde ha stått: .666667-1.23456 ger −.56789, där rätt svar är −.567893.
$3075	EXINST	**INSTR med en tom sträng.** Är b$ tom hoppar EXINST direkt till uträkningen av positionen, men i HL står då antalet tecken som är kvar i a$ och inte en adress. Svaret blir det talet minus a$:s adress och beror på var a$ ligger: INSTR(1,A$,"") gav 16379 och INSTR(1,"","") 314 i emulatorn. Det rimliga svaret hade varit i.
$07E2	OPEN	**LOAD utan filtyp hittar inte .BAS på kassetten.** LOAD utan filtyp ska pröva .BAC och sedan .BAS. OPEN går vidare till .BAS bara om enheten svarar med CY och A = 0, men kassetten svarar med A = $AA när tidsgränsen har gått ut. På ett band med bara PROG.BAS ger LOAD CAS:PROG därför fel 21, och filen måste anges som LOAD CAS:PROG.BAS.
$033E	TSTBRK	**Ett CTRL-C kan gå förlorat.** TSTBRK läser BRKFLG och nollar den med avbrotten påslagna. Kommer avbrottet från CTRL-C mellan läsningen och skrivningen går det förlorat. Fönstret är några mikrosekunder, så det händer sällan.

Gränsfall i talen

Adress	Rutin	Vad som händer
$34DF	FMUL	**Fel 4 fast produkten får plats.** FMUL prövar exponenten före multiplikationen, med summan av de två exponenterna. .1E127*.1E1 ger fel 4, fast produkten .1E127 finns, och så gör också 1E100*1E26, 1E126*1 och 10^126%.
$2E00	POWF	**Potens via logaritmen.** a^b med flyttal räknas som EXP(b*LOG(a)). 0^2 och (-2)^2 ger fel 4, eftersom LOG inte tar tal <= 0 (men (-2)^3% går). Resultatet blir inte exakt: 2^3 ger 8.00005, medan 2^3% ger 8.
$3402	FADDC	**Avrundningen i addition.** Bara den mindre operanden avrundas, och exponentskillnaden begränsas till 6. Därför kan ett hur litet tal som helst runda upp summan: 1+5E-51 ger 1.00001. Och 1-.999999 ger 0, eftersom .999999 blir 1.00000 före subtraktionen.
$2B67	MULI	**Heltal slår runt.** Heltalsaddition och -subtraktion slår runt utan fel (32767%+1% ger -32768). Multiplikationen räknar med beloppen och ger fel 7 först när produkten är över 65535: 200%*200% ger -25536. Teckenbytet prövar inte heller -32768: -(-32768%) ger -32768. Ett tal som görs om till heltal får ligga mellan -65535 och 65535 och tas modulo 65536: A%=65535 ger -1 och A%=-32769 ger 32767, och först 65536 ger fel 7 (Isaksson och Kärrsgård 1980, s. 34).
$14FB	GETSFL	**Negativ nolla.** -0 blir ett tal med teckenbyten 1 och exponenten 0. Det räknas som 0 i jämförelser och SGN. COS(1.570796) ger också en negativ nolla, och så gör VAL("-0") och FIX(-.5), eftersom tecknet sätts tillbaka utan att talet prövas.
$1679	FLTASC	**Det minsta talet går inte att läsa in.** Talen går från .1E−127 till .999999E127. PRINT .1E-127 skriver 1E-128, men 1E-128 ger fel 12 när det läses in, eftersom talet efter E är större än 127.
$3B77	FATN	**Funktionerna har inte alltid sex rätta siffror.** Serierna gör de sista siffrorna osäkra. ATN(.001) blir .000999 och inte .001, eftersom FATN räknar med PI/4 och ett tal nära −1 som nästan tar ut varandra. LOG(1.001)*1000 blir .999482, där rätt svar är .9995, EXP(-.5) blir .606527 i stället för .606531 och EXP(2.302585) 9.99987 i stället för 10. SQR(10) blir 3.16227, eftersom roten huggs av efter sex siffror.
$3A12	ASALGN	**ASCII-aritmetiken avrundar två gånger.** ADD$, SUB$ och MUL$ avrundar först b eller produkten och sedan svaret. ADD$("1","0.0049",2) blir 1.01, eftersom .0049 först blir .005, och MUL$("1","0.0049",2) blir .01. Exakt hälften går mot det större talet, inte bort från noll: ADD$("2.5","0",0) ger 3, men ADD$("-2.5","0",0) ger −2.
$2258	RANDOM	**RANDOMIZE ger bara 128 frön.** RANDOMIZE fyller fröets fem byte med R-registret, som ökar med 6 för varje byte: R, R+6, R+12 och så vidare. ROM:en skriver aldrig till R, och bit 7 är 0 efter start, så det finns bara 128 olika frön. Utan RANDOMIZE börjar RND på samma följd vid varje RUN.
$1CC2	FOR	**FOR-slingor som aldrig tar slut.** STEP 0 räknas som positivt, och slingan tar aldrig slut. Med heltal tar FOR I%=1 TO 32767 inte heller slut: variabeln slår runt till -32768, som är mindre än gränsen. Isaksson och Kärrsgård varnar för samma sak med FOR I%=30000% TO 32000% STEP 1000% (Isaksson och Kärrsgård 1980, s. 35).

Egenheter i språket

Adress	Rutin	Vad som händer
$1DFB	CMPIF	**IF v THEN 100 : S.** Ett radnummer efter THEN är hela THEN-delen. Det som står efter : är en vanlig sats efter IF, och i IF v THEN 100 : S körs S alltså när v är falskt. På samma sätt körs S i IF v THEN T ELSE 100 : S när v är sant.
$27FB	CMPADD	**Unärt minus bara först.** Ett unärt - eller + går bara först på nivån för + och -. 2*-3, 2+-3, 2^-1 och - -2 ger fel 11; 2*(-3) och 1=-1 går. Efter NOT kommer nivån för jämförelser, så NOT NOT 5 och 1+NOT 0 ger också fel 11.
$1EFE	RDVAL	**INPUT och READ tar bort blanktecken.** En sträng utan citattecken läses till , eller CR, och alla blanktecken hoppas över: A B blir "AB" (Christensen 1979). Det gäller även READ av DATA. INPUT LINE tar raden som den är.
$0D3F	CHAIN	**CHAIN med tom sträng.** CHAIN gör NEW innan filen laddas. CHAIN med en tom sträng raderar därför programmet och kör det tomma programmet.
$0C8A	CMDREN	**REN som går över 65535.** Går nästa radnummer över 65535 börjar numreringen om på 10, även när raden som fick det sista numret var programmets sista: REN 65000,500 med två rader ger 10, 20.
$0C8A	CMDREN	**REN med bara startnumret.** Utan steg blir steget samma som startnumret, inte 10: REN 100 ger 100, 200, 300. Bara REN utan tal ger 10, 20, 30, och REN ,20 ger fel 31.
$0912	TXTPRT	**Kolumnen i textfiler.** TXTPRT gör två eller fler blanktecken i följd till $09 och antalet, och kommer CR efter följden stryks den. Följs följden i stället av ett annat tecken blir kolumnen i FCB:ns +6 ett steg för stor.
$0636	ERROR	**ON ERROR fångar bara vissa fel, och bara en gång.** Bara de 16 koderna i listan ERRTRP fångas: 4, 7, 12, 19, 21, 30 och 34–43. Av dem ger ROM:en aldrig 36, 39–41 och 43. Andra fel, till exempel 50 (SQR av ett negativt tal), och STOP stoppar programmet som vanligt. När ett fel har fångats är ON ERROR avstängt, så ON ERROR GOTO måste köras igen innan nästa fel kan fångas. I direktläge fångas inga fel.
$0C5D	CMDMERG	**MERGE sorterar bara tills en rad hamnar sist.** Raderna från filen läggs in med INSLIN, som sorterar in dem. När en rad har hamnat sist i programmet läggs resten av filen till i slutet utan sortering: MERGE av raderna 10, 30, 25 till ett program med 5 och 20 ger 5, 10, 20, 30, 25.
$1AAA	DEF	**DEF FN måste stå först på raden.** Resten av en rad med DEF FN körs inte: 10 DEF FNB(X)=X:PRINT 5 skriver inget. Står DEF FN inte först på raden hittar förlänkningen den inte, och anropet ger fel 57. När raden körs läser DEF då en byte ur den förra satsen som radens längd och fortsätter på en adress som inte är radens slut, och det kan ge till exempel fel 4.
$1EC5	READ	**INPUT och READ skiljer värdena åt på olika sätt.** Står det något annat än , efter ett värde i INPUT (5X,6) glöms resten av raden, och nästa variabel får en ny rad. READ kräver inget ,: det som står efter ett värde blir nästa värde. I DATA "AB"C är C ett eget värde, och i DATA 2 : REM blir : REM nästa värde (fel 12 om det läses som ett tal).
$1513	GETFLT	**Blanka och punkter inne i tal.** Talen läses med RST $18, som hoppar över blanktecken: 1 2 . 5 ger 12.5 och GOTO 1 0 blir GOTO 10, och LIST skriver dem ihop. En andra decimalpunkt ändrar inget (1.2.3 ger 1.23). E+ eller E- utan siffror räknas till talet (1E+ ger 1), men E ensamt gör det inte, så INPUT läser 1E som 1.
$31A0	EXVAL	**VAL vill att texten tar slut efter talet.** EXVAL sparar adressen till CR efter texten och ger fel 12 om talet slutar tidigare. VAL("12X") och VAL("1E") ger därför fel 12, fast INPUT läser samma text som 12 och 1. I 9913 är kontrollen borttagen, och där ger de 12 och 1.
$1D21	NEXT	**Fel efter NEXT och RETURN får fel radnummer.** NEXT hoppar tillbaka till satsen efter FOR utan att sätta om CURLIN, och det gör inte heller RETURN ($22B0). Ett fel eller STOP i resten av raden får därför numret på raden med NEXT eller RETURN, ända tills nästa rad börjar. Med 10 FOR I=1 TO 2:IF I=2 THEN STOP och 20 NEXT I skrivs ”STOP LINE 20”.

Adress	Rutin	Vad som händer
$2154	CMPPRT	**PRINT A B ger fel 11.** Två uttryck får stå direkt efter varandra i PRINT: PRINT A%B skriver båda talen och PRINT BB båda strängarna. Men efter ett flyttal får ingen bokstav komma, och PRINT A B ger fel 11. PRINT 1 2 skriver 12, eftersom talet läses förbi mellanslaget.
$1F82	INPLIN	**INPUT LINE skriver inget frågetecken.** INPUT LINE skriver inget ”?” och ingen ny rad efter raden, så nästa PRINT fortsätter på samma rad. Strängen får CR och LF sist, och raden får vara 118 tecken; resten av en längre rad hamnar i nästa INPUT.
$1EC5	READ	**DATA räknas bara först på raden.** READ letar efter rader som börjar med DATA. Står DATA efter en annan sats, som i 40 PRINT 1 : DATA 8, läses värdena aldrig, och det blir fel 30 när de behövs.
$3126	EXTAB	**TAB räknar i en byte.** TAB(n) ger så många mellanslag som behövs för att komma från markören till kolumn n, men bara den låga byten av skillnaden används. Står markören i kolumn 7 ger TAB(261) 254 mellanslag, och nästa tecken hamnar sex rader längre ned, i kolumn 21.
$01A8	PRTXT	**Tom rad efter 40 tecken, och ESC som försvinner.** När ett tecken har skrivits i kolumn 39 går PRTXT genast till nästa rad. En rad på exakt 40 tecken som följs av CR LF ger därför en tom rad efter sig. En ESC-sekvens måste stå hel i samma text: står färre än tre tecken efter ESC kastas den, och PRINT CHR$(27);"=" skriver bara ett likhetstecken.

Rättat i 9913

Adress	Rutin	Vad som händer
$1787	FAOVF2	**PRINT .0999999 ger .001.** När avrundningen ger en siffra till ökar FLTASC exponenten med 1, även för tal under 1. Därför skrivs .0999999 som .001. Rättat: LD E,A i stället för INC E. Luxors errata ger ett annat exempel, 0.99E-5, som skrivs ut som 0.0000001 i stället för 0.00001 (ABC 80 Bruksanvisning, s. 60).
$1BBD	CMPDMI	**DIM A ger ett slumpvis fel.** DIM A utan index och DIM A(1,2,3) med tre ska ge fel 13. CMPDMI har då två värden på stacken, men CMPERR tar bara bort ett, och återhoppet går till variabelns namn som om det vore en adress ($41F0 för A). Där finns oftast inget minne, och processorn kör skräp. Felet blir oftast 4, men efter ett program som stannat på fel 11 gav DIM A fel 32. Rättat: 9913 ger fel 13.
$1337	DETTAB	**LIST av DIM A$ skriver sönder minnet.** DIM A$ utan index och utan längd går att skriva in och köra. I koden står då bara $D2 (längden 0) och $CC med variabeln, utan något uttryck före. När raden listas får ROTTXT en adress på stacken som inte hör till texten, och LIST skriver sönder minnet. Rättat: 9913 ger fel 15 redan när raden skrivs in (CMPDMI). Luxors errata säger att ABC80 ”går vilse” (ABC 80 Bruksanvisning, s. 60).
$27AE	OPTYPE	**En sträng och ett heltal ger inte fel 17.** När den högra operanden är ett heltal testar OPTYPE den vänstra operandens typ med LD A,(IX+5) direkt före JR NZ, men LD sätter inte flaggorna. Därför görs heltalet om till flyttal, även när den vänstra operanden är en sträng, och det blir en flyttalsoperation på strängens beskrivning i stället för fel 17. "A"=1% och "A"*2% ger fel 50, "A"+1% fel 4, och i ett program skriver PRINT A$+1% ” ?.???4” och ger sedan fel 8. Med ett flyttal till höger (A$+1) eller strängen till höger (1%="A") blir det fel 17 som det ska. Rättat: AND A efter LD. Luxors errata tar S%=S$+S% som exempel och säger att ABC80 då går vilse (ABC 80 Bruksanvisning, s. 60).

Adress	Rutin	Vad som händer
$11C1	DETOK	**REM och DATA ändras av LIST.** Texten efter REM och DATA listas av samma slinga som resten av raden, så ett : i texten får ett mellanslag framför sig. Raden ändras varje gång den listas och läses in igen. Rättat: texten kopieras oförändrad (CMPREM). Luxors errata nämner felet men säger att raden lagras rätt och att READ läser den rätt (ABC 80 Bruksanvisning, s. 60).
$014C	DOCMD	**UNSAVE utan tillägg gör ingenting.** UNSAVE går till tillägget på $605A via EXTJP1. Saknas tillägget kommer CY tillbaka, men READY tittar inte på den, och kommandot gör ingenting. Rättat: via EXTJP2, som ger fel 8. Felet står först på Luxors errata (ABC 80 Bruksanvisning, s. 60).
$3DF5	FSQR	**SQR(0) är inte noll.** SQR(0) ger ett tal med exponenten $40 och mantissan 0. Det skrivs 0, men X=0 är falskt och SGN(X) = 1. Division med talet kommer aldrig tillbaka: 5/SQR(0) hänger i FDIV, och CTRL-C bryter inte. Rättat: 9913 ger 0 (FSQR börjar med RET Z), och 5/SQR(0) ger fel 4. Luxors errata förklarar felet med att tioexponenten inte nollställs och tar PRINT 1/SQR(0) som exempel (ABC 80 Bruksanvisning, s. 60).

Felkoder

ABC80 skriver ett fel som `ERR n`, utan text. Tabellen har ett nummer per rad med felets text och de ställen i ROM:en som ger det. Texterna finns inte i ROM:en; de är hämtade ur ABC80:s fellista. I koden är felet byten $\$80 + n$. Den står efter en `RST $10`, antingen direkt i rutinen eller i en av felingångarna på `$06F7`–`$072B`, som rutinen hoppar till (kolumnen *Ingång*). Ett *ja* under *Fångas* betyder att numret finns i tabellen `ERRTRP`, så att `ON ERROR GOTO` får ta hand om felet i ett körande program; tabellen räknar också upp 36, 39–41 och 43, som ROM:en aldrig ger själv. Under *Ges i* står de namngivna rutiner där felet uppstår. Listan är inte fullständig, och namnen är länkar till raden i listningen.

Nr	Kod	Text	Fångas	Ingång	Ges i
0	`$80`	EJ TILLÅTET ÖKA ”DIM”		`L06F9`	`DIM1`, `DIMSTR`
1	`$81`	FEL ANTAL INDEX		`L06FB`	`LNKSRC`
2	`$82`	OTILLÅTET SOM KOMMANDO		`L06FD`	`NODIR`, `LNKFOR`, `RETURN`
3	`$83`	MINNET FULLT		`L06FF`	`INSLIN`, `CMDMERG`, `FN`, `CONCAT`, `STRFIL`, `EXVAL`, `DIM1`, `DIM2`, `DIMSTR`, `MEMFUL`
4	`$84`	FÖR STORT FLYTTAL	ja	`L0701`	`POWI`, `FADDC`, `FMUL`, `FMRND`, `FDIV`, `FDRND`, `FOVFL`, `FEXP`, `FLOG`
5	`$85`	FÖR STORT INDEX		`L0703`	`VAR1`, `ERRIDX`
6	`$86`	HITTAR EJ DETTA RADNUMMER		`L0705`	`CMDED`, `REFADR`, `LNKELM`
7	`$87`	FÖR STORT HELTAL	ja	`L0707`	`SMULDV`, `FTIERR`
8	`$88`	FINNS EJ I DETTA SYSTEM		`L0709`	`EXTJP1`, `EXTJP2`, `STEXE1`, `STEXE2`, `LNKELM`, `STTAB1`, `STTAB2`, `FN`, `EXPTAB2`
9	`$89`	INDEX UTANFÖR STRÄNGEN		`L070B`	`EXLEFT`, `STRSKP`
10	`$8A`	TEXTEN FÅR EJ PLATS I STRÄNGEN		`L070D`	`ASGNS`
11	`$8B`	FÖRSTÅR EJ			`TOKKW`, `CMPPRT`, `VARNAM`
12	`$8C`	FELAKTIGT TAL	ja	`L0713`	`RDVAL`, `CMPNUM`, `EXVAL`
13	`$8D`	FEL ANTAL ELLER TYP AV ARGUMENT			`LNKFN`, `CMPDEF`, `CMPFN`, `CMPINP`, `CMPPOK`, `TYPCNV`, `CMPARG`
14	`$8E`	OTILLÅTET TECKEN EFTER SATSEN			`TOKLIN`
15	`$8F`	”=” SAKNAS ELLER PÅ FEL PLATS			`TOKKW`, `CMPDEF`, `CMPLET`
16	`$90`	RADNUMMER SAKNAS			`CMDMERG`, `CMPREF`, `CMPON`
17	`$91`	OTILLÅTEN BLANDNING AV TAL OCH STRÄNGAR			`CMPSV`, `CMPGET`, `TOFLT`, `CMPADD`
18	`$92`	”)” SAKNAS ELLER PÅ FEL PLATS			`CMPDEF`, `CMPFN`, `PRIMARY`
19	`$93`	KAN EJ ÖPPNA FLER FILER	ja		
20	`$94`	FÖR LÅNG RAD			`TXTINP`
21	`$95`	HITTAR EJ FILEN	ja	`L070F`	`OPEN`
22	`$96`	OTILLÅTEN SATS		`L0711`	
23	`$97`	”TO” SAKNAS			`CMPFOR`

Nr	Kod	Text	Fångas	Ingång	Ges i
24	$98	”NEXT” SAKNAS		L0715	PRELNK
25	$99	FELAKTIG SATS EFTER ”ON”			CMPON
26	$9A	FEL I ON-UTTRYCK		L071D	ON
27	$9B	”NEXT” UTAN ”FOR”		L0719	LNKNXT
28	$9C	FEL VARIABEL EFTER ”NEXT”		L0717	LNKNXT
29	$9D	”RETURN” UTAN ”GOSUB”		L071B	CMDRUN ($0D6D)
30	$9E	DATA SLUT	ja	L071F	READ
31	$9F	FEL DATA TILL KOMMANDO		L0721	CMDED, CMDLIST, CMDREN
32	$A0	FILEN EJ ÖPPNAD		L0723	CURFIL
33	$A1	”AS FILE” SAKNAS			CMPOPN
34	$A2	SLUT PÅ FILEN	ja		INPUT
35	$A3	CHECKSUMMAFEL VID LÄSNING	ja		RCVBLK
37	$A5	FELAKTIGT RECORDFORMAT	ja		CASOPN, CASGET, RCVBLK
38	$A6	RECORDNUMMER UTANFÖR FILEN	ja		
42	$AA	SKIVAN EJ KLAR	ja		CASRBL
50	$B2	KVADRATROT UR NEGATIVT TAL		L0725	FSQR
51	$B3	ENHETEN UPPTAGEN			
52	$B4	EJ TILL DENNA ENHET		L0727	CASTAB, CMDMERG, CMDSAVE
53	$B5	FELAKTIG RAD		L06F7	STEXE1
57	$B9	FUNKTIONEN EJ DEFINIERAD		L0729	LNKFN
58	$BA	OGILTIGT TECKEN INLÄST			TXTINP
59	$BB	FEL PROGRAMFORMAT			CMDMERG
60	$BC	BIT ADRESS 16 BITAR		L072B	
61	$BD	KOMMA SAKNAS			
62	$BE	DOT-ADRESS UTANFÖR SKÄRMEN			DOTARG
63	$BF	”AS” SAKNAS			
64	$C0	FELAKTIG ”RENAME”			
65	$C1	SPILL I ASCII-ARITMETIK		ERR65	ASCARG
66	$C2	STRÄNG EJ NUMERISK		ERR66	ASCAR, ASCERR, ASCARG

Fyra av felen kommer inte där man väntar sig dem:

- Fel 1 kommer bara när samma namn står med ett index och med två (`A(1)` och `A(1,1)`). `A` och `A(1)`, eller `A$` och `A$(1)`, är olika variabler och går bra (avsnittet Variablerna, i boken).
- Fel 13 är det tänkta felet för `DIM A` och `DIM A(1,2,3)`, men stacken är fel efter felet i `CMPDMI`, och ABC80 ger i stället ett fel som beror på vad som råkar ligga i minnet, oftast 4 (avsnittet `DIM`, i boken). Den senare ROM:en 9913 ger fel 13 (i boken).
- Fel 61 kommer aldrig: kontrollen av kommat i `CMPDOT` slår inte till (avsnittet Grafikpunkterna, i boken).
- Fel 62 kommer bara från `SETDOT` och `CLRDOT`. `DOT` utanför skärmen ger inget fel, utan -1 eller 0 efter en byte i ROM:en (i boken).

Tokens och tabeller

Här står ROM:ens tabeller över kommandon, satser, operatorer och funktioner, ordnade efter den byte (token) som ordet blir i den kompilerade raden. Bredvid ordet står rutinen som kompilerar det och rutinen som kör det, med länk till listningen. Tabellerna hjälper när en kompilerad rad ska läsas byte för byte, eller när man vill veta vilken rutin som gör arbetet för ett ord. För ord som går till en tilläggskrets står stubben, till exempel `L017A` för `BYE`, med kretsens adress inom parentes.

Orden skrivs utan mellanslag, så som de står i ROM:en. `KWSRCH` hoppar över blanka när den jämför, och därför är `INPUT LINE` och `INPUTLINE` samma ord. Står det *tillägg* och en adress tittar ROM:en efter ett `JP` där (`EXTJP1`, `EXTJP2`): finns det hoppar den dit, annars ger ordet fel 8.

B.1 Direktkommandon

Kommandona söks i `CMDKW`, och koden väljer rutinen i `CMDTAB`: koden − `$80` är platsen i tabellen. `SCR` har samma kod som `NEW`.

Kommando	Kod	Rutin
`BYE`	`$80`	`L017A` (tillägg `$604E`)
`RUN`	`$81`	`CMDRUN`
`LIST`	`$82`	`CMDLIST`
`NEW`	`$83`	`NEW`
`SCR`	`$83`	`NEW`
`CLEAR`	`$84`	`CLEAR`
`LOAD`	`$85`	`CMDLOAD`
`MERGE`	`$86`	`CMDMERG`
`REN`	`$87`	`CMDREN`
`ED`	`$88`	`CMDED`
`SAVE`	`$89`	`CMDSAVE`
`UNSAVE`	`$8A`	`L014C` (tillägg `$605A`)

B.2 Satser

En sats med ett ord ur `STKW1` blir en token `$80`–`$96`. Token − `$80` är platsen både i `STTAB1`, med rutinen som kompilerar satsen, och i `STEXE1`, med rutinen som kör den. Några token har inget ord att söka på: kompilatorn skriver dem själv (avsnittet Satsens nyckelord, i boken). Där kompileringen är `RESET` står adressen 0 i `STTAB1`, men den nås aldrig.

Token	Ord	Kompilering	Körning
`$80`	`GOTO`	`CMPREF`	`GOTO`
`$81`	(`GOTO` efter `THEN`/`ELSE`)	`CMPREF`	`GOTO`
`$82`	`LET`	`CMPLET`	`LET`
`$83`	(`LET` utan ordet)	`CMPLET`	`LET`
`$84`	`PRINT`	`CMPPRT`	`PRINTS`
`$85`	`;` (som `PRINT`)	`CMPPRT`	`PRINTS`

Token	Ord	Kompilering	Körning
$86	(tabell 2, kod följer)	RESET	EXEST2
$87	(hopp förbi THEN- eller ELSE-delen)	RESET	EXESKP
$88	IF	CMPIF	IF
$89	INPUT	CMPINP	INPT
$8A	INPUTLINE	CMPINL	INPLIN
$8B	FOR	CMPFOR	FOR
$8C	NEXT	CMPNXT	NEXT
$8D	(inget ord)	fel 8	fel 8
$8E	READ	CMPRD	READ
$8F	RESTORE	CMPRES	RESTOR
$90	GOSUB	CMPREF	GOSUB
$91	RETURN	NOCODE	RETURN
$92	ON	CMPON	ON
$93	DATA	CMPREM	DATA
$94	ONERRORGOTO	CMPREF	ONERRG
$95	(rad som inte gick att kompilera)	RESET	fel 53
$96	DEFFN	CMPDEF	DEF

B.3 Satser, tabell 2

En sats med ett ord ur STKW2 skrivs som $86 följd av $80 + k, där k är platsen i STKW2 och i rutinernas tabeller STTAB2 och STEXE2. $86 $83 har en kompileringsrutin men inget ord, så den kan inte skrivas.

Kod	Ord	Kompilering	Körning
$86 $80	DIM	CMPDIM	DIM
$86 $81	POKE	CMPPOK	POKE
$86 $82	OUT	CMPOUT	OUTST
$86 $83	(inget ord)	CMPINX	fel 8
$86 $84	REM	CMPREM	REM
$86 $85	OPEN	CMPOPN	OPENST
$86 $86	PREPARE	CMPOPN	PREPST
$86 $87	CLOSE	CMPCLS	CLOSST
$86 $88	RANDOMIZE	CMPNOP	RANDOM
$86 $89	STOP	NOCODE	STOP
$86 $8A	END	NOCODE	ENDPRG
$86 $8B	CMD	L0147 (tillägg $7000)	L0166 (tillägg $7003)
$86 $8C	(inget ord)	fel 8	fel 8
$86 $8D	(inget ord)	fel 8	fel 8
$86 $8E	SETDOT	CMPDOT	SETDOT
$86 $8F	CLRDOT	CMPDOT	CLRDOT
$86 $90	GET	CMPGET	GET
$86 $91	CHAIN	CMPSTX	CHAIN
$86 $92	KILL	CMPSTX	L0170 (tillägg $605D)
$86 $93	NAME	L0151 (tillägg $6054)	L0175 (tillägg $6057)
$86 $94	TRACE	NOCODE	TRACE
$86 $95	NOTRACE	NOCODE	NOTRAC

B.4 Övriga nyckelord

Efter STKW1 står sex tabeller med ett ord var. Fem av dem söker en kompileringsrutin i när den väntar ett visst ord: IF söker THEN och ELSE, FOR TO och STEP, och OPEN och PREPARE AS FILE. Ordets token skrivs i satsens kod. KWAS söker ingen rutin i (avsnittet Satsens nyckelord, i boken). LIST hittar orden, eftersom DETOK letar efter token från STKW1 och framåt (avsnittet Raden som text, i boken).

Tabell	Token	Ord
KWTHEN	$BD	THEN
KWELSE	$BC	ELSE
KWSTEP	$BB	STEP
KWTO	$BA	TO
KWASF	$B8	ASFILE
KWAS	$B7	AS

B.5 Operatorer

Uttryckskompilatorn har tio nivåer, från PREC1 med den lägsta prioriteten till PREC10, det primära uttrycket. Sju av dem kompileras av PRECLV (avsnittet En nivå med binära operatorer, i boken). Aritmetiken och jämförelserna har en token per typ: tabellens token för flyttal, +1 för heltal och +2 för strängar. För ^ är +2 ett flyttal upphöjt till ett heltal. Körrutinerna står i EXPTAB1.

Nivå	Operator	Token	Klass	Kompilering	Körning
PREC1	EQV	$DC	logisk	PRECLV	OPEQV
PREC2	IMP	$DD	logisk	PRECLV	OPIMP
PREC3	OR	$DE	logisk	PRECLV	OPOR
PREC3	XOR	$DF	logisk	PRECLV	OPXOR
PREC4	AND	$E0	logisk	PRECLV	OPAND
PREC5	NOT	$E1	unär	CMPNOT	OPNOT
PREC6	<=	$F1	jämförelse	PRECLV	LEF LEI LES
PREC6	<>	$E5	jämförelse	PRECLV	NEF NEI NES
PREC6	<	$E8	jämförelse	PRECLV	LTF LTI LTS
PREC6	>=	$EB	jämförelse	PRECLV	GEF GEI GES
PREC6	>	$EE	jämförelse	PRECLV	GTF GTI GTS
PREC6	=	$E2	jämförelse	PRECLV	EQF EQI EQS
PREC7	+	$F4	aritmetik	CMPADD	ADDF ADDI CONCAT
PREC7	-	$F7	aritmetik	CMPADD	SUBF SUBI
PREC8	*	$F9	aritmetik	PRECLV	MULF MULI
PREC8	/	$FB	aritmetik	PRECLV	DIVF DIVI
PREC9	^, **	$FD	potens	PRECLV	POWF POWI POWFI
PREC10			primärt uttryck	PRIMARY	

B.6 Funktioner

Funktionernas namn står i FNKW, var och en med token $80 + k, där k är funktionens kod. I uttryckskoden blir anropet $CD–$D1 efter antalet argument, 0–4, följt av 2k, och k är platsen i EXPTAB2 (avsnittet Körningen av uttryckskoden, i boken). Formen och resultatets typ kommer ur FNTYPE: x är ett flyttal, i ett heltal och s$ en sträng, och det som står inom [ ] får utelämnas.

Kod	Form	Resultat	Rutin
$00	FN	–	FN
$01	ABS(x)	flyttal	EXABS
$02	ATN(x)	flyttal	EXATN
$03	COS(x)	flyttal	EXCOS
$04	EXP(x)	flyttal	EXEXP1
$05	FIX(x)	flyttal	EXFIX
$06	INT(x)	flyttal	EXINT
$07	LOG(x)	flyttal	EXLOG
$08	LOG10(x)	flyttal	EXLG10
$09	PI	flyttal	EXPI
$0A	RND[(x)]	flyttal	EXRND
$0B	SGN(x)	heltal	EXSGN
$0C	SIN(x)	flyttal	EXSIN
$0D	SQR(x)	flyttal	EXSQR
$0E	TAN(x)	flyttal	EXTAN
$0F	ASC(s$)	heltal	EXASC
$10	CHR$(i[,i[,i[,i]]])	sträng	EXCHR
$11	LEFT$(s$,i)	sträng	EXLEFT
$12	RIGHT$(s$,i)	sträng	EXRGHT
$13	MID$(s$,i,i)	sträng	EXMID
$14	LEN(s$)	heltal	EXLEN
$15	INSTR(i,s$,s$)	heltal	EXINST
$16	SPACE$(i)	sträng	EXSPC
$17	STRING$(i,i)	sträng	EXSTRG
$18	NUM$(x)	sträng	EXNUM
$19	VAL(s$)	flyttal	EXVAL
$1A	SWAP%(i)	heltal	EXSWAP
$1B	PEEK(i)	heltal	EXPEEK
$1C	INP(i)	heltal	EXINP
$1D	CALL(i[,i])	heltal	EXCALL
$1E	ERRCODE	heltal	EXERR
$1F	IEC$(i)	sträng	EXIEC (tillägg $7006)
$20	TAB(i)	sträng	EXTAB
$21	CUR(i,i)	sträng	EXCUR
$23	DOT(i,i)	heltal	EXDOT
$30	ADD$(s$,s$,i)	sträng	EXADD
$31	SUB$(s$,s$,i)	sträng	EXSUB
$32	MUL$(s$,s$,i)	sträng	EXMUL
$33	DIV$(s$,s$,i)	sträng	EXDIV
$34	COMP%(s$,s$)	heltal	EXCOMP

Koderna $22 och $24–$2F har inget namn i FNKW och kan inte skrivas. I EXPTAB2 ger de fel 8.

B.7 Uttryckskod

Den kompilerade uttryckskoden har token $B6–$FF, och EXPTAB1 ger rutinen som kör var och en; token − $B6 är platsen i tabellen. Operatorerna står med sin text. Vad varje token betyder står i avsnittet Körningen av uttryckskoden (i boken).

$B6	ASGNF	$CF	EXFUNC		$E8	LTF	<
$B7	ASGNI	$D0	EXFUNC		$E9	LTI	<
$B8	ASGNS	$D1	EXFUNC		$EA	LTS	<
$B9	ENDF	$D2	PUSH0		$EB	GEF	>=
$BA	EXEND	$D3	EXEXP		$EC	GEI	>=
$BB	ENDS	$D4	GETF		$ED	GES	>=
$BC	ENDVF	$D5	GETI		$EE	GTF	>
$BD	EXEND	$D6	GETS		$EF	GTI	>
$BE	ENDVS	$D7	NEGF		$F0	GTS	>
$BF	ENDDIM	$D8	NEGI		$F1	LEF	<=
$C0	VAR0	$D9	FTOI		$F2	LEI	<=
$C1	VAR0	$DA	ITOF		$F3	LES	<=
$C2	VAR0	$DB	SKPARG		$F4	ADDF	+
$C3	CONSTF	$DC	OPEQV	EQV	$F5	ADDI	+
$C4	VAR1F	$DD	OPIMP	IMP	$F6	CONCAT	+
$C5	VAR1I	$DE	OPOR	OR	$F7	SUBF	-
$C6	VAR1S	$DF	OPXOR	XOR	$F8	SUBI	-
$C7	CONSTI	$E0	OPAND	AND	$F9	MULF	*
$C8	VAR2F	$E1	OPNOT	NOT	$FA	MULI	*
$C9	VAR2I	$E2	EQF	=	$FB	DIVF	/
$CA	VAR2S	$E3	EQI	=	$FC	DIVI	/
$CB	CONSTS	$E4	EQS	=	$FD	POWF	^
$CC	DIMELM	$E5	NEF	<>	$FE	POWI	^
$CD	EXFUNC	$E6	NEI	<>	$FF	POWFI	^
$CE	EXFUNC	$E7	NES	<>			

Symbolförteckning

Namnen från listningen i bokstavsordning, med adressen och sidan där namnet står. I PDF-filen är varje namn en länk till raden. Det är samma namn som i listningen och de är valda för den; ROM:en själv har inga.

Namn som bara är en bokstav och en adress är inte med, eftersom adressen redan säger var de står: J för ingångarna i början av ROM:en, D för texter och tabeller, som D0501 på $0501, och L för hoppmålen i rutinerna. Variablerna i RAM och adresserna utanför ROM:en står i kapitlet Systemvariabler och externa adresser (s. 3).

AADD $37A6 224
ACOMP $37F6 225
ADDF $2AB7 170
ADDI $2B5E 174
ADIV $383B 226
ALLOC $3374 208
AMUL $38B8 227
ARGWLK $33B0 209
ASADDB $3ABA 233
ASADDN $390D 228
ASALG1 $3A32 231
ASALGN $3A12 231
ASALST $3B72 235
ASBAD $39BF 230
ASBLST $3B6D 235
ASBORN $3948 228
ASCAR $2BBA 177
ASCARG $2C6C 179
ASCARN $391A 228
ASCCMP $2C51 178
ASCERR $2C4B 178
ASCMPS $3817 225
ASCNT $3B68 235
ASDIG $3B4D 234
ASDIG2 $395E 229
ASDLP $3868 226
ASDOT $3B47 234
ASENTR $396F 229
ASFIN $3B00 234
ASGNF $2989 165
ASGNI $29B4 165
ASGNS $29C3 165
ASLEN $38FC 228
ASLEN1 $3902 228
ASMADD $37B4 224
ASMLP $38CC 227
ASMSUB $37C9 225
ASNEG $3AD6 233
ASNZ $3B53 235
ASPARS $39C5 230
ASPUT $3926 228
ASPUTB $3954 229
ASREST $3ACD 233
ASRET $3B3E 234
ASSETB $38AB 227
ASSUBN $393B 228
ASUB $37AE 224
ASWIDE $3A67 232
ASWORK $3A9D 232
ASZERA $37E6 225
ATNPOS $3B8A 238
ATNSER $3C02 239
BRKSTP $2323 141
CALLBC $2C47 178
CASBRS $03F4 20
CASCLO $0428 20
CASGET $0509 23
CASHLF $0484 21
CASINT $0594 25
CASOPN $0491 22
CASPRE $03BF 19
CASPUT $040E 20
CASRBL $052B 24
CASTAB $03A4 19
CASW0 $0476 21
CASWBI $0477 21
CASWBL $0439 20
CASWBY $046A 21
CHAIN $0D3F 52
CLEAR $0A80 43
CLOSE $087F 36
CLOSST $20BF 129
CLRDOT $22CF 140
CLRROW $0267 13
CLS $0276 13
CLSALL $20CD 129
CMDED $0AA1 43
CMDKW $0A20 42
CMDLIST $0AEF 44
CMDLOAD $0BA8 46
CMDLP $00DB 8
CMDMERG $0BA9 46
CMDREN $0C8A 50
CMDRUN $0D4C 52
CMDSAVE $0DC6 54
CMDTAB $0A5A 42
CMPADD $27FB 157
CMPARG $264E 152
CMPB9 $1CAF 112
CMPBF $1BB8 107
CMPCLS $2071 128
CMPDEF $1A62 102
CMPDIM $1BAA 107
CMPDMI $1BBD 107
CMPDOT $22B6 139
CMPERR $1BF7 108
CMPEXB $2362 143
CMPEXP $2701 154
CMPFLV $1BA5 107
CMPFN $1AB8 103
CMPFNC $261C 151
CMPFOR $1C83 111
CMPGET $23A8 144
CMPIF $1DFB 118
CMPINL $1F6B 123
CMPINP $1E58 119
CMPINX $2370 143
CMPIVR $239F 144
CMPLET $1B89 106
CMPNOP $226A 137
CMPNOT $2826 158
CMPNUM $2689 152
CMPNXT $1CB7 112
CMPON $1FBF 125
CMPOPN $205D 128
CMPOUT $20FF 130
CMPPOK $2104 130
CMPPRT $2154 132
CMPRD $1E6E 120
CMPREF $1DC7 116
CMPREM $226B 137
CMPRES $2281 137
CMPSTR $25C6 150
CMPSTX $236C 143
CMPSV $237F 143
CMPSVR $2392 143
CMPTYX $2372 143
CMPVAR $25EE 150
CMPVR $2397 143
COLD $00C6 8
CONCAT $2F47 192
CONSTF $2A36 166
CONSTI $2AE3 171
CONSTS $2ED7 189
COSSER $3FF4 254
CRLF $0234 13
CURADR $0293 14
CURFIL $08FC 37

DATA $2273 137
DEF $1AAA 102
DETEXP $1258 72
DETOK $1195 69
DETOP $1398 76
DETTAB $12B6 73
DETVAR $1422 78
DEVDFL $0193 11
DEVIEC $01A1 11
DEVLST $018C 11
DEVPR $019A 11
DIM $1BFA 108
DIM1 $32E6 206
DIM2 $3311 207
DIMELM $1BFE 109
DIMSTR $3338 207
DIVF $2AC9 170
DIVI $2B73 174
DOCMD $0133 9
DOTADR $22EE 140
DOTARG $22E3 140
DVCALL $08A5 36
DVCLOS $0890 36
DVGET $0899 36
DVINP $0893 36
DVOPEN $088A 36
DVPREP $088D 36
DVPRT $0896 36
DVPUT $089C 36
EDTAB $0348 16
ELMADR $2CFA 182
ELMSIZ $1C6C 110
ENDDIM $1C7E 110
ENDF $285F 161
ENDPRG $00C9 8
ENDS $2877 161
ENDVF $2884 161
ENDVS $2887 161
EQF $2A4C 168
EQI $2B2A 173
EQS $2FE4 195
ERR65 $2C4F 178
ERR66 $2C49 178
ERRIDX $2D4F 183
ERRMSG $069B 29
ERROR $062E 28
ERRSTP $0669 28
ERRTRP $0659 28
ESCARG $0227 13
EXABS $2D7F 183
EXADD $2B9F 176
EXASC $2EEC 191
EXATN $2DAF 185
EXCALL $31CC 202
EXCHR $2FC1 193
EXCOMP $2BB3 177
EXCOS $2DA5 185
EXCUR $313B 200
EXDIV $2BAE 177
EXDOT $22D6 140
EXEND $285B 160
EXERR $204F 127
EXESKP $0DB8 54
EXEST2 $0DB3 53
EXESTM $0DA3 53
EXEXP $2842 160
EXEXP1 $2DB4 185
EXEXPH $2841 160
EXFIX $2D96 185
EXFUNC $2853 160
EXIEC $016B 10
EXINP $31C1 202
EXINST $3075 197
EXINT $2D9B 185
EXLEFT $2F07 191
EXLEN $2EE3 191
EXLG10 $2DBE 185
EXLOG $2DB9 185
EXMID $2EFD 191
EXMUL $2BA9 177
EXNUM $3152 200
EXPEEK $2B7E 174
EXPI $2D6E 183
EXPSER $3C7E 241
EXPTAB1 $288B 162
EXPTAB2 $291F 163
EXRGHT $2F1C 192
EXRND $31E0 203
EXSGN $2D86 183
EXSIN $2DA0 185
EXSPC $30F7 199
EXSQR $2DC3 185
EXSTRG $30FB 199
EXSUB $2BA4 177
EXSWAP $2B79 174
EXTAB $3126 200
EXTAN $2DAA 185
EXTJP1 $0156 10
EXTJP2 $017F 10
EXVAL $3172 201
F2DPI $3FA5 253
FACARY $1734 90
FADD $33F7 211
FADDC $3402 211
FADOT $17BB 92
FAEDGT $17EC 93
FAEDIG $16E3 90
FAEFMT $16D1 89
FAEND $17CE 92
FAEOUT $17E3 92
FAEPOS $1805 93
FAEPRN $180D 93
FAEXP $16C1 89
FAEXPN $17F1 93
FAFIX $16E7 90
FAFIX1 $16F5 90
FAFMT $16D5 89
FAFORM $17A6 92
FAFRAC $17C9 92
FAINT $17D4 92
FALEAD $17BF 92
FALIGN $36BB 219
FAOUT $178A 91
FAOVF1 $1778 91
FAOVF2 $1783 91
FAOVFL $1766 91
FAPOS $16C9 89
FARITH $2ACD 170
FARND1 $1741 91
FAROU1 $1721 90
FAROUN $1719 90
FASGN1 $17A4 92
FASIGN $1791 92
FASMAL $16FD 90
FASTR1 $1748 91
FASTRP $1747 91
FATINY $1703 90
FATN $3B77 238
FAZER1 $175C 91
FAZERO $1751 91
FAZERS $16C8 89
FCLR5 $3C39 240
FCMP $2A93 169
FCMPST $2A7A 169
FCOPY $36FC 220
FCOS $3D70 244
FDIV $35EE 217
FDLOOP $3622 217
FDRND $3689 218
FENTER $3766 221
FEXP $3C03 240
FFIX $2DD5 185
FFUNC $2DC6 185
FINDLN $0F39 60
FINT $3C7F 241
FINT1 $3C8B 241
FINT2 $3C8E 241
FINTFL $3CA7 241
FINTLP $3C93 241
FLDX $3BC5 239
FLGE $3D31 243
FLN10 $3D36 243
FLOG $3CC4 242
FLOG10 $3CB7 242
FLTAS6 $1677 88
FLTASC $1679 88
FLTCH $15C4 85
FLTEXP $1642 86
FLTSAV $1670 87
FMLN2 $3D3B 243
FMLOOP $356C 215
FMRND $359D 216
FMUL $34DF 214
FMULE $34FA 214
FN $1B2A 104
FNAME $1478 80
FNDFIL $20E1 130
FNEG $3BC0 239
FNEGX $3BBD 239
FNKW $2431 146
FNKWTX $13F1 77
FNORM $3494 213
FNPAR $33AB 209
FNPART $14DA 82
FNRES $1B7B 105
FNTEXT $13E9 77
FNTYPE $24F2 147
FONE $3FBB 253
FOR $1CC2 113
FORTSF $1D88 115
FORTSI $1D59 115
FOVFL $3758 221
FPI $3FA6 253
FPIH $3FA0 253
FPIQ $3FB4 253
FPOLY $3F61 252
FPOW $2E07 187
FSIN $3D77 244
FSINC $3D78 244
FSQR $3DF5 246
FSTOR1 $3437 212
FSTORE $3434 212
FSUB $3460 212
FSUBM $3473 213

FTAN	$3EAD	248	KWSRCH	$09F5	42	OPTEXT	$13D6	77
FTIERR	$3EF1	250	KWSTEP	$1986	99	OPTYPE	$27A0	156
FTILP	$3EDE	250	KWTHEN	$197A	99	OPXOR	$2B21	173
FTISGN	$3EF8	250	KWTO	$198C	99	OUTST	$212A	131
FTOI	$2DE2	186	LEF	$2A67	168	POKE	$2144	131
FTOINT	$3ECA	249	LEI	$2B56	173	POLYLP	$3F7D	253
FZERO	$375B	221	LES	$2FFF	195	POWF	$2E00	187
GEF	$2A75	168	LET	$1BFD	108	POWFI	$2E5E	188
GEI	$2B4A	173	LINFD	$0238	13	POWI	$2E18	187
GES	$300D	195	LNKADR	$10A4	64	PRCRLF	$21F6	135
GET	$1DB1	116	LNKDEF	$0F59	60	PRDIG	$187D	95
GETDIG	$1837	94	LNKELM	$0EAA	57	PRDIG1	$1889	95
GETEND	$1833	94	LNKEXP	$1032	63	PREC1	$23B5	145
GETF	$29A1	165	LNKFN	$111D	66	PREC10	$23F3	145
GETFLT	$1513	84	LNKFOR	$0F7C	61	PREC2	$23BC	145
GETI	$29BD	165	LNKLST	$106B	64	PREC3	$23C3	145
GETKEY	$02EA	15	LNKNEW	$108E	64	PREC4	$23CA	145
GETLN	$181E	93	LNKNXT	$0FDB	62	PREC5	$23D1	145
GETLN1	$181D	93	LNKPAR	$1050	63	PREC6	$23D7	145
GETLP	$182A	93	LNKREF	$0D01	51	PREC7	$23DE	145
GETOVF	$183F	94	LNKSKP	$102E	63	PREC8	$23E5	145
GETPOS	$08EF	37	LNKSRC	$1074	64	PREC9	$23EC	145
GETS	$2A22	166	LNKVAR	$103C	63	PRECLV	$2705	154
GETSFL	$14FB	82	LNKWR	$10B0	65	PRELNK	$0E6E	56
GETSIN	$1841	94	LOGDBL	$3CCE	242	PREPST	$2077	128
GOSUB	$1DE9	117	LOGEXP	$3D0D	243	PRIMARY	$257F	149
GOTO	$1DDB	116	LOGK	$3D03	243	PRINT	$08D4	37
GTF	$2A5E	168	LOGRED	$3CE3	242	PRINTS	$219F	134
GTI	$2B5A	173	LOGSER	$3D6F	244	PRNUM	$1862	94
GTS	$2FF6	195	LTF	$2A70	168	PRSGN	$1860	94
IDIV	$3788	222	LTI	$2B3E	173	PRSIN	$1855	94
IF	$1E4B	119	LTS	$3008	195	PRTCOM	$221C	135
IM2VEC	$0034	7	MADD3	$3447	212	PRTEXP	$21C0	134
IMUL	$3774	222	MEMFUL	$3398	208	PRTFIT	$21F0	135
IMULDV	$2B6B	174	MEMRES	$335A	208	PRTSEP	$223A	136
INIT	$0068	7	MSHL	$3736	221	PRTXT	$01A8	11
INLIN2	$02AD	14	MSHR	$3714	220	PUSH0	$1C7F	110
INLINE	$02AA	14	MSUB3	$34CB	213	PUTBC	$239B	143
INPLIN	$1F82	124	MUL10A	$3F00	250	PUTTOK	$2368	143
INPT	$1E78	120	MUL10X	$3F16	250	RANDOM	$2258	137
INPUT	$08DD	37	MULF	$2AC3	170	RCVBIT	$061C	26
INSCNV	$27CF	156	MULI	$2B67	174	RCVBLK	$05D0	26
INSLIN	$0750	32	NEF	$2A55	168	RCVBYT	$061A	26
INTDHI	$3F5C	252	NEGF	$2A43	167	RCVSTA	$0556	24
INTDIG	$3F3F	251	NEGHL	$2AEE	171	RDBC	$1017	62
INTTOF	$3F18	251	NEGI	$2AE9	171	RDBC1	$1016	62
ITFPOS	$3F36	251	NEI	$2B34	173	RDDE	$101D	63
ITOF	$2DF0	186	NES	$2FED	195	RDDE1	$101C	63
IXOFF	$375F	221	NEW	$0A70	43	RDVAL	$1EFE	122
JOINA	$140C	77	NEWLIN	$0742	31	READ	$1EC5	121
JOINC	$140A	77	NEXT	$1D21	114	READY	$00CC	8
JPHL	$2852	160	NMI	$0066	7	REFADR	$0D33	52
JPIX	$2A91	169	NMICLK	$0047	7	REM	$2276	137
KEYINT	$031E	15	NOCODE	$22B5	138	RESET	$0000	6
KWADD	$2421	146	NODATA	$10FB	66	RESTOR	$2286	137
KWAND	$2407	146	NODIR	$0DBF	54	RETURN	$22B0	138
KWAS	$1998	99	NOTRAC	$232A	141	RNDADD	$32E5	205
KWASF	$1990	99	NXTLEV	$27E8	157	RNDINI	$225D	137
KWCMP	$2411	146	ON	$2009	126	RNDMUL	$32E0	205
KWDEFN	$196D	99	ONERRG	$2041	127	ROOM	$339B	208
KWELSE	$1980	99	OPAND	$2B14	172	ROTTXT	$1449	78
KWEQV	$23F5	145	OPEN	$07E2	34	ROWTAB	$0374	17
KWIMP	$23FA	145	OPENST	$2074	128	RST08	$0008	6
KWLIST	$1973	99	OPEQV	$2AF5	172	RST10	$0010	6
KWMUL	$2426	146	OPIMP	$2B00	172	RST18	$0018	6
KWNOT	$240C	146	OPNOT	$2B1D	172	RST20	$0020	6
KWOR	$23FF	145	OPNSTR	$07C2	34	RST28	$0028	6
KWPOW	$242B	146	OPOR	$2B0B	172	RST30	$0030	6

RST38 $0038 7
RSTDAT $2293 138
RUBOUT $035B 16
RUNLIN $0D71 53
RUNSTM $0D92 53
SCRPOS $0338 16
SETDOT $22C7 139
SETEXT $08BD 36
SFALSE $2FE9 195
SINCOS $3DDA 245
SINSER $3FD4 254
SINSGN $3DE0 245
SKIPS0 $25BF 149
SKIPSP $25BE 149
SKPADD $10F9 66
SKPARG $1B85 105
SKPELM $0EFB 59
SKPEXP $10C3 65
SKPFN $10EA 65
SKPLEN $10F8 66
SKPRET $10E8 65
SKPSTR $10F4 65
SKPTOK $10DD 65
SMUL $2B83 174
SMULDV $2B87 174
SPACE $11EA 70
SPACES $224A 136
SQRADD $3E71 247
SQRBOR $3E68 247
SQRBYT $3E8D 247
SQRCLR $3E2A 246
SQRCY $3E53 247
SQRDIG $3E5B 247
SQREND $3E9E 248
SQRLP $3E37 246
SQRNXT $3E86 247
SQRRLD $3E95 247
SQRSHL $3E90 247
SQRSUB $3E40 246
STEXE1 $0E14 55
STEXE2 $0E42 55
STKCHK $3389 208
STKW1 $1911 98
STKW2 $199C 99
STMEND $218D 133
STOP $2320 141
STRCMP $3012 195
STRCPN $3068 196
STRFIL $30FE 199
STRPSH $2FBB 193
STRSKP $2F25 192
STRUE $2FF2 195
STTAB1 $1A08 101
STTAB2 $1A36 101
SUBF $2ABD 170
SUBI $2B62 174
TOFLT $256E 148
TOINT $2561 148
TOKKW $18D4 97
TOKLEN $1105 66
TOKLIN $1890 96
TOKSTM $18C7 97
TRACE $2325 141
TRCPRT $232F 141
TSTBRK $033E 16
TXTBLK $0995 39
TXTEND $0985 39
TXTINP $09A0 39
TXTPRT $0912 38
TYPCNV $2551 148
VAR0 $2CCD 181
VAR1 $2CE7 182
VAR1F $2CD9 182
VAR1I $2CDF 182
VAR1S $2CE4 182
VAR2 $2D39 183
VAR2F $2D2B 182
VAR2I $2D31 182
VAR2S $2D36 182
VARIDX $2D11 182
VARNAM $26CB 153
VARSIZ $10B8 65
WRBC $1029 63
WRBC1 $1028 63
WRDE $1023 63
WRDE1 $1022 63
WRDIG $1815 93
WRZERO $1761 91